

C++ Qt6 跨平台开发

来源: <https://gitee.com/apacheecn/apacheecn-c-cpp-zh-pt2/blob/master/docs/cross-platform-dev-qt6-mod-cpp/SUMMARY.md>

C++ Qt6 跨平台开发

来源: <https://gitee.com/apacheecn/apacheecn-c-cpp-zh-pt2/blob/master/docs/cross-plat-dev-qt6-mod-cpp/SUMMARY.md>

原文: <https://gitee.com/apacheecn/apacheecn-c-cpp-zh-pt2/blob/master/docs/cross-plat-dev-qt6-mod-cpp/README.md>

零、前言

原文: <https://gitee.com/apacheecn/apacheecn-c-cpp-zh-pt2/blob/master/docs/cross-plat-dev-qt6-mod-cpp/00.md>

Qt 是一个跨平台的应用开发框架，旨在为桌面、嵌入式和移动平台创建具有令人惊叹的用户界面的优秀软件应用。它为开发人员提供了一套优秀的工具，用于设计和构建优秀的应用，而不必担心平台依赖性。

在本书中，我们将重点介绍 **Qt 6**，这是 **Qt** 框架的最新版本。本书将帮助您创建用户友好且功能强大的图形用户界面。通过在不同平台上提供外观更好、外观一致的应用，您还将获得相对于竞争对手的优势。

希望使用交互式 **GUI** 构建跨平台应用的开发人员将能够将他们的知识运用到本实用指南中。这本书提供了实现概念和相关机制的实践方法，这些概念和相关机制将使您的应用在任何时间内启动并运行。我们还将为您提供基本概念的解释和示例，让您获得完整的学习体验。

您将从探索跨不同平台的 **Qt** 框架开始。您将学习如何在不同的平台上配置 **Qt**，了解不同的 **Qt** 模块，学习核心概念，以及如何使用它们来构建高效的 **GUI** 应用。您将能够跨不同平台构建、运行、测试和部署应用。您还将学习如何定制应用的外观，并开发支持翻译的应用。除了学习完整的应用流程外，本书还将帮助您识别瓶颈以及如何解决这些瓶颈，以提高您的应用的性能。

到本书结束时，您将能够在不同的平台上构建和部署您自己的 **Qt** 应用。

这本书是给谁看的

本书面向希望构建基于 GUI 的应用的开发人员和程序员。它也适用于以前用 C++ 编写过代码的软件工程师。入门门槛不是很高，所以如果您了解基本的 C++ 和 OOPS 概念，那么您就可以开始这段旅程了。

此外，本书还可以帮助那些希望在其他平台上构建和部署的中级 Qt 开发人员。想要开始使用 Qt 编程的在职专业人士或学生，以及刚接触 Qt 的程序员，都会发现这本书很有用。

这本书涵盖了哪些内容

第 1 章，Qt 6 简介将向您介绍 Qt，并说明如何在机器上设置它。在本章结束时，读者将能够从源代码构建 Qt，并开始使用他们选择的平台。

第 2 章，Qt Creator 简介，向您介绍 Qt Creator IDE 及其用户界面。本章还将教您如何在 Qt Creator 中创建和管理项目。您将学习使用 Qt Creator 开发一个简单的 *Hello World* 应用，并了解不同的快捷方式和实用技巧。

第 3 章，*使用 Qt 小部件进行 GUI 设计，*探索 Qt 小部件模块。在这里，您将了解可用于创建 GUI 的各种小部件。您还将了解布局、Qt 设计器，并学习如何创建自己的自定义控件。本章将帮助您使用 Qt 开发您的第一个 GUI 应用。

第 4 章，Qt Quick 和 QML，涵盖 Qt Quick 和 QML、Qt Quick Controls、Qt Quick Designer、Qt Quick Layout 和基本 QML 脚本的基础知识。在本章中，您将学习如何使用 Qt 快速控件以及如何将 C++ 代码与 QML 集成。在本章结束时，您将能够使用 QML 创建具有流畅 UI 的现代应用。

第 5 章，跨平台开发探讨了使用 Qt 进行跨平台开发。您将了解 Qt Creator 中的不同设置。在本章中，您将能够在您最喜欢的桌面和移动平台上运行示例应用。

第 6 章，*信号和插槽，*深入介绍了信号和插槽机制。您将能够在不同的 C++ 类之间以及 C++ 和 QML 之间进行通信。您还将了解事件、事件过滤器和事件循环。

第 7 章，模型视图编程介绍了 Qt 中的模型/视图体系结构及其核心概念。在这里，您将能够编写自定义模型和委托。您可以使用它们来显示基于 Qt Widget 或基于 Qt Quick 的 GUI 应用所需的信息。

第 8 章，*图形和动画，*介绍 2D 图形和动画的概念。您将学习如何使用 Painter API 在屏幕上绘制不同的形状。我们将进一步讨论使用 Qt 的 Graphics View 框架和 Scene Graph 表示图形数据的可能性。本章将引导您创建一个吸引眼球的动画用户界面。本章还涉及状态机框架。

第 9 章，测试和调试探讨了 Qt 应用的不同调试技术。在本章中，您将了解单元测试和 Qt 测试框架。我们还将讨论如何将 Google 测试框架与 Qt Test 结合使用，并了解可用的 Qt 工具和特定于 GUI 的测试技术。

第 10 章，*部署 Qt 应用，*讨论软件部署的重要性。您将学习如何在各种平台上部署 Qt 应用，包括桌面和移动平台。您将了解可用部署工具和创建安装程序包的步骤。

第 11 章，*国际化，*向您介绍国际化。Qt 为将 Qt 小部件和 Qt Quick 应用翻译成本地语言提供了出色的支持。在本章中，您将学习如何制作支持多语言的应用。您还将了解制作翻译感知应用的内置工具和各

种注意事项。

第 12 章，*性能注意事项，*向您介绍性能优化技术，以及如何在 Qt 编程环境中应用这些技术。在这里，我们将讨论诊断性能问题的不同分析工具，主要集中在 Windows 上可用的工具。在本章中，您将学习如何使用 QML Profiler 评测性能并对代码进行基准测试。本章还将帮助您编写高性能的优化 QML 代码。

充分利用这本书

我们将只使用开源软件，因此您不需要购买任何许可证。我们将在每一章的过程中详细介绍安装过程和细节。要安装所需的软件，您需要正常的互联网连接和一台台式 PC 或笔记本电脑。除此之外，在您开始本书之前没有特殊的软件要求。

Main requirements for the book	OS Requirements
Qt 6.0.0 or higher. Preferably Qt 6.1.0	Windows 10 or Ubuntu 20.04 or macOS 10.14, Android 9 or above
Qt Creator 4.14.0 or higher	
Compilers such as MinGW or MSVC or GCC or LLVM	

重要说明

对于 Android 设置，您需要以下设置：

OpenJDK 8(JDK-8.0.275.1)

Android SDK 4.0

NDK R21(21.3.6528147)

叮当作响的工具链

Android OpenSSL

如果您使用的是这本书的电子版，我们建议您自己键入代码或通过 **GitHub** 存储库(下一节提供的链接)访问代码。这样做可以帮助您避免与复制和粘贴代码相关的任何潜在错误。

所有代码示例都已在 **Windows** 平台上使用 **Qt6** 进行了测试。如果你使用 **Qt5**，你可能会看到失败。但是，它们应该也可以在未来的版本中工作。请确保您要安装到计算机上的版本至少为 **Qt 6.0.0** 或更高版本，以便代码与图书兼容。

下载示例代码文件

您可以从 [github](https://github.com/PacktPublishing/Cross-Platform-Development-with-Qt-6-and-Modern-Cpp)<https://github.com/PacktPublishing/Cross-Platform-Development-with-Qt-6-and-Modern-Cpp> 下载本书的示例代码文件。

此外，您还可以在前面提到的 **GitHub** 链接中找到一些 **C++ 17** 特性的额外示例。如果代码有更新，它将在现有的 **GitHub** 存储库中进行更新。

我们还在<https://github.com/PacktPublishing/>上提供了丰富的图书和视频目录中的其他代码包。看看他们！

下载彩色图片

我们还提供了一个 **PDF** 文件，其中包含本书中使用的屏幕截图/图表的彩色图像。您可以在此处下载：https://static.packt-cdn.com/downloads/9781800204584_ColorImages.pdf。

使用的惯例

本书中使用了许多文本约定。

Code in text: 指示文本中的代码字、数据库表名、文件夹名、文件名、文件扩展名、路径名、虚拟 **URL**、用户输入和 **Twitter** 句柄。下面是一个示例：“通常，`exec ()`方法用于显示对话框。”

代码块设置如下：

```
QMessageBox messageBox;  
messageBox.setText("This is a simple QMessageBox.");  
messageBox.exec();
```

当我们希望您注意代码块的特定部分时，相关行或项将以粗体显示：

```
QMessageBox messageBox;  
messageBox.setText("This is a simple QMessageBox.");  
messageBox.exec();
```

任何命令行输入或输出都如下所示：

```
> lrelease *.ts
```

粗体：表示您在屏幕上看到的新术语、重要单词或单词。例如，菜单或对话框中的单词显示在文本中，如下所示。这里有一个例子：“最后一步是构建和运行应用，点击 **Qt Creator** 中的**Run**按钮。”

提示或重要说明

看起来就像这样。

保持联系

欢迎读者的反馈。

一般反馈：如果您对本书的任何方面有疑问，请在邮件主题中提及书名，并向我们发送电子邮件至customercare@Packtpub.com。

勘误表：虽然我们已竭尽全力确保内容的准确性，但错误还是会发生。如果您在这本书中发现了错误，请向我们报告，我们将不胜感激。请访问www.Packtpub.com/support/errata，选择您的图书，单击勘误表提交表链接，然后输入详细信息。

盗版：如果您在互联网上遇到任何形式的非法复制我们的作品，请您提供地址或网站名称，我们将不胜感激。请通过Copyright@Packt.com联系我们，并提供该材料的链接。

如果您有兴趣成为一名作者：如果有一个您擅长的主题，并且您有兴趣撰写或投稿一本书，请访问Auths.Packtpub.com。

评论

请留下评论。一旦你阅读并使用了这本书，为什么不在你购买它的网站上留下评论呢？这样，潜在读者就可以看到并使用您不偏不倚的意见来做出购买决定，我们 **Packt** 可以了解您对我们产品的看法，我们的作者也可以看到您对他们的书的反馈。谢谢！

有关 **Packt** 的更多信息，请访问Packt.com。

第一部分：基础知识

原文: <https://gitee.com/apacheecn/apacheecn-c-cpp-zh-pt2/blob/master/docs/cross-plat-dev-qt6-mod-cpp/sec1.md>

在本节中，您将了解该框架的基础知识和发展，以及如何在不同平台上安装 Qt。在这一节中，您将了解更多关于 Qt 的发展。然后，我们将继续使用 Qt 的最新版本(即 Qt6)构建我们的第一个示例程序。您将了解 Qt Creator IDE 的用法。本节将向您介绍 Qt 小部件、Qt 设计器和创建自定义控件。您将了解样式表、QSS 文件和主题。本节还将向您介绍 Qt Quick 和 QML。

本节包括以下章节：

- 第一章，Qt6 简介
- 第 2 章，Qt Creator 简介
- 第 3 章，使用 Qt 小工具进行 GUI 设计
- 第 4 章、Qt Quick 和 QML

一、Qt 6 简介

原文: <https://gitee.com/apacheecn/apacheecn-c-cpp-zh-pt2/blob/master/docs/cross-plat-dev-qt6-mod-cpp/01.md>

Qt(发音为*cut*, 而不是*que-tee*)是一个跨平台应用开发框架, 旨在为桌面、嵌入式和移动平台创建具有统一用户界面(**UI**)的优秀软件应用。它为开发人员提供了一套出色的工具来设计和构建优秀的应用, 而无需担心平台依赖性。在本章中, 您将了解该框架的基础知识、它的历史, 以及如何在不同的平台上安装 **Qt**。您将了解什么是 **Qt** 以及为什么使用它是有益的。在本章结束时, 您将能够在您选择的平台上安装 **Qt** 并开始使用。

在本章中, 我们将介绍以下主要主题:

- Qt 简介
- 使用 Qt 的原因
- 下载并安装 Qt
- 从源代码构建 Qt 6

技术要求

要开始使用, 您应该有一台运行 Windows、Linux 或 MacOS 的台式机或笔记本电脑。请使用更新的 Windows 10 或 Ubuntu 20.04长期支持(**LTS**)。或者, 使用最新版本的 MacOS(比 MacOS 10.14 更新), 如 MacOS Catalina。

要使您的集成开发环境(**IDE**)平稳运行, 您的系统应至少配备英特尔酷睿 i5 处理器以及至少 **4GB(GB)**的随机存取存储器(**RAM**)。

您需要活动的 Internet 连接才能下载和安装 Qt。作为先决条件，您还应该熟悉 C++，因为 Qt 需要 C++ 编程知识。

Qt 简介

Qt 是一个跨平台的软件开发框架，适用于桌面、嵌入式和移动平台。它遵循减少代码、创建更多代码并随处部署的理念。支持 Windows、Linux、MacOS、VxWorks、QNX、Android、iOS 等平台。软件还支持恩智浦、瑞萨和意法半导体在裸机或 FreeRTOS 上运行的多个微控制器单元(MCU)。

Qt 诞生的初衷是试图提供一个统一的图形用户界面(GUI)，在不同的平台上具有相同的外观、感觉和功能。Qt 通过提供一个只需编写一次代码的框架来实现这一点，并确保它可以在其他平台上运行，只需极少的修改或无需修改。它不是一种编程语言，而是一个用 C++ 编写的框架。Qt 框架和工具在开源和商业许可下是双重许可的。

Qt 使用模块化方法将相关功能组合在一起。Qt Essentials 是所有平台上 Qt 的基础。这些模块是通用的，对于大多数基于 Qt 的应用都很有用。基本模块可供开源使用。Qt Essentials 模块的示例有 Qt Core、Qt GUI、Qt QML、Qt 小部件等。还有一些特殊用途的附加模块，它们提供特定的功能并附带一定的许可义务。附加模块的示例有 Qt 3D、Qt 蓝牙、Qt 图表、Qt 数据可视化等。除此之外，还有一些增值模块，如 Qt Automotive Suite、用于设备创建的 Qt 和用于 MCU 的 Qt，以及商业许可下提供的其他模块。

要了解有关不同 Qt 模块的更多信息，请访问<https://doc.qt.io/qt-6/qtmodules.html>。

Qt 于 1995 年发布供公众使用。从那时起，有了许多改进和重大变化。Qt 6 是 Qt 的新主版。它的主要目标是为 2020 年及以后的需求做好准备，移除过时的模块，并使其更易于维护。有了这个关注点，Qt6 中的架构变化可能会打破与早期版本的向后兼容性。

Qt 6 中的一些基本修改概述如下：

- 强类型简介
- JavaScript 作为Qt 建模语言(QML)的可选功能
- 删除 QML 版本控制
- 消除 QObject 和 QML 之间的重复数据结构
- 避免创建运行时数据结构
- 将 QML 编译为高效的 C++ 和本机代码
- 支持隐藏实现详细信息
- 更好地集成工具

现在我们已经介绍了基础知识，让我们看看使用 Qt...的主要原因

使用 Qt 的原因

Qt 是一个模块化的、跨平台的应用开发框架。对 Qt 最大的误解是很多人认为它是一个 GUI 框架。然而，Qt 不仅仅是一个 GUI 框架。它不仅包括一个 GUI 模块，还包括一组模块，使应用开发更快、更容易在各种平台上扩展。使用 Qt 的最大好处是它能够为各种平台提供可移植性。以下是开发人员使用 Qt 的一些优势：

- 您可以使用 Qt 为您的客户创造令人难以置信的用户体验，提升您的公司品牌。
- 跨平台开发既省时又省钱。您可以使用相同的代码库瞄准多个平

台。

- Qt 以使 C++ 变得容易和可访问而闻名。有了 Qt，开发人员可以轻松创建具有流畅 UI 的高性能、可伸缩的应用。
- 由于采用了开源模式，该框架是面向未来的，同时还拥有一个强大的生态系统。
- 它还支持不同的编程语言，是一个非常灵活和可靠的框架。因此，像 Adobe、微软、三星、AMD、惠普、飞利浦和 MathWorks 这样的大公司都在他们的应用中使用 Qt。许多开源项目，如 VLC(以前称为 VideoLAN Client)、**Open Broadcaster Software(OBS)**和 WPS Office(其中**WPS**代表**Writer**、**Presentation** 和 **Spreadsheet**)也是基于 Qt 构建的。

Qt 的核心值概述如下：

- 跨平台特性
- 高度可扩展
- 非常容易使用
- 内置世界一流的应用编程接口(**API**)、工具和文档
- 可维护性、稳定性和兼容性
- 一个庞大的用户社区

无论您是业余爱好者、学生，还是在公司工作，Qt 都提供了极大的灵活性，可以根据您的需要使用它的模块。许多大学都把 Qt 作为他们的课程之一。因此，Qt 是程序员开始构建具有现成功能的新应用的绝佳选择。让我们从下载 Qt6 并在您的机器上安装开始。

下载安装 Qt

在您的系统上安装 Qt 框架和工具有不同的方式。您可以从 Qt 网站下

载在线或离线安装程序，也可以自己构建源包。Qt 建议首次安装时使用在线安装程序，以后修改安装时使用**Qt** 维护工具。

安装程序允许您下载并安装以下组件：

- Qt 文库
- Qt Creator IDE
- 文档和示例
- Qt 源代码
- 附加模块

在线安装程序允许您根据选择的许可证选择 Qt、工具和附加模块的开源或商业版本进行安装。在线安装程序不包含 Qt 组件，但它是一个下载所有相关文件的下载客户端。下载完成后即可安装。您需要 Qt 帐户才能下载和安装 Qt。商业 Qt 的评估版本为您提供免费试用期访问，包括所有商业包和官方 Qt 支持。安装程序要求您使用您的 Qt 帐户登录。如果您没有 Qt 帐户，您可以在安装过程中注册。安装程序从 Qt 服务器获取附加到帐户的许可证，并根据您的许可证列出模块。如果您不熟悉 Qt，那么我们建议您从开源版本开始。

脱机安装程序是特定于平台的包，其中包括与平台相关的所有 Qt 模块和加载项。由于官方政策的变化，从 Qt 5.15 开始，开源离线安装程序不再可用。如果您拥有商业许可证，则可以在安装过程中提供凭据。您可以在您的**Qt** 帐户门户中找到您的许可证密钥。

您可以从以下链接下载它们：

- 开源：<https://www.qt.io/download-open-source>
- 商业版：<https://www.qt.io/download>

- **Offline:** <https://www.qt.io/offline-installers>

重要音符

Qt 公司为用户提供双重许可选项。作为初学者，您可以开始使用开源许可证来探索 Qt。如果您为一家公司工作，请与您的经理或信息技术(IT)或法律团队讨论，以获得商业许可证或了解法律义务。您可以在<https://www.qt.io/licensing/>了解有关 Qt 许可的更多信息。

下载 Qt

让我们从将 Qt 下载到您的机器开始，如下所示：

1. 要开始，请访问<https://www.qt.io/download>下载页面。
2. 单击下载。试一试。购买****。右上角的按钮。您将在此处看到不同的下载选项。
3. 如果您想尝试商业版本，请点击**Try Qt**部分。如果您已经拥有 Qt 帐户，则可以登录到**Existing Customers**部分下的帐户。
4. Considering that you are new to Qt, we will begin with the open source version. Click on the **Go open source** button, as shown in the following screenshot:

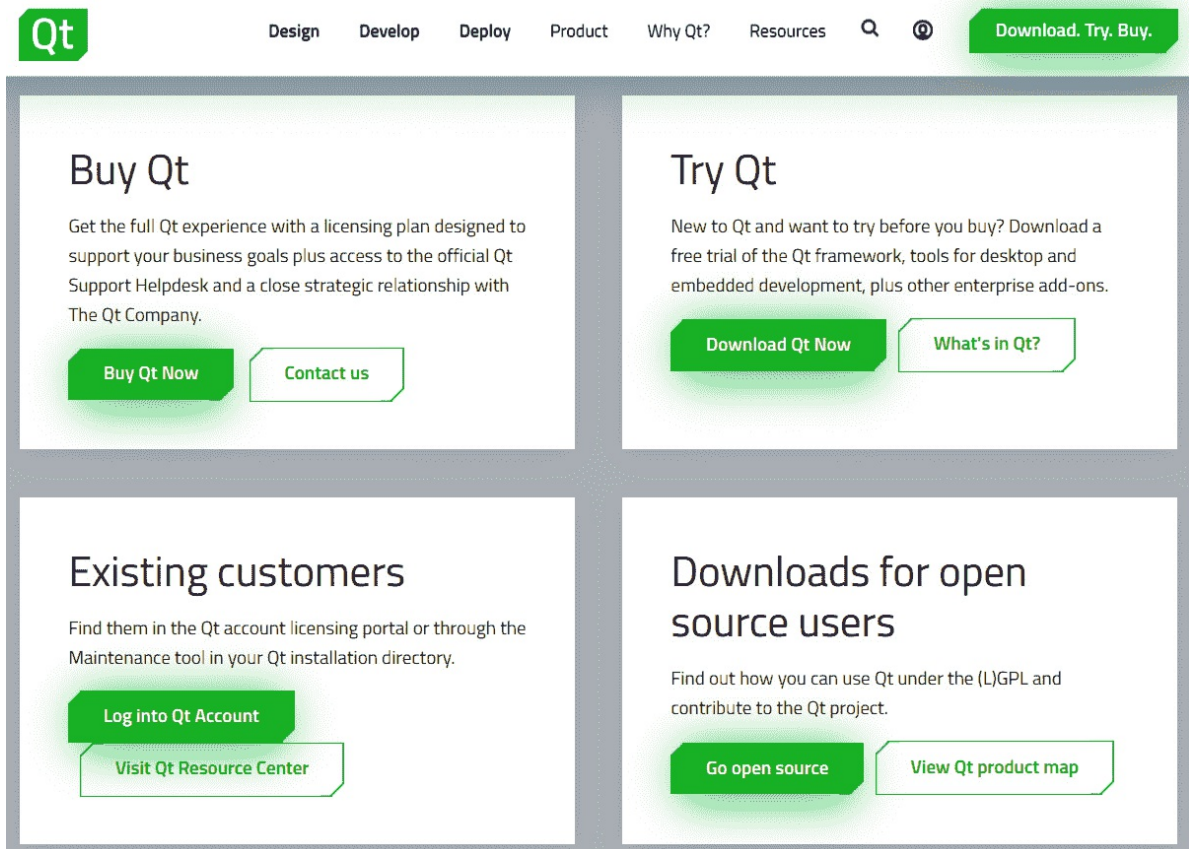


图 1.1-Qt 网站下载选项

5. 在下一个屏幕上，您将看到**Download the Qt Online Installer**按钮。单击它以进入下载链接。
6. The web page will automatically detect the underlying platform details from the browser and will show you the **Download** button. You can also select your intended installer by choosing the other options: you can select **32-bit** or **64-bit** or download for a different platform.

单击下载选项后，您将看到谢谢页面。在此阶段，您可以在download文件夹中找到安装程序。

接下来，让我们从 Windows 平台上的安装过程开始。

在 Windows 上安装 Qt

现在，让我们开始在 Windows 上安装！按照以下步骤进行：

1. 您将在下载文件夹中找到名为 `qt-unified-windows-x86-%VERSION%-online.exe` 的文件。双击可执行文件，您将看到欢迎屏幕。
2. Click the **Next** button, and a credentials screen will appear, asking you to log in with your Qt account. If you don't have one, then you can sign up on the same page, as shown in the following screenshot:

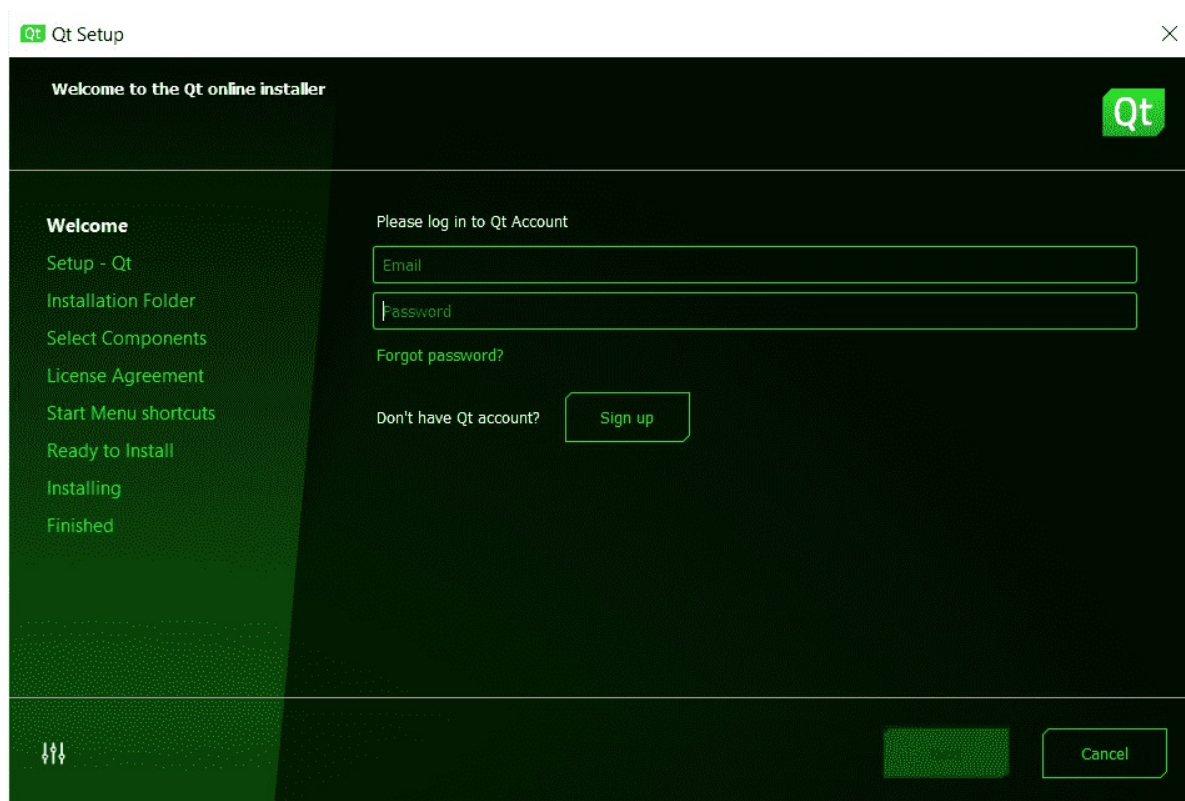


图 1.2-安装程序的登录屏幕

3. 在下一个屏幕中，您将看到开源使用义务协议。如果您使用商业许

可证进行安装，则不会出现此屏幕。 点击第一个复选框，说明我已经阅读并批准了使用开源 **Qt** 的义务，并确认您没有将 **Qt** 用于商业目的。 请务必阅读协议中提到的条款和条件！ 然后，点击下一步按钮。

4. 下一个屏幕将为您提供与在 **Qt Creator** 中跟踪和共享假名数据相关的选项。 您可以根据自己的喜好允许或禁用这些选项。 然后，点击下一步按钮进入下一屏幕。
5. 在下一个屏幕中，您可以指定安装路径。 您可以继续使用默认路径，或者如果默认驱动器上没有足够的空间，也可以将其更改为任何其他路径。 您还可以通过选中底部的复选框选项来选择是否要将常见文件类型与 **Qt Creator** 相关联。 单击下一步按钮。
6. Next, you will be presented with a list where you can select the version(s) of Qt you need to install on your system. You may simply proceed with the default options. If you don't need some of the components, then you can unselect them to reduce the size of the download. You can update the Qt components using the **Maintenance Tool** anytime later. To complete the installation process, click on the **Next** button. The component selection screen can be seen here:

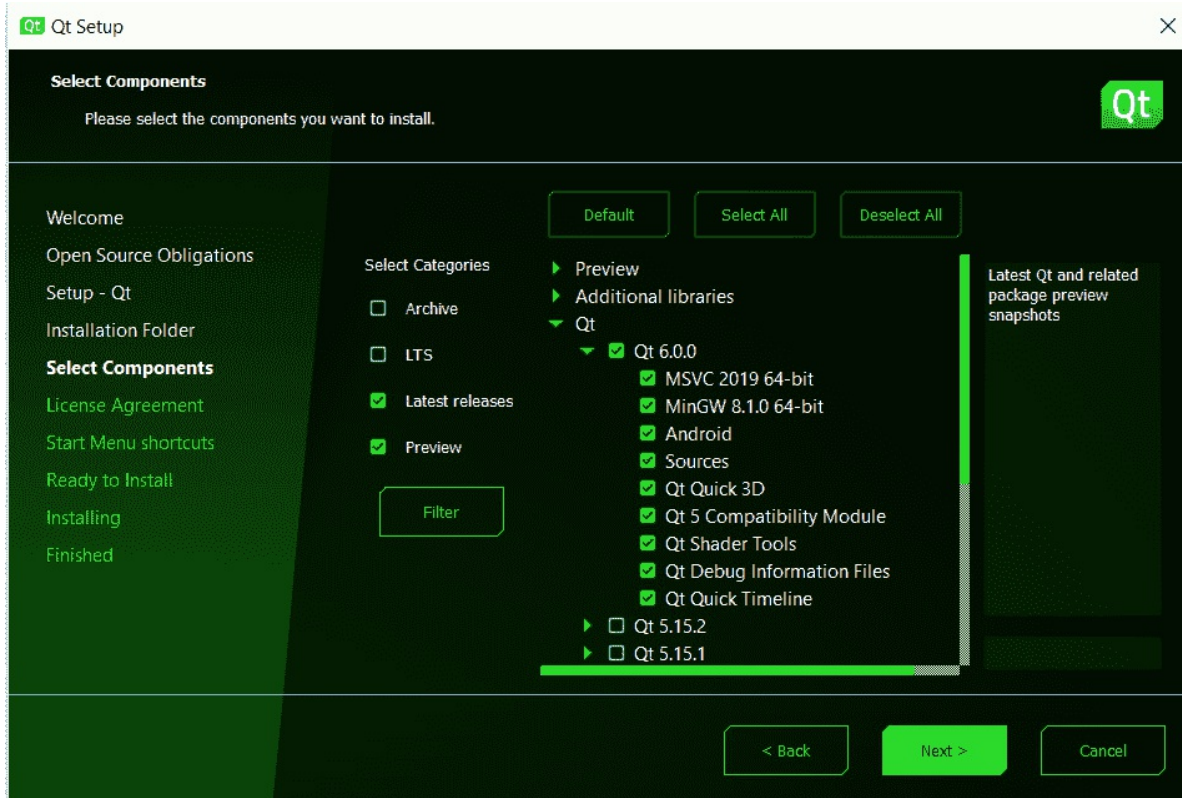


图 1.3-安装程序的组件选择屏幕

7. 在下一个屏幕中，您将看到许可协议。单击第一个单选按钮，显示我已阅读并同意许可协议中包含的条款。同样，请确保您阅读了许可协议中提到的条款和条件，然后单击**Next**按钮。
8. 在下一个屏幕上，您可以在 **Windows** 上创建开始菜单快捷方式。此屏幕将不适用于其他平台。完成此操作后，单击**Next**按钮。
9. 现在，**Qt** 就可以安装到您的系统中了。确保具有正常的互联网连接和数据平衡。单击安装按钮开始安装。下载过程需要时间，具体取决于您的网速。下载所需文件后，安装程序将自动将其安装在先前选择的路径中。
0. 完成安装后，安装程序将为维护工具创建条目，这将帮助您稍后更改库。单击下一步按钮移动到安装程序的最后一个屏幕。

1. 要完成安装过程，请单击**Finish**按钮。如果您选中了启动 **Qt Creator**复选框，则将启动 Qt Creator。我们将在下一章更详细地讨论这一点。现在，Qt 就可以在您的 Windows 计算机上使用了。单击完成按钮退出向导。

在 Linux 上安装 Qt

现在，让我们在 Linux 的最新**LTS** 版本(如 **Ubuntu 20.04**、**CentOS 8.1** 或 **openSUSE 15.1**)上安装 Qt 框架。我们将关注最流行的 Linux 发行版 **Ubuntu**。您可以按照前面提到的相同步骤从 **Qt** 网站下载在线安装程序。

在 **Ubuntu** 上，您将获得一个安装程序文件，

如`qt-unified-linux-x64-%VERSION%-online.run`，其中`%VERSION%`是最新版本-例如：`qt-unified-linux-x86-4.0.1-1-online.run`。

1. 在执行下载的文件之前，您可能必须授予其写入权限。为此，请打开终端并运行以下命令：

```
$ chmod +x qt-unified-linux-x64-%VERSION%-online.run
```

2. 您可以通过双击下载的安装程序文件开始安装过程。安装需要超级用户访问权限。在安装过程中，您可能需要在授权对话框中添加密码。您也可以从终端运行安装程序，如下所示：

```
$ ./qt-unified-linux-x64-%VERSION%-online.run
```

3. 您将看到与 Windows 平台显示的屏幕相似的屏幕。除了特定于操作系统(**OS**)的标题栏更改之外，所有屏幕在安装 **Ubuntu** 或类似 Linux 版本时都保持不变。

在写这本书的时候，Qt6 还没有可用的 Ubuntu 或 Debian 包，因为各自的维护者已经下台了。因此，您可能无法从终端获得 Qt6 包。

在 MacOS 上安装 Qt

如果您是 MacOS 用户，则也可以使用与针对早期平台讨论的相同的方式进行安装。您可以按照前面提到的相同步骤从 Qt 网站下载在线安装程序。

您将获得一个安装程序文件，

如 `qt-unified-mac-x64-%VERSION%-online.dmg`，其中 `%VERSION%` 是最新版本(如 `qt-unified-mac-x64-4.0.1-1-online.dmg`)。

Qt 依赖于 Xcode。要在你的 Mac 上安装 Qt，你需要在你的机器上安装 Xcode，否则它将拒绝安装。如果你是一名苹果开发者，那么你的 Mac 电脑可能安装了 Xcode。如果您的计算机上没有安装 Xcode，那么您可以继续安装 Xcode 的命令行工具，而不是安装 Xcode。这将节省您计算机上的时间和存储空间：

1. 要开始，请在终端上键入以下命令：

```
$ xcode-select --install
```

2. 如果终端显示以下输出，则系统已准备好执行下一步：

```
xcode-select: error: command line tools are already installed,  
"Software Update" to install updates
```

3. 下一步是安装 Qt 框架。双击安装程序文件，启动安装界面。
4. 如果安装程序仍然抱怨 Xcode 未安装，则继续单击确定，直到消息永久消失。记住安装路径。安装完成后，您就可以在计算机上使用

Qt 了。

有关适用于 MacOS 的 Qt 的更多说明，请访问以下链接：

<https://doc.qt.io/qt-6/macos.html>

更新或删除 Qt

一旦安装了 Qt，您就可以使用安装目录下的维护工具修改组件，包括更新、添加和删除组件。所有桌面平台的目录结构都保持相同。安装目录包含文件夹和文件，如以下屏幕截图所示(在 Windows 上)：

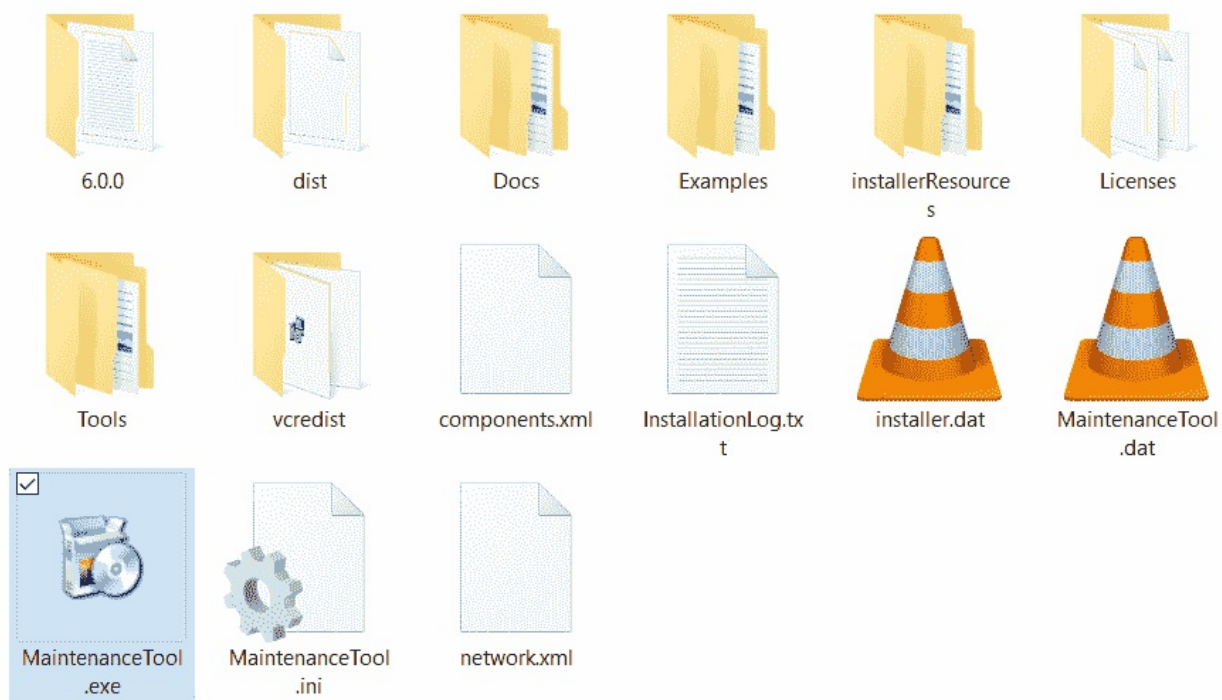


图 1.4-已安装文件夹内的维护工具

让我们从维护流程开始！您可以使用维护工具添加、删除和更新模块。请按以下步骤进行操作：

1. Click on the `MaintenanceTool.exe` executable to launch the

maintenance interface. Click on the **Next** button, and a credentials screen will appear, asking you to log in with your Qt account. The login details will be prefilled from your last login session. You can click **Next** to add or update components or select the **Uninstall only** checkbox to remove Qt from your system. The following screenshot shows what the credentials screen looks like:

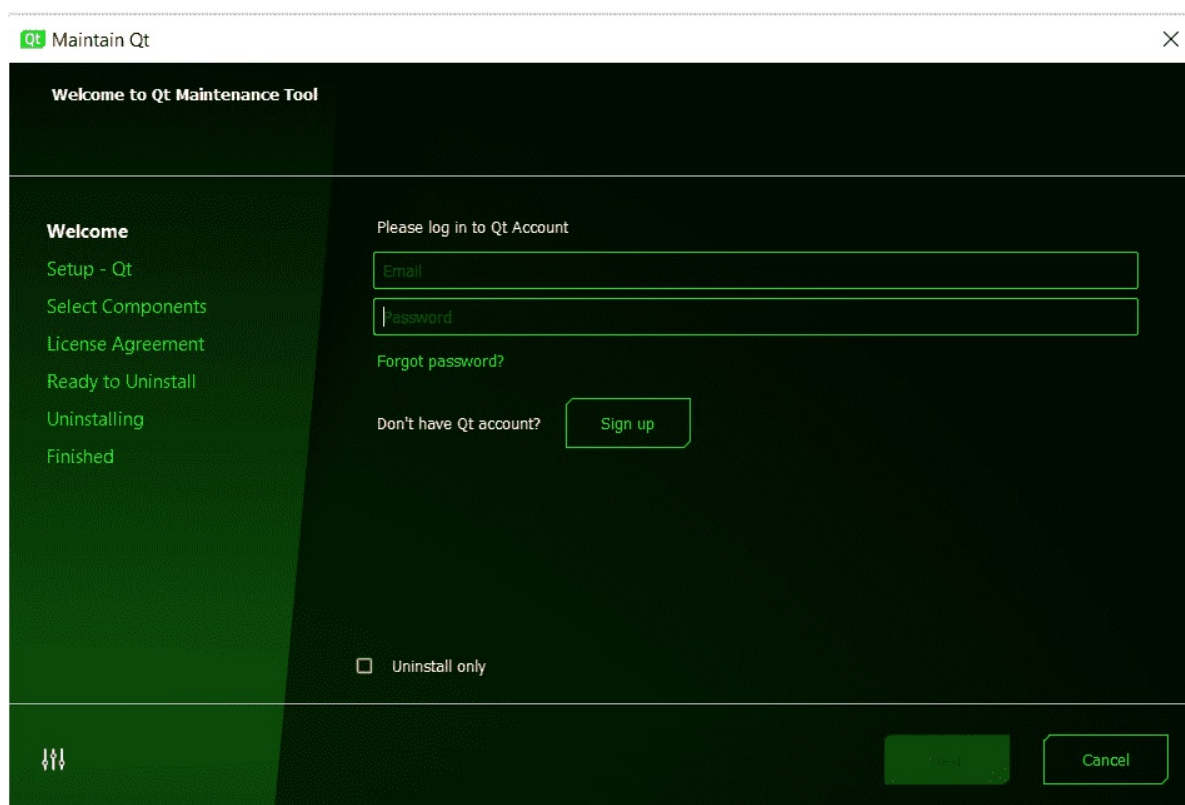


图 1.5-维护工具的欢迎屏幕

2. Once you are logged in, the tool will present you with options to add or remove or update the components, as shown in the following screenshot. Click on the **Next** button to proceed further:

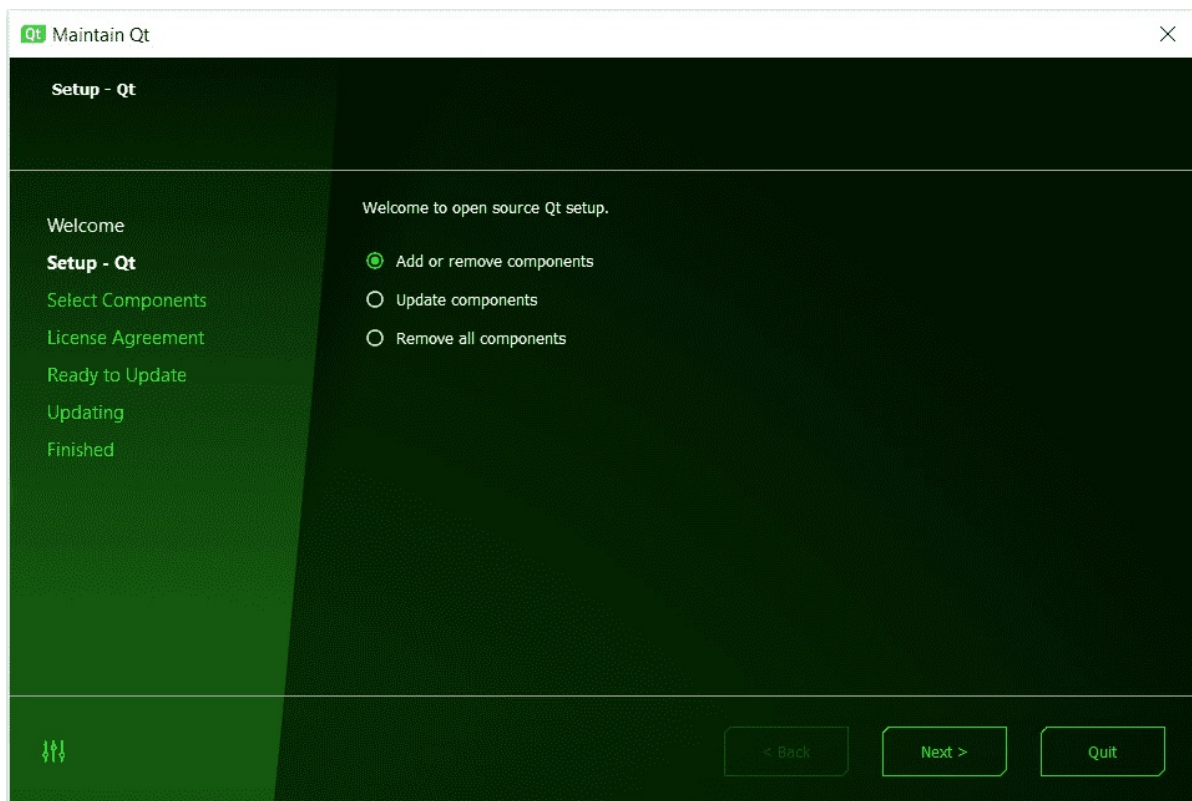


图 1.6-维护工具的设置屏幕

3. On the next screen, you can select new components from the latest releases or the archived version. You can click on the **Filter** button to filter versions as per your requirement. You can also add new platform-specific components such as Android from the component list. If the component is existing and you uncheck it, then it will be removed from your desktop during the update. Once you have selected the components, click on the **Next** button. The following screenshot shows what the component selection screen looks like:

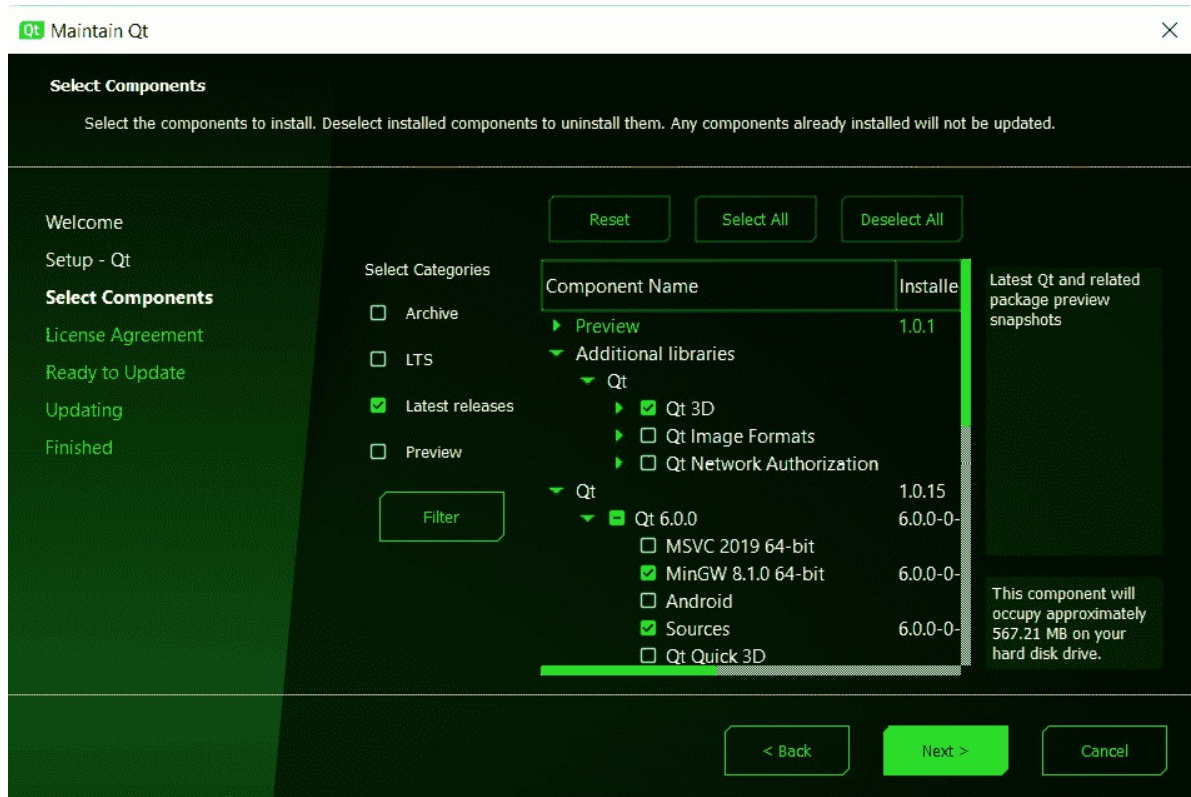


图 1.7-组件选择屏幕

4. You will then come across the update screen. This screen will tell you how much storage space is required for the installation. If you are running out of storage space, then you may go back and remove some existing components. Click on the **Update** button to begin the process, as illustrated in the following screenshot:

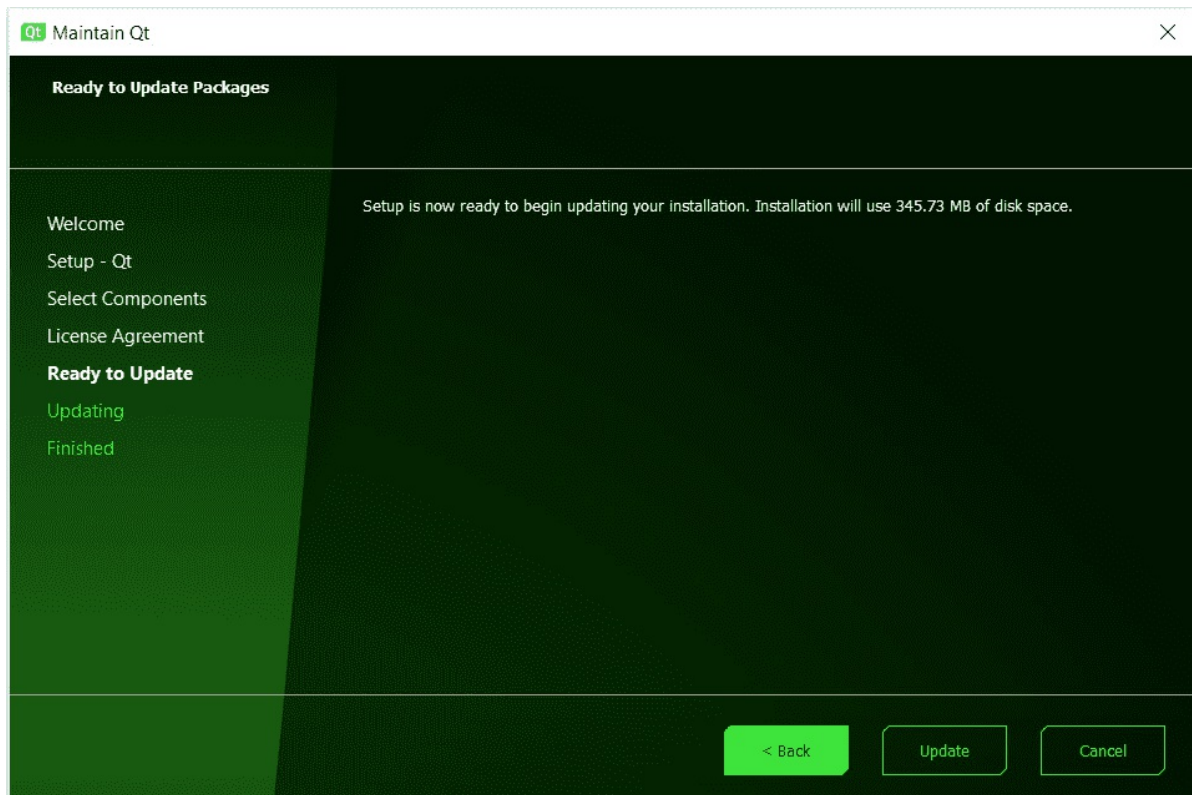


图 1.8-维护工具的 Ready to Update(准备更新)屏幕

5. You can abort the update installation process by clicking on the **Cancel** button. Qt will warn you and ask you for confirmation before aborting the installation process, as illustrated in the following screenshot. Once the process is aborted, click on the **Next** button to exit the wizard:

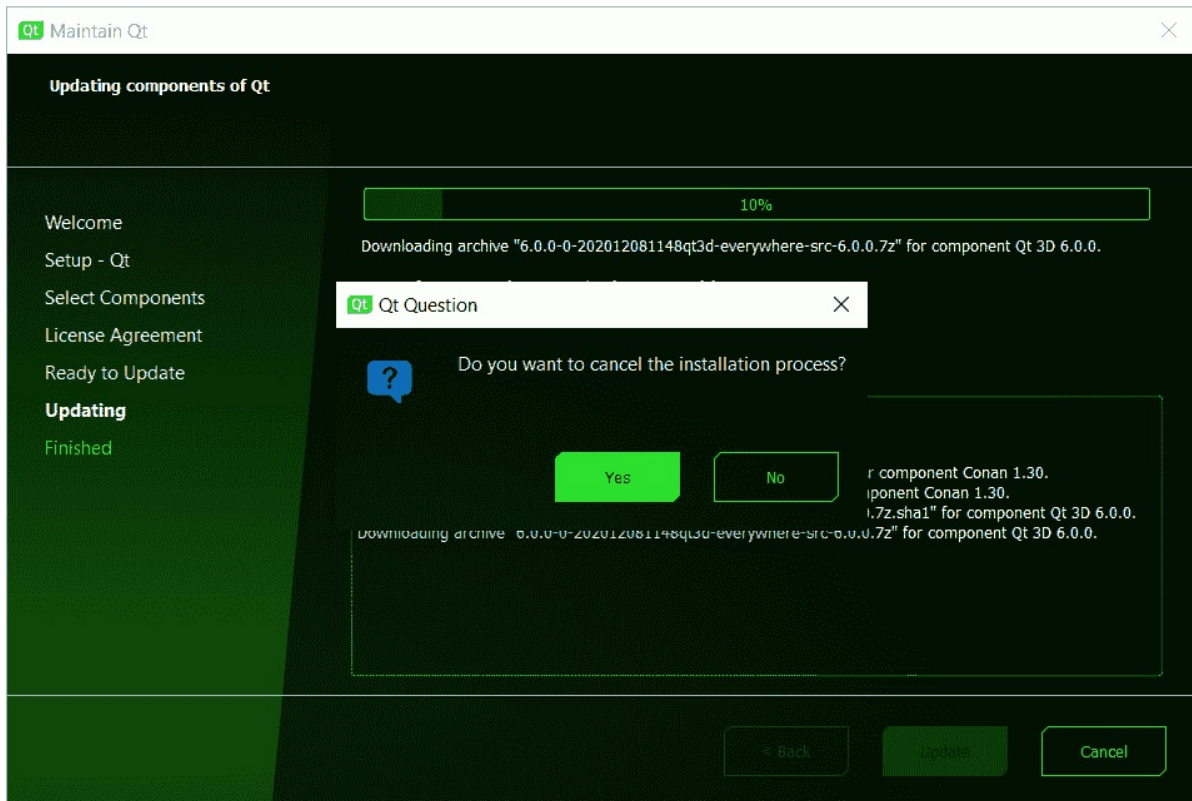


图 1.9←Cancel(取消)对话框

6. Launch the **Maintenance Tool** again to update existing components from the latest releases. You can click on the **Quit** button to exit the **Maintenance Tool**. Please wait while the installer fetches the meta-information from the remote repository. Click on the **Next** button to see the available components. The update option is illustrated in the following screenshot:

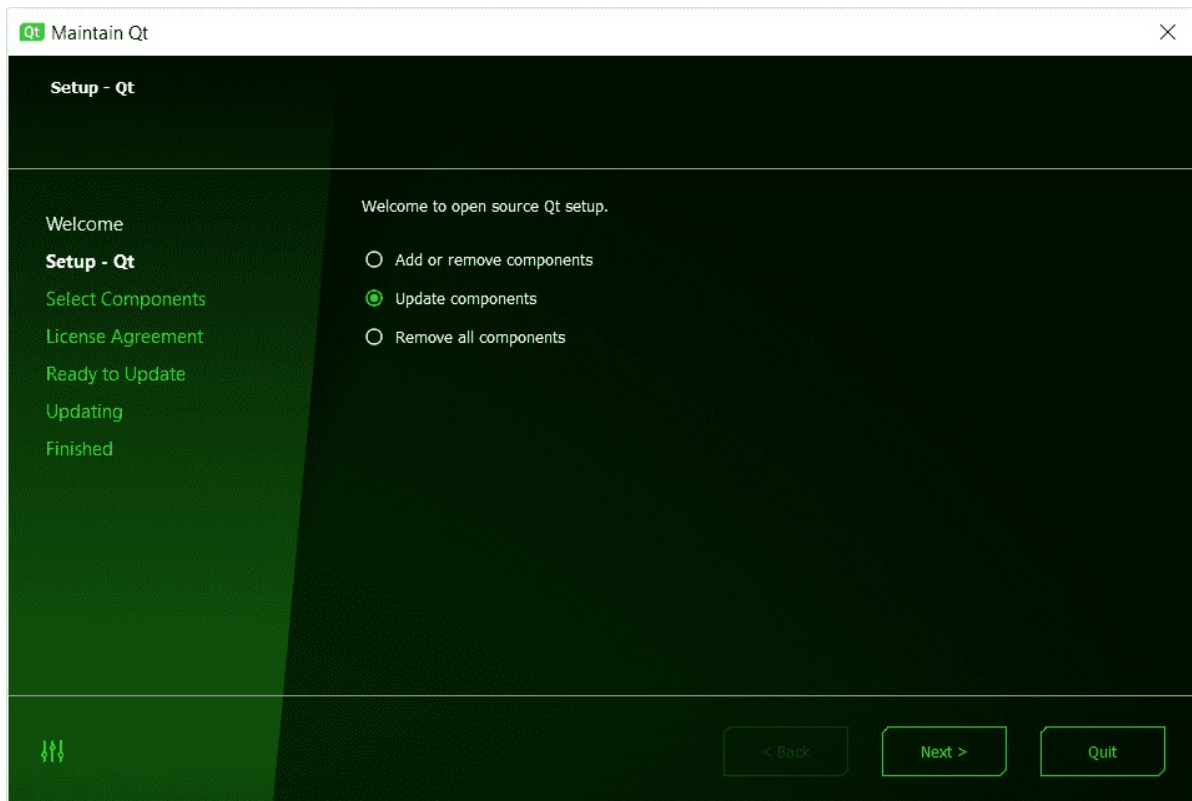


图 1.10-维护工具中的更新选项

7. Next, you can select which components to update from the checkboxes. You can choose to update all or you can update selectively. The installer will show how much storage space will be required for the update, as illustrated in the following screenshot. You can click **Next** to go to the update screen and begin the update. Then, on the next screen, click on the **Update** button to download the update packages:

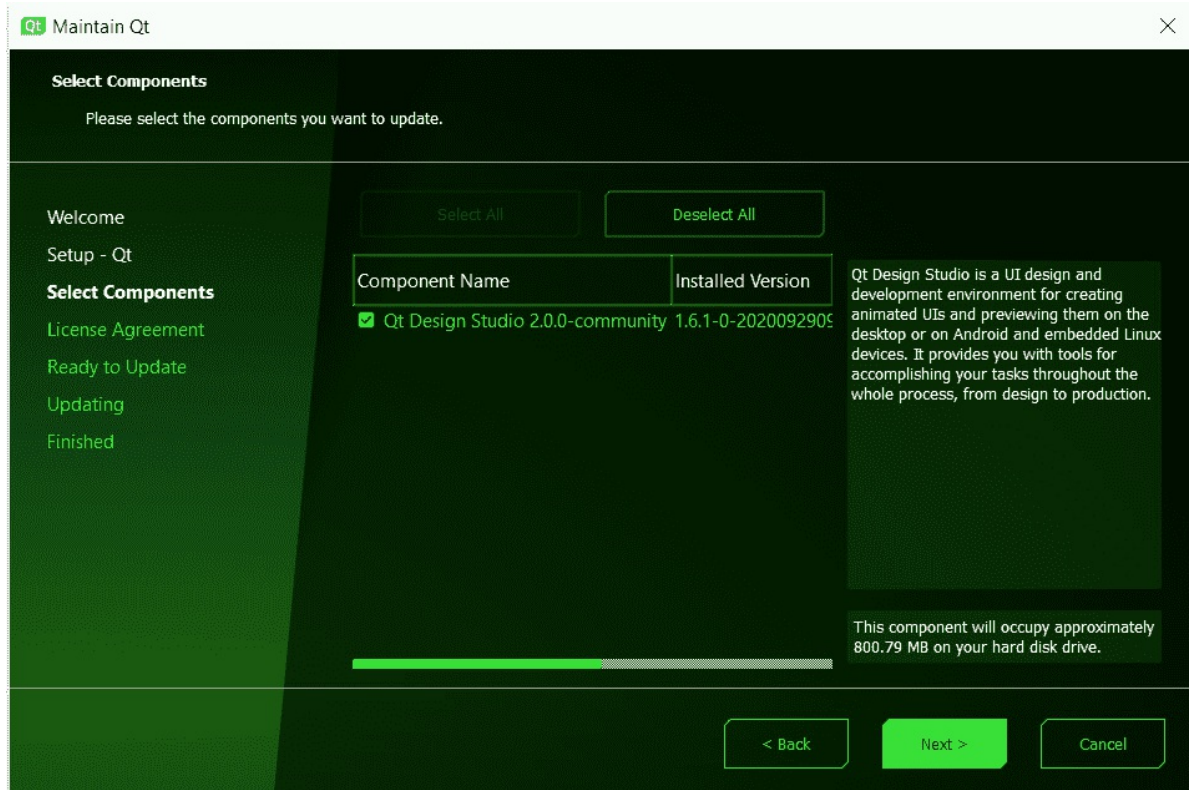


图 1.11-可供更新的组件

8. Once the installation is finished, the installer makes entries for the Maintenance Tool, which will help you make changes to the libraries later. This is illustrated in the following screenshot. Click on the **Next** button to move to the last screen of the installer:

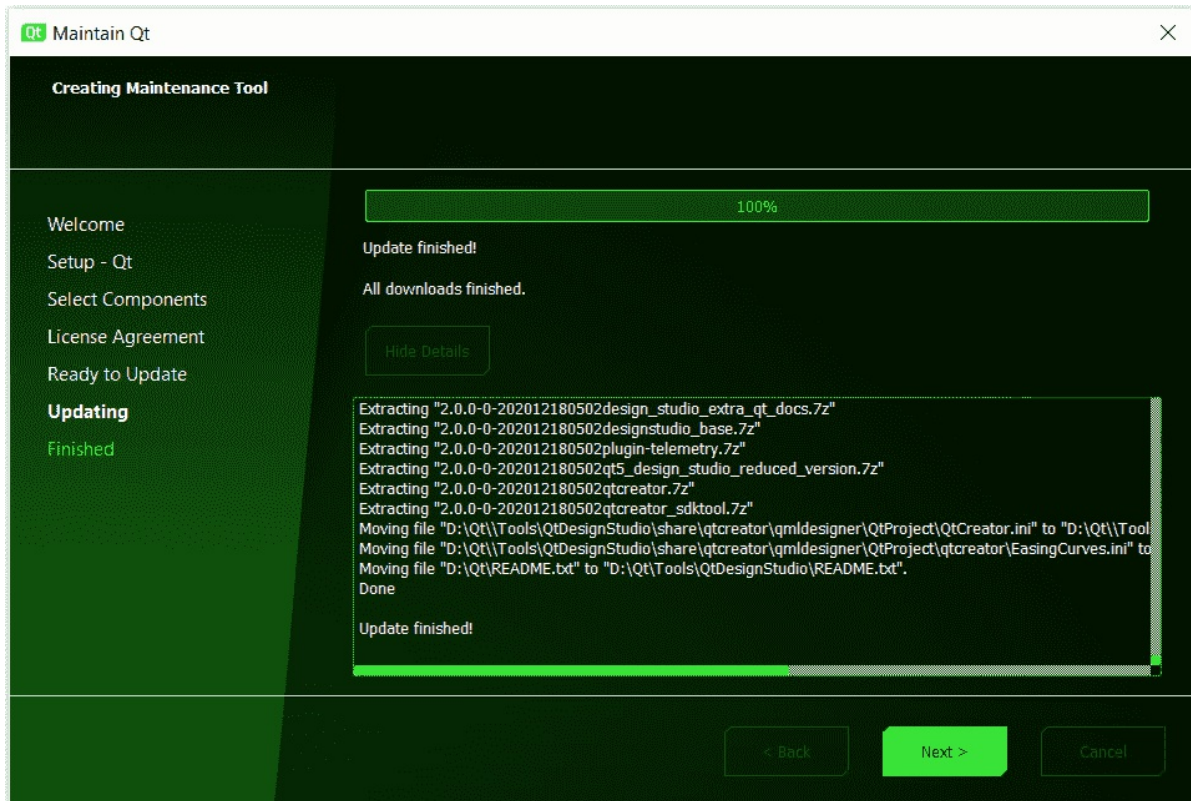


图 1.12-维护工具中的更新完成屏幕

9. 在最后一个屏幕中，您将看到重新启动和完成按钮。单击**Finish**按钮以退出 Qt 向导。
10. 同样，您可以重新启动或启动维护工具，然后选择**Remove All Components**单选按钮。点击下一步按钮开始卸载过程，如下图所示：

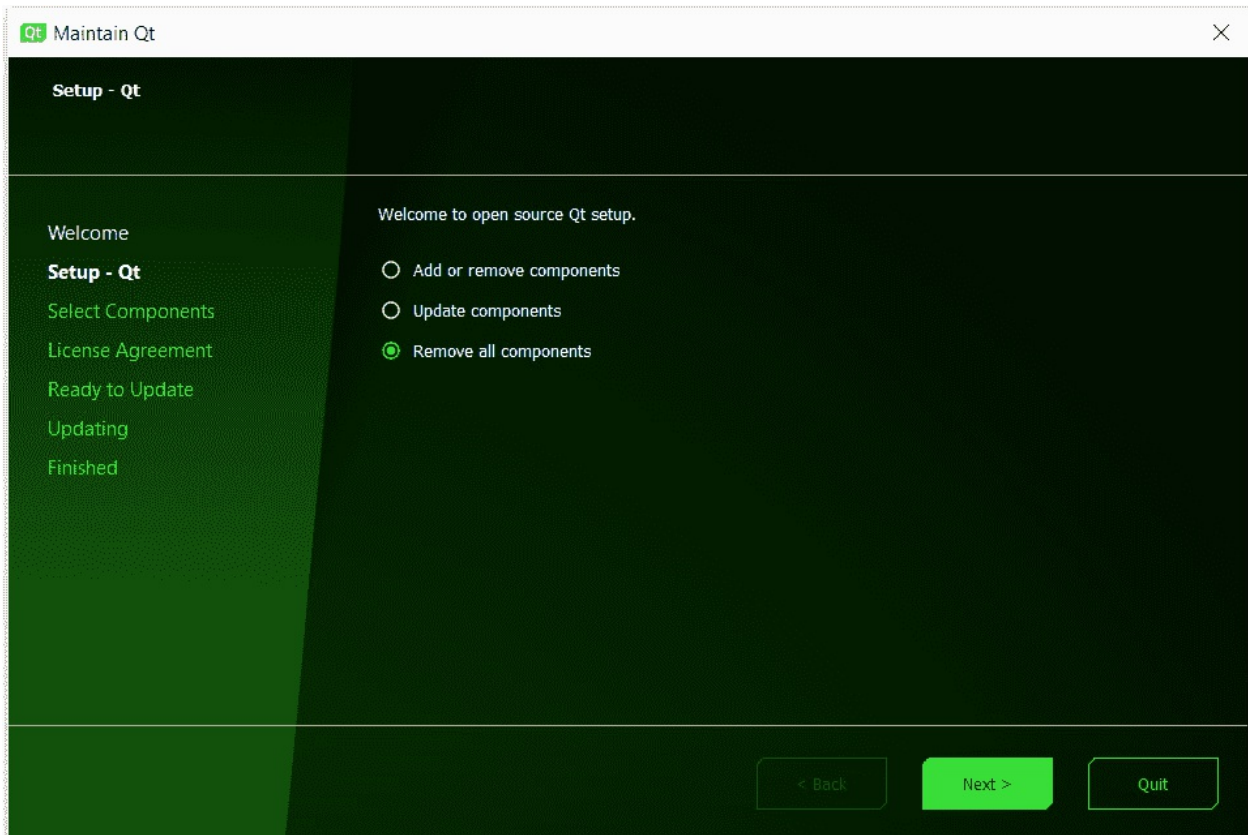


图 1.13-维护工具中的移除选项

请注意，单击卸载按钮后，所有 Qt 组件都将从您的系统中删除；如果您想要再次使用它们，则必须重新安装 Qt。如果您不想从系统中删除 Qt 组件，请单击取消上的，如下面的屏幕截图所示。如果您打算删除现有版本并使用较新版本的 Qt，则选择**Add or Remove Components** 选项，如前所述。这将删除较旧的 Qt 模块并释放磁盘空间：

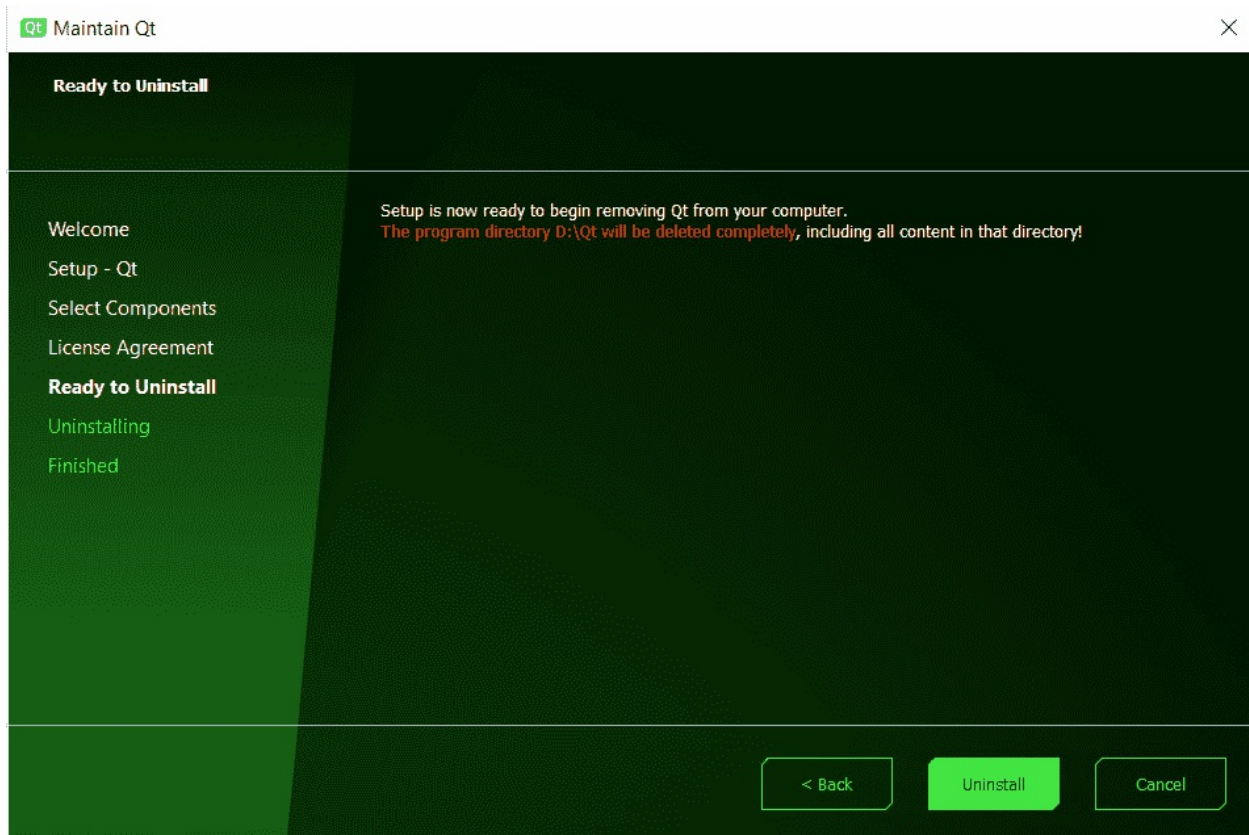


图 1.14-维护工具中的卸载屏幕

在本节中，我们了解了如何通过维护工具修改现有的 Qt 安装。现在，让我们学习如何从源代码构建和安装 Qt。

从源头构建 Qt 6

如果您想自己构建框架和工具，或者尝试使用最新的未发布代码，那么您可以从源代码构建 Qt。如果您打算从源代码开发特定的 Qt 版本，那么您可以从官方发布链接下载 Qt6 源代码，如下所示：https://download.qt.io/official_releases/qt/6.0/。

如果您是商业客户，则可以从您的 Qt 帐户门户下载源包。特定于平台的构建说明将在接下来的小节中讨论。

您也可以从 **GitHub** 存储库克隆，然后签出所需的分支。在撰写本书时，Qt6 分支仍在 Qt5 超级模块内部。您可以通过以下链接克隆存储库：`git://code.qt.io/qt/qt5.git`。

为了便于维护，将来可能会将`qt5.git`存储库重命名为`qt.git`。请参照QTQAINFRA-4200QT 票。关于如何从 Git 构建 Qt 的详细说明可以在以下链接中找到：https://wiki.qt.io/Building_Qt_6_from_Git。

确保您的计算机上安装了最新版本的 **Git**、**Perl** 和 **Python**。在继续下一节中的特定于平台的指令之前，请确保有一个正在运行的 **C++** 编译器。

在 **Windows** 上从源安装 Qt

要从源代码在 Windows 上安装 Qt 6，请执行以下后续步骤：

1. 首先，从 **Git** 或前面提到的开源下载链接下载源代码。您将获得一个压缩文件`qt-everywhere-src--%VERSION%.zip`，其中`%VERSION%`是最新版本(如`qt-everywhere-src-6.0.3.zip`)。请注意，像`-everywhere-src-`这样的后缀可能会在将来被删除。
2. 下载源代码存档后，将其解压缩到所需的目录-例如，`C:\Qt6\src`。
3. 在下一步中，使用支持的编译器和所需的构建工具配置构建环境。
4. 然后，将`CMake`、`ninja`、`Perl`和`Python`的安装目录分别添加到您的`PATH`环境变量中。
5. 下一步是构建 Qt 库。要为您的计算机类型配置 Qt 库，请运行源目录中的`configure.bat`脚本。
6. 在此步骤中，通过在命令提示符下键入以下命令来构建 Qt:


```
>cmake --build . -parallel
```

7. 接下来，在命令提示符下输入以下命令以在您的计算机上安装 Qt:

```
>cmake --install .
```

您的 Windows 计算机现在已准备好使用 Qt。

要了解有关配置选项的更多信息，请访问以下链接：

<https://doc.qt.io/qt-6/configure-options.html>

详细的构建说明可在以下链接中找到：

<https://doc.qt.io/qt-6/windows-building.html>

在 Linux 上从源安装 Qt

要在 Linux 发行版上构建源包，请在您的终端上运行以下指令集：

1. 首先，从 Git 或前面提到的开源下载链接下载源代码。 您将获得一个压缩文件 `qt-everywhere-src-%VERSION%.tar.xz`，其中 `%VERSION%` 是最新版本(如 `qt-everywhere-src-6.0.3.tar.xz`)。 请注意，像 `-everywhere-src-` 这样的后缀可能会在将来被删除。
2. 下载源归档文件后，解压缩该归档文件并将其解压缩到所需的目录-例如， `/qt6`，如以下代码片段所示：

```
$ cd /qt6
$ tar xvf qt-everywhere-opensource-src-%VERSION%.tar.xz
$ cd /qt6/qt-everywhere-opensource-src-%VERSION%
```

3. 要为您的计算机配置 Qt 库，请运行源目录中的 `./configure` 脚本，如以下代码片段所示：

```
$ ./configure
```

4. 要创建库并编译所有示例、工具和教程，请键入以下命令：

```
$ cmake --build . --parallel  
$ cmake --install .
```

5. The next step is to set the environment variables. In `.profile` (if your shell is `bash`, `ksh`, `zsh`, or `sh`), add the following lines of code:

```
PATH=/usr/local/Qt-%VERSION%/bin:$PATH  
export PATH
```

在`.login`中(如果您的 `shell` 是`csh`或`tcsh`)，添加以下代码行：

```
setenv PATH /usr/local/Qt-%VERSION%/bin:$PATH
```

如果您使用不同的 `shell`，请相应地修改您的环境变量。Qt 现在可以在您的 Linux 机器上使用了。

有关 Linux/X11 的详细构建说明，请访问以下链接：

<https://doc.qt.io/qt-6/linux-building.html>

在 **MacOS** 上从源安装 Qt

Qt 对**Xcode**有依赖关系。要在 Mac 上安装 Qt，您的计算机上需要安装 Xcode。如果您的计算机上没有安装 Xcode，则可以继续安装 Xcode 的命令行工具：

1. 要开始，请在终端上键入以下命令：

```
$ xcode-select --install
```

2. 如果终端显示以下输出，则系统已准备好执行下一步：

```
xcode-select: error: command line tools are already installed,  
"Software Update" to install updates
```

3. 要构建源包，请在您的终端上运行以下指令集：

```
$ cd /qt6  
$ tar xvf qt-everywhere-opensource-src-%VERSION%.tar  
$ cd /qt6/qt-everywhere-opensource-src-%VERSION%
```

4. 要为您的 Mac 配置 Qt 库，请运行源目录中的 `./configure` 脚本，如下代码片段所示：

```
$ ./configure
```

5. 要创建库，请运行 `make` 命令，如下所示：

```
$ make
```

6. 如果 `-prefix` 在 `build` 目录之外，则键入以下行以安装库：

```
$ sudo make -j1 install
```

7. The next step is to set the environment variables. In `.profile` (if your shell is `bash`), add the following lines of code:

```
PATH=/usr/local/Qt-%VERSION%/bin:$PATH  
export PATH
```

在 `.login` 中(如果您的 shell 是 `csh` 或 `tcsh`)，添加以下代码行：

```
setenv PATH /usr/local/Qt-%VERSION%/bin:$PATH
```

您的机器现在可以进行 Qt 编程了。

可在此处找到 MacOS 的详细构建说明：

<https://doc.qt.io/qt-6/macOS-building.html>

在本节中，我们学习了如何在您最喜欢的平台上从源代码安装 Qt。现在，让我们来总结一下我们所学的内容。

摘要

本章解释了 Qt 框架的基础知识以及它可以用来做什么。在这里，我们讨论了使用 Qt 的历史、不同的模块和优点。我们还了解了具有许可义务的不同安装方法，给出了 Qt 在不同桌面平台上的逐步安装过程。现在，您的机器已经准备好探索 Qt 了。

在下一章中，我们将讨论 Qt Creator IDE。您将了解 IDE 的 UI、不同的配置以及如何将其用于 Qt 项目。

二、Qt Creator 简介

原文: <https://gitee.com/apacheecn/apacheecn-c-cpp-zh-pt2/blob/master/docs/cross-plat-dev-qt6-mod-cpp/02.md>

Qt Creator是 Qt 自己的集成开发环境(**IDE**), 用于跨平台应用开发。在本章中, 您将学习 Qt Creator IDE 的基础知识, 并介绍 IDE 的用户界面(**UI**)。我们还将了解如何在 Qt Creator 中创建和管理项目。本 Qt 模块介绍了如何使用 Qt Creator 开发简单的 Qt 应用、快捷方式以及面向开发人员的实用提示。

更具体地说, 我们将涵盖以下主要主题:

- Qt Creator 基础知识
- 配置 IDE 和管理项目
- 用户界面
- 编写示例应用
- 高级选项

Qt Creator 可以通过许多有用的工具和示例使您的 Qt 学习变得更容易。您需要最少的 IDE 知识才能入门。在本章结束时, 您将熟悉 Qt Creator 的使用。您还将能够在您最喜欢的桌面平台上构建和运行您的第一个 Qt 应用, 并且了解 IDE 中可用的高级选项, 您可以根据自己的喜好对其进行自定义。

技术要求

本章的技术要求与[第 1 章](#), **QT6** 简介相同。您将需要最新的 Qt 版本, 即 Qt 6.0.0 MinGW 64 位、Qt Creator 4.13.0 或更高版本, 以及 Windows 10、Ubuntu 20.04 LTS 或最新版本的 MacOS(至少高于

MacOS 10.13), 如 MacOS Catalina。Qt 支持 Windows 8.1 或 Ubuntu 18.04 等较早版本的操作系统。但是, 我们建议您升级到首选操作系统的最新版本, 以确保正常运行。在本章中, 我们使用了来自 Windows 10 平台的屏幕截图。

浏览 Qt Creator UI

Qt Creator 是 Qt 公司生产的 IDE。它集成了多个工具, 包括代码编辑器、图形 **UI(GUI)**设计器、编译器、调试器、Qt 设计器、Qt 快速设计器和 Qt 助手等。

Qt Designer 帮助设计基于小部件的 GUI, 而 Qt Quick Designer 提供了在设计模式下创建和编辑基于 QML 的 GUI 的 UI。Qt 助手是一个集成的文档查看器, 只需按 **F1** 键即可打开与给定 Qt 类或函数相关的内容。

让我们首先启动 Qt Creator。二进制文件可以在 `Qt\Tools\QtCreator\bin` 中找到。您将看到如图 2.1 所示的屏幕:

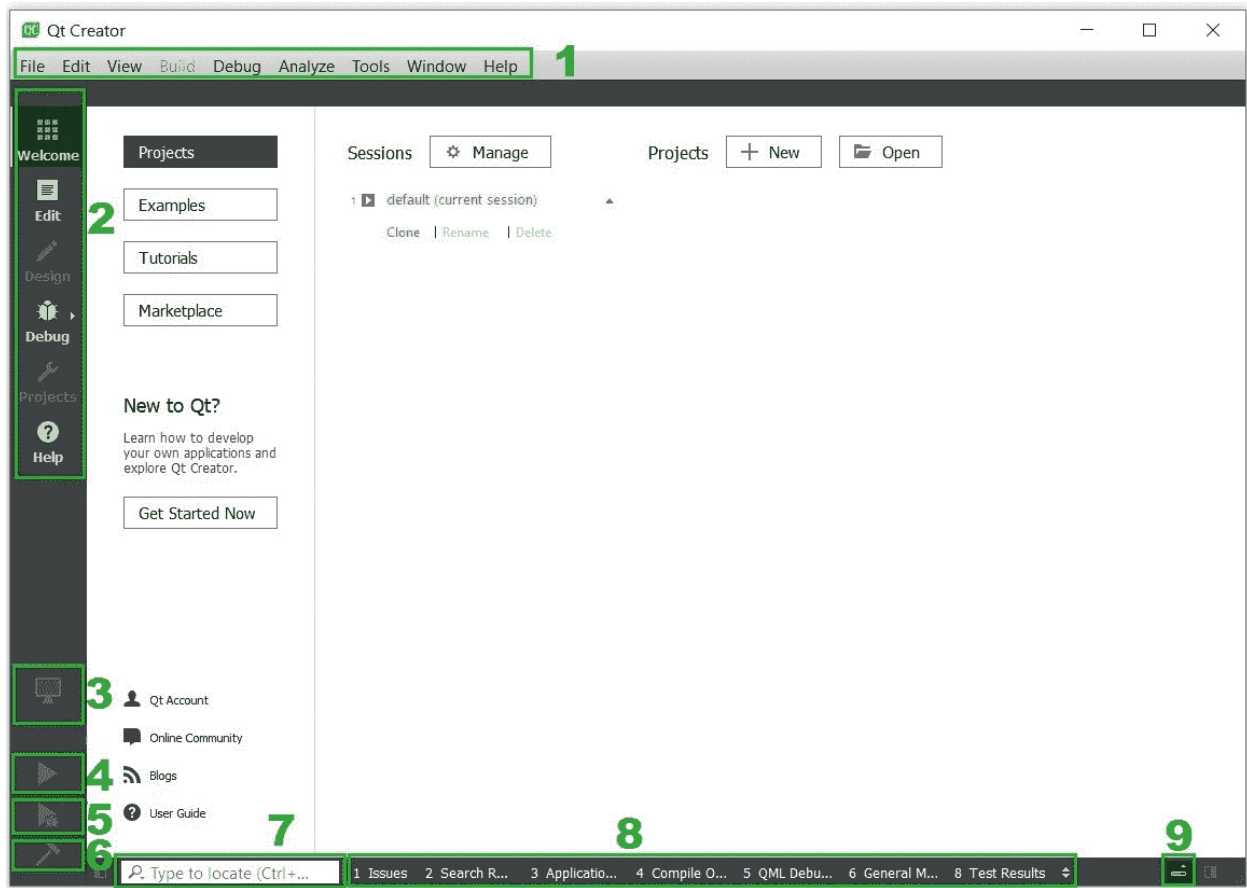


图 2.1-Qt Creator 界面

您可以在 UI 中看到以下 GUI 部分：

1. **IDE 菜单栏**：这为用户提供了一个在窗口中查找大多数特定于应用的功能的标准位置。这些功能包括创建项目、打开和关闭文件、开发工具、分析选项、帮助内容以及退出程序的方式。
2. **模式选择器**：此部分根据活动任务提供不同的模式。欢迎按钮提供打开示例、教程、最近的课程和项目的选项。编辑按钮打开代码窗口并帮助导航项目。设计按钮根据 UI 文件的类型打开 Qt Designer 或 Qt Quick Designer。**Debug**提供了分析应用的选项。**Projects**按钮帮助管理项目设置，**Help**按钮用于浏览帮助内容。
3. **套件选择器**：这有助于选择激活的项目配置和更改套件设置。

4. 运行按钮：该按钮在生成激活项目后运行它。
5. 调试按钮：这有助于使用调试器调试活动项目。
6. 构建按钮：该按钮用于构建激活的项目。
7. **Locator**：用于从任何打开的项目中打开文件。
8. 输出窗格：这包括几个窗口，用于显示编译和应用输出等项目信息。它还显示构建问题、控制台消息以及测试和搜索结果。
9. 进度指示器：该控件显示与正在运行的任务相关的进度。

当您第一次启动 Qt Creator 时，您还可以从交互式 UI 教程中获益。您也可以从菜单栏中的**Help|UI Tour**选项启动它，如图 2.2所示：

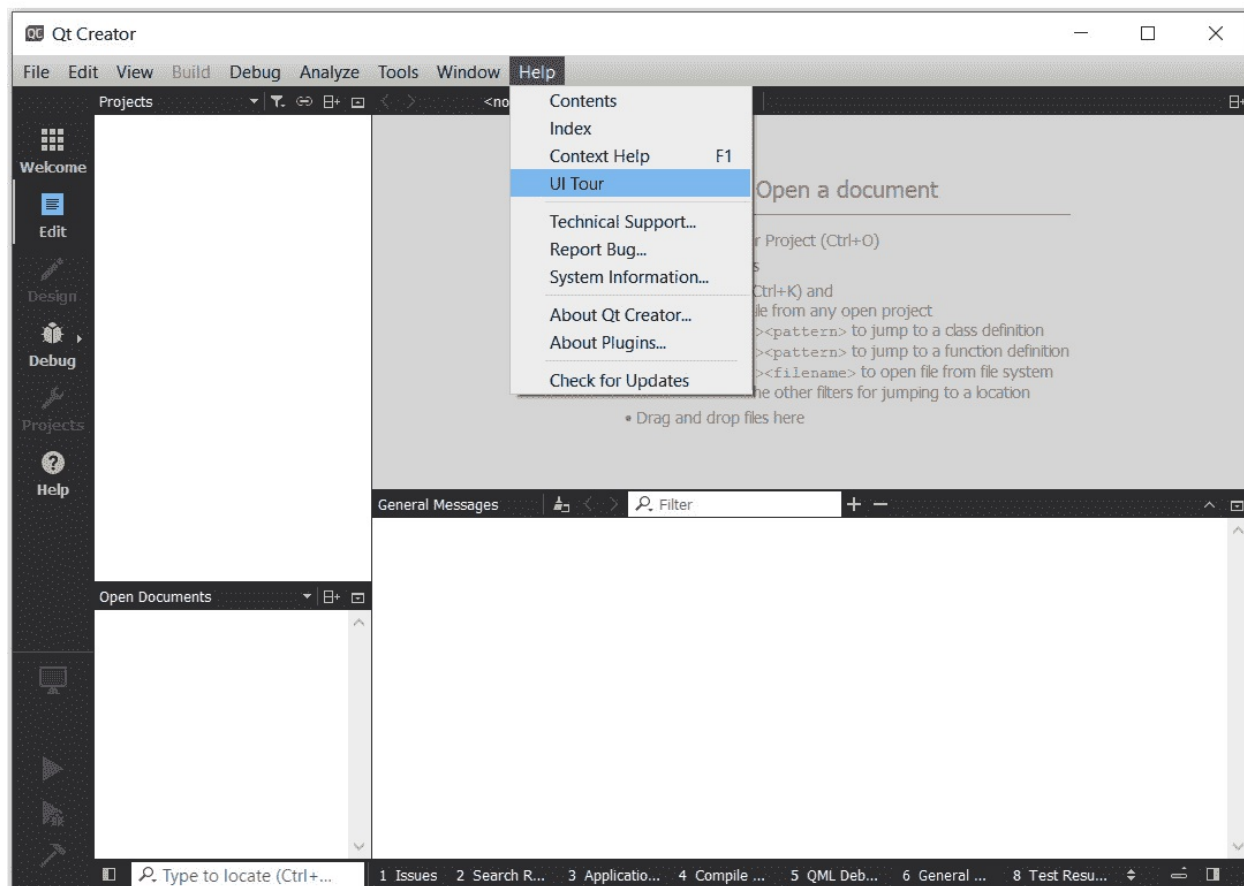


图 2.2-Qt Creator UI 巡视菜单选项

笔记 / 便条 / 票据 / 注解

如果您按`Alt`键，您将在菜单标题中看到带下划线的助记符。按相应的键打开相应的上下文菜单。

在本节中，我们了解了 IDE 中的各个部分。在下一节中，我们将使用 Qt Creator IDE 构建一个简单的 Qt 应用。

构建一个简单的 Qt 应用

让我们从一个简单的*Hello World*项目开始。*Hello World*程序是一个非常简单的程序，它显示**Hello World! **并检查 SDK 配置是否没有错误。这些项目使用最基本、最少的代码。对于这个项目，我们将使用由 Qt Creator 创建的项目框架。

按照以下步骤构建您的第一个 Qt 应用：

1. To create a new project in Qt, click on the **File** menu option on the menu bar or hit `Ctrl + N`. Alternatively, you can also click on the **+** **New** button located at the welcome screen to create a new project, as shown in *Figure 2.3*:

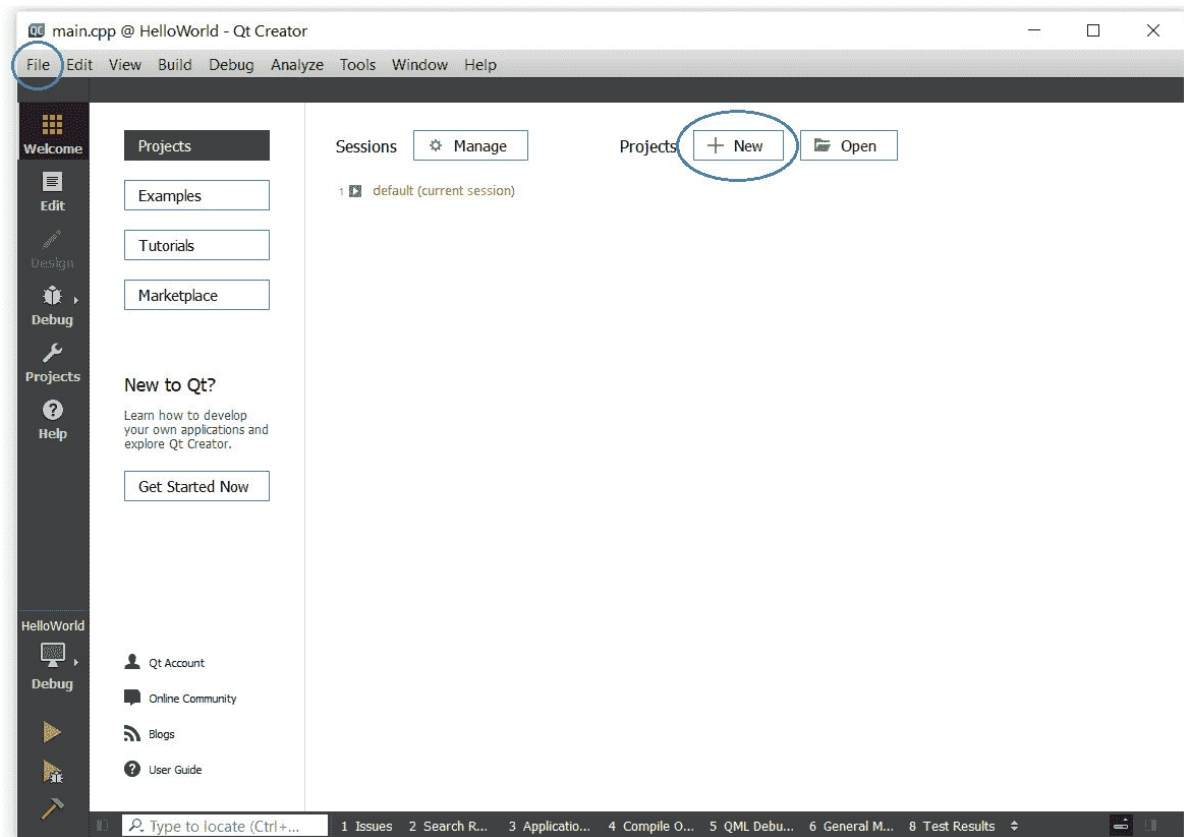


图 2.3-新建项目界面

2. Next, you can select a template for the project. You can create different types of applications, including a console application or GUI application. You can also create non-Qt projects as well as library projects. In the upper-right section, you will see a dropdown to filter templates specific to the desired target platform. Select the **Qt Widgets Application** template and then click on the **Choose...** button:

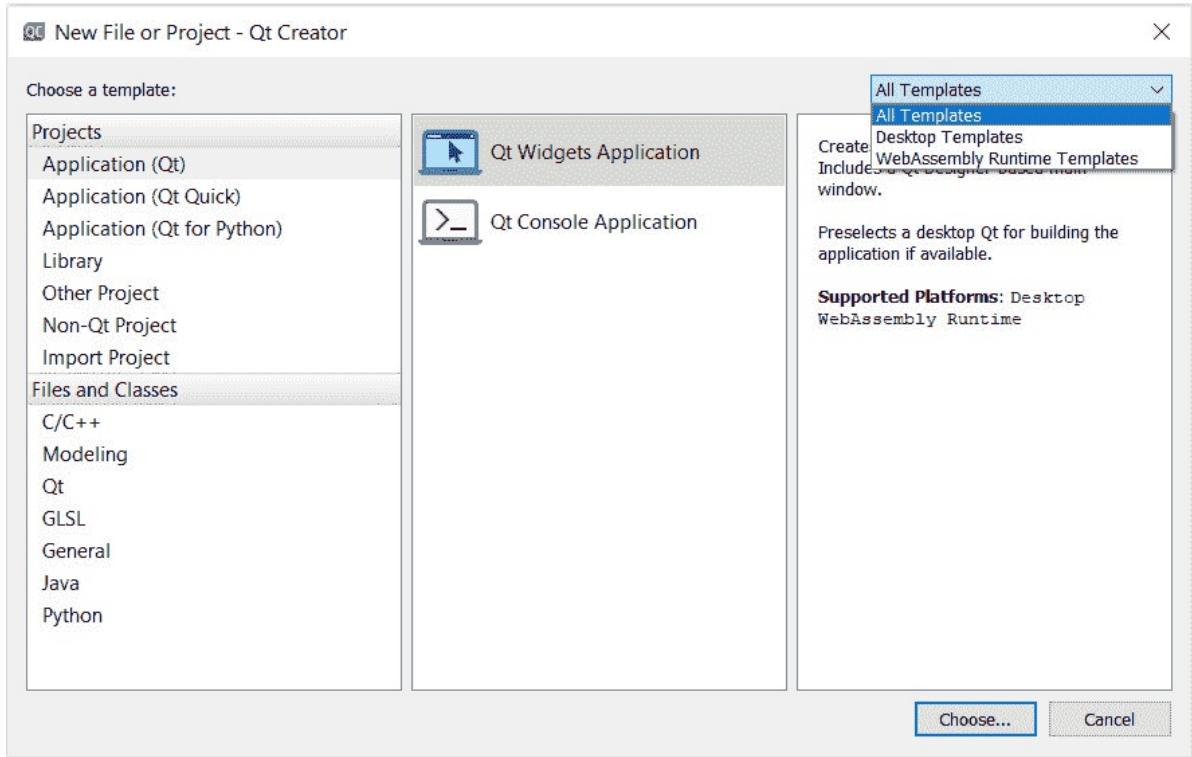


图 2.4-项目模板界面

3. In the next step, you will be asked to choose the project name and project location. You can navigate to the desired project location by clicking the **Browse...** button. Then click on the **Next** button to proceed to the next screen:

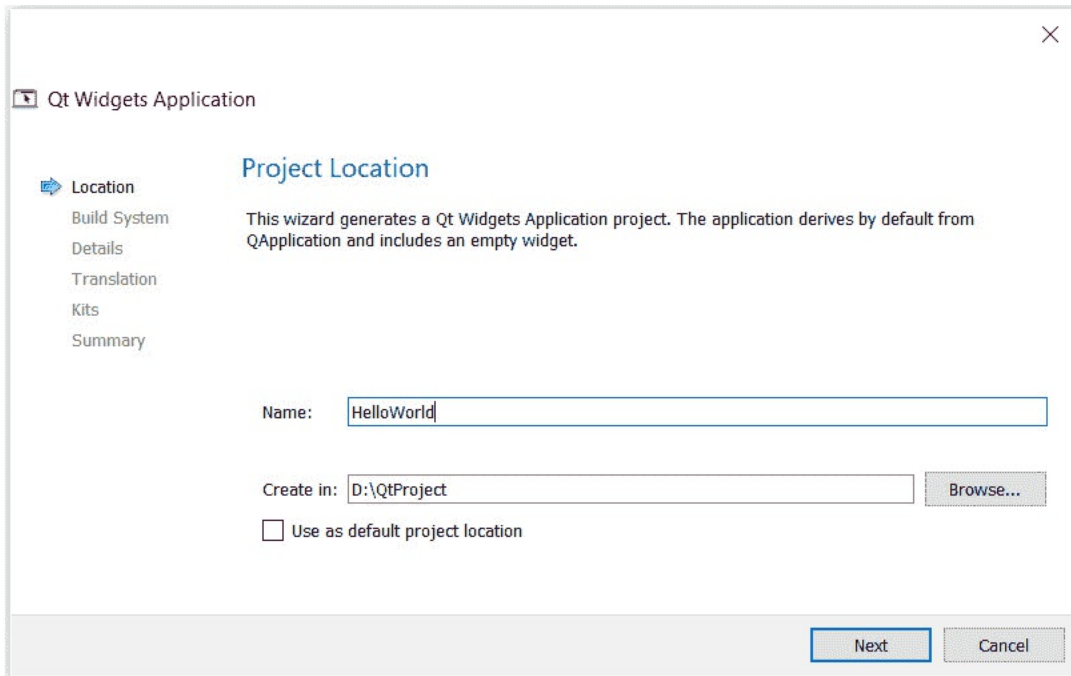


图 2.5-新建项目位置屏幕

4. You can now select the build system. By default, Qt's own build system **qmake** will be selected. We will discuss qmake more later in [Chapter 6, Signals and Slots](#). Click on the **Next** button to proceed to the next screen:

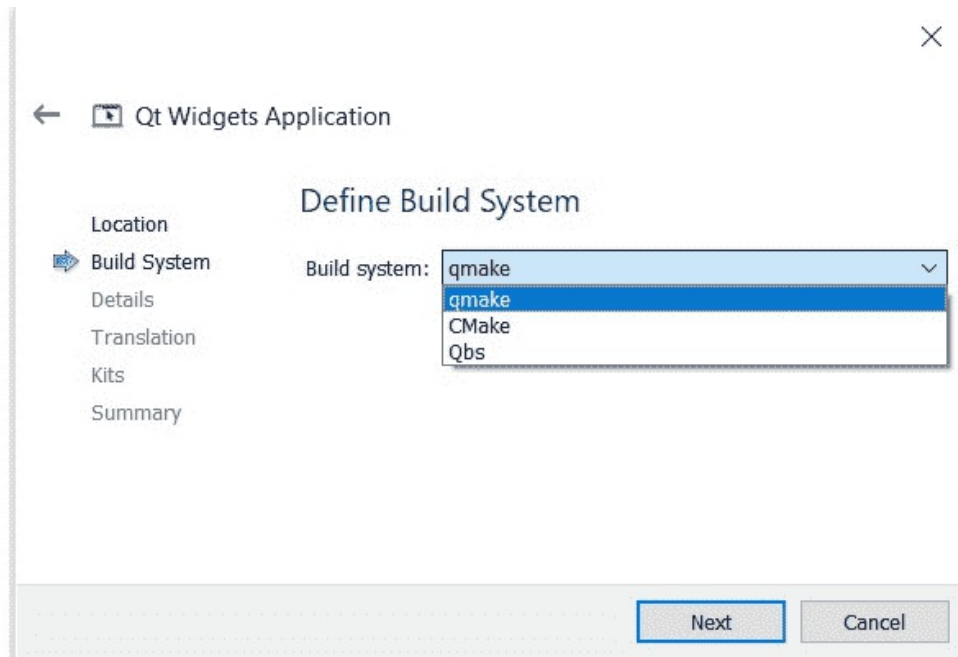


图 2.6-构建系统选择屏幕 n

5. Next, you can specify the class information and the base class to be used to automatically generate the project skeleton. If you need a desktop application with features of `MainWindow` such as `menubar`, `toolbar`, and `statusbar`, then select **QMainWindow** as the base class. We will discuss more on `QMainWindow` in [Chapter 3, GUI Design Using Qt Widgets](#). Click on the **Next** button to proceed to the next screen:

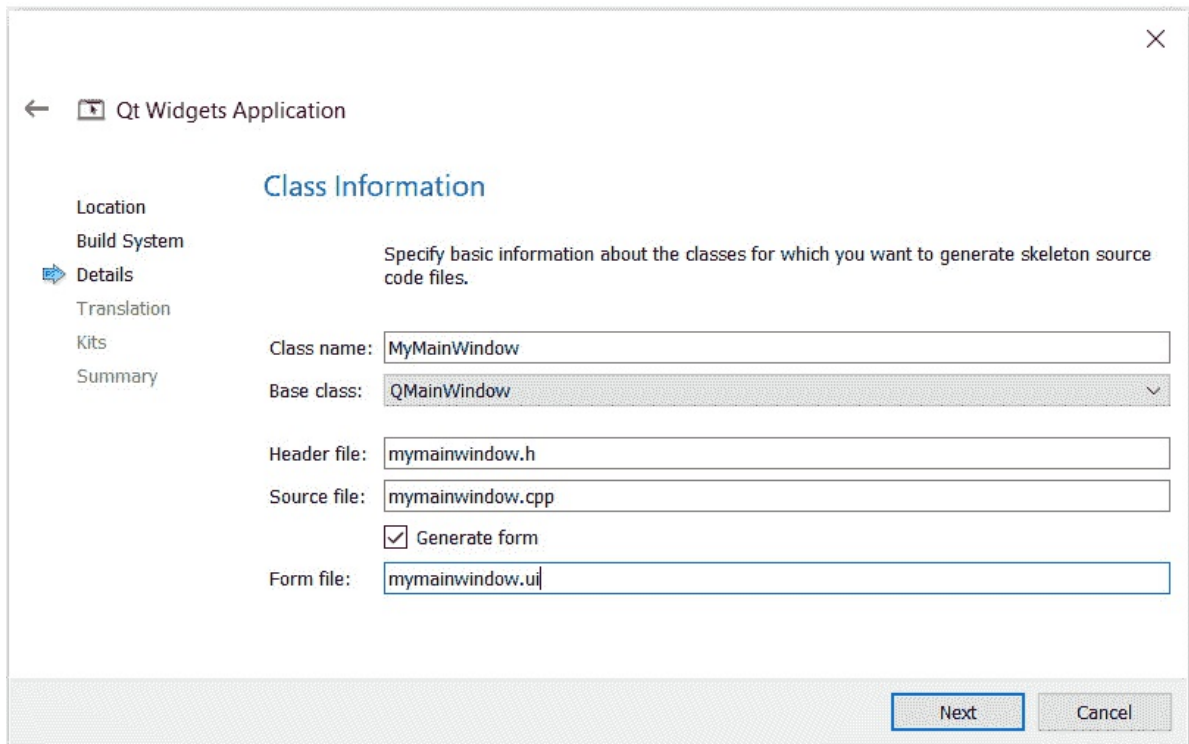


图 2.7-源代码框架生成屏幕

6. In this next step, you can specify the language for translation. Qt Creator comes with the *Qt Linguist* tool, which allows you to translate your application into different languages. You may skip this step for now. We will discuss **Internationalization (i18n)** in [Chapter 11, Internationalization](#). Click on the **Next** button to proceed to the next screen:

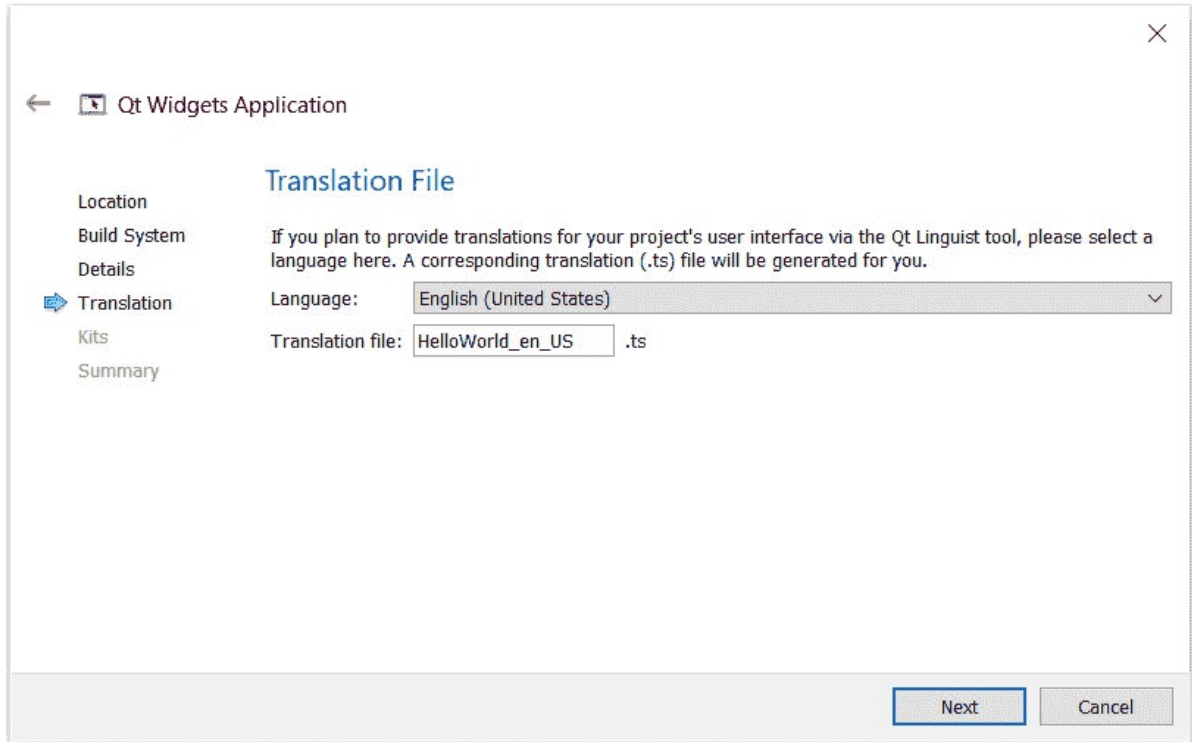


图 2.8-翻译文件创建屏幕 en

7. In the next step, you can select a kit to build and run your project. To build and run the project, at least one kit must be active and selectable. If your desired kit is shown as grayed out, then you may have some kit configuration issues. When you install Qt for a target platform, the build and run settings for the development targets usually get configured automatically. Click on the checkbox to select one of the desktop kits such as **Desktop Qt 6.0.0 MinGW 64-bit**. Click on the **Next** button to proceed to the next screen:

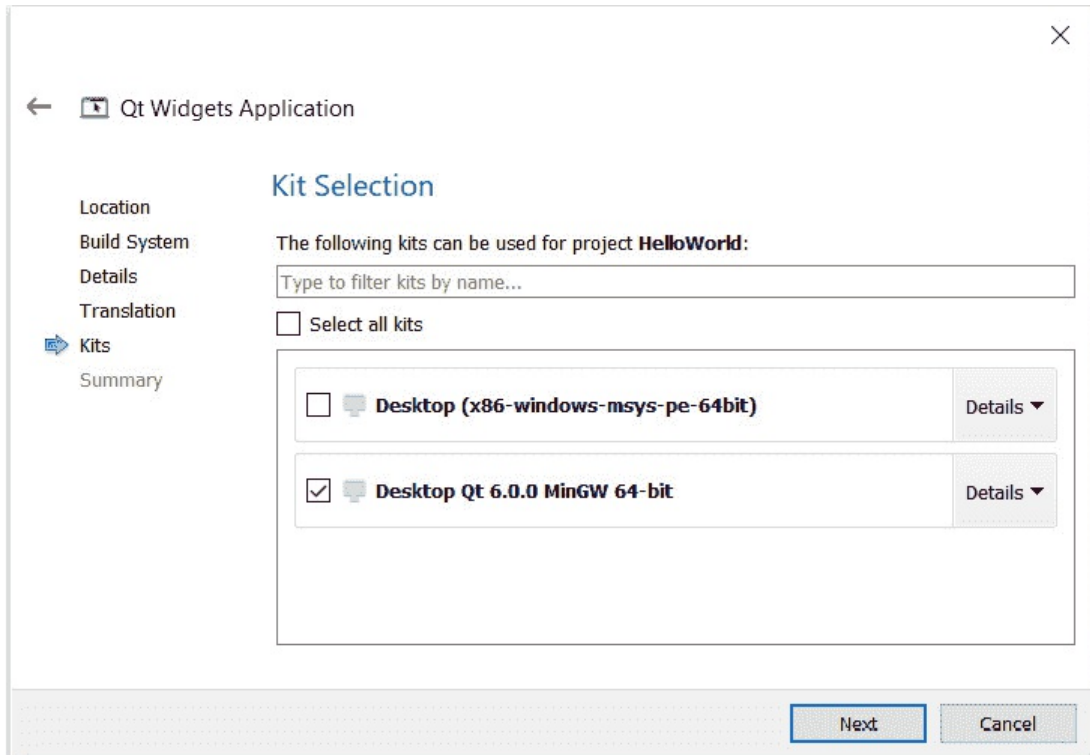


图 2.9-套件选择屏幕

8. Version control allows you or your team to submit code changes to a centralized system so that each and every team member can obtain the same code without passing files around manually. You can add your project into the installed version control system on your machine. Qt has support for several version control systems within the Qt Creator IDE. You may skip version control for this project by selecting . Click on the **Finish** button to complete the project creation:

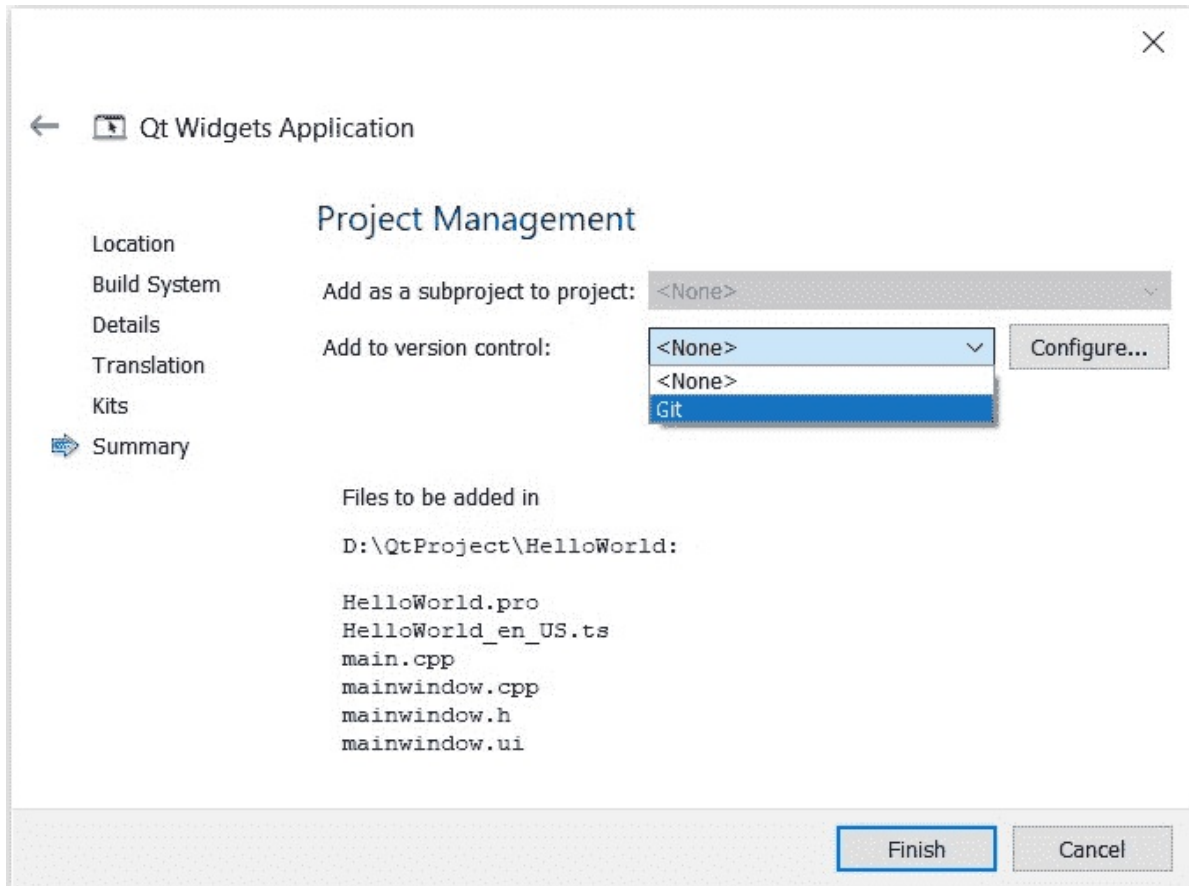


图 2.10-项目管理屏幕

9. Now you will see the generated files on the left side of the editor window. Click on any file to open it in the coding window, the most used component of the Qt Creator. The code editor is used in **Edit** mode. You can write, edit, refactor, and beautify your code in this window. You can also modify the fonts, font size, colors, and indentation. We will learn more about these in the *Understanding advanced options* section later in this chapter:

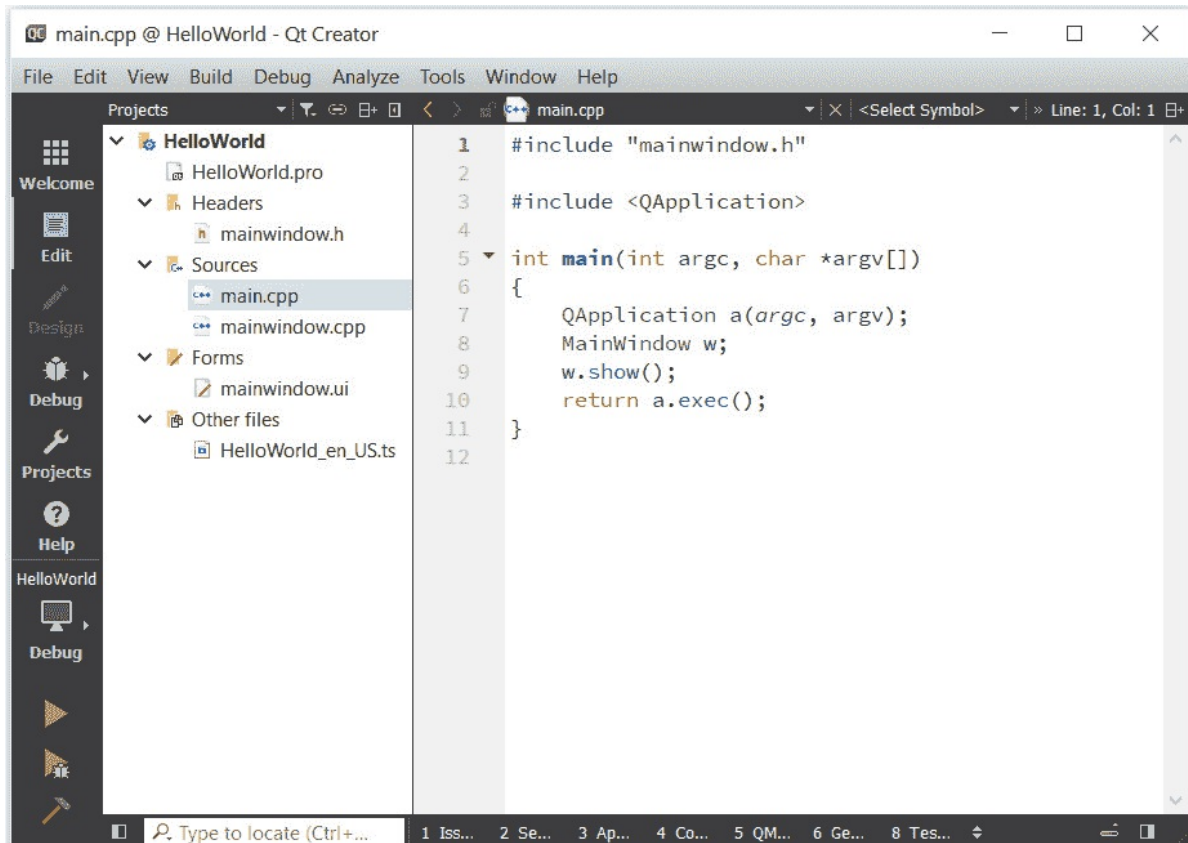


图 2.11-生成的文件和代码编辑器窗口

0. You can now see a `.pro` file inside your project folder. In the current project, the `HelloWorld.pro` file is the project file. This contains all the information required by qmake to build the application. This file is autogenerated during the project creation and contains the relevant details in a structured fashion. You can specify files, resources, and target platforms in this file. You need to run qmake again if you make any modifications to the `.pro` file contents, as shown in *Figure 2.12*. Let's skip modifying the contents for this project:

![Figure 2.12 - The contents of the project file](img/Figure_2.12)

图 2.12-项目文件的内容

1. 您可以在编辑器窗口的左侧找到扩展名为`.ui`的表单文件。让我们双击`mainwindow.ui`文件将其打开。在这里，您可以看到该文件在另一个界面下打开：**Qt Designer**。您可以看到，模式选择面板已切换到设计模式。我们将在下一章更多地讨论 **Qt** 设计器。
2. 现在将**display widgets**类别下列出的**Label**控件拖到右侧表单的中心，如图 2.13所示。
3. Next, double-click on the item you dragged in, and type `Hello World!`. Hit the *Enter* key on your keyboard or click with the mouse anywhere outside the control to save the text:

![Figure 2.13 - The designer screen](img/Figure_2.13_B16231.jpg)

图 2.13-设计器屏幕

4. 最后一步是按套件选择器按钮下方的**Run**按钮。一旦阅读器点击**Run**按钮，项目将自动构建。**Qt Creator** 足够聪明，能够计算出项目需要首先构建。您可以单独构建和运行应用。编译几秒钟后，您将看到一个窗口，其中的文本显示为**Hello World!**：

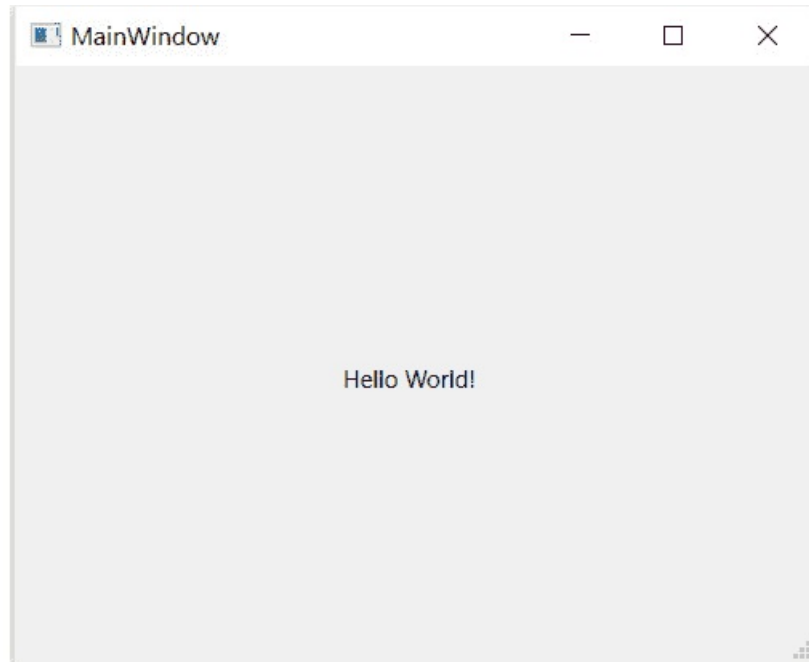


图 2.14-示例 GUI 应用的显示输出

祝贺您，您已经创建了您的第一个基于 Qt 的 GUI 应用！现在，我们来看看 Qt Creator 中可用的不同高级选项。

了解高级选项

安装 Qt Creator 时，会使用默认配置进行安装。您可以自定义 IDE 并配置其外观或设置您喜欢的编码样式。

进入顶部菜单栏，点击**Tools**选项，然后选择**Options...**。您将在左侧栏上看到可用的类别列表。每个类别都提供了一组用于自定义 Qt Creator 的选项。作为初学者，您可能根本不需要更改设置，但让我们熟悉一下可用的不同选项。我们将从查看管理工具包开始。

管理套件

Qt Creator 可以自动检测已安装的 Qt 版本和可用的编译器。它对用于构建和运行项目的配置进行分组，以使其跨平台兼容。这组配置以套

件的形式存储。每个工具包都包含一组描述环境的参数，例如目标平台、编译器和 Qt 版本。

首先单击左边栏中的**Kits**选项。这将自动检测并列出可用的套件，如图 2.15 所示。如果任何套件显示黄色或红色警告标记，则表示配置有故障。在这种情况下，您可能必须选择正确的编译器和 Qt 版本。您还可以通过单击添加按钮来创建自定义套件。如果您想使用新的套件，那么不要忘记单击**Apply**按钮。我们将继续使用默认桌面配置，如下所示：

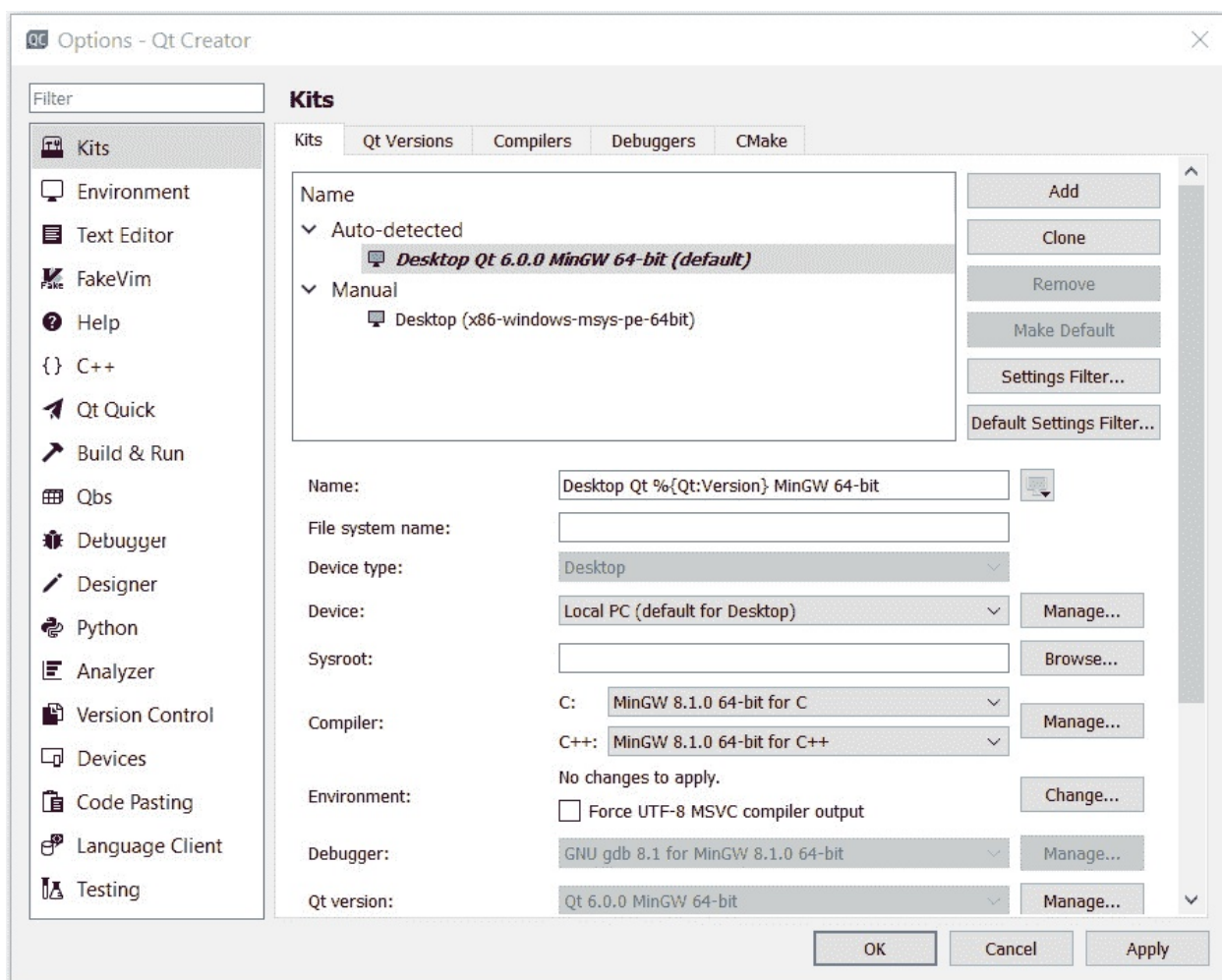


图 2.15-套件配置屏幕

现在，让我们继续到工具包部分下的**Qt Versions**选项卡。

在此选项卡中，您可以看到系统上可用的 Qt 版本。理想情况下，会自动检测到该版本。如果未检测到，则单击**Add...** 按钮，然后浏览到 **qmake** 的路径以添加所需的 Qt 版本。Qt 对其版本使用定义的编号方案。例如，Qt 6.0.0 表示 Qt 6.0 和 6 作为主要 Qt 版本的第一个补丁发布。每个版本都对可接受的更改量有限制，以确保 API 稳定。Qt 尝试维护版本之间的兼容性。但是，由于主要版本中的代码清理和体系结构更改，这并不总是可能的：

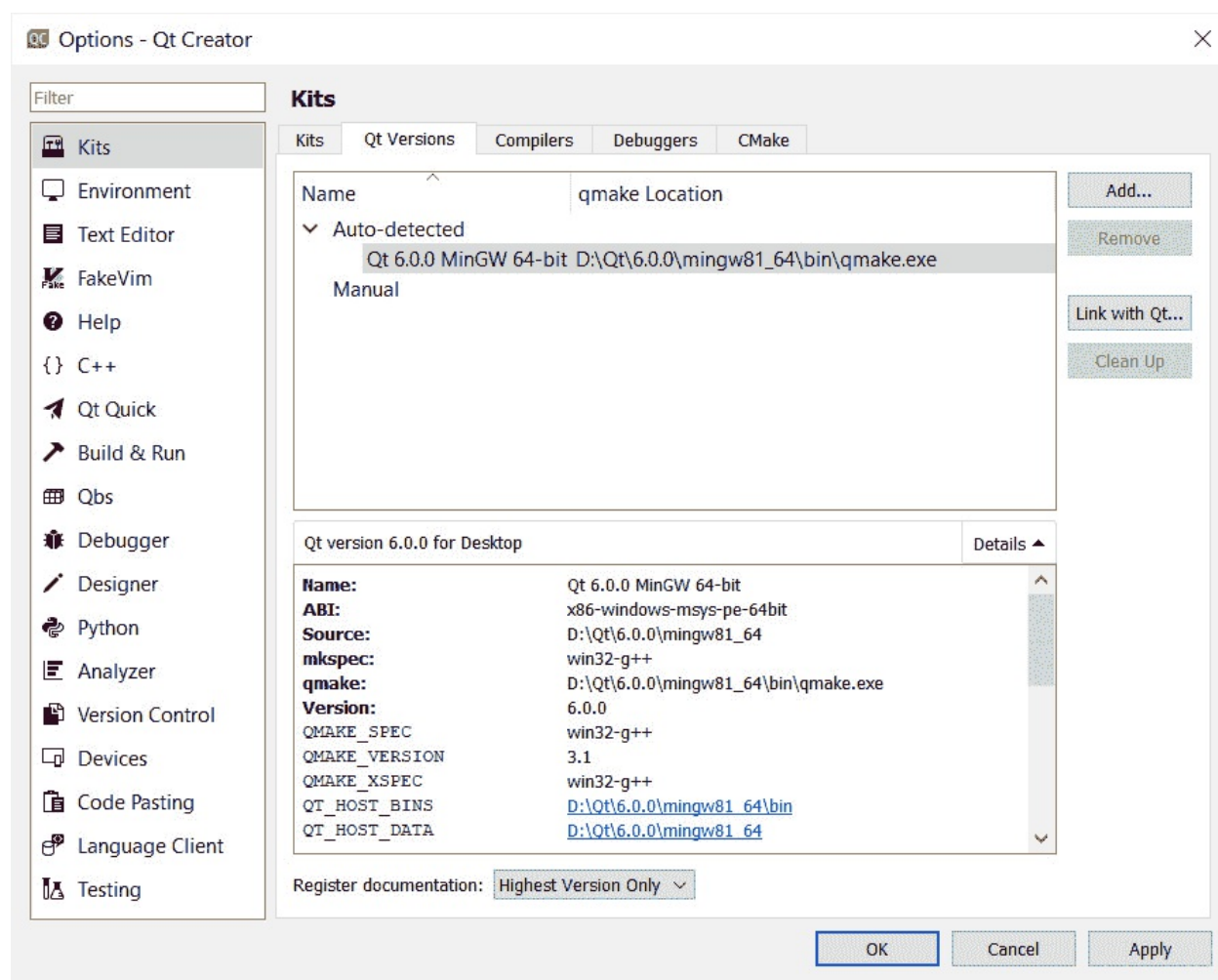


图 2.16-可用的 Qt 版本

重要音符

Qt 软件版本使用 `Major.Minor.Patch` 的版本控制格式。主要版本可能会破坏二进制文件和源代码的向后兼容性，但可以保持源代码兼容性。次要版本具有二进制和源代码向后兼容性。补丁版本对二进制文件和源代码既有向后兼容性，也有向前兼容性。

我们不会讨论工具包部分下的所有选项卡，因为其他选项卡需要关于编译器、调试器和构建系统的知识。如果您是经验丰富的开发人员，您可以浏览选项卡并根据需要进行更改。让我们继续到左侧栏中的环境类别。

环境

该选项允许用户选择他们喜欢的语言和主题。默认情况下，Qt Creator 使用系统语言。它不支持许多语言，但大多数流行语言都可用。如果更改为其他语言，请单击 **Apply** 按钮并重新启动 Qt Creator 以查看更改。请注意，这些环境选项与构建环境不同。您将看到一个类似于的界面，如图 2.17 所示：

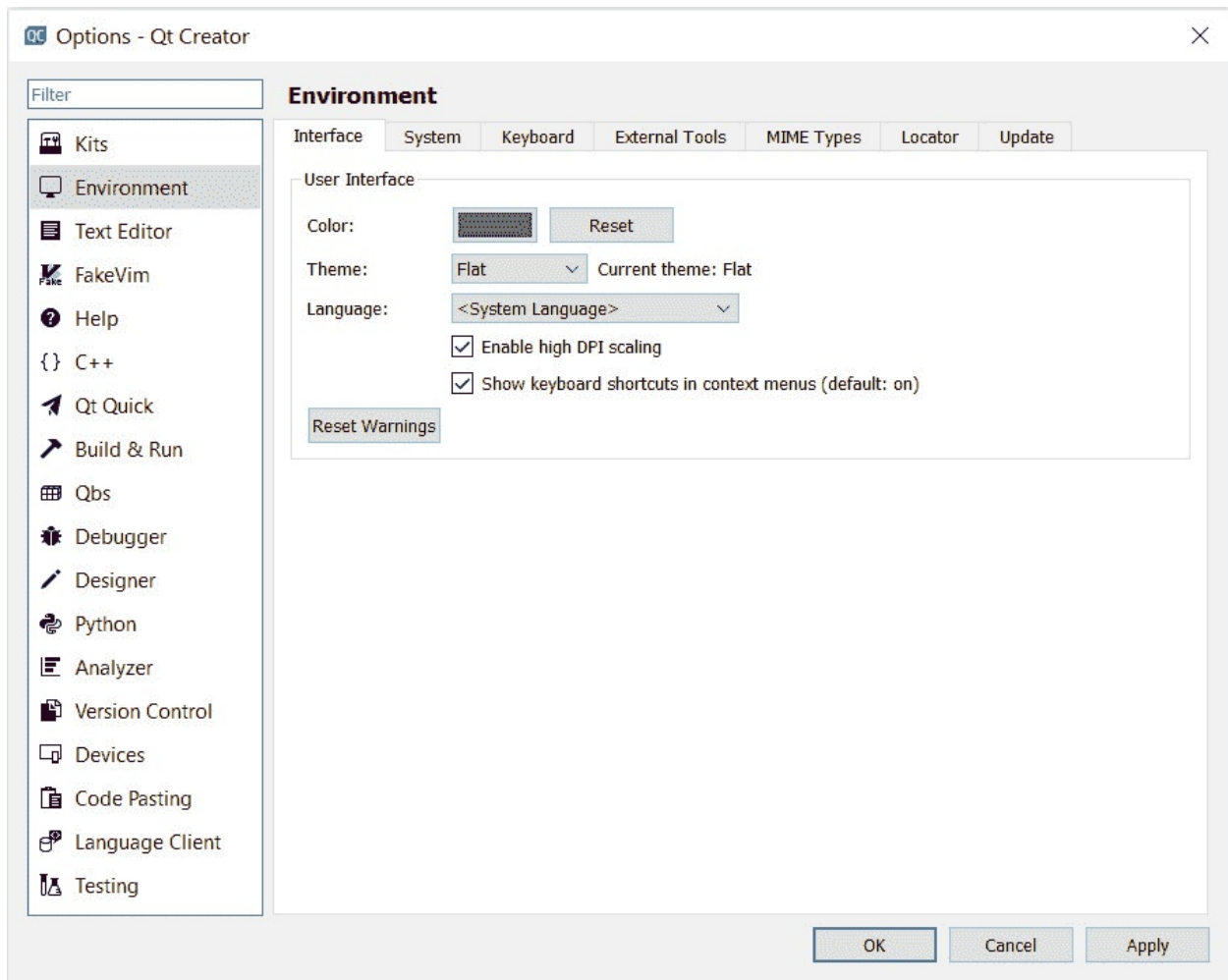


图 2.17-环境设置选项

您还会看到一个复选框，上面写着启用高 **DPI** 缩放。Qt Creator 在不同的操作系统上以不同方式处理高每英寸点数(**DPI**)缩放，如下所示：

- 在 Windows 上，Qt Creator 检测默认比例因子并相应地使用它。
- 在 Linux 上，Qt Creator 将是否启用高 DPI 扩展的决定权留给用户。这样做是因为有许多 Linux 版本和窗口系统。
- 在 MacOS 上，Qt Creator 强制 Qt 使用系统缩放因子作为 Qt Creator 缩放因子。

要覆盖默认的方法，可以切换复选框选项并单击**Apply**按钮。这些更改

将在您重新启动 IDE 后反映出来。现在让我们看一下键盘选项卡。

键盘快捷键

键盘部分允许用户浏览现有的键盘快捷键并创建新的快捷键。 **Qt Creator** 有许多内置的键盘快捷键，对开发人员非常有用。如果您最喜欢的快捷键丢失，您也可以创建自己的快捷键。您还可以为列表中未显示的功能指定自己的键盘快捷键，例如，在文本编辑器中选择单词或行。

下面列出了一些日常开发常用的快捷方式：

New File or Project	CTRL + N
Open File or Project	CTRL + O
Save Current File	CTRL + S
Save All	CTRL + SHIFT + S
Close Current File	CTRL + W
Close All	CTRL + SHIFT + W
Quit Qt Creator	CTRL + Q
Return to Edit mode	ESCAPE
Switch Current File	CTRL + TAB
Force Code Completion	CTRL + SPACE
Start Debugging	F5
Stop Debugging	SHIFT + F5
Step Over	F10
Step Into	F11
Step Out	SHIFT + F11
Toggle Breakpoint	F9
Build Current Project	CTRL + B
Run	CTRL + R
Build All	CTRL + SHIFT + B
Locate	CTRL + K

图 2.18-一些常用的键盘快捷键

快捷键按类别分组。要在列表中查找键盘快捷键，请在**Filter**字段中输入函数名或快捷键。在图 2.19中，我们搜索了与new相关的可用快捷方式：

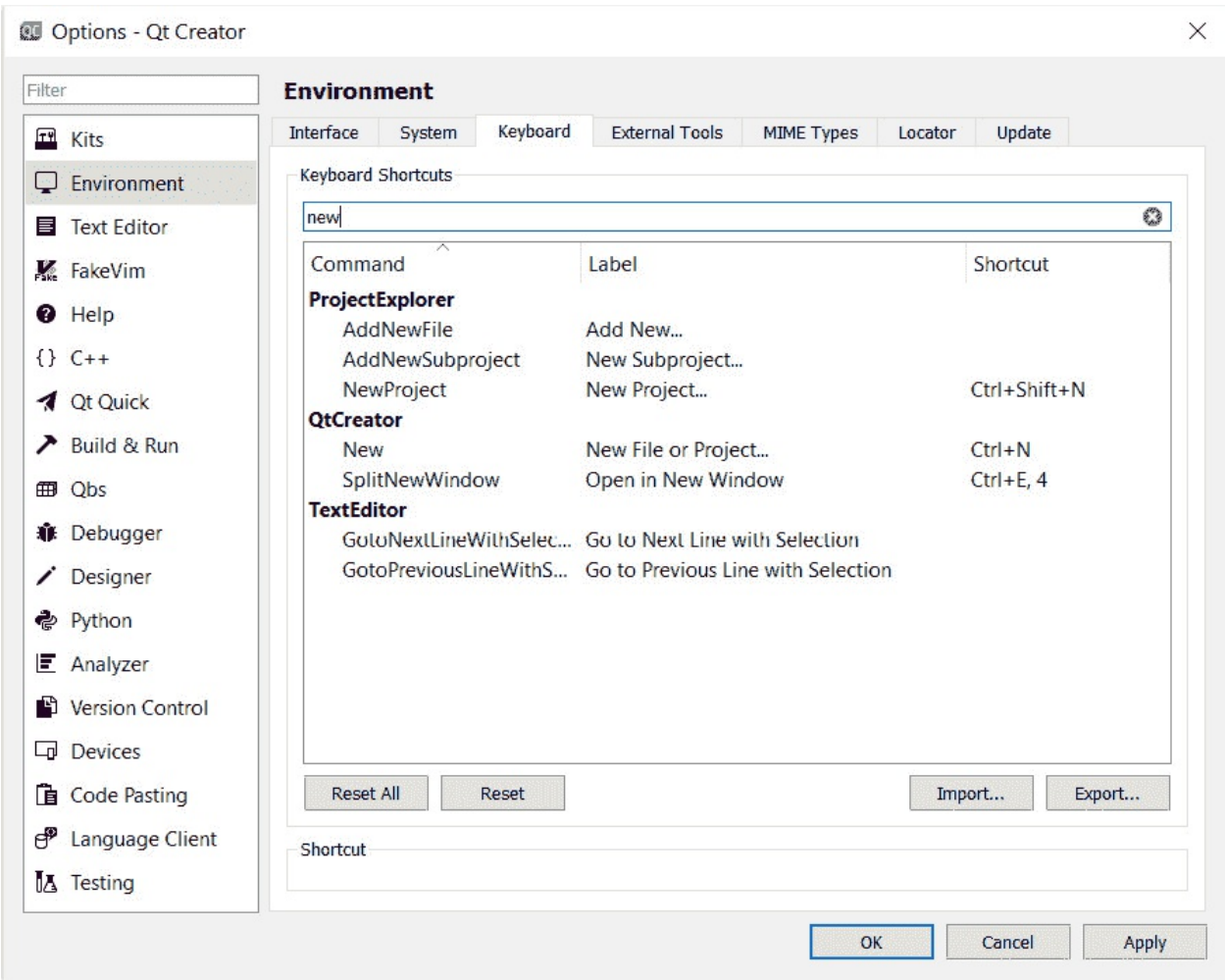


图 2.19-键盘快捷键选项

前面的屏幕截图显示了关键字new的可用快捷键列表。您可以看到**Ctrl+N**用于创建新文件或项目。您还可以导入或导出.kms格式的键盘映射方案文件。

重要音符

内置的 **Qt** 快捷键比我们在这里讨论的要多得多。您可以在以下文章中阅读有关快捷键的更多信息：

<https://doc.qt.io/qtcreator/creator-keyboard-shortcuts.html>

https://wiki.qt.io/Qt_Creator_Keyboard_Shortcuts

https://shortcutworld.com/Qt-Creator/win/Qt-Creator_Shortcuts

Qt Creator 键盘快捷键和窗口管理器快捷键之间存在冲突的可能性。在这种情况下，窗口管理器快捷键将覆盖 **Qt Creator** 快捷键。您还可以在窗口管理器中配置键盘快捷键。如果这是受限的，那么您可以改为更改 **Qt Creator** 快捷键。现在，让我们进入下一个侧栏类别。

文本编辑器

左边栏中的下一个类别是文本编辑器。在这里，您可以在第一个选项卡中选择配色方案、字体和字体大小。下一个选项卡列出了文本编辑器中的不同行为。正如您在图 2.20 中看到的，**Qt** 对键盘上的 **Tab** 键使用空格缩进：

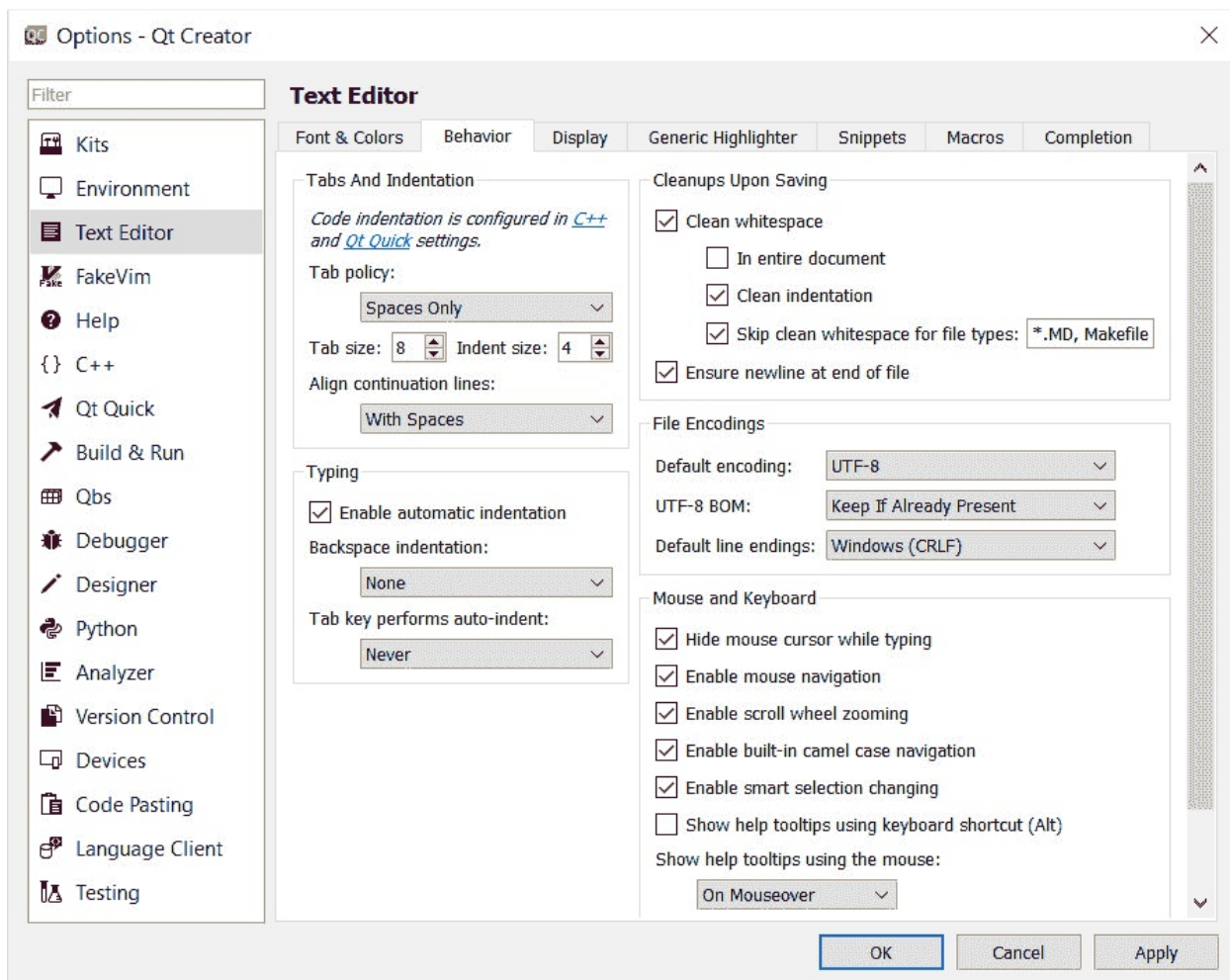


图 2.20-文本编辑器行为选项卡

一些开发人员更喜欢制表符缩进而不是空格缩进。可以在**C++**和**Qt Quick**设置中更改缩进行为。由于有作为不同侧栏类别的专用设置，因此文本编辑器中的此部分可能会在将来的版本中弃用。

您可以在文件编码组中找到当前文件的文件编码。要修改文件编码，请从下拉列表中选择新建编码。要查看使用新编码的文件，请单击应用按钮。

我们不会讨论所有的侧栏类别，因为这些都是非常高级的选项。一旦您学习了基础知识，您可以稍后探索它们。在下一节中，我们将讨论

编码窗口的管理。

分割编码窗口

您可以拆分编码窗口，并在同一屏幕或外部屏幕上查看多个文件。您可以选择不同的方式同时查看多个文件(这些选项位于菜单栏中的窗口选项下)：

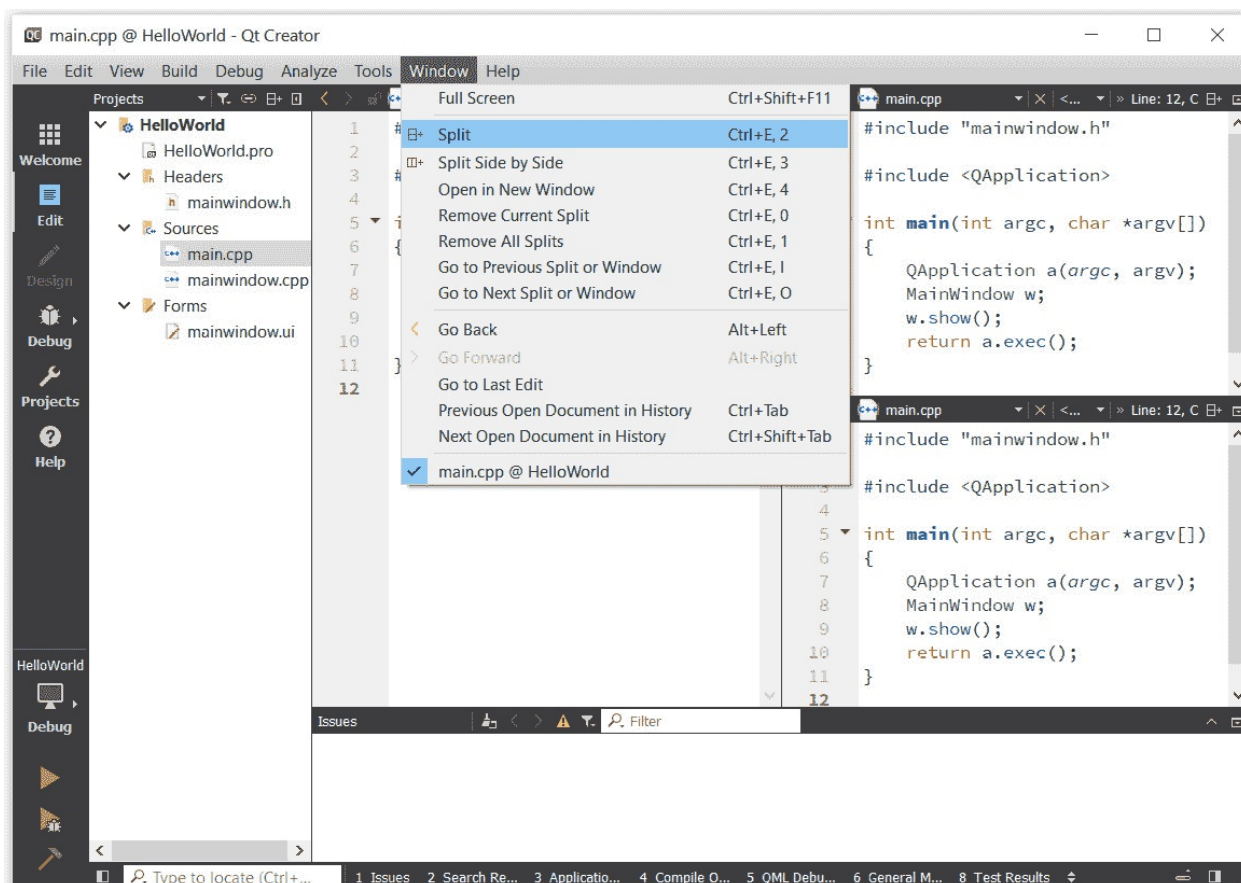


图 2.21-显示分屏选项的屏幕截图

现在，让我们讨论拆分编码窗口和删除拆分窗口的各种方法：

- 要将编码窗口拆分为顶视图和底视图，请按**Ctrl+E**，然后按**2**、或选择菜单栏中的窗口选项，然后单击拆分选项。这将在当前活动窗口下方创建一个额外的编码窗口。

- 要将编码窗口拆分为相邻视图，请选择并排拆分或按**Ctrl+E**，然后按**3**。并排拆分在当前活动编码窗口的右侧创建视图。
- 要在分离窗口中打开编码窗口，请按**Ctrl+E**和**4**，或选择在新窗口中打开。为方便起见，您可以将窗口拖到外部监视器上。
- 要在拆分视图和分离的编辑器窗口之间移动，请选择下一个拆分或按**Ctrl+E**，然后按**O**。
- 要删除拆分视图，请单击要删除的窗口并选择删除当前拆分，或按**Ctrl+E**，然后按**0**。
- 要删除所有拆分编码窗口，请选择删除所有拆分或按**Ctrl+E**，然后按**1**。

在本节中，您了解了如何拆分编码编辑器窗口。这对在编码时一次引用多个代码文件很有帮助。在下一节中，我们将讨论 **IDE** 菜单栏中的 **Build** 菜单。

生成选项

在菜单栏中，您可以看到**Build**选项。如果单击该选项，则会看到各种构建选项，如图 2.22所示。在这里，您可以生成、重建或清理项目。在复杂的项目中，您可能有多个子项目。您可以单独生成子项目，以减少总生成时间：

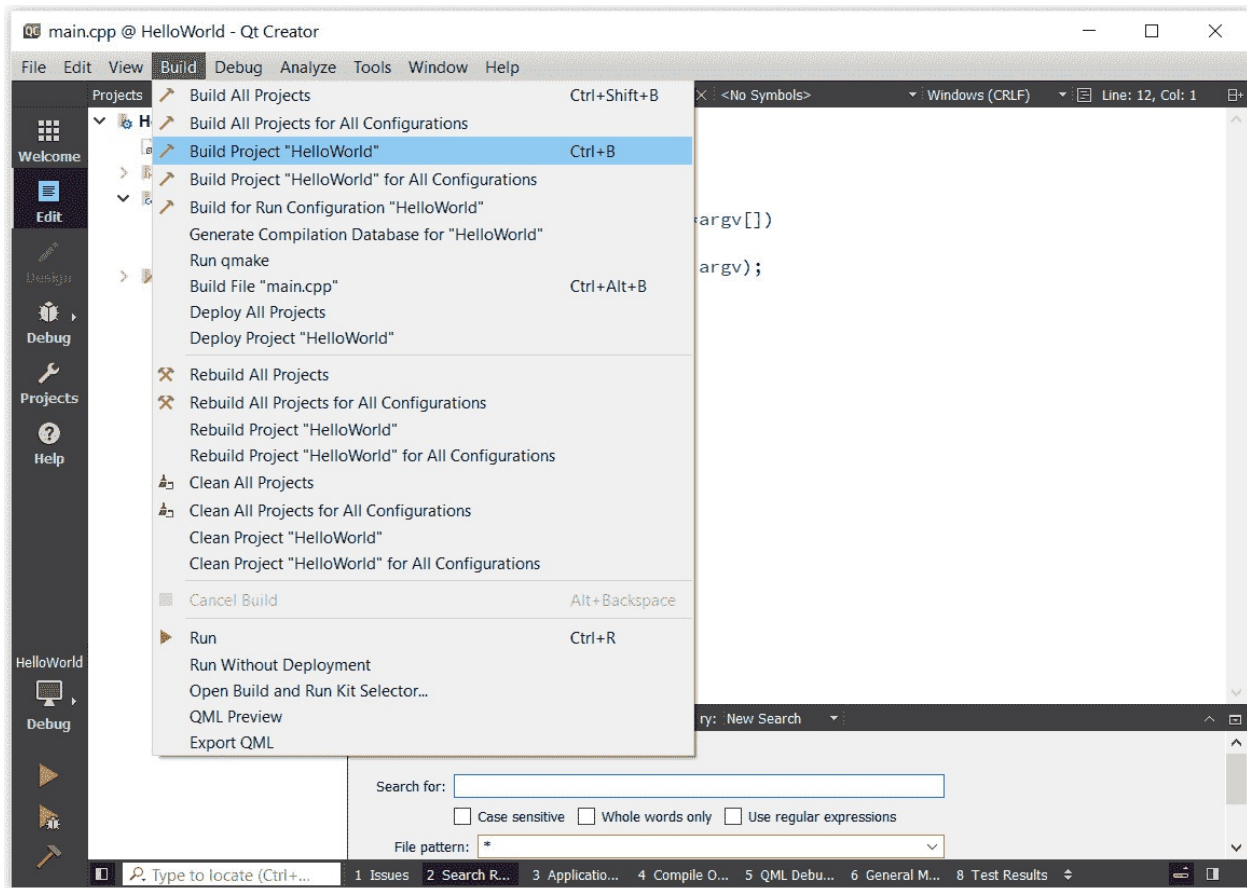


图 2.22-构建菜单选项

Qt Creator 项目向导允许您在创建新项目时选择构建系统，包括 **qmake**、**CMake** 和 **QBS**。它使开发人员可以自由地使用 Qt Creator 作为代码编辑器，并控制构建项目时使用的步骤或命令。默认情况下，会为您的新项目安装和配置 **qmake**。您可以通过以下链接了解有关使用其他构建系统的更多信息：<https://doc.qt.io/qtcreator/creator-project-other.html>。

现在，让我们讨论在哪里以及如何查找框架的文档。

Qt 助手

Qt Creator 还包括名为 **Qt Assistant** 的内置文档查看器。这非常方便，

因为您只需将鼠标光标悬停在源代码中的类名上，然后按下**F1**键，就可以查找某个 Qt 类或函数的解释。然后将打开 Qt 助手，并向您显示与该 Qt 类或函数相关的文档：

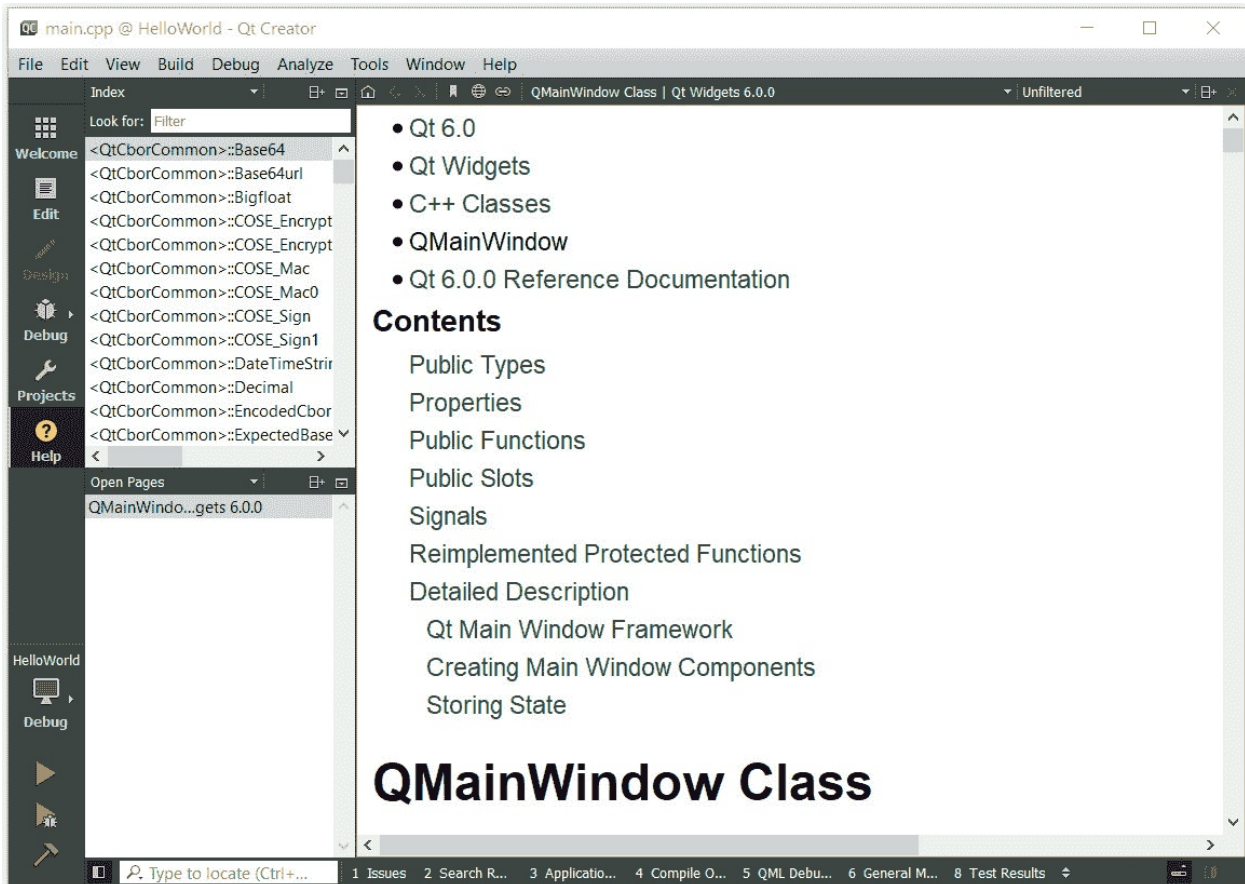


图 2.23-集成帮助界面

Qt 助手还支持交互式帮助，并允许您为 Qt 应用创建帮助文档。

笔记 / 便条 / 票据 / 注解

在 Windows 平台上，Qt 助手是 Qt Creator 菜单栏上的一个菜单选项。在 Linux 发行版上，您可以打开终端，键入 `assistant`，然后按 **Enter**。在 MacOS 上，它安装在 `/Developer/Applications/Qt` 目录中。

在本节中，我们了解了 Qt 助手和帮助文档。现在，让我们总结一下这

一章的要点。

摘要

本章解释了 **Qt Creator IDE** 的基本原理以及它的用途。 **Qt Creator** 是一个 **IDE**，有一套很棒的工具。 它可以帮助您轻松地多个平台创建出色的 **GUI** 应用。 开发人员不需要仅仅为了创建一个简单的按钮而编写冗长的代码，也不需要仅仅为了对齐文本标签而更改大量代码--在我们设计 **GUI** 时， **Qt Designer** 会自动为我们生成代码。 我们只需单击几下即可创建一个 **GUI** 应用，我们还了解了 **IDE** 中提供的各种高级选项，包括如何管理工具包和快捷方式。 内置的 **Qt** 助手对有用的示例提供了很大的帮助，并且可以帮助我们编写自己的文档。

在下一章中，我们将讨论使用 **Qt** 小部件的 **GUI** 设计。 在这里，您将了解不同的小部件，如何创建您自己的 **GUI** 元素，以及如何创建自定义 **GUI** 应用。

三、将 Qt 小部件用于 GUI 设计

原文: <https://gitee.com/apacheecn/apacheecn-c-cpp-zh-pt2/blob/master/docs/cross-plat-dev-qt6-mod-cpp/03.md>

Qt 小部件是一个提供一组用户界面(UI)元素的模块，用于构建经典 UI。在本章中，您将了解Qt 小部件模块，并了解基本小部件。我们将了解什么是小部件，以及可用于创建图形用户界面(GUI)的各种小部件。除此之外，您还将了解Qt Designer的布局，您还将学习如何创建自己的自定义控件。我们将密切关注 Qt 在轻松设计外观流畅的 GUI 方面能为我们提供什么。在本章的开头，我们将向您介绍 Qt 提供的小部件类型及其功能。之后，我们将完成一系列步骤，并使用 Qt 设计我们的第一个表单应用。然后，您将了解样式表、Qt 样式表(QSS 文件)和主题化。

本章将介绍以下主要主题：

- Qt 小工具简介
- 使用 Qt Designer 创建 UI
- 管理布局
- 创建自定义小部件
- 创建 Qt 样式表和自定义主题
- 浏览自定义样式
- 使用小部件、窗口和对话框

在本章结束时，您将了解 GUI 元素及其对应的 C++ 类的基础知识，如何在不编写任何代码的情况下创建自己的 UI，以及如何使用样式表自定义 UI 的外观。

技术要求

本章的技术要求包括 Qt 6.0.0 MinGW 64 位、Qt Creator 4.14.0 和 Windows 10/Ubuntu 20.04/MacOS 10.14。本章使用的所有代码都可以从以下 [giHub](https://github.com/PacktPublishing/Cross-Platform-Development-with-Qt-6-and-Modern-Cpp/tree/master/Chapter03) 链接下载：<https://github.com/PacktPublishing/Cross-Platform-Development-with-Qt-6-and-Modern-Cpp/tree/master/Chapter03>。

笔记 / 便条 / 票据 / 注解

本章中使用的屏幕截图来自 Windows 环境。根据计算机中的底层平台，您将看到类似的屏幕。

Qt 小工具简介

小部件是 **GUI** 的基本元素。它也称为**UI** 控件。它接受来自底层平台的不同用户事件，如鼠标和键盘事件(以及其他事件)。我们使用不同的小部件创建 **UI**。曾经有一段时间，所有的 **GUI** 控件都是从头开始编写的。Qt 小部件通过开发具有现成的 **GUI** 控件的桌面 **GUI** 来缩短时间，并且 Qt 广泛使用继承的概念。所有小工具都继承自 `QObject`。

`QWidget` 是一个基本小部件，是所有 **UI** 小部件的基类。它包含描述小部件所需的大多数属性，以及几何图形、颜色、鼠标、键盘行为、工具提示等属性。让我们看看下图中的 `QWidget` 继承层次结构：

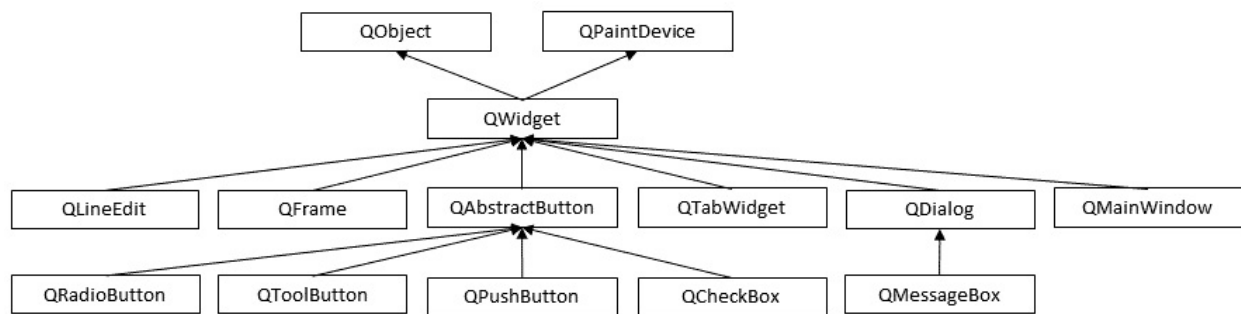


图 3.1-QWidget 类层次结构

大多数 Qt 小部件名称都是不言而喻的，因为它们以`q`开头，所以很容易识别。下面列出了其中的一些内容：

- `QPushButton`是，用于命令应用执行某个操作。
- `QCheckBox`允许用户做出二元选择。
- `QRadioButton`允许用户从一组互斥选项中仅选择一个。
- `QFrame`显示帧。
- `QLabel`用于显示文本或图像。
- `QLineEdit`允许用户输入和编辑单行纯文本。
- `QTabWidget`用于显示与选项卡式小部件堆栈中的每个选项卡相关的页面。

使用 Qt 小部件的优势之一是它的育儿系统。从`QObject`继承的任何对象都具有父子关系。这种关系给开发人员带来了很多便利，例如：

- 当小部件被销毁时，由于父子层次结构，它的所有子部件也会被销毁。这避免了内存泄漏。
- 您可以使用`findChild()`和`findChildren()`查找给定`QWidget`类的子类。
- `QWidget`中的子构件自动出现在父构件内部。

典型的 C++ 程序在 `Main` 返回时终止，但在 GUI 应用中我们不能这样做，否则应用将无法使用。因此，我们需要 GUI 一直存在，直到用户关闭窗口。要实现这一点，程序应该循环运行，直到发生这种情况。GUI 应用等待用户输入事件。

让我们使用`QLabel`通过一个简单的 GUI 程序显示文本，如下所示：

```
#include <QApplication>
#include <QLabel>
int main(int argc, char *argv[])
{
    QApplication app(argc, argv);
    QLabel myLabel;
    myLabel.setText("Hello World!");
    myLabel.show();
    return app.exec();
}
```

请记住将下面的行添加到 `helloworld.pro` 文件中，以启用 Qt 小部件模块：

```
QT += widgets
```

对 `.pro` 文件进行更改后，需要运行 `qmake`。如果您正在使用命令行，则继续执行以下命令：

```
>qmake
>make
```

现在，点击 **Run** 按钮构建并运行应用。您很快就会看到显示了 **Hello World!** 的 UI，如以下屏幕截图所示：

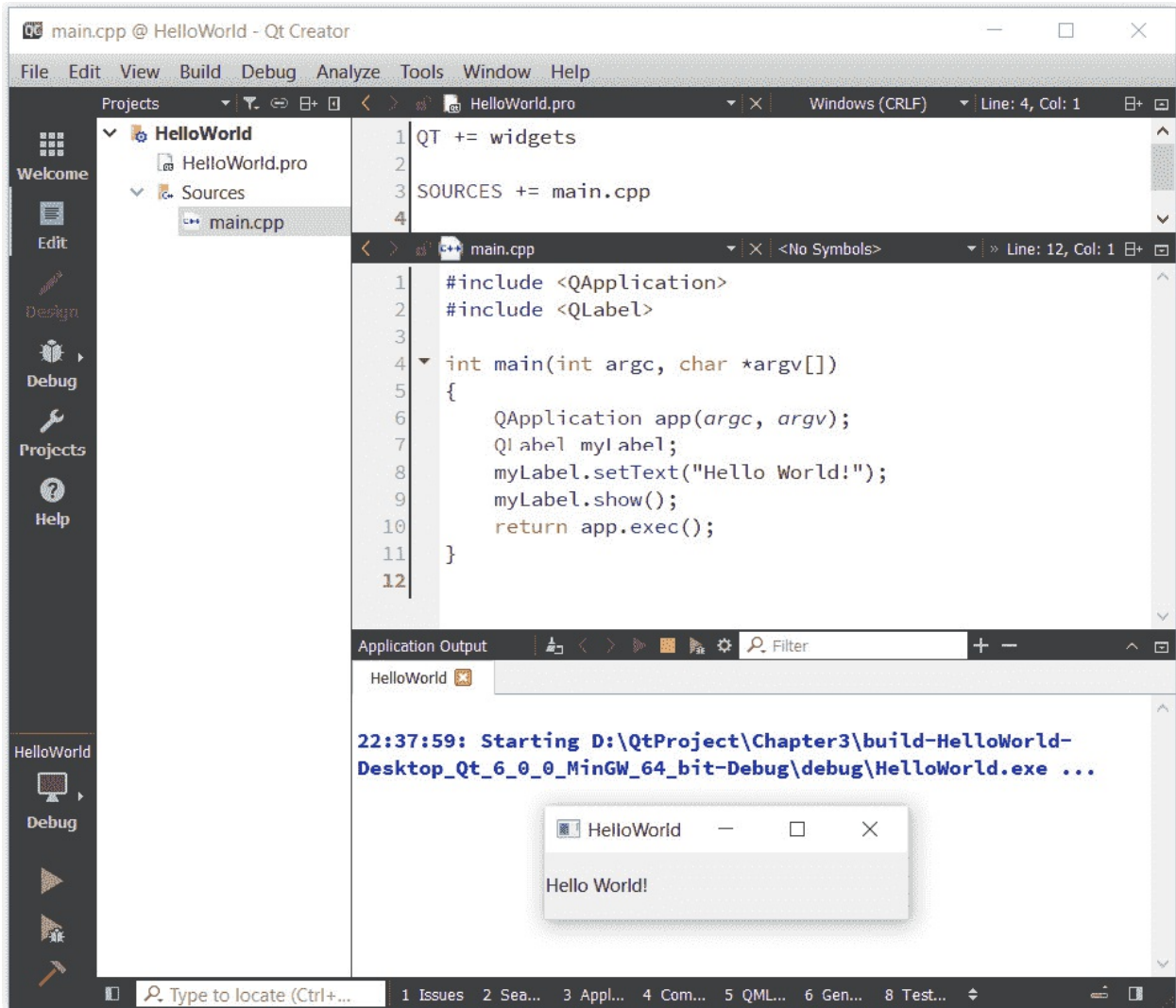


图 3.2-简单 GUI 应用

您还可以在 Windows 上从命令行运行应用，如下所示：

```
>helloworld.exe
```

您可以在 Linux 发行版上从命令行运行该应用，如下所示：

```
$/helloworld
```

在命令行模式下，如果在应用路径中找不到库，您可能会看到几个错误对话框。您可以将 Qt 库和插件文件复制到该二进制文件夹来解决该问

题。为了避免这些问题，我们将坚持使用 Qt Creator 来构建和运行我们的示例程序。

在本节中，我们学习了如何使用 Qt 小部件模块创建一个简单的 GUI。在下一节中，我们将探索可用的小部件，并使用 Qt Designer 创建 UI。

使用 Qt Designer 创建 UI

在开始学习如何设计我们自己的 UI 之前，让我们先熟悉一下 Qt Designer 的界面。下面的屏幕截图显示了 Qt Designer 的不同部分。在设计我们的 UI 时，我们将逐步了解这些部分：

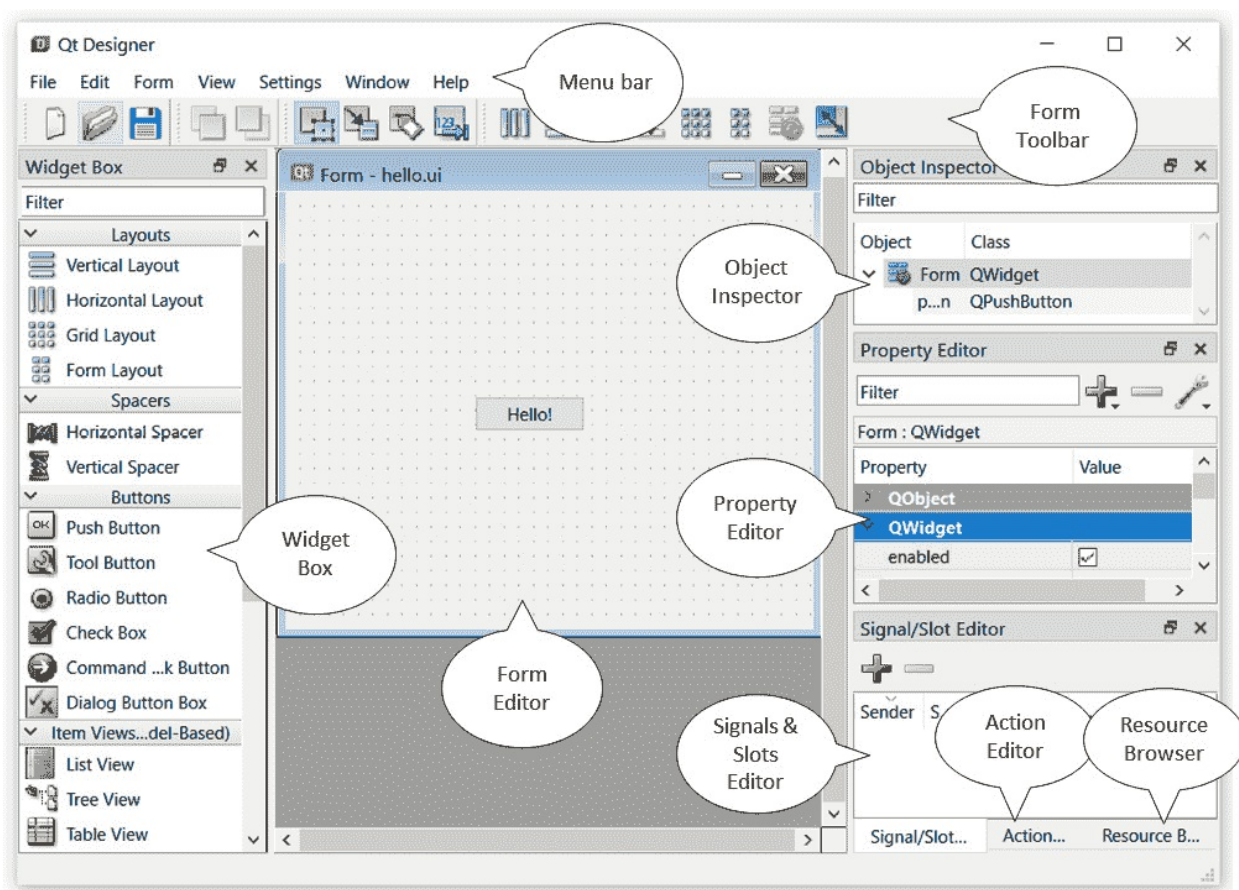


图 3.3-Qt 设计器 UI

Qt 小部件模块附带了现成的小部件。所有这些小部件都可以在小部件

框部分下找到。Qt 提供了通过拖放方法创建 UI 的选项。我们只需将这些小部件从**Widget Box**区域拖放到**Form Editor**区域，即可浏览这些小部件。您可以通过抓取项目，然后在所需区域上按下并释放鼠标或触控板来完成此操作。在项目到达窗体编辑器区域之前，不要释放鼠标或触控板。

下面的屏幕截图显示了**Widget Box**部分提供的不同类型的小部件。我们在表单编辑器区域添加了几个现成的小部件，如**Label**、**Push Button**、**Radio Button**、**Check Box**、**Combo Box**、**Progress Bar**和**Line Edit**。这些小部件是非常常用的小部件。您可以在属性编辑器中浏览小组件特定的属性：

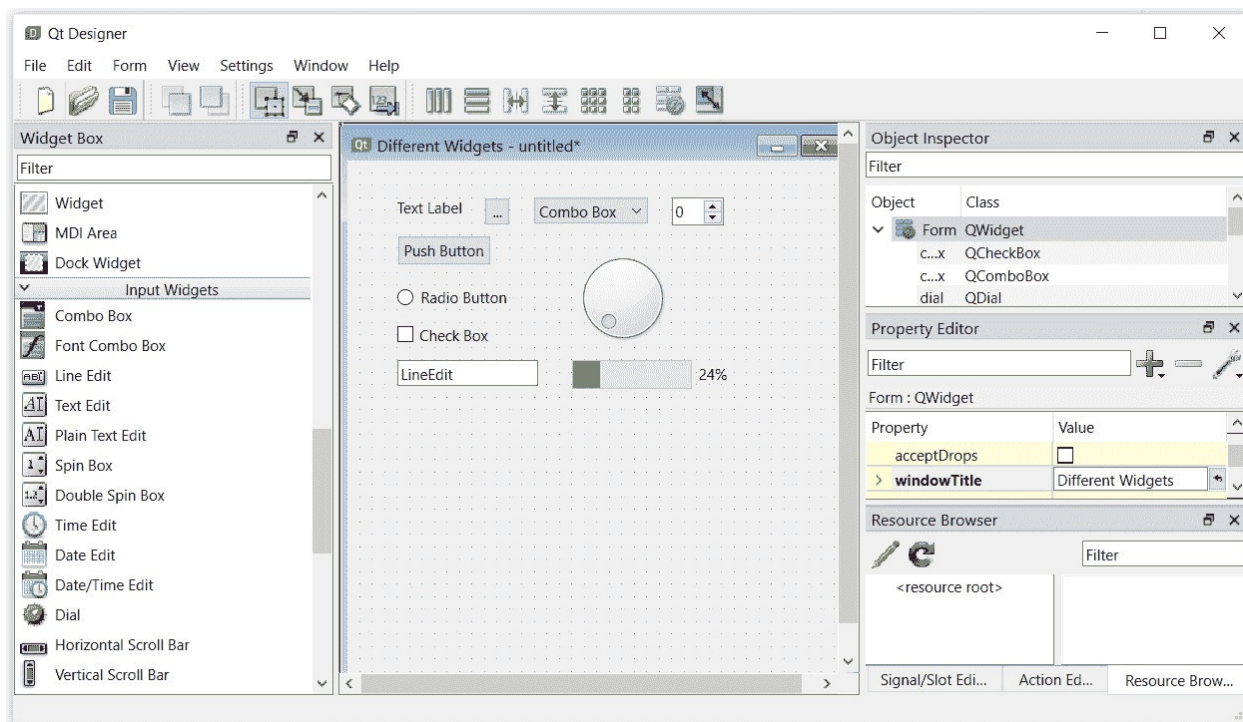


图 3.4-不同类型的 GUI 小部件

您可以通过选择预览...来预览您的 UI。窗体菜单下的选项，如下面的截图所示，也可以点击**Ctrl+R**。您将看到一个带有 UI 预览的窗口：

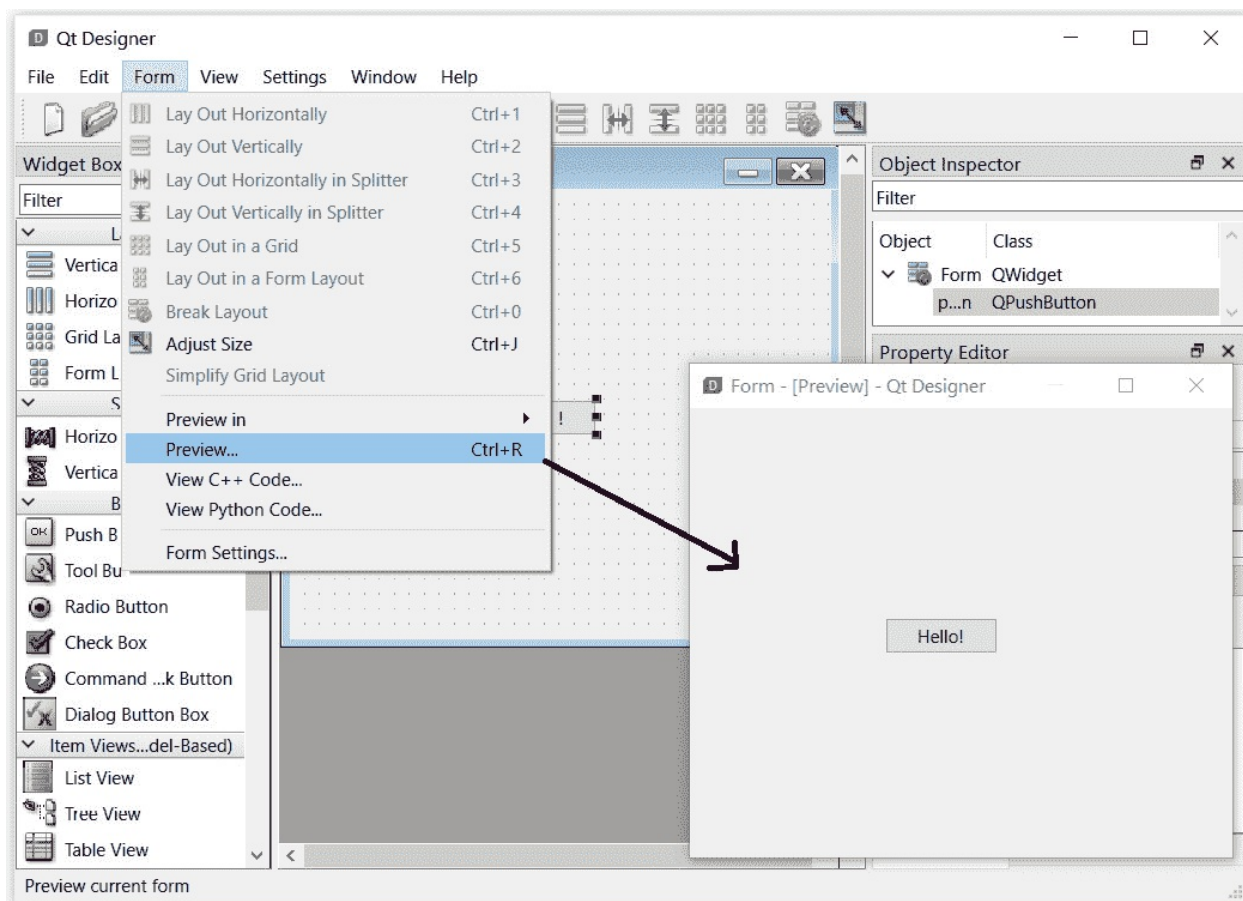


图 3.5-预览您的自定义用户界面

通过选择查看 **C++** 代码...，您可以找到为您的 **UI** 创建的 **C++** 代码。窗体菜单下的选项，如下面的屏幕截图所示。您将看到一个窗口，其中包含生成的代码。您可以在创建动态 **UI** 时重用代码：

现 `setGeometry()`、`sizeHint()`、`addItem()`、`itemAt()`、`takeAt()` 和 `minimumSize()` 函数来创建自己的布局管理器。请注意，一旦删除布局管理器，布局管理也将停止。

下表提供了主要布局类的简要说明：

- `QVBoxLayout` 垂直排列小工具。
- `HBoxLayout` 水平排列小工具。
- `GridLayout` 在网格中布局小部件。
- `FormLayout` 管理输入小工具的表单及其关联标签。
- `StackedLayout` 提供一堆小部件，其中一次只有一个小部件可见。

`QLayout` 通过继承 `QObject` 和 `QLayoutItem` 使用多个继承。`QLayout` 的子类是 `QBoxLayout`、`GridLayout`、`FormLayout` 和 `StackedLayout`。

`QVBoxLayout` 和 `HBoxLayout` 继承自添加了方位信息的 `QBoxLayout`。

让我们使用 Qt 设计器模块来布局几个 `QPushButton`。

QVBoxLayout

在 `QVBoxLayout` 类中，控件垂直排列，并且它们在布局中从上到下对齐。此时，您可以执行以下操作：

1. 将四个按钮拖到表单编辑器上。
2. 重命名按钮并按键盘上的 **Ctrl** 键选择按钮。
3. In the **Form** toolbar, click on the vertical layout button. You can find this by hovering on the toolbar button that says **Lay Out Vertically**.

您可以在下面的屏幕截图中看到按钮是以自上而下的方式垂直排列

的：

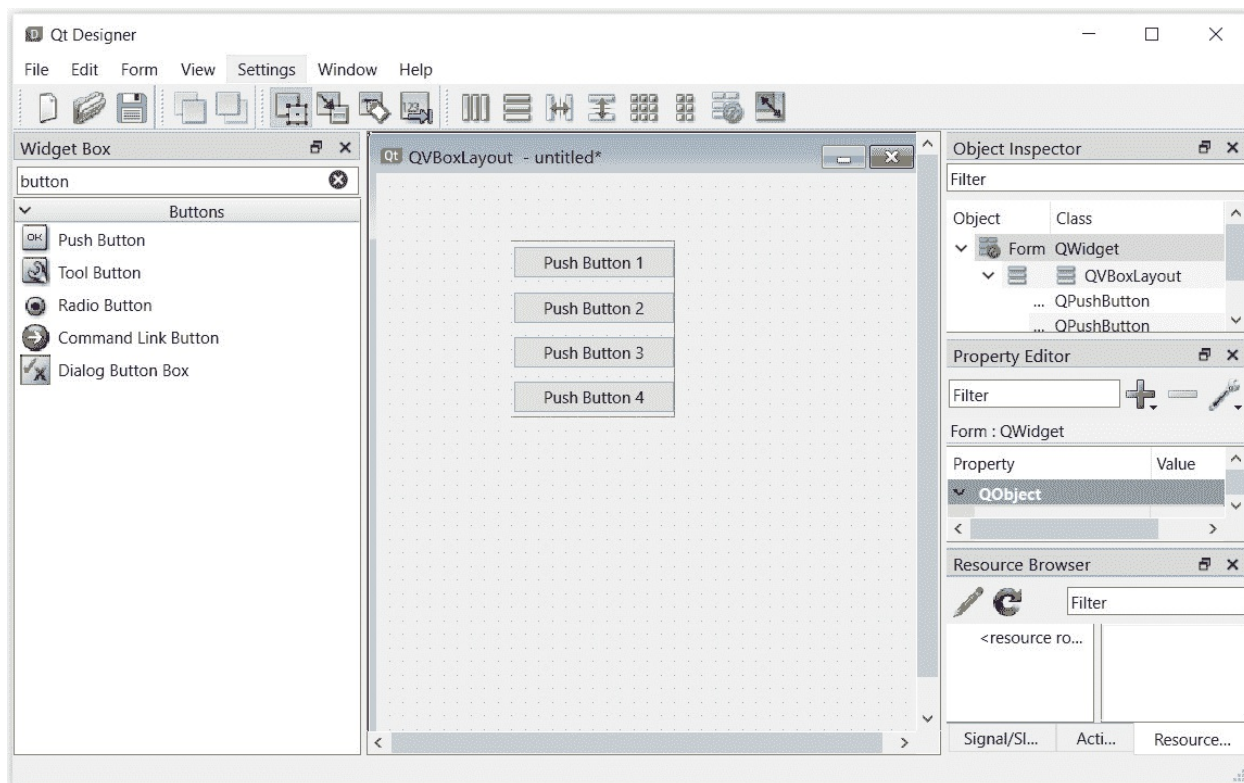


图 3.7-使用 QVBoxLayout 进行布局管理

您还可以通过 C++ 代码动态添加垂直布局，如以下代码片段所示：

```
QWidget *widget = new QWidget;
QPushButton *pushBtn1 = new QPushButton("Push Button
                                     1");
QPushButton *pushBtn2 = new QPushButton("Push Button
                                     2");
QPushButton *pushBtn3 = new QPushButton("Push Button
                                     3");
QPushButton *pushBtn4 = new QPushButton("Push Button
                                     4");
QVBoxLayout *verticalLayout = new QVBoxLayout(widget);
verticalLayout->addWidget(pushBtn1);
verticalLayout->addWidget(pushBtn2);
verticalLayout->addWidget(pushBtn3);
verticalLayout->addWidget(pushBtn4);
widget->show ();
```

本程序演示如何使用垂直布局对象。请注意，`QWidget`实例`widget`将成为应用的主窗口。这里，布局被直接设置为顶级布局。添加到`addWidget()`方法的第一个按钮占据布局的顶部，而最后一个按钮占据布局的底部。`addWidget()`方法将一个小部件添加到布局的末尾，并带有拉伸因子和对齐方式。

如果没有在构造函数中设置父窗口，则必须稍后使用`QWidget::setLayout()`安装布局并重新设置`widget`实例的父级。

接下来，我们来看一下`QHBoxLayout`类。

QHBoxLayout

在`QHBoxLayout`类中，小部件水平排列，并且从左到右对齐。

我们现在可以执行以下操作：

1. 将四个按钮拖到表单编辑器上。
2. 重命名按钮并按键盘上的`Ctrl`键选择按钮。
3. In the **Form** toolbar, click on the horizontal layout button. You can find this by hovering on the toolbar button that says **Lay Out Horizontally**.

在此截图中，您可以看到按钮以从左到右的方式水平排列：

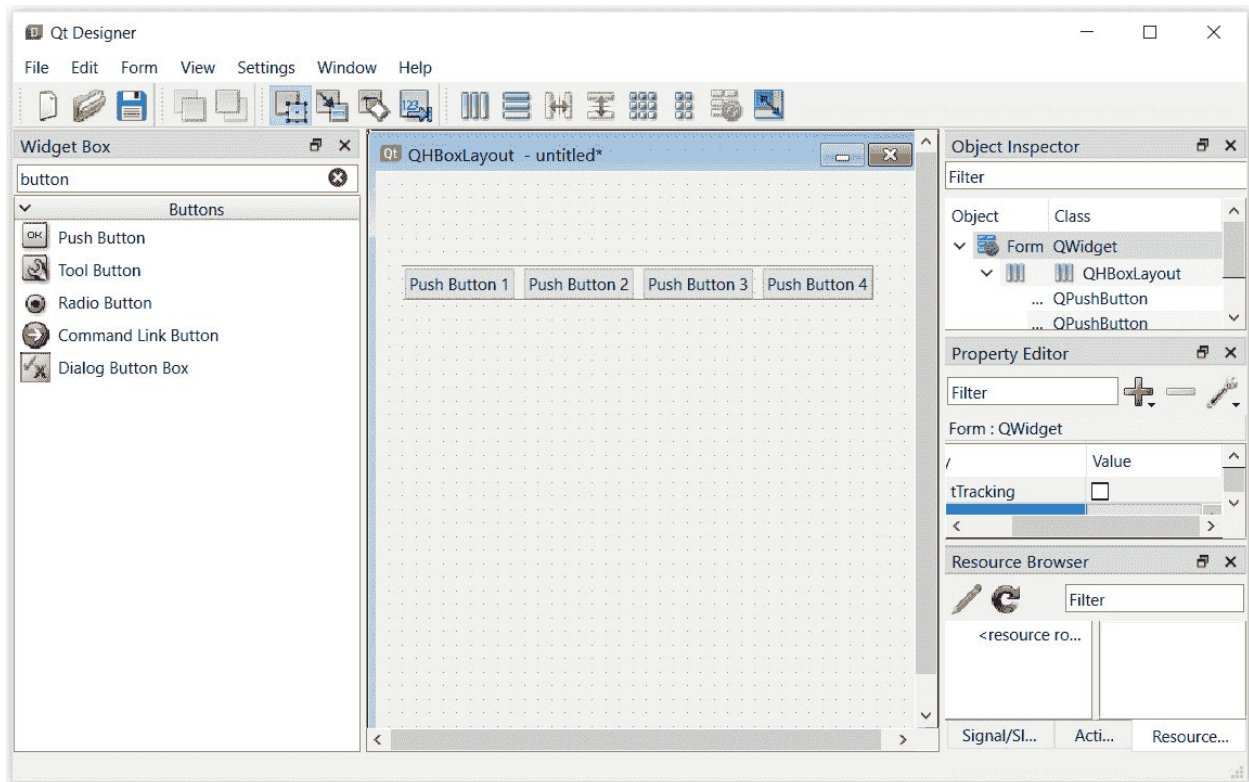


图 3.8-使用 QHBoxLayout 进行布局管理

您还可以通过 C++ 代码动态添加水平布局，如以下代码片段所示：

```
QWidget *widget = new QWidget;
QPushButton *pushBtn1 = new QPushButton("Push
                                   Button 1");
QPushButton *pushBtn2 = new QPushButton("Push
                                   Button 2");
QPushButton *pushBtn3 = new QPushButton("Push
                                   Button 3");
QPushButton *pushBtn4 = new QPushButton("Push
                                   Button 4");
QHBoxLayout *horizontalLayout = new QHBoxLayout(
                                   widget);

horizontalLayout->addWidget(pushBtn1);
horizontalLayout->addWidget(pushBtn2);
horizontalLayout->addWidget(pushBtn3);
horizontalLayout->addWidget(pushBtn4);
widget->show ();
```


前面的示例演示了如何使用水平布局对象。与垂直布局示例类似，`QWidget`实例将成为应用的主窗口。在这种情况下，布局被直接设置为顶级布局。默认情况下，添加到`addWidget()`方法的第一个按钮占据布局的最左侧，而最后一个按钮占据布局的最右侧。当小部件添加到布局中时，您可以使用`setDirection()`方法更改增长方向。

在下一节中，我们将查看`QGridLayout`类。

Колибриобработается

在`QGridLayout`类中，小部件通过指定行数和列数排列在网格中。它类似于具有行和列的网格状结构，小部件作为项插入。

在这里，我们应该做到以下几点：

1. 将四个按钮拖到表单编辑器上。
2. 重命名按钮并按键盘上的`Ctrl`键选择按钮。
3. In the **Form** toolbar, click on the grid layout button. You can find this by hovering on the toolbar button that says **Lay Out in a Grid**.

您可以在下面的屏幕截图中看到按钮以网格形式排列：

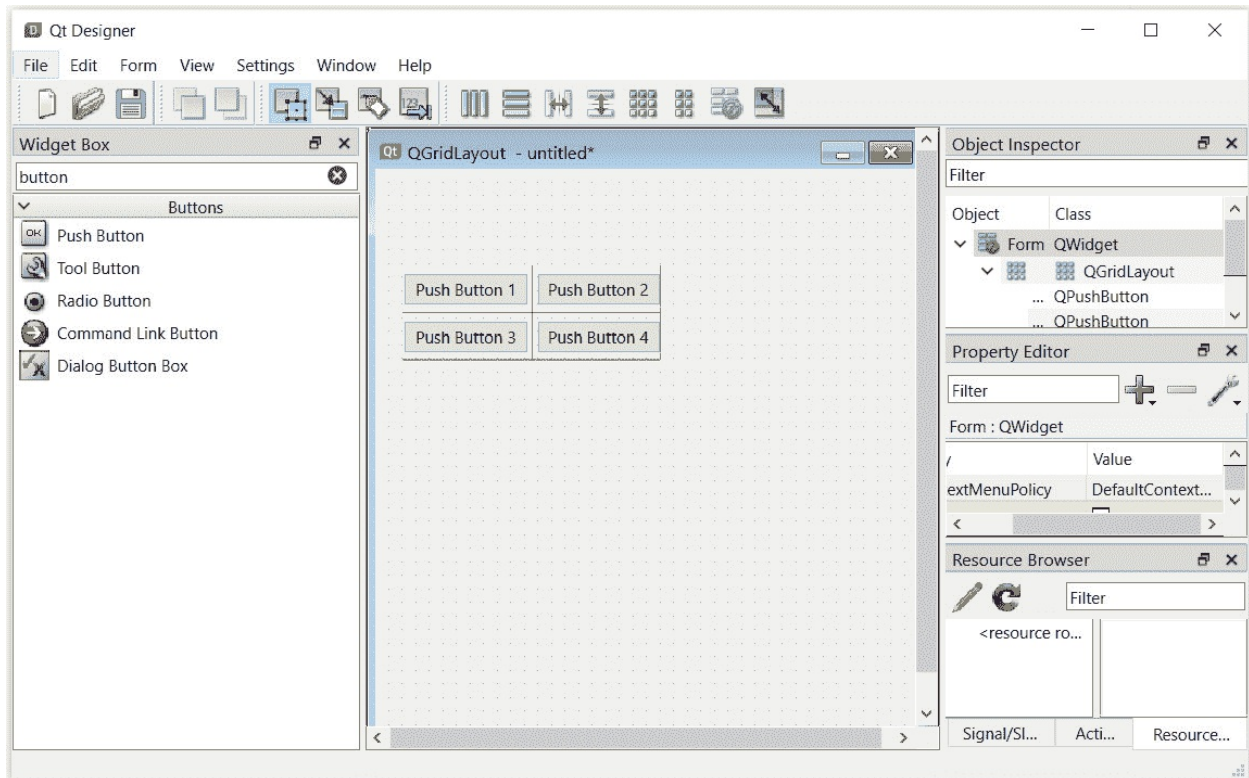


图 3.9-使用 QGridLayout 进行布局管理

您还可以通过 C++ 代码动态添加网格布局，如以下代码片段所示：

```
QWidget *widget = new QWidget;
QPushButton *pushBtn1 = new QPushButton(
    "Push Button 1");
QPushButton *pushBtn2 = new QPushButton(
    "Push Button 2");
QPushButton *pushBtn3 = new QPushButton(
    "Push Button 3");
QPushButton *pushBtn4 = new QPushButton(
    "Push Button 4");
QGridLayout *gridLayout = new QGridLayout(widget);
gridLayout->addWidget(pushBtn1);
gridLayout->addWidget(pushBtn2);
gridLayout->addWidget(pushBtn3);
gridLayout->addWidget(pushBtn4);
widget->show();
```

前面的代码片段解释了如何使用网格布局对象。 布局概念与前几节相

同。您可以从 Qt 文档浏览 `QFormLayout` 和 `QStackedLayout` 布局。让我们继续下一节，介绍如何创建自定义小部件并将其导出到 Qt Designer 模块。

创建自定义小部件

Qt 提供了个随时可用的基本个 **GUI** 元素。在 Qt 建模语言(**QML**)出现之后，Qt 小部件并没有积极地开发，因此您可能需要更具体的小部件，并且可能希望将其提供给其他人使用。自定义小部件可以是放置在一起的一个或多个 Qt 小部件的组合，也可以从头开始编写。我们将从 `QLabel` 创建一个简单的标签小部件，作为我们的第一个自定义小部件。一个自定义小部件集合可以有多个自定义小部件。

按照以下步骤构建您的第一个 Qt 自定义小部件库：

1. To create a new Qt custom widget project in Qt, click on the **File menu** option on the menu bar or hit *Ctrl + N*. Alternatively, you can also click on the **New Project** button located on the **Welcome** screen. Select the **Other Project** template and then select **Qt Custom Designer Widget**, as shown in the following screenshot:

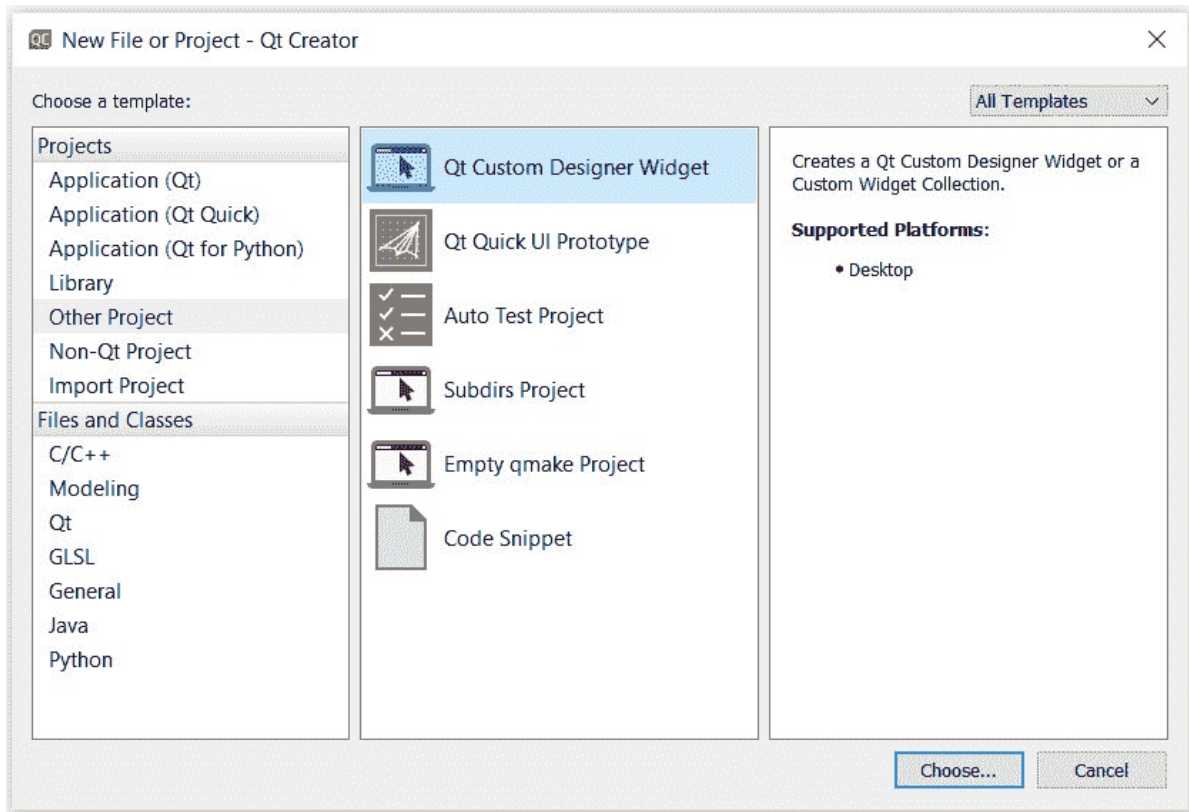


图 3.10-创建自定义小部件库项目

2. In the next step, you will be asked to choose the project name and project location. You can navigate to the desired project location by clicking the **Browse...** button. Let's name our sample project `MyWidgets`. Then, click on the **Next** button to proceed to the next screen. The following screenshot illustrates this step:

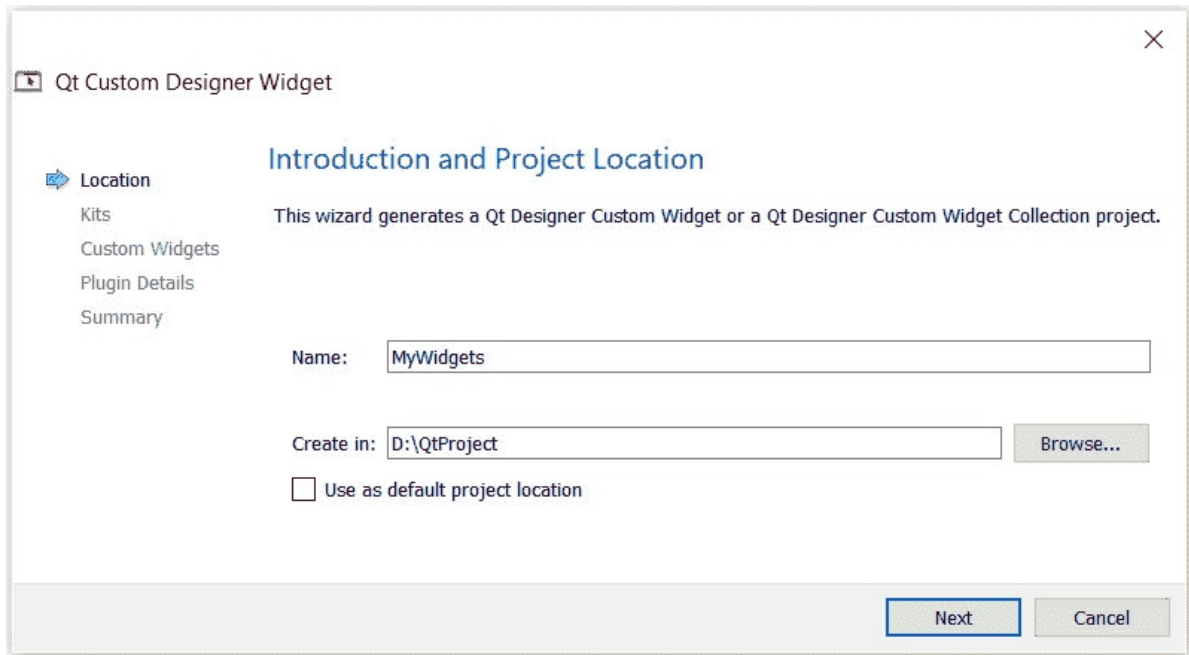


图 3.11-创建客户 m 控件库项目

3. In the next step, you can select a kit from a set of kits to build and run your project. To build and run the project, at least one kit must be active and selectable. Select the default **Desktop Qt 6.0.0 MinGW 64-bit** kit. Click on the **Next** button to proceed to the next screen. The following screenshot illustrates this step:

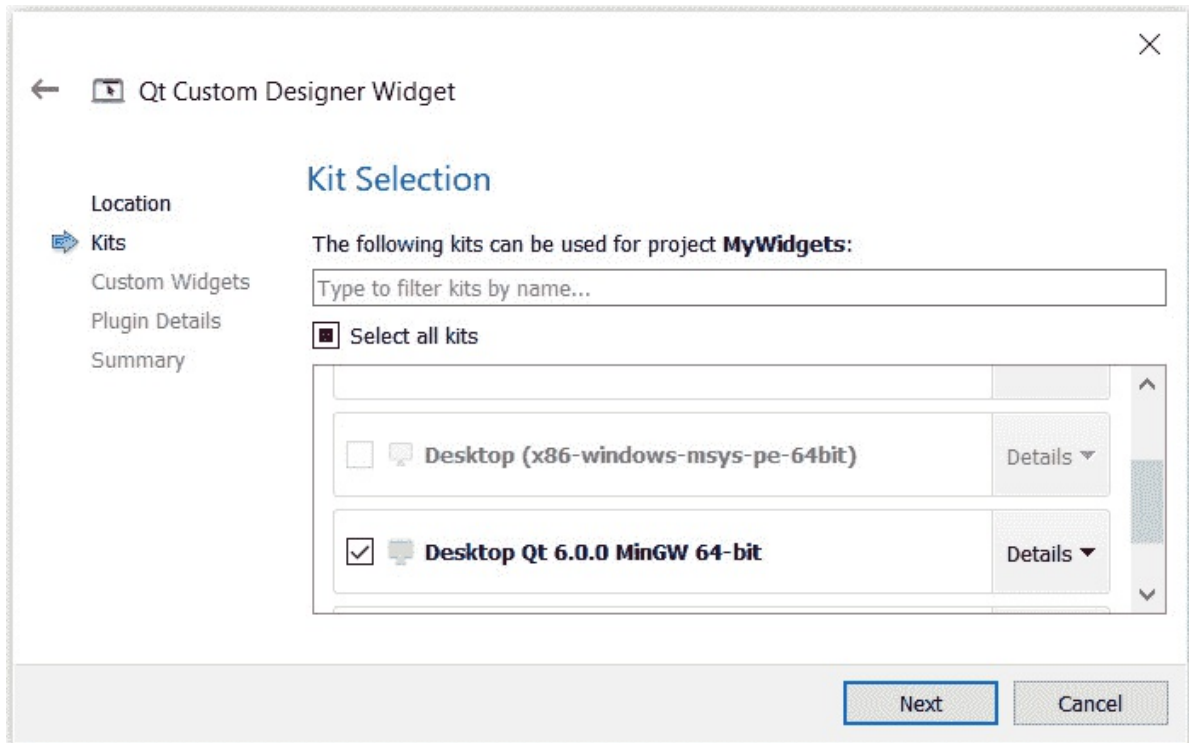


图 3.12-套件选择屏幕

4. In this step, you can define your custom widget class name and inheritance details. Let's create our own custom label with the class name `MyLabel`. Click on the **Next** button to proceed to the next screen. The following screenshot illustrates this step:

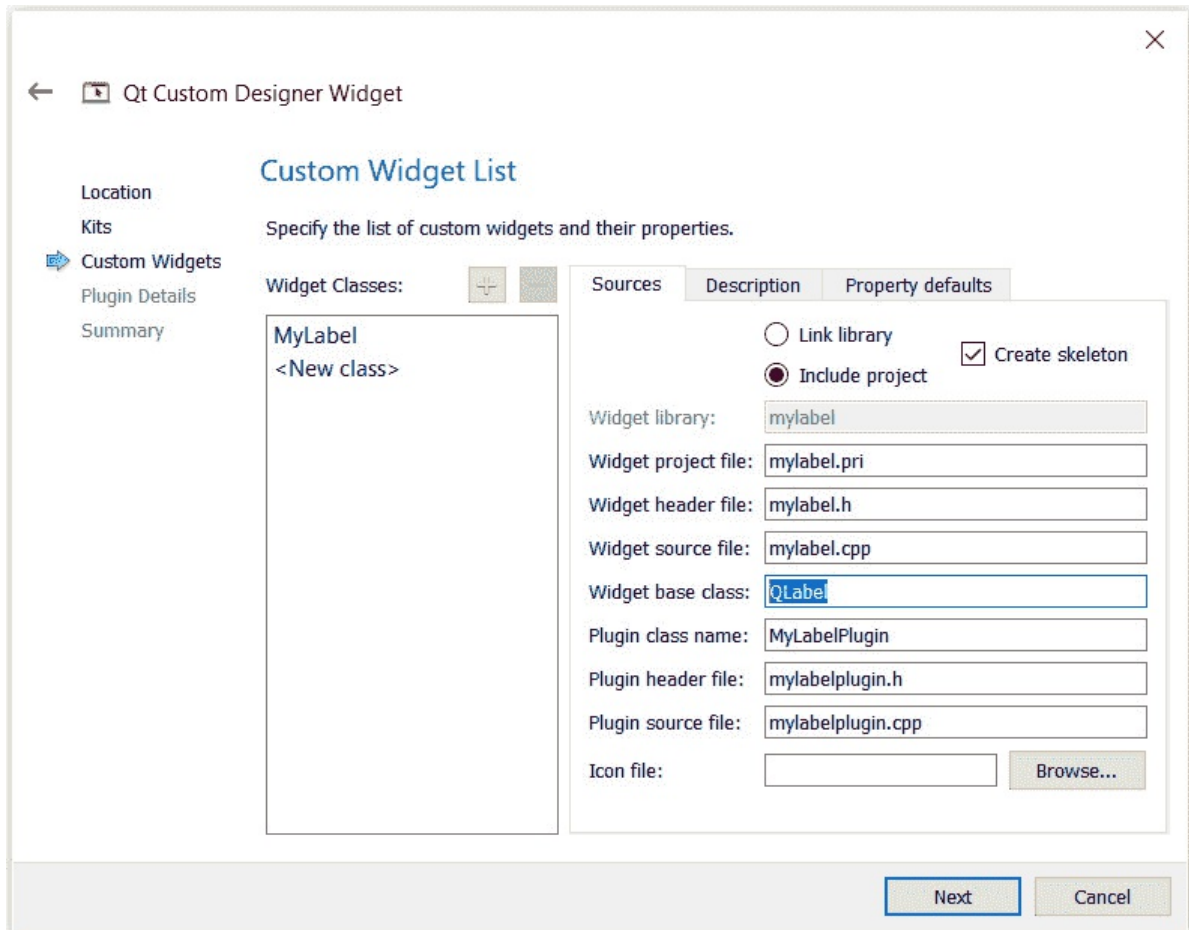


图 3.13-从现有小部件的屏幕创建自定义小部件

5. In the next step, you can add more custom widgets to create a widget collection. Let's create our own custom frame with the class name `MyFrame`. You can add more information to the **Description** tab or can modify it later. Click on the checkbox that says **The widget is a container** to use the frame as a container. Click on the **Next** button to proceed to the next screen. The following screenshot illustrates this step:

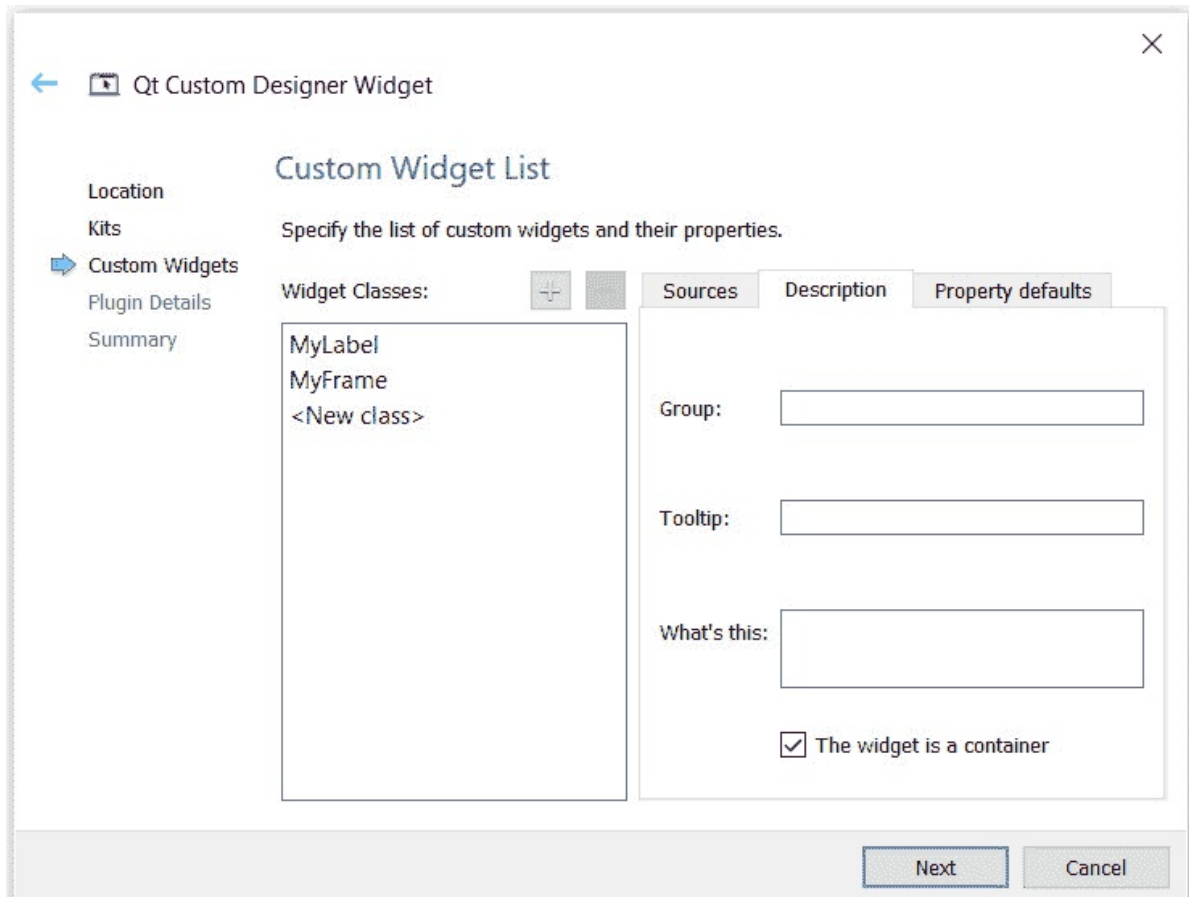


图 3.14-创建自定义小部件容器

6. In this step, you can specify the collection class name and the plugin information to automatically generate the project skeleton. Let's name the collection class `MyWidgetCollection`. Click on the **Next** button to proceed to the next screen. The following screenshot illustrates this step:

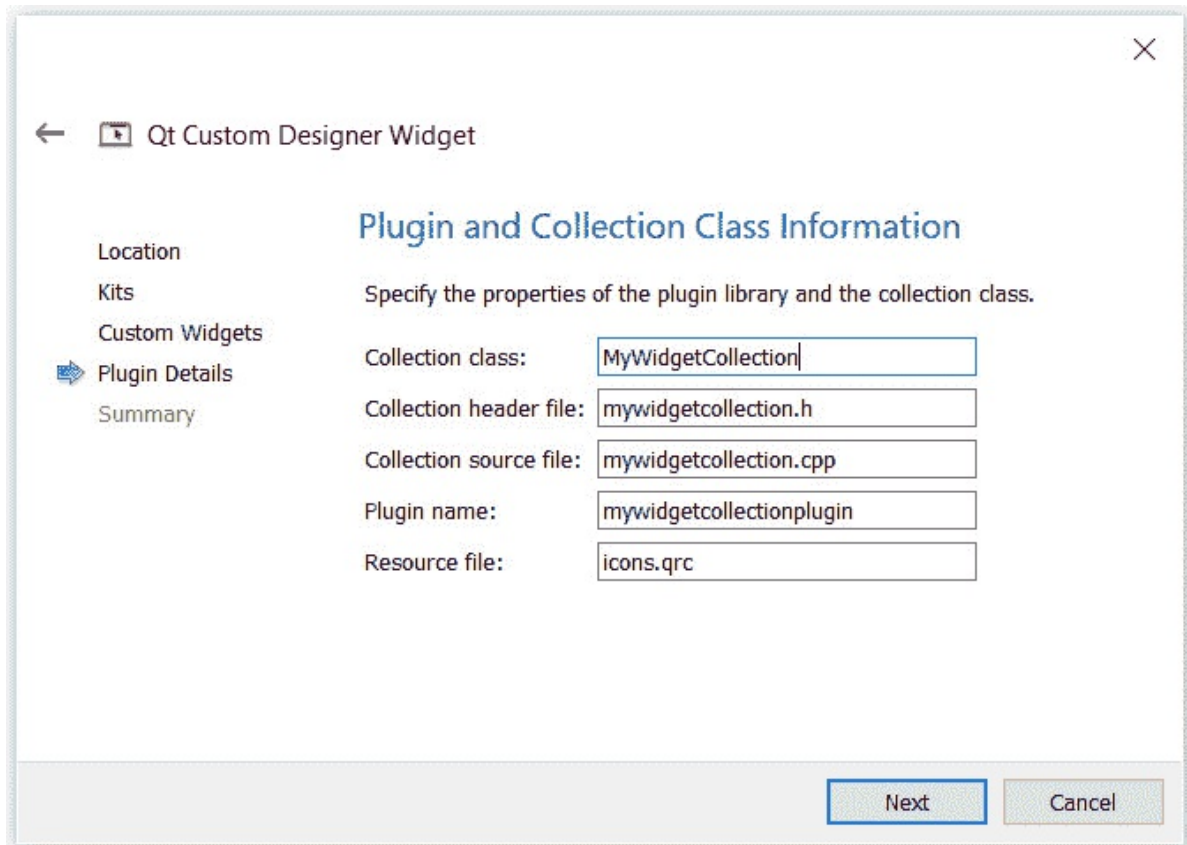


图 3.15-指定插件和收集类信息的选项

7. The next step is to add your custom widget project to the installed version control system. You may skip version control for this project. Click on the **Finish** button to create the project with the generated files. The following screenshot illustrates this step:

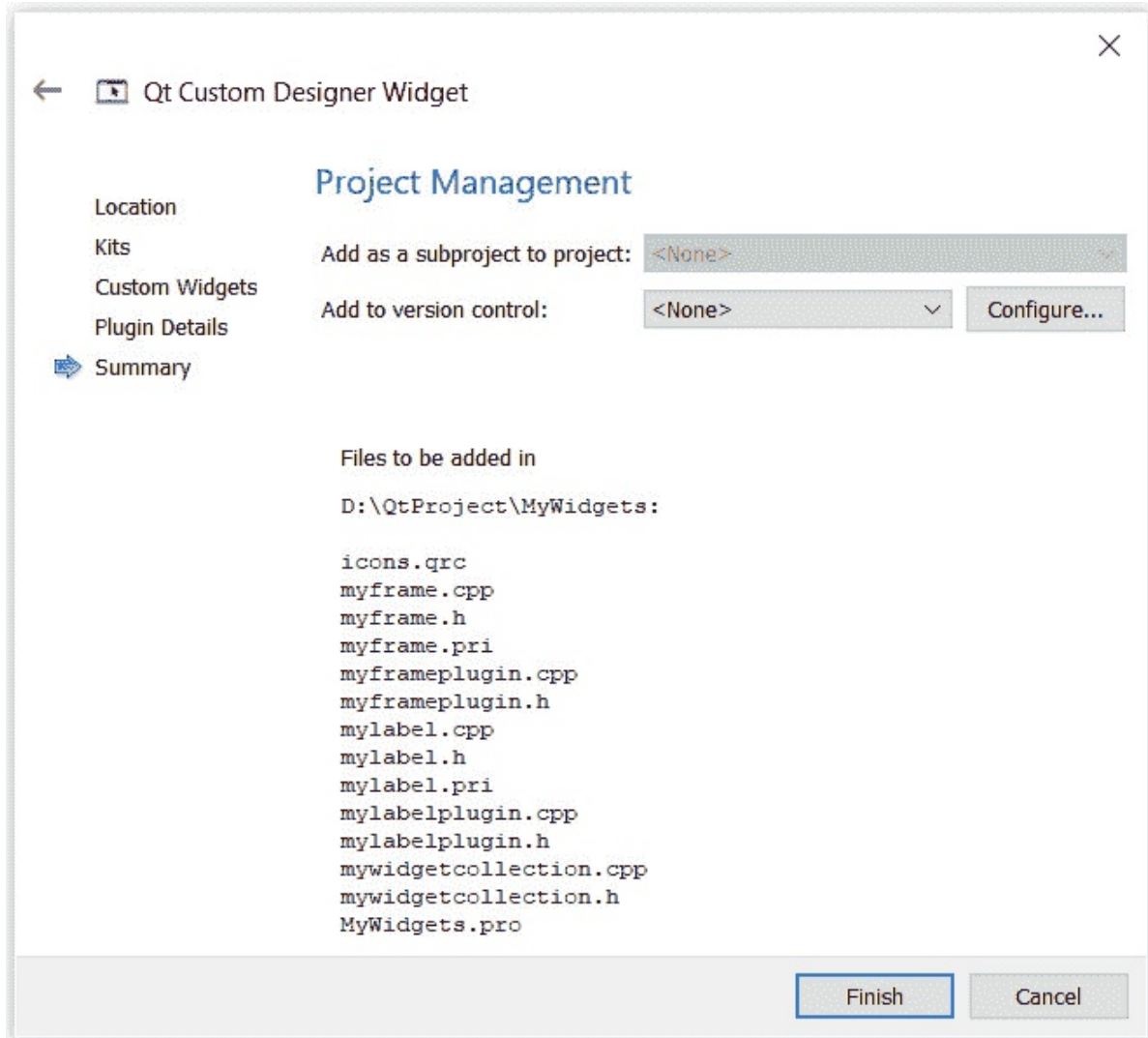
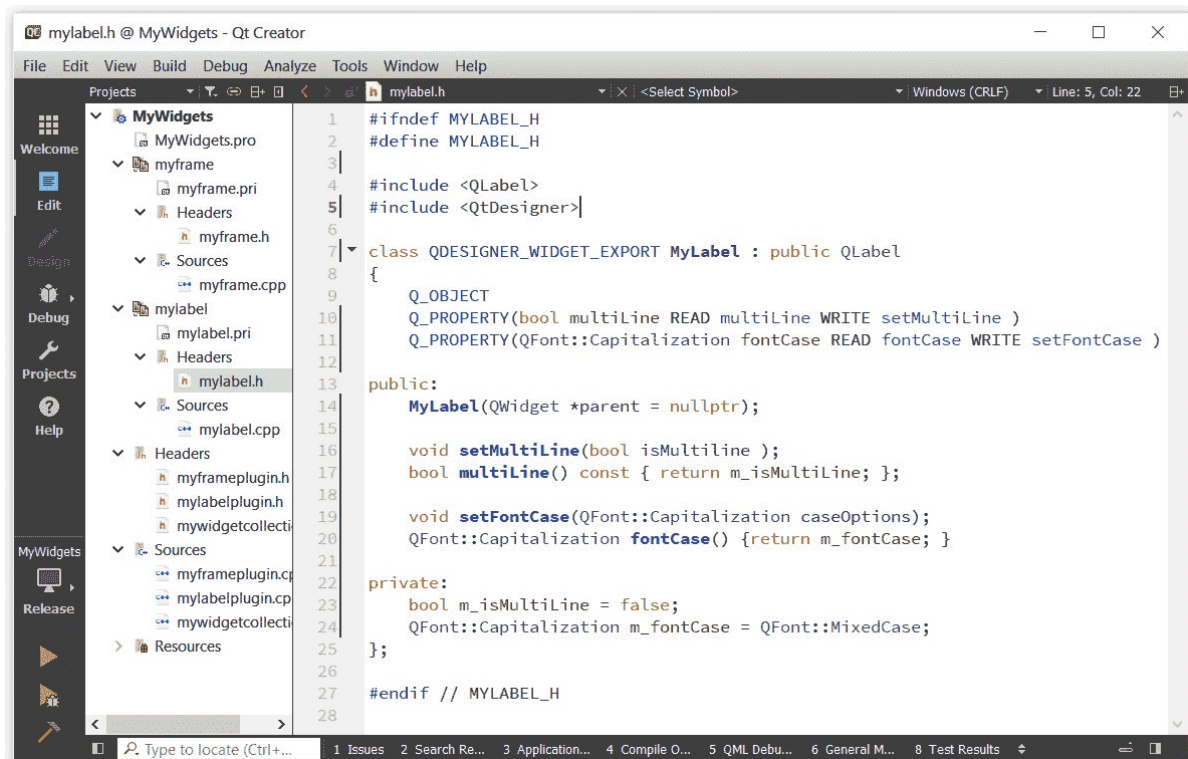


图 3.16-项目管理屏幕

8. Expand the **Project Explorer** views and open the `mylabel.h` file. We will modify the contents to extend the functionalities. Add a `QDESIGNER_WIDGET_EXPORT` macro before the custom widget class name to ensure the class is exported properly in the **dynamic-link library (DLL)** or the shared library. Your custom widget may work without this macro, but it is a good practice to add this macro. You will have to add `#include <QtDesigner>` to the header file after you insert the macro. The following screenshot illustrates this step:



重要音符

在某些平台上，构建系统可能会删除 Qt Designer 模块创建新小工具所需的符号，使它们无法使用。使用 `QDESIGNER_WIDGET_EXPORT` 宏可确保符号保留在这些平台上。在创建跨平台库时，这一点很重要。在其他 `er` 平台上没有副作用。

- Now, open the `mylabelplugin.h` file. You will find that the plugin class is inherited from a new class named `QDesignerCustomWidgetInterface`. This class allows Qt Designer to access and create custom widgets. Please note that you must update the header file as follows to avoid deprecated warnings:

```
#include <QtUiPlugin/QDesignerCustomWidgetInterface>
```

0. 您会发现在`mylabelplugin.h`中自动创建了几个函数。不要删除这些函数。您可以指定出现在 Qt 设计器模块中的`name()`、`group()`和`icon()`函数中的值。请注意，如果您没有在`icon()`中指定图标路径，那么 Qt Designer 将使用默认的 Qt 图标。下面的代码片段说明了`group()`函数：

```
```cpp
QString MyFramePlugin::group() const
{
 return QLatin1String("My Containers");
}
```
```

1. 您可以在中看到下面的代码片段：`isContainer()`在`MyLabel`中返回`false`，在`MyFrame`中返回`true`，因为`MyLabel`不是为容纳其他小部件而设计的。Qt 设计器调用`createWidget()`获取`MyLabel`或`MyFrame`的实例：

```
```cpp
bool MyFramePlugin::isContainer() const
{
 return true;
}
```
```

2. 要创建具有定义的几何图形或任何其他属性的小部件，请在`domXML()`方法中指定这些属性。此函数返回可扩展标记语言(**XML**)代码段，小部件工厂使用该代码段创建具有定义属性的自定义小部件。让我们将`MyLabel`宽度指定为100像素(**px**)，将高度指定为16**px**，如下所示：

```
```cpp
QString MyLabelPlugin::domXml() const
{
 return "<ui language=\"c++ \""
```

```

 displayname="MyLabel">\n"
 " <widget class="MyLabel"
 name="myLabel">\n"
 " <property name="geometry">\n"
 " <rect>\n"
 " <x>0</x>\n"
 " <y>0</y>\n"
 " <width>100</width>\n"
 " <height>16</height>\n"
 " </rect>\n"
 " </property>\n"
 " <property name="text">\n"
 " <string>MyLabel</string>\n"
 " </property>\n"
 " </widget>\n"
"</ui>\n";
}

```

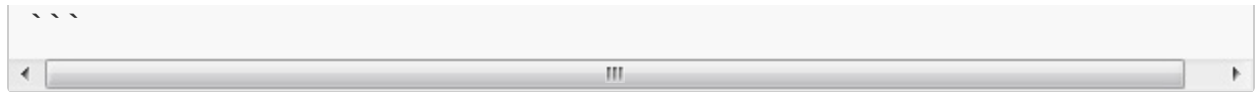
3. 现在，让我们看一下文件 `MyWidgets.pro`。它包含 `qmake` 构建自定义小部件集合库所需的所有信息。您可以在以下代码片段中看到该项目是库类型，并配置为用作插件：

```

` ` ` cpp
CONFIG += plugin debug_and_release
CONFIG += c++ 17
TARGET = $$qtLibraryTarget(
 mywidgetcollectionplugin)
TEMPLATE = lib
HEADERS = mylabelplugin.h myframeplugin.h mywidgetcollection.h
SOURCES = mylabelplugin.cpp myframeplugin.cpp \
 mywidgetcollection.cpp
RESOURCES = icons.qrc
LIBS += -L.
greaterThan(QT_MAJOR_VERSION, 4) {
 QT += designer
} else {
 CONFIG += designer
}
target.path = $$[QT_INSTALL_PLUGINS]/designer
INSTALLS += target
include(mylabel.pri)
include(myframe.pri)

```





4. We have gone through the custom widget creation process. Let's run `qmake` and build the library in the **Release** mode. Right-click on the project and click on the **Build** option, as shown in the following screenshot. The project will be built within a few seconds and will be available inside the `inside release` folder. On the Windows platform, you can manually copy the `mywidgetcollectionplugin.dll` created plugin library to the `D:\Qt\6.0.0\mingw81_64\plugins\designer` path. This path and extension vary for different operating systems:

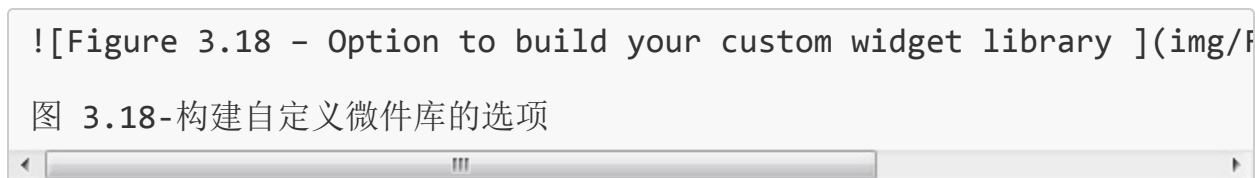


图 3.18-构建自定义微件库的选项

5. We have created our custom plugin. Now, close the plugin project and click on the `designer.exe` file present inside `D:\Qt\6.0.0\mingw81_64\bin`. You can see `MyFrame` under the **Custom Widgets** section, as shown in the following screenshot. Click on the **Create** button or use a widget template. You can also register your own form as a template by doing platform-specific modifications. Let's use the Qt Designer-provided widget template:

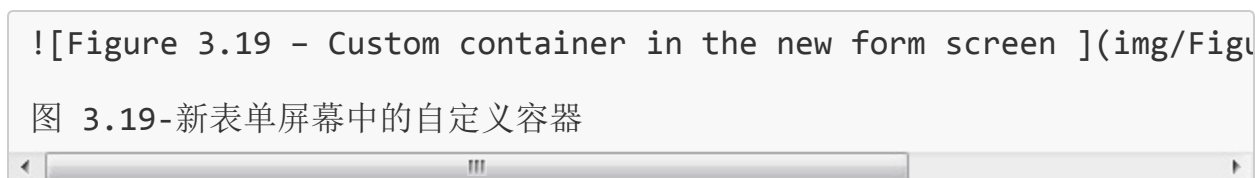


图 3.19-新表单屏幕中的自定义容器

6. 您可以在底部左侧的**Widget Box**部分中看到我们的自定义小部件。

将**MyLabel**小部件拖到表单上。您可以在属性编辑器下找到创建的属性，如**MULTLINE**和**FontCase**以及**QLabel**属性，如以下屏幕截图所示：

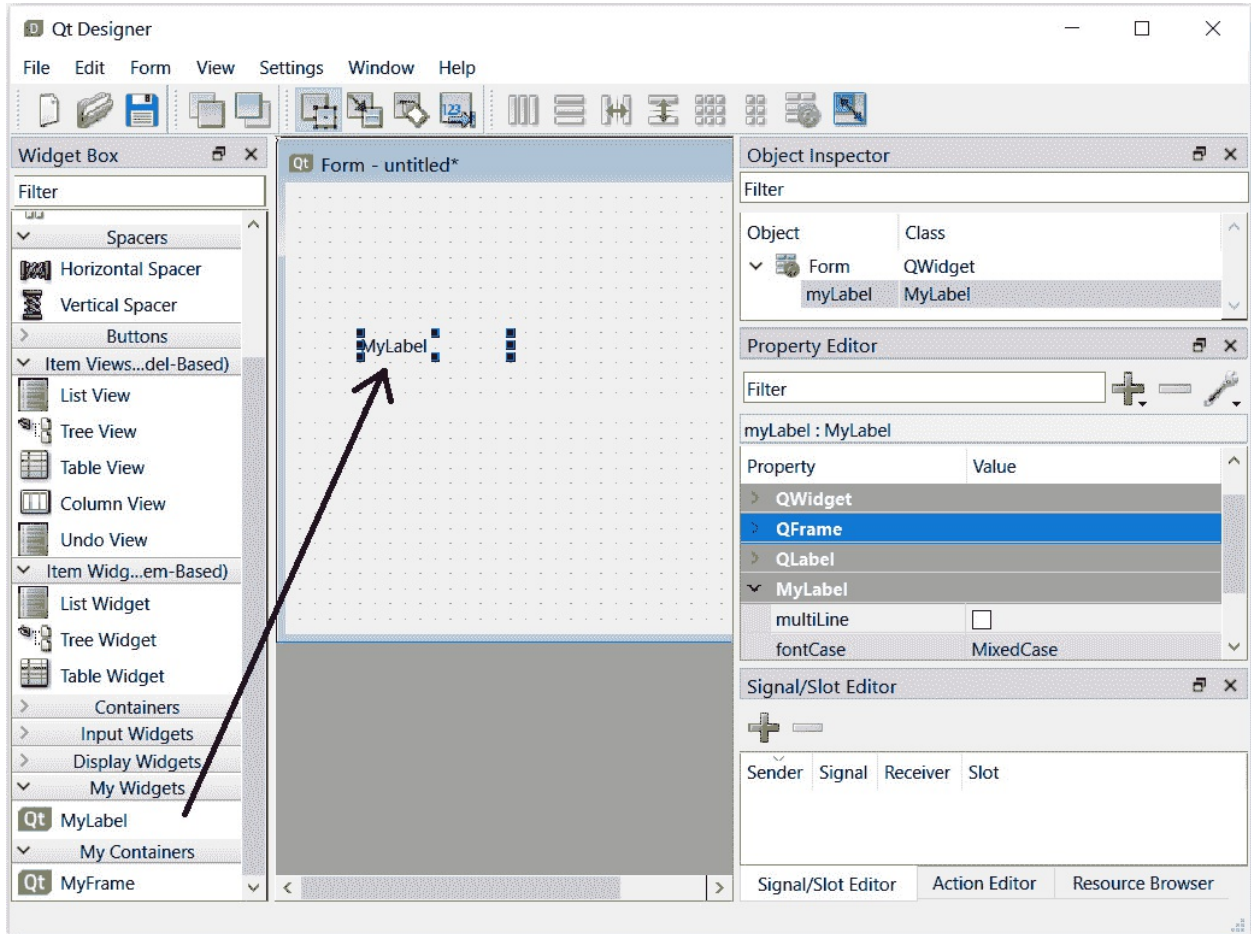


图 3.20-Qt Designer 中可用的导出小部件

您还可以在以下 Qt 文档链接中找到带有示例的详细说明：

<https://doc.qt.io/qt-6/designer-creating-custom-widgets.html>

祝贺你！您已经成功地创建了具有新属性的自定义小部件。您可以通过组合多个小部件来创建复杂的自定义小部件。在下一节中，您将学习如何定制小部件的外观。

# 创建 Qt 样式表和自定义主题

---

在上一节中，我们创建了自定义小部件，但小部件仍然具有原生外观。Qt 提供了几种定制 UI 外观的方法。Qt 样式表是更改小部件外观的最简单方法之一，无需进行太复杂的编码。Qt 样式表语法与超文本标记语言/级联样式表(HTML/CSS)语法相同。样式表由样式规则的序列组成。样式规则由选择器和声明组成。选择器指定将受样式规则影响的小部件，声明指定小部件的属性。样式规则的声明部分是作为键-值对的属性列表，包含在{}内，并用分号分隔。

让我们看一下简单的QPushButton样式表语法，如下所示：

```
QPushButton { color: green; background-color: rgb (193, 255, 216);}
```

您还可以通过使用样式表编辑器在 Qt Designer 中应用样式表来更改小工具的外观，如下所示：

1. 打开 Qt Designer 模块并创建一个新窗体。在表单上拖动并放置一个按钮。
2. 然后，右键单击按钮或表单中的任意位置以获得上下文菜单。
3. Next, click on the **Change styleSheet...** option, as shown in the following screenshot:

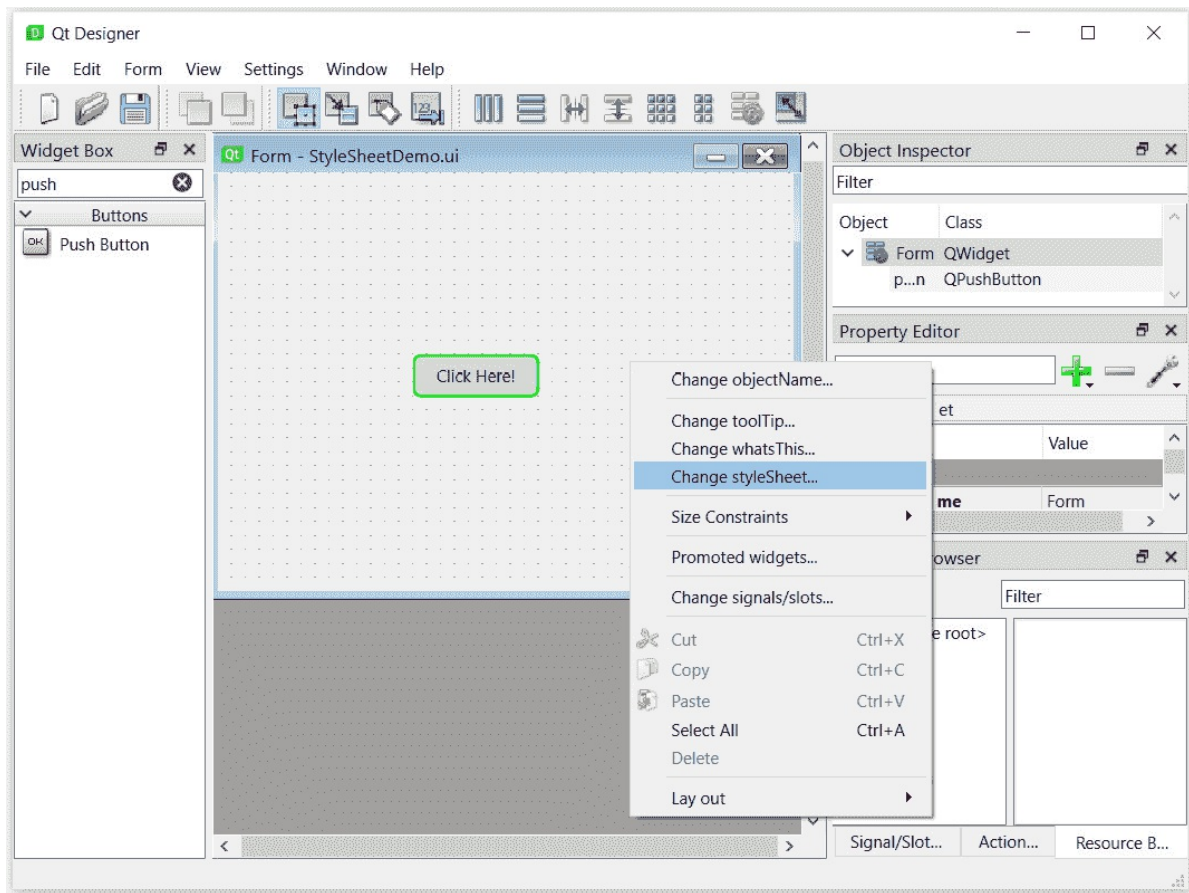


图 3.21-使用 Qt Designer 添加样式表

4. 我们已经使用下面的样式表创建了前面的外观。还可以在属性编辑器

```
QPushButton {
 background-color: rgb(193, 255, 216);
 border-width: 2px;
 border-radius: 6;
 border-color: lime;
 border-style: solid;
 padding: 2px;
 min-height: 2.5ex;
 min-width: 10ex;
}
QPushButton:hover {
 background-color: rgb(170, 255, 127);
}
QPushButton:pressed {
```

```
background-color: rgb(170, 255, 127);
font: bold;
}
```

中更改 `QWidget` 属性中的样式表

在前面的示例中，只有 `Push Button` 将获得样式表中描述的样式，所有其他小部件都将具有本机样式。您还可以为每个按钮创建不同的样式，并通过在样式表中提及它们的对象名称将样式应用到各个按钮，如下所示：

`QPushButton#pushButtonID`

重要音符

要了解有关样式表及其用法的更多信息，请阅读以下链接中的文档：

<https://doc.qt.io/qt-6/stylesheet-reference.html>

<https://doc.qt.io/qt-6/stylesheet-syntax.html>

<https://doc.qt.io/qt-6/stylesheet-customizing.html>

## 使用 **QSS** 文件

您可以将所有样式表代码组合到一个已定义的 `.qss` 文件中。这有助于确保在所有屏幕上保持整个应用的外观。 **QSS** 文件类似于 `.css` 文件，其中包含 **GUI** 元素的外观定义，如颜色、背景颜色、字体和鼠标交互行为。可以使用任何文本编辑器创建和编辑它们。您可以创建一个扩展名为 `.qss` 的新样式表文件，然后将其添加到资源文件 (`.qrc`)。您可能也有也可能没有所有项目的 `.ui` 文件。 **GUI** 控件可以通过代码动态创建。您可以将样式表应用于小部件或整个应用，如以下代码片段所示。以下是我们对自定义小部件或表单执行此操作的方式：

```
MyWidget::MyWidget(QWidget *parent)
 : QWidget(parent)
{
 setStyleSheet("QWidget { background-color: green }");
}
```

下面是我们如何将其应用于整个应用：

```
#include "mywidget.h"
#include <QApplication>
#include <QFile>
int main(int argc, char *argv[])
{
 QApplication app(argc, argv);
 QFile file(":/qss/default.qss");
 file.open(QFile::ReadOnly);
 QString styleSheet = QLatin1String(file.readAll());
 app.setStyleSheet(styleSheet);
 Widget mywidget;
 mywidget.show();
 return app.exec();
}
```

前面的程序说明了如何将样式表文件用于整个 Qt GUI 应用。您需要添加。 `qss` 文件放入资源中。 打开。 使用 `QFile` 创建 `qss` 文件，并将定制的 QSS 规则作为参数传递给 `QApplication` 对象上的 `setStyleSheet()` 方法。 您将看到所有屏幕都将应用样式表。

在本节中，您了解了如何使用样式表自定义应用的外观，但更改应用外观的方法还有很多。 这些方法取决于您的项目需求。 在下一节中，您将了解自定义样式。

## 浏览自定义样式

Qt 提供了几个 `QStyle` 子类，这些子类模拟 Qt 支持的不同平台的样式。 这些样式在 Qt GUI 模块中随时可用。 您可以构建自己的自定义样式并

将其导出为插件。Qt 使用 `QStyle` 来呈现 Qt 小部件，以确保它们的外观，就像本地小部件一样。

在 Unix 发行版上，您可以通过运行以下命令为您的应用获得 Windows 风格的 UI：

```
$. /helloworld -style windows
```

您可以使用 `QWidget::setStyle()` 方法在单个小工具上设置样式。

## 创建自定义样式

您可以通过创建自定义样式来自定义 GUI 的外观。创建自定义样式有两种不同的方法。在静态方法中，您可以子类化 `QStyle` 类并重新实现虚拟函数以提供所需的行为，或者从头开始重写 `QStyle` 类。 `QCommonStyle` 通常用作基类，而不是 `QStyle`。在动态方法中，您可以子类化 `QProxyStyle` 并在运行时修改系统样式的行为。您还可以使用 `drawPrimitive()`、`drawItemText()` 和 `drawControl()` 等 `QStyle` 函数开发样式感知的自定义小部件。

本节是高级 Qt 主题。您需要深入了解 Qt 才能创建您自己的样式插件。如果你是初学者，你可以跳过这一节。您可以通过以下链接在 Qt 文档中了解 `QStyle` 类和自定义样式：

<https://doc.qt.io/qt-6/qstyle.html>

## 使用自定义样式

有几种方法可以在 Qt 应用中应用自定义样式。最简单的方法是在创建 `QApplication` 对象之前调用 `QApplication::setStyle()` 静态函数，如下所示：



```
#include "customstyle.h"
int main(int argc, char *argv[])
{
 QApplication::setStyle(new CustomStyle);
 QApplication app(argc, argv);
 Widget helloworld;
 helloworld.show();
 return app.exec();
}
```

您还可以将自定义样式作为命令行参数应用，如下所示：

```
>./customstyledemo -style customstyle
```

自定义样式可能很难实现，但可以更快、更灵活。QSS 很容易学习和实现，但性能可能会受到影响，特别是在应用启动时，因为 QSS 解析可能需要时间。您可以选择对您或您的组织方便的方法。我们已经学习了如何自定义 GUI。现在，让我们来了解一下本章最后一节中的小部件、窗口和对话框是什么。

## 使用小工具、窗口和对话框

小工具是可以在屏幕上显示的 GUI 元素。这可能包括标签、按钮、列表视图、窗口、对话框等。所有小部件都在屏幕上向用户显示特定信息，其中大多数允许用户通过键盘或鼠标进行交互。

窗口是没有其他父部件的顶级部件。通常，除非指定了任何窗口标志，否则窗口都有标题栏和边框。窗口样式和某些策略由底层窗口系统确定。Qt 中的一些常见窗口类有 `QMainWindow`、`QMessageBox` 和 `QDialog`。主窗口通常遵循桌面应用的预定义布局，该布局包括菜单栏、工具栏、中央小工具区域和状态栏。`QMainWindow` 需要一个中央小部件，即使它只是一个占位符。可以在主窗口中删除其他组件。图 3.22 说明了 `QMainWindow` 的布局结构。我们通常调用 `show()` 方法来显示

小部件或主窗口。

`QMenuBar` 位于 `QMainWindow` 的顶部。您可以添加菜单选项，如文件、编辑、查看和帮助。在下面显示 `QMenuBar` 的屏幕截图中，有 `QToolBar`。

`QDockWidget` 提供一个可以停靠在 `QMainWindow` 内部或作为顶级窗口浮动的小部件。中心小部件是您可以在其中添加表单或子小部件的主视图区域。使用子小部件创建您自己的视图区域，然后调用 `setCentralWidget()`：

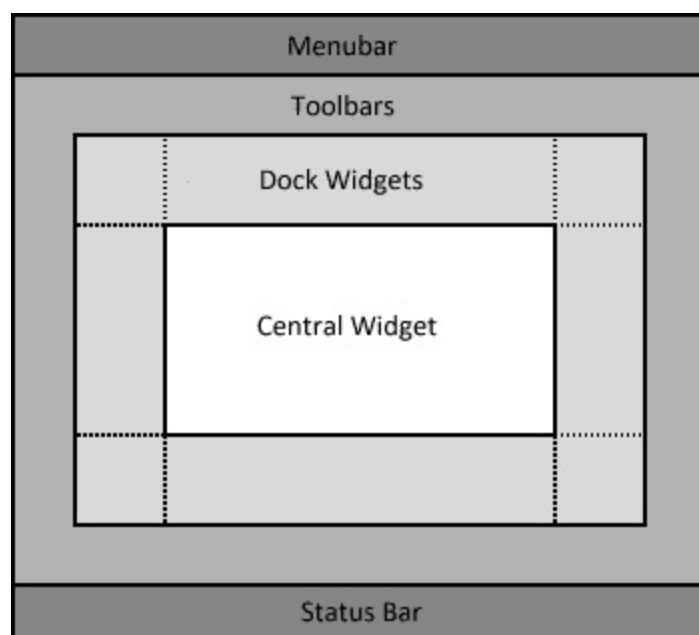


图 3.22-`QMainWindow` 布局

重要音符

`QMainWindow` 不应与 `QWindow` 混淆。 `QWindow` 是一个方便的类，表示底层窗口系统中的一个窗口。通常，应用使用 `QWidget` 或 `QMainWindow` 作为其 UI。但是，如果您希望保持最小的依赖关系，则可以直接呈现到 `QWindow`。

对话框是用于提供通知或接收用户输入的临时窗口，通常有 **OK** 和

**Cancel**类型的按钮。 `QMessageBox` 是一种用于向用户显示信息和警报或向用户提问的对话框。通常， `exec()` 方法用于显示对话框。该对话框显示为模式对话框，并且在用户关闭它之前本质上是阻塞的。可以使用以下代码片段创建一个简单的消息框：

```
QMessageBox messageBox;
messageBox.setText("This is a simple QMessageBox.");
messageBox.exec();
```

结果是，所有这些都是小工具。窗口是最顶层的小部件，对话框是一种特殊的窗口。

## 摘要

---

本章解释了 **Qt** 小部件模块的基本原理以及如何创建自定义 **UI**。在这里，您学习了如何使用 **Qt Designer** 设计和构建 **GUI**。传统的桌面应用通常是使用 **Qt Designer** 构建的。自定义小部件插件等功能允许您通过 **Qt Designer** 创建和使用您自己的小部件集合。我们还讨论了如何使用样式表和样式自定义应用的外观，以及查看小部件、窗口和对话框的使用和区别。现在，您可以使用自己的自定义小部件创建具有扩展功能的 **GUI** 应用，并为桌面应用创建自己的主题。

在下一章中，我们将讨论 `QtQuick` 和 **QML**。在这里，您将了解 `QtQuick` 控件、**Qt** 快速设计器，以及如何构建自定义 **QML** 应用。我们还将讨论使用 **Qt Quick** 进行 **GUI** 设计而不是小部件的另一种选择。

## 四、Qt Quick 和 QML

---

原文: <https://gitee.com/apacheecn/apacheecn-c-cpp-zh-pt2/blob/master/docs/cross-plat-dev-qt6-mod-cpp/04.md>

Qt 由两个不同的模块组成，用于开发图形用户界面(GUI)应用。第一种方法是使用 Qt 小部件和 C++，我们在上一章中已经了解到这一点。第二种方法是使用 Qt 快速控件和Qt 建模语言(QML)，我们将在本章中介绍。

在本章中，您将学习如何使用 Qt 快速控件和 QML 脚本语言。您将学习如何使用 Qt Quick Layout 和定位器，以及如何制作响应迅速的 GUI 应用。您将学习如何将后端 C++ 代码与前端 QML 集成。您将学习 Qt Quick 和 QML 的基础知识，以及如何开发触摸友好和面向视觉的 Qt 应用。您还将了解鼠标和触摸事件，以及如何开发触摸感知应用。

在本章中，我们将介绍以下主要主题：

- QML 和 Qt 快速入门
- 了解 Qt 快速控件
- 创建一个简单的 Qt Quick 应用
- 使用 Qt Quick Designer 设计用户界面(UI)
- QML 中的定位器和布局
- QML 与 C++ 的集成
- 将 QML 与JavaScript(JS)集成
- 处理鼠标和触摸事件

在本章结束时，您将了解 QML 的基础知识、与 C++ 的集成，以及如何创建自己的 Fluid UI。

# 技术要求

---

本章的技术要求包括在 Windows 10、Ubuntu 20.04 或 MacOS 10.14 等最新桌面平台上安装 Qt 6.0.0 和 Qt Creator 4.14.0 的最低版本。

本章使用的所有代码都可以从以下 giHub 链接下

载：<https://github.com/PacktPublishing/Cross-Platform-Development-with-Qt-6-and-Modern-Cpp/tree/master/Chapter04>。

## 重要音符

本章中使用的屏幕截图取自 Windows 平台。根据计算机中的底层平台，您将看到类似的屏幕。

# QML 和 Qt 快速入门

---

QML 是一种 UI 标记语言。它是一种声明性语言，是 Qt 框架的一部分。它可以构建流畅、触控友好的用户界面，并随着触摸屏移动设备的发展而出现。它是为高度动态而创建的，开发人员可以通过最少的代码轻松创建流畅的 UI。Qt QML 模块实现了 QML 架构，并提供了开发应用的框架。它定义和实现了语言和基础设施，并提供了应用编程接口(API)来集成 QML 语言与 JS 和 C++。

Qt Quick 为 QML 提供了一个类型和功能库。它包括交互类型、视觉类型、动画、模型、视图和图形效果。它用于移动应用，在移动应用中，触摸输入、流畅的动画和用户体验至关重要。Qt QML 模块为 QML 应用提供了语言和基础设施，而 Qt Quick 模块提供了许多可视化元素、动画和更多模块来开发面向触摸和视觉吸引力的应用。您可以使用 QML 和 Qt Quick 控件，而不是使用 Qt 小部件进行 UI 设计。Qt Quick 支持多种平台，如 Windows、Linux、Mac、iOS 和 Android。

您可以用 C++ 创建自定义类，并将其移植到 Qt Quick 以扩展其功能。此外，该语言还提供了与 C++ 和 JS 的平滑集成。

## 了解 QML 类型系统

让我们熟悉一下的 QML 类型系统和各种 QML 类型。QML 文件中的类型可以来自各种来源。下面概述了 QML 文件中使用的不同类型：

- QML 原生提供的基本类型，如 `int`、`bool`、`real` 和 `list`
- JS 类型，如 `var`、`Date` 和 `Array`
- QML 对象类型，如 `Item`、`Rectangle`、`Image` 和 `Component`
- QML 模块(如 `BackendLogic`)通过 C++ 注册的类型
- 作为 QML 文件提供的类型，如 `MyPushButton`

基本类型可以包含简单的值，如 `int` 或 `bool` 类型。除了原生基本类型之外，Qt Quick 模块还提供了其他基本类型。QML 引擎还支持 JS 对象和数组。任何标准 JS 类型都可以使用泛型 `var` 类型创建和存储。请注意，`variant` 类型已过时，仅支持较旧的应用。QML 对象类型是可以从中创建 QML 对象的类型。可以通过创建定义类型的 `.qml` 文件来定义自定义 QML 对象类型。QML 对象类型可以具有属性、方法、信号等。

要在 QML 文件中使用基本的 QML 类型，请使用以下代码行导入 `QtQml` 模块：`import QtQml`

`Item` 是 Qt Quick 中所有视觉元素的基本类型。Qt Quick 中的所有可视项目都继承自 `Item`，它是一个透明的可视元素，可以用作容器。Qt Quick 提供了 `Rectangle` 作为绘制矩形的可视类型，并提供了 `Image` 类型来显示图像。`Item` 提供可视元素的通用属性集。我们将在整本书中探索这些类型的用法。

您可以通过以下链接了解有关 QML 类型的更多信息：

<https://doc.qt.io/qt-6/qmltypes.html>

在本节中，我们学习了 QML 和 Qt Quick 的基础知识。在下一节中，我们将讨论 Qt 快速控件。

## 了解 Qt 快速控件

---

**Qt Quick Controls**提供了一组个 UI 元素，可用于使用 Qt Quick 构建流畅的 UI。为了避免小部件的歧义，我们将使用术语控件来表示 UI 元素。**Qt Quick Controls 1**最初设计用于支持桌面平台。随着移动设备和嵌入式系统的发展，该模块需要进行更改以满足性能预期。因此，**Qt Quick Controls 2**应运而生，它进一步增强了对移动平台的支持。**Qt Quick Controls 1**从 Qt 5.11 起已弃用，并已从 Qt 6.0 中删除。**Qt Quick Controls 2**现在简称为 **Qt Quick Controls**。

可以使用 `.qml` 文件中的以下 `import` 语句将 QML 类型导入到您的应用中：

```
import QtQuick.Controls
```

### 重要音符

在 Qt6 中，QML 导入和版本控制系统有一些变化。版本号一直是可选的。如果在未指定版本号的情况下导入模块，则会自动导入模块的最新版本。如果您导入的模块仅具有主版本号，则该模块将使用指定的主要版本和最新的次要版本导入。Qt6 引入了自动导入功能，即写成 `import <module> auto`。这可确保导入的模块和导入的模块具有相同的版本号。



在 Qt 6 中对 Qt 快速控件的更改可在以下链接中找到：

<https://doc.qt.io/qt-6/qtquickcontrols-changes-qt6.html>

Qt 快速控件提供了用于创建 UI 的 QML 类型。这里给出了 Qt 快速控件的示例：

- **ApplicationWindow**：带有样式的顶级窗口，支持页眉和页脚
- **BusyIndicator**：指示后台活动-例如，在加载内容时
- **Button**：可点击以执行命令或回答问题的按钮
- **CheckBox**：可切换开或关的复选按钮
- **ComboBox**：选择选项的组合按钮和弹出式列表
- **Dial**：旋转以设定值的圆形刻度盘
- **Dialog**：带有标准按钮和标题的弹出对话框
- **Label**：使用继承字体设置样式的文本标签
- **Popup**：类似弹出式 UI 控件的基本类型
- **ProgressBar**：表示操作进度
- **RadioButton**：可打开或关闭的独占单选按钮
- **ScrollBar**：垂直或水平互动滚动条
- **ScrollView**：可滚动视图
- **Slider**：用于通过沿轨迹滑动手柄来选择值
- **SpinBox**：允许用户从一组预设值中进行选择
- **Switch**：可以打开或关闭的按钮
- **TextArea**：多行文本输入区
- **TextField**：单行文本输入字段
- **ToolTip**：为任何控件提供工具提示
- **Tumbler**：可以选择的项目的可旋转滚轮

要配置 Qt Quick Controls 模块以使用 **qmake** 进行构建，请将以下行添

加到项目的 `.pro` 文件中：

```
QT += quickcontrols2
```

在本节中，我们了解了 Qt Quick 提供的不同类型的 UI 元素。在下一节中，我们将讨论 Qt Quick 提供的不同样式以及如何应用它们。

## 设置 Qt 快速控件的样式

Qt 快速控件附带一组标准样式。下面列出了它们：

- 基本
- 融合
- \*T0想象一下元 T1
- 材料
- 通用

在 Qt 快速控件中应用样式有两种方法，如下所示：

- 编译时间
- 运行时

您可以通过导入相应的样式模块来应用编译时样式，如下所示：

```
import QtQuick.Controls.Universal
```

您可以使用以下方法之一应用运行时样式：

QQuickStyle::setStyle()	QQuickStyle::setStyle("Universal");
-style command line argument	./application -style universal
Environment variable QT_QUICK_CONTROLS_STYLE	QT_QUICK_CONTROLS_STYLE=universal ./application
qtquickcontrols2.conf configuration file	[Controls] Style=Universal

图 4.1-运行时应用样式的不同方式

在本节中，我们了解了 Qt Quick 中的可用样式。在下一节中，我们将创建第一个 Qt Quick GUI 应用。

## 创建一个简单的 Qt Quick 应用

让我们使用 Qt6 创建我们的第一个 Qt Quick 应用。Hello World 程序是一个非常简单的程序，它显示 `Hello World!`。该项目使用最少也是最基本的代码。对于此项目，我们将使用由 Qt Creator 创建的项目框架。那么，让我们开始吧！请按以下步骤进行操作：

1. To create a new Qt Quick application, click on the **File menu** option on the menu bar or hit *Ctrl + N*. Alternatively, you can also click on the **New Project** button located on the welcome screen. Then, a window will pop up for you to choose a project template. Select **Qt Quick Application - Empty** and click the **Choose...** button, as shown in the following screenshot:

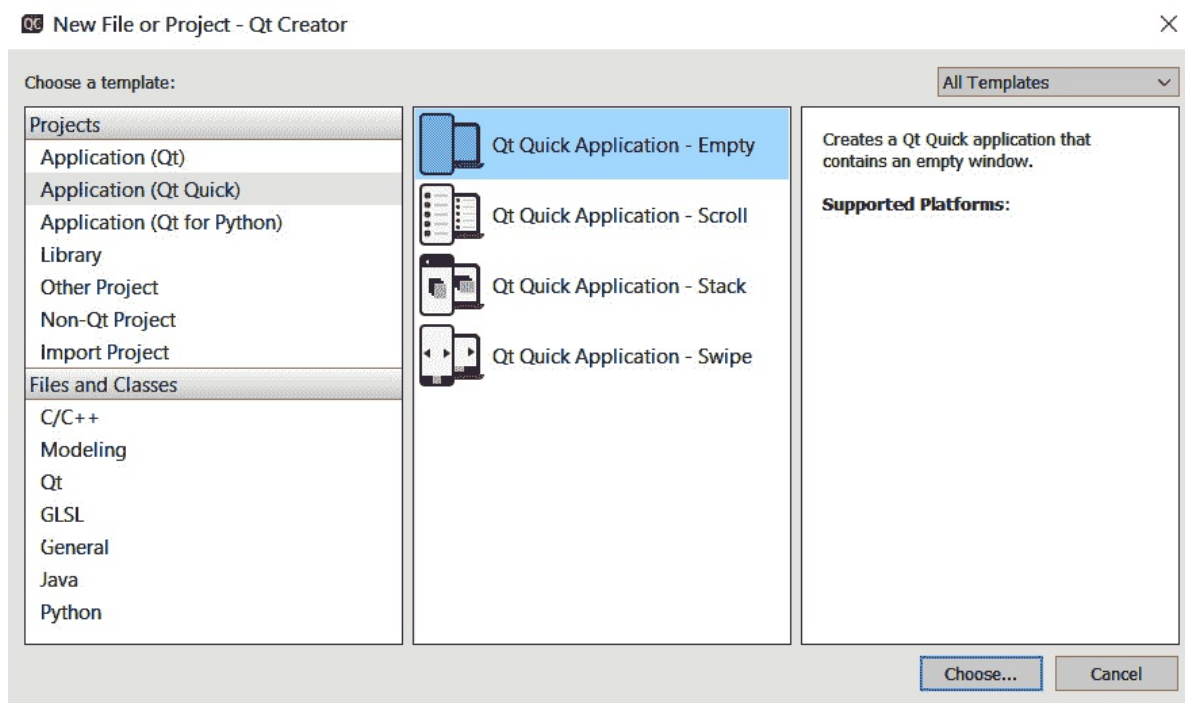


图 4.2-新建 Qt 快速应用向导

2. In the next step, you will be asked to choose a project name and a project location. You can navigate to the desired project location by clicking the **Browse...** button. Let's name our sample project `SimpleQtQuickApp`. Then, click on the **Next** button to proceed to the next screen, as shown in the following screenshot:

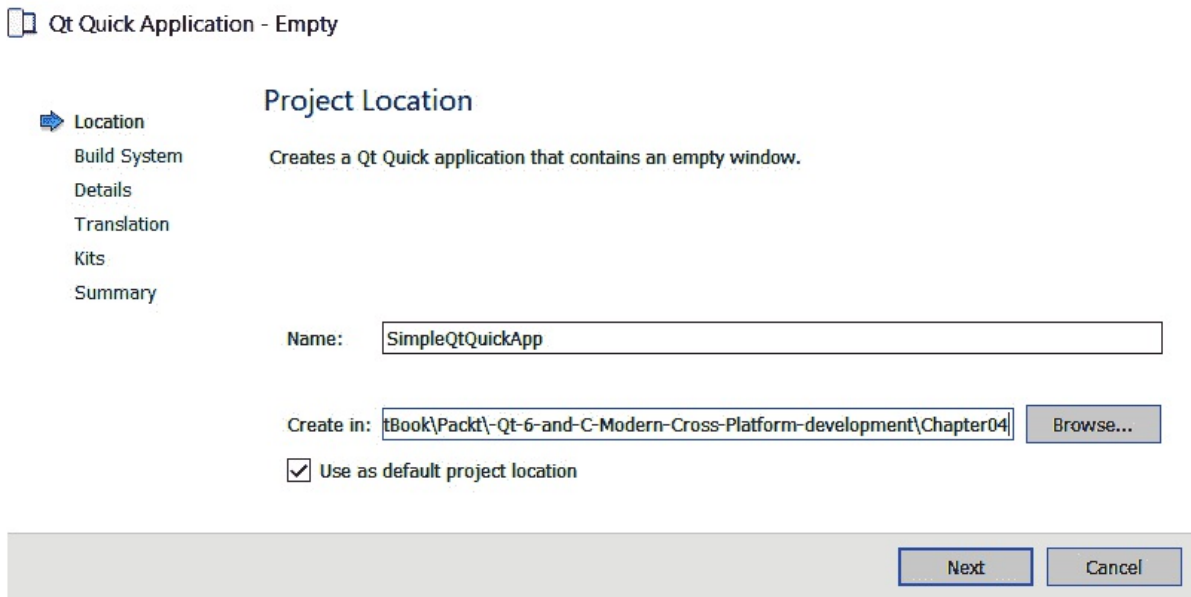


图 4.3-项目位置选择屏幕

3. In the next step, you can select a kit from a set of kits to build and run your project. To build and run a project, at least one kit must be active and selectable. Select the default **Desktop Qt 6.0.0 MinGW 64-bit** kit. Click on the **Next** button to proceed to the next screen. This can be seen in the following screenshot:

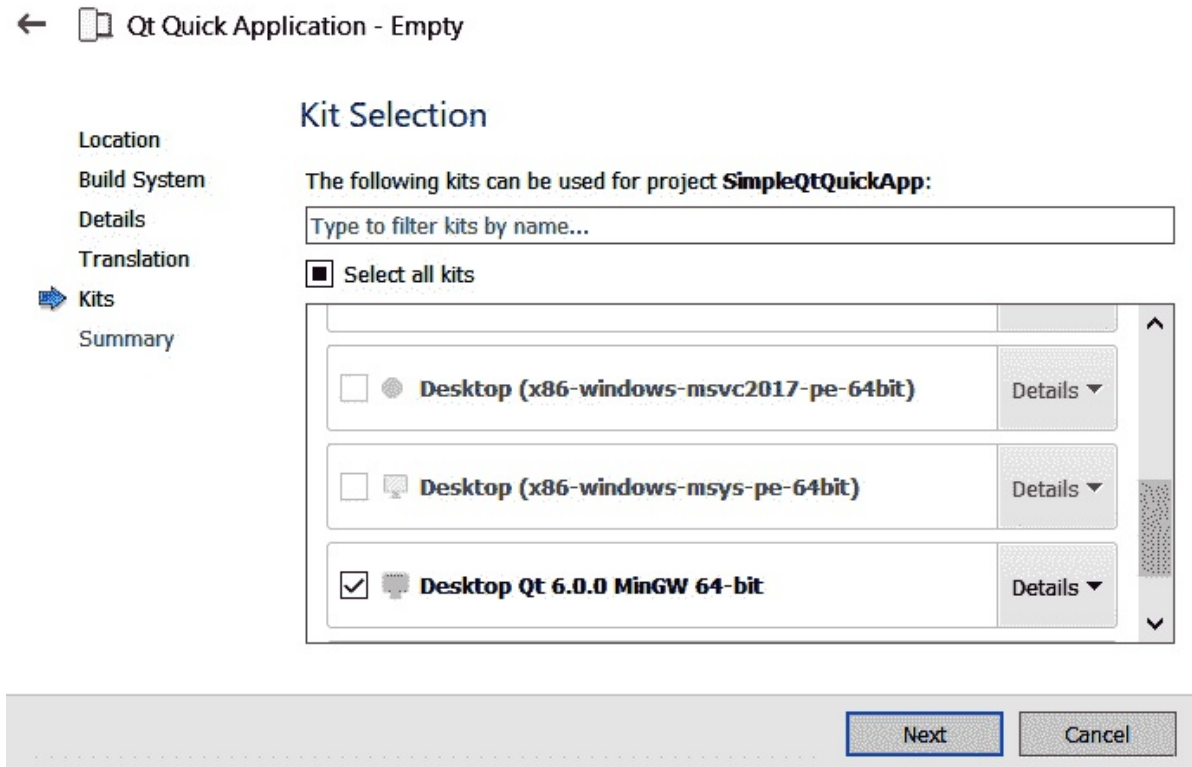


图 4.4-套件选择屏幕

4. The next step is to add your Qt Quick project to the installed **version control system (VCS)**. You may skip version control for this project. Click on the **Finish** button to create a project with the generated files, as shown in the following screenshot:

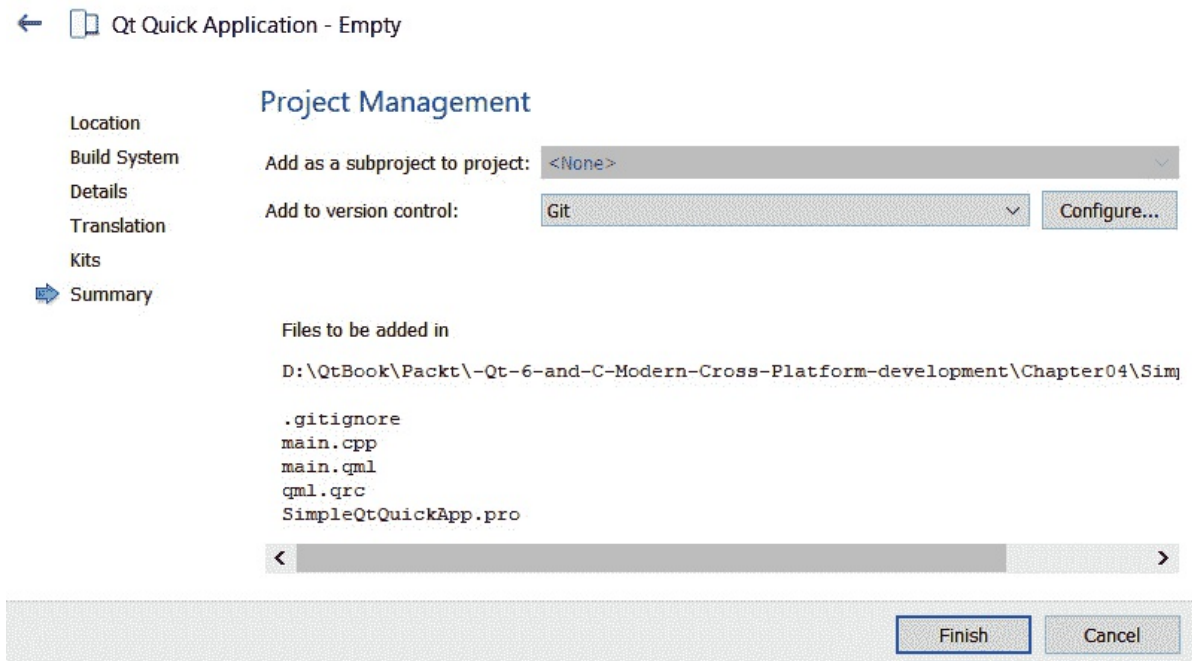


图 4.5-项目管理屏幕

5. Once a project has been created, Qt Creator will automatically open up a file from your project, called `main.qml`. You will see a type of script that is very different from your usual C/C++ projects, as shown in the following screenshot:

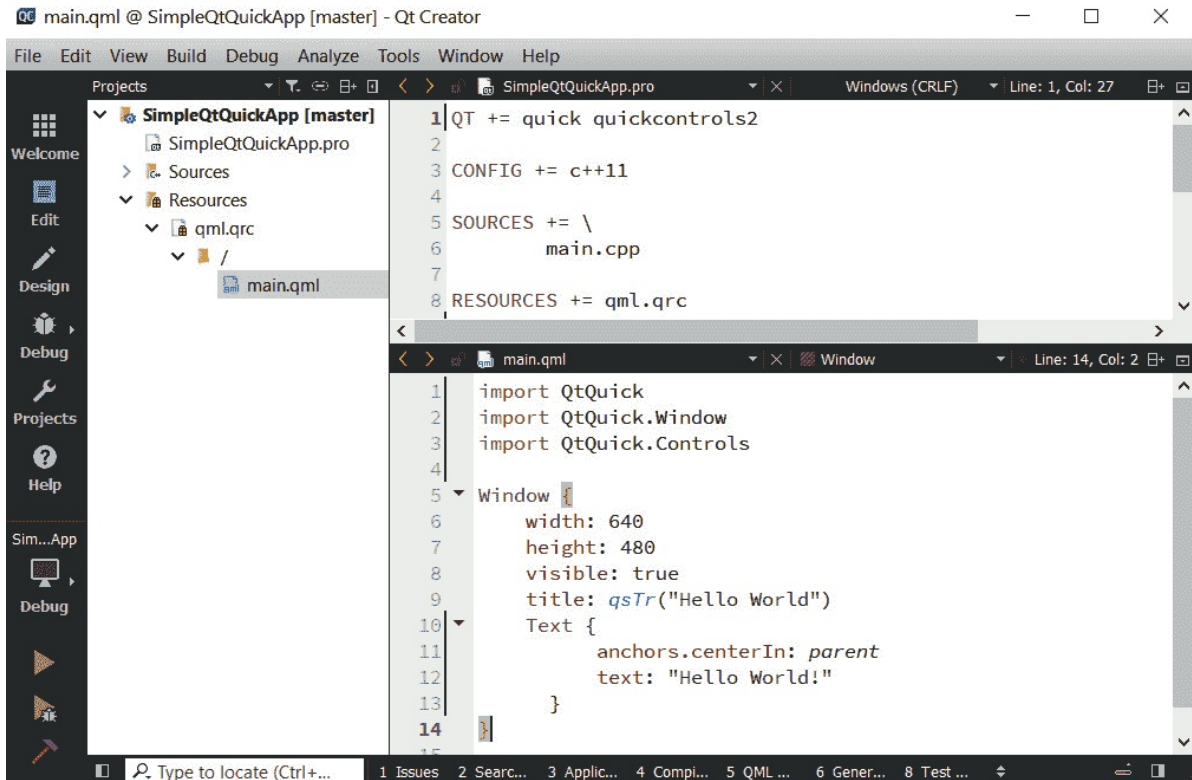


图 4.6-代码编辑器屏幕显示 main.qml 文件

QML 运行时是用 C++ 在 `QtQml` 模块中实现的。它包含一个负责执行 QML 的 QML 引擎。它还包含 QML 元素可以访问的上下文和属性。Qt 提供了用于实例化 QML 组件的 `QQmlEngine` 类。您还可以使用 `QQmlApplicationEngine` 类以一种方便的方式使用单个 QML 文件加载应用，如下所示：

```
#include <QGuiApplication>
#include <QQmlApplicationEngine>
int main(int argc, char *argv[])
{
 QGuiApplication app(argc, argv);
 QQmlApplicationEngine engine;
 const QUrl url(QStringLiteral("qrc:/main.qml"));
 engine.load(url);
 return app.exec();
}
```



您还可以使用 `QQuickView` 类，它为显示 Qt Quick UI 提供了一个窗口。这种方法有点陈旧。通过 QML，`QQmlApplicationEngine` 具有方便的中心应用功能，而 `QQuickView` 通常由 C++ 控制。下面的代码片段显示了如何使用 `QQuickView` 加载 `.qml` 文件：

```
#include <QGuiApplication>
#include <QQuickView>
int main(int argc, char *argv[])
{
 QGuiApplication app(argc, argv);
 QQuickView view;
 view.setResizeMode(
 QQuickView::SizeRootObjectToView);
 view.setSource(QUrl("qrc:/main.qml"));
 view.show();
 return app.exec();
}
```

`QQuickView` 不支持将 `Window` 用作根项目。如果您想从 QML 创建根窗口，那么选择 `QQmlApplicationEngine`。使用 `QQuickView` 时，您可以直接使用任何 Qt Quick 元素，如以下代码片段所示：

```
import QtQuick
Item {
 width: 400
 height: 400
 Text {
 anchors.centerIn: parent
 text: "Hello World!"
 }
}
```

6. Next, you can build and run the Qt Quick project by clicking on the green arrow button located at the bottom-left corner of the **integrated development environment (IDE)**, as shown in the following screenshot:

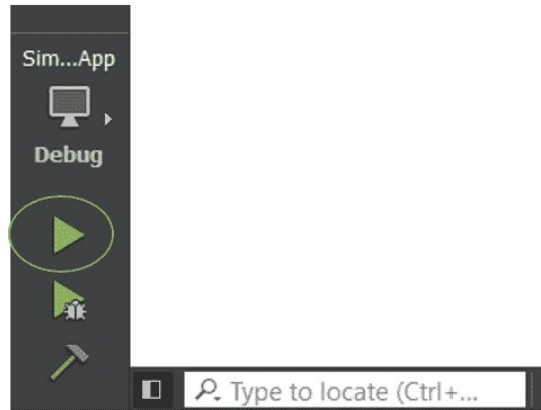


图 4.7-Qt Creator 中的构建和运行选项

7. 现在，点击**Run**按钮构建并运行应用。很快，您将看到一个带有**\*\*Hello World! \*\***的 UI，如以下屏幕截图所示：

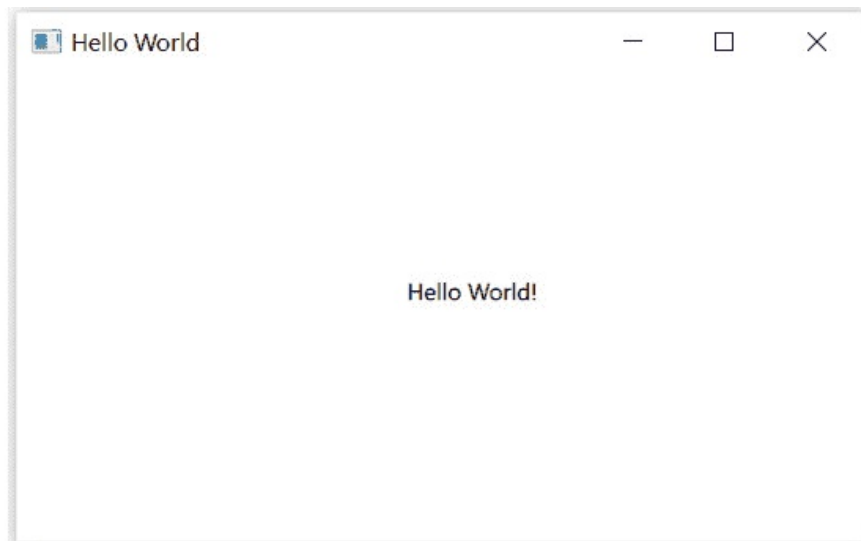


图 4.8-Hello World UI 的输出

您可以在 Windows 上从命令行运行该应用，如下所示：

```
>SimpleQtQuickApp.exe
```

您还可以在 Linux 发行版上从命令行运行该应用，如下所示：

```
$/SimpleQtQuickApp
```

在命令行模式下，如果在应用路径中找不到库，可能会看到几个错误对话框。您可以将 **Qt** 库和插件文件复制到该二进制文件夹来解决该问题。为了避免这些问题，我们将坚持使用 **Qt Creator** 来构建和运行我们的示例程序。您可以转到项目界面并根据您的喜好选择一个工具包，从而在不同的工具包之间进行切换。请记住，您需要在更改 `.pro` 文件后运行 `qmake`。如果您正在使用命令行，则继续执行以下命令：

```
>qmake
>make
```

您还可以使用 **QML** 入口点创建一个 **Qt Quick2UI** 项目，而无需使用任何 **C++** 代码。要使用它，您需要设置一个 **QML** 运行时环境，比如 `qmlscene`。**Qt Creator** 使用 `.qmlproject` 处理仅限 **QML** 的项目：

1. To create a Qt Quick 2 UI project, select **Qt Quick 2 UI Prototype** from the new project template screen, as shown in the following screenshot:

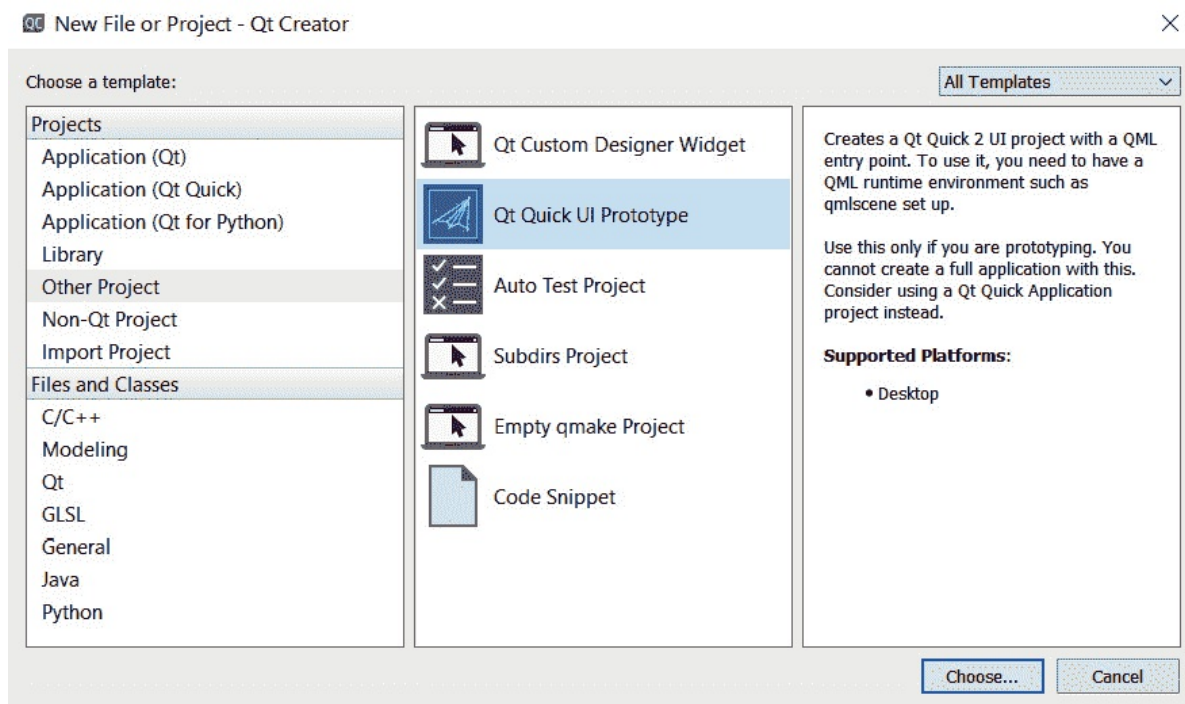


图 4.9-Qt Quick UI Prototype 向导

2. 继续单击下一步按钮，查看项目详细信息、套件选择和项目管理屏幕。这些屏幕与前面讨论的 Qt 快速应用项目相同。单击**Finish**按钮以创建具有骨架的项目。现在，看看 `QtQuickUIPrototype.qmlproject` 和 `QtQuickUIPrototype.qml` Qt Creator 生成的文件的内容。
3. Let's modify the contents of `QtQuickUIPrototype.qml` to add a `Text` element and display `Hello World!`, as illustrated in the following screenshot:

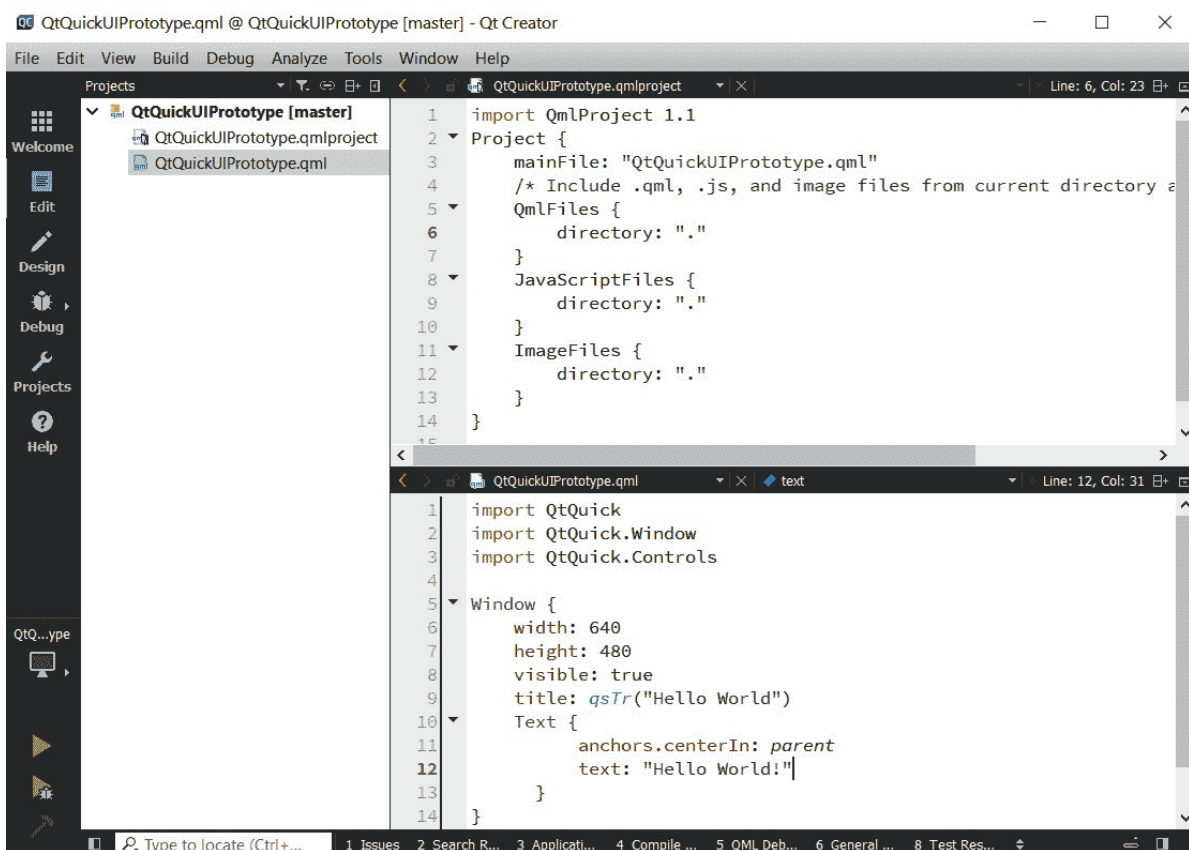


图 4.10-Qt Quick UI 原型项目示例内容

4. 现在，点击**Run**按钮构建并运行应用。很快，您将看到一个带有

**\*\*Hello World! \*\***的 UI。

您还可以从命令行运行该应用，如下所示：

```
>qmlscene QtQuickUIPrototype.qml
```

您可能需要在命令行中提到`qmlscene`和`qml`文件路径。仅当您正在制作原型时才使用此选项。您不能使用此命令创建完整的应用。对于完整的应用，请考虑使用 **Qt Quick** 应用项目。

在本节中，我们学习了如何使用 **Qt Quick** 模块创建一个简单的 **GUI**。在下一节中，我们将学习如何使用 **Qt Quick Designer** UI 设计自定义 UI。

## 使用 **Qt Quick Designer** 设计 UI

---

在本节中，您将学习如何使用 **Qt Quick Designer** 设计您的 UI。与 **Qt** 小部件中的`.ui`文件类似，您还可以在 **QML** 中创建一个 UI 文件。该文件的文件扩展名为`.ui.qml`。**QML** 文件有两种类型：一种扩展名为`.qml`，另一种扩展名为`.ui.qml`。**QML** 引擎将其视为标准的`.qml`文件，但禁止其中的逻辑实现。它为多个`.qml`文件创建可重用 UI 定义。通过 UI 定义和逻辑实现的分离，增强了 **QML** 代码的可维护性。

在开始学习如何设计我们自己的 UI 之前，让我们先熟悉一下 **Qt Quick Designer** 的界面。下面的屏幕截图显示了 **Qt** 快速设计器的不同部分。在设计我们的 UI 时，我们将逐步了解这些部分：

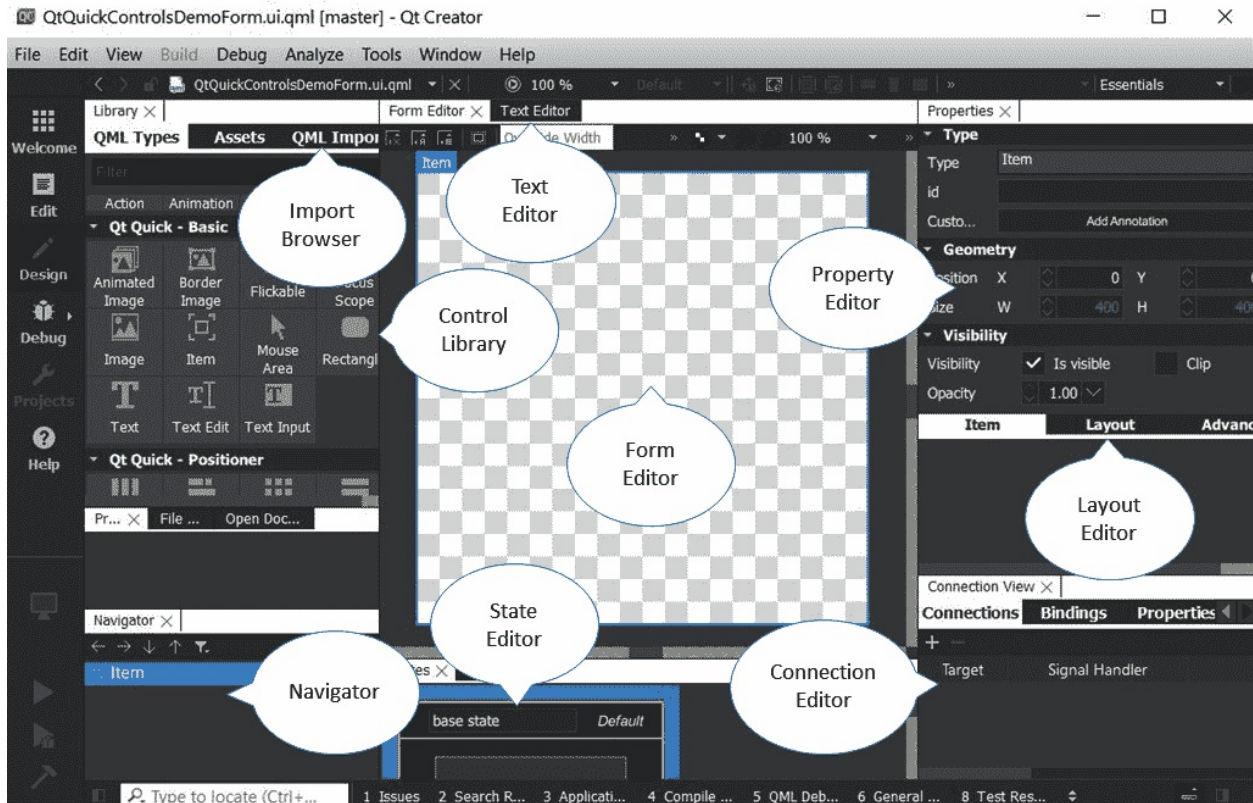


图 4.11-Qt Quick Designer 用户界面的部分

Qt Quick Designer 的 UI 由以下主要部分组成：

- **Navigator:** 它以树形结构列出当前 QML 文件中的项目。它类似于我们在上一章中了解到的 Qt 设计器中的对象运算符窗口。
- **控件库:** 此窗口显示 QML 中所有可用的 Qt 快速控件。您可以将控件拖放到画布窗口以修改您的 UI。
- **Resources:** 这将在列表中显示所有资源，然后可用于 UI 设计。
- **Import Browser:** Import Browser 便于将不同的 QML 模块导入到当前 QML 文件中，为您的 QML 项目添加新功能。您还可以创建自己的自定义 QML 模块并从此处导入。
- **文本编辑器:** 它有六个工具按钮，每个按钮用于特定的操作，如复制和粘贴。

- 属性编辑器：类似于 Qt 设计器的属性编辑器。Qt 快速设计器中的属性部分显示选定项的属性。也可以在文本编辑器中更改项目的属性。
- 表单编辑器：表单编辑器是一个画布，您可以在其中为 Qt Quick 应用设计 UI。
- 状态编辑器：此窗口列出 QML 项目中的不同状态，并描述 UI 定义及其行为。
- 连接编辑器：此部分类似于 Qt Designer 中的信号/插槽编辑器。在这里，您可以定义 QML 组件的信号和槽机制。

您现在已经熟悉了 Qt 快速设计器 UI。让我们创建一个 Qt Quick UI 文件并探索 Qt Quick 控件，如下所示：

1. To create a Qt Quick UI, select **QtQuick UI File** from the **New File** template screen, as shown in the following screenshot. Proceed through the next screens to create a Qt Quick form with a `.ui.qml` file extension. By default, Qt Creator will open up Qt Quick Designer. You can switch to code-editing mode by clicking the **Edit** button on the left-side panel:



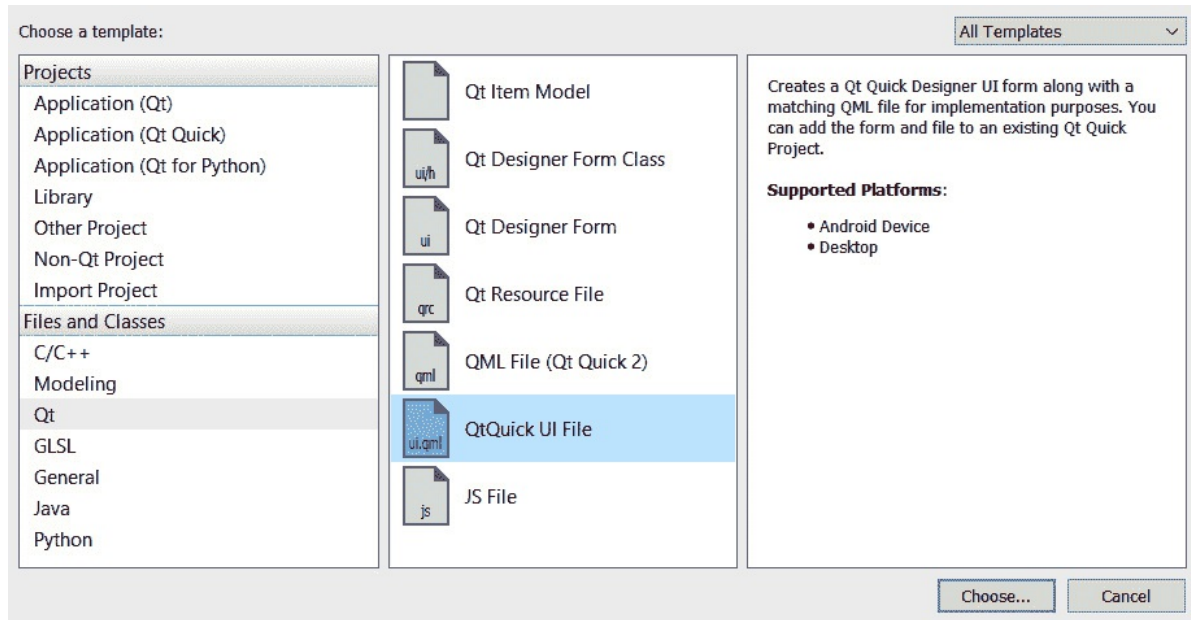


图 4.12-Qt 快速 UI 文件向导

2. Let's add a few QML elements to the **Form Editor** by grabbing a control by a mouse press and dropping it onto the canvas area by a mouse release, as shown in the following screenshot. This action is known as **drag and drop (DnD)**. You can find several basic QML types, such as `Item`, `Rectangle`, `Image`, `Text`, and so on. `Item` is a transparent UI element that can be used as a container:

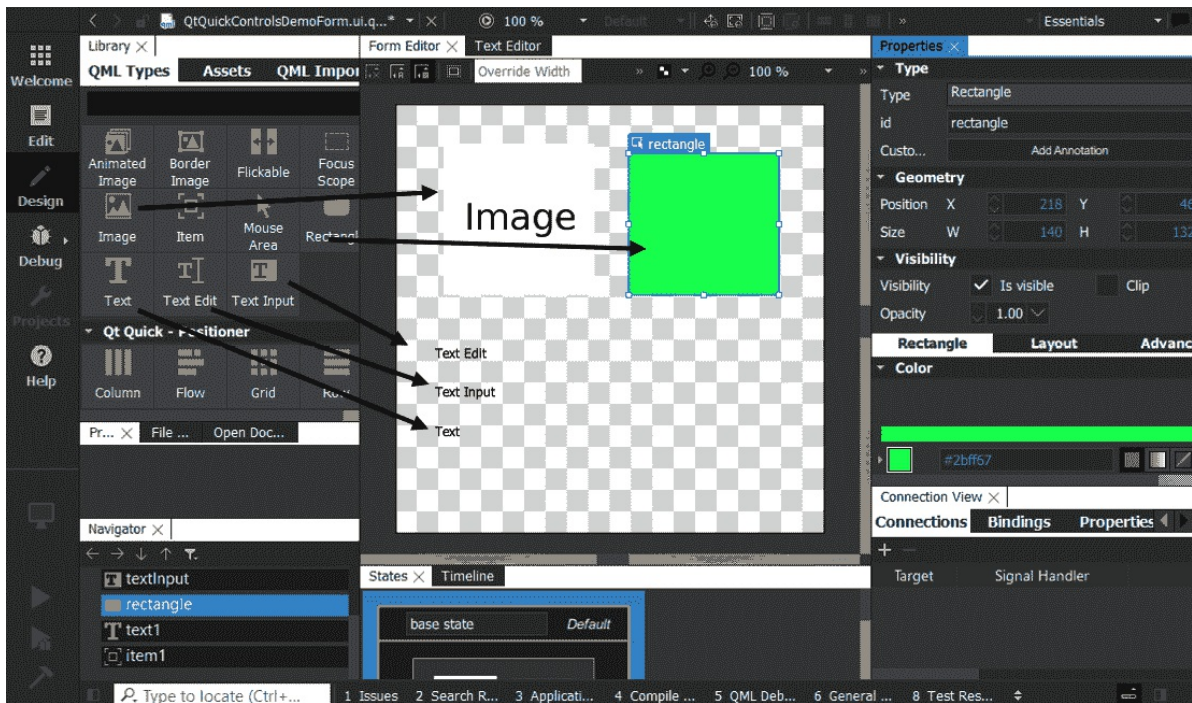


图 4.13-显示基本 QML 类型的 Qt 快速设计器

3. By default, the library contains only a few basic QML types. You can import Qt Quick modules to Qt Quick Designer through the **QML Import Browser**. Let's import any of the `QtQuick.Controls` packages, as shown in the next screenshot:

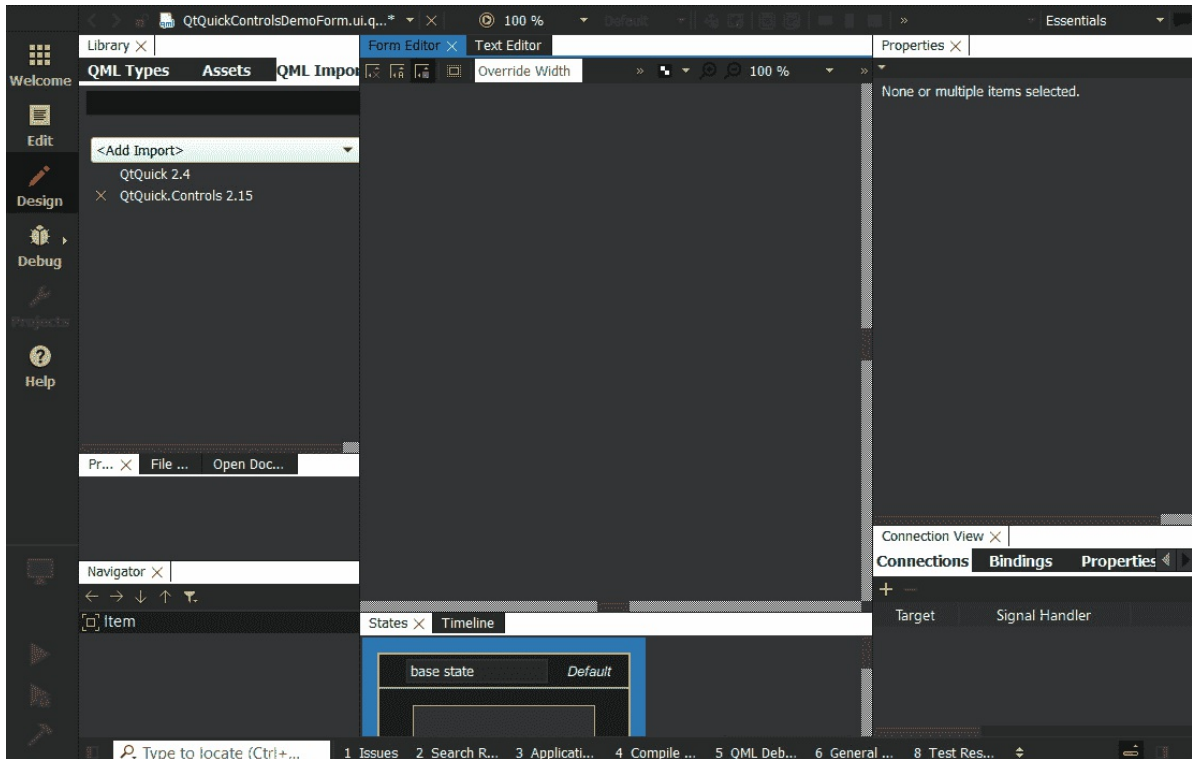


图 4.14-显示 QML 模块导入选项的 Qt 快速设计器

4. 导入模块后，您可以在库中看到带有**Qt Quick-Controls 2**的部分，如以下屏幕截图所示：

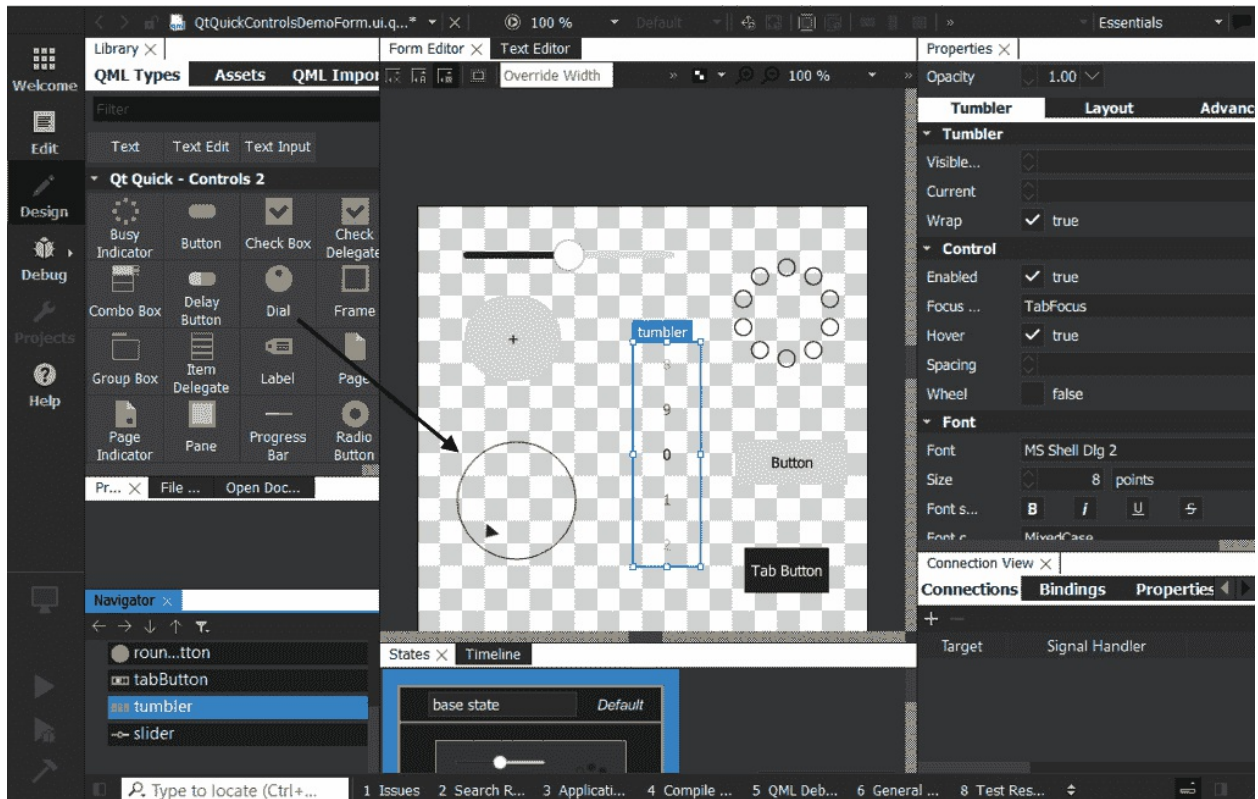


图 4.15-显示 Qt 快速控件的 Qt 快速设计器

在本节中，我们熟悉了 Qt Quick Designer 的界面。在下一节中，您将了解不同的定位器和布局。

## QML 中的定位器和布局

在 QML 中定位项目有种不同的方式。可以通过提及 $x$ 和 $y$ 坐标或使用锚点、定位器或布局来手动定位控件。让我们讨论一下如何通过上述方法定位控件。

### 手动定位

通过设置控件相应的 $x$ 和 $y$ 属性，可以将控件定位在特定的 $x$ 和 $y$ 坐标处。根据可视的坐标系规则，这将使控件相对于其父控件的左上角定位。

下面的代码片段显示如何将 `Rectangle` 项放置在位置 `(50,50)`：

```
import QtQuick
Rectangle {
 // Manually positioned at 50,50
 x: 50 // x position
 y: 50 // y position
 width: 100; height: 80
 color: "blue"
}
```

运行前面的代码时，您将看到在(50,50)位置创建了一个蓝色矩形。更改x和y值，您将看到相对于左上角位置是如何变化的。Qt 允许您在由分号分隔的一行中编写多个属性。您可以在同一行中写入x和y位置，并用分号分隔。

在本节中，您学习了如何通过指定可视化项的坐标来定位它。在下一节中，我们将讨论锚的使用。

## 用锚定位

Qt Quick 提供了一种将一个控件锚定到另一个控件的方法。每个项目有七条不可见锚

线：left、right、top、bottom、baseline、horizontalCenter

和verticalCenter。您可以为每一面设置边距或不同的边距。如果特定项目有多个锚点，则可以对它们进行分组。

让我们来看一下下面的示例：

```
import QtQuick
import QtQuick.Window
Window {
 width: 400; height: 400
 visible: true
 title: qsTr("Anchoring Demo")
 Rectangle {
 id: blueRect
 anchors {
```

```
 left: parent.left; leftMargin:10
 right: parent.right; rightMargin: 40
 top: parent.top; topMargin: 50
 bottom: parent.bottom; bottomMargin: 100
 }
 color: "blue"
 Rectangle {
 id: redRect
 anchors.centerIn: blueRect
 color:"red"
 width: 150; height: 100
 }
}
```

如果运行此示例，您将在输出窗口中看到一个蓝色矩形内有一个红色矩形，该矩形具有不同的边距，如下所示：

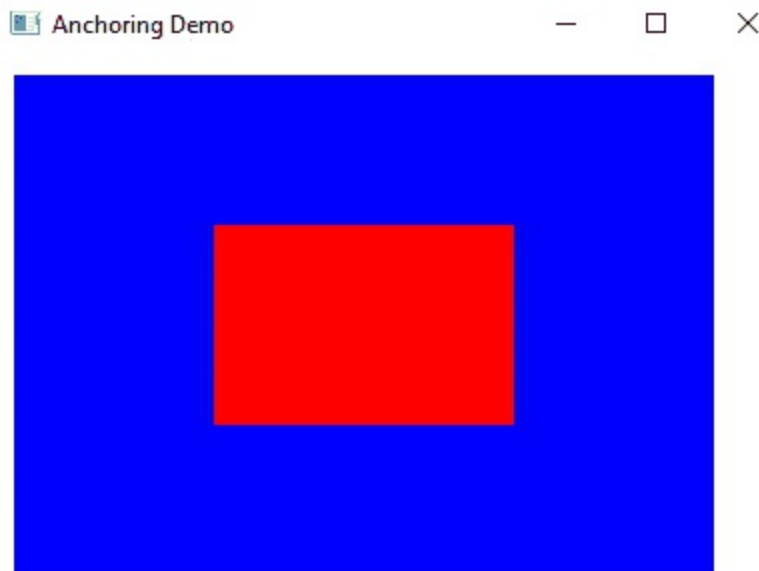


图 4.16-将控件锚定在窗口内

在本节中，您学习了如何使用锚点定位可视项目。在下一节中，我们将讨论定位器的使用。

## 定位器



定位器是容器，它管理声明性 UI 中视觉元素的位置。定位器的行为类似于**Qt** 小部件中的布局管理器。

**Qt Quick** 元素的基本集合中提供了一组标准的定位器。它们概述如下：

- 列将其子级定位在列中。
- 行将其子项放置在一行中。
- 网格将其子项定位在网格中。
- **flow**将其子项定位为页面上的单词。

让我们来看看如何在 **Qt Quick Designer** 中使用它们。首先，创建三个具有不同颜色的矩形项，然后将它们放置在**Row**元素中，如以下屏幕截图所示：

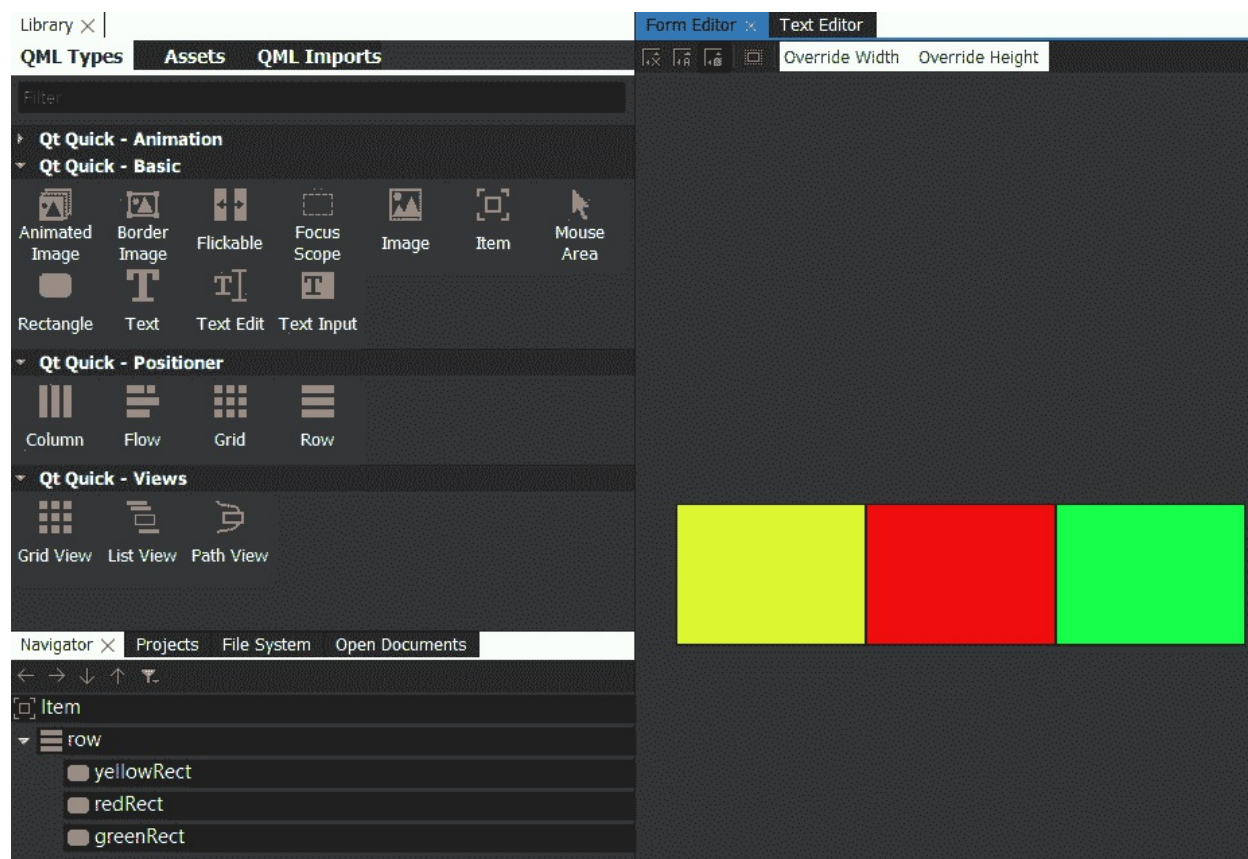




图 4.17-行定位器内部的矩形

您还可以编写代码在定位器中定位控件。如果您使用 **Qt 快速设计器**，**Qt Creator** 会自动生成代码。可以通过表单编辑器旁边的文本编辑器选项卡查看和修改生成的代码。代码如以下代码片断所示：

```
Row {
 id: row
 Rectangle {
 id: yellowRect
 width: 150; height: 100
 color: "yellow"
 border.color: "black"
 }
 Rectangle {
 id: redRect
 width: 150; height: 100
 color: "red"
 border.color: "black"
 }
 Rectangle {
 id: greenRect
 width: 150; height: 100
 color: "green"
 border.color: "black"
 }
}
```

在本节中，我们了解了不同的定位器。在下一节中，我们将讨论中继器和型号以及定位器的使用。

## 中继器

中继器使用提供的模型创建多个可视元素，以及来自模板的元素与定位器一起使用，并使用来自模型的数据。中继器放置在定位器内，并创建遵循定义的定位器排列的视觉元素。当有许多类似的物品时，带有中继器的定位器在按规则布局排列时会更容易维护。

让我们使用 `Repeater` 创建位于一行中的五个矩形，如下所示：

```
import QtQuick
import QtQuick.Window
Window {
 width: 400; height: 200
 visible: true
 title: qsTr("Repeater Demo")
 Row {
 anchors.centerIn: parent
 spacing: 10
 Repeater {
 model: 5
 Rectangle {
 width: 60; height: 40
 border{ width: 1; color: "black";}
 color: "green"
 }
 }
 }
}
```

运行前面的示例时，您将看到五个矩形排成一行，如下所示：



图 4.18-行定位器内部的矩形

在本节中，我们了解了如何将中继器与定位器配合使用。在下一节中，我们将研究 **Qt 快速布局**。

**Qt Quick Layout** 是一组 **QML** 类型，可用于在 **UI** 中排列视觉元素。**Qt Quick Layout** 可以调整其子项的大小，因此它们用于可调整大小的 **UI**。定位器和布局之间的基本区别在于，布局可以在调整窗口大小时调整其子窗口的大小。

可以使用以下 `import` 语句将 Qt 快速版面导入到 QML 文件中：

```
import QtQuick.Layouts
```

QML 中有五种不同的类型的布局，如下所示：

- `RowLayout`：这会将元素排成一行。它类似于 `GridLayout`，但只有一行。
- `ColumnLayout`：这会在列中排列元素。它类似于 `GridLayout`，但只有一列。
- `GridLayout`：这允许在网格中动态排列元素。
- `Layout`：这为推送到 `ColumnLayout`、`RowLayout` 或 `GridLayout` 布局类型上的项提供附加属性。
- `StackLayout`：这会以类似堆栈的方式排列元素，一次只有一个元素可见。

让我们看一下下面的 `RowLayout` 示例：

```
import QtQuick
import QtQuick.Window
import QtQuick.Layouts
Window {
 width: 640; height: 480
 visible: true
 title: qsTr("Layout Demo")
 RowLayout {
 id: layout
 anchors.fill: parent
 spacing: 6
 Rectangle {
 color: 'yellow'
 Layout.fillWidth: true
 Layout.minimumWidth: 50
 Layout.preferredWidth: 150
 Layout.maximumWidth: 200
 Layout.minimumHeight: 100
 Layout.margins: 10
 }
 }
}
```

```
 }
 Rectangle {
 color: 'red'
 Layout.fillWidth: true
 Layout.minimumWidth: 50
 Layout.preferredWidth: 100
 Layout.preferredHeight: 80
 Layout.margins: 10
 }
}
}
```

请注意，`Row`类型是定位器，而`RowLayout`类型是布局。像往常一样，什么时候使用它们主要取决于你的目标。让我们进入下一节，看看如何将 QML 与 C++ 集成。

## QML 与 C++ 的集成

---

QML 应用通常需要在 C++ 中处理更高级的性能密集型任务。要做到这一点，最常见、最快捷的方法是将 C++ 类公开给 QML 运行时，前提是 C++ 实现派生自 `QObject`。

QML 可以很容易地与 C++ 代码集成。可以从 C++ 加载和操作 QML 对象。QML 与 Qt 的元对象系统的集成允许从 QML 调用 C++ 功能。这有助于构建混合使用 C++、QML 和 JS 的混合应用。要向 QML 公开 C++ 数据或属性或方法，应该从 `QObject` 类派生。这是可能的，因为所有 QML 对象类型都是使用 `QObject` 派生类实现的，从而允许 QML 引擎通过 Qt 元对象系统加载和检查对象。

您可以通过以下方式将 QML 与 C++ 集成：

- 利用上下文属性将 C++ 对象嵌入到 QML 中
- 向 QML 引擎注册类型
- 创建 QML 扩展插件

在接下来的几节中，让我们逐一讨论每种方法。

## 重要音符

要快速确定哪种集成方法适合您的项目，请查看位于以下链接的 Qt 文档中所示的流程图：

<https://doc.qt.io/qt-6/qtqml-cppintegration-overview.html>

## 将 **C++** 对象嵌入到具有上下文属性的 **QML** 中

您可以使用上下文属性将 C++ 对象公开到 QML 环境中。上下文属性适用于简单的应用。它们会将您的对象导出为全局对象。上下文在被 QML 引擎实例化之后向 QML 环境公开。

让我们看一下下面的示例，其中我们已经将 `radius` 导出到 QML 环境。您还可以用类似的方式导出 C++ 模型：

```
#include <QGuiApplication>
#include <QQmlApplicationEngine>
#include <QQmlContext>
int main(int argc, char *argv[])
{
 QGuiApplication app(argc, argv);
 QQmlApplicationEngine engine;
 engine.rootContext()->setContextProperty("radius", 50);
 const QUrl url(QStringLiteral("qrc:/main.qml"));
 engine.load(url);
 return app.exec();
}
```

您可以直接在 QML 文件中使用导出值，如下所示：

```
import QtQuick
import QtQuick.Window
Window {
 width: 640; height: 480
 visible: true
}
```

```
 title: qsTr("QML CPP integration")
 Text {
 anchors.centerIn: parent
 text: "C++ Context Property Value: "+ radius
 }
}
```

您还可以注册 **C++** 类并在 **QML** 环境中实例化它。让我们在下一节中学习如何实现这一点。

## 向 **QML** 引擎注册 **C++** 类

注册 **QML** 类型允许开发人员从 **QML** 环境控制 **C++** 对象的生命周期。这不能通过 **Context** 属性实现，也不会填充全局名称空间。尽管如此，所有类型都需要首先注册，因此，所有库都需要在应用启动时链接，这在大多数情况下都不是问题。

这些方法可以是公共槽，也可以是标记有 `Q_INVOKABLE` 的公共方法。现在，让我们将 **C++** 类导入到 **QML** 文件中。看一下下面的 **C++** 类：

```
#ifndef BACKENDLOGIC_H
#define BACKENDLOGIC_H
#include <QObject>
class BackendLogic : public QObject
{
 Q_OBJECT
public:
 explicit BackendLogic(QObject *parent = nullptr) {
 Q_UNUSED(parent);}
 Q_INVOKABLE int getData() {return mValue; }
private:
 int mValue = 100;
};
#endif // BACKENDLOGIC_H
```

您需要使用 `qmlRegisterType()` 将 `main.cpp` 文件中的 **C++** 类注册为模块，如下所示：

```
qmlRegisterType<BackendLogic>("backend.logic", 1, 0, "BackendLogic")
```

任何 `QObject` 派生的 C++ 类都可以注册为 QML 对象类型。一旦类注册到 QML 类型系统，就可以像使用任何其他 QML 类型一样使用该类。现在，C++ 类已经准备好在 `.qml` 文件中实例化了。您必须导入模块并创建一个对象，如以下代码片段所示：

```
import QtQuick
import QtQuick.Window
import backend.logic
Window {
 width: 640; height: 480
 visible: true
 title: qsTr("QML CPP integration")
 BackendLogic {
 id: backend
 }
 Text {
 anchors.centerIn: parent
 text: "From Backend Logic : "+ backend.getData()
 }
}
```

当您运行前面的程序时，您可以看到该程序正在从后端 C++ 类获取数据并将其显示在 UI 中。

您还可以使用 `qmlRegisterSingletonType()` 将 C++ 类公开为 QML 单例。通过使用 QML 单例，可以防止全局命名空间中出现重复对象。我们跳过这一部分，因为它需要了解设计模式。有关详细文档，请访问以下链接：

<https://doc.qt.io/qt-6/qqmlengine.html#qmlRegisterSingletonType>

在 Qt6 中，可以通过使用 `QML_ELEMENT` 宏来实现 C++ 集成。此宏将封闭类型声明为在 QML 中可用，并使用其类或命名空间名称作为 QML 元



素名称。要在 C++ 头文件中使用此宏，必须将 `qml.h` 头文件包含为 `#include <QtQml>`。

让我们来看看下面的例子：

```
#ifndef USINGELEMENT_H
#define USINGELEMENT_H
#include <QObject>
#include <QtQml>
class UsingElements : public QObject
{
 Q_OBJECT
 QML_ELEMENT
public:
 explicit UsingElements(QObject *parent = nullptr) {
 Q_UNUSED(parent);}
 Q_INVOKABLE int readValue() {return mValue; }
private:
 int mValue = 500;
};
#endif // USINGELEMENT_H
```

在 `.pro` 文件中，您必须将 `qmltypes` 选项添加到 `CONFIG` 变量中，并且要提到 `QML_IMPORT_NAME` 和 `QML_IMPORT_MAJOR_VERSION`，如以下代码片段所示：

```
CONFIG += qmltypes
QML_IMPORT_NAME = backend.element
QML_IMPORT_MAJOR_VERSION = 1
```

现在，您的 C++ 类已经准备好在 `.qml` 文件中实例化了。您必须导入模块并创建一个对象，如以下代码片段所示：

```
import QtQuick
import QtQuick.Window
import backend.element
Window {
 width: 640; height: 480
 visible: true
```

```
title: qsTr("QML CPP integration")
UsingElements {
 id: backendElement
}
Text {
 anchors.centerIn: parent
 text: "From Backend Element : "+
 backendElement.readValue()
}
}
```

在本节中，您了解了如何将 **C++** 类导出到 **QML** 环境中，并从 **QML** 访问其函数。在本例中，只有在调用该方法时才检索数据。您还可以通过添加带有 **NOTIFY** 信号的 **Q\_PROPERTY()** 宏，在 **C++** 中更改数据时得到通知。在使用之前，您需要了解信号和插槽机制。因此，我们将跳过这一部分，在 [第 6 章](#)、信号和插槽中进一步讨论。在下一节中，我们将讨论如何创建 **QML** 扩展插件。

## 创建 **QML** 扩展插件

**QML** 扩展插件提供了与 **C++** 集成的最灵活方式。它允许您在第一个 **QML** 文件调用导入标识符时加载的插件中注册类型。您可以跨项目使用插件，这在构建复杂项目时非常方便。

**Qt Creator** 有一个创建 **Qt Quick 2 QML** 扩展插件的向导。选择一个模板，如下面的屏幕截图所示，然后继续下面的屏幕。该向导将为 **QML** 扩展插件项目创建一个基本框架。插件类必须从 **QqmlExtensionPlugin** 派生，并且应该实现 **registerTypes()** 函数。需要使用 **Q\_PLUGIN\_METADATA** 宏将插件标识为 **QML** 扩展插件：

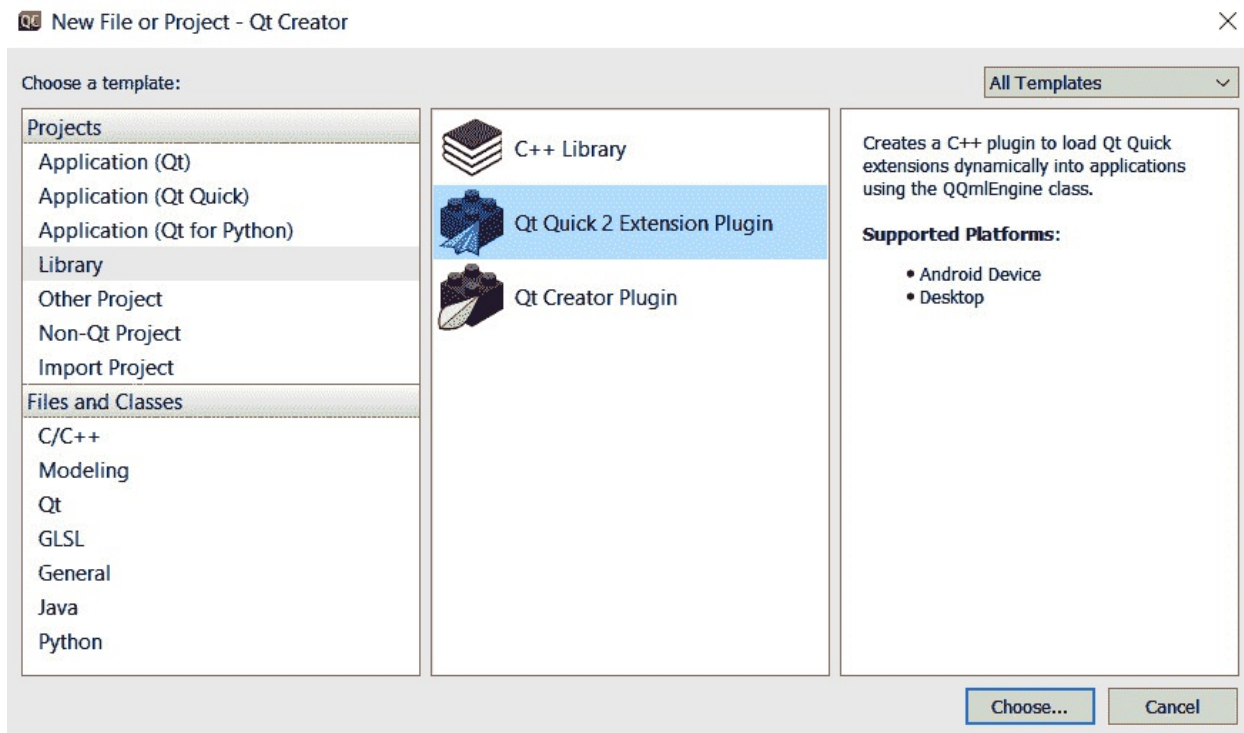


图 4.19-Qt Quick 2 QML 扩展插件向导

本节是高级 QT 主题。您需要深入了解 Qt 才能创建您自己的 QML 扩展插件。如果您是初学者，您可以跳过本部分，但您可以在以下链接的 Qt 文档中了解有关 QML 扩展插件的更多信息：

<https://doc.qt.io/qt-6/qtqml-modules-cppplugins.html>

让我们继续下一节，了解如何在 C++ 类中调用 QML 方法。

## 调用 C++ 类内部的 QML 方法

所有 QML 方法都向元对象系统公开，并且可以使用 `QMetaObject::invokeMethod()` 从 C++ 调用。您可以指定参数的类型和冒号字符后的返回值，如下面的代码片段所示。例如，当您希望将具有特定签名的 C++ 中的信号连接到 QML 定义的方法时，这会很有用。如果省略类型，C++ 签名将使用 `QVariant`。

让我们看一个使用 `QMetaObject::invokeMethod()` 调用 QML 方法的应用。

在 QML 文件中，我们添加一个名为 `qmlMethod()` 的方法，如下所示：

```
import QtQuick
Item {
 function qmlMethod(msg: string) : string {
 console.log("Received message:", msg)
 return "Success"
 }
 Component.onCompleted: {
 console.log("Component created successfully.")
 }
}
```

在 `main.cpp` 文件中，按照以下代码片段调用 `QMetaObject::invokeMethod()`：

```
#include <QGuiApplication>
#include <QQmlApplicationEngine>
#include <QQmlComponent>
int main(int argc, char *argv[])
{
 QGuiApplication app(argc, argv);
 QQmlApplicationEngine engine;
 QQmlComponent component(&engine,
 "qrc:/CustomItem.qml");
 QObject *myObject = component.create();
 QString retValue = "";
 QString msg = "Message from C++";
 QMetaObject::invokeMethod(myObject, "qmlMethod",
 Q_RETURN_ARG(QString,
 retValue),
 Q_ARG(QString, msg));
 qDebug() << "QML method returned:" << retValue;
 delete myObject;
 return app.exec();
}
```

请注意，必须指定参数和返回类型。基本类型和对象类型都可以作为类型名称。如果 QML 类型系统中没有提到该类型，则在调用 `QMetaObject::invokeMethod` 时必须将 `QVariant` 声明为具有 `Q_RETURN_ARG()` 和 `Q_ARG()` 的类型。或者，如果您不需要任何返回值，也可以只使用两个参数调用 `invokeMethod()`，如下所示：

```
QMetaObject::invokeMethod(myObject, "qmlMethod");
```

在本节中，您学习了从 QML 方法接收数据。在下一节中，您将学习如何在 C++ 中访问 QML 对象指针。

## 向 C++ 公开 QML 对象指针

有时，您可能希望通过 C++ 修改 QML 对象的属性，例如修改控件的文本、更改控件的可见性或更改自定义属性。QML 引擎允许您将 QML 对象注册为 C++ 类型，这会自动公开 QML 对象的属性。

让我们看一下下面的示例，在该示例中，我们已经将一个 QML 对象导出到 C++ 环境：

```
#ifndef CUSTOMOBJECT_H
#define CUSTOMOBJECT_H
#include <QObject>
#include <QVariant>
class CustomObject : public QObject
{
 Q_OBJECT
public:
 explicit CustomObject(QObject *parent = nullptr);
 Q_INVOKABLE void setObject(QObject* object)
 {
 object->setProperty("text", QVariant("Clicked!"));
 }
};
#endif // CUSTOMOBJECT_H
```

在 QML 文件中，您需要创建 `C++` 类的实例并调用 `C++` 方法。正如您在下面的代码片段中看到的那样，该属性是在 `C++` 类内部操作的：

```
import QtQuick
import QtQuick.Window
import QtQuick.Controls
import MyCustomObject
Window {
 width: 640; height: 480;
 visible: true
 title: qsTr("QML Object in C++")
 CustomObject{
 id: customObject
 }
 Button {
 id: button
 anchors.centerIn: parent
 text: qsTr("Click Me!")
 onClicked: {
 customObject.setObject(button);
 }
 }
}
```

## 重要音符

Qt QML 模块提供了几个用于注册不可实例化类型的宏。

`QML_ANONYMOUS` 注册不可实例化且无法从 QML 引用的 `C++` 类型。

`QML_INTERFACE` 注册现有的 Qt 接口类型。该类型不能从 QML 实例化，并且不能用它声明 QML 属性。`QML_UNCREATABLE` 注册一个命名的 `C++` 类型，该类型不可实例化，但应可识别为 QML 类型系统的类型。

`QML_SINGLETON` 注册可以从 QML 导入的单例类型。

祝贺你！您已经了解了如何集成 QML 和 `C++`。在下一节中，我们将讨论如何将 JS 与 QML 一起使用。

# QML 与 JS 的集成

QML 与 JS 有很好的集成，并使用 **JavaScript** 对象符号(**JSON**)-，类似于语法，允许将表达式和方法定义为 JS 函数。它还允许开发人员导入 JS 文件并使用现有功能。QML 引擎提供的 JS 环境与 Web 浏览器提供的 JS 环境相比有一些限制。Qt Quick 应用的逻辑可以在 JS 中定义。JS 代码可以内联写在 QML 文件中，也可以写在单独的 JS 文件中。

让我们看看如何在 QML 文档中使用内联 JS。下面的示例演示了 `btnClicked()` 内联 JS 函数。单击 `Button` 控件时将调用该方法：

```
import QtQuick
import QtQuick.Window
import QtQuick.Controls
Window {
 width: 640; height: 480;
 visible: true
 title: qsTr("QML JS integration")
 function btnClicked(controlName) {
 controlName.text = "JS called!"
 }
 Column {
 anchors.centerIn: parent
 Button {
 text: "Call JS!"
 onClicked: btnClicked(displayText)
 }
 Text {
 id: displayText
 }
 }
}
```

前面的示例展示了如何将 JS 代码与 QML 集成。我们使用了 `btnClicked()` 内联 JS 函数。当您运行应用时，您将收到一条消息，提示 **JS** 已调用！。



如果您的逻辑非常长，或者在多个 QML 文档中有用处，那么可以使用单独的 JS 文件。您可以按如下方式导入 JS 文件：

```
import "<JavaScriptFile>" as <Identifier>
```

例如，您可以运行以下代码行：

```
import "constants.js" as Constants
```

在前面的示例中，我们将 `constants.js` 导入到 QML 环境中。 `Constants` 是我们的 JS 文件的标识符。

您还可以创建一个共享的 JS 库。您只需在 JS 文件的开头包含以下代码行：

```
.pragma library
```

## 重要音符

如果脚本是单个表达式，则建议将其内联编写。如果脚本只有几行长，则使用块。如果脚本超过几行，或者不同的对象需要该脚本，则创建一个函数并根据需要调用它。对于长脚本，创建一个 JS 文件并将其导入到 QML 文件中。避免使用 `Qt.include()`，因为它已被弃用，并将从未来的 Qt 版本中删除。

要了解更多关于在 QML 中导入 JS 的信息，请阅读以下文档：

<https://doc.qt.io/qt-6/qtqml-javascript-imports.html>

在本节中，您了解了如何将 JS 与 QML 集成。在下一节中，我们将讨论如何在 QML 中导入目录。

## 在 QML 中导入目录

您可以在另一个 QML 文件中直接导入包含 QML 文件的本地目录，而无需添加资源。您可以使用目录的绝对或相对文件系统路径来完成此操作，从而为将 QML 类型安排为文件系统上的可重用目录提供了一种方便的方法。

目录导入的常见形式如下所示：

```
import "<DirectoryPath>" [as <Qualifier>]
```

例如，如果您的目录名是 `customqmlelements`，则可以按如下方式导入：

```
import "../customqmlelements"
```

还可以将目录作为限定的本地命名空间导入，如以下代码片段所示：

```
import "../customqmlelements" as CustomQMLElements
```

您还可以从资源路径导入文件，如下所示：

```
import "qrc:/qml/customqmlelements"
```

您还可以从远程服务器导入 QML 文件的目录。有两种不同类型的 `qmldir` 文件：QML 目录列表文件和 QML 模块定义文件。这里，我们讨论 `qmldir` QML 目录列表文件的使用。可以使用远程位置的统一资源定位符(**URL**)导入目录。请注意，通过网络导入时，只有 `qmldir` 文件中指定的 QML 和 JS 文件可以访问。为了避免恶意代码，您必须小心处理网络文件。

以下文档提供了有关 `qmldir` QML 目录列表文件的详细信息：

<https://doc.qt.io/qt-6/qtqml-syntax-directoryimports.html>

您可以通过以下链接了解有关不同类型的 `qmldir` 文件的更多信息：

<https://doc.qt.io/qt-6/qtqml-modules-qmlDir.html>

在本节中，您学习了如何在 QML 中导入目录。在下一节中，我们将讨论如何在 QML 中处理鼠标和触摸事件。

## 处理鼠标和触摸事件

QML 通过允许 QML 应用处理鼠标和触摸事件的输入处理程序为鼠标和触摸事件提供出色的支持。QML 类型

(如 `MouseArea`、`MultiPointTouchArea` 和 `TapHandler`) 用于检测鼠标和触摸事件。在下一节中，我们将来看看这些 QML 类型。

### 鼠标区域

`MouseArea` 是不可见项目，它与可见项目(如 `Item` 或 `Rectangle` 一起使用，以便为该项目提供鼠标和触摸处理事件。`MouseArea` 接收 `Item` 定义区域内的鼠标事件。您可以通过使用 `anchors.fill` 属性将 `MouseArea` 锚定到其父区域来定义此区域。如果将 `Visible` 属性设置为 `false`，则鼠标区域对鼠标事件变得透明。

让我们看看如何在下面的示例中使用 `MouseArea`：

```
import QtQuick
import QtQuick.Window
Window {
 width: 640; height: 480
 visible: true
 title: qsTr("Mouse Area Demo")
 Rectangle {
 anchors.centerIn: parent
 width: 100; height: 100
 color: "green"
 MouseArea {
 anchors.fill: parent
 onClicked: { parent.color = 'red' }
```

```
 }
 }
}
```

在前面的示例中，您可以看到只有 `rectangle` 区域接收到鼠标事件。`Windows` 的其他部分没有获得鼠标事件。您可以根据鼠标事件执行相应的操作。`MouseArea` 还提供方便的信号，为我们提供有关鼠标事件(如鼠标悬停、鼠标按下、按住、鼠标退出和鼠标释放事件)的信息。编写相应的信号处理程序，并使用 `entered()`、`exited()`、`pressed()` 和 `released()` 信号进行实验。您还可以检测到按下了哪个鼠标按钮，并执行相应的操作。

## 多点触控区域

`MultiPointTouchArea` QML 类型允许处理多点触摸屏中的多个触摸点。就像 `MouseArea` 一样，`MultiPointTouchArea` 是一个看不见的东西。您可以跟踪多个触摸点并相应地处理手势。当它被禁用时，触摸区域对触摸和鼠标事件都变得透明。在 `MultiPointTouchArea` 类型中，鼠标事件作为单个触摸点处理。可以将 `mouseEnabled` 属性设置为 `false` 以停止处理鼠标事件。

让我们看一下下面的示例，其中我们有两个矩形跟在我们的触摸点后面：

```
import QtQuick
import QtQuick.Window
Window {
 width: 640; height: 480
 visible: true
 title: qsTr("Multitouch Example")
 MultiPointTouchArea {
 anchors.fill: parent
 touchPoints: [
 TouchPoint { id: tp1 },
```

```

 TouchPoint { id: tp2 }
]
}
Rectangle {
 width: 100; height: 100
 color: "blue"
 x: tp1.x; y: tp1.y
}
Rectangle {
 width: 100; height: 100
 color: "red"
 x: tp2.x; y: tp2.y
}
}

```

在 `MultiPointTouchArea` 类型中，`TouchPoint` 定义触摸点。它包含有关触点的详细信息，如压力、当前位置和区域。现在，在您的移动设备上运行该应用并进行验证！

在本节中，您了解了如何使用 `MouseArea` 和 `MultiPointTouchArea` 处理鼠标和触摸事件。让我们在下一节中了解 `TapHandler`。

## TapHandler

`TapHandler` 是处理鼠标上的单击事件和触摸屏上的点击事件的处理程序。您可以使用 `TapHandler` 来响应点击和触摸手势，并且它允许您同时处理多个嵌套项中的事件。有效轻击手势的识别取决于 `gesturePolicy`。`gesturePolicy` 的默认值为 `TapHandler.DragThreshold`，其事件点不得显著移动。如果 `gesturePolicy` 设置为 `TapHandler.WithinBounds`，则 `TapHandler` 对新闻事件进行独占抓取，但在事件点离开父项目边界后立即释放抓取。类似地，如果将 `gesturePolicy` 设置为 `TapHandler.ReleaseWithinBounds`，则 `TapHandler` 独占抓取压力机，并将其保留到释放，以便检测此手势。

让我们创建一个可以识别不同的鼠标按键事件和手写笔轻击

的 `TapHandler` 类型，如下所示：

```
import QtQuick
import QtQuick.Window
Window {
 width: 640; height: 480
 visible: true
 title: qsTr("Hello World")
 Item {
 anchors.fill: parent
 TapHandler {
 acceptedButtons: Qt.LeftButton
 onPressed: console.log("Left Button Clicked!")
 }
 TapHandler {
 acceptedButtons: Qt.MiddleButton
 onPressed: console.log("Middle Button Clicked!")
 }
 TapHandler {
 acceptedButtons: Qt.RightButton
 onPressed: console.log("Right Button Clicked!")
 }
 TapHandler {
 acceptedDevices: PointerDevice.Stylus
 onPressed: console.log("Stylus Tap!")
 }
 }
}
```

您可以使用输入处理程序来处理触摸事件和手势，以替代 `MouseArea`。输入处理程序使复杂触摸交互的形成变得更简单，这是使用 `MouseArea` 或 `TouchArea` 都难以实现的。

Qt 提供了一些现成的控件来处理一般手势，如捏、轻弹和轻扫。

`PinchArea` 是一种方便的 QML 类型，可以处理简单的捏手势。它是一种不可见的物品，与另一个可见物品一起使用。 `Flickable` 是另一种方便的 QML 类型，它为轻击手势提供了一个界面。浏览相关文档和示例以了解有关这些 QML 元素的更多信息。

让我们看看下一节中的 `SwipeView`。

## SwipeView

滑动是触摸式设备中另一个常见的手势。您可以使用 `SwipeView` 侧向滑动来导航页面。它使用基于滑动的导航模型，并提供了一种简化的水平分页滚动方式。您可以在底部添加页面指示器，以显示当前活动页面。

让我们看一个简单的例子，如下所示：

```
import QtQuick
import QtQuick.Window
import QtQuick.Controls
Window {
 width: 640; height: 480
 visible: true
 title: qsTr("Swipe Demo")
 SwipeView {
 id: swipeView
 currentIndex: 0
 anchors.fill: parent
 Rectangle { id: page1; color: "red" }
 Rectangle { id: page2; color: "green" }
 Rectangle { id: page3; color: "blue" }
 }
 PageIndicator {
 id: pageIndicator
 count: swipeView.count
 currentIndex: swipeView.currentIndex
 anchors {
 bottom: swipeView.bottom
 horizontalCenter: parent.horizontalCenter
 }
 }
}
```

如您所见，我们只需将子项添加到 `SwipeView` 即可。您可以将 `SwipeView` 当前索引设置为 `PageIndicator` 当前索引。 `SwipeView` 是导航模型之一，



也包括 `StackView` 和 `Drawer`。您可以探索这些 QML 类型来体验移动设备上的手势。

在本节中，您了解了如何使用各种 QML 类型来处理鼠标、触摸和手势事件。接下来，我们将总结一下我们在本章中学到的内容。

## 摘要

---

本章解释了 Qt Quick 模块的基本原理以及如何创建自定义 UI。您学习了如何使用 Qt Quick Designer 设计和构建 GUI，并了解了 Qt Quick 控件，以及如何构建自定义 Qt Quick 应用。您还学习了如何将 QML 与 C++ 和 JS 集成。现在，您应该了解 Qt 小部件和 Qt Quick 之间的异同，并能够为您的项目选择最合适的框架。在本章中，我们了解了 Qt Quick 以及如何使用 QML 创建应用。您还学习了如何将 QML 与 JS 集成，并了解了鼠标和触摸事件。

在下一章中，我们将讨论使用 Qt Creator 进行跨平台开发。您将学习在 Windows、Linux、Android 和 MacOS 操作系统(OS)上配置和构建应用。我们将学习如何将我们的 Qt 应用移植到不同的平台，而不会遇到太多挑战。我们走吧！

## 第二部分：跨平台开发

---

原文：<https://gitee.com/apacheecn/apacheecn-c-cpp-zh-pt2/blob/master/docs/cross-plat-dev-qt6-mod-cpp/sec2.md>

本节将向您介绍跨平台开发。跨平台开发的理念是，一个软件应用应该在多个平台上很好地工作，而不需要对代码进行重大更改。这节省了移植和维护代码库的时间。这遵循了 Qt 的理念：“少代码，多创建，到处部署”。在本节中，您将了解 Qt Creator IDE，它的用法，以及如何不同的平台上开发和运行相同的应用。

本节包括以下章节：

- [第 5 章](#)，跨平台开发

## 五、跨平台开发

---

原文: <https://gitee.com/apacheecn/apacheecn-c-cpp-zh-pt2/blob/master/docs/cross-plat-dev-qt6-mod-cpp/05.md>

Qt 自最初发布以来就因其跨平台能力而广为人知--它是创建该框架的主要愿景。您可以在您喜欢的桌面平台(如 Windows、Linux 和 MacOS)上使用 Qt Creator, 创建流畅、现代、触摸友好的图形用户界面(GUI)和桌面、移动或嵌入式应用, 使用相同的代码库或稍加修改。您可以轻松地修改代码并将其部署到目标平台上。Qt 有几个内置工具来分析您的应用及其在各种支持平台上的性能。此外, 与其他跨平台框架不同, 它易于使用, 并且有一个直观的用户界面(UI)。

在本章中, 您将学习跨平台开发要点以及如何在不同平台上构建应用。有了它, 您将能够在您最喜欢的桌面和移动平台上运行示例应用。

在本章中, 我们将介绍以下主要主题:

- 了解跨平台开发
- 了解编译器
- 使用 `qmake` 构建
- Qt 项目(`.pro`)文件
- 了解生成设置
- 特定于平台的设置
- 在 Microsoft Visual Studio 中使用 Qt
- 在 Linux 上运行 Qt 应用
- 在 MacOS 和 iOS 上运行 Qt 应用
- 其他 Qt 支持的平台
- 从 Qt5 移植到 Qt6

在本章结束时，您将了解 Qt 项目文件、基本设置以及如何在移动设备上运行 Qt 应用。我们开始吧！

## 技术要求

---

本章的技术要求包括在 Windows 10、Ubuntu 20.04 或 MacOS 10.14 等最新桌面平台上安装 Qt 6.0.0 和 Qt Creator 4.14.0 的最低版本。

本章使用的所有代码都可以从以下 GitHub 链接下载：

<https://github.com/PacktPublishing/Cross-Platform-Development-with-Qt-6-and-Modern-Cpp/tree/master/Chapter05/HelloWorld>

### 重要音符

本章中使用的截图是在 Windows 平台上拍摄的。根据计算机中的底层平台，您将看到类似的屏幕。

## 了解跨平台开发

---

市场上有几个跨平台的框架，但是 Qt 是一个更好的选择，因为它的成熟和可用的社区支持。传统的 C++ 开发人员很容易更快地适应 Qt 并开发高质量的应用。Qt 框架允许开发与多种平台(如 Windows、Linux、MacOS、**QNX**(最初称为**Quick Unix[Qunix]**)、iOS 和 Android)兼容的应用。凭借其一次编码的能力和随时随地部署的理念，它以更好的代码质量促进了更快的应用开发。Qt 在内部处理特定于平台的实现，还使您能够在微控制器驱动的设备上使用令人印象深刻的 GUI 构建令人惊叹的超轻量级应用。

要在嵌入式平台上使用 Qt 开发应用，您需要商业许可证才能使用 Qt 创建设备。Qt 还支持一些微控制器单元(MCU)平台，如 Renesas、

STM32 和 NXP。在写这本书的时候，Qt for MCU 1.8 已经发布，它提供了内存占用量很小的超轻量级模块。

下面列出了使用 Qt 框架进行跨平台开发的一些优势：

- 降低开发成本的成本效益
- 更好的代码可重用性
- 方便 / 便利设施 / 便利 / 厕所
- 更快的上市时间(TTM)
- 更广泛的市场覆盖范围
- 提供近乎原生的体验
- 高性能

也有一些缺点，例如：

- 平台特定功能不可用，无法访问所有平台应用编程接口(API)
- 本机组件和非本机组件之间的通信挑战
- 某些特定于设备的功能和硬件兼容性挑战
- 延迟的平台更新

在本节中，您对 Qt 的跨平台特性有了一个基本的了解，并了解了跨平台开发的优缺点。在任何平台上运行应用之前，您需要使用编译器为目标平台编译应用。在下一节中，我们将了解 Qt 框架支持的编译器。

## 了解编译器

---

在本节中，您将了解什么是编译器以及如何使用它进行跨平台开发。编译器是一种软件，它将您的程序转换为计算机可以读取和执行的机器码或低级指令。这些低级机器指令因平台而异。您可以使用不同的编译器编译 Qt 应用，比如GNU 编译器集合(GCC)，或者您也可以使用供

应商提供的编译器。在 **Qt Creator** 中，您可以在**Kits**选项卡下找到工具包支持的编译器，以及在 **Windows**、**Linux** 或 **MacOS** 等特定平台上构建应用的其他基本工具。并非所有受支持的编译器都随 **Qt** 安装程序一起提供，但您可以在推荐的工具包中找到自动列出的最广泛使用的编译器。**Qt** 可能会放弃对某些套件配置的支持，或将其替换为最新版本。

目前，**Qt** 支持以下编译器：

- **GCC**
- 用于 **Windows** 的最小 **GNU(MinGW)**
- 帖子主题：Re: Колибри программирования
- 低级虚拟机(**LLVM**)
- 英特尔 **C++** 编译器(**ICC**)
- T1ClangT2 和 T0\_\_\_\_\_
- 拿子游戏 / 取物游戏
- **QCC**

此外，**Qt Creator** 裸机设备插件提供以下编译器：

- **IAR** 嵌入式工作台(**IAREW**)
- 凯尔
- 小型设备 **C** 编译器(**SDCC**)

除了前面的编译器，**Qt** 在构建 **Qt** 项目时还使用特定的内置编译器。下面列出了这些内容：

- 元对象编译器(**moc**)
- 用户界面编译器(**uic**)
- 资源编译器(**rcc**)

您可以使用上述编译器为目标平台构建应用或添加自定义编译器配置。在下一节中，您将学习如何创建自定义编译器配置。

## 添加自定义编译器

要添加 **Qt Creator** 未自动检测到或不可用的编译器，请使用自定义选项。您可以指定目录的编译器和工具链路径，并相应地进行配置。

要添加自定义编译器配置，请执行以下步骤：

1. 要在 **Qt** 中创建新的编译器配置，请单击菜单栏上的**Tools**菜单，然后从左侧窗格中选择**Kits**选项卡。
2. Then, click on the **Compilers** tab and select **Custom** from the **Add** dropdown. You will see **C** and **C++** options in the context menu. Select the type as per your requirement. You can see an overview of this in the following screenshot:



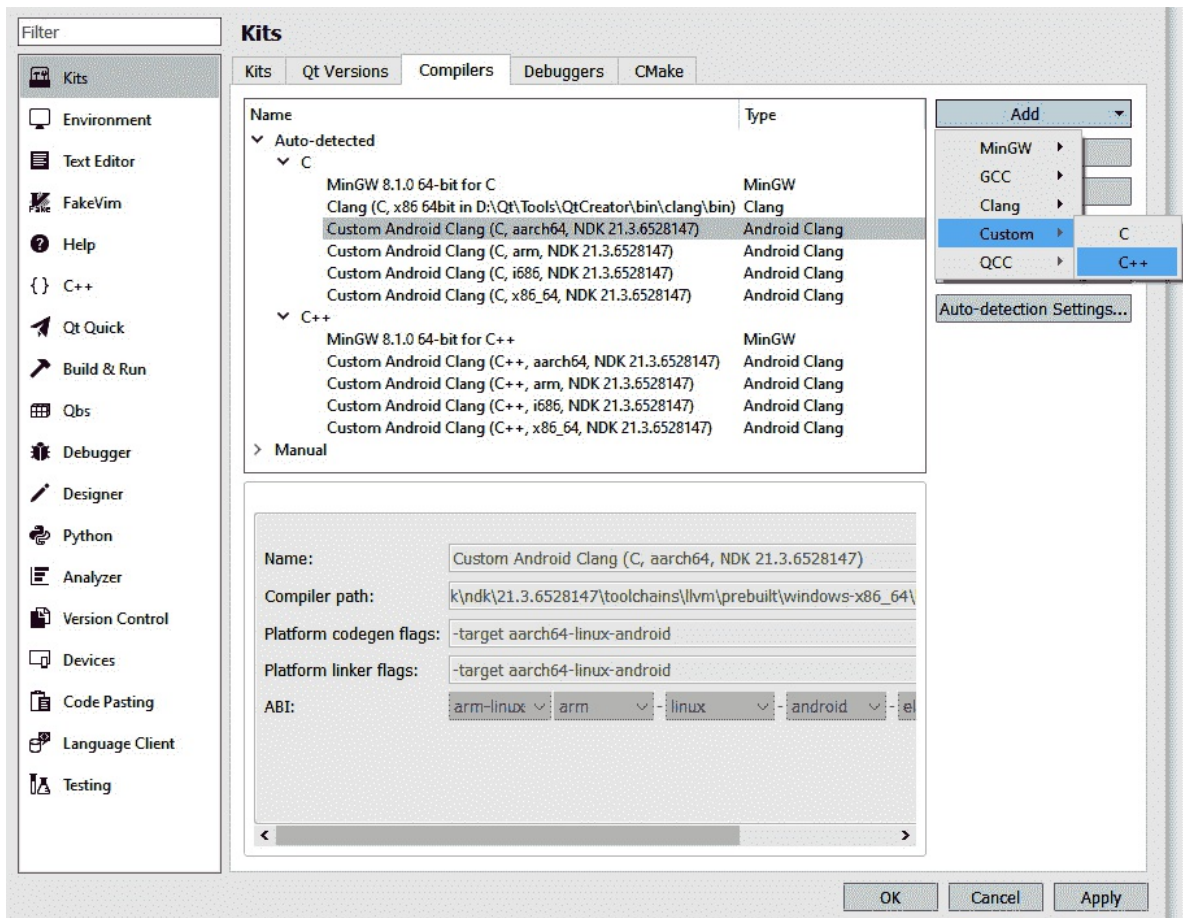


图 5.1-自定义编译器选项

3. 在下一步中，使用编译器的自定义名称填写**Name**字段。
4. 接下来，在编译器路径字段中，选择编译器所在目录的路径。
5. 接下来，在**Make Path**字段中，浏览到make工具所在目录的路径。
6. In the next step, specify the **application binary interface (ABI)** version in the **ABI** field.

您可以在下面的屏幕截图中看到这方面的概述：

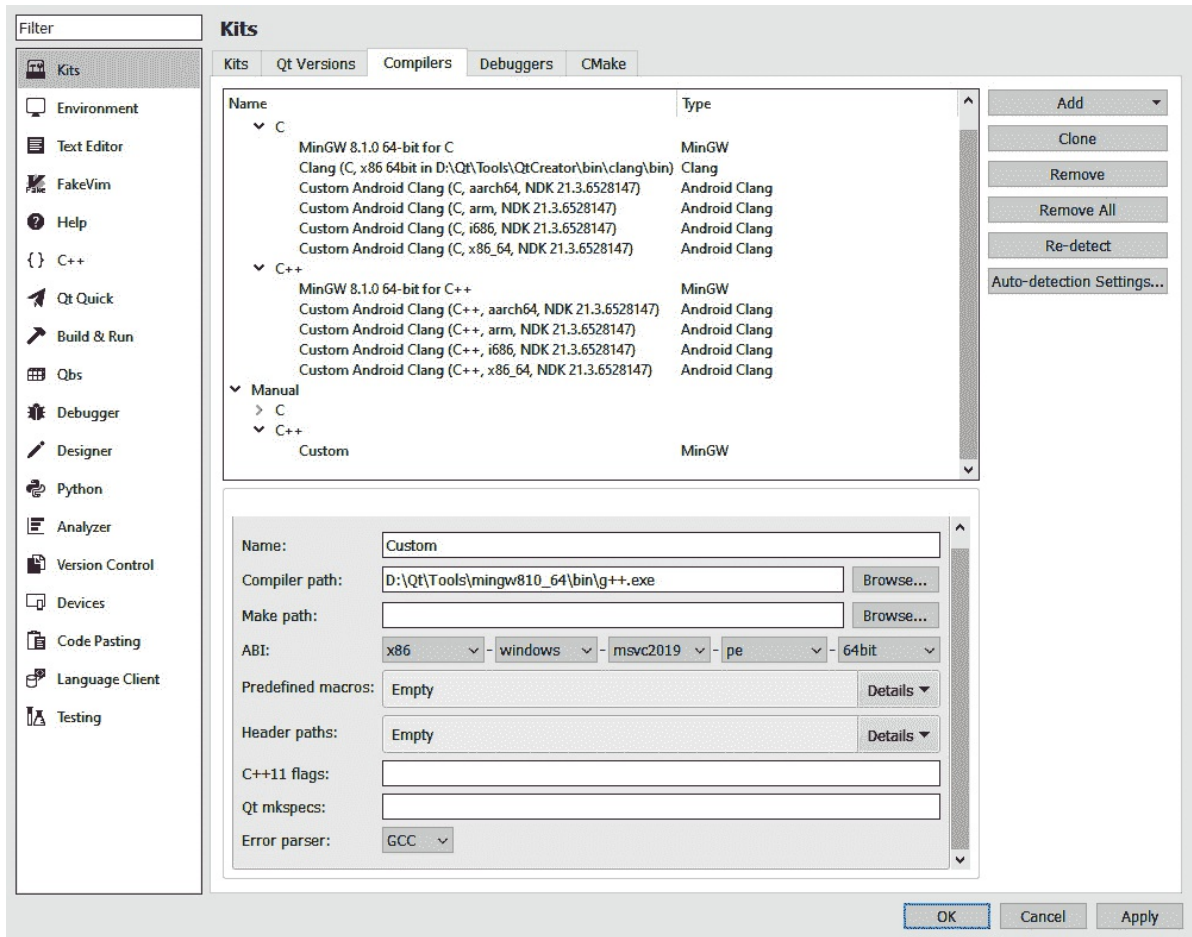


图 5.2-自定义编译器的必填字段

7. 接下来，您可以在预定义宏字段中指定默认所需的宏。按以下格式在单独的行上指定每个宏： `MACRO[=value]`。
8. 在下一步中，在标头路径字段中指定编译器检查标头的目录路径。
9. 接下来，在 **C++ 11** 标志字段中，指定打开 **C++ 11** 支持的标志。
0. 在下一步中，在 **Qt mkspecs** 字段中指定 **mkspecs** (一组编译规则) 的位置。
1. 接下来，在 **Error Parser** 字段中，选择合适的错误解析器。
2. 单击应用按钮保存配置。

在这一节中，您了解了受支持的编译器以及如何在 Qt Creator 中创建新的编译器配置，但是要构建和运行项目，我们需要更多的工具，而不仅仅是一个编译器。为了方便起见，Qt 提供了 `qmake` 作为内置的构建工具。在下一节中，我们将讨论 `qmake` 是什么。

## 使用 `qmake` 进行建筑

`make` 是一个构建工具，它读取名为 `Makefile` 的项目配置文件，并构建可执行程序 and 库。 `qmake` 是 Qt 提供的构建工具，它简化了跨多个平台的开发项目的构建过程。它将每个项目文件中的信息展开为执行编译和链接所需命令的 `Makefile`。它也可以用于非 Qt 项目。 `qmake` 基于项目文件中的信息生成 `Makefile`，并包含支持 Qt 开发的补充功能，自动包括 `moc` 和 `uic` 的构建规则。 `qmake` 还可以为 Microsoft Visual Studio 创建项目，而不需要开发人员更改项目文件。

作为一个社区驱动的框架，Qt 对开发人员来说非常灵活，让他们可以自由地为他们的项目选择最合适的工具，而不需要强迫他们使用自己的构建系统。Qt 支持以下类型的构建系统：

- `qmake`
- 我是 CMake。
- QBS
- 介子
- 不可信的建筑

您可以从 Qt Creator UI 或命令行运行 `qmake`。您应该在每次更改项目文件时运行 `qmake`。以下是从命令行运行 `qmake` 的语法：

```
>qmake [mode] [options] files
```

`qmake` 提供两种不同的操作模式。在默认模式下，`qmake` 使用项目文件中的信息生成 `Makefile`，但它也可以生成项目文件。这些模式如下所示：

- `-makefile`
- `-project`

在 **Makefile** 模式下，`qmake` 将生成一个 `Makefile`，用于创建项目。在 `Makefile` 模式下运行 `qmake` 的语法如下所示：

```
>qmake -makefile [options] files
```

在项目模式下，`qmake` 将生成项目文件。在项目模式下运行 `qmake` 的语法如下所示：

```
>qmake -project [options] files
```

如果将 **Visual Studio** 用作集成开发环境(**IDE**)，则可以将现有的 `qmake` 项目导入到 **Visual Studio** 中。`qmake` 可以创建包含开发环境所需的所有基本信息的 **Visual Studio** 项目。它可以使用以下命令在子目录中递归生成 `.vcproj` 文件，在主目录中递归生成 `.sln` 文件：

```
>qmake -tp vc -r
```

例如，您可以通过运行以下命令为 `HelloWorld` 项目生成 **Visual Studio** 项目：

```
>qmake -tp vc HelloWorld.pro
```

请注意，每次修改项目文件时，都需要运行 `qmake` 来生成更新的 **Visual Studio** 项目。

您可以在以下链接中找到有关 `qmake` 的更多详细信息：

<https://doc.qt.io/qt-6/qmake-manual.html>

大多数 `qmake` 项目文件使用 `name = value` 和 `name += value definitions` 列表定义项目使用的源文件和头文件，但 `qmake` 中还有其他高级功能，它们使用其他运算符、函数、平台范围和条件来创建跨平台应用。有关 `qmake` 语言的更多详细信息，请访问以下链接：<https://doc.qt.io/qt-6/qmake-language.html>。

Qt 团队在 Qt6 上投入了大量精力，通过转向广泛采用的、流行的构建工具 **CMake** 来使其面向未来。通过使用 **Conan** 作为一些附加组件的包管理器，实现了使 Qt 更加模块化。Qt6 中的一些 Qt 模块在 Qt 在线安装程序中不再以二进制包的形式提供，而是以 Conan 配方的形式提供。您可以通过以下链接了解更多关于构建系统更改和添加 `cmake` 作为默认构建工具的信息：<https://doc.qt.io/qt-6/qt6-buildsystem.html>。

## 重要音符

在 Qt5 中，构建系统建立在 `qmake` 之上，但在 Qt6 中，**CMake** 是用于从源代码构建 Qt 的构建系统。此更改仅影响希望从源代码构建 Qt 的开发人员。您仍然可以使用 `qmake` 作为您的 Qt 应用的构建工具。

在本节中，您了解了 `qmake`。我们将跳过高级 `qmake` 主题进行自我探索。在下一节中，我们将讨论由 `qmake` 解析的 Qt 项目文件。

## Qt 项目(.pro)文件

---

在前面的示例中，QtCreator 创建的 `.pro` 文件实际上是 Qt 项目文件。`.pro` 文件包含 `qmake` 构建应用、库或插件所需的所有信息。项目文件既支持简单生成系统，也支持复杂生成系统。简单的项目文件可以使用简单的声明，定义标准变量来指示项目中使用的源文件和头文件。复

杂的项目可能使用多个流结构来优化构建过程。项目文件包含一系列用于指定资源的声明，例如指向源文件和头文件的链接、项目所需的库、不同平台的自定义生成过程等。

Qt 项目文件有多个部分，并使用某些预定义的 `qmake` 变量。让我们看一下前面的 `HelloWorld` 示例 `.pro` 文件：

```
QT += core gui
greaterThan(QT_MAJOR_VERSION, 4): QT += widgets
CONFIG += c++ 17
You can make your code fail to compile if it uses
deprecated APIs.
In order to do so, uncomment the following line.
#DEFINES += QT_DISABLE_DEPRECATED_BEFORE=0x060000
disables all the APIs deprecated before Qt 6.0.0
SOURCES += \
 main.cpp \
 widget.cpp
HEADERS += \
 widget.h
FORMS += \
 widget.ui
Default rules for deployment.
qnx: target.path = /tmp/${TARGET}/bin
else: unix:!android: target.path = /opt/${TARGET}/bin
!isEmpty(target.path): INSTALLS += target
```

项目文件只是告诉 `qmake` 项目中需要的 Qt 模块是什么，以及可执行程序的名称。它还链接到需要包含在项目中的头文件、源文件、表单文件和资源文件。所有这些信息对于 `qmake` 创建配置文件和构建应用至关重要。对于更复杂的项目，您可以为不同的操作系统配置不同的项目文件。

下表介绍了最常用的变量及其用途：

- **QT**：项目中使用的 Qt 模块列表

- **CONFIG**: 常规项目配置选项
- **DESDIR**: 放置可执行文件或二进制文件的目录
- **FORMS**: 要由 UI 编译器处理的 UI 文件列表(**uic**)
- **HEADERS**: 生成项目时使用的头(**.h**)文件的文件名列表
- **RESOURCES**: 要包含在最终项目中的资源(**.qrc**)文件列表
- **SOURCES**: 生成项目时要使用的源代码(**.cpp**)文件列表
- **TEMPLATE**: 用于项目的模板

您可以向项目中添加不同的 Qt 模块、配置和定义。让我们来看看我们如何才能做到这一点。要添加其他模块，只需在 **QT +=** 后添加 **MODULE** 关键字，如下所示：

```
QT += core gui sql
```

您还可以在前面添加一个条件，以确定何时将特定模块添加到您的项目中，如下所示：

```
greaterThan(QT_MAJOR_VERSION, 4): QT += widgets
```

您还可以将配置设置添加到项目中。例如，如果要在编译项目时指定 **c++ 17** 规范，则将以下行添加到 **.pro** 文件：

```
CONFIG += c++ 17
```

您可以向项目文件添加注释，从散列符号(**#**)开始，生成系统将忽略相应的文本行。现在，让我们来看一下 **TEMPLATE** 变量。这决定了构建过程的输出是应用、库还是插件。有不同的变量可用于概述 **qmake** 将生成的文件类型。它们如下所列：

- **app** 用于构建应用。
- **lib** 是用来建库的。



- `aux` 用于构建任何内容。 如果不需要调用编译器来创建目标-例如，因为您的项目是用解释型语言编写的，则使用此选项。
- `subdirs` 用于使用 `SUBDIRS` 变量指定的子目录。 每个子目录必须包含其自己的项目文件。
- `vcapp` 用于创建 Visual Studio 项目文件以生成应用。
- `vcplib` 用于创建用于生成库的 Visual Studio 项目文件。
- `vcsubdirs` 用于创建 Visual Studio 解决方案文件以在子目录中生成项目。

Qt 项目文件有时需要依赖于 `include` 功能。 在 Qt 项目文件中，您还可以定义两个重要变量： `INCLUDEPATH` 和 `DEPENDPATH`。 您可以使用 `SUBDIRS` 变量编译一组依赖库或模块。

现在，让我们讨论一下什么是 `.pri` 文件。

## 了解 `.pro` 和 `.pri` 文件之间的差异

您可以创建 `.pri` 文件以在复杂项目中包含项目文件。 这提高了可读性，并隔离了不同的模块。 `.pri` 文件通常称为项目包含文件或 `qmake` 包含文件，其格式类似于 `.pro` 文件。 主要区别在于使用目的； `.pro` 文件是我们期望直接在其上运行 `qmake` 的文件，而 `.pri` 文件包含在 `.pro` 文件中。 您可以将源文件、头文件、 `.ui` 文件和 `.qrc` 文件等常见配置添加到 `.pri` 文件中，并根据项目需要从多个 `.pro` 文件中包含它们。

您可以在 `.pro` 文件中包含 `.pri` 文件，如下所示：

```
include($$PWD/common.pri)
```

在本节中，您了解了什么是 Qt 项目文件，以及其中使用的不同变量。在下一节中，我们将讨论不同的构建设置。

# 了解生成设置

---

在编译或生成项目之前，编译器需要某些细节，这些细节称为生成设置。这是编译过程中非常重要的一部分。在本节中，您将了解生成设置以及如何以正确的方式配置它们。同一项目可以有多个生成配置。通常，**Qt Creator** 会自动创建调试、发布和概要文件构建配置。调试版本包含调试应用所需的附加调试符号，而发布版本是没有此类符号的优化版本。通常，开发人员使用调试配置进行测试，使用发布配置创建最终的二进制文件。配置文件版本是一个优化的发布版本，提供了单独的调试信息，最适合分析应用。

可以在项目模式下指定生成设置。如果 IDE 中没有打开任何项目，您可能会发现**Projects**按钮被禁用。您可以通过单击**Add**下拉按钮，然后选择您想要添加的配置类型来添加新的构建配置。选项可能取决于为项目选择的构建系统。您可以根据需要添加多个生成配置。您可以单击克隆...。按钮根据当前构建配置添加构建配置，或单击**Rename...**。按钮重命名当前选定的生成配置。单击**Remove**按钮删除构建配置。

您可以在下面的屏幕截图中看到这方面的概述：



## Build Settings

Edit build configuration:

Debug ▾

Add ▾

Remove

Rename...

Clone...

### General

Shadow build:	<input checked="" type="checkbox"/>
Build directory:	Desktop_Qt_6_0_2_MinGW_64_bit-Debug <span>Browse...</span>
Separate debug info:	Leave at Default ▾
QML debugging and profiling:	Enable ▾
⚠ Might make your application vulnerable. Only use in a safe environment.	
Qt Quick Compiler:	Leave at Default ▾
qmake system() behavior when parsing:	Enable Disable Leave at Default

### Build Steps

qmake: qmake.exe QtQuickCompilerDemo.pro	Details ▾
Make: mingw32-make.exe -j8 in D:\QtBook\Packt\Qt-6-and-C-Mod	Details ▲
Override D:\Qt\Tools\mingw810_64\bin\mingw32-make.exe:	<span>Browse...</span>
Make arguments:	
Parallel jobs:	8 <input type="checkbox"/> Override MAKEFLAGS
Disable in subdirectories:	<input type="checkbox"/>
<span>Add Build Step ▾</span>	

图 5.3-构建设置和 Qt 快速编译器选项

通常，QtCreator 在与源目录不同的目录中构建项目，称为影子构建。这将分离为每个构建和运行工具包生成的文件。如果您只想使用单个工具包构建和运行，则可以取消选中**Shadow Build**复选框。Qt Creator 项目向导创建一个 Qt Quick 项目，该项目可以编译为使用Qt 资源系统。要使用默认设置，请选择**Leave at Default**。要编译 Qt 快速代码，请在**Qt Quick Compiler**字段中选择**Enable**，如图 5.3所示。

您可以通过以下链接阅读有关不同构建配置的更多信息：

<https://doc.qt.io/qtcreator/creator-build-settings.html>

在本节中，我们讨论了生成设置。在生成跨平台应用时，将特定于平台的配置添加到项目文件中非常重要。在下一节中，我们将了解特定于平台的设置。

## 特定于平台的设置

您可以为不同的平台定义不同的配置，因为不是每种配置都能适合所有用例。例如，如果要包含不同操作系统的不同头路径，可以将以下代码行添加到`.pro`文件：

```
win32: INCLUDEPATH += "C:/mylibs/windows_headers"
unix:INCLUDEPATH += "/home/user/linux_headers"
```

在前面的代码片段中，我们添加了一些特定于 **Windows** 和特定于 **Linux** 的头文件。您也可以将`if`语句等配置放在 **C++** 中，如下所示：

```
win32 {
 SOURCES += windows_code.cpp
}
```

上述代码仅适用于 **Windows** 平台，这就是我们在其前面添加了`win32`关键字的原因。如果您的目标平台是基于 **Linux** 的，那么您可以添加一个`unix`关键字来添加特定于 **Linux** 的配置。

要在 **Windows** 平台上为应用设置自定义图标，应将以下代码行添加到项目(`.pro`)文件中：

```
RC_ICONS = myapplication.ico
```

若要在 MacOS 上为应用设置自定义图标，应将以下代码行添加到项目 (.pro) 文件中：

```
ICON = myapplication.icns
```

请注意，Windows 和 MacOS 的图标格式不同。对于 Linux 发行版，有一种不同的方法来为每种风格制作桌面条目。

在本节中，我们讨论了一些特定于平台的设置。在下一节中，我们将学习如何将 Visual Studio 与 Qt vs 工具结合使用。

## 在 Microsoft Visual Studio 中使用 Qt

---

一些开发人员选择 Visual Studio 作为他们的首选 IDE。因此，如果您最喜欢的 IDE 是 Visual Studio，那么您可以将 Qt VS 工具与 Microsoft Visual Studio 集成。这将允许您使用标准的 Windows 开发环境，而不必担心与 Qt 相关的构建步骤或工具。您可以直接从 Microsoft Visual Studio 安装和更新 Qt VS 工具。

您可以从 Visual Studio Marketplace 中找到相应版本的 Qt Visual Studio 工具。对于 Visual Studio 2019，您可以从以下链接下载该工具：[https://marketplace.visualstudio.com/items?](https://marketplace.visualstudio.com/items?itemName=TheQtCompany.QtVisualStudioTools2019)

[itemName=TheQtCompany.QtVisualStudioTools2019](https://marketplace.visualstudio.com/items?itemName=TheQtCompany.QtVisualStudioTools2019)。您也可以从以下 Qt 下载链接下载 vs 加载

项：[https://download.qt.io/official\\_releases/vsaddin/](https://download.qt.io/official_releases/vsaddin/)。

以下是 Qt VS Tools 的一些重要的功能：

- 用于创建新项目 and 类的向导
- moc、uic 和 rcc 编译器的自动生成设置
- 导入导出 .pro 和 .pri 文件

- 自动将 Qt vs Tools 项目转换为qmake项目
- 综合 Qt 资源管理
- 能够创建 Qt 翻译文件并与Qt 语言学家集成
- 集成Qt 设计器
- 集成的 Qt 文档
- Qt 数据类型的调试扩展插件

要开始使用 Visual Studio 环境中的功能，必须设置 Qt 版本。从选项中选择适当的版本，然后重新启动 IDE。Visual Studio 和 Qt 使用不同的文件格式保存项目。您可以在 Visual Studio 中使用.pro文件和qmake或.vcproj文件来生成项目。由于 Visual Studio 用于特定于 Windows 的开发，因此建议使用 Qt Creator 作为跨平台开发的 IDE。

如果没有.vcproj文件，则可以通过命令行或 VS 工具从.pro文件生成一个。我们已经在*Building with qmake*一节中讨论了命令行指令。也可以使用 VS Tools 中的Open选项将.pro文件转换为.vcproj文件。请注意，生成的.vcproj文件仅包含特定于 Windows 的设置。

在本节中，我们讨论了vs外接程序。在下一节中，我们将学习如何在 Linux 上运行示例应用。我们将跳过关于在 Windows 上构建和运行 Qt 应用的讨论，因为我们已经在前面的章节中讨论过这一点。

## 在 Linux 上运行 Qt 应用

---

在 Linux 上构建和运行 Qt 应用类似于在 Windows 上运行，但是 Linux 有很多发行版，因此很难构建一个完美地运行在所有 Linux 变体上的应用。在大多数发行版中，应用将平稳运行。我们将重点关注 Ubuntu20.04 作为我们的目标平台。当您在 Ubuntu 上安装 Qt 时，它会自动检测工具包和配置。您还可以使用适当的编译器和 Qt 版本配置

工具包，如以下屏幕截图所示：

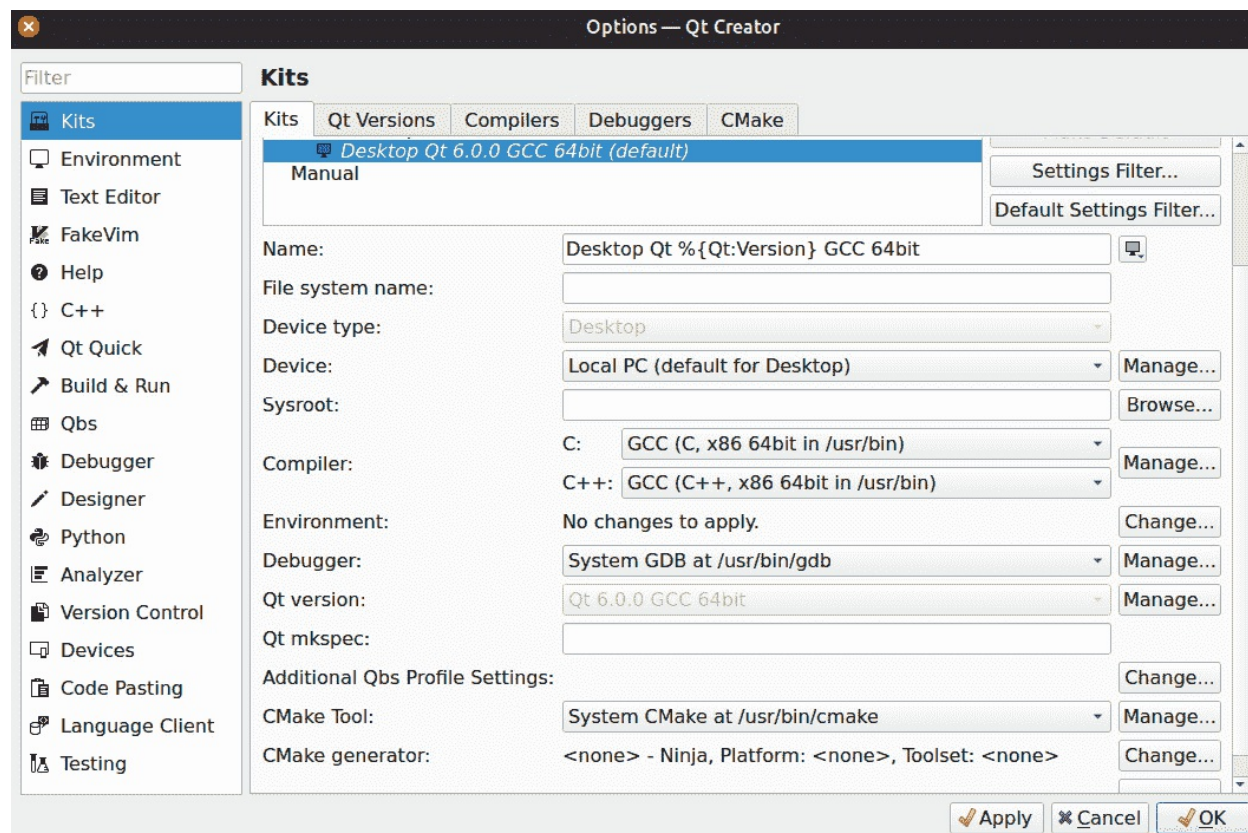


图 5.4-Ubuntu 上的台式机套件配置

让我们在 Ubuntu 上运行我们的HelloWorld示例。 点击左侧窗格上的 **Run**按钮。 很快就会出现显示\*\*Hello World! \*\*的 UI，如下图所示：



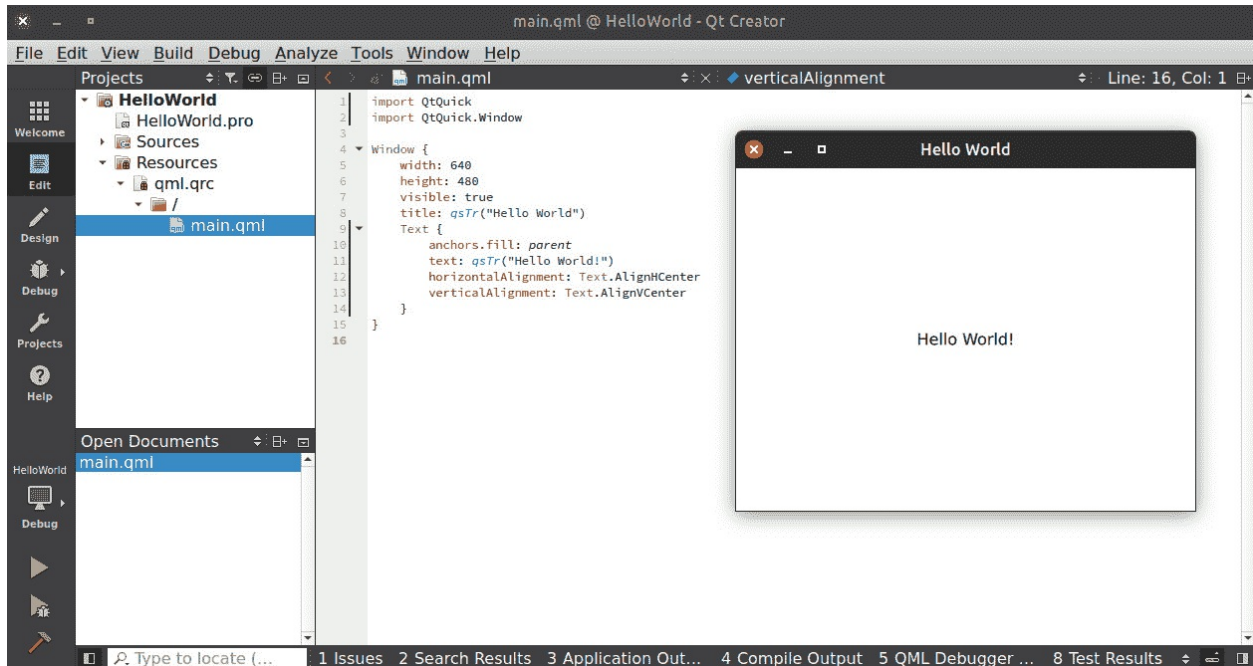


图 5.5-Ubuntu 上运行的应用

您还可以从命令行运行应用，如以下代码片段中所示的：

```
$/./HelloWorld
```

在本节中，我们讨论了如何在 Linux 发行版上运行我们的应用。在下一节中，我们将学习如何在 MacOS 和 iOS 上运行 Qt 应用。

## 在 MacOS 和 iOS 上运行 Qt 应用

我们已经在前面的章节中讨论了如何在 Windows 和 Linux 平台上构建和运行应用。让我们继续学习如何在 MacOS 和 iOS 等平台上运行我们的应用。要在 MacOS 和 iOS 上构建 Qt 应用，您需要 App Store 中的 Xcode。Xcode 是 MacOS 的 IDE，由一套用于在 MacOS 和 iOS 中开发应用的软件开发工具组成。如果您已经安装了 Xcode，Qt Creator 将检测它的存在并自动检测合适的工具包。至于套件选择，Qt for MacOS 支持 Android、clang64 位、iOS 和 iOS 模拟器套件。

您可以在下面的屏幕截图中看到 macOS 上的示例桌面套件配置：

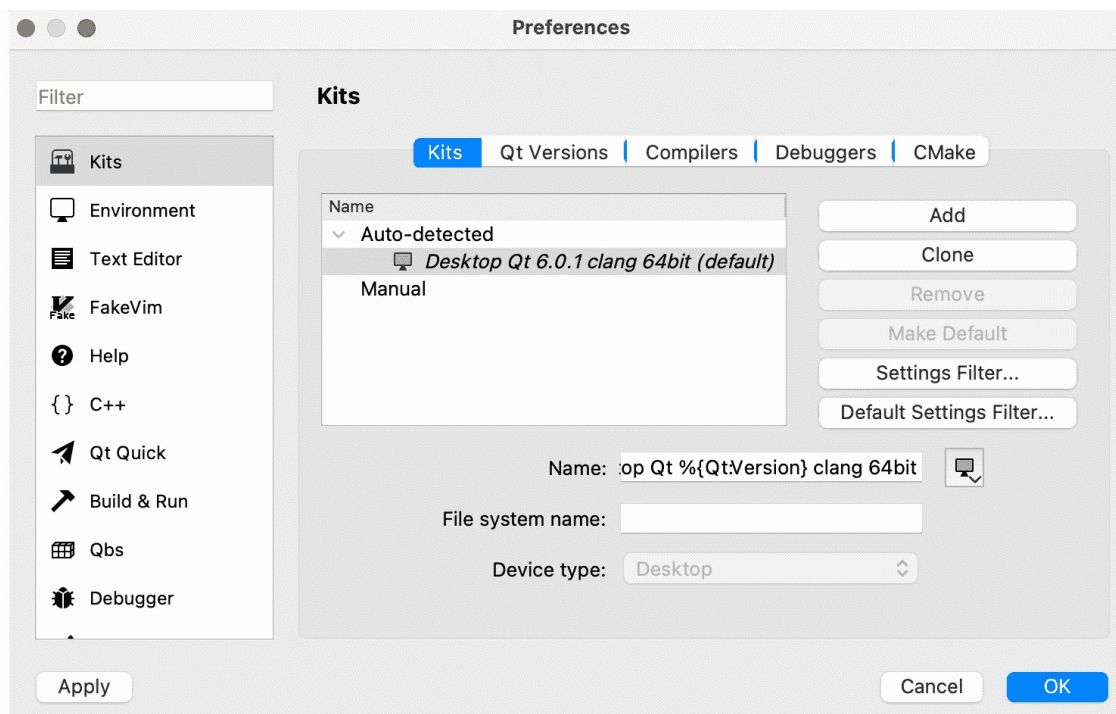


图 5.6-MacOS 上的台式机套件配置

如果您不想使用自动检测到的调试器，也可以在调试器选项卡中手动添加调试器，如以下屏幕截图所示：

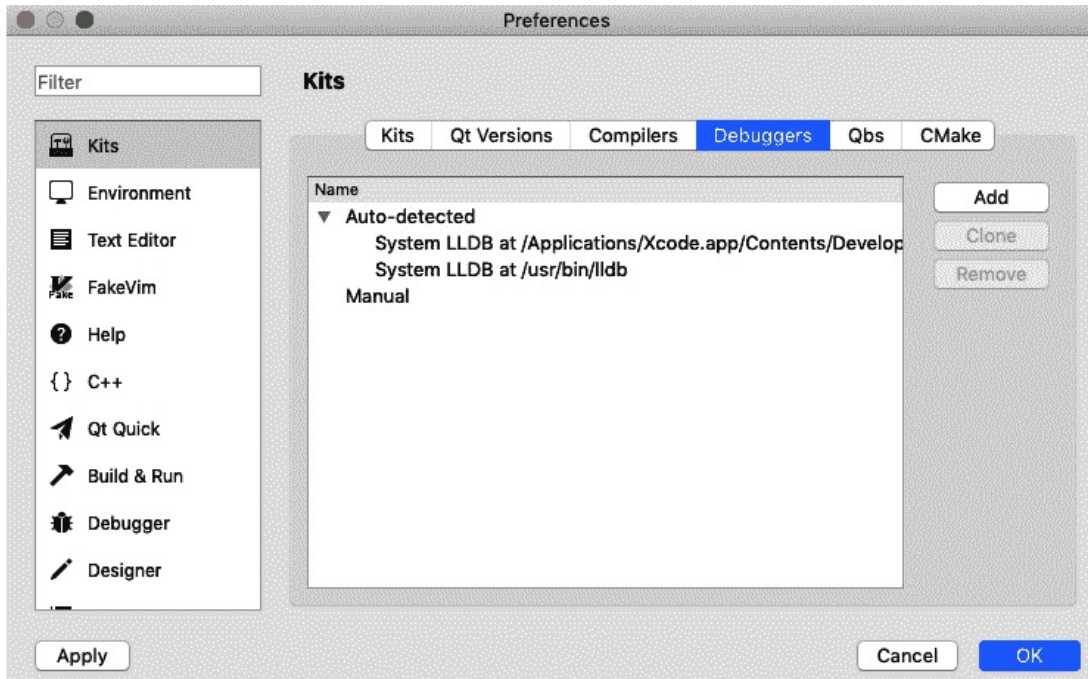


图 5.7-MacOS 上的调试器选项

在 MacOS 上运行应用类似于在 Windows 上运行它。只需点击**Run**按钮，您将立即看到运行应用。

移动平台与桌面平台(如 Windows、Linux 和 MacOS)同等重要。让我们来了解一下如何设置一个在 iOS 上运行应用的环境。

## 为 iOS 配置 Qt Creator

在 iOS 上运行 Qt 应用真的很简单。您可以连接 iOS 设备，然后从设备选择列表中选择合适的设备类型。您可以从套件选择屏幕中选择设备类型。您还可以在 IOS 模拟器上运行该应用，如以下屏幕截图所示：

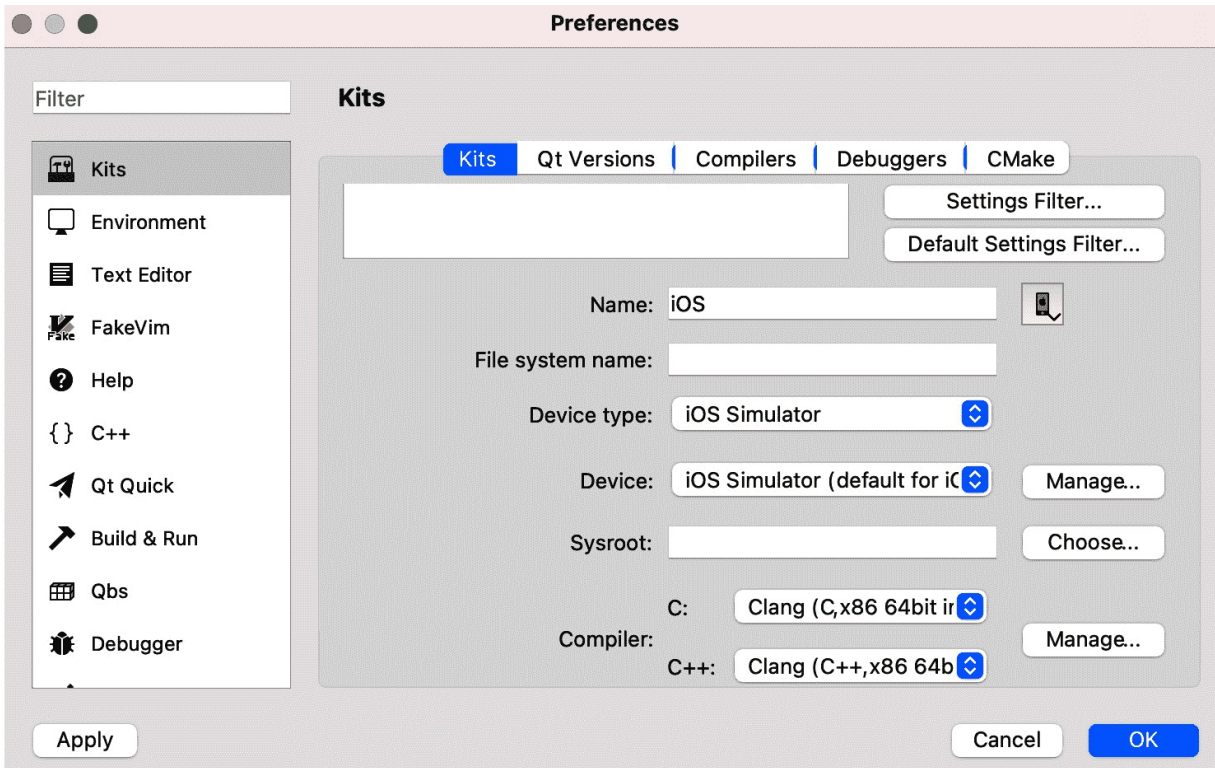


图 5.8-MacOS 上的 IOS 模拟器选项

在配置工具包之后，只需插入 iPhone 并点击运行按钮即可。您可以在以下屏幕截图中看到示例输出：

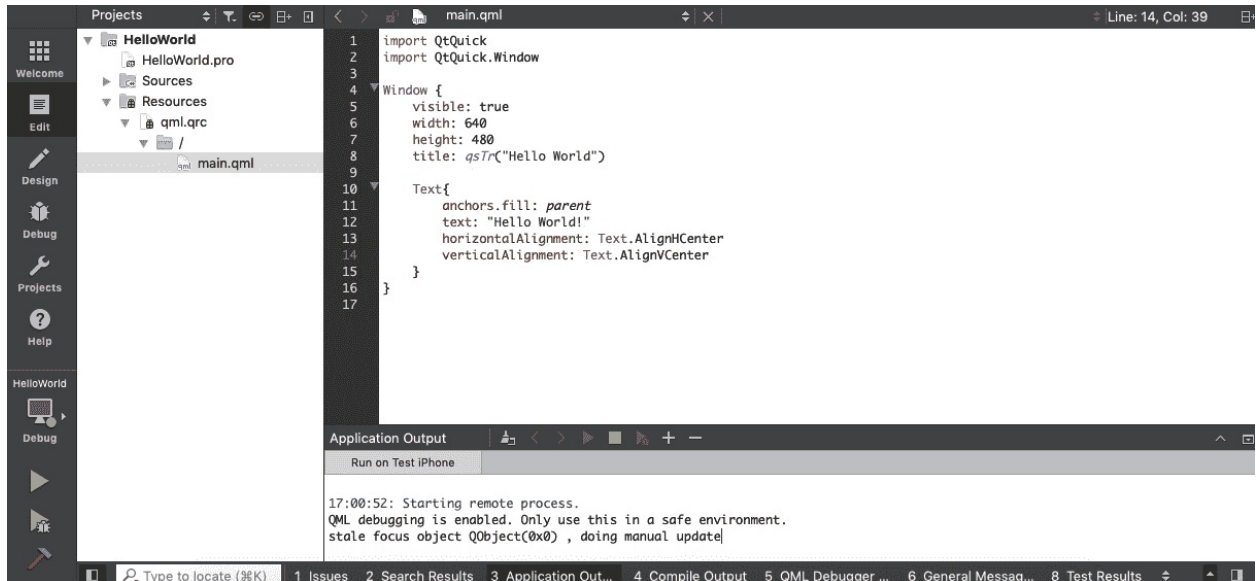


图 5.9-Qt Creator 在 iPhone 上运行应用



在 iOS 平台上构建和运行应用相对容易。然而，分发应用并不容易，因为 App Store 是一个非常封闭的生态系统。您应该拥有 Apple ID，并且需要先登录 iOS 应用，然后才能将它们分发给您的用户。您无法避免这些步骤，但现在让我们跳过部署部分。

您可以通过以下链接了解有关 App Store 提交的更多信息：

<https://developer.apple.com/app-store/submissions>

在本节中，我们了解了如何在 MacOS 和 iOS 上运行应用。在下一节中，我们将学习如何为 Android 平台配置和构建应用。

## 配置安卓 Qt Creator

Android 是当今最流行的移动平台，因此开发者想要为 Android 构建应用。虽然 Android 是一个基于 Linux 的操作系统，但它与其他 Linux 发行版有很大的不同。要使用它，您必须配置 Qt Creator 并安装某些软件包。

为了让您的 Qt Creator Android 配置能够顺利运行，请使用带有 CLANG 工具链的 OpenJDK 8、NDK R21。您可以使用所需的参数从 `ANDROID_SDK_ROOT\cmdline-tools\latest\bin` 运行 `sdkmanager`，以使用所需的依赖项进行配置。

您可以通过以下链接了解更多 Android 的具体要求和使用说明：

<https://doc.qt.io/qt-6/android-getting-started.html>

让我们按照以下步骤开始为 Android 配置您的计算机：

1. 要在 Android 上构建 Qt 应用，您必须在开发 PC 上安装 Android 软件开发工具包(SDK)、Android 原生开发工具包(NDK)、Java 开发

工具包(JDK)和 OpenSSL。您可以在每个相应字段旁边找到带有地球仪图标或**Download**按钮的下载选项，以便从相应软件包的页面下载。

2. 安装完所有必需的软件包后，重新启动 Qt Creator。Qt Creator 应该能够自动检测构建和平台工具。
3. However, you may have to configure further to fix errors in **Android** settings. You may find the SDK manager, the platform SDK, and essential packages missing, as shown in the following screenshot:

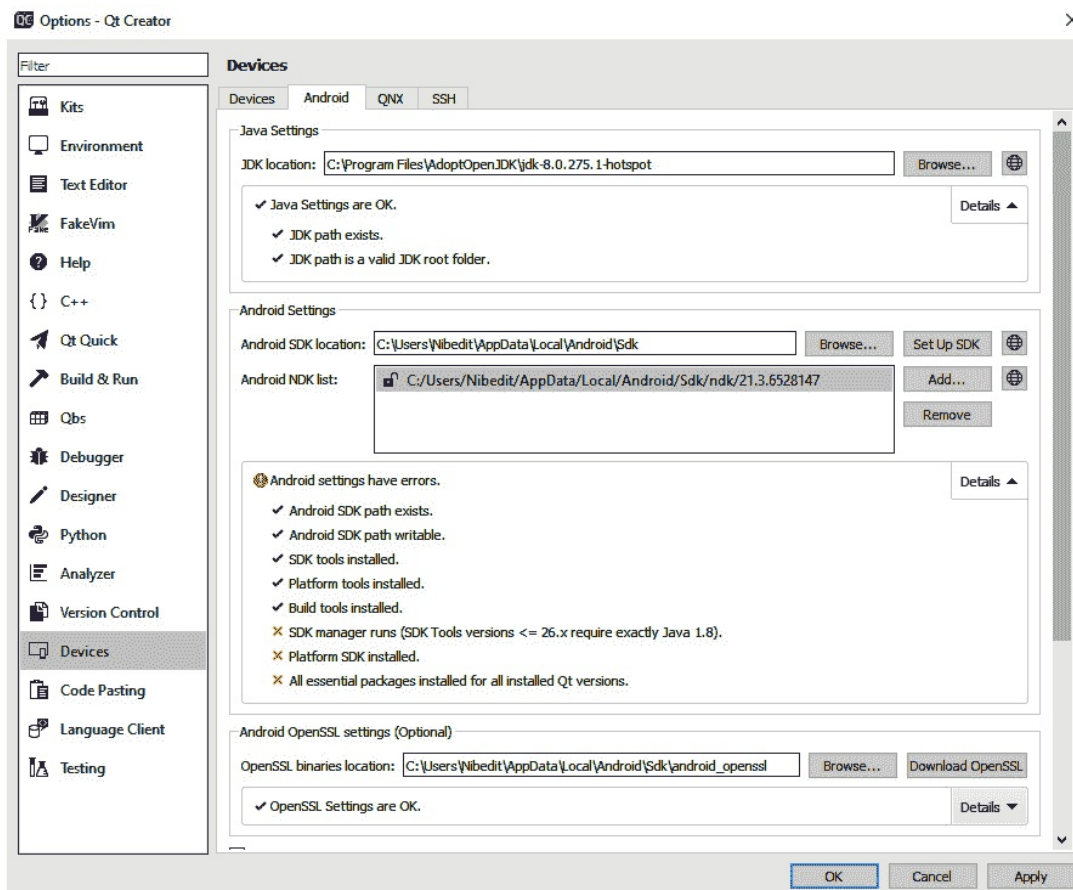


图 5.10-Android 设置屏幕

4. 在**Android** 设置下选择正确的 SDK 和 NDK 路径。单击应用按钮上的保存更改。
5. Click on the **SDK Manager** tab and click on the **Update Installed** button. You may see a message box prompting you to install missing packages, as illustrated in the following screenshot. Click on the **Yes** button to install the packages:

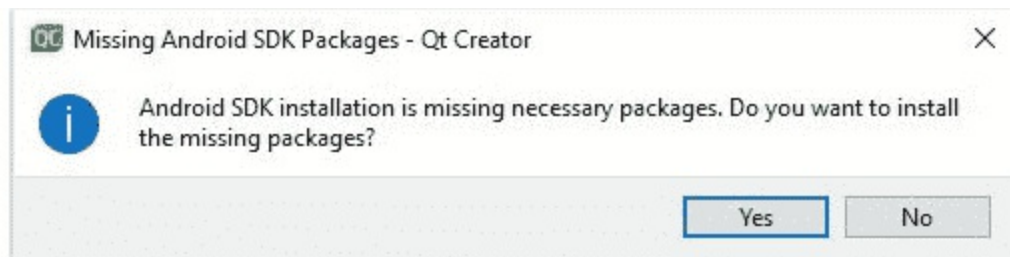


图 5.11-显示缺少 Android 软件包的信息消息

6. You may get another message warning of Android SDK changes, listing missing essential packages, as illustrated in the following screenshot. Click on the **OK** button:

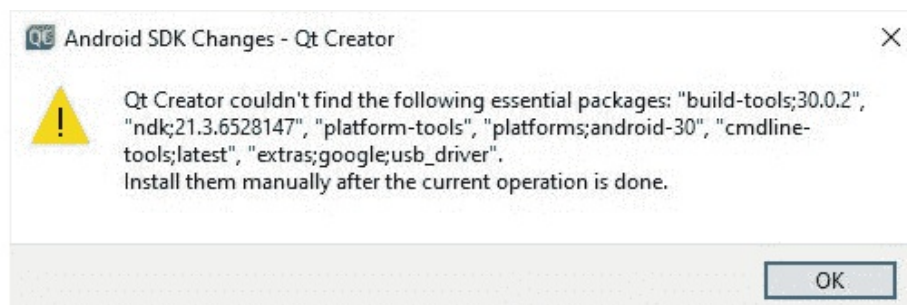


图 5.12-有关缺少 Android 软件包的警告

7. Click on the **Advanced Options...** button to launch the **SDK Manager Arguments** screen, type `--verbose`, and click on the **OK** button. You can see an overview of this in the following screenshot:



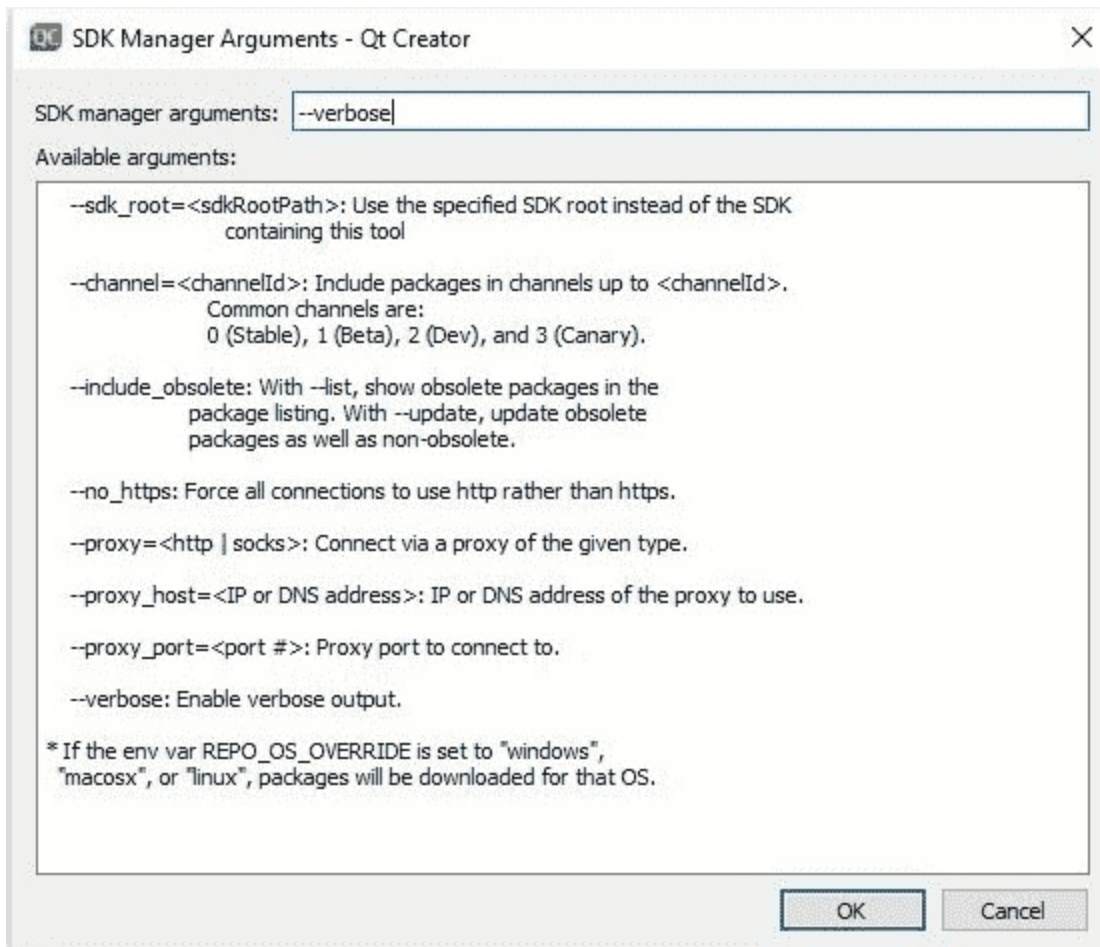


图 5.13-Android SDK 管理器工具

8. Once the issues are resolved, you will see that all Android settings have been properly configured, as shown in the following screenshot:

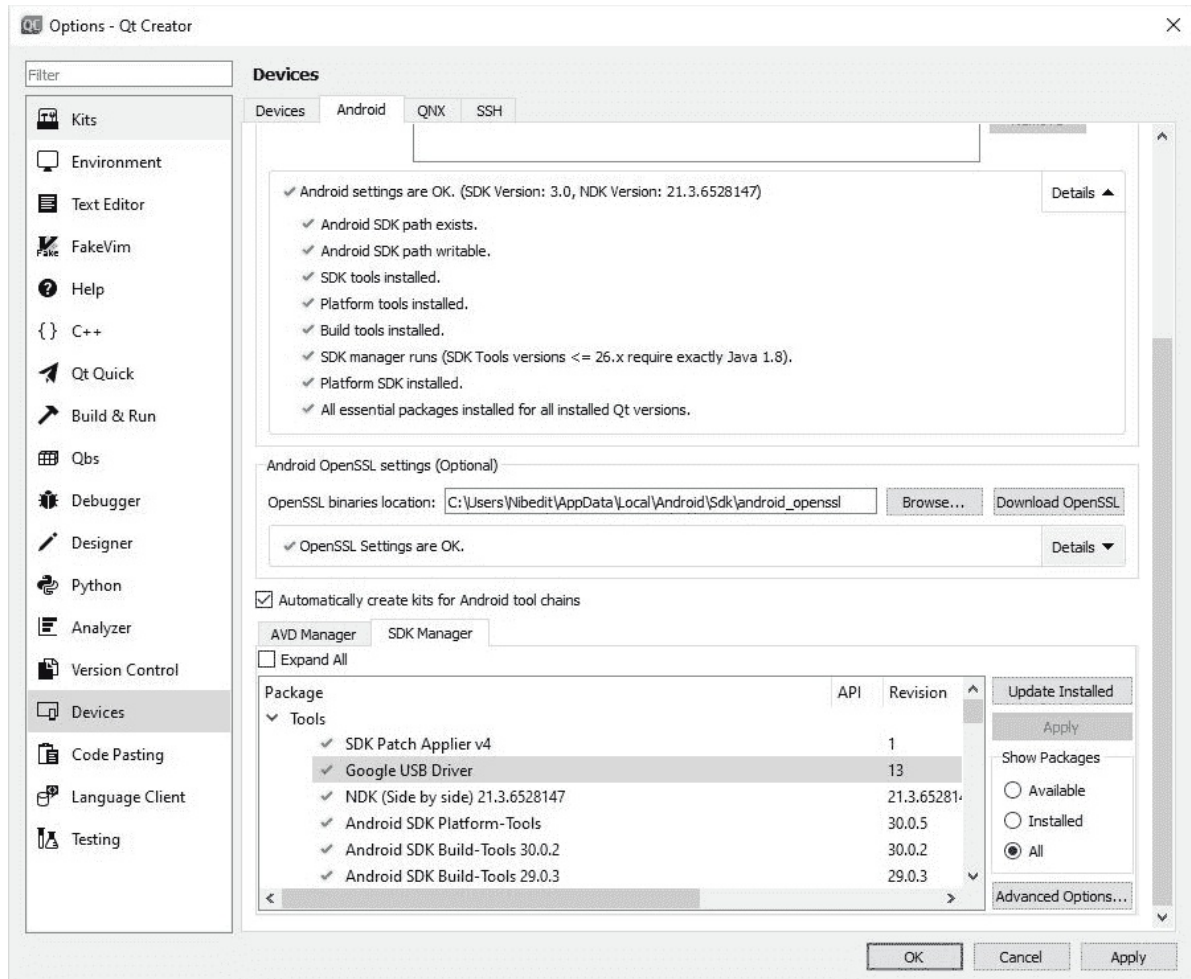


图 5.14-Qt Creator 中的正确 Android 配置

- If the issues are still not resolved or if you want to install a specific platform, you can enter the appropriate command, as shown in the following screenshot. You may also install the required packages from the command line. Qt will automatically detect the build tools and platforms available in the SDK location:

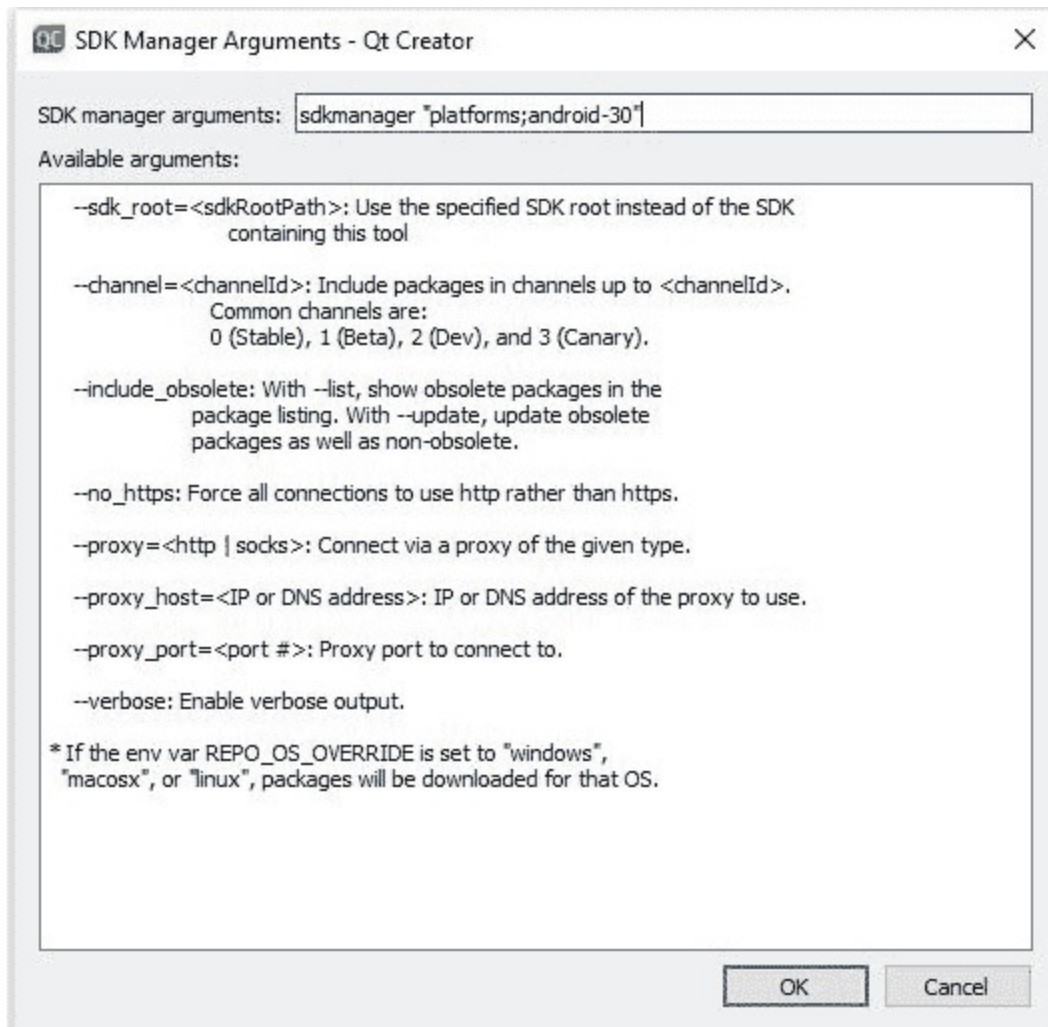


图 5.15-Android SDK 管理器工具

0. Once the Android settings are properly configured, you can see the Android kit is ready for development, as illustrated in the following screenshot:



1. Select an Android kit from the **Kit** selection option, as illustrated in the following screenshot:

![Figure 5.17 - Android Kit selection option ](img/Figure\_5.17\_B16

图 5.17-Android 套件选择选项

2. In this step, you can select a target Android version and configure your Android application by creating a `AndroidManifest.xml` file with Qt Creator. You can set the package name, version code, SDK version, application icon, permissions, and so on. The settings can be seen in the following screenshot:

![Figure 5.18 - Android manifest option in build settings ](img/Fi

图 5.18-构建设置中的 Android 清单选项

3. 您的机器现在已经为 **Android** 开发做好了准备。但是，您的 **Android** 硬件需要启用开发人员选项或使用 **Android** 模拟器。要启用**Developer**模式，请转到设置，点击系统，然后单击关于电话。
4. 然后，点击软件信息并找到内部版本号。持续点击**Builder Number**，直到看到**Developer**模式激活。可能需要轻敲七下才能激活显影模式。现在，返回到设置窗格，在这里您会发现**Developer**选项是一个条目。
5. 您的 **Android** 设备已准备好运行 **Android** 应用。单击**Run**按钮，并从**Compatible Device**列表屏幕中选择一个设备。
6. 接下来，在 **Android** 设备上的允许 **USB** 调试提示上点击允许。您将看到**\*\*Hello World! \*\***消息在您的 **Android** 设备上运行。您可以在**build**文件夹中找到生成的**.apk**文件。

祝贺你!。您已经成功地开发了 **Android** 应用。与 **iOS** 不同，**Android** 是一个开放系统。您可以复制或分发。**apk**文件放到运行同一 **Android**

版本的其他 **Android** 设备中，然后安装。然而，如果你想在 **Google Play Store** 上分发你的应用，那么你必须注册为 **Google Play** 开发者，并在包上签名。

在本节中，我们了解了如何为 **Android** 平台配置和构建。在下一节中，我们将讨论本书撰写时 **Qt6** 支持的其他平台。

## 其他 **Qt** 支持的平台

---

**Qt5** 支持多种平台，从桌面和移动平台到嵌入式和 **Web** 平台。**Qt6** 尚未支持 **Qt5** 中支持的所有平台，但随着 **Qt6** 的成熟，这些平台将逐渐得到支持。目前，在商业许可下，**Qt6** 的最新版本只支持嵌入式 **Linux**。您可能需要等待一段时间才能将应用移植到不同嵌入式平台上的 **Qt6**。否则，如果您想要立即为您喜欢的嵌入式平台迁移到 **Qt6**，您必须从源代码构建并进行必要的修改。

下面的链接提供了 **Qt6.2** 中嵌入式 **Linux** 支持的快照：<https://doc-snapshots.qt.io/qt6-dev/embedded-linux.html>。此链接可能会随着 **Qt** 升级到下一版本而更新。

**Qt** 还为商业许可下的嵌入式 **Linux** 系统提供了 **Boot to Qt** 软件堆栈。它是安装在目标系统上的轻量级、**Qt** 优化的完整软件堆栈。**Boot to Qt** 软件栈使用的是传统的嵌入式 **Linux** 内核，它是用 **POKY** 和 **Yocto** 软件包设计的。

通过以下链接了解有关引导至 **Qt** 的更多信息：

<https://doc.qt.io/QtForDeviceCreation/b2qt-index.html>

**Qt for WebAssembly** 允许为 **Web** 平台构建 **Qt** 应用。它不一定需要任何客户端安装，并且节省了服务器资源。它是一个平台插件，可以

让你构建可以嵌入到网页中的 Qt 应用。Qt6 中的开源开发者还不能使用该插件。商业许可证持有者可能会提前访问该插件。

您可以通过以下链接了解有关 WebAssembly 插件的 Qt 的更多信息：

[https://wiki.qt.io/Qt\\_for\\_WebAssembly](https://wiki.qt.io/Qt_for_WebAssembly)

在本节中，我们了解了 Qt6 中支持的其他平台。在下一节中，我们将讨论如何将您的应用从 Qt5 移植到 Qt6。

## 从 Qt5 移植到 Qt6

---

Qt6 是对 Qt 框架的重大更改，因此它破坏了一些向后兼容性。因此，在升级到 Qt6 之前，请确保您的 Qt5 应用已更新到 Qt5.15。从 Qt5.15 到 Qt6 的移植将更容易，更改次数最少。但是，Qt 5.15 中标记为不推荐使用或过时的 API 可能已从 Qt 6.0 中删除。

Qt5 和 Qt6 中的 CMake API 在语义上几乎相同。因此，Qt5.15 引入了无版本的目标和命令，允许编写完全独立于 Qt 版本的 CMake 代码。无版本导入的目标对于同时需要 Qt5 和 Qt6 编译的项目最有用。默认情况下不建议使用它们，因为缺少目标属性。您可以通过以下链接了解更多信息：<https://doc.qt.io/qt-6/cmake-qt5-and-qt6-compatibility.html>。

Qt6 中删除了一些类和模块，但为了便于移植，这些类和模块保留在 Qt5Compat 中。除了构建系统更改之外，您可能需要修复过时类的 Include 指令-例如，Qt6 中的 QLinkedList、QRegExp 和 QTextCodec 等类被新类替换。但是为了便于移植，您需要将 core5compat 添加到您的 .pro 文件中，如下所示：

```
QT += core5compat
```



牵伸机构也有变化。如果在项目中使用 OpenGL 样式的**OpenGL** 着色语言(**GLSL**)，则必须切换到 Vulkan 样式的 GLSL。根据新的更改，您可以在与 Vulkan 兼容的 GLSL 中编写着色器并使用 `qsb` 工具。着色器代码应编译为标准可移植中间表示**-Vulkan(SPIR-V)**格式。我们将在[第 8 章](#)、图形和动画中详细讨论图形。有关详细信息，请访问以下链接：<https://doc.qt.io/qt-6/qtshadertools-index.html>。

对**Qt** 建模语言(**QML**)也有一些更改。Qt Quick Extras 模块已与 Qt Quick Controls 合并。像 `QtGraphicalEffects` 这样的模块已经从 Qt6 中删除，并将以不同的许可证提供。Qt Quick MultiEffect 在 Qt Marketplace 中提供，并提供更好的性能。您还可以考虑更新 QML 中以前的信号连接，以使用 JavaScript 函数声明，如以下代码片段所示：

```
Connections {
 target: targetElement
 function onSignalName() { //Do Something }
}
```

Qt 状态机模块在很大程度上与 Qt5 版本的源代码兼容，因此您应该能够继续处理他们的项目，而不需要--或者只需要很小的--更改。要使用状态机模块的类，请将以下代码行添加到 Qt 项目(`.pro`)文件：

```
QT += statemachine
```

要在 QML 文件中导入状态机模块，请使用以下 `import` 语句：

```
import QtQml.StateMachine
```

Qt 提供了详细的移植指南。如果您希望将 Qt 5 应用移植到 Qt 6，请查看以下文档：



<https://doc.qt.io/qt-6/portingguide.html>

<https://www.qt.io/blog/porting-from-qt-5-to-qt-6-using-qt5compat-library>

<https://doc.qt.io/qt-6/porting-to-qt6-using-clazy.html>

在本节中，您学习了如何将应用从 **Qt5** 移植到 **Qt6**。在下一节中，我们将总结我们在本章中学到的内容。

## 摘要

---

本章解释了使用 **Qt Creator** 进行跨平台开发。您了解了各种编译器、构建工具以及特定于构建和平台的设置。在本章中，您学习了如何在桌面和移动平台上配置和构建应用，以及如何在 **iPhone** 和 **Android** 设备上运行应用。我们讨论了如何将 **Qt** 项目移植到不同的平台，而不会遇到太多挑战。

在下一章中，您将了解信号和槽机制、**Qt** 元对象系统和事件处理。我们继续吧！

## 第三部分：高级编程、调试和部署

---

原文：<https://gitee.com/apacheecn/apacheecn-c-cpp-zh-pt2/blob/master/docs/cross-plat-dev-qt6-mod-cpp/sec3.md>

在本节中，您将了解高级编程和开发方法。您将了解如何在各种平台上调试、测试和部署 **Qt** 应用。您还将了解国际化以及如何构建高性能应用。

本节包括以下章节：

- 第 6 章，信号和插槽
- 第 7 章，模型视图编程
- 第 8 章，图形和动画
- 第 9 章，测试和调试
- 第 10 章，部署 **Qt** 应用
- 第 11 章，国际化
- 第 12 章，性能注意事项

## 六、信号和插槽

---

原文: <https://gitee.com/apacheecn/apacheecn-c-cpp-zh-pt2/blob/master/docs/cross-plat-dev-qt6-mod-cpp/06.md>

在前面的章节中，我们学习了如何使用 Qt 控件和 Qt Quick 创建 GUI 应用。但是要使我们的应用可用，我们需要添加一个通信机制。信号和槽机制是 Qt 的独特功能之一，使其有别于其他框架。信号和槽是通过 Qt 的元对象系统实现的。

在本章中，您将深入了解信号和插槽及其内部工作原理。您将能够接收来自不同类别的通知并采取相应的操作。

在本章中，我们将讨论以下主题：

- 了解 QT 信号和插槽
- QT 信号和插槽的工作原理
- 了解 QT 的产权制度
- 了解信号和处理程序事件系统
- 了解事件和事件循环
- 使用事件过滤器管理事件
- 拖放

在本章结束时，您将能够在带有 QML 的 C++ 类之间以及 QML 组件之间进行通信。

## 技术要求

---

本章的技术要求包括在最新的桌面平台(如 Windows 10、Ubuntu 20.04 或 MacOS 10.14)上安装 Qt(6.0.0)和 Qt Creator(4.14.0)的最低版本。

本章中的所有代码都可以从以下 **GitHub** 链接下载：

<https://github.com/PacktPublishing/Cross-Platform-Development-with-Qt-6-and-Modern-Cpp/tree/master/Chapter06>

重要音符

本章中的屏幕截图是在 **Windows** 计算机上拍摄的。根据您的机器上的底层平台，您将看到类似的屏幕。

## 了解 **QT** 信号和插槽

---

在 **GUI** 编程中，当用户使用任何 **UI** 元素执行任何操作时，应该更新另一个元素，或者应该完成某个任务。要实现这一点，我们需要对象之间的通信。例如，如果用户单击标题栏上的**Close**按钮，则预计窗口将关闭。不同的框架使用不同的方法来实现这种通信。回调是最常用的方法之一。回调是作为参数传递给另一个函数的函数。回调可能有多个缺点，并且在确保回调参数的类型正确性方面可能会遇到复杂问题。

在 **Qt** 框架中，我们有一个称为信号和槽的回调技术的替代品。信号是传递的消息，用于传达对象的状态已更改。该信号可以携带关于已经发生的改变的信息。槽是响应于特定信号而调用的特殊函数。因为插槽是功能，所以它们包含执行特定操作的逻辑。**Qt** 小部件有许多预定义的信号，但您始终可以扩展类并向其中添加您自己的信号。同样，您也可以添加自己的插槽来处理预期的信号。信号和插槽使观察者模式的实现变得容易，同时避免了样板代码。

为了能够通信，您必须连接相应的信号和插槽。让我们了解一下信号和插槽连接的连接机制和语法。

## 了解语法

要将信号连接到插槽，我们可以使用 `QObject::connect()`。这是一个线程安全函数。标准语法如下：

```
QMetaObject::Connection QObject::connect(
 const QObject *senderObject, const char *signalName,
 const QObject *receiverObject, const char *slotName,
 Qt::ConnectionType type = Qt::AutoConnection)
```

在前面的连接中，第一个参数是发送器对象，而下一个参数是来自发送器的信号。第三个参数是 **Receiver** 对象，而第四个参数是 **Slot** 方法。最后一个参数是可选的，它描述了要建立的连接类型。它确定通知是立即传递到插槽还是稍后排队。在 **Qt 6** 中有六种不同类型的连接。让我们来看一下连接类型：

- **Qt: : AutoConnection:** 这是默认的连接类型。此连接类型在信号发出时确定。如果发送方和接收方都在同一线程中，则使用 `Qt::DirectConnection`；否则使用 `Qt::QueuedConnection`。
- **Qt: : DirectConnection:** 在这种情况下，信号和槽都位于同一线程中。插槽在信号发出后立即被调用。
- **Qt: : QueuedConnection:** 在这个示例中，槽位于另一个线程中。一旦控制返回到接收器线程的事件循环，就会调用该槽。
- **Qt: : BlockingQueuedConnection:** 这类似于 `Qt::QueuedConnection`，不同之处在于表示信号线程阻塞，直到插槽返回。如果发送方和接收方都在同一线程中，则不能使用此连接以避免死锁。
- **Qt: : UniqueConnection:** 此可以使用 `bitwise OR` 与前面提到的

任何一种连接类型组合。这是用来避免重复连接的。如果连接已存在，则连接将失败。

- **Qt::SingleShotConnection:** In this case, the slot is called only once and the connection is disconnected once the signal is emitted. This can be also used with other connection types. This connection type was introduced in Qt 6.0.

### 重要音符

使用 `Qt::BlockingQueuedConnection` 时必须小心，以避免死锁。您正在向同一线程发送一个事件，然后锁定该线程，等待处理该事件。由于线程被阻塞，事件将永远不会被处理，并且线程将被永久阻塞，从而导致死锁。如果您知道自己在做什么，请使用此连接类型。在使用此连接类型之前，您必须了解这两个线程的实现细节。

有几种方法可以连接信号和插槽。在指定信号和槽函数时，必须分别使用 `SIGNAL()` 和 `SLOT()` 宏。最常用的语法如下：

```
QObject::connect(this, SIGNAL(signalName()),
 this, SLOT(slotName()));
```

这是自 Qt 开始以来一直存在的原始语法。然而，它的实现已经改变了好几次。在不破坏基本应用编程接口(**API**)的情况下添加了新功能。建议使用新的函数指针语法，如下所示：

```
connect(sender, &MyClass::signalName, this,
 &MyClass::slotName);
```

这两种语法各有优缺点。有关基于字符串的和基于函数式的连接之间的差异的更多信息，请访问

<https://doc.qt.io/qt-6/signalsandslots-syntaxes.html>

如果连接失败，则前面的语句返回 `false`。您还可以连接到函数器或 C++ 11 lambdas，如下所示：

```
connect(sender, &MyClass::signalName, this, [=]()
 { sender->doSomething(); });
```

您可以检查返回值，以验证信号是否成功连接到插槽。如果签名不兼容，或者信号和插槽丢失，连接可能会失败。

### 重要音符

`Qt::UniqueConnection` 不适用于 lambdas、非成员函数和函数器；它只能用于连接到成员函数。

信号和插槽的签名可以包含参数，并且这些参数可以具有默认值。如果信号至少具有与槽一样多的参数，并且相应参数的类型之间可能存在隐式转换，则可以将信号连接到槽。让我们看看具有不同参数数量的可行连接：

```
connect(sender, SIGNAL(signalName(int)), this,
 SLOT(slotName(int)));
connect(sender, SIGNAL(signalName(int)), this,
 SLOT(slotName()));
connect(sender, SIGNAL(signalName()), this,
 SLOT(slotName()));
```

但是，以下方法不起作用，因为槽中的参数比信号多：

```
connect(sender, SIGNAL(signalName()), this,
 SLOT(slotName(int)));
```

您建立的每个连接都会发出一个信号，因此重复连接会发出两个信号。您可以使用 `disconnect()` 断开连接。



您还可以将 Qt 与第三方信号/插槽机制一起使用。 如果要对同一项目使用这两种机制，请将以下配置添加到 Qt 项目(.pro)文件中：

```
CONFIG += no_keywords
```

让我们创建一个具有简单信号和插槽连接的示例。

## 声明信号和插槽

要创建信号和槽，必须在自定义的类中声明信号和槽。 该类的头文件将如下所示：

```
#ifndef MYCLASS_H
#define MYCLASS_H
#include <QObject>
class MyClass : public QObject
{
 Q_OBJECT
public:
 explicit MyClass(QObject *parent = nullptr);
signals:
 void signalName();
public slots:
 void slotName();
};
#endif // MYCLASS_H
```

如您所见，我们在类中添加了 `Q_OBJECT` 以促进信号和槽机制。 您可以在头文件中使用 `signals` 关键字声明一个信号，如前面的代码片段所示。 同样，可以使用关键字 `slots` 声明插槽。 信号和槽都可以有参数。 在本例中，我们对发送方和接收方使用相同的对象，以简化解释。 在大多数情况下，信号和插槽将位于不同的类别。

接下来，我们将讨论如何将信号连接到插槽。

## 将信号连接到插槽

以前，我们声明了一个自定义信号和槽。现在，让我们看看如何将它们连接起来。您可以定义信号和插槽连接，并在MyClass内部发出信号，如下所示：

```
#include "myclass.h"
#include <QDebug>
MyClass::MyClass(QObject *parent) : QObject(parent)
{
 QObject::connect(this, SIGNAL(signalName()),
 this, SLOT(slotName()));
 emit signalName();
}
void MyClass::slotName()
{
 qDebug()<< "Slot called!";
}
```

您需要在连接之后发出信号来调用插槽。在前面的示例中，我们使用了传统的信号和槽声明方式。您可以将连接替换为最新的语法，如下所示：

```
connect(this, &MyClass::signalName, this,
 &MyClass::slotName);
```

不仅可以将一个信号连接到一个插槽，还可以连接多个插槽和信号。类似地，许多信号可以连接到一个插槽。我们将在下一节中学习如何做到这一点。

## 将单个信号连接到多个插槽

您可以将相同信号连接到多个插槽。这些插槽的调用顺序与建立连接的顺序相同。让我们考虑名为signalX()的信号连接到名为slotA()、slotB()和slotC()的三个槽。当发出signalA()时，将调用所有三个插槽。

让我们来看看建立联系的传统方式：

```
QObject::connect(this, SIGNAL(signalX()),this,
 SLOT(slotA()));
QObject::connect(this, SIGNAL(signalX()),this,
 SLOT(slotB()));
QObject::connect(this, SIGNAL(signalX()),this,
 SLOT(slotC()));
```

您还可以按照新语法创建连接，如下所示：

```
connect(this, &MyClass:: signalX, this, &MyClass:: slotA);
connect(this, &MyClass:: signalX, this, &MyClass:: slotB);
connect(this, &MyClass:: signalX, this, &MyClass:: slotC);
```

在下一节中，我们将学习如何将多个信号连接到单个插槽。

## 将多个信号连接到单个插槽

在前面的部分中，您了解了如何在单个信号和多个插槽之间创建连接。现在，让我们看一下以下代码，以了解如何将多个信号连接到单个插槽：

```
QObject::connect(this, SIGNAL(signalX()),this,
 SLOT(slotX()));
QObject::connect(this, SIGNAL(signalY()),this,
 SLOT(slotX()));
QObject::connect(this, SIGNAL(signalZ()),this,
 SLOT(slotX()));
```

这里，我们使用了三个不同的信号，分别称为`signalX()`、`signalY()`和`signalZ()`，但是有一个定义为`slotX()`的。当这些信号中的任何一个发出时，就会调用该插槽。

在下一节中，我们将学习如何将一个信号连接到另一个信号。

## 将一个信号连接到另一个信号

有时，您可能需要转发信号，而不是直接连接到插槽。您可以将一个信号连接到另一个信号，如下所示：

```
connect(sender, SIGNAL(signalA()),forwarder,
 SIGNAL(signalB())));
```

您还可以根据新语法创建连接，如下所示：

```
connect(sender,&ClassName::signalA,forwarder,&ClassName::
 signalB);
```

在前面的行中，我们已经将 `signalA()` 连接到 `signalB()`。因此，当发出 `signalA()` 时，也会发出 `signalB()`，并调用与 `signalB()` 连接的相应插槽。让我们考虑一下，我们的 GUI 中有一个按钮，我们希望将按钮单击作为不同的信号转发。下面的代码片段显示了如何转发信号：

```
#include <QWidget>
class QPushButton;
class MyClass : public QWidget
{
 Q_OBJECT
public:
 MyClass(QWidget *parent = nullptr);
 ~MyClass();
signals:
 void signalName();
private:
 QPushButton *myButton;
};
MyClass::MyClass(QWidget *parent)
 : QWidget(parent)
{
 myButton = new QPushButton(this);
 connect(myButton, &QPushButton::clicked,
 this, &MyClass::signalName);
}
```

在前面的示例中，我们将按钮单击信号转发到自定义信号。如前所

述，我们可以调用连接到自定义信号的插槽。

在本节中，我们学习了如何建立连接以及如何使用信号和插槽。现在，您可以在不同的班级之间通信并共享信息。在下一节中，我们将了解信号和槽背后的工作机制。

## QT 信号和槽的工作机理

---

在前面的小节中，我们了解了信号和插槽语法以及如何连接它们。现在，我们将了解它是如何工作的。

在创建连接时，Qt 查找信号和插槽的索引。Qt 使用查找字符串表来查找相应的索引。然后，创建一个 `QObjectPrivate::Connection` 对象并将其添加到内部链表中。由于一个信号可以连接到多个插槽，因此每个信号可以具有连接的插槽列表。每个连接都包含接收方的名称和插槽的索引。每个对象都有一个连接向量，该连接向量与 `QObjectPrivate::Connection` 的链表中的每个信号相关联。

下图说明了 `ConnectionList` 如何在发送方和接收方对象之间创建连接：

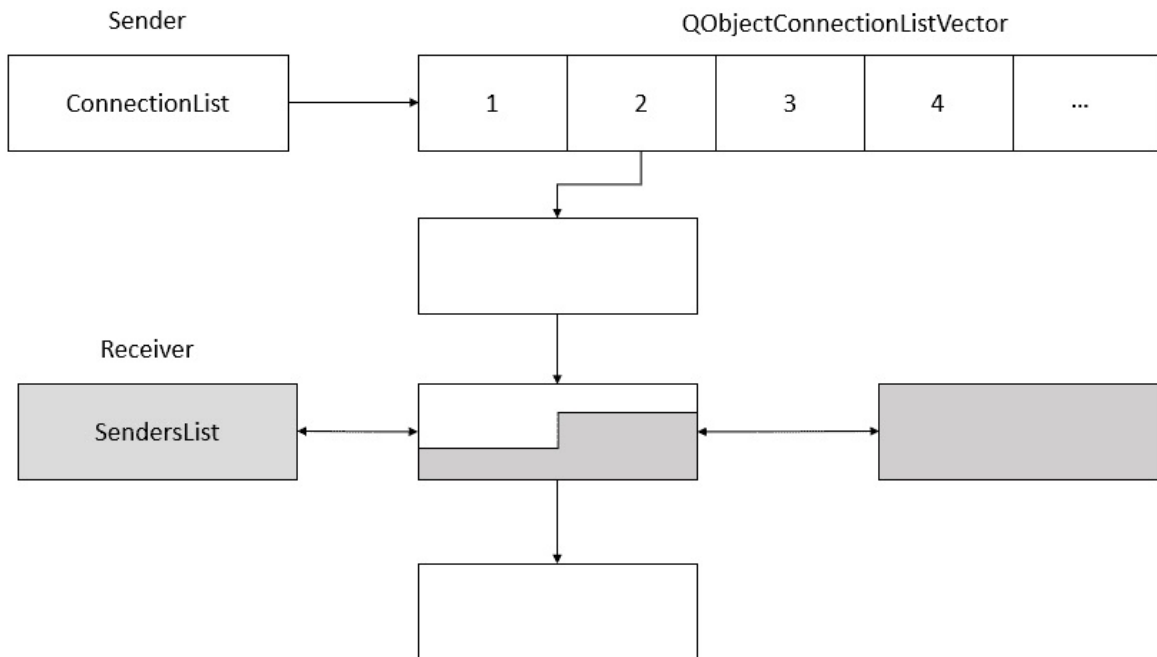


图 6.1-发送方和接收方之间的连接机制图示

`ConnectionList` 是一个单链表，其中包含与对象之间的所有连接。  
`signalVector` 包含给定信号的连接列表。每个 `Connection` 也是个发送者个链表的一部分。使用链表是因为它们允许更快地添加和删除对象。每个对象还具有对象连接到的反向连接列表，以便自动删除。有关内部实施的详细信息，请参阅最新的 `qobject_p.h`。

在 *Woboq* 网站上有相当多的文章介绍 **Signal** 和插槽是如何工作的。您也可以在 *woboq* 网站上浏览 **Qt** 源代码。如果您需要更多信息，请访问以下链接：

<https://woboq.com/blog/how-qt-signals-slots-work.html>.

现在，让我们了解一下 **Qt** 的元对象系统。

## Qt 的元对象系统

**Qt** 的元对象系统是信号和槽机制背后的核心机制。它提供对象间通

信、动态属性系统和运行时类型信息等功能。

元对象系统由三部分机制实现。这些机制如下：

- `QObject`
- `Q_OBJECT` 宏
- 元对象编译器

`QObject`类是所有 Qt 对象的基类。这是一个非常强大的机制，它促进了信号和插槽机制。`QObject`类为可以利用元对象系统的对象提供基类。`QObject`派生类将自身排列在对象树中。这将在类之间创建父子关系。当您使用另一个`QObject`派生类作为父对象创建`QObject`派生类时，该对象将自动添加到父对象的`children()`列表中。父级取得对象的所有权。GUI 编程既需要运行时效率，又需要高度的灵活性。Qt 通过结合 C++ 的速度和 Qt 对象模型的灵活性实现了这一点。Qt 通过实现基于 `QObject` 继承的标准 C++ 技术来提供所需的功能。

您可以通过以下链接了解有关 Qt 对象模型的更多信息：

<https://doc.qt.io/qt-6/object.html>.

`Q_OBJECT`宏出现在类声明的私有部分中。它用于启用 Qt 的元对象系统提供的信号、槽和其他服务。

元对象编译器(**moc**)为`QObject`个派生类生成实现元对象特性的附加代码。它提供了在运行时自省对象的功能。默认情况下，C++ 不支持自省。因此，Qt 创建了**moc**。它是一个处理 Qt 的 C++ 扩展的代码生成程序。该工具读取 C++ 头文件，如果找到`Q_OBJECT`宏，则使用元目标代码创建另一个 C++ 源文件。生成的文件包含自省所需的代码。这两个文件都被编译并链接在一起。除了为对象之间的通信提供信号和



槽机制外，元对象代码还提供了几个附加功能来查找类名和继承详细信息，并且还有助于在运行时设置属性。Qt 的 `moc` 提供了一种超越编译语言功能的干净方法。

您可以通过对 `QObject` 派生类使用 `qobject_cast()` 来执行类型强制转换。`qobject_cast()` 函数的作用类似于标准 C++ `dynamic_cast()`。优点是它不需要运行时类型标识(RTTI)，并且可以跨动态库边界工作。您可以从 `QObject` 派生一个类，但是如果不添加 `Q_OBJECT` 宏，那么信号和槽以及其他元对象系统特性将不可用。没有元代码的 `QObject` 派生类等同于包含元对象代码的最接近的祖先类。还有一个更轻量级的 `Q_OBJECT` 宏版本，称为 `Q_GADGET`，可用于利用 `QMetaObject` 提供的一些功能。使用 `Q_GADGET` 的类没有信号或槽。

我们在这里看到了一些新的关键字，比如 `Q_OBJECT`、`signals`、`slots`、`emit`、`SIGNAL` 和 `SLOT`。这些被称为 C++ 的 Qt 扩展。它们是非常简单的宏，可以由 `qobjectdefs.h` 中定义的 `moc` 看到。其中，`emit` 是 `moc` 未解析的空宏。保留它只是为了给开发者一些提示。

您可以了解为什么 Qt 在 <https://doc.qt.io/qt-6/why-moc.html> 将 `moc` 用于信号和插槽。

在本节中，我们了解了 Qt 的元对象系统。在下一节中，我们将讨论 `moc` 生成的代码，并讨论一些底层实现。

## MOC 生成的代码

在本节中，我们将查看 Qt6 中 `moc` 生成的代码。当您构建前面的信号和槽示例时，您将在 `build` 目录下看到生成的文件：`moc_myclass.cpp` 和 `moc_predefs.h`。让我们使用文本编辑器打开 `moc_myclass.cpp` 文件：

```

#include <memory>
#include "../SignalSlotDemo/myclass.h"
#include <QtCore/qbytearray.h>
#include <QtCore/qmetatype.h>
#if !defined(Q_MOC_OUTPUT_REVISION)
#error "The header file 'myclass.h' doesn't include
 <QObject>."
#elif Q_MOC_OUTPUT_REVISION != 68
#error "This file was generated using the moc from 6.0.2\
 It"
#error "cannot be used with the include files from this
 version of Qt."
#error "(The moc has changed too much.)"
#endif

```

您可以在文件顶部看到有关 **Qt** 元对象编译器版本的信息。请注意，重新编译项目时，在此文件中所做的所有更改都将丢失。因此，不要修改此文件中的任何内容。我们正在看这份文件，以了解其工作机制。

让我们看一下 `QMetaObject` 的整数数据。如您所见，这里有两列；第一列是计数，第二列是此数组中的索引：

```

static const uint qt_meta_data_MyClass[] = {
 // content:
 9, // revision
 0, // classname
 0, 0, // classinfo
 2, 14, // methods
 0, 0, // properties
 0, 0, // enums/sets
 0, 0, // constructors
 0, // flags
 1, // signalCount
 // signals: name, argc, parameters, tag, flags, initial
 // metatype offsets
 1, 0, 26, 2, 0x06, 0 /* Public */,
 // slots: name, argc, parameters, tag, flags, initial
 // metatype offsets
 3, 0, 27, 2, 0x0a, 1 /* Public */,
 // signals: parameters
 QMetaType::Void,

```

```
// slots: parameters
 QMetaType::Void,
 0 // eod
};
```

在本例中，我们有一个方法，该方法的描述从索引 **14** 开始。您可以在 `signalCount` 中找到可用信号的数量。对于每个函数，`moc` 还将每个参数的返回类型、它们的类型以及它们的索引保存到名称中。在每个元对象中，方法都有一个索引，从 **0** 开始。它们被安排为信号，然后是插槽，然后是其他功能。这些索引是相对索引，不包括父对象的索引。

当您进一步查看代码时，您会发现 `MyClass::metaObject()` 函数。对于动态元对象，此函数返回 `QObject::d_ptr->dynamicMetaObject()`。函数 `metaObject()` 通常返回类的 `taticMetaObject`：

```
const QMetaObject *MyClass::metaObject() const
{
 return QObject::d_ptr->metaObject
? QObject::d_ptr->dynamicMetaObject()
: &staticMetaObject;
}
```

当传入的字符串数据与当前类匹配时，必须将此指针转换为空指针并将其传递给外部世界。如果不是当前类，则调用父类的 `qt_metacast()` 继续查询：

```
void *MyClass::qt_metacast(const char *_cname)
{
 if (!_cname) return nullptr;
 if (!strcmp(_cname,
 qt_meta_stringdata_MyClass.stringdata0))
 return static_cast<void*>(this);
 return QObject::qt_metacast(_cname);
}
```

Qt 的 Meta-Object 系统使用 `qt_metacall()` 函数来访问特定 `QObject` 对象的元信息。当我们发出信号时，会调用 `qt_metacall()`，然后再调用实际信号函数：

```
int MyClass::qt_metacall(QMetaObject::Call _c, int _id, void **_a)
{
 _id = QObject::qt_metacall(_c, _id, _a);
 if (_id < 0)
 return _id;
 if (_c == QMetaObject::InvokeMetaMethod) {
 if (_id < 2)
 qt_static_metacall(this, _c, _id, _a);
 _id -= 2;
 } else if (_c == QMetaObject::
 RegisterMethodArgumentMetaType) {
 if (_id < 2)
 *reinterpret_cast<QMetaType *>(_a[0]) =
 QMetaType();
 _id -= 2;
 }
 return _id;
}
```

当您调用一个信号时，它会调用 `moc` 生成的代码，该代码在内部调用 `QMetaObject::activate()`，如下面的代码片段所示。然后，`QMetaObject::activate()` 查看内部数据结构，以找出连接到该信号的插槽。

您可以在 `qobject.cpp` 中找到该函数的详细实现：

```
void MyClass::signalName()
{
 QMetaObject::activate(this, &staticMetaObject, 0,
 nullptr);
}
```

通过这样做，您可以探索完整的生成代码并进一步查看符号。现在，

让我们看一下调用插槽的moc生成的代码。插槽由其在qt\_static\_metacall函数中的索引调用，如下所示：

```
void MyClass::qt_static_metacall(QObject *_o,
 QMetaObject::Call _c, int _id, void **_a)
{
 if (_c == QMetaObject::InvokeMetaMethod) {
 auto *_t = static_cast<MyClass *>(_o);
 (void)_t;
 switch (_id) {
 case 0: _t->signalName(); break;
 case 1: _t->slotName(); break;
 default: ;
 }
 } else if (_c == QMetaObject::IndexOfMethod) {
 int *result = reinterpret_cast<int *>(_a[0]);
 {
 using _t = void (MyClass::*)();
 if (*reinterpret_cast<_t *>(_a[1]) ==
 static_cast<_t*>(&MyClass::signalName)) {
 *result = 0;
 return;
 }
 }
 }
 (void)_a;
}
```

指向参数的数组指针与信号的格式相同。\_a[0]未被触及，因为此处的所有内容都返回 void：

```
bool QObject::isSignalConnected(const QMetaMethod &signal) const
```

如果信号连接到至少一个接收器，则返回true；否则返回false。

当一个对象被销毁时，将迭代一个QObjectPrivate::senders列表，并将所有Connection::receiver设置为0。此外，将Connection::receiver->connectionLists->dirty设置为true。还会迭

代每个 `QObjectPrivate::connectionLists` 以删除发送者列表中的连接。

在这一节中，我们浏览了 `moc` 生成的代码的某些部分，并理解了信号和槽背后的工作机制。在下一节中，我们将了解 Qt 的属性系统。

## 了解 QT 的产权制度

Qt 的属性系统与其他一些编译器供应商类似。但是，它提供了跨平台的优势，可以在不同平台上使用 Qt 支持的标准编译器。若要添加属性，必须将 `Q_PROPERTY()` 宏添加到 `QObject` 派生类。此属性的作用类似于类数据成员，但它提供了可通过元对象系统使用的额外功能。简单的语法如下所示：

```
Q_PROPERTY(type variableName READ getterFunction
 WRITE setterFunction NOTIFY signalName)
```

在前面的语法中，我们使用了一些最常见的参数。但是语法中支持的参数还有更多。您可以通过阅读 Qt 文档找到更多信息。让我们来看一下下面的代码片段，它使用了 `MEMBER` 参数：

```
Q_PROPERTY(QString text MEMBER m_text NOTIFY
 textChanged)
signals:
 void textChanged(const QString &newText);
private:
 QString m_text;
```

在前面的代码片段中，我们使用 `MEMBER` 关键字将成员变量导出为 Qt 属性。这里的类型是 `QString`，`NOTIFY` 信号用于实现 QML 属性绑定。

现在，让我们探索如何使用元对象系统读写属性。

## 使用元对象系统读取和写入属性

让我们创建一个名为`MyClass`的类，它是`QWidget`的子类。让我们将`Q_OBJECT`宏添加到其私有部分，以启用属性系统。在本例中，我们希望在`MyClass`中创建一个属性来跟踪版本的值。属性的名称将是`version`，其类型将是`QString`，这在`MyClass`中定义。让我们看一下下面的代码片段：

```
class MyClass : public QWidget
{
 Q_OBJECT
 Q_PROPERTY(QString version READ version WRITE
 setVersion NOTIFY versionChanged)
public:
 MyClass(QWidget *parent = nullptr);
 ~MyClass();
 void setVersion(QString version)
 {
 m_version = version;
 emit versionChanged(version);
 }
 QString version() const { return m_version; }
signals:
 void versionChanged(QString version);
private:
 QString m_version;
};
```

要获得属性更改通知，您必须在`version`值更改后发出`versionChanged()`。

让我们看一下前面示例的`main.cpp`文件：

```
int main(int argc, char *argv[])
{
 QApplication a(argc, argv);
 MyClass myClass;
 myClass.setVersion("v1.0");
 myClass.show();
 return a.exec();
}
```



在前面的代码片段中，该属性是通过调用 `setVersion()` 来设置的。您可以看到，每次更改版本时都会发出 `versionChanged()` 信号。

还可以使用 `QObject::property()` 读取属性，并使用 `QObject::setProperty()` 写入属性。您还可以使用 `QObject::property()` 查询动态属性，类似于编译时 `Q_PROPERTY()` 声明。

您还可以按如下方式设置该属性：

```
QObject *object = &myClass;
object->setProperty("version", "v1.0");
```

在这一部分，我们讨论了产权制度。在下一节中，我们将了解 Qt Designer 中的信号和插槽。

## 在 Qt Designer 中使用信号和插槽

---

如果您正在使用 Qt 小部件模块，则可以使用 QtDesigner 编辑表单中的信号和插槽连接。Qt 默认小部件带有许多信号和槽。让我们看看我们如何在 Qt Designer 中实现信号和槽，而无需编写任何代码。

您可以将 **Dial** 控件和 **Slider** 控件拖到窗体上。您可以通过底部选项卡上的信号和插槽编辑器添加连接，如以下屏幕截图所示：

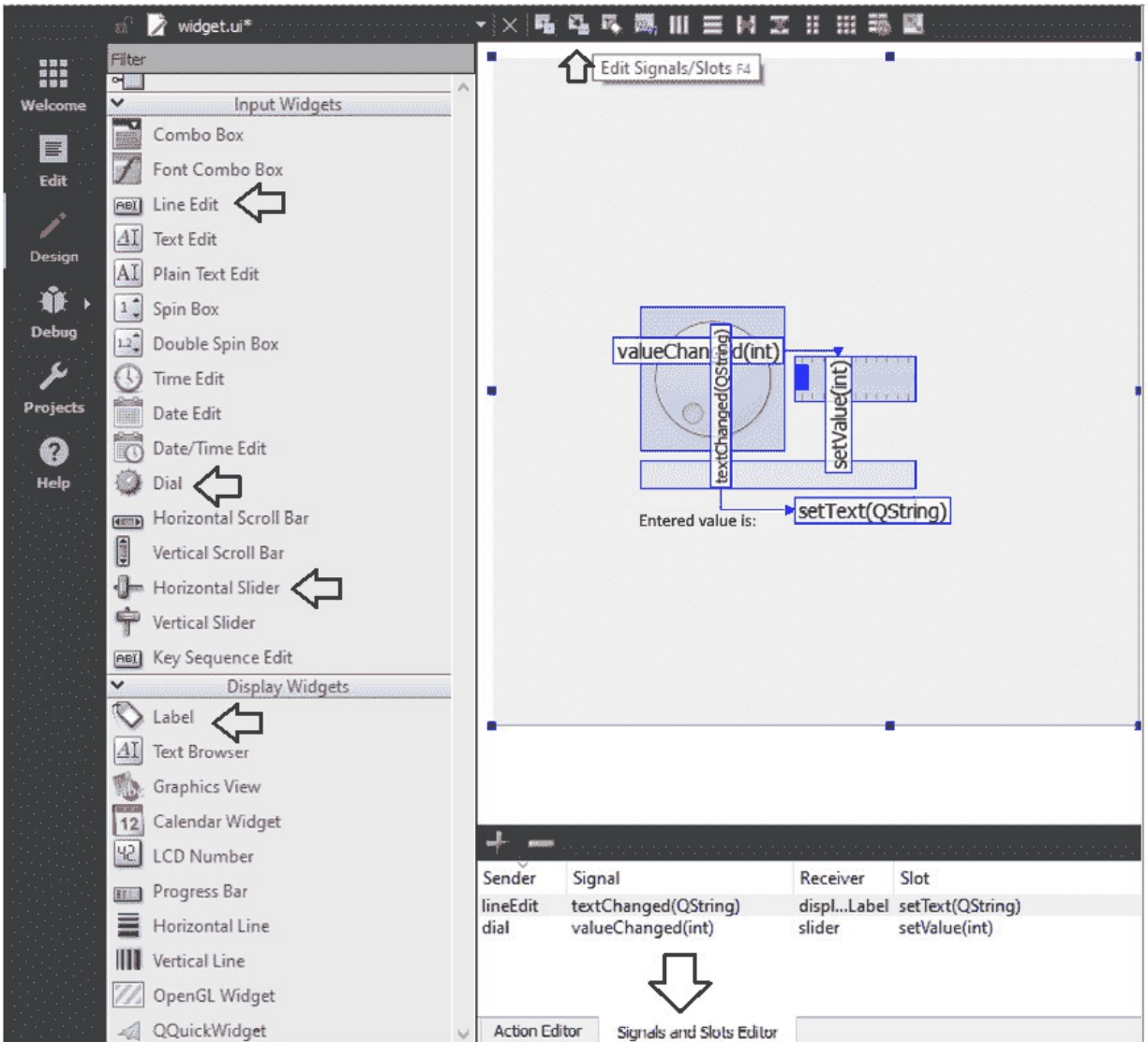


图 6.2-使用 Qt Designer 创建信号和插槽连接

或者，您可以按 **F4** 或从顶部工具栏中选择编辑信号/插槽按钮。然后，您可以选择控件，并通过拖动连接到接收器来创建连接。如果您有为您的自定义类定义的自定义信号或插槽，它们将自动显示在信号和插槽编辑器中。然而，大多数开发人员更喜欢在 **C++** 源文件中定义连接。

在本节中，我们讨论了使用 **Qt Designer** 在 **Qt** 小部件中实现信号和槽。现在，让我们看看如何在 **QML** 中处理信号。

# 了解 QML 中的信号和处理程序事件系统

在此之前，我们了解了如何连接 C++ 源文件中的信号和插槽，并将它们与 Qt 小部件模块一起使用。现在，让我们看看如何使用 QML 进行通信。QML 有一个信号和处理程序机制，类似于信号和槽。在 QML 文档中，信号是一个事件，信号通过信号处理程序响应。与 C++ 中的槽类似，在 QML 中发出信号时会调用信号处理程序。在 Qt 术语中，方法是连接到信号的槽；QML 中定义的所有方法都被创建为 Qt 槽。因此，QML 中没有单独的槽声明。信号是对象发出的某个事件已发生的通知。您可以将逻辑放在 JavaScript 或响应信号的方法中。

让我们看看如何编写信号处理程序。您可以按如下方式声明信号处理程序：

```
onSignalName : {
 //Logic
}
```

这里，`signalName` 是信号的名称。信号名称的第一个字母在编写处理程序时大写。因此，这里的信号处理程序被命名为 `onSignalName`。信号和信号处理程序应该在同一对象中定义。信号处理程序内部的逻辑是一段 JavaScript 代码。

例如，当用户在鼠标区域内点击时，发出 `clicked()` 信号。要处理 `clicked()` 信号，我们必须添加 `onClicked:{...}` 信号处理程序。

信号处理程序是在发出关联信号时由 QML 引擎调用的简单函数。当您向 QML 对象添加信号时，Qt 会自动向对象定义添加相应的信号处理程序。

让我们从向 QML 文档添加自定义信号开始。

## 在 QML 中添加信号

要在 QML 类中添加信号，必须使用关键字 `signal`。定义新信号的语法如下：

```
signal <name>([(<type> <parameter name>[...])])
```

下面是一个这样的示例：

```
signal composeMessage(string message)
```

可以使用或不使用任何参数来声明信号。如果没有为信号声明参数，则可以保留 `()` 方括号。您可以通过将其作为函数调用来发出信号：

```
Rectangle {
 id: mailBox
 signal composeMessage(string message)
 anchors.fill: parent
 Button {
 id: sendButton
 anchors.centerIn: parent
 width: 100
 height: 50
 text: "Send"
 onClicked: mailBox.composeMessage("Hello World!")
 }
 onComposeMessage: {
 console.log("Message Received", message)
 }
}
```

在前面的示例中，我们向 QML 文件添加了一个自定义信号 `composeMessage()`。我们使用了相应的信号处理程序，即 `onComposeMessage()`。然后，我们添加了一个按钮，该按钮在单击时会发出 `composeMessage()` 信号。运行此示例时，您将看到单击该按钮时会自动调用信号处理程序。

在本节中，您了解了如何声明信号以及如何实现相应的信号处理程序。在下一节中，我们将把信号连接到一个函数。

## 将信号连接到函数

您可以将信号连接到 **QML** 文档中定义的任何函数。您可以使用将一个信号连接到一个函数或另一个信号。当信号连接到函数时，无论何时发出信号，都会自动调用该函数。此机制使信号能够由函数接收，而不是由信号处理程序接收。

在以下代码片段中，使用 `connect()` 函数将 `composeMessage()` 信号连接到 `transmitMessage()` 函数：

```
Rectangle {
 id: mailBox
 signal composeMessage(string message)
 anchors.fill: parent
 Text {
 id: textElement
 anchors {
 top: parent.top
 left: parent.left
 right: parent.right
 }
 width: 100
 height: 50
 text: ""
 horizontalAlignment: Text.AlignHCenter
 }
 Component.onCompleted: {
 mailBox.composeMessage.connect(transmitMessage)
 mailBox.composeMessage("Hello World!")
 }
 function transmitMessage(message) {
 console.log("Received message: " + message)
 textElement.text = message
 }
}
```

在 QML 中，信号处理使用以下语法实现：

```
sender.signalName.connect(receiver.slotName)
```

您也可以使用 `disconnect()` 函数删除连接。您可以像这样断开连接：

```
sender.signalName.disconnect(receiver.slotName)
```

现在，我们来看看如何在 QML 中转发信号。

## 将信号连接到信号

您可以将信号连接到 QML 中的另一个信号。您可以使用 `connect()` 函数来实现这一点。

让我们通过查看以下示例来了解如何做到这一点：

```
Rectangle {
 id: mailBox
 signal forwardButtonClick()
 anchors.fill: parent
 Button {
 id: sendButton
 anchors.centerIn: parent
 width: 100
 height: 50
 text: "Send"
 }
 onForwardButtonClick: {
 console.log("Forwarded Button Click Signal!")
 }
 Component.onCompleted: {
 sendButton.clicked.connect(forwardButtonClick)
 }
}
```

在前面的示例中，我们将 `clicked()` 信号连接到 `forwardButtonClick()` 信号。您可以在 `onForwardButtonClick()` 信号处理程序内部的根级别实现

必要的逻辑。您还可以从按钮单击处理程序发出信号，如下所示：

```
onClicked: {
 mailBox.forwardButtonClick()
}
```

在本节中，我们讨论了如何连接并处理两个信号。在下一节中，我们将讨论如何使用信号和槽在 **C++** 类和 **QML** 之间通信。

## 定义属性属性并了解属性绑定

以前，我们学习了如何通过注册类的 `Q_PROPERTY` 来定义 **C++** 中的类型，然后将其注册到 **QML** 类型系统。还可以在 **QML** 文档中创建自定义属性。属性绑定是 **QML** 的核心特性，它允许我们在各种对象属性之间创建关系。您可以使用以下语法在 **QML** 文档中声明属性：

```
[default] property <propertyType> <propertyName> : <value>
```

通过这种方式，您可以将特定参数公开给外部对象或更有效地维护内部状态。让我们看一下下面的属性声明：

```
property string version: "v1.0"
```

当您声明自定义属性时，**Qt** 会隐式创建该属性的属性更改信号。关联的信号处理程序是 `on<PropertyName>Changed`，其中 `<PropertyName>` 是属性的名称，第一个字母大写。对于前面声明的属性，关联的信号处理程序是 `onVersionChanged`，如下所示：

```
onVersionChanged:{...}
```

如果为该属性分配了静态值，则在显式为其分配新值之前，该属性将保持不变。要动态更新这些值，应该在 **QML** 文档内使用属性绑定。我们早先使用了简单的属性绑定，如以下代码片断所示：

```
width: parent.width
```

但是，我们可以将其与后端 C++ 类公开的属性结合起来，如下所示：

```
property string version: myClass.version
```

在前一行中，`myClass` 是向 QML 引擎注册的后端 C++ 对象。在这种情况下，无论何时从 C++ 端发出 `versionChanged()` `CHANGE` 信号，QML `version` 属性都会自动更新。

接下来，我们将讨论如何在 C++ 和 QML 之间集成信号和槽。

## 在 C++ 和 QML 之间集成信号和插槽

在 C++ 中，要与 QML 层交互，可以使用信号、槽和 `Q_INVOKABLE` 函数。也可以使用 `Q_PROPERTY` 宏来创建特性。要响应来自对象的信号，可以使用 `Connections` `QML` 类型。当 C++ 文件中的属性发生更改时，`Q_PROPERTY` 会自动更新这些值。如果该属性与任何 QML 属性有绑定，它将自动更新 QML 内的属性值。在这种情况下，信号插槽机制是自动建立的。

让我们看一下使用上述机制的以下示例：

```
class CPPBackend : public QObject
{
 Q_OBJECT
 Q_PROPERTY(int counter READ counter WRITE setCounter
 NOTIFY counterChanged)
public:
 explicit CPPBackend(QObject *parent = nullptr);
 Q_INVOKABLE void receiveFromQml();
 int counter() const;
 void setCounter(int counter);
signals:
 void sendToQml(int);
 void counterChanged(int counter);
```



```
private:
 int m_counter = 0;
};
```

在前面的代码中，我们声明了一个基于 `Q_PROPERTY` 的通知。当发出 `counterChanged()` 信号时，我们可以得到新的 `counter` 值。但是，我们将 `receiveFromQml()` 函数用作 `Q_INVOKABLE` 函数，以便可以在 QML 文档中直接调用它。我们发出的是 `sendToQml()`，它在 `main.qml` 内部处理：

```
void CPPBackend::setCounter(int counter)
{
 if (m_counter == counter)
 return;
 m_counter = counter;
 emit counterChanged(m_counter);
}
void CPPBackend::receiveFromQml()
{
 // We increase the counter and send a signal with new
 // value
 ++ m_counter;
 emit sendToQml(m_counter);
}
```

现在，让我们看看 QML 实现：

```
Window {
 width: 640
 height: 480
 visible: true
 title: qsTr("C++ QML Signals & Slots Demo")
 property int count: cppBackend.counter
 onCountChanged:{
 console.log("property is notified. Updated value
 is:",count)
 }
 Connections {
 target: cppBackend
 onSendToQml: {
```

```

 labelCount.text = "Fetched value is "
 + cppBackend.counter
 }
}
Row{
 anchors.centerIn: parent
 spacing: 20
 Text {
 id: labelCount
 text: "Fetched value is " + cppBackend.counter
 }
 Button {
 text: qsTr("Fetch")
 width: 100 ;height: 20
 onClicked: {
 cppBackend.receiveFromQml()
 }
 }
}
}

```

在前面的示例中，我们使用 `Connections` 连接到 C++ 信号。单击按钮时，我们将调用 `receiveFromQml()` C++ 函数，我们将在该函数中发出信号。我们还声明了 `count` 属性，它也侦听 `counterChanged()`。我们在关联的信号处理程序中处理数据；即 `onCountChanged`。我们还可以根据通知更新 `labelCount` 数据：

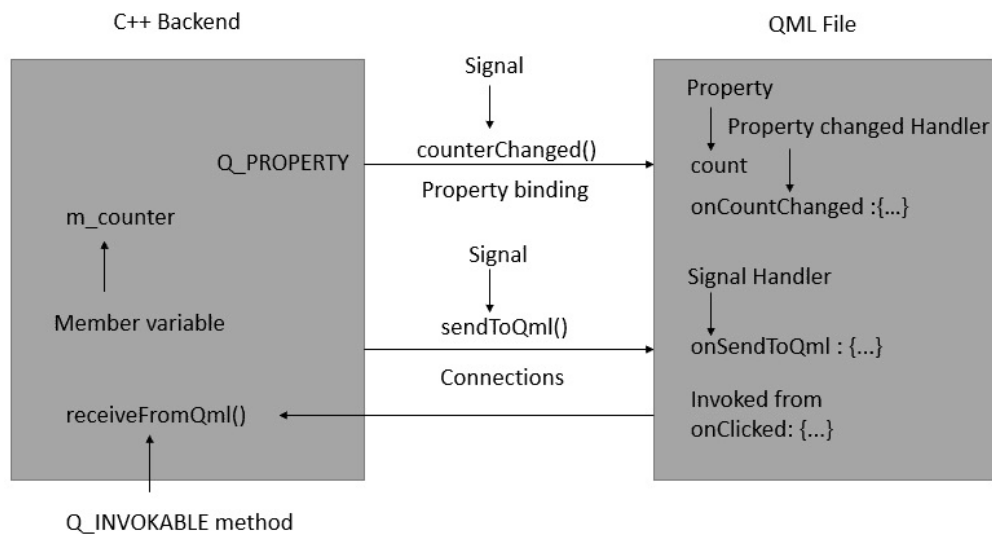


图 6.3-本例中使用的机制

上面的图说明了本例中的通信机制。出于解释目的，我们在同一示例中保留了多种方法来解释 C++ 和 QML 之间的通信机制。

在本节中，您通过示例了解了信号和插槽机制。在下一节中，我们将了解事件和 Qt 中的事件循环。

## 了解事件和事件循环

---

Qt 是一个基于事件的系统，所有的 GUI 应用都是事件驱动的。在事件驱动的应用中，通常有一个主循环侦听事件，然后在检测到其中一个事件时触发回调函数。事件可以是自发的，也可以是合成的。自发事件来自外部环境。合成事件是由应用生成的自定义事件。Qt 中的事件是表示已发生的事情的通知。Qt 事件是从 `QEvent` 派生的值类型，`QEvent` 为每个事件提供一个类型枚举。Qt 应用内发生的所有事件都封装在从 `QEvent` 类继承的对象中。所有 `QObject` 派生类都可以重写 `QObject::event()` 函数，以便处理其实例所针对的事件。事件可以来自应用内部和外部。

当事件发生时，Qt 通过构造适当的 `QEvent` 子类实例来生成事件对象，然后通过调用 `event()` 函数将其传递给 `QObject` 的特定实例。与信号和插槽机制不同，在信号和插槽机制中，连接到信号的插槽通常立即执行，而事件必须等待轮到它，直到事件循环调度所有较早到达的事件。您必须根据您的预期实现选择正确的机制。下图说明了如何在事件驱动的应用中创建和管理事件：

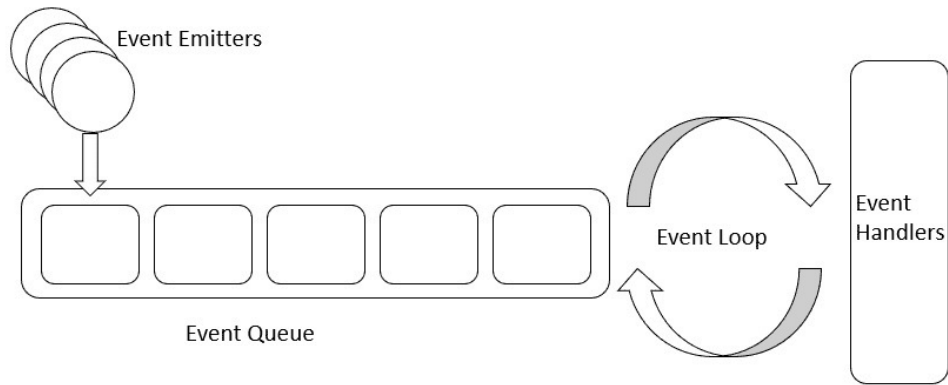


图 6.4-使用事件循环的事件驱动应用的图示

我们可以通过调用 `QCoreApplication::exec()` 进入 Qt 的主事件循环。应用将一直运行，直到调用 `QCoreApplication::exit()` 或 `QCoreApplication::quit()`，从而终止循环。`QCoreApplication` 可以处理 GUI 线程中的每个事件，并将事件转发到 `QObject`s。请注意，事件不会立即传递；相反，它们会在事件队列中排队，然后一个接一个地稍后处理。事件分派器循环通过该队列，将它们转换为 `QEvent` 对象，然后将事件分派到目标 `QObject`。

简化的事件循环调度器可能如下所示：

```
while(true)
{
 dispatchEventsFromQueue();
 waitForEvents();
}
```

与事件循环相关的一些重要的 Qt 类如下所示：

- **QAbstractEventDispatcher** 被子类化以管理 Qt 的 `event` 队列。
- **QEventLoop** 提供本地 `event` 循环。
- **QCoreApplication** 为非基于 GUI 的应用提供 `event` 循环。
- **QGuiApplication** 包含基于 GUI 的应用的主 `event` 循环。

- **QThread**用于创建自定义线程和管理线程。
- **QSocketNotifier**用于监视文件描述符的活动。
- **QTimer**用于在具有`event`循环的任何线程中创建计时器。

您可以在 Qt 文档中阅读有关这些类的内容。以下链接提供了对事件系统的进一步了解：

[https://wiki.qt.io/Threads\\_Events\\_QObjects](https://wiki.qt.io/Threads_Events_QObjects).

在本节中，我们讨论了事件和 Qt 的事件循环。在下一节中，我们将学习如何使用事件过滤器过滤事件。

## 使用事件过滤器管理事件

在本节中，您将学习如何管理事件以及如何筛选特定事件并执行任务。您可以通过重新实现事件处理程序并安装事件筛选器来实现事件筛选。您可以通过将感兴趣的小部件子类化并重新实现该事件处理程序来重新定义事件处理程序应该执行的操作。

Qt 提供了五种不同的事件处理方法，如下所示：

- 重新实现特定的事件处理程序，如`paintEvent()`
- 重新实现`QObject::event()`功能
- 在`QObject`实例上安装事件筛选器
- 在`QApplication`实例上安装事件筛选器
- 子类化`QApplication`并重新实现`notify()`

下面的代码处理鼠标左键在自定义构件上的单击，同时将所有其他按钮单击传递给基类`QWidget`：

```
void MyClass::mousePressEvent(QMouseEvent *event)
{
```

```

 if (event->button() == Qt::LeftButton)
 {
 // Handle left mouse button here
 }
 else
 {
 QWidget::mousePressEvent(event);
 }
}

```

在上一个示例中，我们只过滤了左按钮按下事件。您可以在相应的块中添加所需的操作。下图说明了高级事件处理机制：

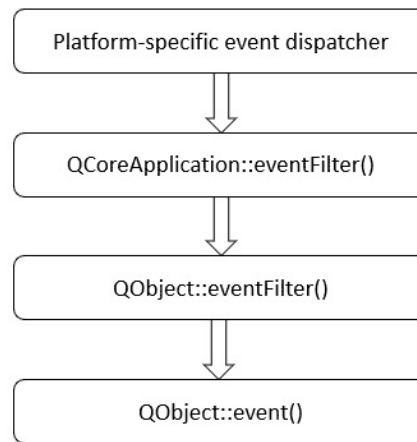


图 6.5-事件过滤器机制的图示

事件过滤器既可以安装在应用实例上，也可以安装在本地对象上。如果事件筛选器安装在 `QCoreApplication` 对象中，则所有事件都将通过此事件筛选器。如果它安装在 `QObject` 派生类中，则已发送到该对象的事件将通过事件筛选器。有时，可能没有任何合适的 Qt 事件类型可用于特定操作。在这种情况下，您可以通过从 `QEvent` 创建子类来创建自定义事件。您可以重新实现 `QObject::event()` 来过滤预期的事件，如下所示：

```

#include <QWidget>
#include <QEvent>

```

```

class MyCustomEvent : public QEvent
{
public:
 static const QEvent::Type MyEvent
 = QEvent::Type(QEvent::User + 1);
};
class MyClass : public QWidget
{
 Q_OBJECT
public:
 MyClass(QWidget *parent = nullptr);
 ~MyClass();
protected:
 bool event(QEvent *event);
};

```

在这里，我们创建了一个名为MyCustomEvent的自定义事件类，并创建了一个自定义类型。

现在，让我们通过重新实现event()来过滤这些事件：

```

bool MyClass::event(QEvent *event)
{
 if (event->type() == QEvent::KeyPress)
 {
 QKeyEvent *keyEvent= static_cast<QKeyEvent
 *>(event);
 if (keyEvent->key() == Qt::Key_Enter)
 {
 // Handle Enter event event
 return true;
 }
 }
 else if (event->type() == MyCustomEvent::MyEvent)
 {
 MyCustomEvent *myEvent = static_cast<MyCustomEvent
 *>(event);
 // Handle custom event
 return true;
 }
 return QWidget::event(event);
}

```

如您所见，我们已将其他事件传递给 `QWidget::event()` 进行进一步处理。如果要阻止事件进一步传播，则为 `return true`；否则为 `return false`。

事件筛选器是接收发送到对象的所有事件的对象。筛选器可以停止事件，也可以将其转发给对象。如果对象已安装为监视对象的事件过滤器，则它会筛选事件。还可以通过使用事件过滤器并执行必要的任务来监视另一个对象的事件。下面的示例显示如何使用事件过滤器方法重新实现最常用的事件之一-按键事件。

让我们来看一下下面的代码片段：

```
#include <QMainWindow>
class QTextEdit;
class MainWindow : public QMainWindow
{
 Q_OBJECT
public:
 MainWindow(QWidget *parent = nullptr);
 ~MainWindow();
protected:
 bool eventFilter(QObject *obj, QEvent *event) override;
private:
 QTextEdit *textEdit;
};
```

在前面的代码中，我们创建了一个名为 `MainWindow` 的类并覆盖了 `eventFilter()`。让我们使用 `installEventFilter()` 在 `textEdit` 上安装过滤器。您可以在一个对象上安装多个事件过滤器。但是，如果在单个对象上安装了多个事件过滤器，则将首先激活最后安装的过滤器。您还可以通过调用 `removeEventFilter()` 来删除事件筛选器：

```
#include "mainwindow.h"
#include <QTextEdit>
#include <QKeyEvent>
```



```
MainWindow::MainWindow(QWidget *parent)
 : QMainWindow(parent)
{
 textEdit = new QTextEdit;
 setCentralWidget(textEdit);
 textEdit->installEventFilter(this);
}
```

在前面的代码中，我们在`textEdit`对象上安装了`eventFilter`。现在，我们来看一下`eventFilter()`函数：

```
bool MainWindow::eventFilter(QObject *monitoredObj, QEvent *event)
{
 if (monitoredObj == textEdit)
 {
 if (event->type() == QEvent::KeyPress)
 {
 QKeyEvent *keyEvent = static_cast<QKeyEvent*>
 (event);
 qDebug() << "Key Press detected: " <<
 keyEvent->text();
 return true;
 }
 else
 {
 return false;
 }
 }
 else
 {
 return QMainWindow::eventFilter(monitoredObj,
 event);
 }
}
```

这里，`textEdit`是监视对象。每次按下某个键时，如果`textEdit`处于焦点位置，则会捕获该事件。因为可能有更多的子对象，并且`QMainWindow`可能需要事件，所以不要忘记将未处理的事件传递给基类以进行进一步的事件处理。

## 重要音符

一旦您使用了 `eventFilter()` 函数中的事件，请确保将其设置为 `return true`。如果 **Receiver** 对象被删除，而您 `return false`，则可能导致应用崩溃。

您还可以将信号和插槽机制与事件相结合。您可以通过过滤事件并发出对应于该事件的信号来实现这一点。我希望您已经了解了 **Qt** 中的事件处理机制。现在，让我们来看看拖放。

## 拖放

---

在本节中，我们将学习有关拖放(**DND**)的。在 **GUI** 应用中，**DND** 是一种定点设备手势，其中用户通过抓住虚拟对象，然后在另一个虚拟对象上释放来选择虚拟对象。当用户做出被识别为开始拖拽动作的信号的某个手势时，拖放操作开始。

让我们讨论一下如何使用 **Qt** 小部件实现拖放。

### 在 **Qt** 小工具中拖放

在使用拖放的基于 **Qt** 小部件的 **GUI** 应用中，用户开始从特定小部件拖动，并将拖动的对象放到另一个小部件上。这需要我们重新实现几个函数，它会处理相应的事件。需要重新实现以实现拖放的最常见功能如下：

```
void dragEnterEvent(QDragEnterEvent *event) override;
void dragMoveEvent(QDragMoveEvent *event) override;
void dropEvent(QDropEvent *event) override;
void mousePressEvent(QMouseEvent *event) override;
```

重新实现上述功能后，使用以下语句启用拖放目标小部件：

```
setAcceptDrops(true);
```

要开始拖动，请创建一个 `QDrag` 对象并将指针传递给开始拖动的小工具。拖放操作由 `QDrag` 对象处理。此操作要求附加数据描述为多用途 **Internet** 邮件扩展(MIME)类型：

```
QMimeData *mimeType = new QMimeData;
mimeType->setData("text/csv", csvData);
QDrag *dragObject = new QDrag(event->widget());
dragObject->setMimeData(mimeType);
dragObject->exec();
```

前面的代码显示了如何创建拖动对象和设置自定义 **MIME** 类型。这里，我们使用 `text/csv` 作为 **MIME** 类型。您可以通过拖放操作提供多种类型的 **MIME** 编码数据。

要截取拖放事件，可以重新实现 `dragEnterEvent()`。当拖动正在进行并且鼠标进入小部件时，将调用此事件处理程序。

您可以在 **Qt Creator** 的示例部分找到几个相关的示例。由于 **Qt** 小部件现在不是很流行，我们跳过了使用小部件的拖放示例。在下一节中，我们将讨论 **QML** 中的拖放。

## 在 **QML** 中拖放

在上一节中，我们讨论了使用小部件进行拖放。由于 **QML** 用于创建现代的、触摸友好的应用，因此拖放是一个非常重要的功能。**Qt** 为实现 **drag** 和 **drop** 提供了几种方便的 **QML** 类型。在内部，相应事件的处理方式类似。这些函数在 `QQuickItem` 类中声明。

例如，`dragEnterEvent()` 在 `QQuickItem` 中也可用，用于拦截拖放事件，如下所述：

```
void QQuickItem::dragEnterEvent(QDragEnterEvent *event)
```

让我们讨论一下如何使用可用的 QML 类型来实现这一点。使用 `Drag` 附加属性，可以使任何 `Item` 成为 QML 场景中拖放事件的来源。 `DropArea` 是不可见的项，当将项拖到它上面时，它可以接收事件。当项目上的拖动操作处于活动状态时，对其位置所做的任何更改都将生成一个拖动事件，该事件将发送到任何交叉点 `DropArea`。 `DragEvent` QML 类型提供有关拖动事件的信息。

下面的代码片段显示了 QML 中的一个简单拖放操作：

```
Rectangle {
 id: dragItem
 property point beginDrag
 property bool caught: false
 x: 125; y: 275
 z: mouseArea.drag.active || mouseArea.pressed ? 2 : 1
 width: 50; height: 50
 color: "red"
 Drag.active: mouseArea.drag.active
 Drag.hotSpot.x: 10 ; Drag.hotSpot.y: 10
 MouseArea {
 id: mouseArea
 anchors.fill: parent
 drag.target: parent
 onPressed: dragItem.beginDrag = Qt.point(dragItem.x,
 dragItem.y)
 onReleased: {
 if(!dragItem.caught) {
 dragItem.x = dragItem.beginDrag.x
 dragItem.y = dragItem.beginDrag.y
 }
 }
 }
}
```

在前面的代码中，我们创建了一个 ID 为 `dragItem` 的可拖动项。它包含一个 `MouseArea` 来捕获鼠标按下事件。拖动并不局限于鼠标拖动。任何

可以生成拖动事件的东西都可以触发拖动操作。可以通过调用 `Drag.cancel()` 或将 `Drag.active` 状态设置为 `false` 来取消拖动。

通过调用 `Drag.drop()`，可以通过删除事件完成删除操作。让我们添加一个 `DropArea`：

```
Rectangle {
 x: parent.width/2
 width: parent.width/2 ; height:parent.height
 color: "lightblue"
 DropArea {
 anchors.fill: parent
 onEntered: drag.source.caught = true
 onExited: drag.source.caught = false
 }
}
```

在前面的代码片段中，我们使用一个浅蓝色矩形来区分屏幕上的 `DropArea`。当 `dragItem` 进入 `DropArea` 区域时，我们正在捕捉 `dragItem`。当 `dragItem` 离开 `DropArea` 区域时，丢弃操作被禁用。因此，当放置不成功时，该项目将返回到其原始位置。

在本节中，我们了解了拖放操作及其对应的事件。我们讨论了如何使用 `Qt` 小部件模块以及在 `QML` 中实现它们。现在，让我们总结一下我们本章中学到的内容。

## 摘要

---

在本章中，我们研究了 `Qt` 中信号和槽的核心概念。我们讨论了连接信号和插槽的不同方式。我们还学习了如何将一个信号连接到多个插槽，以及如何将多个信号连接到一个插槽。然后，我们研究了如何在 `Qt` 小部件中以及在 `QML` 中使用它们，以及信号和槽连接背后的机制。在那之后，您学习了如何使用信号和槽在 `C++` 和 `QML` 之间通信。

本章还讨论了 **Qt** 中的事件和事件循环。我们探索了如何使用事件而不是信号槽机制。完成此操作后，我们创建了一个带有自定义事件处理程序的示例程序来捕获事件并对其进行过滤。

在了解了事件之后，我们实现了一个简单的拖放示例。现在，您可以在类之间、**C++** 和 **QML** 之间通信，并基于事件实现必要的操作。

在下一章中，我们将学习模型视图编程以及如何创建自定义模型。

## 七、模型视图编程

---

原文: <https://gitee.com/apacheecn/apacheecn-c-cpp-zh-pt2/blob/master/docs/cross-plat-dev-qt6-mod-cpp/07.md>

模型/视图编程用于将数据从 **Qt** 中的视图中分离出来，以处理数据集。模型/视图(**M/V**)体系结构区分了允许开发人员自由修改和以多种方式用户界面(**UI**)上显示信息的功能。我们将讨论该体系结构的每个组件、**Qt** 提供的相关便利类，以及如何通过实际示例使用它们。在本章中，我们将讨论模型视图模式，并理解基本的核心概念。

在本章中，我们将讨论以下主题：

- **M/V** 架构的基础知识
- 使用模型和视图
- 创建自定义模型和代理
- 在 **Qt** 小部件中使用 **M/V** 显示信息
- 在 **QML** 中使用 **M/V** 显示信息
- 在 **QML** 中使用 **C++** 模型

在本章结束时，您将能够创建数据模型并在自定义 **UI** 上显示信息。您将能够编写自定义模型和委托。您还将学习如何通过 **Qt** 小部件和 **QML** 在 **UI** 中表示信息。

## 技术要求

---

本章的技术要求包括在 **Windows 10**、**Ubuntu 20.04** 或 **MacOS 10.14** 等最新桌面平台上安装的 **Qt 6.0.0** 和 **Qt Creator 4.14.0** 的最低版本。

本章使用的所有代码都可以从以下 **giHub** 链接下

载: <https://github.com/PacktPublishing/Cross-Platform-Development-with-Qt-6-and-Modern-Cpp/tree/master/Chapter07>。

## 重要音符

本章中使用的截图是在 **Windows** 平台上拍摄的。根据您的机器上的底层平台，您将看到类似的屏幕。

## 了解 M/V 架构

传统上，在构建 **UI** 时经常使用模型-视图-控制器(**MVC**)设计模式。顾名思义，它由三个术语组成：模型、视图和控制器。模型是具有动态数据结构和逻辑的独立组件，视图是可视元素，控制器决定 **UI** 如何响应用户输入。在 **MVC** 出现之前，开发人员通常将这些组件放在一起。尽管开发人员希望将控制器与其他组件尽可能地区分开来，但并不总是可以将控制器与其他组件解耦。**MVC** 设计将组件解耦，以增加灵活性和重用性。下图说明了传统 **MVC** 模式的组件：

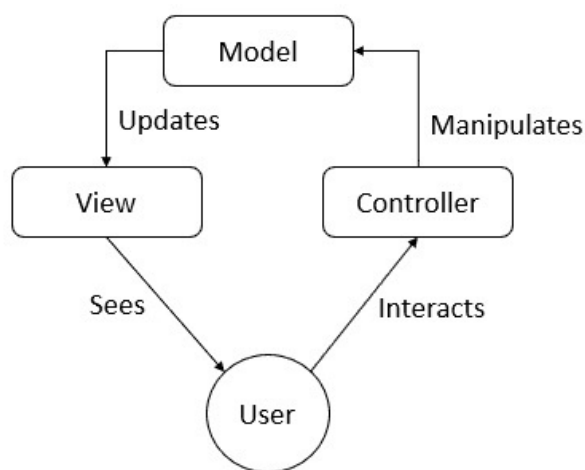


图 7.1-传统 MVC 设计模式

在 **MVC** 模式中，用户可以看到视图并与控制器交互。控制器向模型发



送数据，模型更新视图。如果将视图和控制器组件组合在一起，则会产生 M/V 体系结构。它提供了更多功能的架构。它基于相同的原则，但使实现变得简单得多。修改后的体系结构允许我们在几个不同的视图中显示相同的数据。开发人员可以在不更改底层数据结构的情况下实现新类型的视图。为了给我们的用户输入处理带来这种灵活性，Qt 引入了委托的概念。视图接收通过委托更新的数据，而不是控制器。它有两个主要目的：

- 帮助视图呈现每个值
- 要在用户想要进行某些更改时帮助视图，请执行以下操作

因此，在某种程度上，控制器已经与视图结合，并且视图还通过委托执行控制器的一些工作。拥有委托的好处在于它提供了呈现和修改数据元素的方法。

让我们用图表来理解 M/V 实现及其组件：

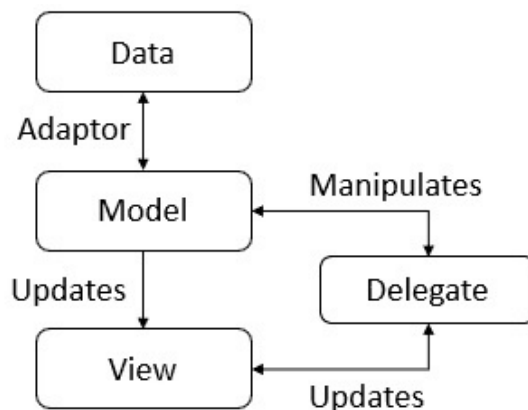


图 7.2-Qt 模型-视图-委托框架

如图 7.2所示，M/V 组件分为模型、视图和委托三个部分。模型与数据库交互，并作为体系结构的其他组件的接口工作。通信的目的由数据源和模型的实现确定。视图获得对称为模型索引的个数据项的引用。

视图可以使用此模型索引从数据模型中检索单个项目数据。在标准视图中，委托呈现数据项。当项目被修改时，代表通过使用模型索引通知模型。

图 7.3说明了模型如何向视图传递数据，视图显示在各个代表上：

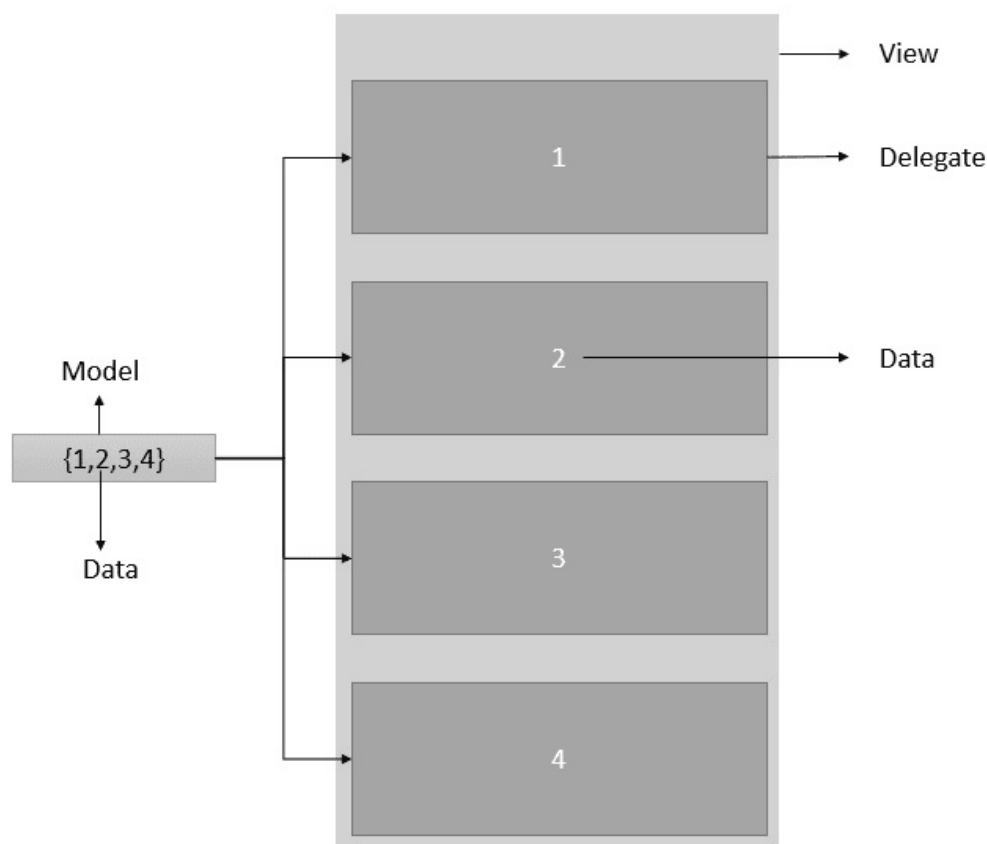


图 7.3-模型-视图-委派实现图解

Qt 框架提供了一组实现 M/V 体系结构的标准类，用于管理数据之间的关系以及用户查看数据的方式。通过分离功能，该体系结构提供了定制数据表示的灵活性，并允许将广泛的数据源与视图相结合。

模型、视图和委托使用信号和插槽机制相互通信。该模型发出一个信号，通知数据源中发生的数据更改。当用户与 **View** 交互时，**View** 会发出一个信号来通知用户操作。委派出发出信号，通知模型和视图有关

编辑状态的信息。

现在，您已经了解了 M/V 体系结构的基础知识。以下各节说明如何在 Qt 中使用 M/V 模式。我们将从 Qt 框架提供的标准类开始，然后我们将讨论 M/V 在 Qt 小部件中的使用。您将学习如何根据 M/V 体系结构创建新组件。我们走吧！

## 型号

M/V 消除了标准窗口小部件可能出现的数据一致性挑战。它使得对同一数据使用多个视图变得更容易，因为一个模型可以传递给多个视图。Qt 为 M/V 实现提供了几个抽象类，具有公共接口和某些功能实现。您可以子化抽象类并添加其他组件期望的预期功能。在 M/V 实现中，该模型提供了一个标准接口，供 **View** 和 **Delegate** 用来访问数据。

Qt 提供了一些现成的 **Model** 类，如 `QStandardItemModel`、`QFileSystemModel` 和 `QSqlTableModel`。`QAbstractItemModel` 是 Qt 定义的标准接口。`QAbstractItemModel` 的子类表示层次结构中的数据。图 7.4 说明了模型类的层次结构：

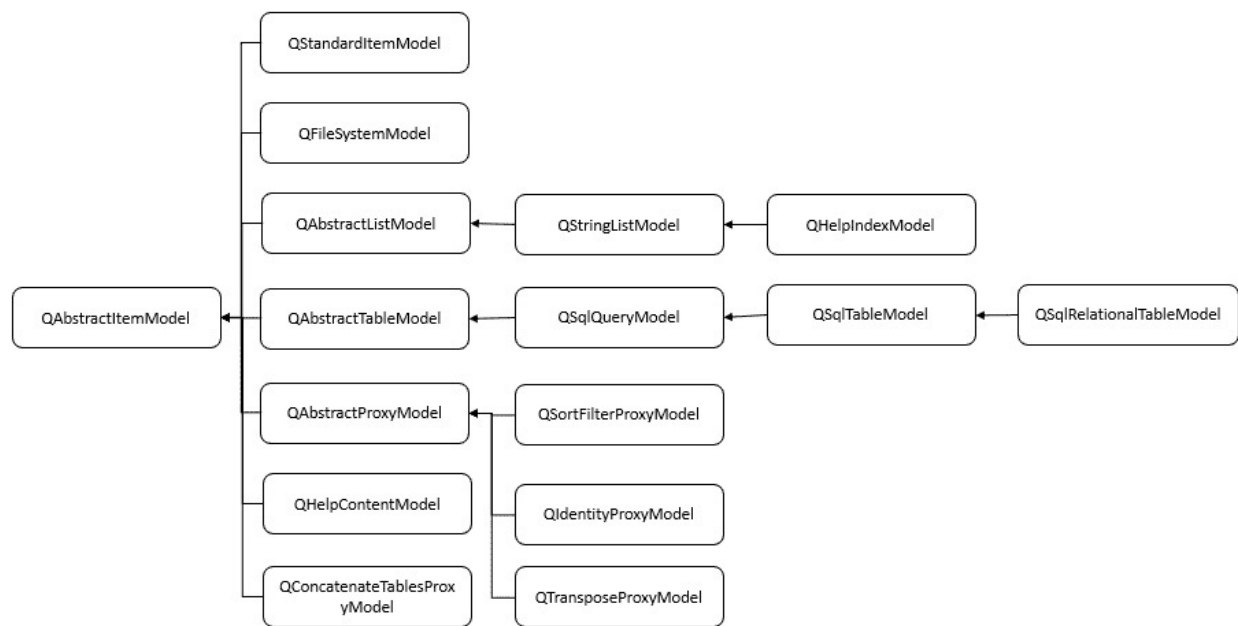


图 7.4-Qt 中模型类的层次结构

视图使用这种方法来访问模型中的各个数据项，但它们不受向用户呈现此信息的方式的限制。通过模型传递的数据可以保存在数据结构或数据库中，也可以保存在其他一些应用组件中。所有项目模型都基于 `QAbstractItemModel` 类。

图 7.5显示了数据在不同类型的模型中的排列方式：

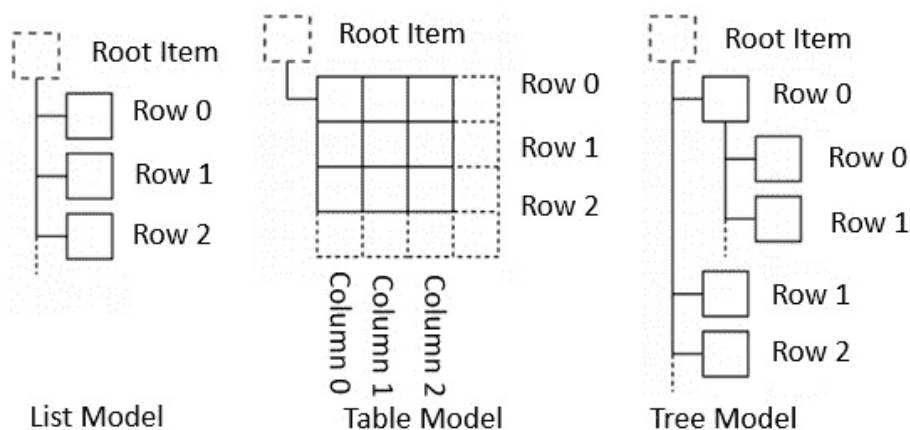


图 7.5-不同类型的数据模型和排列

数据通过模型以行和列形式的表格表示形式表示，或使用数据的分层表示形式表示。在 **M/V** 模式中，小部件不会将数据存储在单个单元格后面。他们直接使用数据。您可能需要创建一个包装器来使您的数据与 `QAbstractItemModel` 接口兼容。视图使用此接口读取和写入数据。从 `QAbstractItemModel` 派生的任何类都称为 **Model**。它提供了一个接口来处理以列表、表和树的形式表示数据的视图。要实现列表或表状数据结构的自定义模型，可以从 `QAbstractListModel` 和 `QAbstractTableModel` 派生以使用可用的功能。这些子类提供适合于专用列表和表格的模型。

Qt 框架提供了两种标准类型的模型。这些建议如下：

- `QStandardItemModel`
- `QFileSystemModel`

`QStandardItemModel` 是一个多用途模型，可以在其中存储自定义数据。每个元素引用一个项目。它可用于显示列表、表格和树视图所需的各种数据结构。它提供了一种传统的基于项目的方法来处理模型。

`QStandardItem` 提供了在 `QStandardItemModel` 中使用的项目。

`QFileSystemModel` 是一个保存有关目录内容的信息的模型。它只表示本地文件系统上的个文件和个目录，不包含任何数据项。它提供了一个现成的模型来创建示例应用，您可以使用模型索引来操作数据。现在，让我们讨论一下什么是代表。

## 代表

委派可以控制视图中显示的项目的显示方式。与 **MVC** 模式不同，**M/V** 模式没有完全不同的组件来处理用户交互。视图主要负责向用户显示模型数据并允许他们与其交互。为了增加用户操作获取方式的灵活

性，委托处理交互。它支持将某些小部件用作模型中可编辑项的编辑器。委托用于提供交互功能，并在视图中呈现各个字段。

`QAbstractItemDelegate`类定义了管理委托的基本接口。Qt 提供了几种现成的委托类，可以与内置小部件一起使用来修改特定的数据类型。

为了更好地理解它，我们将查看 Qt 框架中委托类的层次结构(请参见图 7.6):



图 7.6-Qt 框架中委托类的层次结构

正如我们在上图中看到的，`QAbstractItemDelegate`是委托的抽象基类。默认委托实现由`QStyledItemDelegate`提供。Qt 的标准视图将其用作默认委托。为视图中的项目绘制和创建编辑器的其他选项是`QStyledItemDelegate`和`QItemDelegate`。您可以使用`QItemDelegate`定制项目的显示功能和编辑器小部件。这两个类之间的不同之处在于，与`QItemDelegate`不同，`QStyledItemDelegate`使用当前样式来绘制其项目。`QStyledItemDelegate`可以处理最常见的数据类型，如`int`和`QString`。建议在创建新委托或使用 Qt 样式表时将`QStyledItemDelegate`子类化。通过编写自定义委托，可以使用自定义数据类型或自定义呈现。

在本节中，我们讨论了不同类型的模型和委托。让我们讨论一下 Qt 小部件提供的 **View** 类。

## Qt 小工具中的视图

几个方便的类是从标准视图类派生的，以实现 M/V 模式。这种便利类的示例有 `QListWidget`、`QTableWidget` 和 `QTreeWidget`。根据 Qt 文档，这些类的适应性不如 **View** 类，它们不能用于随机模型。根据您的项目需求，您必须选择合适的小部件类来实现 M/V 模式。

如果要使用基于项目的界面并利用 M/V 模式的，则建议将以下视图类与 `QStandardItemModel` 一起使用：

- `QListView` 显示项目列表。
- `QTableView` 在表格中显示模型中的数据。
- `QTreeView` 在分层列表中显示数据的模型项。

Qt 框架中 View 类的层次如下：

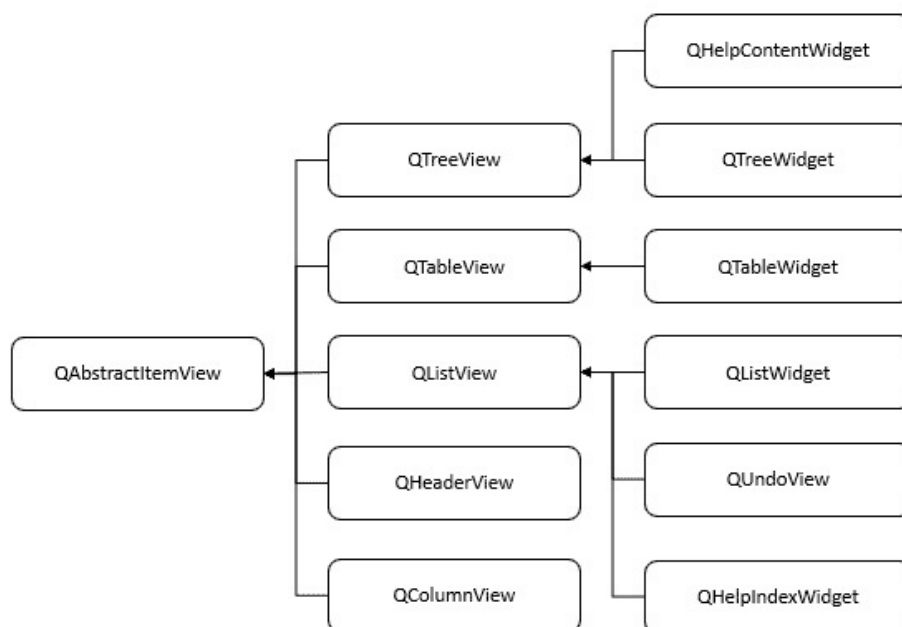


图 7.7-Qt 框架中视图类的层次结构

`QAbstractItemView` 是上述类的抽象基类。即使这些类提供了现成的实现，也可以派生出专门的视图。最适合用于 `QFileSystemModel` 的视图是 `QListView` 和 `QTreeView`。这些视图中的每一个都有其独特的表示数据

项的方式。例如，`QTreeView`将树层次结构显示为一系列水平列表。所有这些视图都必须有一个与之相关联的模型。`Qt` 提供了几个预定义的模型。如果现成的型号不符合您的标准，您可以添加您自己的定制型号。

与视图类(其类名以`View`结尾)不同，方便小部件(其类名以`Widget`结尾)不需要 **Model** 支持，可以直接使用。使用方便小部件的主要优点是，使用它们所需的工作量最少。

让我们看看 `Qt` 小部件模块中的个不同的 **View** 类，以及哪些现成的模型可以与它们一起使用：

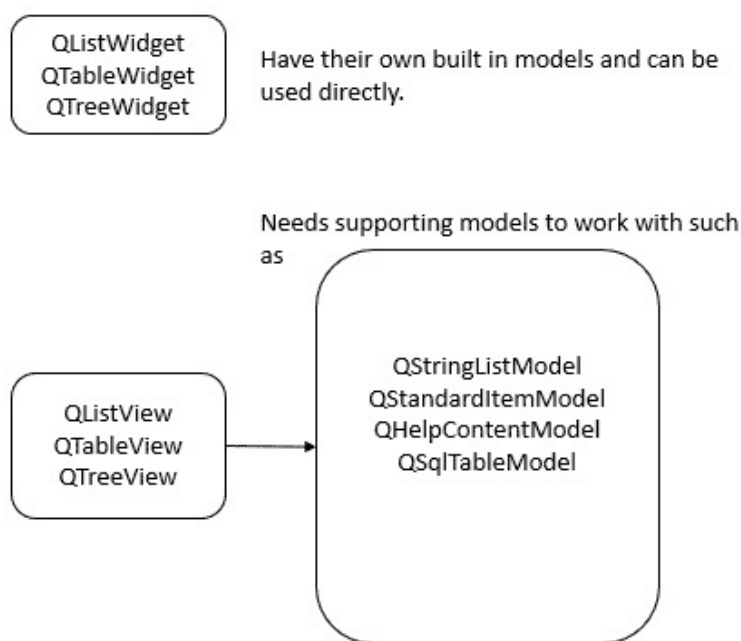


图 7.8-在 M/V 模式中用作视图的不同类型的 `Qt` 小部件

委托用于在`QListView`、`QTableView`或`QTreeView`中显示各个字段数据。当用户开始与项目交互时，委托将提供编辑器小部件以进行编辑。

您可以通过以下链接找到个前述类的比较概述，并了解相应小部件的用法：



<https://doc.qt.io/qt-6/modelview.html>

在本节中，您了解了 M/V 体系结构，并熟悉了使用的术语。让我们使用 Qt 小部件通过一个简单的 GUI 应用实现 M/V。

## 使用 **M/V** 模式创建一个简单的 **Qt** 小部件应用

---

现在是我们使用 **Qt** 小部件创建一个简单示例的时候了。本节中的示例说明了如何将预定义的 `QFileSystemModel` 与内置的 `QListView` 和 `QTreeView` 小部件结合使用。双击视图时会自动执行委派。

按照这些步骤创建实现 M/V 模式的简单应用：

1. 使用 **Qt Creator** 创建一个新项目，从项目创建向导中选择 **Qt widgets** 模板。它将生成一个具有预定义项目框架的项目。
2. Once the application skeleton is created, open the `.ui` form and add `QListView` and `QTreeView` to the form. You may add two labels to distinguish the Views as shown here:

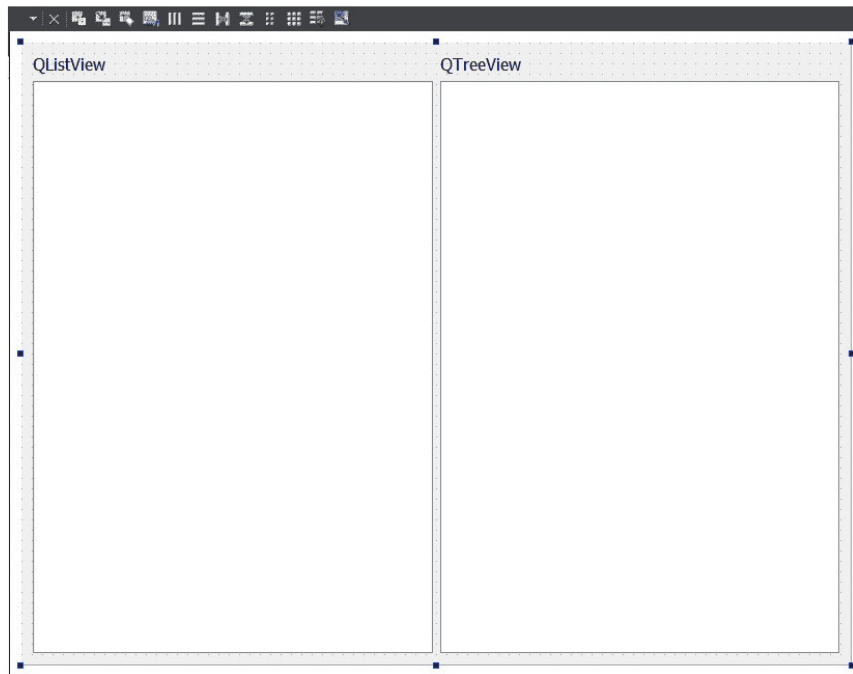


图 7.9-使用 Qt Designer 使用 QListView 和 QTreeView 创建 UI

3. Open the `mainwindow.cpp` file and add the following contents:

```
#include "mainwindow.h"
#include "ui_mainwindow.h"
#include <QFileSystemModel>
MainWindow::MainWindow(QWidget *parent)
 : QMainWindow(parent)
 , ui(new Ui::MainWindow)
{
 ui->setupUi(this);
 QFileSystemModel *model = new QFileSystemModel;
 model->setRootPath(QDir::currentPath());
 ui->treeView->setModel(model);
 ui->treeView->setRootIndex(
 model->index(QDir::currentPath()));
 ui->listView->setModel(model);
 ui->listView->setRootIndex(
 model->index(QDir::currentPath()));
}
```

在前面的 C++ 实现中，我们使用了一个预定义的 `QFileSystemModel`

作为视图的模型。

4. Next, hit the **Run** button in the left pane. You will see a window as shown in *Figure 7.10* once you hit the **Run** button:

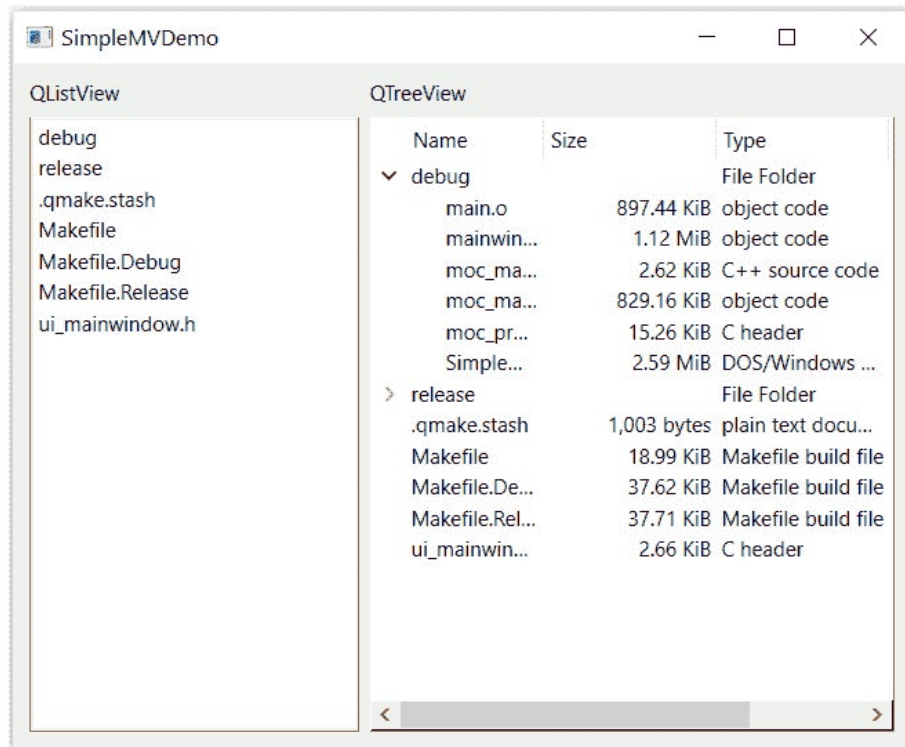


图 7.10-示例应用的输出显示 QListView 和 QTreeView

5. 让我们修改现有的应用，以使用从 `QAbstractItemModel` 派生的自定义模型。在下面的示例中，我们创建了一个从 `QAbstractItemModel` 派生的简单 `ContactListModel` 自定义类：

```
void ContactListModel::addContact(QAbstractItemModel *model,
const QString &name,const QString &phoneno, const QString &ema:
{
 model->insertRow(0);
 model->setData(model->index(0, 0), name);
 model->setData(model->index(0, 1), phoneno);
 model->setData(model->index(0, 2), emailid);
}
QAbstractItemModel* ContactListModel::
```

```

 getContactListModel()
 {
 QStandardItemModel *model = new
 QStandardItemModel(0, 3, this);
 model->setHeaderData(0,Qt::Horizontal,
 QObject::tr("Name"));
 model->setHeaderData(1,Qt::Horizontal,
 QObject::tr("Phone No"));
 model->setHeaderData(2,Qt::Horizontal,
 QObject::tr("Email ID"));
 addContact(model,"John","+1
 1234567890","john@abc.com");
 addContact(model,"Michael","+44
 213243546","michael@abc.com");
 addContact(model,"Robert","+61
 5678912345","robert@xyz.com");
 addContact(model,"Kayla","+91
 9876554321","kayla@xyz.com");
 return model;
 }

```

6. 接下来，修改 UI 表单以实现 `QTableView`，并将联系人列表模型设置为它，如下面的片段所示：

```

ContactListModel *contactModel = new ContactListModel;
ui->tableView->setModel(
 contactModel->getContactListModel());
ui->tableView->horizontalHeader()->setStretchLastSection(true);

```

7. 您可以将 `QStringListModel` 添加到 `QListView` 以使用简单列表模型：

```

QStringListModel *model = new QStringListModel(this);
QStringList List;
List << "Item 1" << "Item 2" << "Item 3" <<"Item 4";
model->setStringList(List);
ui->listView->setModel(model);

```

8. 接下来，点击左窗格中的 **Run** 按钮。您将看到一个窗口，如图 7.11 所示：



```
QWidget *createEditor(QWidget* parent,
 const QStylestyleOptionViewItem& styleOption,
 const QModelIndex & modelIndex)
void setEditorData(QWidget* editor,
 const QModelIndex& modelIndex)
void updateEditorGeometry(QWidget* editor,
 const QStylestyleOptionViewItem& styleOption,
 const QModelIndex& modelIndex)
};
```

您必须根据项目需要覆盖虚拟方法并添加相应的逻辑。您可以通过以下链接了解有关自定义委托和示例的更多信息：

<https://doc.qt.io/qt-6/model-view-programming.html>

在本节中，我们学习了如何创建使用 M/V 模式的 GUI 应用。在下一节中，我们将讨论如何在 QML 中实现它。

## 了解 QML 中的模型和视图

与 Qt 小部件一样，QtQuick 也实现了模型、视图和委托来显示数据。该实现模块化了数据的可视化，以使开发人员能够管理该数据。您可以更改一个视图与另一个视图，只需对数据进行最小程度的更改。

要可视化数据，请将视图的 `model` 属性绑定到模型，并将 `delegate` 属性绑定到组件或其他兼容类型。

让我们讨论一下可用于在 Qt Quick 应用中实现 M/V Pattern 的 QML 类型。

### Qt Quick 中的视图

视图是显示数据并用于项集合的容器。这些容器功能丰富，可以自定义以满足特定的样式或行为要求。

Qt Quick 图形类型的基本集合中提供了一组标准视图：

- `ListView`：在水平或垂直列表中布局项目
- `GridView`：以网格方式布局项目
- `TableView`：以表格形式布局项目
- `PathView`：在路径上布局项目

`ListView`、`GridView`和`TableView`继承自`FlickableQML`类型。`PathView`继承`Item`。`TreeViewQML`类型已过时。让我们来看看这些 QML 类型的继承：

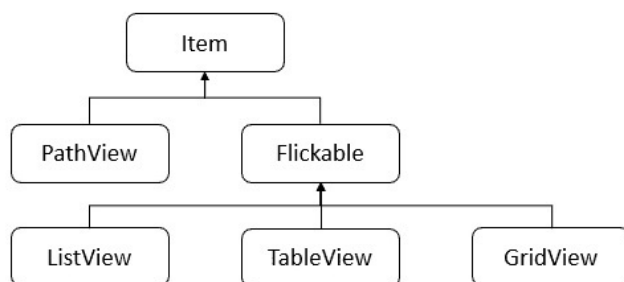


图 7.12-Qt Quick 中的视图类层次结构

每种 QML 类型的属性和行为都不同。它们是根据 GUI 要求使用的。如果您想了解更多关于 QML 类型的信息，可以参考它们各自的文档。在下一节中，让我们探索 Qt Quick 中的模型。

## Qt Quick 中的模型

Qt 提供了几种方便的 QML 类型来实现 M/V 模式。这些模块提供了非常简单的模型，而不需要在 C++ 中创建自定义模型类。这种便利类的示例有`ListModel`、`TableModel`和`XmlListModel`。

`QtQml.Models`模块提供以下定义数据模型的 QML 类型：

- `ListModel`定义自由格式列表数据源。

- `ListElement` 定义 `ListModel` 中的数据项。
- `DelegateModel` 封装模型和委托。
- `DelegateModelGroup` 封装一组筛选的可视数据项。
- `ItemSelectionModel` 继承 `QItemSelectionModel` 并跟踪视图的选定项。
- `ObjectModel` 定义要用作模型的一组项目。
- `Instantiator` 动态实例化对象。
- `Package` 描述命名项的集合。

要在 Qt Quick 应用中使用上述 QML 类型，请使用以下行导入模块：

```
import QtQml.Models
```

让我们讨论一下 Qt Quick 中提供的现成模型。 `ListModel` 是包含数据角色的 `ListElement` 定义的简单容器。它与 `ListView` 一起使用。

`Qt.labs.qmlmodels` 为模型提供实验性的 QML 类型。这些模型可用于快速原型制作和显示非常简单的数据。 `TableModel` 类型将 JavaScript/JSON 对象存储为表模型的数据，并将其与 `TableView` 一起使用。您可以通过导入模块来使用这些实验类型，如下所示：

```
import Qt.labs.qmlmodels
```

如果您希望从 XML 数据创建一个 Model，那么您可以使用 `XmlListModel`。它可以用作具有 `ListView`、`PathView` 和 `GridView` 等视图的模型。要使用此模型，您必须按如下方式导入模块：

```
import QtQuick.XmlListModel
```

可以将 `ListModel` 和 `XmlListModel` 与 `TableView` 配合使用，以在 `TableView` 中创建一行。要处理多行多列，可以使用 `TableModel`，也可以通过子类化 `QAbstractItemModel` 来创建自定义 C++ 模型。



您还可以对模型使用 `Repeater`。整数可以用作定义项目数的模型。在这种情况下，模型没有任何数据角色。让我们创建一个使用 `ListView` 和一个 `Text` 项作为委托组件的简单示例：

```
import QtQuick
import QtQuick.Window
Window {
 width: 640
 height: 480
 visible: true
 title: qsTr("Simple M/V Demo")
 ListView {
 anchors.fill: parent
 model: 10
 delegate: itemDelegate
 }
 Component {
 id: itemDelegate
 Text { text: " Item : " + index }
 }
}
```

在前面的示例中，我们使用了一个基于整数的模型。我们创建了一个简单的委托，它是一个文本元素。为简单起见，我们没有使用复杂的委托组件。您还可以直接使用 `Text` 作为委托，而不使用组件。

现在，让我们探索一下如何将 `ListModel` 与 `ListView` 一起使用。

`ListModel` 是在 **QML** 中指定的类型的简单层次结构。可用的角色由 `ListElement` 属性指定。让我们使用 `ListModel` 和 `ListView` 创建一个简单的应用。

假设您想要创建一个简单的地址簿应用。您可能需要为联系人填写几个字段。在下面的代码片段中，我们使用了包含一些联系人的姓名、电话号码和电子邮件地址的 `ListModel`：

```
ListModel {
```

```

id: contactListModel
ListElement {
 name: "John" ; phone: "+1 1234567890" ;
 email: "john@abc.com"
}
ListElement {
 name: "Michael" ; phone: "+44 213243546" ;
 email: "michael@abc.com"
}
ListElement {
 name: "Robert" ; phone: "+61 5678912345" ;
 email: "robert@xyz.com"
}
ListElement {
 name: "Kayla" ; phone: "+91 9876554321" ;
 email: "kayla@xyz.com"
}
}

```

现在我们已经创建了模型。接下来，我们必须使用委托来显示它。因此，让我们用三个 `Text` 元素修改前面创建的委托组件。根据您的需要，您可以使用图标、文本或自定义类型创建复杂的委托类型。您可以添加突出显示的项目并基于焦点更新背景。您需要为视图提供委派，才能直观地表示列表中的项目：

```

Component {
 id: contactDelegate
 Row {
 id: contact
 spacing: 20
 Text { text: " Name: " + name; }
 Text { text: " Phone no: " + phone }
 Text { text: " Email ID: " + email }
 }
}
ListView {
 anchors.fill: parent
 model: contactListModel
 delegate: contactDelegate
}

```

在前面的示例中，我们将 `ListElement` 与 `ListModel` 一起使用。视图根据委派定义的模板显示每个项目。模型中的项可以通过 `index` 属性或通过项的属性来访问。

您可以通过以下链接了解有关不同类型的模型以及如何操作模型数据的更多信息：

<https://doc.qt.io/qt-6/qtquick-modelviewsdata-modelviewsdata-modelview.html>

在本节中，您了解了 QML 中的 M/V。您可以尝试自定义模型和代理，并创建个性化视图。在你的手机上查看你的电话簿或最近的来电清单，然后试着去实现它。在下一节中，您将讲述 ARN 如何将 QML 前端与 C++ 模型集成。

## 在 QML 中使用 C++ 模型

---

到目前为止，我们已经讨论了如何在 Qt 小部件和 QML 中使用模型和视图。但在大多数现代应用中，您将需要用 C++ 编写的模型和用 QML 编写的前端。Qt 允许我们用 C++ 定义模型，然后在 QML 中访问它们。这对于向 QML 公开现有的 C++ 数据模型或其他复杂的数据集很方便。对于复杂的逻辑操作，本机 C++ 始终是正确的选择。它可以超越用 QML 和 JavaScript 编写的逻辑。

创建 C++ 模型的原因有很多。C++ 是类型安全的，可以编译成目标代码。它提高了应用的稳定性，并减少了错误的数量。它非常灵活，可以提供比 QML 类型更多的功能。您可以与现有代码集成，也可以与用 C++ 编写的第三方库集成。

您可以使用以下类定义 C++ 模型：

- `QStringList`
- `QVariantList`
- `QObjectList`
- `QAbstractItemModel`

前三个类有利于公开更简单的数据集。 `QAbstractItemModel` 为创建复杂模型提供了更灵活的解决方案。 `QStringList` 包含 `QString` 实例的列表，并通过 `modelData` 角色提供该列表的内容。类似地， `QVariantList` 包含个 `QVariant` 类型列表，并通过 `modelData` 角色提供该列表的内容。如果 `QVariantList` 更改，则必须重置模型。 `QObjectList` 嵌入 `QObject*` 列表，该列表提供列表中对象的属性作为角色。可以作为 `modelData` 属性访问 `QObject*`。为方便起见，可以在委托的上下文中直接访问对象的属性。

Qt 还提供了 C++ 类来处理 SQL 数据模型，

如 `QSqlQueryModel`、`QSqlTableModel` 和 `QSqlRelationalTableModel`。

`QSqlQueryModel` 提供基于 SQL 查询的只读模型。这些类减少了对基本 SQL 操作(如 `INSERT`、`CREATE` 或 `UPDATE`)运行 SQL 查询的需要。这些类派生自 `QAbstractTableModel`，使得在 `View` 类中显示数据库中的数据变得很容易。

您可以通过访问以下链接了解有关不同类型的 C++ 模型的更多信息：

<https://doc.qt.io/qt-6/qtquick-modelviewsdata-cppmodels.html>

在本节中，我们讨论了 C++ 模型以及使用它们的原因。现在，您可以从 C++ 后端获取数据，并将其呈现在用 QML 开发的 UI 中。在下一节中，我们将使用上述概念创建一个简单的 Qt Quick 应用，并解释如何在 QML 中使用它们。

# 使用 Qt Quick 创建一个简单的 M/V 应用

在前面几节中，我们讨论了 Qt 的模型-视图-委托框架。您了解了如何创建自定义模型和委托，以及如何使用 C++ 模型。但您一定想知道如何与我们的 QML 前端集成。在本节中，我们将创建一个 C++ 模型并将其公开给 QML 引擎。我们还将讨论如何将自定义模型注册为 QML 类型。

让我们创建一个从 C++ 代码获取 Model 并将其显示在基于 Qt Quick 的应用中的应用：

```
#include <QGuiApplication>
#include <QQmlApplicationEngine>
#include <QQmlContext>
#include <QStringListModel>
int main(int argc, char *argv[])
{
 QGuiApplication app(argc, argv);
 QQmlApplicationEngine engine;
 QStringList stringList;
 stringList << "Item 1" << "Item 2" << "Item 3"
 << "Item 4";
 engine.rootContext()->setContextProperty("myModel",
 QVariant::fromValue(stringList));
 const QUrl url(QStringLiteral("qrc:/main.qml"));
 engine.load(url);
 return app.exec();
}
```

在前面的代码片段中，我们创建了一个基于 `QStringListModel` 的简单模型。字符串列表包含四个不同的字符串。我们已经使用 `setContextProperty()` 将该模型公开给 QML 引擎。现在，让我们在 QML 文件中使用 Model：

```
import QtQuick
import QtQuick.Window
Window {
```

```

width: 640
height: 480
visible: true
title: qsTr("QML CPP M/V Demo")
ListView {
 id: listview
 width: 120
 height: 200
 model: myModel
 delegate: Text { text: modelData }
}
}

```

前面的示例使用 `QQmlContext::setContextProperty()` 直接在 QML 组件中设置 **Model** 值。另一种替代方法是将 C++ **Model** 类注册为 QML 类型，如下所示：

```
qmlRegisterType<MyModel>("MyModel",1,0,"MyModel");
```

前一行将允许将 **Model** 类直接创建为 QML 文件中的 QML 类型。第一个字段是 C++ 类名，然后是所需的包名，然后是版本号，最后一个参数是 QML 中的类型名。您可以使用以下行将其导入到 QML 文件中：

```
Import MyModel 1.0
```

让我们在 QML 文件中创建 `MyModel` 的一个实例，如下所示：

```

MyModel {
 id: myModel
}
ListView {
 width: 120
 height: 200
 model: myModel
 delegate: Text { text: modelData }
}

```

还可以使用 `setInitialProperties()` 使用与 `QQuickView` 一起使用模型，如

以下代码所示：

```
QQuickView view;
view.setResizeMode(QQuickView::SizeRootObjectToView);
view.setInitialProperties({
 {"myModel", QVariant::fromValue(myModel)}});
view.setSource(QUrl("qrc:/main.qml"));
view.show();
```

在前面的代码片段中，我们使用 `QQuickView` 创建了一个 UI，并将一个自定义 C++ 模型传递给了 QML 环境。

在本节中，我们学习了如何将简单的 C++ 模型与 QML 集成。您可以添加信号和属性来扩展自定义类的功能。接下来，让我们在本章中总结一下我们所学到的东西。

## 摘要

---

在本章中，我们介绍了 Qt 中的模型-视图-代表模式的核心概念。我们解释了它与传统 MVC 模式的不同之处。我们讨论了 M/V 的不同使用方式以及 Qt 中提供的便利类。我们学习了如何在 Qt 小部件和 Qt Quick 中应用 M/V 概念。我们讨论了如何将 C++ 模型与 QML 视图集成。我们还创建了几个示例，并在我们的 Qt 应用中实现了这些概念。现在，您可以创建自己的模型、委托和视图。我希望您已经理解了该框架的重要性以及使用它来满足您的需求的充分理由。

在[第 8 章](#)，图形和动画中，我们将了解图形框架以及如何向 Qt Quick 项目添加动画。

## 八、图形和动画

---

原文: <https://gitee.com/apacheecn/apacheecn-c-cpp-zh-pt2/blob/master/docs/cross-plat-dev-qt6-mod-cpp/08.md>

在本章中，您将学习 **Qt** 图形框架的基础知识以及如何在屏幕上渲染图形。您将了解如何在 **Qt** 中绘制总图。我们将首先讨论使用 **QPainter** 的 **2D** 图形。我们将探索如何使用画笔绘制不同的形状。然后，您将了解 **QGraphicsView** 和 **QGraphicsScene** 使用的 **Graphics View** 体系结构。稍后，我们将讨论 **Qt Quick** 使用的场景图机制。在本章中，您还将学习如何通过添加动画和状态使用户界面更有趣。

在本章中，我们将讨论以下内容：

- 了解 **Qt** 的图形框架
- **QPainter** 和 **2D** 图形
- 图形视图框架
- **OpenGL** 实现
- **Qt** 快速场景图
- **QML** 中的动画
- **Qt** 中的状态机

在本章结束时，您将了解 **Qt** 使用的图形框架。您将能够在屏幕上绘制并向您的 **UI** 元素添加动画。

## 技术要求

---

本章的技术要求包括在最新版本的桌面平台(如 **Windows 10**、**Ubuntu 20.04** 或 **MacOS 10.14**)上安装 **Qt 6.0.0** 和 **Qt Creator 4.14.0** 的最低版本。



本章使用的所有代码都可以从以下 [giHub](https://github.com/PacktPublishing/Cross-Platform-Development-with-Qt-6-and-Modern-Cpp/tree/master/Chapter08) 链接下载：<https://github.com/PacktPublishing/Cross-Platform-Development-with-Qt-6-and-Modern-Cpp/tree/master/Chapter08>。

## 重要音符

本章中使用的屏幕截图取自 **Windows** 平台。根据您的机器上的底层平台，您将看到类似的屏幕。

## 了解 Qt 的图形框架

Qt 是最流行的 GUI 应用框架之一。开发人员可以使用 Qt 构建令人惊叹的跨平台 GUI 应用，而不必担心底层的图形实现。Qt 渲染硬件接口 (RHI) 将来自 Qt 应用的图形指令解释为目标平台上可用的图形 API。

RHI 是硬件加速图形 API 的抽象接口。rhi 模块中最重要的类是 `QRhi`。`QRhi` 实例由特定图形 API 的后端支持。后端的选择在运行时发生，并由创建 `QRhi` 实例的应用或库决定。您可以通过在项目文件中添加以下行来添加模块：

```
QT += rhi
```

RHI 支持的不同类型的图形 API 如下：

- **OpenGL**
- **OpenGL ES**
- 火山
- **Direct3D**
- 金属

图 8.1 显示了 Qt 图形框架中图形堆栈的主要层：

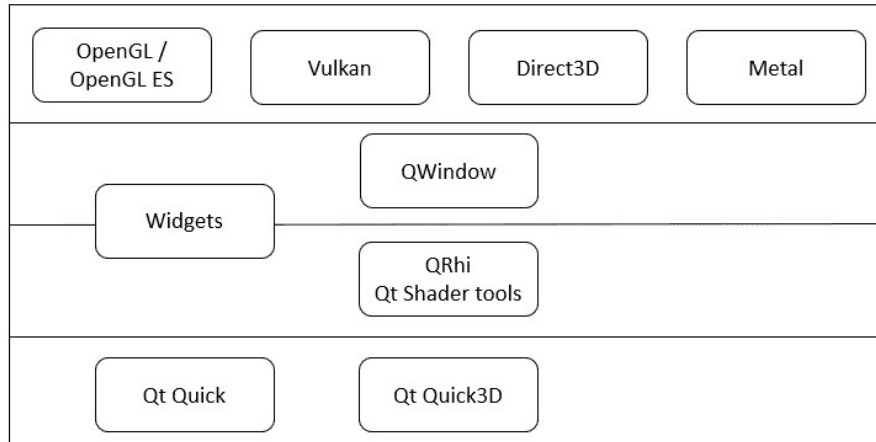


图 8.1-Qt 6 图形堆栈的主要层

让我们来熟悉一下上图所示的图形 API。**OpenGL**是最流行的图形 API，支持跨语言和跨平台的应用。用于与 GPU 交互，实现硬件加速渲染。**OpenGL ES**是面向嵌入式设备的 OpenGL API 的风格。它允许在嵌入式和移动设备上渲染高级 2D 和 3D 图形。**iOS** 设备上的 **OpenGL ES**也称为，即**EAGL**。OpenGL ES 在 Web 平台上也以 WebGL 的形式提供。OpenGL 和 OpenGL ES 是由科罗诺斯集团 (Khronos Group)开发和维护的，该集团是一个由技术硬件和软件公司组成的联盟。您可以通过以下链接了解有关 OpenGL 的更多信息：

<https://www.opengl.org/about/>

**Vulkan**是新一代图形 API，有助于为现代 GPU 创建跨平台的高性能应用。它是由 Khronos 集团创建的。Vulkan 显式的 API 设计允许在广泛的桌面、嵌入式和移动平台上高效实现。Qt6 提供了对 Vulkan API 的支持。要使用 Vulkan，Qt 应用需要 LunarG Vulkan SDK。通过以下链接了解有关伏尔坎的更多信息：

<https://www.lunarg.com/vulkan-sdk/>

**Direct3D**是 Microsoft 专有图形 API，它提供了通过利用底层 GPU 功

能渲染 2D 和 3D 图形的功能。微软公司创建它是为了在 Windows 平台上使用。它是一个低级 API，可用于使用渲染管道绘制基本体或使用计算着色器执行并行操作。

Direct3D 展示了 3D 图形硬件的高级图形功能，包括模板缓冲、W 缓冲、Z 缓冲、透视校正纹理映射、空间抗锯齿、可编程 HLSL 着色器和效果。Direct3D 与其他 DirectX 技术的集成使其能够提供多项功能，包括视频映射、2D 覆盖平面中的硬件 3D 渲染，甚至精灵，并允许在交互式媒体连接中使用 2D 和 3D 图形。Direct3D 通常用于虚拟化 3D 硬件接口。相比之下，OpenGL 旨在成为一种 3D 硬件加速渲染系统，可以在软件中进行仿真。这两个 API 基本上是以两种截然不同的方式设计的。下面的链接提供了对 Direct3D 的进一步了解：

<https://docs.microsoft.com/en-in/windows/win32/getting-started-with-direct3d>

**Metal**是 Apple 的低级计算机图形 API，它提供对图形处理单元(GPU)的近乎直接的访问，允许您优化 iOS、MacOS 和 TVOS 应用的图形和计算能力。它还具有低开销架构，包括预编译的 GPU 着色器、细粒度资源管理和多线程支持。在 Metal 发布之前，苹果提供了适用于 MacOS 的 OpenGL 和适用于 iOS 的 OpenGL ES，但由于高度抽象的硬件，存在性能问题。另一方面，由于其特定于苹果的 API，Metals 的性能比 OpenGL 更好。Metals 支持比 OpenGL 多 100 倍的绘制调用，从而支持全新一代的专业图形输出。您可以通过以下链接阅读有关金属的更多信息：

<https://developer.apple.com/documentation/metal>

在这一节中，我们熟悉了 Qt 的图形框架和 RHI。现在您对该框架有了

一个基本的了解。在下一节中，我们将进一步讨论使用 QPainter 的 2D 图形。

## QPainter 和 2D 图形

Qt 附带了高级窗口、绘画和排版系统。Qt GUI 模块中最重要的类是 `QWindow` 和 `QGuiApplication`。本模块包括 2D 图形、成像、字体和高级排版的类。此外，GUI 模块附带用于集成窗口系统、OpenGL 集成、事件处理、2D 图形、基本图像、字体和文本的类。Qt 的用户界面技术在内部使用这些类，但它们可以直接用于编写使用低级 OpenGL 图形 API 的应用。

根据平台的不同，`QWindow` 类支持使用 OpenGL 和 OpenGL ES 进行渲染。Qt 包括 `QOpenGLPaintDevice` 类，它允许使用 OpenGL 加速的 `QPainter` 渲染和几个方便的类。这些方便的类通过隐藏扩展处理的复杂性以及 OpenGL ES 2.0 和桌面 OpenGL 之间的差异，简化了在 OpenGL 中编写代码。`QOpenGLFunctions` 是一个方便的类，它提供了对桌面 OpenGL 上的 OpenGL ES 2.0 函数的跨平台访问，而无需手动解析 OpenGL 函数指针。

要在基于 `qmake` 的应用上使用这些 API 和类，您必须在项目文件 (`.pro`) 中包含 `gui` 模块，如下所示：

```
QT += gui
```

如果您使用的是基于 `Cmake` 的构建系统，则将以下内容添加到 `CMakeLists.txt` 文件：

```
find_package(Qt6 COMPONENTS Gui REQUIRED)
target_link_libraries(mytarget PRIVATE Qt6::Gui)
```

`QPainter`类主要用于绘制操作，它为各种任务提供了 API，例如将矢量图形、文本和图像绘制到不同的表面或`QPaintDevice`实例上，包括`QImage`、`QOpenGLPaintDevice`、`QWidget`和`QPrinter`。对于 Qt 小部件用户界面，Qt 使用软件渲染器。

以下是 Qt GUI 的高级绘图 API：

- 涂装系统
- 坐标系
- 绘制和填充

我们将在以下几节中探讨这些 API。

## 了解涂装系统

Qt 的 Paint System 为在屏幕上绘图提供了几个方便的类。使用的最重要的类是`QPainter`、`QPaintDevice`和`QPaintEngine`。您可以使用`QPainter`在小工具和其他绘制设备上绘制。这个类可以用来绘制从简单的线条到复杂的形状，如饼和弦。它还用于绘制像素图和文本。如果绘制设备是小工具，则在`paintEvent()`函数内或由`paintEvent()`调用的函数调用的函数内使用`QPainter`。`QPaintDevice`是允许使用`QPainter`实例进行二维绘图的对象基类。`QPaintEngine`提供定义`QPainter`如何绘制到指定平台上的指定设备的接口。`QPaintEngine`类是由`QPainter`和`QPaintDevice`内部使用的抽象类。

让我们来看看与绘画相关的类的层次结构，以便更好地了解在使用绘画系统时如何选择正确的类。

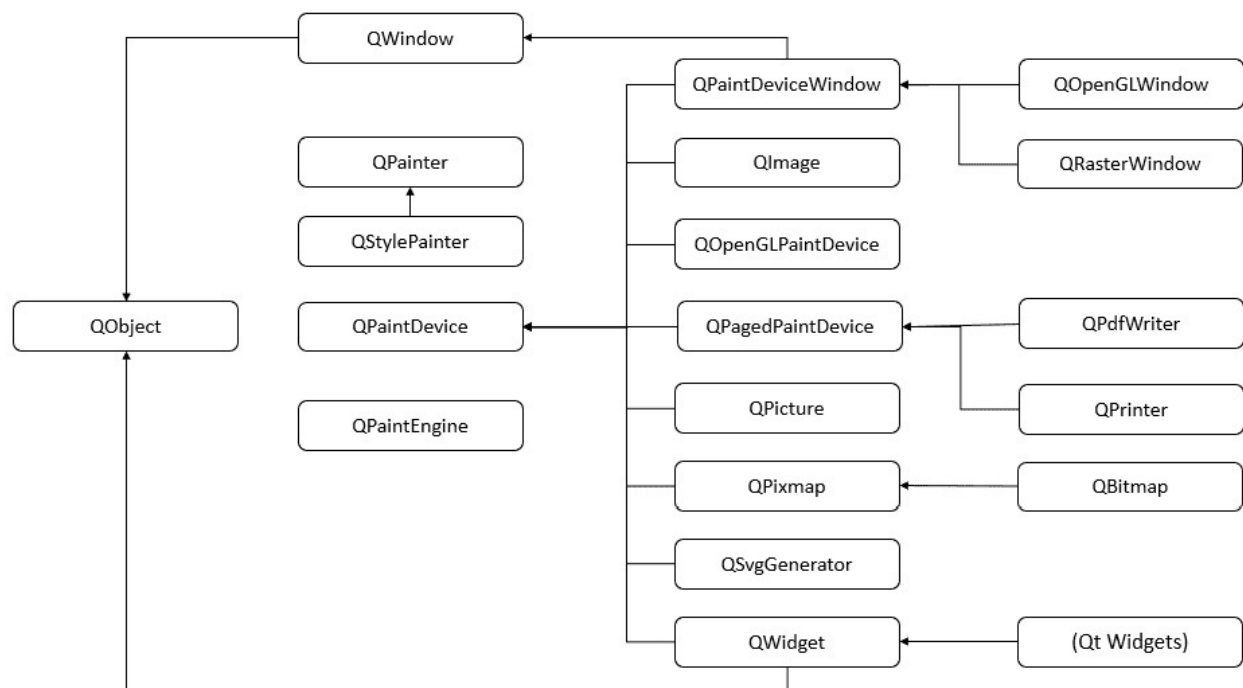


图 8.2-Qt 中绘制类的层次结构

前面的分层方法说明了所有绘制方法都遵循相同的机制。因此，很容易为新功能添加配置，并为不受支持的功能提供默认实现。

让我们在下一节讨论一下坐标系。

## 使用坐标系

`QPainter` 类控制坐标系。它与 `QPaintDevice` 和 `QPaintEngine` 类一起构成了 Qt 绘画系统的基础。绘制设备的默认坐标系的原点在左上角。

`QPainter` 的主要功能是执行绘图操作。虽然 `QPaintDevice` 类是二维空间的抽象，可以使用 `QPainter` 在其上绘制，但 `QPaintEngine` 类提供了一个具有在不同类型设备上绘制的接口的绘制器。`QPaintDevice` 类是可以绘制的对象的基础类，它继承了 `QWidget`、`QImage`、`QPixmap`、`QPicture` 和 `QOpenGLPaintDevice` 类的绘制功能。

您可以在以下文档中了解有关坐标系的详细信息：

<https://doc.qt.io/qt-6/coordsys.html>

## 绘制和填充

`QPainter` 为画笔提供高度优化的功能，以满足 GUI 上的大多数绘图要求。它可以绘制从简单图形基元(如 `QPoint`、`QLine`、`QRect`、`QRegion` 和 `QPolygon` 类)到复杂形状(如矢量路径)的各种形状。矢量路径由 `QPainterPath` 类表示。`QPainterPath` 用作绘画操作的容器，允许构建和重复使用图形形状。它可以用来填充、勾勒和剪裁。`QPainter` 还可以绘制对齐的文本和像素图。要填充由 `QPainter` 绘制的形状，可以使用 `QBrush` 类。它具有颜色、样式、纹理和渐变属性，并使用颜色和样式进行定义。

在下一节中，我们将使用到目前为止讨论的 API 来使用 `QPainter` 进行绘制。

## 用 `QPainter` 绘图

`QPainter` 具有几个方便的函数来绘制最基本的形状，如 `drawLine()`、`drawRect()`、`drawEllipse()`、`drawArc()`、`drawPie()` 和 `drawPolygon()`。您可以使用 `fillRect()` 函数填充形状。`QBrush` 类描述由 `QPainter` 绘制的形状的填充图案。画笔可用于定义样式、颜色、渐变和纹理。

让我们看一下下面的 `paintEvent()` 函数，其中我们使用 `QPainter` 绘制文本和不同的形状：

```
void PaintWindow::paintEvent(QPaintEvent *event)
{
 QPainter painter;
 painter.begin(this);
 //draws a line
 painter.drawLine(QPoint(50, 50), QPoint(200, 50));
}
```

```

//draws a text
painter.drawText(QPoint(50, 100), "Text");
//draws an ellipse
painter.drawEllipse(QPoint(100,150),50,20);
//draws an arc
QRectF drawingRect(50, 200, 100, 50);
int startAngle = 90 * 16;
int spanAngle = 180 * 16;
painter.drawArc(drawingRect, startAngle, spanAngle);
//draws a pie
QRectF drawingRectPie(150, 200, 100, 50);
startAngle = 60 * 16;
spanAngle = 70 * 16;
painter.drawPie(drawingRectPie, startAngle, spanAngle);
painter.end();
QWidget::paintEvent(event);
}

```

在前面的示例中，我们已经创建了一个 `QPainter` 实例，并使用可用的默认绘图函数绘制了直线、文本、椭圆、圆弧和饼图。将上述代码添加到自定义类并运行项目时，您将看到以下输出：

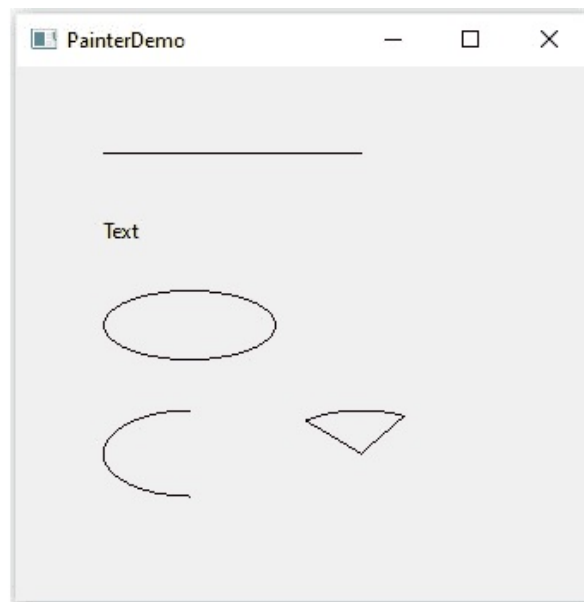


图 8.3-使用 QPainter 示例绘制的输出

Qt 提供了几个屏幕外绘制类，每个类都有自己的一组优缺点。



`QImage`、`QBitmap`、`QPixmap`和`QPicture`是涉及的类。在大多数情况下，您必须在`QImage`和`QPixmap`之间进行选择。

Qt 中的`QImage`类允许轻松读取、写入和操作图像。`QImage`是使用资源、合并多个图像并进行一些绘制时要使用的类：

```
QImage image(128, 128, QImage::Format_ARGB32);
QPainter painter(&image);
```

第一行创建一个 128 像素正方形的图像，将每个像素编码为 32 位整数-8 位用于红色、绿色和蓝色的每个不透明通道。第二行创建一个可以在`QImage`实例上绘制的`QPainter`实例。接下来，我们执行您在上一节中刚刚看到的绘图，完成后，我们将图像写入 PNG 文件，并显示以下行：

```
image.save("image.png");
```

`QImage`支持多种图像格式，包括 PNG 和 JPEG。`QImage`还有一个`load`方法，可以从文件或资源加载图像。

`QBitmap`类是一种单色屏幕外绘制设备，它提供 1 位深度的像素图。

`QPixmap`类提供屏幕外绘制设备。`QPicture`类是序列化`QPainter`命令的绘制设备。

您还可以使用`QImageReader`和`QImageWriter`类对如何加载和保存图像进行更细粒度的控制。要添加对 Qt 提供的图像格式以外的图像格式的支持，可以使用`QImageIOHandler`和`QImageIOPlugin`创建图像格式插件。

`QPainterPath`类帮助绘制可以创建和重用的不同图形形状。以下代码片段演示了如何使用`QPainterPath`：

```
void MyWidget:: paintEvent(QPaintEvent *event)
{
```

```
QPainter painter(this);
QPolygon polygon;
polygon << QPoint(100, 185) << QPoint(175, 175)
 << QPoint(200, 110) << QPoint(225, 175)
 << QPoint(300, 185) << QPoint(250, 225)
 << QPoint(260, 290) << QPoint(200, 250)
 << QPoint(140, 290) << QPoint(150, 225)
 << QPoint(100, 185);
QBrush brush;
brush.setColor(Qt::yellow);
brush.setStyle(Qt::SolidPattern);
QPen pen(Qt::black, 3, Qt::DashDotDotLine,
 Qt::RoundCap, Qt::RoundJoin);
painter.setPen(pen);
QPainterPath path;
path.addPolygon(polygon);
painter.drawPolygon(polygon);
painter.fillPath(path, brush);
QWidget::paintEvent(event);
}
```

在前面的代码中，我们创建了一个具有所需绘制路径的自定义绘制多边形对象。

## 笔记 / 便条 / 票据 / 注解

请注意，在执行绘制操作时，请确保绘制背景和绘制内容之间没有延迟。否则，如果延迟超过 **16** 毫秒，您将注意到屏幕上的闪烁。您可以通过将背景渲染到像素图中，然后将内容绘制到该像素图上来避免这种情况。最后，您可以将该像素图绘制到小部件上。这种方法称为双缓冲。

在本节中，我们不仅学习了如何在屏幕上绘制图像，还学习了如何在屏幕上绘制图像并将其另存为图像文件。在下一节中，我们将了解 Graphics View 框架的基础知识。

## 图形视图框架简介

---

图形视图框架是一个功能强大的图形引擎，它允许您可视化大量定制的 **2D** 图形项目并与之交互。如果您是一名经验丰富的程序员，您可以使用图形视图框架来绘制您的 **GUI**，并将其完全手动设置为动画。为了一次绘制成百上千个相对轻量级的自定义项目，**Qt** 提供了一个独立的视图框架，即 **Graphics View** 框架。如果您要从头开始创建自己的小工具集，或者如果要在屏幕上一次显示大量项目(每个项目都有自己的位置和数据)，则可以使用图形视图框架。这对于处理和显示大量数据的应用(如地理信息系统或计算机辅助设计软件)尤其重要。

图形视图提供了一个用于管理大量自定义创建的 **2D** 图形项目并与之交互的界面，以及一个用于可视化项目的视图小部件，并支持缩放和旋转。该框架由启用场景项目交互功能的事件传播体系结构组成。这些项响应按键事件；鼠标按下、移动、释放和双击事件；以及跟踪鼠标移动。**Graphics View** 使用二进制空间划分(**BSP**)树来提供非常快速的项目发现，即使有数百万个项目，它也可以实时可视化大型场景。

该框架遵循基于项目的 **approach** 来进行模型/视图编程。它包括三个组件，场景、视图和项。多个视图可以使用同一场景，并且场景可以包含多个项目。**Qt** 为实现 **Graphics View** 框架提供的便利类有 **QGraphicsScene**、**QGraphicsView** 和 **QGraphicsItem**。

**QGraphicsItem** 公开一个接口，您的子类可以重写该接口来管理鼠标和键盘事件、拖放、接口层次结构和冲突检测。每个项目都有自己的局部坐标系，使用辅助对象函数可以快速将项目的坐标转换为场景的坐标。图形视图框架使用一个或多个 **QGraphicsView** 实例显示 **QGraphicsScene** 类的内容。要查看场景的不同部分，可以将多个视图附加到同一场景，每个视图都有自己的平移和旋转。因为 **QGraphicsView** 小部件是滚动区域，所以您还可以将滚动条附加到视图并允许用户在视图中滚动。视

图接收键盘和鼠标输入，为场景生成场景事件，并将这些场景事件调度到场景，然后场景再将这些相同的事件调度到场景的项目。以前，该框架是游戏开发的首选。

## 重要音符

我们将跳过该框架和示例的用法细节，因为它在 Qt Quick2 问世后就不再流行了。Qt Quick2 附带了 Scene Graph API，它提供了 Graphics View 框架早先提供的大部分功能。如果您仍想了解有关图形视图框架的更多信息，可以阅读以下文档：

<https://doc.qt.io/qt-6/graphicsview.html>

在本节中，我们讨论了 Qt 的 Graphics View 框架。在下一节中，我们将了解 OpenGL 与 Qt 的集成。

## 了解 Qt OpenGL 模块

---

Qt Quick 和 Qt 小部件是在 Qt 中开发用户界面(UI)的两种主要方法。它们的存在是为了支持各种类型的 UI，并且构建在单独的图形引擎上，这些引擎已经针对每种类型进行了优化。可以在 Qt 中将 OpenGL 图形 API 代码与这两种 UI 类型结合使用。当应用包含其自己的依赖于 OpenGL 的代码或与基于 OpenGL 的第三方渲染器集成时，此选项非常有用。OpenGL/OpenGL es XML API 注册表用于生成 OpenGL 头。

Qt OpenGL 模块旨在与需要 OpenGL 访问的应用配合使用。Qt OpenGL 模块中的便利类可以帮助开发人员更轻松、更快速地构建应用。此模块负责维护与 Qt 5 应用和 Qt GUI 的兼容性。QOpenGLWidget 是一个可以将 OpenGL 场景添加到使用 QWidget 的 UI 的小工具。

随着 Qt 中引入了 Qt RHI 作为渲染基础，大部分用 `QOpenGL` 表示的类已经被移到了 Qt 6 中的 Qt OpenGL 模块中。这些类仍然可以使用，并且完全支持完全依赖 OpenGL 的应用。它们不再被认为是必需的，因为 Qt 已经扩展到支持其他图形 API，如基础中的 Direct3D、Metal 和 Vulkan。

现有的应用代码大部分将继续工作，但它现在应该在项目文件中包括 Qt OpenGL，如果它们以前是通过 Qt GUI 间接包含的，还应该包括头文件。

Qt6 不再直接使用与 OpenGL 兼容的 GLSL 源代码片段。着色器改为使用 Vulkan 样式的 GLSL 编写，反射并转换为其他着色语言，然后打包到 `QRhi` 可以使用的可序列化 `QShader` 对象中。

Qt 6 中的着色器准备管道如下所示：

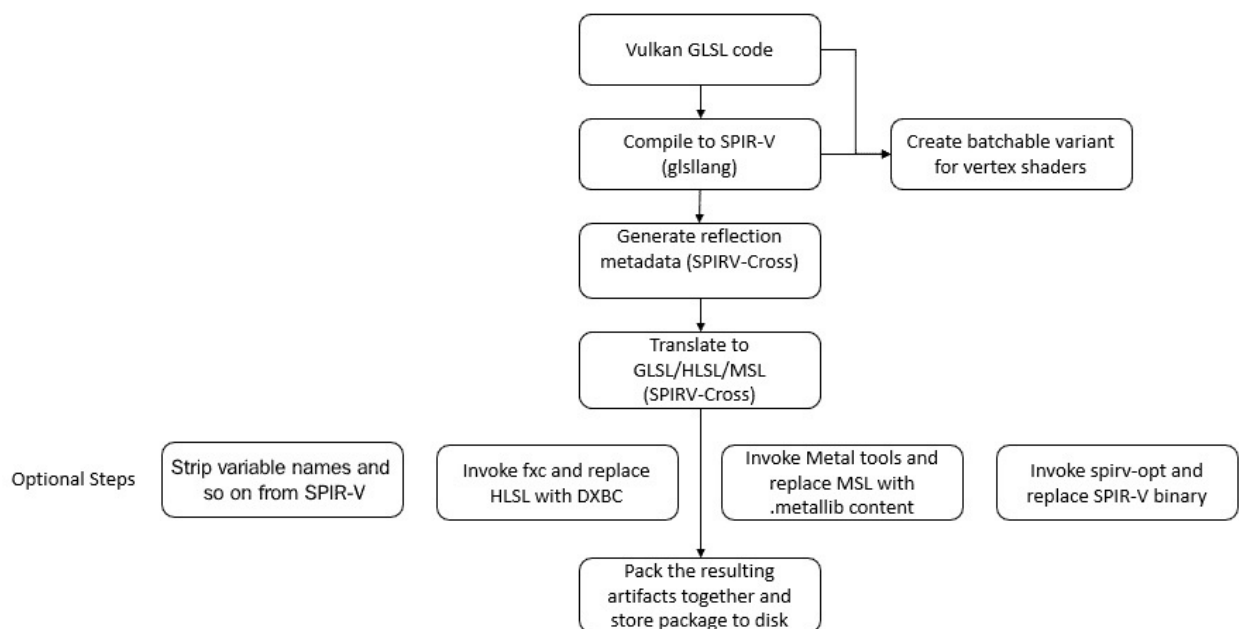


图 8.4-Qt 博客中描述的着色器准备管道的插图

在 Qt 6.1 中，Qt Data 可视化仅支持 OpenGL RHI 后端。它需要将环

环境变量 `QSG_RHI_BACKEND` 设置为 `opengl`。您可以在系统级别执行此操作，或在 `main()` 中定义它，如下所示：

```
qputenv("QSG_RHI_BACKEND","opengl");
```

在下一节中，我们将讨论框架如何与 Qt 小部件一起使用。

## Qt OpenGL 和 Qt 小工具

Qt 窗口小部件通常由高度优化和精确的软件光栅化器呈现，最终内容使用适合于运行应用的平台的方法显示在屏幕上。但是，Qt 小工具和 OpenGL 可以组合在一起。 `QOpenGLWidget` 类是这方面的主要入口点。此类可用于为小部件树的特定部分启用 OpenGL 呈现，Qt OpenGL 模块的类可用于帮助编写任何应用端 OpenGL 代码。

### 重要音符

角度是第三方 OpenGL ES 到 Direct3D 的转换器，在 Windows 上不再包含在 Qt 6 中。对于使用 OpenGL 实现的基于 `QWindow` 或 `QWidget` 的应用，除了在运行时直接调用 OpenGL API 之外别无选择。对于 Qt Quick 和 Qt Quick 3D 应用，除了 OpenGL 之外，Qt 6 还引入了对 Direct3D 11、Vulkan 和 Metal 的支持。在 Windows 上，默认选择仍为 Direct3D，因此也支持 OpenGL 以外的图形 API，从而简化了角度的删除。

在本节中，我们学习了如何使用 Qt 的 Open GL 模块。让我们继续下一节，在那里我们将详细讨论 Qt Quick 中的图形。

## Qt Quick 中的 Graphics

Qt Quick 是为利用硬件加速渲染而设计的。默认情况下，它将构建在

最适合目标平台的低级图形 API 上。例如，在 Windows 上，它将默认为 Direct3D，而在 MacOS 上，它将默认为 Metal。对于渲染，Qt Quick 应用使用场景图。场景图形渲染器可以进行更高效的图形调用，从而提高性能。场景图形具有可访问的 API，允许您创建复杂但快速的图形。Qt Quick 2D 渲染器也可用于渲染 Qt Quick。此光栅绘制引擎允许在不支持 OpenGL 的平台上渲染 Qt Quick 应用。

默认情况下，Qt 在目标平台上使用最合适的图形 API。但是，可以将 Qt 的呈现路径配置为使用特定的 API。在许多情况下，选择特定的 API 可以提高性能，并允许开发人员在支持特定图形 API 的平台上进行部署。要更改 `QQuickWindow` 中的渲染路径，可以使用 `QRhi` 界面。

在接下来的部分中，我们将了解一些功能，这些功能将进一步增强您在 Qt Quick 中的图形相关技能。让我们首先讨论如何在 Qt Quick 中使用 OpenGL。

## Qt OpenGL 和 Qt Quick

在支持 OpenGL 的平台上，可以手动选择它作为活动图形 API。为了在使用 Qt Quick 时使用这个功能，除了调整项目文件和包含头文件外，应用还应该手动将渲染后端设置为 OpenGL。

在 Qt 6 中，没有使用 Qt Quick 进行 OpenGL 渲染的直接方式。Qt 快速场景图的基于 QRhi 的渲染路径现在是新的默认设置。除了默认设置之外，配置哪个 QRhi 后端以及使用哪个图形 API 的方法从 Qt 5.15 开始基本上没有改变。Qt6 中的一个关键区别是改进了 API 命名。现在，您可以通过调用 `QQuickWindow::setGraphicsApi()` 函数来设置 RHI 后端，而之前的设置是通过调用 `QQuickWindow::setSceneGraphBackend()` 函数来实现的。

您可以在下面的文章中了解有关这些更改的更多信息：

<https://www.qt.io/blog/graphics-in-qt-6.0-qrhi-qt-quick-qt-quick-3d>

## 使用 **QPainter** 自定义 **Qt** 快捷项目

您还可以在 **Qt Quick** 应用中使用 **QPainter**。这可以通过子类化 **QQuickPaintedItem** 来实现。在这个子类的帮助下，您可以使用 **QPainter** 实例呈现内容。要渲染其内容，**QQuickPaintedItem** 子类使用间接 **2D** 表面，方法是使用软件光栅化或使用 **OpenGL FrameBuffer** 对象(**FBO**)。渲染是一个分两步进行的操作。绘制前先栅格化油漆表面。但是，使用场景图绘制要比这种光栅化方法快得多。

让我们探索一下 **Qt Quick** 使用的场景图机制。

## 了解 **Qt** 快速场景图

---

**Qt Quick 2** 使用专用场景图，该场景图使用图形 **API**(包括 **OpenGL**、**OpenGL ES**、**Metal**、**Vulkan** 或 **Direct 3D**)进行遍历和渲染。使用图形的场景图而不是传统的命令式绘制系统(**QPainter**和类似系统)，允许在帧之间保留要渲染的场景，并允许在渲染开始之前知道要渲染的整个基元集。这允许进行各种优化，包括批渲染以减少状态更改和丢弃模糊的基元。

让我们假设一个 **GUI** 由 **10** 个元素组成，每个元素都有不同的背景色、文本和图标。这将使用传统的绘制技术提供 **30** 个绘制调用和相同数量的状态更改。相反，场景图重新组织要呈现的基元，以便一个调用可以绘制所有背景、图标和文本，从而将绘制调用的总数减少到三个。这种类型的批处理和状态更改减少可以显著提高某些硬件上的性能。

场景图与 **Qt Quick 2** 有着千丝万缕的联系，不能单独使用。



`QQuickWindow`类管理和呈现场景图形，自定义的`Item`类型可以通过调用`QQuickItem::updatePaintNode()`将其图形基元添加到场景图形中。

场景图以图形方式表示`Item`场景，并且是一个自包含的结构，它有足够的信息来呈现所有项目。配置完成后，无论项目的状态如何，都可以对其进行操作和呈现。在多个平台上，场景图形甚至在单独的渲染线程上渲染，而 **GUI** 线程则为下一帧准备状态。

在接下来的几节中，我们将深入研究以提高我们对场景图结构的理解，然后学习渲染机制。此外，我们将在使用 **Qt Quick3D** 的同时混合场景图形和 **Native Graphics API**。

## Qt 快速场景图结构

场景图由种预定义的节点类型组成，每种类型都有特定的用途。虽然我们称它为场景图，但节点树是一个更精确的定义。树由 **QML** 场景中的`QQuickItem`类型构建，然后场景由绘制场景的渲染器进行内部处理。节点本身中没有激活的绘图代码。

虽然节点树主要是由现有的 **Qt Quick QML** 类型在内部构建的，但用户可以添加具有自己内容的完整子树，包括表示 **3D** 模型子树。

- 交叉点 / 节 / 淋巴结 / 结节
- 物质的 / 重要的 / 本质的 / 有关的

`QSGGeometryNode`是用户最重要的节点。它通过指定图形的几何图形和材质来创建自定义图形。`QSGGeometry`类描述图形基元的形状或网格，并用于定义几何体。它可以定义任何东西，可以是直线、矩形、多边形、一组不相连的矩形或复杂的 **3D** 网格。该材质定义如何填充特定形状的像素。一个节点可以有多个子节点。几何体节点按照子顺序渲

染，父节点可以在其子节点后面找到。

该材质描述如何填充 `QSGGeometryNode` 中的几何体内部。它封装了图形管道的顶点和碎片阶段的图形着色器，并提供了极大的灵活性，即使大多数 **Qt Quick** 项目只使用非常基本的材质，如纯色和纹理填充。

**Scene** 图形 API 是低级的，优先考虑性能而不是便利性。从头开始创建最基本的自定义几何图形和材质需要大量的代码输入。因此，API 包含几个方便的类，使得最常用的自定义节点可以轻松访问。

在下一节中，我们将讨论如何在场景图中完成渲染。

## 使用场景图渲染

场景图在内部呈现在 `QQuickWindow` 类中，并且没有公共 API 来访问它。但是，在呈现管道中有几个点，用户可以在其中插入应用代码。通过直接调用场景图的图形 API (**OpenGL**、**Vulkan**、**Metal** 等)，这些点可用于添加自定义场景图内容或插入任意渲染命令。渲染循环确定积分点。

在场景图中有两种类型的渲染循环：

- `basic` 是单线程渲染器。
- `threaded` 是在不同线程上渲染的多线程渲染器。

**Qt** 尝试根据平台和底层图形功能选择适当的呈现循环。当这还不够时，或者在测试期间，可以使用环境变量 `QSG_RENDER_LOOP` 来强制使用特定类型的渲染器循环。您可以通过启用 `qt.scenegraph.general` 日志记录类别来查找正在使用的呈现循环类型。

在大多数使用场景图的应用中，渲染在单独的渲染线程上进行。这样

做是为了提高多核处理器的并行性，并更好地利用停滞时间，例如等待阻塞的交换缓冲区调用。这提供了显著的性能改进，但它限制了可以在何时何地与时景图进行交互。

下面的图描述了如何使用线程化渲染循环和 OpenGL 来渲染帧。除了 OpenGL 上下文细节之外，其他图形 API 的步骤也是相同的：

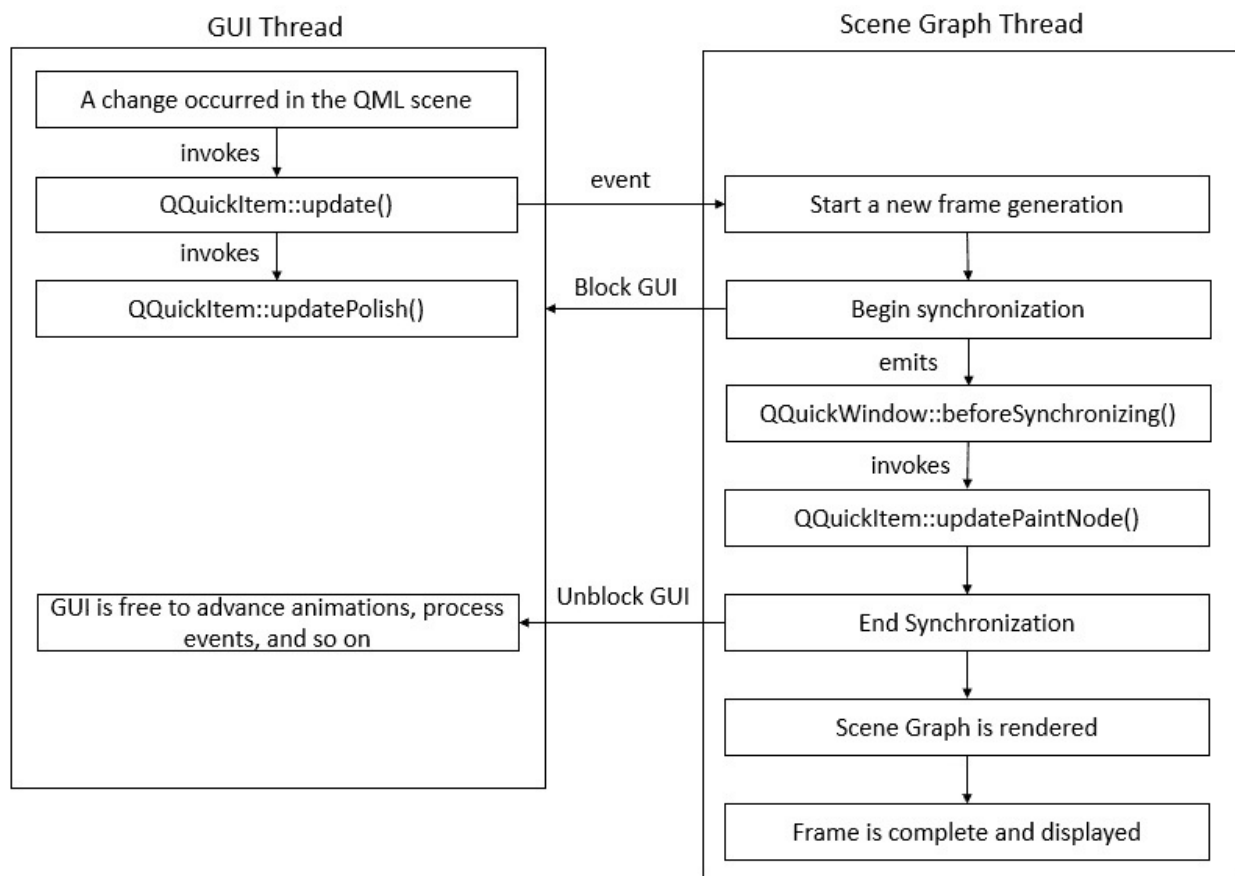
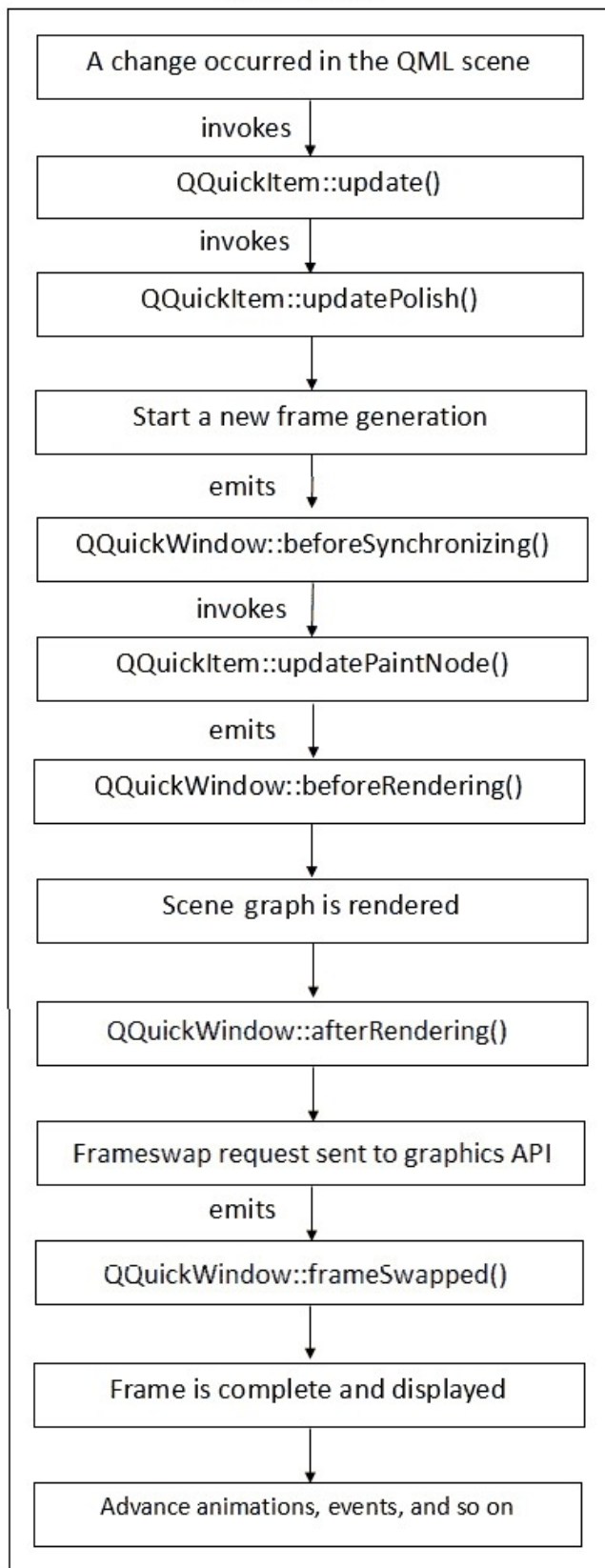


图 8.5-线程渲染循环中遵循的渲染序列

当前，在装有 Direct3D 11 或更高版本的 Windows 上默认使用线程渲染器。通过在环境中将 `QSG_RENDER_LOOP` 设置为 `threaded`，可以强制使用线程渲染器。但是，线程化呈现循环依赖于图形 API 实现进行限制。在 MacOS 和 OpenGL 上使用 Xcode 10 或更高版本构建时，不支持线程渲染循环。对于金属，没有这样的限制。

如果您的系统不能提供基于 **Vsync** 的节流，那么可以通过将环境变量 `QSG_RENDER_LOOP` 设置为 `basic` 来使用基本呈现循环。以下步骤描述如何在基本渲染循环或非线程渲染循环中渲染帧：

## GUI Thread



## 图 8.6-多线程渲染循环中遵循的渲染序列

如果未使用平台的标准 OpenGL 库，则默认情况下，在启用 OpenGL 的平台上使用多线程渲染循环。这主要是针对后者的预防性策略，因为并不是所有 OpenGL 驱动程序和窗口系统的组合都经过验证。即使您使用的是多线程呈现循环，您也可以考虑像使用线程呈现程序一样编写代码，否则您的代码将无法移植。

要查找有关场景图形渲染器工作原理的详细信息，请访问以下链接：

<https://doc-snapshots.qt.io/qt6-dev/qtquick-visualcanvas-scenegraph.html>

在本节中，您将了解场景图背后的渲染机制。在下一节中，我们将讨论如何将场景图与 Native Graphics API 混合使用。

## 将场景图形与本地图形一起使用

场景图提供用于将场景图与本地图形 API 混合的两种方法。第一种方法是直接向底层图形引擎发出命令，第二种方法是通过在场景图中生成纹理节点。通过连接到 `QQuickWindow::beforeRendering()` 和 `QQuickWindow::afterRendering()` 信号，应用可以将 OpenGL 调用直接调用到与场景图相同的上下文中。使用诸如 Metal 或 Vulkan 等 API 的应用可以通过 `QSGRendererInterface` 请求本地对象，例如场景图的命令缓冲区。然后，用户可以在 Qt Quick 场景内部或外部呈现内容。将两者混合使用的优点是不需要额外的帧缓冲区或内存来执行渲染，并且避免了可能代价高昂的纹理步骤。缺点是 Qt Quick 选择何时调用信号。OpenGL 引擎仅允许在此期间进行绘制。

从 Qt 6.0 开始，必须在调用 `QQuickWindow::beginExternalCommands()`

和 `QQuickWindow::endExternalCommands()` 函数之前调用 **Native Graphics API** 的直接使用。此方法与 `QPainter::beginNativePainting()` 相同，并且服务于相同的目的。它允许场景图标识任何缓存状态或关于当前记录的渲染过程中的状态的假设。如果存在任何内容，则它将变为无效，因为代码可能已通过直接与 **Native Graphics API** 交互而更改了它。

## 重要音符

将 **OpenGL** 内容与场景图形渲染相结合时，应用不要在 **OpenGL** 上下文中设置缓冲区限制、启用属性或模具缓冲区中的特定值或其他类似内容，这一点非常重要。如果您忘记了这一点，那么您将看到意外的行为。自定义呈现代码必须是线程感知的。

**scene** 图还支持多个日志记录类别。这些对于查找性能问题和错误的根本原因很有用。除了公共 **API** 之外，场景图还具有适配层。该层允许您实现特定于硬件的适配。它有一个内部专有的插件 **API**，允许硬件适配团队最大限度地利用他们的硬件。

## 重要音符

如果您正在观察与图形相关的问题，或者要了解当前使用哪种类型的渲染循环或图形 **API**，请通过将环境变量 `QSG_INFO` 设置为 `1` 或至少启用 `qt.scenegraph.general` 和 `qt.rhi.*` 来启动应用。在初始化期间，这将打印调试图形问题所需的一些关键信息。

## 使用 **Qt Quick 3D** 的 **3D Graphics**

**Qt Quick 3D** 是 **Qt Quick Add-on**，它提供了用于创建 **3D** 内容和 **3D** 用户界面的高级 **API**。它扩展了 **Qt Quick** 场景图，允许您将 **3D** 内容集

成到 2D Qt Quick 应用中。Qt Quick 3D 是一个高级 API，用于在 Qt Quick 平台上创建 3D 内容和 3D 用户界面。我们不依赖于引入同步问题和额外抽象层的外部引擎，而是为现有的 Qt Quick 场景图提供了空间内容扩展，并为扩展后的场景图提供了一个渲染器。使用空间场景图时，还可以混合 Qt Quick 2D 和 3D 内容。

您的 `.qml` 文件中的以下 `import` 语句可用于将 QML 类型导入到您的应用中：

```
import QtQuick3D
```

除了基本 Qt Quick 3D 模型之外，以下模块导入还提供了其他功能：

```
import QtQuick3D.Effects
import QtQuick3D.Helpers
```

Qt Quick 3D 可在商业许可下购买。从源代码构建时，请确保首先构建来自 `qtdeclarative` 和 `qtshaderutils` 存储库的模块和工具，因为 Qt Quick 3D 没有它们就无法使用。

让我们在下一节中讨论着色器工具和着色器效果。

## 着色器效果

为了将着色器导入 3D 场景，Qt Quick 3D 有自己的框架。着色器效果使图形处理单元的全部原始能力能够通过顶点和片段着色器直接利用。过多的着色器效果可能会导致功耗增加，有时还会降低性能，但如果谨慎而谨慎地使用着色器，则可以将复杂且具有视觉吸引力的效果应用于可视对象。

两个着色器都绑定到 `vertexShader` 和 `fragmentShader` 属性。每个着色器



的代码都需要一个`main(){...}`函数，该函数由 GPU 执行。前缀为`qt_`的变量由 Qt 提供。若要了解着色器代码中的变量，请查看 OpenGL API 参考文档。

使用 Qt Quick 在 QML 应用中使用`ShaderEffect`或子类化`QSGMaterialShader`时，应用必须以`.qsb`文件的形式提供烘焙着色器包。Qt ShaderTools 模块包括一个名为`QSB`的命令行工具。它包含第三方库，如`glslang`和`SPIRV-Cross`，AS 以及外部工具，如`FXC`和`spirv-opt`，并生成`.qsb`文件。特别是，`ShaderEffectQML`类型和`QSGMaterial`子类可以利用 QSB 输出。它还可用于检查`.qsb`包的内容。输入文件扩展名用于确定着色器的类型。因此，扩展必须是以下之一：

- `.vert`-顶点着色器
- `.frag`-碎片着色器
- `.comp`-计算着色器

该示例假设`myeffect.vert`和`myeffect.frag`包含 Vulkan 风格的 GLSL 代码，由`qsb`工具处理以生成`.qsb`文件。现在，我们通过以下命令使用`qsb`转换该 Vulkan 样式着色器：

```
>qsb --glsl 100es,120,150 --hlsl 50 --msl 12 -o <Output_File.qsb>
```

您可以在以下命令中看到使用上述语法的示例：

```
>C:\Qt\6.0.2\mingw81_64\bin>qsb --glsl 100es,120,150 --hlsl 50 --n
```

没有必要同时指定和`vertexShader`和`fragmentShader`。许多`ShaderEffect`实现实际上只提供片段着色器，而不是依赖内置的顶点

着色器。

您可以通过以下链接了解有关着色器工具的详细信息：

<https://doc.qt.io/qt-6/qtshader-tools-qsb.html>

让我们在示例中使用着色器效果：

```
import QtQuick
import QtQuick.Window
Window {
 width: 512
 height: 512
 visible: true
 title: qsTr("Shader Effects Demo")
 Row {
 anchors.centerIn: parent
 width: 300
 spacing: 20
 Image {
 id: originalImage
 width: 128; height: 94
 source: "qrc:/logo.png"
 }
 ShaderEffect {
 width: 160; height: width
 property variant source: originalImage
 vertexShader: "grayeffect.vert.qsb"
 fragmentShader: "grayeffect.frag.qsb"
 }
 }
}
```

在前面的示例中，我们排列了两个图像。第一个是原始图像，第二个是带有着色器效果的图像。

在本节中，您了解了 **Qt Quick** 中不同类型的着色器效果，以及如何使用 **qsb** 工具创建兼容的片段文件。在下一节中，您将学习如何使用 **Canvas** 绘图。

# 使用画布 **QML** 类型

画布允许您绘制被引用的直线和曲线、简单和复杂的形状、图形和图形图像。还可以添加文本、颜色、阴影、渐变和图案，以及低级像素操作。您可以将`Canvas`输出另存为图像。它提供了一个使用`Context2D`对象进行绘制的 **2D** 画布，并实现了一个绘制信号处理程序。

让我们来看看下面的例子：

```
import QtQuick
import QtQuick.Window
Window {
 width: 512
 height: 512
 visible: true
 title: qsTr("Canvas Demo")
 Canvas {
 id: canvas
 anchors.fill: parent
 onPaint: {
 var context = getContext("2d")
 context.lineWidth = 2
 context.strokeStyle = "red"
 context.beginPath()
 context.moveTo(100,100)
 context.lineTo(250,100)
 context.lineTo(250,150)
 context.lineTo(100,150)
 context.closePath()
 context.stroke()
 }
 }
}
```

在前面的示例中，首先，我们从`getContext("2d")`获得上下文。然后我们画了一个红色边框的矩形。输出如下所示：

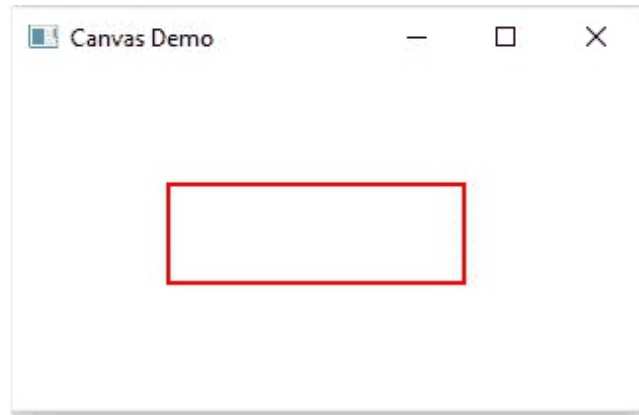


图 8.7-使用画布绘制矩形的示例应用的输出

在本节中，您熟悉了使用 `Canvas` 绘图。在下一节中，我们将讨论 `Qt Quick` 中的粒子系统。

## 了解粒子模拟

使用粒子系统，可以模拟爆炸、烟花、烟雾、雾和风等效果。`Qt Quick` 包括一个粒子系统，支持这些类型的复杂 **2D** 模拟，包括对重力和湍流等环境效果的支持。粒子最常用于游戏中，为列表或活动通知器中当前选定的项目添加微妙且视觉上吸引人的效果。

`ParticleSystem`、`Painters`、`Emitters` 和 `Affectors` 是该粒子系统中的四种主要的 **QML** 类型。`ParticleSystem` 系统包括喷涂器、发射器和效应器类型。`ParticleSystem` 类型连接所有这些类型并管理共享时间线。它们必须共享相同的 `ParticleSystem` 才能交互。受此约束的约束，您可以拥有任意多个粒子系统，因此逻辑上的分隔是针对要与之交互的所有类型使用一个 `ParticleSystem` 类型，或者如果类型数量较少且易于控制，则只使用一个类型。

要使用 `ParticleSystem`，请使用以下行导入模块：

```
import QtQuick.Particles
```

发射器产生粒子。发射器在粒子发射后不能再更改该粒子。可以使用 `affectors` 类型在粒子发射后影响它们。

每种类型的 `affector` 对粒子的影响不同：

- `Age`：修改粒子的寿命
- `Attractor`：将粒子引向特定位置
- `Friction`：减慢与粒子当前速度成比例的移动
- `Gravity`：将加速度设置为一定角度
- `Turbulence`：基于噪声图像的类液体行为
- `Wander`：随机更改路线
- `GroupGoal`：更改粒子组的状态
- `SpriteGoal`：更改精灵粒子的状态

让我们通过以下示例来了解 `ParticleSystem` 的用法：

```
ParticleSystem {
 id: particleSystem
 anchors.fill: parent
 Image {
 source: "qrc:/logo.png"
 anchors.centerIn: parent
 }
 ImageParticle {
 system: particleSystem
 source: "qrc:/particle.png"
 colorVariation: 0.5
 color: "#00000000"
 }
 Emitter {
 id: emitter
 system: particleSystem
 enabled: true
 x: parent.width/2; y: parent.height/2
 maximumEmitted: 8000; emitRate: 6000
 size: 4 ; endSize: 24
 }
}
```

```
 sizeVariation: 4
 acceleration: AngleDirection {
 angleVariation: 360; magnitude: 360;
 }
 }
}
```

在前面的代码中，我们使用了 **Qt** 徽标，该徽标在其周围发射粒子。我们已经创建了 **ImageParticle** 的一个实例，该实例创建由 **Emitter** 发射的粒子。 **AngleDirection** 类型用于确定粒子发射的角度和方向。因为我们希望在徽标周围发射粒子，所以我们对这两个属性都使用了 **360**。上例的输出如图 8.8 所示：

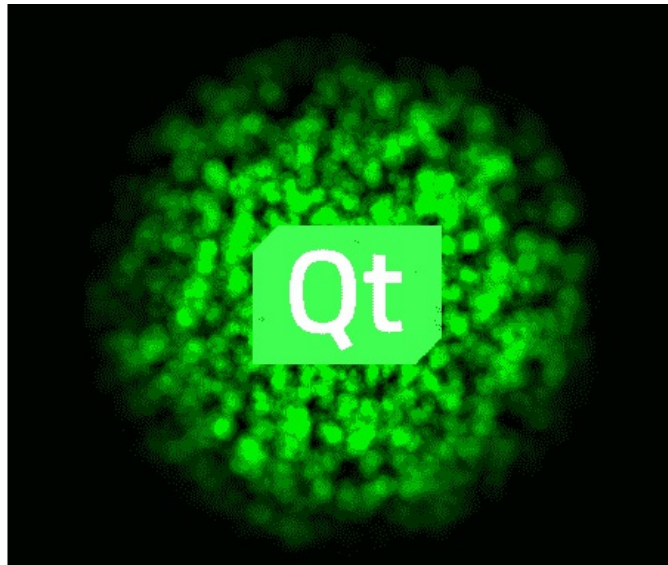


图 8.8-上述粒子系统示例的输出

您可以在以下网站上了解有关这些 QML 类型的更多信息：

<https://qmlbook.github.io/>

在本节中，我们讨论了 **Qt Quick** 中不同类型的绘制机制和组件。在下一节中，我们将学习如何在 **Qt** 小部件中制作动画。

# Qt 小工具中的动画

动画框架通过允许对 GUI 元素的属性进行动画处理，简化了动画制作 GUI 元素的过程。缓动曲线用于控制动画。缓动曲线描述控制动画速度的功能，从而产生各种加速和减速模式。Qt 支持线性、二次、三次、四次、正弦、指数、圆形和弹性缓动曲线。Qt 提供的属性动画类 (称为 `QPropertyAnimation` 类) 是为 GUI 元素设置动画的较常见方法之一。该类是动画框架的一部分，它使用 Qt 的计时器系统在指定的时间段内更改 GUI 元素的属性。

为了为我们的 GUI 应用创建动画，Qt 为我们提供了几个子系统，包括计时器、时间线、动画框架、状态机框架和图形视图框架。

让我们在下面的代码中讨论如何在 `QPushButton` 中使用属性动画：

```
QPropertyAnimation *animateButtonA = new
QPropertyAnimation(ui->pushButtonA, "geometry");
animateButtonA->setDuration(2000);
animateButtonA->setStartValue(ui->pushButtonA->geometry());
animateButtonA->setEndValue(QRect(100, 150, 200, 300));
```

在前面的代码片段中，我们以动画形式将按钮从一个位置移动到另一个位置，并更改了按钮大小。只需在调用 `start()` 函数之前将缓动曲线添加到属性动画中，即可添加缓动曲线来控制动画。您还可以尝试不同类型的缓动曲线，以查看哪种曲线最适合您。

属性动画和动画组都继承自 `QAbstractAnimator` 类。因此，可以将一个动画组添加到另一个动画组，以创建更复杂的嵌套动画组。Qt 目前提供了两种类型的动画组类：`QParallelAnimationGroup` 和 `QSequentialAnimationGroup`。

让我们使用 `QSequentialAnimationGroup` 组来管理其中动画的状态：

```
QSequentialAnimationGroup *group = new QSequentialAnimationGroup;
group->addAnimation(animateButtonA);
group->addAnimation(animateButtonB);
group->addAnimation(animateButtonC);
```

您可以通过以下链接了解有关 Qt 动画框架的更多信息：

<https://doc.qt.io/qt-6/animation-overview.html>

在本节中，我们讨论了 Qt 小部件中的动画。在下一节中，您将学习如何在 Qt Quick 中制作动画。

## Qt Quick 中的动画和过渡

在本节中，您将学习如何在 Qt Quick 中创建动画和添加过渡。要创建动画，需要为要设置动画的属性类型选择适当的动画类型，然后将动画应用于所需的行为。

Qt Quick 有几种不同类型的动画，例如以下动画：

- **Animator**：它是一种特殊类型的动画，直接在 Qt Quick 的场景图上操作。
- **AnchorAnimation**：用于制作更改锚点的动画。
- **ParallelAnimation**：它并行运行动画。
- **ParentAnimation**：它用于为父更改设置动画。
- **PathAnimation**：它沿路径为项目设置动画。
- **PauseAnimation**：它允许在动画过程中暂停。
- **PropertyAnimation**：它以动画形式显示属性值的更改。
- **SequentialAnimation**：它按顺序运行动画。
- **ScriptAction**：在动画过程中，它允许执行 JavaScript。
- **PropertyAction**：它可以在动画过程中立即更改属性，而无需为属性



更改设置动画。

图 8.9显示了动画类的层次结构：

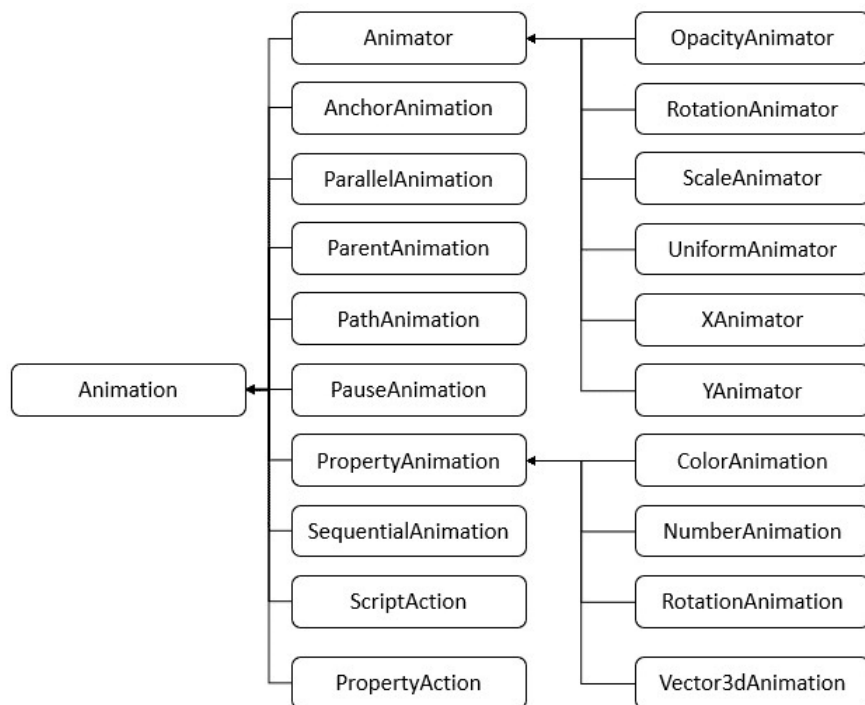


图 8.9-Qt Quick 中动画类的层次结构

`PropertyAnimation`提供了一种方法以动画形式更改属性值。

`PropertyAnimation`的不同子类如下：

- `ColorAnimation`：为颜色值的更改制作动画
- `NumberAnimation`：为`qreal`类型值的更改制作动画
- `RotationAnimation`：为旋转值的更改制作动画
- `Vector3dAnimation`：为`QVector3d`值的更改制作动画

它可用于以多种方式定义动画：

- 在`Transition`中
- 在`Behavior`中

- 截至元 T\_
- 在 `signal` 处理程序中
- 单机版

属性值通过对其应用动画类型来设置动画。要创建平滑过渡，动画类型将插入属性值。状态变换还可以将动画指定给状态更改：

- `SmoothedAnimation`：它是专门化的 `NumberAnimation` 子类。在动画中，当目标值更改时，`SmoothAnimation` 可确保平滑更改。
- `SpringAnimation`：通过其专用属性(包括质量、阻尼和 $\epsilon$ )，它提供了类似弹簧的动画。

可以通过不同方式为对象设置动画：

- 直接属性动画
- 预定义的目标和属性
- 动画作为行为
- 状态更改期间的转换

动画是通过将动画对象应用于属性值来创建的，以便随着时间的推移逐渐更改属性。通过在属性值更改之间插入值，可以在这些属性动画中使用平滑移动。属性动画允许通过缓和曲线进行不同的插值和计时控制。

下面的代码片段演示了两个使用预定义属性的 `PropertyAnimation` 对象：

```
Rectangle {
 id: rect
 width: 100; height: 100
 color: "green"
 PropertyAnimation on x { to: 200 }
 PropertyAnimation on y { to: 200 }
}
```

在上例中，动画将在加载`Rectangle`后立即开始，并自动应用于其`x`和`y`值。这里，我们使用了`<AnimationType> on <Property>`语法。因此，不需要将目标和属性值设置为`x`和`y`。

动画可以顺序或并行显示。顺序动画连续播放一组动画，而并行动画同时播放一组动画。因此，当动画在`SequentialAnimation`或`ParallelAnimation`内分组时，它们将按顺序或并行播放。

`SequentialAnimation`也可用于播放`Transition`动画，因为过渡动画会自动并行播放。您可以对动画进行分组，以确保组中的所有动画都应用于同一属性。

在下面的示例中，让我们使用`SequentialAnimation`来设置矩形的`color`的动画：

```
import QtQuick
import QtQuick.Window
Window {
 width: 640
 height: 480
 visible: true
 title: qsTr("Sequential Animation Demo")
 Rectangle {
 anchors.centerIn: parent
 width: 100; height: 100
 radius: 50
 color: "red"
 SequentialAnimation on color {
 ColorAnimation { to: "red"; duration: 1000 }
 ColorAnimation { to: "yellow"; duration: 1000 }
 ColorAnimation { to: "green"; duration: 1000 }
 running: true
 loops: Animation.Infinite
 }
 }
}
```

在前面的示例中，我们使用`<AnimationType> on <Property>`语法对`color`

属性使用了 `SequentialAnimation`。因此，子 `ColorAnimation` 对象会自动添加到此属性中，不需要 `target` 或 `property` 动画值。

可以使用 `Behavior` 动画设置默认属性动画。在 `Behavior` 类型中指定的动画将应用于属性，并为任何属性值更改设置动画。要有意启用或禁用行为动画，可以使用 `enabled` 属性。可以使用多种方法将行为动画指定给属性。其中一种方法是 `Behavior on <property>` 声明。它可以方便地将行为动画指定给属性。

`Animator` 类型与正常的 `Animation` 类型不同。让我们创建一个使用 `Animator` 旋转图像的简单示例：

```
import QtQuick
import QtQuick.Window
Window {
 width: 640
 height: 480
 visible: true
 title: qsTr("Animation Demo")
 Image {
 anchors.centerIn: parent
 source: "qrc:/logo.png"
 RotationAnimator on rotation {
 from: 0; to: 360;
 duration: 1000
 running:true
 loops: Animation.Infinite
 }
 }
}
```

在前面的示例中，我们使用了 `RotationAnimator` 类型，它用于设置 `ImageQML` 类型的旋转动画。

在本节中，我们讨论了 `Qt Quick` 中不同类型的动画，并创建了几个示例。在下一节中，我们将讨论如何控制动画。

## 控制动画

控制 **ng** 动画可以用多种方式来完成。 `Animation` 类型是所有动画类型的祖先。此类型不允许创建 `Animation` 对象。它为用户配备了使用动画类型所需的属性和方法。所有动画类型都由 `start()`、`stop()`、`resume()`、`pause()`、`restart()` 和 `complete()` 组成，它们控制动画的执行方式。

动画的起始值和结束值之间的插值由缓动曲线定义。不同的缓动曲线可能超出定义的插值范围。缓动曲线使创建动画效果(如反弹、加速、减速和循环动画)变得更容易。

在 **QML** 对象中，每个属性动画可以具有不同的缓动曲线。该曲线可以用各种参数来控制，其中一些参数是特定曲线所特有的。有关缓和曲线的详细信息，请访问缓和文档。

在本节中，您了解了如何在 **Qt Quick** 中控制动画。在下一节中，您将学习如何使用状态和转换。

## Qt Quick 中的状态、状态机和转换

---

**Qt** 快速状态是属性配置，其中属性值可以更改以反映不同的状态。状态的更改会导致属性的突然更改；动画会平滑过渡以创建视觉上吸引人的状态更改。声明性状态机框架提供了在 **QML** 中创建和执行状态图的类型。考虑将 **QML** 状态和转换用于具有独立于应用逻辑状态的多个可视状态的用户界面。

您可以通过添加以下语句将状态机模块和 **QML** 类型导入到应用中：

```
import QtQml.StateMachine
```

请注意，在 QML 中定义状态有两种方式。一个由 `QtQuick` 提供，另一个由 `QtQml.StateMachine` 模块提供。

## 重要音符

在单个 QML 文件中使用 `QtQuick` 和 `QtQml.StateMachine` 时，请确保在 `QtQuick` 之后导入 `QtQml.StateMachine`。在这种方法中，`State` 类型由声明性状态机框架提供，而不是由 `QtQuick` 提供。为了避免 `QtQuick` 的 `State` 项有任何歧义，您可以将 `QtQml.StateMachine` 导入到不同的名称空间。

要插入由状态更改引起的属性更改，`Transition` 类型可以包括动画类型。将过渡绑定到 `transitions` 属性以将其分配给对象。

按钮可以有两种状态：`pressed` 和 `released`。对于每个州，我们可以分配不同的属性配置。过渡将以动画形式从 `pressed` 过渡到 `released`。类似地，当从 `released` 切换到 `pressed` 状态时也会有动画。

让我们看一下下面的示例。

使用 `Rectangle` QML 类型创建圆形 LED，并向其添加 `MouseArea`。将默认状态指定为 `OFF`，将颜色指定为 `green`。按下鼠标时，我们想要将 LED 颜色更改为 `red`，一旦松开鼠标，LED 再次变为 `green`：

```
Rectangle {
 id: led
 anchors.centerIn: parent
 width: 100
 height: 100
 radius: 50
 color: "green"
 state: "OFF"
 MouseArea {
 anchors.fill: parent
```

```
 onPressed: led.state = "ON"
 onReleased: led.state = "OFF"
 }
 }
 }
```

接下来，定义州。在本例中，我们有两个状态`ON`和`OFF`。这里，我们根据状态更改操作`color`属性：

```
states: [
 State {
 name: "ON"
 PropertyChanges { target: led; color: "red"}
 },
 State {
 name: "OFF"
 PropertyChanges { target: led; color: "green"}
 }
]
```

可以将动画添加到过渡。让我们将`ColorAnimation`添加到过渡中，使其平滑和吸引人：

```
transitions: [
 Transition {
 from: "ON"
 to: "OFF"
 ColorAnimation { target: led; duration: 100}
 },
 Transition {
 from: "OFF"
 to: "ON"
 ColorAnimation { target: led; duration: 100}
 }
]
```

在前面的示例中，我们使用了两种状态`ON`和`OFF`。我们使用`MouseArea`根据鼠标按下和释放事件更改状态。当状态为`ON`时，矩形颜色变为`red`，当状态为`OFF`时，颜色变为`green`。这里，我们还使用`Transition`在状态之间切换。

当 `to` 和 `from` 属性绑定到状态名称时，转换将与状态更改相关联。对于简单或对称转换，将 `to` 属性设置为通配符 `"*"` 表示转换适用于任何状态更改：

```
transitions: Transition {
 to: "*"
 ColorAnimation { target: led; duration: 100 }
}
```

您可以通过以下链接了解有关状态机 QML API 的更多信息：

<https://doc.qt.io/qt-6/qmlstatemachine-qml-guide.html>

在本节中，您了解了 Qt Quick 中的状态机。在下一节中，您将学习如何在 Qt 小部件中使用状态机。

## Qt 小工具中的状态机

状态机框架中的类可用于创建和执行状态图。状态机框架提供了一个 API 和执行模型，用于在 Qt 应用中有效地嵌入状态图元素和语义。该框架与 Qt 的元对象系统紧密集成。

Qt6 中对状态机框架进行了重大更改。Qt6.0.x 核心模块中缺少 API。在 Qt6.1 中，该模块被恢复为状态机模块。因此，您将无法在 Qt6.0.x 版本中运行它，您必须将 `statemachine` 添加到 `.pro` 文件中才能使用该框架。

如果您使用的是基于 `qmake` 的构建系统，则将以下行添加到您的 `.pro` 文件中：

```
QT += statemachine
```

如果您使用的是基于 `CMake` 的构建系统，则将以下内容添加



到CMakeLists.txt:

```
find_package(Qt6 COMPONENTS StateMachine REQUIRED)
target_link_libraries(mytarget PRIVATE Qt6::StateMachine)
```

您的 C++ 源文件中需要以下标头:

```
#include <QStateMachine>
#include <QState>
```

让我们创建一个实现状态机的个简单的 Qt 小部件应用。通过添加QLabel和QPushButton修改 UI 窗体:

1. 将以下代码添加到自定义 C++ 类的构造函数中:

```
QState *green = new QState();
green->assignProperty(ui->pushButton, "text", "Green");
green->assignProperty(ui->led,
 "styleSheet", "background-color: rgb(0, 190, 0);");
green->setObjectName("GREEN");
```

2. 在前面的代码中, 我们创建了一个显示绿色 LED 的状态。接下来, 我们将为红色 LED 创建另一个状态:

```
QState *red = new QState();
red->setObjectName("RED");
red->assignProperty(ui->pushButton, "text", "Red");
red->assignProperty(ui->led, "styleSheet", "background-color: r
```

3. 切换按钮时添加状态更改事件的转换:

```
green->addTransition(ui->pushButton,
 &QAbstractButton::clicked, red);
red->addTransition(ui->pushButton,
 &QAbstractButton::clicked, green);
```

4. 现在创建一个状态机实例并向其添加状态:

```
QStateMachine *machine = new QStateMachine(this);
machine->addState(green);
machine->addState(red);
machine->setInitialState(green);
```

5. 最后一步是启动状态机：

```
machine->start();
```

6. 运行上一个示例时，您将看到如下所示的输出窗口：

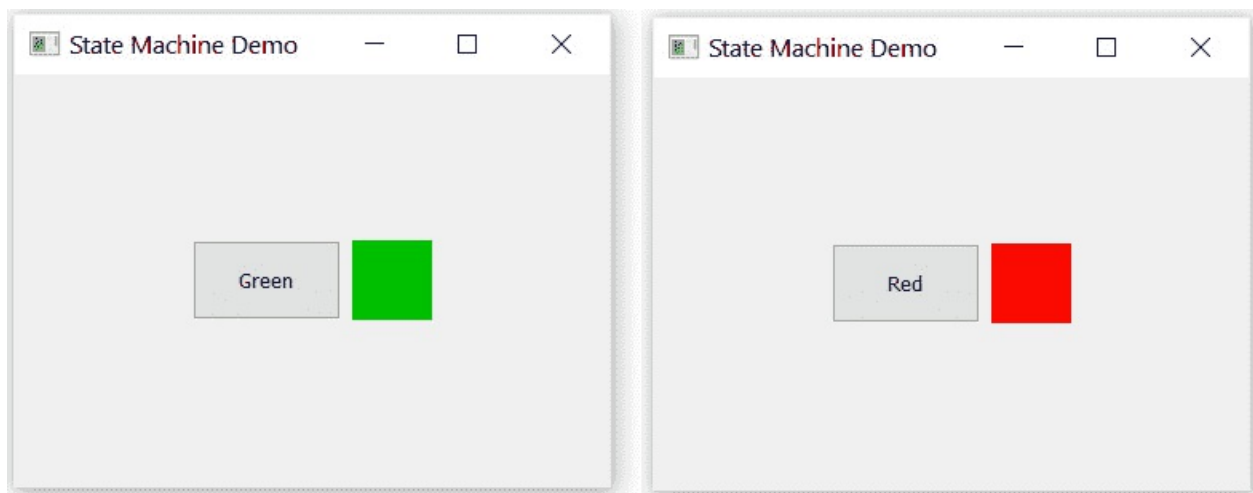


图 8.10-在 Qt 小部件中使用状态机的应用输出

上图强调，在父状态机中，只能将子状态机的状态指定为转换目标。另一方面，父状态机的状态不能指定为子状态机中转换的目标。

下面的文章很好地捕捉了使用状态机时的性能注意事项：

<https://www.embedded.com/how-to-ensure-the-best-qt-state-machine-performance/>

在本节中，我们了解了状态机及其在 Qt 小部件中的用法。我们讨论了如何在 Qt 小部件和 Qt Quick 中实现状态机。让我们总结一下我们在本章中学到的内容。

# 摘要

---

在本章中，我们讨论了不同的图形 **API**，并学习了如何使用 `QPainter` 类在屏幕上和屏幕外绘制图形。我们还研究了 **Graphics View** 框架和场景图形渲染机制。我们在本章中看到了 **Qt** 如何提供 `QPaintDevice` 接口和 `QPainter` 类来执行图形操作。我们还讨论了图形视图类、**OpenGL** 框架和着色器工具。在本章的最后，我们探索了 **Qt** 小部件和 **Qt Quick** 中的动画和状态机框架。

在[第 9 章](#)，测试和调试中，我们将学习 **Qt** 中的调试和测试。它将帮助您找到问题的根本原因并修复缺陷。

## 九、测试和调试

---

原文: <https://gitee.com/apacheecn/apacheecn-c-cpp-zh-pt2/blob/master/docs/cross-plat-dev-qt6-mod-cpp/09.md>

调试和测试是软件开发的重要组成部分。在本章中，您将学习如何调试 Qt 项目，了解不同的调试技术，以及 Qt 支持的调试器。调试是发现错误或不需要的行为的根本原因并解决它的过程。我们还将讨论使用 Qt 测试框架进行单元测试。Qt Test 是一个针对基于 Qt 的应用和库的单元测试框架。它具有大多数单元测试框架提供的所有功能。此外，它还支持测试图形用户界面(GUI)。此模块有助于以方便的方式为基于 Qt 的应用和库编写单元测试。您还将学习使用不同的 GUI 测试工具测试 GUI 的技术。

具体来说，我们将讨论以下主题：

- 在 Qt 中调试
- 调试策略
- 调试 C++ 应用
- 调试 Qt Quick 应用
- Qt 中的测试
- 与 Google 的 C++ 测试框架集成
- 测试 Qt 快速应用
- GUI 测试工具

在本章结束时，您将熟悉 Qt 应用的调试和测试技术。

## 技术要求

---

本章的技术要求包括在最新版本的桌面平台(如 Windows 10、Ubuntu

20.04 或 MacOS 10.14)上安装 Qt 6.0.0 和 Qt Creator 4.14.0 的最低版本。

本章使用的所有代码都可以从以下 giHub 链接下

载: <https://github.com/PacktPublishing/Cross-Platform-Development-with-Qt-6-and-Modern-Cpp/tree/master/Chapter09>。

重要音符

本章中使用的屏幕截图取自 Windows 平台。根据您的机器上的底层平台，您将看到类似的屏幕。

## 在 Qt 中调试

---

在软件开发中，经常会出现技术问题。要解决这些问题，我们必须首先找出并解决所有问题，然后才能向公众发布我们的应用，以保持质量和声誉。调试是一种定位这些潜在技术问题的技术。

在接下来的几节中，我们将讨论软件工程师为确保软件的稳定性和质量而使用的流行调试技术。

### Qt 支持的调试器

Qt 支持几种不同类型的调试器。根据项目使用的平台和编译器的不同，您使用的调试器可能会有所不同。以下是广泛用于 Qt 的调试器列表：

- **GNU Symbol Debugger(gdb)**是 GNU Project 开发的跨平台调试器。
- **Microsoft Console Debugger(CDB)**是 Microsoft for Windows 的调试器。

- **Low Level Virtual Machine Debugger(LLDB)**是 LLVM 开发小组开发的跨平台调试器。
- **QML/JavaScript** 调试器是 Qt 公司提供的 QML 和 JavaScript 调试器。

如果您在 Windows 上使用 MinGW 编译器，则不需要使用 **gdb** 进行任何手动设置，因为它通常包含在 Qt 安装中。如果您使用的是不同的操作系统，如 Linux，则可能需要在将其链接到 Qt Creator 之前手动安装。Qt Creator 自动检测 **gdb** 的存在，并将其添加到其调试器列表中。

您还可以使用**Valgrind**来调试您的应用。您可以通过指定 `--vgdb=yes` 或 `--vgdb=full` 来激活 **Valgrindgdbserver**。您可以指定 `--vgdb-error=number` 在显示一定数量的错误后激活 **gdbserver**。如果将该值设置为0，则 **gdbserver** 将在初始化时处于活动状态，从而允许您在应用启动之前设置断点。值得注意的是，**vgdb** 包含在**Valgrind**发行版中。它不需要单独安装。

如果您喜欢的平台是 Windows，您可以在您的机器上安装 **CDB**。默认情况下，**Visual Studio** 的内置调试器将不可用。因此，在安装 **Windows SDK** 时，您必须选择 **Debug Tools for Windows** 作为可选组件，从而单独安装 **CDB** 调试器。Qt Creator 通常会识别 **CDB** 的存在，并将其添加到选项下的调试器列表中。

**Android** 调试比在常规桌面环境中调试更具挑战性。**Android** 开发需要不同的包，如 **JDK**、**Android SDK**、**Android NDK** 等。在桌面平台上，您需要**Android Debug Bridge(ADB)**驱动程序才能进行 **USB** 调试。您必须启用开发人员模式并接受 **Android** 设备上的 **USB** 调试才能继续。

MacOS 和 iOS 上使用的调试器是**LLDB**。默认情况下，它包含在 Xcode 中。Qt Creator 将自动检测其存在并将其与套件链接。如果您熟悉调试器并知道自己在做什么，还可以将非 **gdb** 调试器添加到您最喜欢的 IDE 中。

调试器插件根据您计算机上可用的内容为每个包确定合适的本机调试器。您可以通过添加新的调试器来克服此首选项。您可以在**Options** 菜单下的**Kits**设置下的**Debuggers**选项卡中找到可用的调试器，如图 9.1所示：

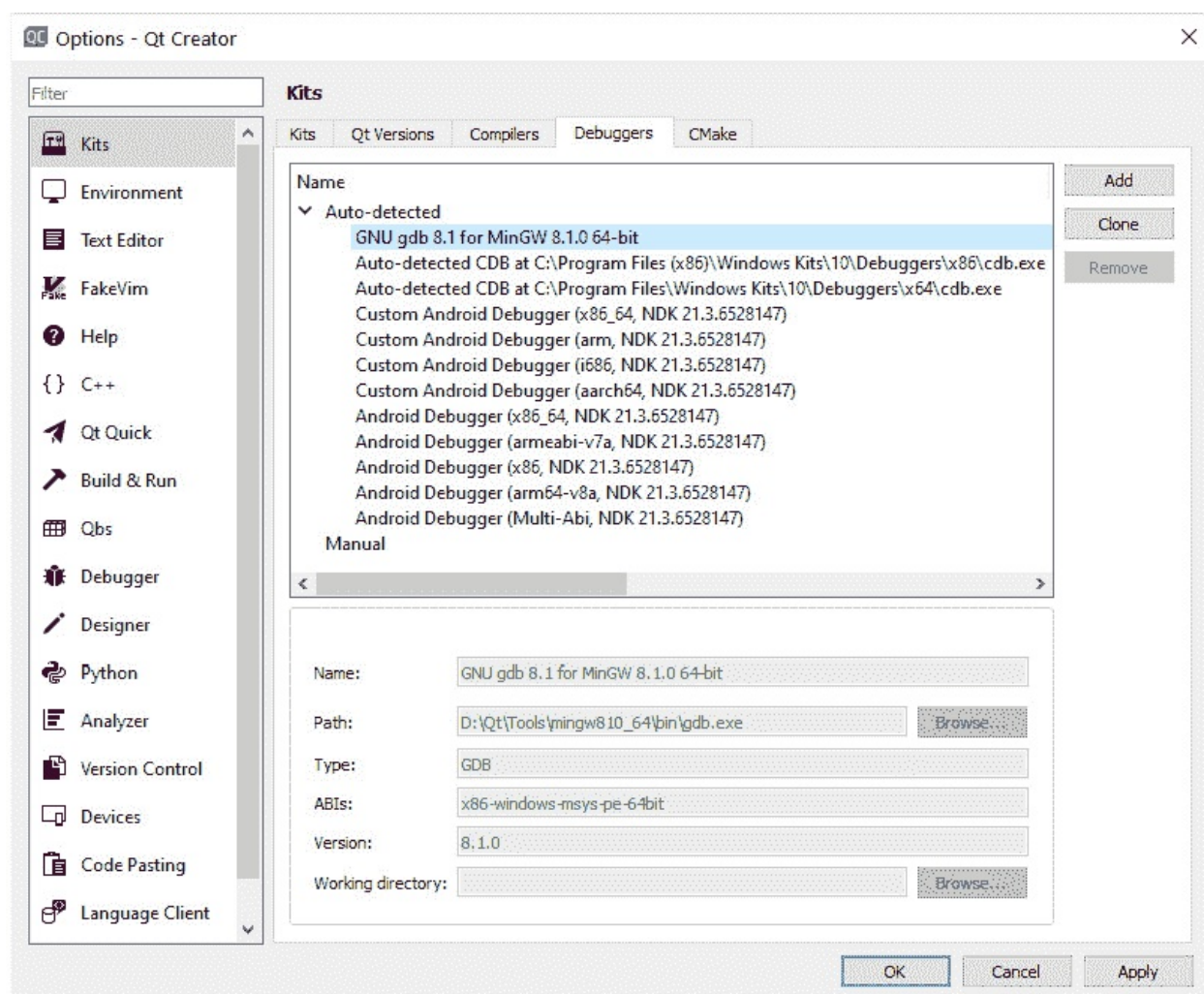


图 9.1-显示 Add 按钮的 Kits Selection(工具包选择)屏幕下的

## Debuggers(调试器)选项卡

在**Debuggers**选项卡中，您可以在右侧看到**Add**、**Clone**和**Remove**按钮。您可以克隆现有的调试器配置，并对其进行修改以满足您的需求。或者，如果您知道调试器的详细信息和配置，则可以使用**Add**按钮创建新的调试器配置。您还可以通过单击删除按钮来删除有故障或过时的调试器配置。不要忘记单击应用按钮保存您的更改。请注意，您不能修改自动检测到的调试器配置。

在本节中，我们了解了各种受支持的调试器。在下一节中，我们将讨论如何调试应用。

## 调试策略

---

有不同的调试策略可以找到问题的根本原因。在尝试定位应用中的错误之前，彻底了解程序或库是至关重要的。如果你不知道自己在做什么，你就找不到错误。只有对系统及其操作方式有了透彻的了解，才能识别应用中的错误。以前的经验有助于检测类似类型的错误以及解决错误。个别专家的知识决定了开发人员定位错误的难易程度。您可以添加调试打印语句和断点来分析程序流。您可以进行正向分析或反向分析来跟踪错误的位置。

调试时使用以下步骤查找根本原因并解决问题：

1. 找出问题所在。
2. 找到问题所在。
3. 分析问题。
4. 解决问题。
5. 解决副作用。



不管是哪种编程语言或平台，调试应用时要知道的最重要的事情是代码的哪一部分导致了问题。您可以通过多种方式找到有缺陷的代码。

如果缺陷是由您的 **QA** 团队或用户提出的，请询问问题发生的时间。查看日志文件或任何错误消息。注释掉可疑的代码部分，然后再次构建并运行应用，看看问题是否仍然存在。如果问题是可重现的，则在找到导致问题的代码之前，通过打印消息和注释掉代码行来进行向前和向后分析。

您还可以在内置调试器中设置断点，以搜索目标功能中的变量更改。如果其中一个变量更新为意想不到的值，或者对象指针变成无效指针，那么您可以很容易地识别它。检查您在安装程序中使用的所有模块，并确保您和您的用户具有相同的应用版本号。如果您使用的是不同的版本或不同的分支，那么请使用指定的版本标签签出分支，然后调试代码。

在下一节中，我们将讨论如何通过打印调试消息和添加断点来调试 **C++** 代码。

## 调试 **C++** 应用

---

可以使用 `QDebug` 类将变量的值打印到应用输出窗口。`QDebug` 类似于标准库中的 `std::cout`，但它的优点是成为 **Qt** 的一部分，这意味着它支持开箱即用的 **Qt** 类，并且无需转换即可显示其值。

要启用调试消息，我们必须包括 `QDebug` 报头，如下所示：

```
#include <QDebug>
```

**Qt** 提供了几个全局宏，用于生成不同类型的调试消息。它们可以用于

不同的目的，如下所述：

- `QDebug()` 提供自定义调试消息。
- `qInfo()` 提供信息性消息。
- `qWarning()` 报告警告和可恢复的错误。
- `qCritical()` 提供严重错误消息并报告系统错误。
- `qFatal()` 在退出前提供致命错误消息。

您可以使用 `QDebug()` 查看您的功能是否正常工作。查找完错误后，删除包含 `QDebug()` 的代码行，以避免不需要的控制台日志。让我们通过一个示例来看看如何使用 `QDebug()` 将变量输出到输出窗格。创建示例 `QWidget` 应用并添加函数 `setValue(int value)`，然后在函数定义中添加以下代码：

```
int value = 500;
QDebug() << "The value is : " << value;
```

上述代码将在 **Qt Creator** 底部的输出窗口中显示以下输出：

```
The value is : 500
```

您可以通过查看函数在应用中被使用和调用的次数来判断值是否被另一个函数更改。如果多次打印调试消息，则会从多个位置调用它。检查是否向所有调用函数发送了正确的值。要消除输出控制台窗口中不必要的控制台日志，请在查找完问题后删除包含 `QDebug()` 的代码行。或者，您可以实现条件编译。

让我们进一步了解 **Qt Creator** 中的调试和调试选项：

1. You can see a **Debug** menu in the menu bar. When you click on it, you will see a context menu with submenus as shown in *Figure*

9.2:

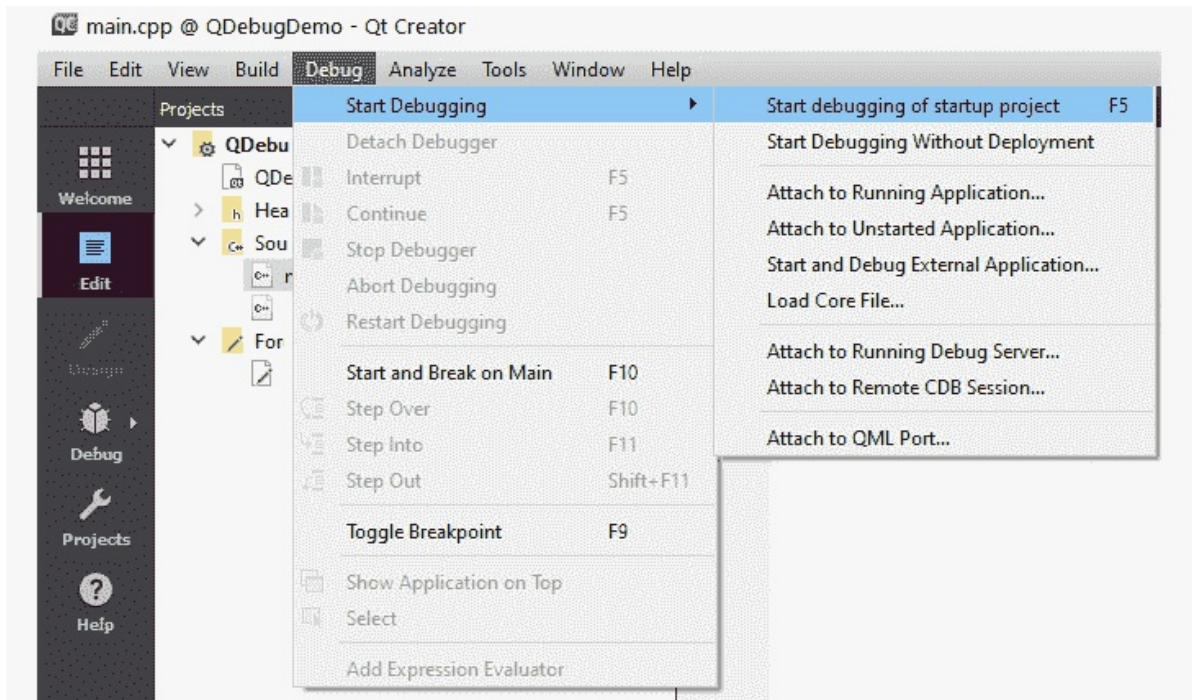


图 9.2Qt 创建器中的←调试菜单

2. To start debugging, press *F5* or click on the start **Debug** button at the bottom left of Qt Creator as shown here:



图 9.3-Qt Creator 中的开始调试按钮

3. 如果 Qt Creator 以错误消息抱怨调试器，则检查您的项目包是否有

调试器。

4. 如果错误仍然存在，请关闭 **Qt Creator** 并转到您的项目文件夹，您可以在其中删除 `.pro.user` 文件。
5. 然后在 **Qt Creator** 中重新加载项目。您的项目将由 **Qt Creator** 重新配置，调试模式现在应该可以工作了。

调试应用的一个很好的方法是设置断点：

1. 当您在 **Qt Creator** 中右键单击脚本的行号时，您将看到一个包含三个选项的弹出式菜单。
2. 您还可以单击行号来添加断点。单击行号以设置断点。您将看到行号上出现一个红点。
3. Next, press the *F5* key on the keyboard or click on the start **Debug** button. Once you run the application in debug mode, you will notice a yellow arrow appearing on top of the first red dot:

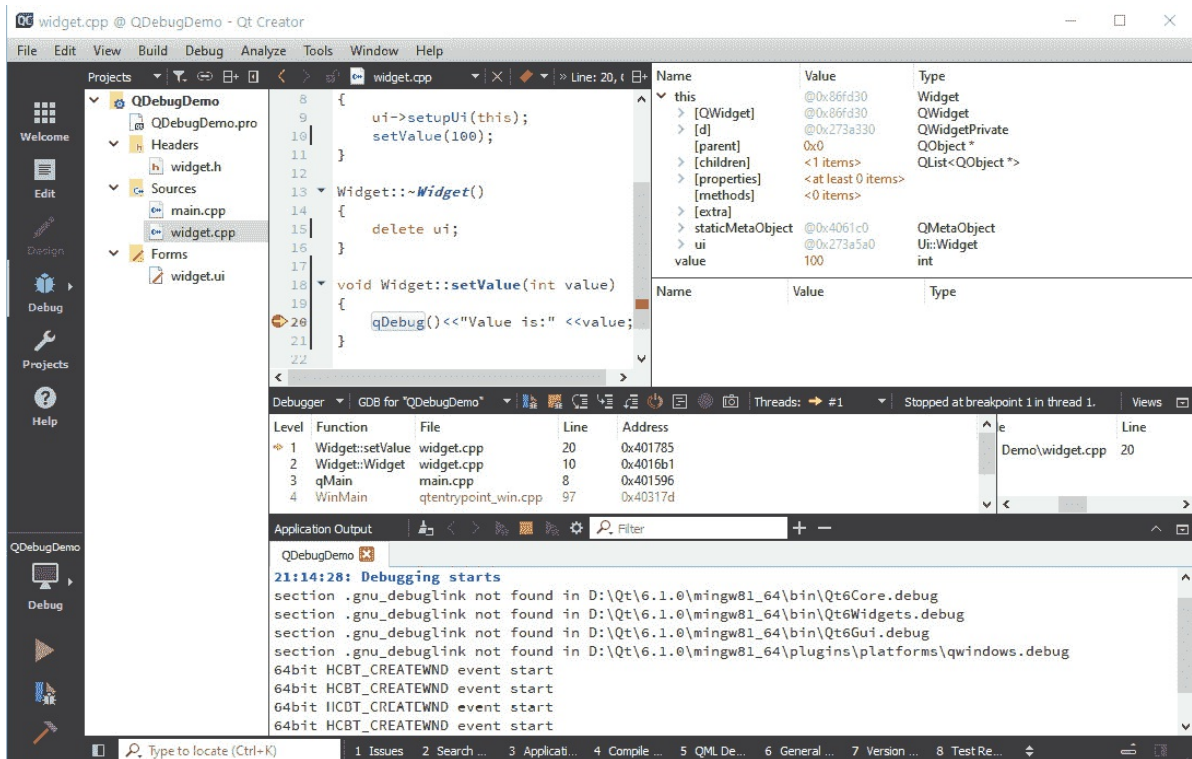


图 9.4-Qt Creator 显示调试窗口和断点

4. 调试器在第一个断点处停止。变量及其含义和类型现在将显示在 Qt Creator 右侧的**Locals**和**Expression**窗口中。
5. 此方法可用于快速检查应用。要删除断点，只需再次单击红点图标或从右键单击上下文菜单中：

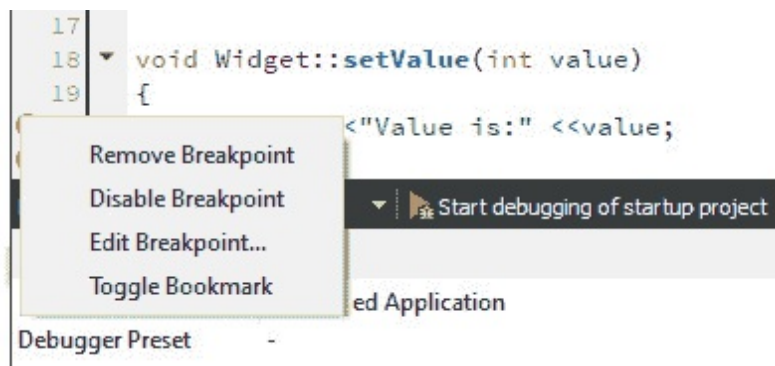


图 9.5-显示断点标记上的右键单击选项的上下文菜单

重要的是要记住，您必须在调试模式下运行您的应用。这是因为当您在调试模式下编译时，您的应用或库将具有其他调试符号，这些符号允许调试器从二进制文件的源代码访问信息，例如标识符、变量和函数的名称。这就是应用或库二进制文件在调试模式下编译时文件大小较大的原因。

您可以在以下文档中了解更多功能及其用法：

<https://doc.qt.io/qt-6/debug.html>

### 重要音符

某些防病毒应用阻止调试器检索信息。**Avira** 就是这样一种抗病毒药物。如果在生产 **PC** 上安装了，则在 **Windows** 平台上启动调试器可能会失败。

在下一节中，我们将讨论如何调试 **Qt Quick** 应用并定位 **QML** 文件内部的问题。

## 调试 **Qt Quick** 应用

---

在上一节中，我们讨论了如何调试 **C++** 代码。但是您可能仍然想知道如何调试用 **QML** 编写的代码。**Qt** 还提供了调试 **QML** 代码的功能。当您开发 **Qt Quick** 应用时，有很多选项可以解决问题。在本节中，我们将讨论与 **QML** 相关的各种调试技术以及如何使用它们。

就像 **QDebug** 类一样，有不同的控制台 **API** 可用于在 **QML** 中进行调试。这些建议如下：

- **Log**：用于打印一般消息。
- **Assert**：用于验证表达式。

- `Timer`: 用于测量两次调用之间花费的时间。
- `Trace`: 这用于打印 JavaScript 执行的堆栈跟踪。
- `Count`: 此参数用于查找调用函数的次数。
- `Profile`: 这用于分析 QML 和 JavaScript 代码。
- `Exception`: 用于打印错误信息。

`ConsoleAPI` 提供了几个方便的函数来打印不同类型的调试消息，如 `console.log()`、`console.debug()`、`console.info()`、`console.warn()` 和 `console.error()`。您可以打印带有参数值的消息，如下所示：

```
console.log("Value is:", value)
```

您还可以通过在 `Components.onCompleted:{...}` 中添加消息来检查组件的创建：

```
Components.onCompleted: {
 console.log("Component created")
}
```

要验证表达式是否为真，可以使用 `console.assert()`，例如：

```
console.assert(value == 100, "Reached the maximum limit");
```

您会发现，`console.time()` 和 `console.timeEnd()` 记录了两次呼叫之间花费的时间。调用阶段的 JavaScript 执行的堆栈跟踪由 `console.trace()` 打印。函数名、文件名、行号和列号都包含在堆栈跟踪详细信息中。

`console.count()` 返回一段代码的当前执行次数，以及一条消息。QML 和 JavaScript 评测在您使用 `console.profile()` 时激活，在调用 `console.profileEnd()` 时停用。您可以使用 `console.exception()` 打印错误消息以及 JavaScript 执行的堆栈跟踪。

您可以按照我们在前面小节中讨论的相同方式添加断点，如下所示：

- 要单步执行堆栈中的代码，请单击工具栏上的单步执行按钮或按**F11**。
- 要退出，请按**Shift+F11**。要命中断点，请在方法末尾添加一个断点，然后单击继续。
- 打开 **QML** 调试器控制台输出窗格，在当前上下文中运行 **JavaScript** 命令。

您可以在运行 **Qt Quick** 应用时发现问题并观察值。它将帮助您找到导致意外行为并需要修改的代码部分。

在本节中，我们了解了如何在 **QML** 环境中进行调试。在下一节中，我们将讨论 **Qt** 中的测试框架。

## Qt 测试

---

单元测试是使用自动化工具测试简单应用、类或功能的一种方式。我们将讨论它是什么，以及为什么我们想要这样做，然后再讨论如何使用 **Qt Test** 将它合并到我们的方法中。单元测试是将应用分解成其最小的功能单元，然后在计划框架内根据实际情况测试每个单元的过程。单元是可以测试的应用中最小的组件。过程性编程中的单元测试通常关注函数或进程。

面向对象编程中的单元通常是接口、类或单个函数。单元测试在实现过程的早期识别问题。这包括程序员实现中的小故障，以及单元规范中的缺陷或不完整部分。在创建过程中，单元测试是由待测试单元的开发人员开发的简短代码片段。有许多单元测试工具可以测试您的 **C++** 代码。让我们探索一下 **Qt** 测试框架的好处和特性。



## Qt 中的单元测试

Qt Test 是一个针对基于 Qt 的应用和库的单元测试平台。Qt 测试包括传统单元测试应用中存在的所有特性，以及用于测试图形用户界面的插件。它有助于为基于 Qt 的程序和库编写单元测试。图 9.6 显示了选项下的测试部分：

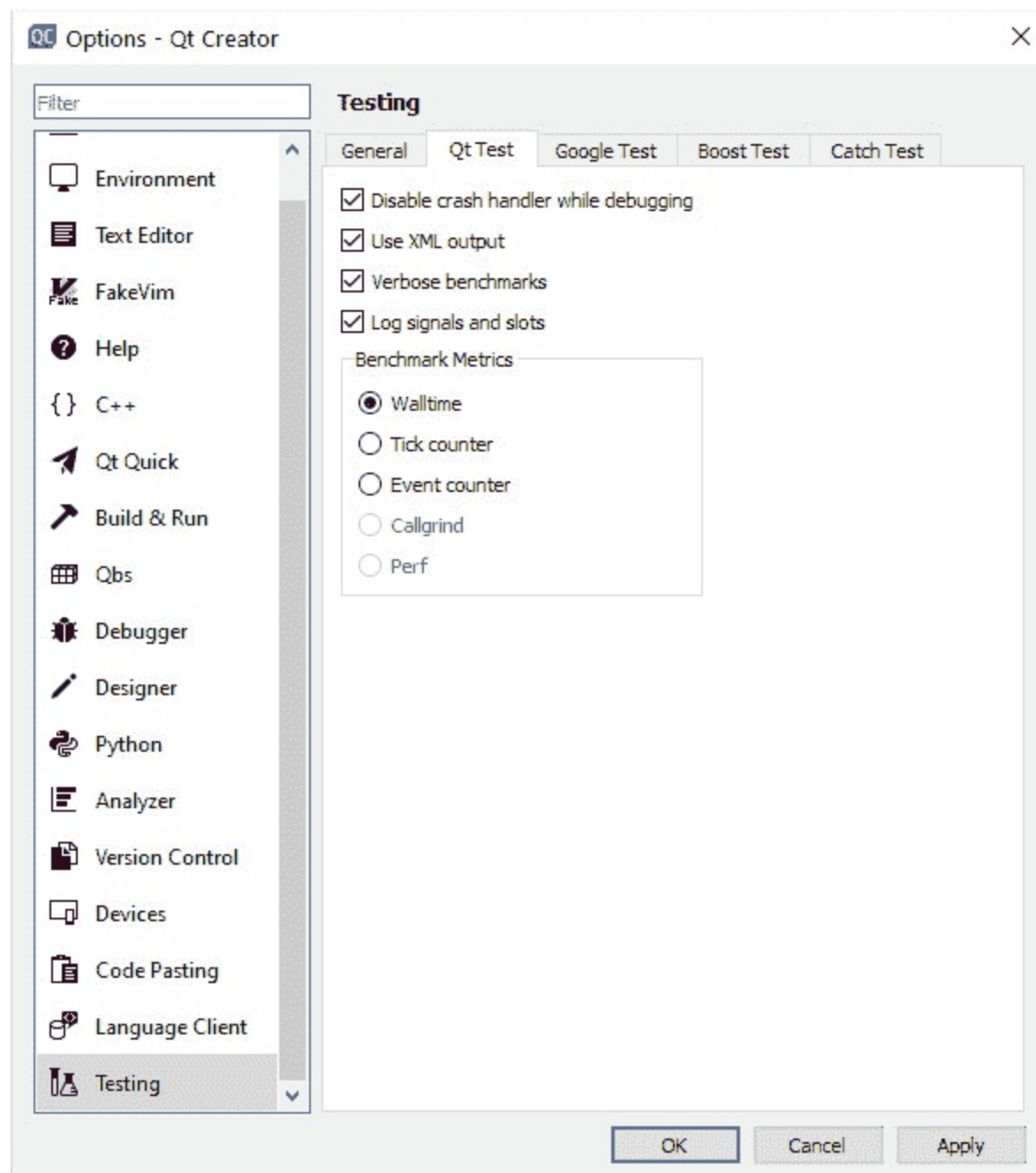


图 9.6-显示 Qt Creator Options(Qt Creator 选项)菜单下的 Qt Test Preferences(Qt 测试首选项)的屏幕截图

以前，单元测试可能是手动完成的，特别是对于 GUI 测试，但是现在有一个工具允许您编写代码来自动验证代码，这乍一看可能有悖常理，但它可以正常工作。Qt 测试是基于 Qt 的单元测试专用测试框架。

您必须在项目文件(.pro)中添加testlib才能使用 Qt 的内置单元测试模块：

```
QT += core testlib
```

接下来，运行qmake以添加可用于您的项目的模块。为了让测试系统找到并实现它，您必须使用QTest标头并将测试函数声明为私有插槽。

QTest标头包含与 Qt 测试相关的所有函数和语句。要使用QTest功能，只需将以下行添加到 C++ 文件：

```
#include <QTest>
```

您应该为每个可能的场景编写测试用例，然后在每次基线代码更改时运行测试，以确保系统继续按预期运行。它是一个非常有用的工具，可以确保所做的任何编程更新都不会破坏现有功能。

让我们使用 Qt Creator 的内置向导创建一个简单的测试应用。从新建项目菜单中选择自动测试项目，如图 9.7所示：

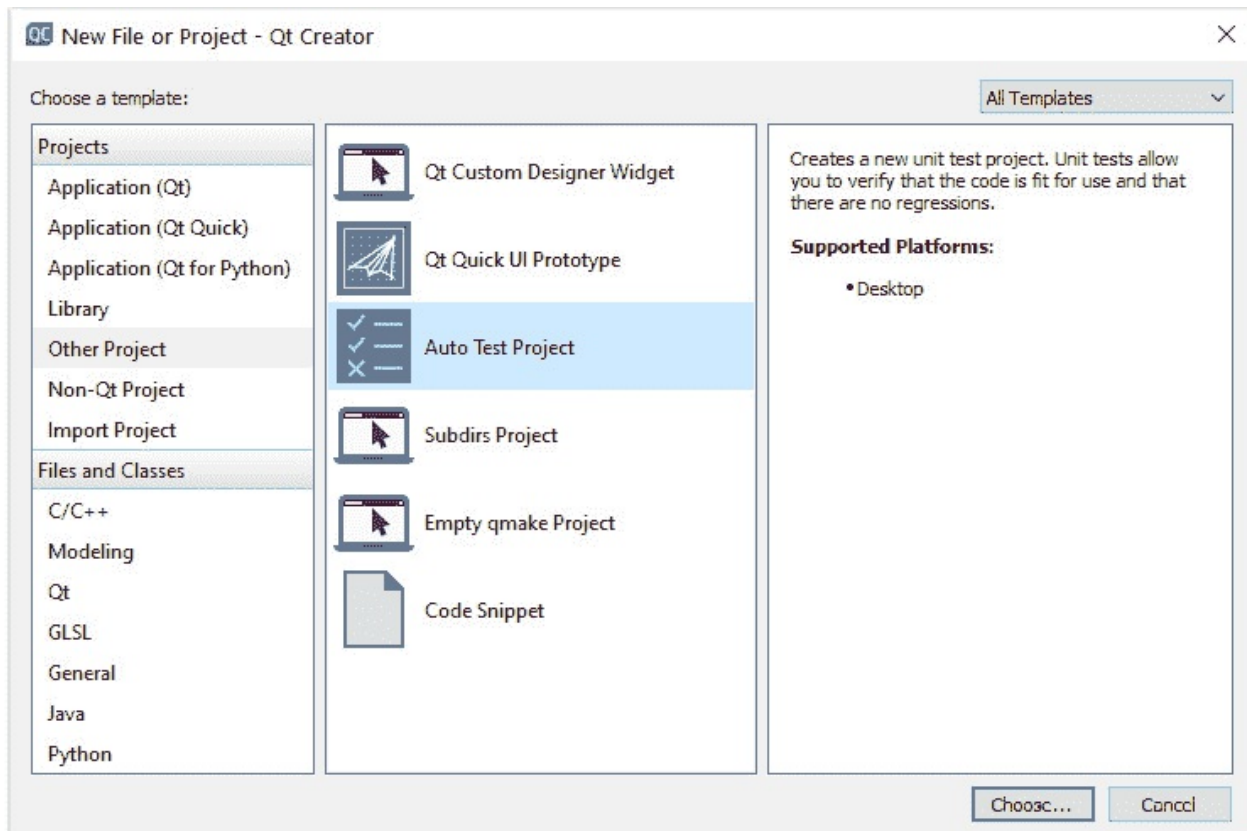


图 9.7-项目向导中的新建自动测试项目选项

一旦测试项目框架生成，您就可以修改生成的文件以满足您的需要。打开测试项目的 `.pro` 文件，并添加以下代码行：

```
QT += testlib
QT -= gui
CONFIG += qt console warn_on depend_includepath testcase
CONFIG -= app_bundle
TEMPLATE = app
SOURCES += tst_testclass.cpp
```

让我们创建一个名为 `TestClass` 的 C++ 类。我们将把我们的测试函数添加到这个类中。此类必须从 `QObject` 派生。让我们来看 `tst_testclass.cpp`：

```
#include <QtTest>
class TestClass : public QObject
```

```

{
 Q_OBJECT
public:
 TestClass() {}
 ~TestClass(){}
private slots:
 void initTestCase(){}
 void cleanupTestCase() {}
 void test_compareStrings();
 void test_compareValues();
};

```

在前面的代码中，我们已经声明了两个测试函数来测试样本字符串和值。您需要使用声明的测试用例的测试场景来实现测试功能。让我们比较两个字符串并做一个简单的算术运算。您可以使用 `QCOMPARE` 和 `QVERIFY` 等宏来测试值：

```

void TestClass::test_compareStrings()
{
 QString string1 = QLatin1String("Apple");
 QString string2 = QLatin1String("Orange");
 QCOMPARE(string1.localeAwareCompare(string2), 0);
}
void TestClass::test_compareValues()
{
 int a = 10;
 int b = 20;
 int result = a + b;
 QCOMPARE(result, 30);
}

```

要执行所有测试用例，您必须在文件底部添加像 `QTEST_MAIN()` 这样的宏。 `QTEST_MAIN()` 宏扩展为运行所有测试函数的简单 `main()` 方法。 `QTEST_APPLESS_MAIN()` 宏对于不使用 `QApplication` 对象的简单独立非 GUI 测试非常有用。 如果不需要 GUI 但需要事件循环，请使用 `QTEST_GUILESS_MAIN()`：

```

QTEST_APPLESS_MAIN(TestClass)

```

```
#include "tst_testclass.moc"
```

为了使测试用例成为独立的可执行文件，我们为类添加了 `QTEST_APPLESS_MAIN()` 宏和 `moc` 生成的文件。您可以使用许多其他宏来测试应用。有关更多信息，请访问以下链接：

<http://doc.qt.io/qt-6/qtest.html#macros>

当您运行前面的示例时，您将看到带有测试结果的输出，如下所示：

```
***** Start testing of TestClass *****
Config: Using QtTest library 6.1.0, Qt 6.1.0 (x86_64-little_endian
64bit HCBT_CREATEWND event start
PASS : TestClass::initTestCase()
FAIL! : TestClass::test_compareStrings() Compared values are not
 Actual (string1.localeAwareCompare(string2)): -1
 Expected (0) : 0
..\TestProject\tst_testclass.cpp(26) : failure location
PASS : TestClass::test_compareValues()
PASS : TestClass::cleanupTestCase()
Totals: 3 passed, 1 failed, 0 skipped, 0 blacklisted, 7ms
***** Finished testing of TestClass *****
```

您可以看到一个测试用例失败，因为它不符合测试标准。类似地，您可以添加更多测试用例并从另一个类获取参数来测试功能。您还可以在 **Qt Creator** 菜单栏的测试上下文菜单中使用运行所有测试选项来运行所有测试，如图 9.8 所示：

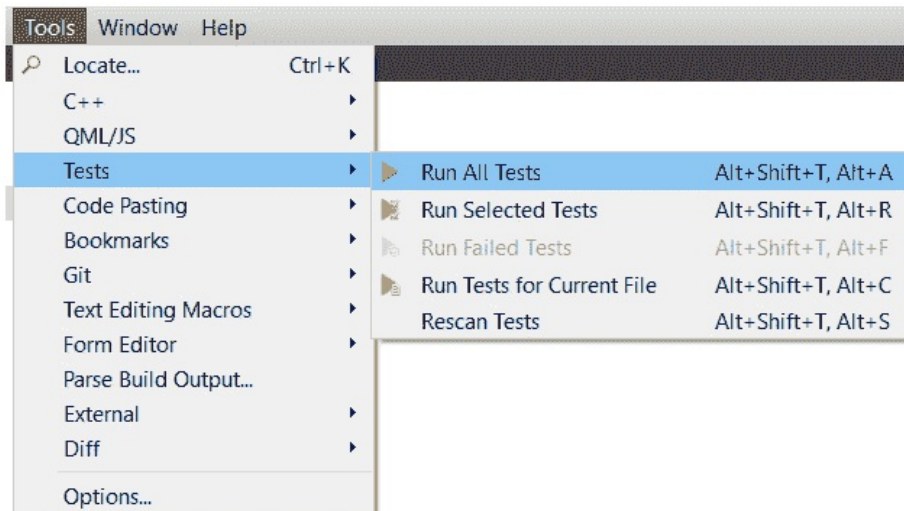


图 9.8-“Tools(工具)”菜单下的“Tests(测试)”选项

您还可以在左侧的项目资源管理器视图中查看所有测试用例。从项目资源管理器下拉菜单中选择**Tests**。您可以在此窗口中启用或禁用某些测试用例。图 9.9显示了我们之前编写的两个测试用例。您还可以看到，我们没有对此测试项目使用其他测试框架：

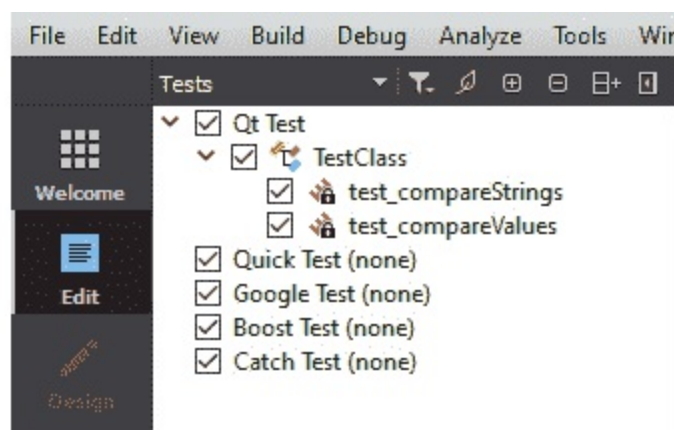


图 9.9-项目资源管理器下拉菜单中的测试资源管理器选项

您可以使用几个**QTest**方便的函数来模拟 GUI 事件，如键盘或鼠标事件。让我们用一个简单的代码片段来看看它们的用法：

```
QTest::keyClicks(testLineEdit, "Enter");
```

```
QCOMPARE(testLineEdit->text(), QString("Enter"));
```

在前面的代码中，测试代码模拟 `lineEdit` 控件上的键盘文本 `Enter` 事件，然后验证输入的文本。您还可以使用 `QTest::mouseClick()` 模拟鼠标单击事件。您可以按如下方式使用它：

```
QTest::mouseClick(testPushBtn, Qt::LeftButton);
```

Qt 的测试框架在测试驱动开发(TDD)中也很有用。在 TDD 中，您首先编写测试，然后编写实际逻辑。测试最初将失败，因为没有实现。然后编写通过测试所需的最少代码，然后再进行下一个测试。这就是在实现必要功能之前迭代开发功能的方式。

在这一节中，我们学习了如何创建测试用例和模拟 GUI 交互事件。在下一节中，您将学习如何使用 Google 的 C++ 测试框架。

## 与 Google 的 C++ 测试框架集成

**GoogleTest**是由 Google 开发的测试和模仿框架。**GoogleMock**项目已经合并到 **GoogleTest** 中。**GoogleTest** 要求编译器至少支持 C++ 11 标准。它是一个跨平台的测试框架，支持主要的桌面平台，如 Windows、Linux 和 MacOS。它可以帮助您编写具有模拟等高级功能的更好的 C++ 测试。您可以将 Qt Test 与 **GoogleTest** 集成，以充分利用这两个框架。如果您打算同时使用这两个测试框架特性，那么您应该使用 **GoogleTest** 作为主要测试框架，并且在测试用例中，您可以使用 Qt Test 的特性。

Qt Creator 内置了对 **GoogleTest** 的支持。您可以在**Options**屏幕上的**Testing**部分找到**Google Test**选项卡，并设置全局 **GoogleTest** 首选项，如图 9.10所示：

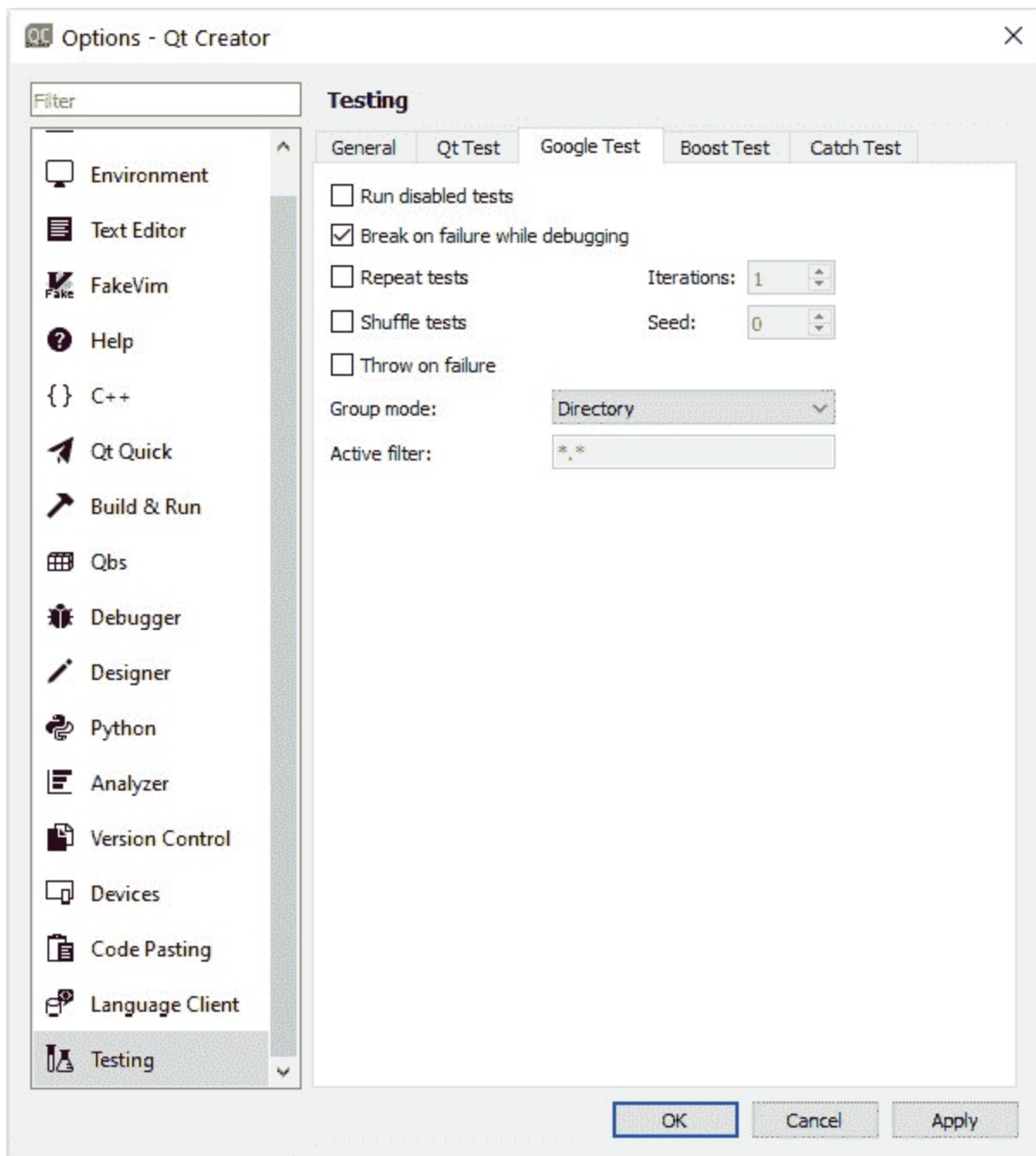


图 9.10-Options(选项)菜单下 Testing(测试)部分的 Google Test(Google 测试)选项卡

您可以从以下链接下载 GoogleTest 源代码:

<https://github.com/google/googletest>



您可以在以下文档中了解有关功能及其用法的更多信息：

<https://google.github.io/googletest/primer.html>

下载源代码后，在创建示例应用之前构建库。您还可以随测试项目一起构建统一的 **GoogleTest** 源代码。生成库后，请按照以下步骤运行 **GoogleTest** 应用：

1. 要使用 Qt Creator 的内置向导创建一个简单的 **GoogleTest** 应用，请从**New Project**菜单中选择**Auto Test Project**。然后按照屏幕进行操作，直到您看到项目和测试信息。
2. On the **Project and Test Information** screen, select **Google Test** for **Test framework**. Then add information for the **Test suite name** and **Test case name** fields as shown in *Figure 9.11*:

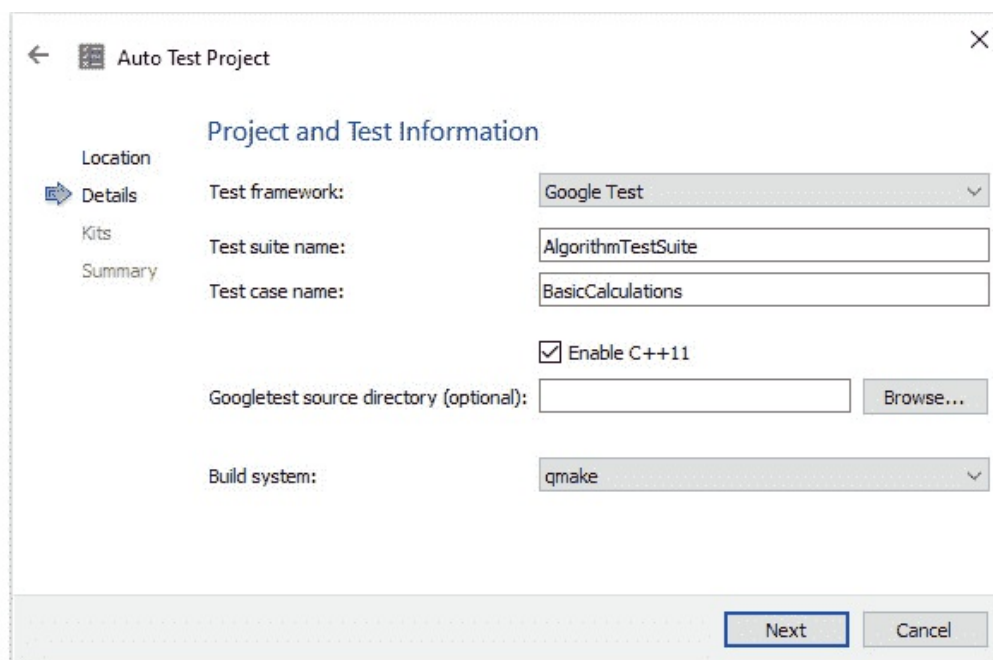


图 9.11-项目创建向导中的 Google 测试选项

3. In the next step, you can fill in the **Googletest source directory**

field or you can add it later by editing the `.pro` file.

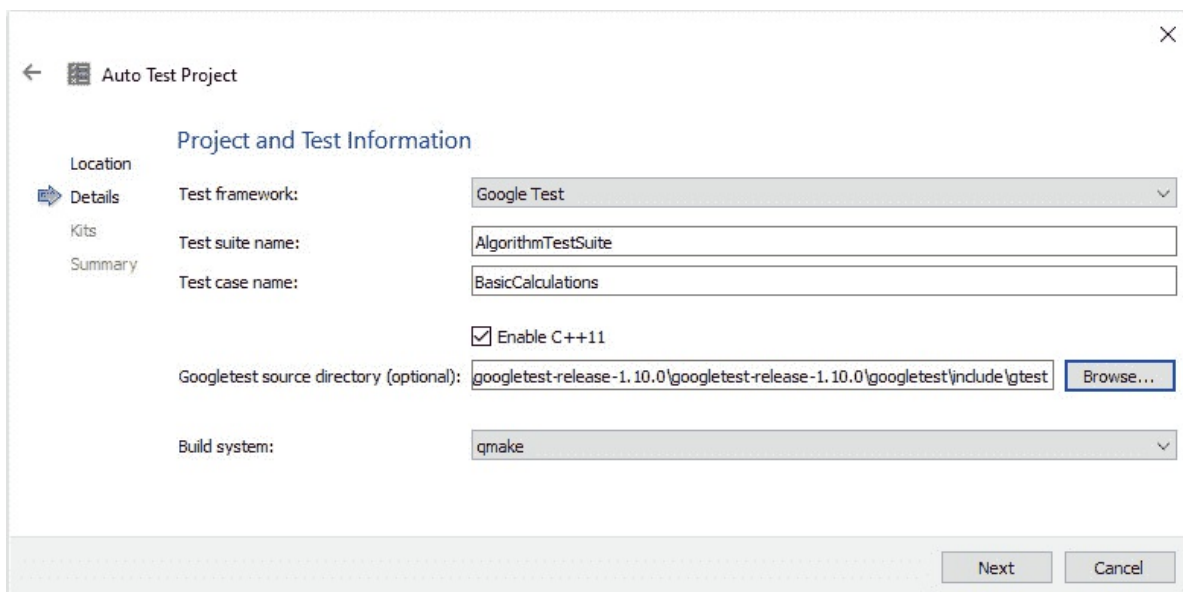


图 9.12-在项目创建向导中添加 GoogleTest 源目录的选项

4. 单击下一步并按照说明生成项目框架。
5. 要使用 GoogleTest，您必须将头文件添加到测试项目中：

```
#include "gtest/gtest.h"
```

6. 您可以看到 Main 函数已经由向导创建：

```
#include "tst_calculations.h"
#include "gtest/gtest.h"
int main(int argc, char *argv[])
{
 ::testing::InitGoogleTest(&argc, argv);
 return RUN_ALL_TESTS();
}
```

7. 您可以使用以下语法创建一个简单测试用例：

```
TEST(TestCaseName, TestName) { //test logic }
```

8. GoogleTest also provides macros such as `ASSERT_*` and `EXPECT_*` to check conditions and values:

```
ASSERT_TRUE(condition)
ASSERT_EQ(expected,actual)
ASSERT_FLOAT_EQ(expected,actual)
EXPECT_DOUBLE_EQ (expected, actual)
```

在大多数情况下，在运行多个测试之前执行一些自定义初始化工作是标准的过程。如果您想评估测试的时间/内存占用，则必须编写一些特定于测试的代码。测试夹具有助于设置特定的测试要求。

`fixture`类派生自`::testing::Test`类。请注意，使用的不是`TEST`，而是`TEST_F`宏。您可以在构造函数或`SetUp()`函数中分配资源和执行初始化。同样，您可以在析构函数或`TearDown()`函数中解除分配。文本装置内的测试函数定义如下：

```
TEST_F(TestFixtureName, TestName) { //test logic }
```

9. To create and use a test fixture, create a class derived from the `::testing::Test` class as follows:

```
class PushButtonTests: public ::testing::Test
{
protected:
 virtual void SetUp()
 {
 pushButton = new MyPushButton(0);
 pushButton ->setText("My button");
 }
};
TEST_F(PushButtonTests, sizeConstraints)
{
 EXPECT_EQ(40, pushButton->height());
 EXPECT_EQ(200, pushButton->width());
 pushButton->resize(300,300);
 EXPECT_EQ(40, pushButton->height());
 EXPECT_EQ(200, pushButton->width());
}
```

```
}
TEST_F(PushButtonTests, enterKeyPressed)
{
 QSignalSpy spy(pushButton, SIGNAL(clicked()));
 QTest::keyClick(pushButton, Qt::Key_Enter);
 EXPECT_EQ(spy.count(), 1);
}
```

在前面的代码中，我们在`SetUp()`函数中创建了一个自定义按钮。然后我们测试了两个测试函数来测试大小和进入键处理。

0. 运行前面的测试时，您将在“输出”窗口中看到测试结果。

**GoogleTest** 在运行时为用`TEST_F()`指定的每个测试构建一个新的测试装置。它通过调用`SetUp()`函数立即初始化并运行测试。然后调用`TearDown()`进行清理，并移除测试夹具。重要的是要注意，同一测试套件中的不同测试可以有不同的测试装置对象。在构建下一个测试夹具之前，**GoogleTest** 总是删除前一个夹具。它不会为多个测试重用测试夹具。一个测试对夹具所做的任何修改都不会影响其他测试。

我们讨论了如何用一个简单的测试用例创建一个 **GoogleTest** 项目，以及如何设计一个测试夹具或测试套件。现在您可以为现有的 **C++** 应用创建测试用例了。**GoogleTest** 是一个非常成熟的测试框架。它还集成了早先在 **GoogleMock** 下提供的模仿机制。探索不同的特性，并使用测试用例进行实验。

还有一个现成的 **GUI** 工具，它集成了两个测试框架来测试您的 **Qt** 应用。**GTest Runner**是一个基于 **Qt** 的自动化测试运行器和 **GUI**，具有适用于 **Windows** 和 **Linux** 平台的强大功能。但是，代码不会主动维护，也不会升级到 **Qt 6**。您可以通过以下链接了解有关 **GTest Runner** 功能和用法的更多信息：

<https://github.com/nholthaus/gtest-runner>

在本节中，您学习了如何一起使用 `QTest` 和 `GoogleTest`。您已经看到了这两个测试框架的功能。您可以使用 `GoogleTest` 框架的 `GoogleMock` 特性创建模拟对象。现在，您可以为自定义 C++ 类或自定义小部件编写自己的测试夹具。在下一节中，我们将讨论 `Qt Quick` 中的测试。

## 测试 Qt 快速应用

**Qt Quick Test** 是为 `Qt Quick` 应用的单元测试创建的框架。测试用例是用 `JavaScript` 编写的，并且使用 `TestCaseQML` 类型。名称以 `test_` 开头的函数被标识为需要执行的测试用例。测试工具递归地搜索 `tst_*.qml` 文件所需的源目录。您可以将所有测试 `.qml` 文件保存在一个目录下，并定义 `QUICK_TEST_SOURCE_DIR`。如果未定义，则在测试执行期间将只包括当前目录中可用的 `.qml` 个文件。**Qt** 不能确保 **Qt 快速测试** 模块的二进制兼容性。您必须使用适当版本的模块。

您必须将 `QUICK_TEST_MAIN()` 添加到 C++ 文件中才能开始执行测试用例，如下所示：

```
#include <QtQuickTest>
QUICK_TEST_MAIN(testqml)
```

您需要添加 `qmltest` 模块才能启用 **Qt 快速测试**。将以下代码行添加到 `.pro` 文件：

```
QT += qmltest
TEMPLATE = app
TARGET = tst_calculations
CONFIG += qmltestcase
SOURCES += testqml.cpp
```

让我们看一个基本算术计算的演示，看看该模块是如何工作的。我们将进行一些加法、减法和乘法等计算，并故意犯一些错误，从而导致测试用例失败：

```
import QtQuick
import QTest
TestCase {
 name: "Logic Tests"
 function test_addition() {
 compare(4 + 4, 8, "Logic: 4 + 4 = 8")
 }
 function test_subtraction() {
 compare(9 - 5, 4, "Logic: 9 - 5 = 4")
 }
 function test_multiplication() {
 compare(3 * 3, 6, "Logic: 3 * 3 = 6")
 }
}
```

运行前面的示例时，您将看到带有测试结果的输出，如下所示：

```
***** Start testing of testqml *****
Config: Using QTest library 6.1.0, Qt 6.1.0 (x86_64-little_endian)
PASS : testqml::Logic Tests::initTestCase()
PASS : testqml::Logic Tests::test_addition()
FAIL! : testqml::Logic Tests::test_multiplication()Logic: 3 * 3 =
 Actual (): 9
 Expected (): 6
C:\Qt6Book\Chapter09\QMLTestDemo\tst_calculations.qml(15) : failure
PASS : testqml::Logic Tests::test_subtraction()
PASS : testqml::Logic Tests::cleanupTestCase()
Totals: 4 passed, 1 failed, 0 skipped, 0 blacklisted, 3ms
***** Finished testing of testqml *****
```

请注意，`cleanupTestCase()`是在测试执行完成后立即调用的。此函数可用于在所有东西被销毁之前进行清理。

您还可以执行数据驱动测试，如下所示：

```

import QtQuick
import QtTest
TestCase {
 name: "DataDrivenTests"
 function test_table_data() {
 return [
 {tag: "10 + 20 = 30", a: 10, b: 20, result: 30},
 {tag: "30 + 60 = 90", a: 30, b: 60, result: 90},
 {tag: "50 + 50 = 100", a: 50, b: 50, result: 50}
]
 }
 function test_table(data) {
 compare(data.a + data.b, data.result)
 }
}

```

请注意，表数据可以使用以 `_data` 结尾的函数名提供给测试。运行前面的示例时，您将看到带有测试结果的输出，如下所示：

```

***** Start testing of main *****
Config: Using QtTest library 6.1.0, Qt 6.1.0 (x86_64-little_endian)
PASS : main::DataDrivenTests::initTestCase()
PASS : main::DataDrivenTests::test_table(10 + 20 = 30)
PASS : main::DataDrivenTests::test_table(30 + 60 = 90)
FAIL! : main::DataDrivenTests::test_table(50 + 50 = 100) Compared
 Actual (): 100
 Expected (): 50
C:\Qt6Book\Chapter09\QMLDataDrivenTestDemo\tst_datadriventests.qml
PASS : main::DataDrivenTests::cleanupTestCase()
Totals: 4 passed, 1 failed, 0 skipped, 0 blacklisted, 3ms
***** Finished testing of main *****

```

您还可以在 **QML** 中运行基准测试。**Qt** 基准测试框架将多次运行名称以 `benchmark_` 开头的函数，并记录运行的平均计时值。它类似于 **QTestLib** 的 C++ 版本中的 `QBENCHMARK` 宏。您可以在测试函数名前加

上 `benchmark_once` 前缀，以获得 `QBENCHMARK_ONCE` 宏的效果。让我们来看看下面的基准测试示例：

```
import QtQuick
import QtTest
TestCase {
 id: testObject
 name: "BenchmarkingMyItem"
 function benchmark_once_create_component() {
 var component = Qt.createComponent("MyItem.qml")
 var testObject = component.createObject(testObject)
 testObject.destroy()
 component.destroy()
 }
}
```

在前面的示例中，我们创建了一个自定义 **QML** 元素。我们想测量创建元素所需的时间。因此，我们编写了前面的基准代码。正常的基准测试运行多次，并显示操作的持续时间。在这里，我们已经对创作进行了一次基准测试。此技术在评估 **QML** 代码的性能时非常有用。

当您运行前面的示例时，您将看到带有测试结果的输出，如下所示：

```
***** Start testing of testqml *****
Config: Using QtTest library 6.1.0, Qt 6.1.0 (x86_64-little_endian)
PASS : testqml::BenchmarkingMyItem::initTestCase()
PASS : testqml::BenchmarkingMyItem::benchmark_once_create_component()
PASS : testqml::BenchmarkingMyItem::benchmark_once_create_component()
RESULT : testqml::benchmark_once_create_component:
 0 msecs per iteration (total: 0, iterations: 1)
PASS : testqml::BenchmarkingMyItem::cleanupTestCase()
QWARN : testqml::UnknownTestFunc() QQmlEngine::setContextForObject()
Totals: 4 passed, 0 failed, 0 skipped, 0 blacklisted, 5ms
***** Finished testing of testqml *****
```

要多次运行基准测试，您可以从测试用例中删除 `once` 关键字，如下所示：`function benchmark_create_component() {...}`。您还可以使



用 `Qt.createQmlObject()` 测试动态创建的对象。

还有一个名为 **qmlbench** 的基准测试工具，用于对 Qt 应用的整体性能进行基准测试。它是 **qt-labs** 下提供的功能丰富的基准测试工具。该工具还有助于测量用户界面的刷新率。您可以通过以下链接了解有关此工具的更多信息：

<https://github.com/qt-labs/qmlbench>

与 C++ 实现类似，您还可以在 QML 中模拟键盘事件，如 `keyPress()`、`keyRelease()` 和 `keyClick()`。事件被传递给当前关注的 QML 对象。让我们来看看下面的例子：

```
import QtQuick
import QtTest
MouseArea {
 width: 100; height: 100
 TestCase {
 name: "TestRightKeyPress"
 when: windowShown
 function test_key_click() {
 keyClick(Qt.Key_Right)
 }
 }
}
```

在前面的示例中，键盘事件是在 QML 查看窗口显示之后发送的。在此之前尝试传递事件将不会成功。为了跟踪窗口显示的时间，使用了 `when` 和 `windowShown` 属性。

运行前面的示例时，您将看到带有测试结果的输出，如下所示：

```
***** Start testing of testqml *****
Config: Using QtTest library 6.1.0, Qt 6.1.0 (x86_64-little_endian)
PASS : testqml::TestRightKeyPress::initTestCase()
QWARN : testqml::TestRightKeyPress::test_key_click() QQmlEngine::
```

```
PASS : testqml::TestRightKeyPress::test_key_click()
PASS : testqml::TestRightKeyPress::cleanupTestCase()
Totals: 3 passed, 0 failed, 0 skipped, 0 blacklisted, 25ms
***** Finished testing of testqml *****
```

您可以使用`SignalSpy`观看信号发射。在下面的示例中，我们使用`SignalSpy`来检测`Button`上的`clicked`信号。当信号发出时，`clickSpy`计数增加：

```
import QtQuick
import QtQuick.Controls
import QtTest
Button {
 id: pushButton
 SignalSpy {
 id: clickSpy
 target: pushButton
 signalName: "clicked"
 }
 TestCase {
 name: "PushButton"
 function test_click() {
 compare(clickSpy.count, 0)
 pushButton.clicked();
 compare(clickSpy.count, 1)
 }
 }
}
```

运行前面的示例时，您将看到带有测试结果的输出，如下所示：

```
***** Start testing of testqml *****
Config: Using QtTest library 6.1.0, Qt 6.1.0 (x86_64-little_endian)
PASS : testqml::PushButton::initTestCase()
PASS : testqml::PushButton::test_click()
PASS : testqml::PushButton::cleanupTestCase()
Totals: 3 passed, 0 failed, 0 skipped, 0 blacklisted, 5ms
***** Finished testing of testqml *****
```

`QUICK_TEST_MAIN_WITH_SETUP`宏用于在运行任何 QML 测试之前执行 C++ 代码。此对于设置 QML 引擎上的上下文属性非常有用。一个测试应用可以包括几个 `TestCase` 个实例。应用在运行所有测试用例后终止。您可以从 **Tests** 资源管理器启用或禁用测试用例：

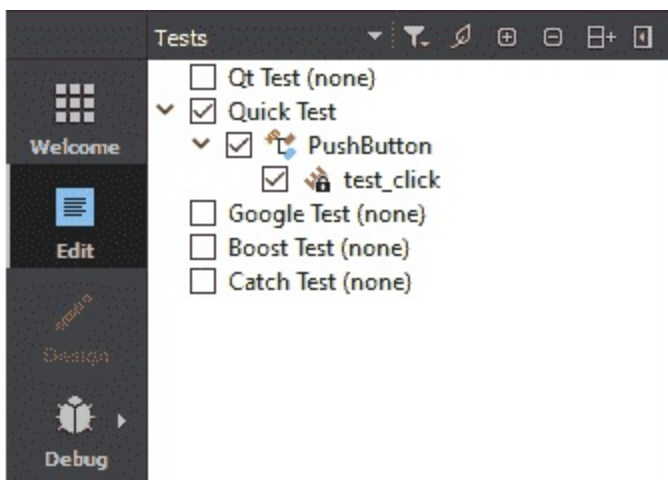


图 9.13-测试资源管理器显示带有可用测试用例的快速测试

在本节中，我们讨论了测试 QML 对象的不同测试方法。在下一节中，我们将熟悉 GUI 测试，并了解一些流行的工具。

## GUI 测试工具

您可以轻松地将一个或多个类作为单元测试进行评估，但我们必须手动编写所有测试用例。GUI 测试是一项特别具有挑战性的任务。我们如何在不用 C++ 或 QML 编码的情况下记录用户交互，比如鼠标点击？这个问题让开发人员感到困惑。市场上有很多 GUI 测试工具可以帮助我们做到这一点。有些很贵，有些是开源的。我们将在本节中讨论几个这样的工具。

但是，您可能不需要完整的 GUI 测试框架。有些问题可以通过简单的技巧解决。例如，在使用 GUI 时，您可能还必须检查不同的属性，例

如可视元素的对齐和边界。最简单的方法之一是添加一个 `Rectangle` 来检查边界，如下面的代码所示：

```
Rectangle {
 id: container
 anchors {
 left: parent.left
 leftMargin: 100
 right: parent.right
 top: parent.top
 bottom: parent.bottom
 }
 Rectangle {
 anchors.fill : parent
 color: "transparent"
 border.color: "blue" }
 Text {
 text: " Sample text"
 anchors.centerIn: parent
 Rectangle {
 anchors.fill : parent
 color: "transparent"
 border.color: "red"
 }
 }
}
```

当您运行前面的代码片段时，您将看到带有彩色元素边界的 **GUI**，如下面的屏幕截图所示：

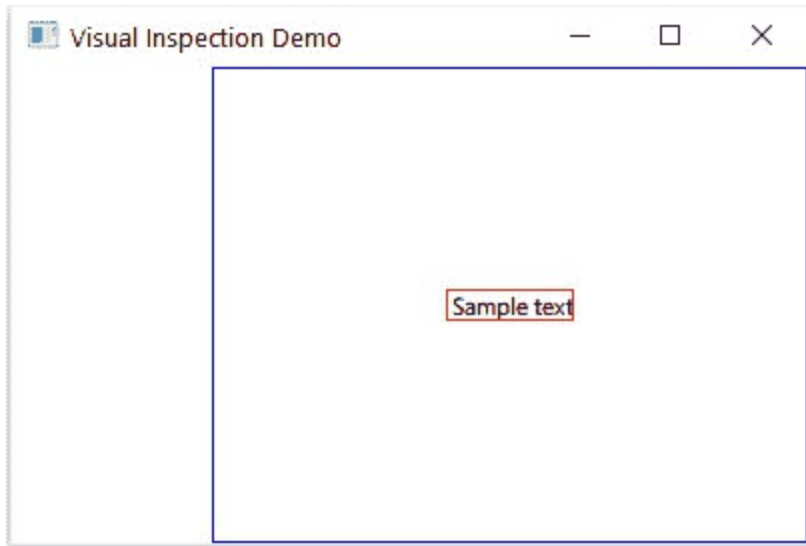


图 9.14-使用矩形输出 GUI 元素的可视边界

在前面的示例中，您可以看到文本元素位于带有蓝色边框的矩形的中心位置。如果没有蓝色边框，您可能会纳闷为什么不将其放在 GUI 的中心位置。您还可以查看每个元素的边界和边距。当文本元素宽度小于字体宽度时，您将观察到剪裁。您还可以查看用户界面元素之间是否存在任何重叠区域。这样，您就可以在不使用 `SG_VISUALIZE` 环境变量的情况下发现 GUI 的特定元素中的问题。

让我们讨论几个 GUI 测试工具。

## Linux 桌面测试项目(LDTP)

**Linux 桌面测试项目(LDTP)**为测试和改进 Linux 桌面平台提供了高质量的测试自动化基础设施和尖端工具。LDTP 是一个可以在所有平台上运行的 GUI 测试框架。它使用可访问性库查看应用的用户界面。该框架还包括用于记录测试用例的工具，具体取决于用户与 GUI 的交互方式。

要单击按钮，请使用以下语法：

```
click('<window name>', '<button name>')
```

若要获取给定对象的当前滑块值，请使用以下代码：

```
getslidervalue('<window name>', '<slider name>')
```

要将 LDTP 用于您的 GUI 应用，您必须为所有 QML 对象添加一个可访问的名称。您可以使用对象名称作为可访问的名称，如下所示：

```
Button {
 id: quitButton
 objectName: "quitButton"
 Accessible.name: objectName
}
```

在前面的代码中，我们向 QML 控件添加了一个可访问的名称，以便 LDTP 工具可以找到此按钮。LDTP 需要用户界面的窗口名称来定位子控件。假设窗口名称为**Example**，则要生成单击事件，请在 LDTP 脚本上使用以下命令：

```
>click('Example', 'quitButton')
```

前面的 LDTP 命令定位 `quitButton` 并生成按钮单击事件。

您可以通过以下链接了解有关其功能和用途的更多信息：

<https://ldtp.freedesktop.org/user-doc/>

## [#T0]伽玛雷

KDAB 开发了一个名为 **GammaRay** 的软件自省工具来检查 Qt 应用。您可以在运行时使用 `QObject` 自检机制观察和操作应用。这可以在本地机器和远程嵌入式目标上运行。它扩展了指令级调试器的功能，同时遵循与底层框架相同的标准。这对于使用场景图、模型/视图、状态机

等框架的复杂项目特别有用。有几种工具可用于检查对象及其属性。然而，由于它与 **Qt** 复杂框架的深度关联，它在其他工具中脱颖而出。

您可以从以下链接下载 **GammaRay**：

<https://github.com/KDAB/GammaRay/wiki/Getting-GammaRay>

您可以通过以下链接了解有关其功能和用途的更多信息：

<https://www.kdab.com/development-resources/qt-tools/gammaray/>

## **=0\_Squish**

**Squish**是一款跨平台 GUI 测试自动化工具，适用于桌面、移动、嵌入式和 Web 应用。您可以为使用 **Qt** 小部件或 **Qt Quick** 编写的跨平台应用自动执行 GUI 测试。世界上数以千计的组织使用 **Squish** 通过功能回归测试和系统测试来测试他们的 GUI。

您可以通过以下链接了解有关该工具的更多信息：

<https://www.froglogic.com/squish/>

在本节中，我们讨论了各种 GUI 测试工具。探索它们，并在您的项目中尝试它们。让我们在本章中总结一下我们所学的内容。

## 摘要

---

在本章中，我们学习了调试是什么，以及如何使用不同的调试技术来识别 **Qt** 应用中的技术问题。除此之外，我们还研究了 **Qt** 在各种操作系统上支持的各种调试器。最后，我们了解了如何使用单元测试来简化一些调试措施。我们讨论了单元测试，您学习了如何使用 **Qt** 测试框架。您了解了如何调试 **Qt Quick** 应用。我们还讨论了 **Qt** 支持的各种

其他测试框架和工具。现在您可以为您的自定义类编写单元测试了。如果有人意外修改了某些特定逻辑，单元测试将失败并自动发出警报。

在[第 10 章](#)，部署 **Qt** 应用中，您将了解如何在各种平台上部署 **Qt** 应用。它将帮助您为目标平台创建可安装的软件包。



## 十、部署 Qt 应用

---

原文: <https://gitee.com/apacheecn/apacheecn-c-cpp-zh-pt2/blob/master/docs/cross-plat-dev-qt6-mod-cpp/10.md>

在前面的章节中，您学习了如何使用 **Qt6** 开发和测试应用。您的应用已经准备好并在桌面上运行，但它不是独立的。您必须遵循某些步骤才能交付您的应用，以便最终客户可以使用它。此过程称为部署。通常，您的最终用户需要一个文件，他们可以双击该文件来启动您的软件。软件部署包括向可能没有任何技术知识的目标用户提供软件所需的同步步骤和活动。

在本章中，您将学习如何在不同的平台上部署 **Qt** 项目。从头到尾，您将了解可用的部署工具和创建部署包时需要考虑的要点。

在本章中，我们将介绍以下主题：

- 部署策略
- 静态构建与动态构建
- 在桌面平台上部署
- **Qt** 安装程序框架
- 其他安装工具
- 在 **Android** 上部署

在本章结束时，您将能够创建可部署的包并与其他人共享。

## 技术要求

---

本章的技术要求包括在最新的桌面平台(如 Windows 10 或 Ubuntu 20.04 或 MacOS 10.14)上安装 Qt 6.0.0 和 Qt Creator 4.14.0 的最低版

本。

本章使用的所有代码都可以从以下 **giHub** 链接下载：<https://github.com/PacktPublishing/Cross-Platform-Development-with-Qt-6-and-Modern-Cpp/tree/master/Chapter10/HelloWorld>。

## 重要音符

本章中使用的截图是在 **Windows** 平台上拍摄的。根据计算机中的底层平台，您将看到类似的屏幕。

# 了解部署需求

---

使软件在目标设备(无论是测试服务器、生产环境、用户桌面还是移动设备)上工作的过程称为，称为软件部署。通常，最终用户需要一个他们可以打开以访问您的应用的文件。用户将不想经历几个过程来获得各种外来文件。通常，用户会寻找双击或轻拍即可启动的软件。用户将不想经历一系列步骤来获得多个未知文件。在本章中，我们将讨论部署 **Qt** 应用时需要考虑的步骤和事项。我们将讨论在 **Windows**、**Mac**、**Linux** 和 **Android** 平台上部署该应用。

我们刚刚运行了到目前为止构建的应用的调试版本。您应该制作用于生成部署包的发布二进制文件。这两种方法之间的不同之处在于，调试版本包含有关您编写的代码的信息，这使得在遇到问题时进行调试要容易得多。但是，您不希望向用户发送多个文件，因为这对他们毫无用处。用户只想运行您的应用。这就是为什么您必须向他们提供应用的发布版本。因此，要发布这款应用，我们将在发布模式下创建它，这将为我们的提供一个发布二进制文件，我们可以将其交付给我们的用户。获得二进制文件后，您将需要根据您希望部署应用的平台创建单

独的包。如果你想在 Windows 上部署，你需要采取一种特定的方法，同样的方法也适用于 Linux、MacOS 或 Android。

标准的 Qt 部署包由单个可执行文件组成，但它需要存在其他文件才能运行。除可执行文件外，还需要以下文件：

- 动态库
- 第三方库
- 附加模块
- 可分发文件
- Qt 插件
- 翻译文件
- 帮助文件
- 许可证

当我们在 Qt Creator 中启动 Qt 项目时，默认设置为使用动态链接。因此，我们的应用将需要 Qt 动态链接库。我们还需要您最喜欢的编译器 (mingw/msvc/clang/GCC) 的 C++ 运行时和标准库实现。这些文件在 Windows 上通常为 .dll 文件，在 Linux 上为 .so 文件，在 MacOS 上为 .so 或 .dylib 文件。如果您的项目是一个大型复杂项目，则可能有多个库。您的应用包可能还需要第三方库，如 OpenGL、libstdc++、libwinpthread 和 OpenSSL。

如果您的应用是基于 Qt Quick 的，那么您还需要 QtQuick、QtQml、QtStateMachine、QtCharts 和 Qt3D 等标准模块。它们以动态库的形式提供，还有一些额外的文件提供 QML 模块元数据，或者作为纯 QML 文件提供。不幸的是，实现 Qt 的 C++ 和 QMLAPI 的动态库不足以让我们的可执行文件运行。Qt 还使用插件来启用扩展，以及相当标准的 GUI 功能，如图像文件加载和显示。类似地，一些插件封装了运行 Qt

的平台。

如果您使用的是 **Qt** 的翻译支持，那么您还需要部署翻译文件。我们将在第 11 章，国际化中更多地讨论翻译。如果您使用的是 **Qt** 帮助框架，甚至是简单的 **PDF** 手册，则可能还需要部署文档文件。您可能还需要为应用部署一些图标或脚本或许可协议。您还必须确保 **Qt** 库可以自行定位平台插件、文档和翻译，以及预期的可执行文件。

## 在静态库和动态库之间选择

您可以使用静态链接或动态链接构建 **Qt** 应用。当您构建应用时，链接器使用这两种方法中的任何一种将所有使用的库函数复制到可执行文件中。我们假设您已经知道这两种方法。在本节中，我们将讨论何时对 **Qt** 应用使用静态链接和何时使用动态链接。

静态库或静态链接库源于链接器将所有必需的库函数放入可执行文件。静态链接会生成更大的二进制文件，需要更多存储空间和主存空间。静态库在 **Linux** 中由 `.a` 文件扩展名表示，在 **Windows** 中由 `.lib` 文件扩展名表示。

动态库或动态链接共享库不需要传输代码。相反，库的名称只是包含在二进制文件中。启动应用时，二进制文件和库都会加载到内存中。动态库在运行时链接。它们在 **Linux** 中由 `.so` 文件扩展名表示，在 **Windows** 中由 `.dll` 文件扩展名表示。

静态构建由单个可执行文件组成。但在动态构建中，您必须注意动态库。静态构建更简单，因为它们可能已经在可执行文件中具有 **Qt** 插件和 **QML** 导入。静态构建还促进了链接时间优化(**LTO**)，这可以提高整体应用性能。由于它避免了下载 **Qt** 库并确保它们位于目标系统上库的默认搜索路径中的负担，因此静态链接通常是发布应用最安全、最简单

的方法。但是，除非 Qt 是从指定了 `-static` 配置选项的源构建的，否则静态链接不是很有用。此模式的 Qt 应用部署仅在具有商业许可证的情况下可用。如果您是开放源码开发人员，则应避免静态链接您的应用。由于我们在本书中使用的是开源的 Qt 版本，因此我们将不再详细介绍静态构建。相反，我们将坚持常规的动态构建和部署。

您可以通过以下链接了解有关使用上述方法部署 Qt 应用的更多信息：

<https://doc.qt.io/qt-6/deployment.html>.

在接下来的几节中，我们将重点关注领先的桌面和移动平台。我们不会讨论嵌入式平台，因为这超出了本书的范围。

## 在桌面平台上部署

---

您已经看到，在部署 Qt 应用时有很多需要考虑的问题。幸运的是，Qt 提供了一个工具，可以通过扫描生成的应用二进制文件、识别所有依赖项并将它们复制到部署目录来帮助我们完成此过程。我们将在不同的平台上部署我们的应用，以实现不同的目标，但概念将保持不变。一旦我们构建了我们的二进制文件，我们需要做的第一件事就是添加依赖项，这样用户就可以毫无困难地执行应用。

我们可以通过两种方式加载依赖项。我们可以手动完成，也可以使用 Qt 框架或第三方提供的某些工具。在 Windows 上，我们可以使用 `windeployqt` 加载依赖项。在 MacOS 上，我们可以使用 `macdeployqt` 加载我们的二进制文件的依赖项。还有另一个名为 `linuxdeployqt` 的工具，您可以使用它将依赖项添加到您的二进制文件中。`linuxdeployqt` 很好地满足了我们的需求，我们将在本章中讨论它。但是，该 Linux 部署实用工具不是官方工具，也不受 Qt 支持。生成二进制文件后，需

要查找并添加依赖项。您可以手动执行此操作，也可以根据您所处的位置使用这些工具之一来部署您的应用。

在本章中，我们将使用一个简单的*HelloWorld*示例来讨论如何在不同平台上部署应用。我们将找到依赖项并创建一个独立的包。让我们从 Windows 部署开始。

## 在 Windows 上部署

大多数为 Windows 构建的桌面应用通常以两种方式交付。首先，该应用作为独立应用提供，无需安装。在这种方法中，应用通常是一个可执行文件(.exe)，所有依赖库都在同一目录中。这种类型的应用被称为便携式应用。该应用不会将条目添加到已安装的应用列表中。因此，您在添加或删除程序列表中找不到任何选项。当您没有在 Windows 桌面上安装新应用的权限时，这很有用。第二种类型的应用通常以.exe或.msi格式出现。您将学习如何创建可安装的.exe文件。在本部分中，我们将讨论如何使用这两种方法创建独立的部署包。

按照以下步骤创建便携应用：

1. 首先，创建一个简单的 Qt 应用。您可以选择 Qt Widget 或基于 Qt Quick 的应用。这里我们将讨论基于 Qt Widget 的应用。这两种类型的应用是相同的。
2. 创建示例应用后，您可以选择通过在main.cpp文件中添加几行代码来添加应用名称、版本、组织名称和域，如下所示：

```
QApplication app (argc, argv);
app.setOrganizationName("Awesome Company");
app.setOrganizationDomain("www.abc.com");
app.setApplicationName("Deployment Demo");
app.setApplicationVersion("1.0.0");
```

3. Once you created the application, build it in **Release** mode. You can change the **Build** mode in the build settings. **Release** mode creates a smaller binary as it eliminates the debug symbols. You can quickly change the build mode from the kit selector section by clicking on it and then selecting the **Release** option, as shown in *Figure 10.1*:

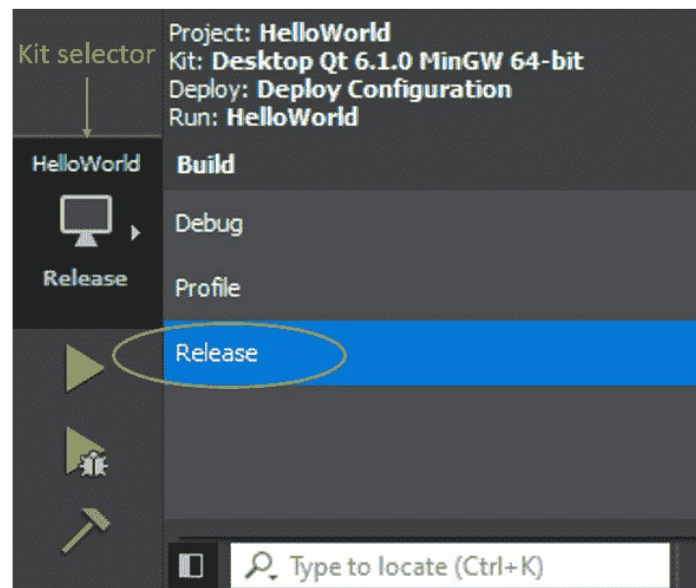


图 10.1-Qt Creator 中的发布选项

4. You can see that the binaries are created inside the **Release** directory. In this example, we are using *shadow build*. You can also change the release directory from the **General** section under the **Build Settings** screen:



<< build-HelloWorld-Desktop_Qt_6_1_0_MinGW_64_bit-Release > release				Search release	
Name	Date modified	Type	Size		
HelloWorld.exe	18-05-2021 22:59	Application	24 KB		
main.o	18-05-2021 22:59	O File	2 KB		
moc_predefs.h	18-05-2021 22:59	C++ Header file	16 KB		
moc_widget.cpp	18-05-2021 22:59	C++ Source file	3 KB		
moc_widget.o	18-05-2021 22:59	O File	9 KB		
widget.o	18-05-2021 22:59	O File	4 KB		

图 10.2-带有版本二进制文件的目录

5. 现在，创建一个部署目录，并从版本目录复制可执行文件。
6. Now, double-click on the executable file. You will notice that the application failed to launch and that several error dialogs appear. The error dialogs will mention which library is missing. If you don't see these errors, then you might have already added the library path in the system environment. You can try it on a clean system where Qt libraries are not installed:

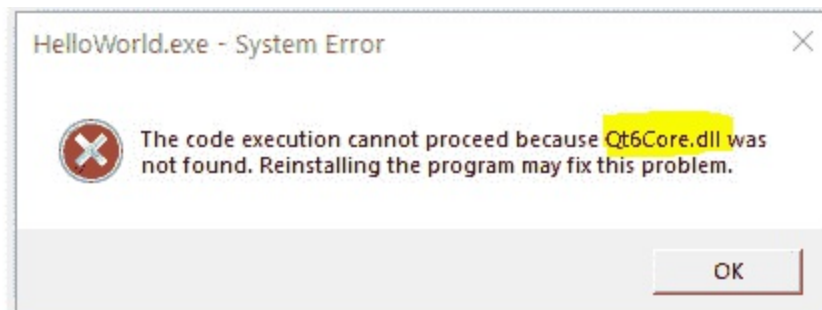


图 10.3-显示 Qt 库依赖项时出错

7. 下一步是查找在 IDE 外部独立运行应用所需的缺少的 Qt 库。
8. 由于我们在这里使用的是 Qt 的开放源码版本和动态链接方法，您将注意到缺少的库将具有 .dll 扩展名。在这里，我们看到缺少的库是 Qt6Core.dll。



9. The number of errors will depend on the number of modules used in the program. You can find the Qt dependent libraries from the `QTDIR/6.x.x/<CompilerName>/bin` directory. Here, `QTDIR` is where Qt 6 is installed. In our example, we have used Qt 6.1.0 as the version and `mingw81_64` as the compiler, hence the path is `D:/Qt/6.1.0/mingw81_64/bin`. This path can vary as per your Qt installation path, Qt version, and chosen compiler. The following screenshot shows the presence of the dynamic libraries under the `bin` directory:

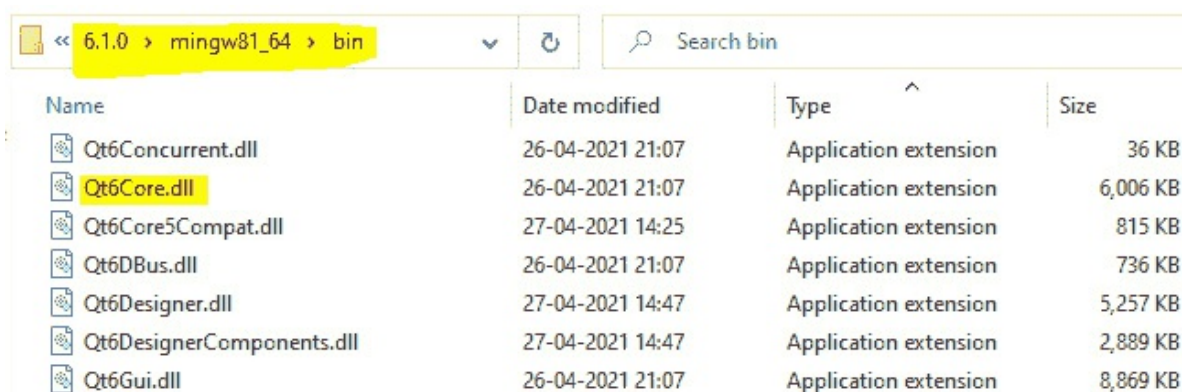


图 10.4-bin 目录内所需的 Qt 库

0. 如图 10.4所示，将丢失的 `.dll` 文件复制到最近创建的部署目录。
1. 重复该过程，直到将错误消息中提到的所有缺少的库复制到部署目录。您可能还必须将特定于编译器的库与应用一起部署。您还可以使用依赖关系遍历(**Depends.exe**)工具按查找依赖库。此工具是特定于 **Windows** 的免费工具。它提供了依赖库的列表。然而，在最近的版本中，该工具并不是非常有用，而且经常无法提供所需的信息。您可以尝试的工具很少，例如 **PeStudio**、**Mitec EXE Explorer** 和 **CFF Explorer**。请注意，我还没有研究过这些工具。

2. Once you have copied all the missing libraries, try to run the application again. This time, you will notice that a new error pops up. On this occasion, the message relates to the platform plugin:

![Figure 10.5 - Error dialog indicating a missing Qt platform plugin]

图 10.5-指示缺少 Qt 平台插件的错误对话框

3. Create a directory called `platforms` inside the deployment directory:

![Figure 10.6 - Directory showing the Qt windows platform plugin]

图 10.6-显示 Qt Windows 平台插件的目录

4. Then, copy the `qwindows.dll` file from `C:\Qt\6.x.x\<compiler_name>\plugins\platforms` to the new `platforms` subdirectory. *Figure 10.7* illustrates the organization of the files in the deployment directory:

![Figure 10.7 - Copy platforms plugin inside the release directory]

图 10.7-在发布目录中复制平台插件

5. Now, double-click on the `HelloWorld.exe` file. You will observe that the **HelloWorld!** GUI appears in no time. Now, the Qt Widgets application can be launched on a Windows platform that doesn't have Qt 6 installed:

![Figure 10.8 - Standalone application running with resolved dependencies]

图 10.8-独立应用在解析依赖项的情况下运行

6. 下一步也是最后一步是将文件夹压缩并与您的朋友共享。

祝贺你!。您已经成功部署了您的第一个独立应用。但是,这种方法不适用于我们有许多依赖文件的大型项目。Qt 提供了几个方便的工具来应对此类挑战并轻松创建安装包。在下一节中,我们将讨论 Windows 部署工具以及它如何帮助我们应对这些挑战。

## Windows 部署工具

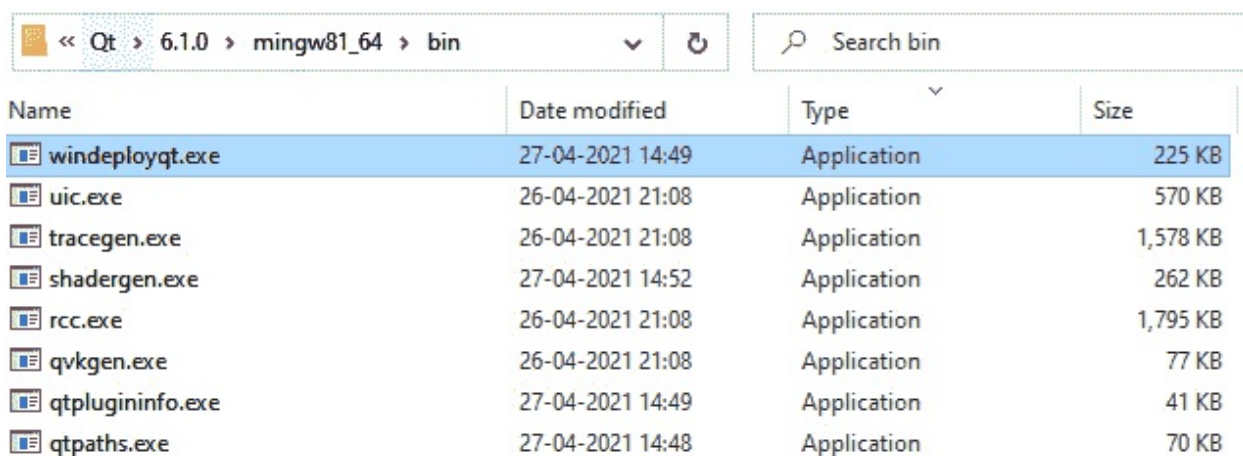
Windows 部署工具随 Qt 6.x 安装包一起提供。您可以在 `<QTDIR>/bin/` 下找到,名称为 `windeployqt.exe`。您可以从 Qt 命令提示符运行此工具,并将可执行文件作为参数传递,或者将目录作为参数传递。如果您正在构建 Qt Quick 应用,则必须额外添加 `.qml` 文件的目录路径。

让我们来看看 `windeployqt` 中提供的一些重要命令行选项。请浏览以下列表中的一些有用选项:

- `-?`或`-h`或`--help`显示有关命令行选项的帮助。
- `--help-all`显示包括 Qt 特定选项的帮助。
- `--libdir <path>`将依赖库复制到路径。
- `--plugindir <path>`将依赖插件复制到路径。
- `--no-patchqt`指示不要修补 Qt6Core 库。
- `--no-plugins`指示跳过插件部署。
- `--no-libraries`指示跳过库部署。
- `--qmldir <directory>`扫描源目录中的 QML 导入。
- `--qmlimport <directory>`将给定路径添加到 QML 模块搜索位置。
- `--no-quick-import`指示跳过 Qt 快速导入的部署。
- `--no-system-d3d-compiler`指示跳过 D3D 编译器的部署。

- `--compiler-runtime` 在桌面上部署编译器运行时。
- `--no-compiler-runtime` 阻止在桌面上部署编译器运行时。
- `--no-opengl-sw` 阻止部署软件光栅化库。

您可以在 `bin` 文件夹中找到 `windeployqt` 工具，如以下截图所示：



Name	Date modified	Type	Size
 windeployqt.exe	27-04-2021 14:49	Application	225 KB
 uic.exe	26-04-2021 21:08	Application	570 KB
 tracegen.exe	26-04-2021 21:08	Application	1,578 KB
 shadergen.exe	27-04-2021 14:52	Application	262 KB
 rcc.exe	26-04-2021 21:08	Application	1,795 KB
 qvkgen.exe	26-04-2021 21:08	Application	77 KB
 qtplugininfo.exe	27-04-2021 14:49	Application	41 KB
 qtpaths.exe	27-04-2021 14:48	Application	70 KB

图 10.9-`bin` 目录内的 `winployment qt` 工具

使用 `windeployqt` 最简单的方法是将其路径添加到 `path` 变量。要将其添加到路径，请在 **Windows** 计算机上打开系统属性，然后单击高级系统设置。您将发现出现系统属性窗口。在系统属性窗口的底部，您将看到环境变量... 按钮。单击它并选择 `path` 变量，如下面的屏幕截图所示。然后，单击编辑... 按钮。添加 **Qt bin** 目录的路径，点击确定按钮：

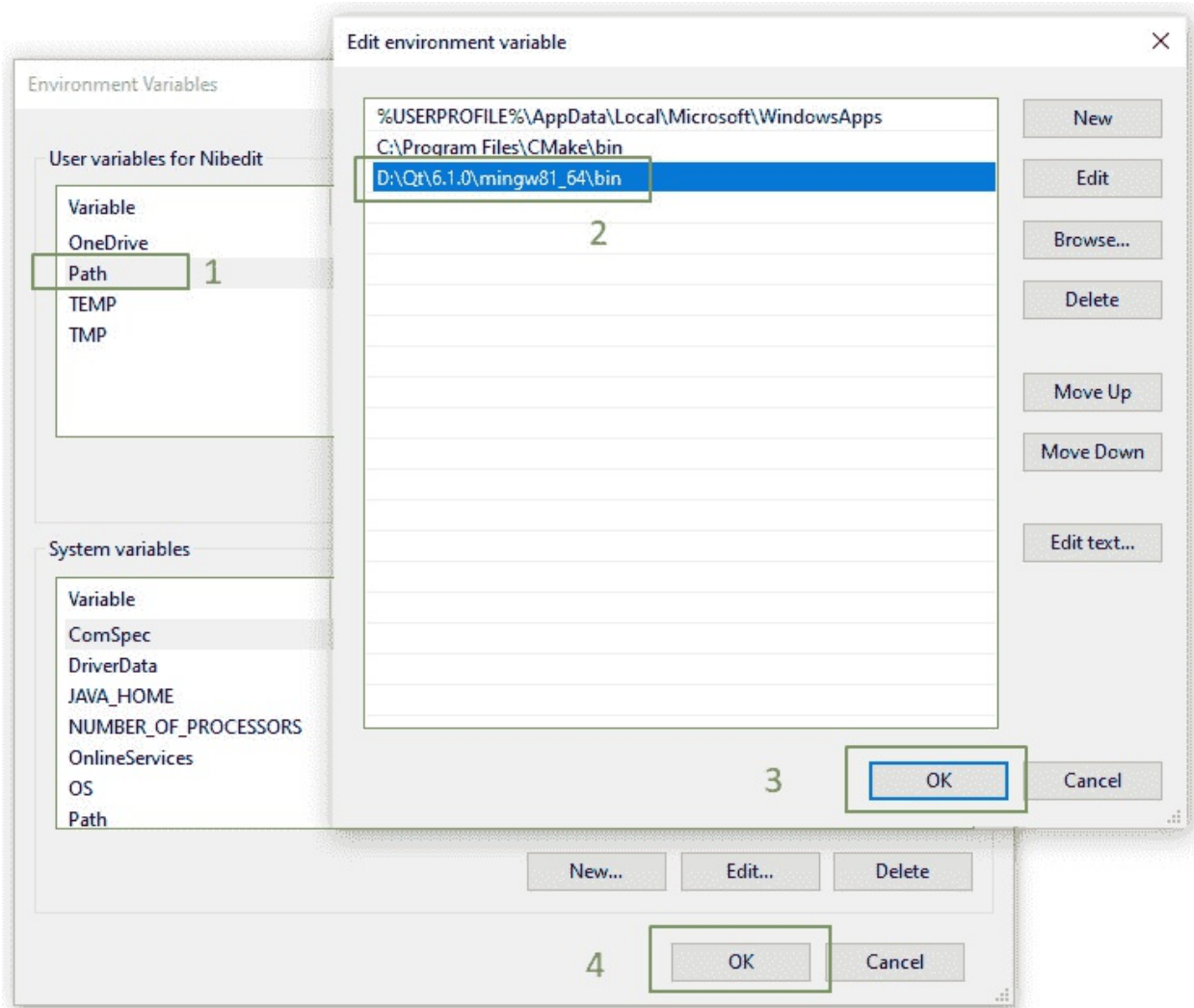


图 10.10-将 bin 目录添加到系统环境路径

关闭系统属性屏幕并启动 **Qt** 命令提示符。然后，您可以使用以下语法为基于 **Qt** 小部件的应用创建部署包：

```
>windeployqt <your-executable-path>
```

如果您使用的是 **Qt Quick**，请遵循以下语法：

```
>windeployqt --qmlidir <qmlfiles-path> <your-executable-path>
```

然后，该工具会将确定的依赖项复制到部署目录，确保我们在一位置

拥有所有必需的组件。 它还将为插件和您期望的其他 Qt 资源构建子目录结构。 如果 ICU 和其他文件不在 `bin` 目录中，则在运行该工具之前，必须将它们添加到 **PATH** 变量。

让我们从相同的 *HelloWorld* 示例开始。 要使用 `windeployqt` 创建示例的展开，请执行以下步骤：

1. 创建部署目录，并将 `HelloWorld.exe` 文件复制到部署目录。
2. 现在您可以调用部署工具了，如下所示：

```
D:\Chapter10\HelloWorld\deployment>windeployqt HelloWorld.exe
```

3. 输入该命令后，该工具将开始收集有关依赖项的信息：

```
>D:\Chapter10\HelloWorld\deployment\HelloWorld.exe 64 bit, rel
Adding Qt6Svg for qsvgicon.dll
Direct dependencies: Qt6Core Qt6Widgets
All dependencies : Qt6Core Qt6Gui Qt6Widgets
To be deployed : Qt6Core Qt6Gui Qt6Svg Qt6Widgets
```

4. 您将看到，该工具不仅列出了依赖项，还将所需的文件复制到目标目录。
5. Open the deployment directory and you will find that multiple files and directories have been added:



Name	Date modified	Type	Size
iconengines	19-05-2021 00:25	File folder	
imageformats	19-05-2021 00:25	File folder	
platforms	19-05-2021 00:25	File folder	
styles	19-05-2021 00:25	File folder	
translations	19-05-2021 00:25	File folder	
D3Dcompiler_47.dll	11-03-2014 16:24	Application extension	4,077 KB
HelloWorld.exe	18-05-2021 22:39	Application	24 KB
libgcc_s_seh-1.dll	12-05-2018 11:41	Application extension	75 KB
libstdc++-6.dll	12-05-2018 11:41	Application extension	1,384 KB
libwinpthread-1.dll	12-05-2018 11:41	Application extension	51 KB
opengl32sw.dll	04-06-2020 13:20	Application extension	20,150 KB
Qt6Core.dll	26-04-2021 21:07	Application extension	6,006 KB
Qt6Gui.dll	26-04-2021 21:07	Application extension	8,869 KB
Qt6Svg.dll	27-04-2021 10:59	Application extension	350 KB
Qt6Widgets.dll	26-04-2021 21:07	Application extension	6,219 KB

图 10.11-winployqt 将所有需要的文件复制到部署目录

6. 在前一节中，我们必须自己识别和复制所有依赖项，但该任务现在委托给windeployqt工具。
7. 如果您使用的是Qt Quick 应用，请运行以下命令：

```
>D:\Chapter10\qmldeployment>windeployqt.exe --qmldir D:\Chapter
```

8. 您可以看到，该工具已经收集了依赖项，并将所需文件复制到部署目录：

```
D:\Chapter10\qmldeployment\HelloWorld.exe 64 bit, release exec
Scanning D:\Chapter10\HelloWorld:
QML imports:
 'QtQuick' D:\Qt\6.1.0\mingw81_64\qml\QtQuick
 'QtQuick.Window' D:\Qt\6.1.0\mingw81_64\qml\QtQuick\Window
 'QtQml' D:\Qt\6.1.0\mingw81_64\qml\QtQml
 'QtQml.Models' D:\Qt\6.1.0\mingw81_64\qml\QtQml\Models
 'QtQml.WorkerScript' D:\Qt\6.1.0\mingw81_64\qml\QtQml\WorkerS
Adding Qt6Svg for qsvgicon.dll
Direct dependencies: Qt6Core Qt6Gui Qt6Qml
All dependencies : Qt6Core Qt6Gui Qt6Network Qt6OpenGL Qt6Qm
To be deployed : Qt6Core Qt6Gui Qt6Network Qt6OpenGL Qt6Qm
```



9. 现在，您可以双击启动独立应用。

0. 下一步是将文件夹压缩并与您的朋友共享。

**Windows** 部署工具的命令行选项可用于微调标识和复制过程。基本说明可在以下链接中找到：

<https://doc.qt.io/qt-6/windows-deployment.html>.

[https://wiki.qt.io/Deploy\\_an\\_Application\\_on\\_Windows](https://wiki.qt.io/Deploy_an_Application_on_Windows).

干杯！您已经学习了如何使用 **Windows** 部署工具部署 **Qt** 应用。然而，还有很多工作要做。**Qt Installer Framework** 提供了几个方便的工具来应对这类挑战，并轻松创建可安装的软件包。在下一节中，我们将讨论 **Linux** 部署工具以及如何使用它创建独立的应用。

## 在 **Linux** 上部署

在 **Linux** 发行版中，我们有多项选项来部署我们的应用。您可以使用安装程序，但也可以选择应用捆绑包等选项。有一种名为应用映像的技术可以使部署过程非常简单。您还可以打包您的应用以供包管理器使用。您可以选择 **Debian**、**Ubuntu** 或 **Fedora** 上的 `apt` 等选项，您的应用可以通过此方法使用。但是，您可以选择一种简单得多的方法，比如应用映像选项，它将为您提供一个文件。您可以将该文件提供给您的用户，他们只需双击即可运行该应用。

**Qt** 文档提供了在 **Linux** 上部署的某些说明。您可以查看以下链接：

<https://doc.qt.io/qt-6/linux-deployment.html>.

**Qt** 没有为 **Linux** 发行版提供任何类似于 `windeployqt` 的现成工具。这可



能是由于大量的 Linux 版本造成的。但是，有一个非官方的开源 Linux 部署工具，名为 `linuxdeployqt`。这会将应用作为输入，并通过将项目资源复制到捆绑包中将其转换为自包含的包。用户可以将生成的捆绑包作为 `AppDir` 或 `AppImage` 获取，也可以将其包含在交叉分发包中。对于 `cMake`、`qmake` 和 `make` 这样的系统，它可以作为构建过程的一部分来部署用 C、C++ 和其他编译语言编写的应用。它可以打包运行基于 Qt 的应用所需的特定库和组件。

您可以通过以下链接下载 `linuxdeployqt`：

<https://github.com/probonopd/linuxdeployqt/releases>.

您将在下载之后获得 `linuxdeployqt-x86_64.AppImage`，并在运行之前执行 `chmod a+x`。

您可以在 <https://github.com/probonopd/linuxdeployqt> 上阅读完整的文档并找到源代码。

如果您想轻松地使用单个应用包，那么可以使用 `-appimage` 标志运行 `linuxdeployqt`。

还有更多的部署工具，如 **Snap** 和 **Flatpak** 来打包应用及其依赖项，无需任何修改即可跨多个 Linux 发行版运行。

您可以通过以下链接了解如何创建快照：  
照：<https://snapcraft.io/docs/creating-a-snap>

您可以通过访问以下链接了解有关 Flatpak 的更多信息：  
息：<https://docs.flatpak.org/en/latest/qt.html>

在下一节中，我们将讨论 MacOS 部署工具，以及如何使用它为 Mac

用户创建独立的应用。

## 在 **MacOS** 上部署

您可以通过与前几节讨论的类似的过程来为 **MacOS** 生成安装程序文件。我们将讨论生成应用捆绑包可以遵循的步骤。您可以在 **MacOS** 上测试该软件包，然后将其发送给 **Mac** 用户。该过程与 **Linux** 上的过程基本相同。毕竟，**MacOS** 是基于 **Unix** 的。因此，您可以创建安装程序，我们将其称为 **MacOS** 上的捆绑包。

您可以在 `QTDIR/bin/macdeployqt` 中找到 **MacOS** 部署工具。它旨在自动化创建可部署应用包的过程，该应用包将 **Qt** 库作为私有框架包含在内。除非您指定 `-no-plugins` 选项，否则 **Mac** 部署工具还会部署 **Qt** 插件。默认情况下，总是部署 **Qt** 插件，例如平台、图像格式、打印支持和辅助功能。只有当应用使用 **SQL** 驱动程序和 **SVG** 插件时，才会部署它们。未部署设计器插件。如果要在应用捆绑包中包含第三方库，则必须在构建后手动将库复制到捆绑包中。

几年前，**Apple** 发布了一个名为 **Apple File System (APFS)** 的新文件系统。旧版本的 **MacOS** 无法读取 **APFS** 格式的 `.dmg` 文件。为了与 **Qt** 当前支持的所有 **MacOS** 版本兼容，默认情况下，`macdeployqt` 使用较旧的 **HFS+** 文件系统。要选择不同的文件系统，请使用 `-fs` 选项。

您可以在下面的链接中找到详细说明：<https://doc.qt.io/qt-6/macos-deployment.html>。

在下一节中，我们将讨论 **Qt** 安装程序框架，以及如何使用它为您的用户创建完整的安装包。

## 使用 **Qt** 安装程序框架

---

**Qt 安装程序框架(QIFW)**是一组跨平台工具和实用程序，用于为支持的桌面 Qt 平台(包括 Linux、Windows 和 MacOS)创建安装程序。它允许您在所有支持的桌面 Qt 平台上分发您的应用，而不必重写源代码。Qt **Installer Framework** 工具创建的安装程序包括一组帮助用户完成安装、更新和删除过程的页面。您提供可安装的内容以及有关它的信息，例如产品名称、安装程序和法律协议。

您可以通过向预定义页面添加小部件或添加整个页面来个性化安装程序，为消费者提供更多选择。您可以通过编写脚本向安装程序添加操作。根据您的使用情形，您可以为最终客户提供离线和/或在线安装。它在 Windows、Linux 和 Mac 上运行良好。我们将使用它为我们的应用创建安装程序，我们将详细了解它在 Windows 上是如何工作的。Linux 和 MacOS 遵循的流程与 Windows 类似。因此，我们将只讨论 Windows 平台。您可以在您最喜欢的平台上尝试类似的步骤。

您可以通过以下链接了解有关预定义页面的更多信息：<https://doc.qt.io/qtinstallerframework/ifw-use-cases-install.html>。

在开始旅程之前，请确认您的计算机上已安装 Qt Installer Framework。如果没有，请启动**Qt Maintenance Tool**并从**Select Components**页面安装，如以下截图所示：

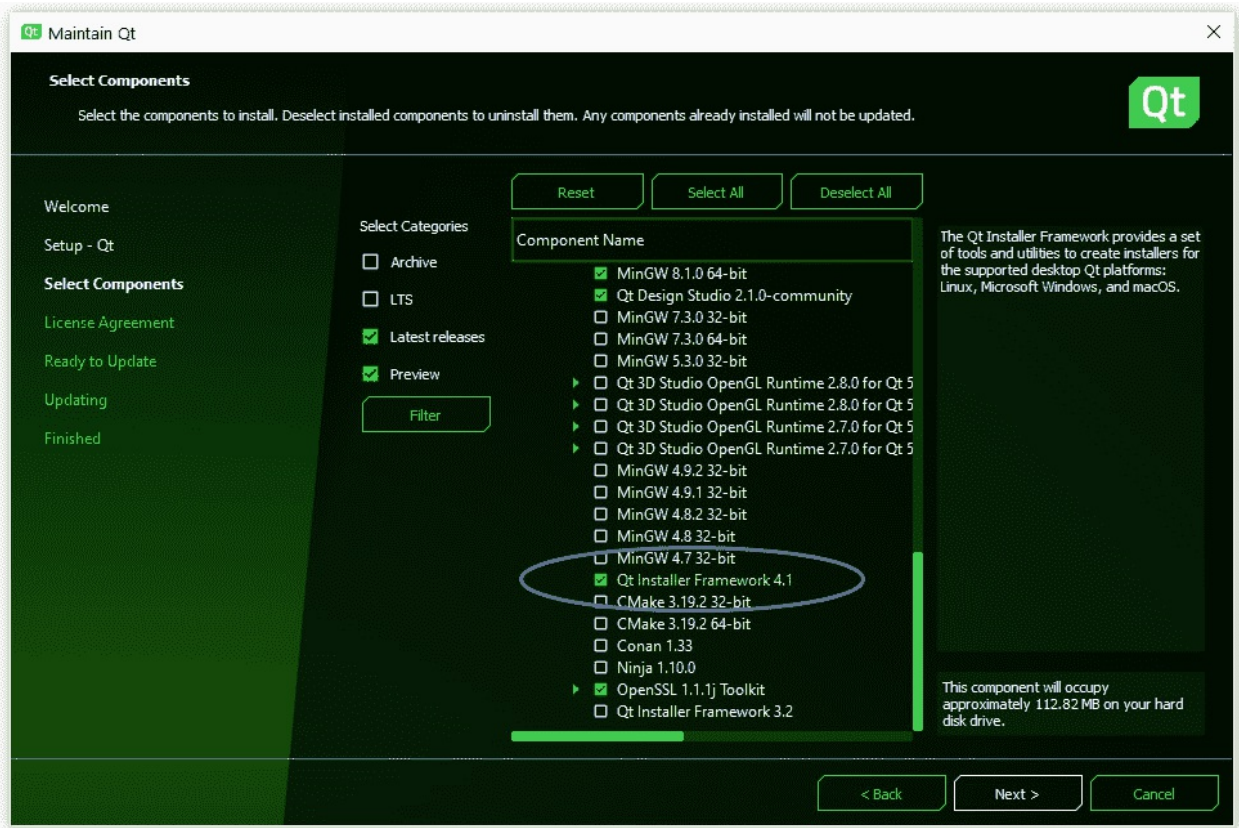


图 10.12-Qt 维护工具中的 Qt 安装程序框架下载选项

成功安装应用后，您将在`QTDIR\Tools\QtInstallerFramework\`下找到安装文件：

Name	Date modified	Type	Size
 archivegen.exe	10-03-2020 15:28	Application	19,694 KB
 binarycreator.exe	10-03-2020 15:28	Application	20,303 KB
 devtool.exe	10-03-2020 15:29	Application	20,151 KB
 installerbase.exe	10-03-2020 15:28	Application	21,034 KB
 repogen.exe	10-03-2020 15:28	Application	20,078 KB

图 10.13-Windows 上 Qt Installer Framework 目录内的工具

您可以看到，在 Qt Installer Framework 目录中创建了五个可执行文件：

- `archivegen` 工具用于将文件和目录打包到 7zip 存档中。
- `binarycreator` 工具用于创建在线和离线安装程序。
- `devtool` 用于使用新的安装程序库更新现有安装程序。
- `installerbase` 工具是打包所有数据和元信息的核心安装程序。
- `repogen` 工具用于生成在线存储库。

在本节中，我们将使用 `binarycreator` 工具为我们的 Qt 应用创建安装程序。此工具可用于生成脱机和在线安装程序。某些选项具有默认值，因此您可以省略它们。

要在 Windows 计算机上创建脱机安装程序，可以在 Qt 命令提示符下输入以下命令：

```
><location-of-ifw>\binarycreator.exe -t <location-of-ifw>\install
```

同样，要在 Linux 或 Mac 计算机上创建脱机安装程序，可以在 Qt 命令提示符下输入以下命令：

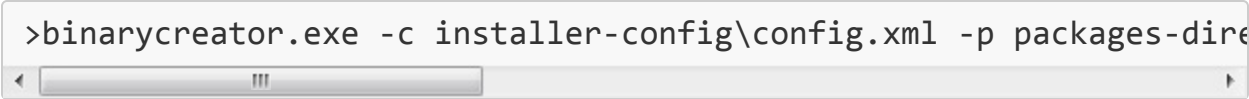
```
><location-of-ifw>/binarycreator -t <location-of-ifw>/installerbas
```

例如，要创建脱机安装程序，请执行以下命令：

```
>binarycreator.exe --offline-only -c installer-config\config.xml -
```

上述说明将为 SDK 创建一个包含所有依赖项的脱机安装程序。

要创建仅在线安装程序，您可以使用 `--online-only`，它定义了要从 Web 服务器上的在线存储库安装的所有软件包。例如，要创建在线安装程序，请执行以下命令：



```
>binarycreator.exe -c installer-config\config.xml -p packages-dir
```

您可以在以下页面了解有关**binarycreator**和不同选项的更多信息：<https://doc.qt.io/qtinstallerframework/ifw-tools.html#binarycreator>。

使用**binarycreator**的最简单方法是将其路径添加到**PATH**变量。要将其添加到路径，请在 **Windows** 计算机上打开系统属性，然后单击高级系统设置。您将发现出现系统属性窗口。在系统属性窗口的底部，您将看到环境变量... 按钮。单击它并选择**path**变量，如下面的屏幕截图所示。然后，单击编辑... 按钮。添加**QIFWbin** 目录的路径，然后单击确定按钮。以下屏幕截图说明了如何执行此操作：



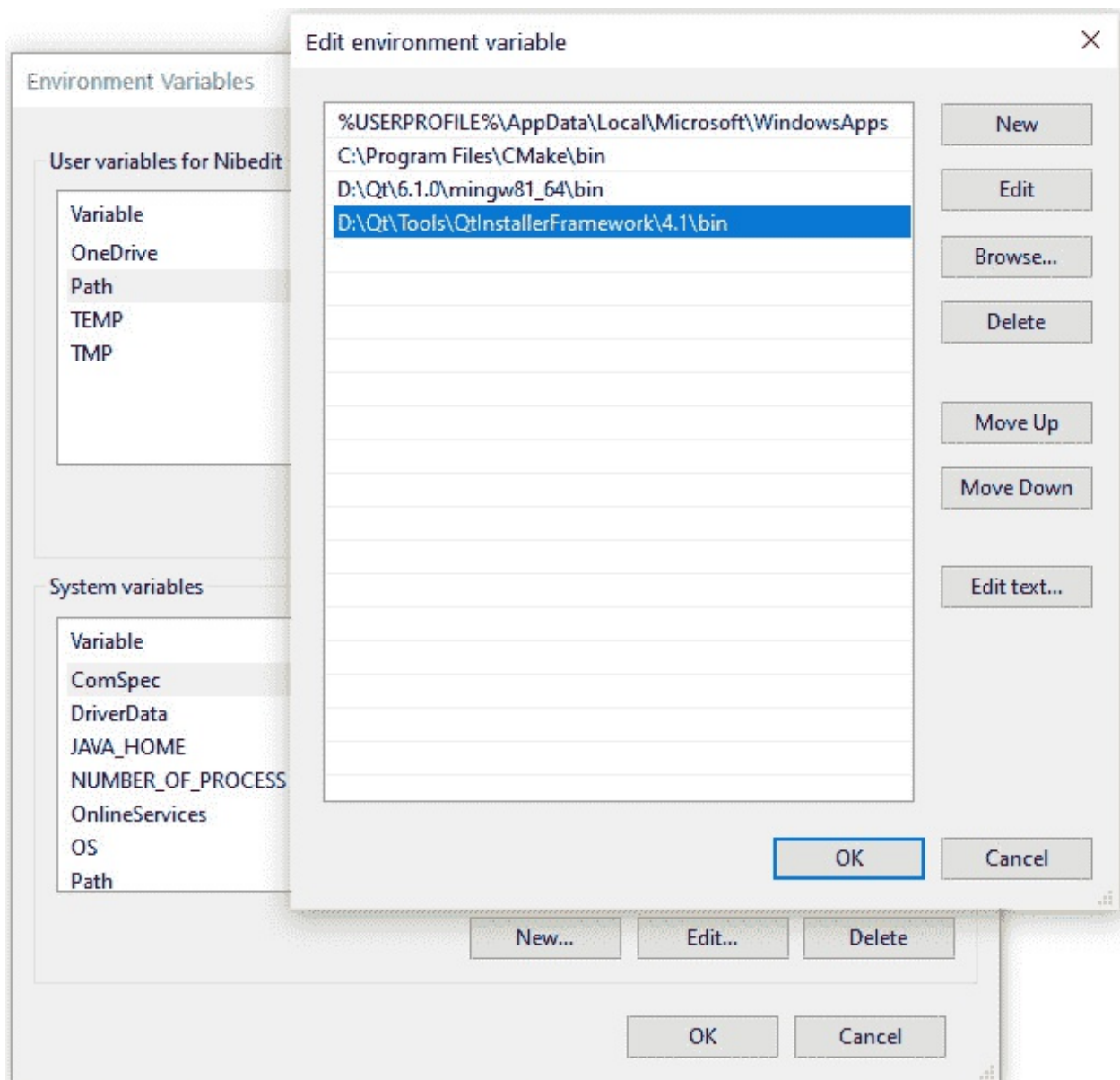


图 10.14-将 QIFW bin 目录添加到系统环境路径

关闭系统属性屏幕并启动 Qt 命令提示符。

让我们继续部署示例 *HelloWorld* 应用。我们将为我们的用户创建一个可安装的软件包，这样他们就可以双击并安装它：

1. 创建一个与安装程序的设计相匹配的目录结构，并允许将来对其进行扩展。目录中必须存在 `config` 和 `packages` 子目录。QIFW 部署的

目录放在哪里并不重要；重要的是它具有这样的结构。

2. Make a configuration file with instructions for building the installer binaries and online repositories. Create a file called `config.xml` in the config directory with the following content:

```
<?xml version="1.0" encoding="UTF-8"?>
<Installer>
 <Name>Deployment Example </Name>
 <Version>1.0.0</Version>
 <Title>Deployment Example</Title>
 <Publisher>Packt</Publisher>
 <StartMenuDir>Qt6 HelloWorld</StartMenuDir>
 <TargetDir>@HomeDir@/HelloWorld</TargetDir>
</Installer>
```

`Title` 标记提供出现在标题栏中的安装程序的名称。使用 `Name` 标记将应用名称添加到页面名称和介绍性文本中。软件版本号由 `Version` 标记指定。 `Publisher` 标记定义软件的发行商。 **Windows** 开始菜单中产品的默认程序组名称由 `StartMenuDir` 标记指定。显示给用户的默认目标目录是当前用户的主目录中的 `InstallationDirectory`，由 `TargetDir` 标记指定。您可以在文档中阅读更多关于标记的内容。

您还可以在 `config.xml` 中指定应用捆绑包图标。在 **Windows** 上，它使用 `.ico` 进行扩展，并可用作 `.exe` 文件的应用图标。在 **Linux** 上，您可以指定带有 `.png` 扩展名的图标，这可以用作窗口图标。在 **MacOS** 上，您可以用 `.icns` 指定图标，这可以用作新生成的包的图标。

3. 现在在 `packages` 目录内创建一个子目录。这将是您的 `component` 名称。您可以使用组织的名称和应用名称或组织域作为 `component`，例如 `CompanyName.ApplicationName`。目录名起到类似域标识符的作



用，标识所有组件。

4. 创建一个软件包信息文件，其中包含有关可能安装的组件的详细信息。在这个简单的示例中，安装程序只需处理一个组件。让我们在 `packages\{component}\meta` 目录中创建一个名为 `package.xml` 的包信息文件。
5. Add the file in side the meta-directory with the following content to provide information about the component to the installer.

让我们创建 `package.xml`，并向其中添加以下内容：

```
<?xml version="1.0"?>
<Package>
 <DisplayName>Hello World</DisplayName>
 <Description>This is a simple deployment example.
</Description>
 <Version>1.0.1</Version>
 <ReleaseDate>2021-05-19</ReleaseDate>
</Package>
```

在安装过程中，组件选择页面上会显示来自个元素的信息：

- `DisplayName` 标记指定组件列表中的组件名称。
  - `Description` 标记指定选择组件时显示的文本。
  - `Version` 标记使您能够在更新可用时向用户提供更新。
  - `Default` 标记指定默认情况下是否选择元件。值 `true` 将元件设置为选定。
  - 您可以向安装程序添加许可信息。包含许可协议文本的文件名由 `License` 标记指定，该文件显示在许可检查页面上。
6. You can copy the required content inside the `data` subdirectory under the `package` directory. Copy all the files and directories to the `data` subdirectory, which were earlier created with `windeployqt`.

The following screenshot shows the content copied inside the `data` subdirectory:

Name	Date modified	Type	Size
iconengines	19-05-2021 00:25	File folder	
imageformats	19-05-2021 00:25	File folder	
platforms	19-05-2021 00:25	File folder	
styles	19-05-2021 00:25	File folder	
translations	19-05-2021 00:25	File folder	
D3Dcompiler_47.dll	11-03-2014 16:24	Application extension	4,077 KB
HelloWorld.exe	18-05-2021 22:39	Application	24 KB
libgcc_s_seh-1.dll	12-05-2018 11:41	Application extension	75 KB
libstdc++-6.dll	12-05-2018 11:41	Application extension	1,384 KB
libwinpthread-1.dll	12-05-2018 11:41	Application extension	51 KB
opengl32sw.dll	04-06-2020 13:20	Application extension	20,150 KB
Qt6Core.dll	26-04-2021 21:07	Application extension	6,006 KB
Qt6Gui.dll	26-04-2021 21:07	Application extension	8,869 KB
Qt6Svg.dll	27-04-2021 10:59	Application extension	350 KB
Qt6Widgets.dll	26-04-2021 21:07	Application extension	6,219 KB

图 10.15-在数据子目录内复制的 winployment qt 生成的内容

7. 下一步是使用 `binarycreator` 工具创建安装程序。在 Qt 命令提示符下输入以下指令：

```
>binarycreator.exe -c config/config.xml -p packages HelloWorld
```

8. You can see that an installer file got generated inside our deployment directory:

↑ << build-HelloWorld-Desktop_Qt_6_1_0_MinGW_64_bit-Release > deployment			
Name	Date modified	Type	Size
config	18-05-2021 23:55	File folder	
packages	19-05-2021 00:38	File folder	
HelloWorld.exe	19-05-2021 00:40	Application	35,992 KB

图 10.16-在部署目录内创建的安装程序包

您也可以按照相同的步骤在 **Linux** 或 **MacOS** 上运行以下命令来生成安装程序文件：

```
$. /binarycreator -c config/config.xml -p packages HelloWorld
```

9. 我们已经得到了预期的结果。现在，让我们运行安装程序来验证部署包是否已正确创建。
0. Double-click on the installer file to begin the installation. You will see a nice setup wizard appear on screen:

![Figure 10.17 - Setup wizard running a deployment example ](img/f

图 10.17-运行部署示例的设置向导

1. 继续浏览页面以完成安装。退出安装向导。
2. 现在，从 **Windows**开始菜单启动该应用。您应该立即看到**HelloWorld**用户界面出现。
3. You can also find the installed application in **Add/Remove Programs**:

![Figure 10.18 - Deployment example entry in the Windows program ]

图 10.18-Windows 程序列表中的部署示例条目

4. 您可以使用随安装包一起安装的维护工具来更新、卸载和向应用添加组件。您可以在安装目录中找到该工具，如以下屏幕截图所示：

Name	Date modified	Type	Size
iconengines	19-05-2021 00:44	File folder	
imageformats	19-05-2021 00:44	File folder	
installerResources	19-05-2021 00:44	File folder	
platforms	19-05-2021 00:44	File folder	
styles	19-05-2021 00:44	File folder	
translations	19-05-2021 00:44	File folder	
components.xml	19-05-2021 00:44	XML Document	1 KB
D3Dcompiler_47.dll	11-03-2014 16:24	Application extension	4,077 KB
HelloWorld.exe	18-05-2021 22:39	Application	24 KB
InstallationLog.txt	19-05-2021 00:44	Text Document	3 KB
installer.dat	19-05-2021 00:44	DAT File	1 KB
libgcc_s_seh-1.dll	12-05-2018 11:41	Application extension	75 KB
libstdc++-6.dll	12-05-2018 11:41	Application extension	1,384 KB
libwinpthread-1.dll	12-05-2018 11:41	Application extension	51 KB
maintenancetool.dat	19-05-2021 00:44	DAT File	5 KB
<b>maintenancetool.exe</b>	<b>19-05-2021 00:44</b>	<b>Application</b>	<b>21,512 KB</b>
maintenancetool.ini	19-05-2021 00:44	Configuration settings	5 KB
network.xml	19-05-2021 00:44	XML Document	1 KB
opengl32sw.dll	04-06-2020 13:20	Application extension	20,150 KB
Qt6Core.dll	26-04-2021 21:07	Application extension	6,006 KB
Qt6Gui.dll	26-04-2021 21:07	Application extension	8,869 KB
Qt6Svg.dll	27-04-2021 10:59	Application extension	350 KB
Qt6Widgets.dll	26-04-2021 21:07	Application extension	6,219 KB

图 10.19-安装目录中的维护工具

祝贺你!。您已经为示例应用创建了安装程序包。现在,您可以将开发的 Qt 应用发送给您的用户和朋友。

您还可以使用自定义的安装向导页提供进一步的自定义。您可以在以下链接中找到可与 QIFW 配合使用的模板及其安装程序的完整列表:

<https://doc.qt.io/qtinstallerframework/ifw-customizing-installers.html>

<https://doc.qt.io/qtinstallerframework/qtifwexamples.html>.

您可以在此处了解该框架的更多特

性：<https://doc.qt.io/qtinstallerframework/ifw-overview.html>。

在本节中，我们创建了一个可安装的软件包，并将其发送给我们的最终用户。在下一节中，我们将学习如何在 **Android** 平台上部署。

## 在 **Android** 上部署

---

除了 **Windows**、**Linux** 和 **MacOS** 等桌面平台之外，由于用户数量庞大，移动平台也同样重要。许多开发人员希望让他们的应用在移动平台上可用。让我们来看看这是如何做到的。我们将简要讨论在 **Android** 上部署的注意事项。

在**第 5 章**，跨平台开发中，您已经了解了如何创建 `.apk` 文件，这是 **Android** 平台的部署包。所以，我们不会再讨论这些步骤了。在本节中，我们将在上载到 **Play Store** 之前讨论一些必要的更改：

1. 从工具包选择屏幕使用 **Android** 工具包创建一个简单的 *HelloWorld* 应用。
2. 将构建模式更改为版本模式。
3. Open the build settings of your project. You will find several options on the screen:

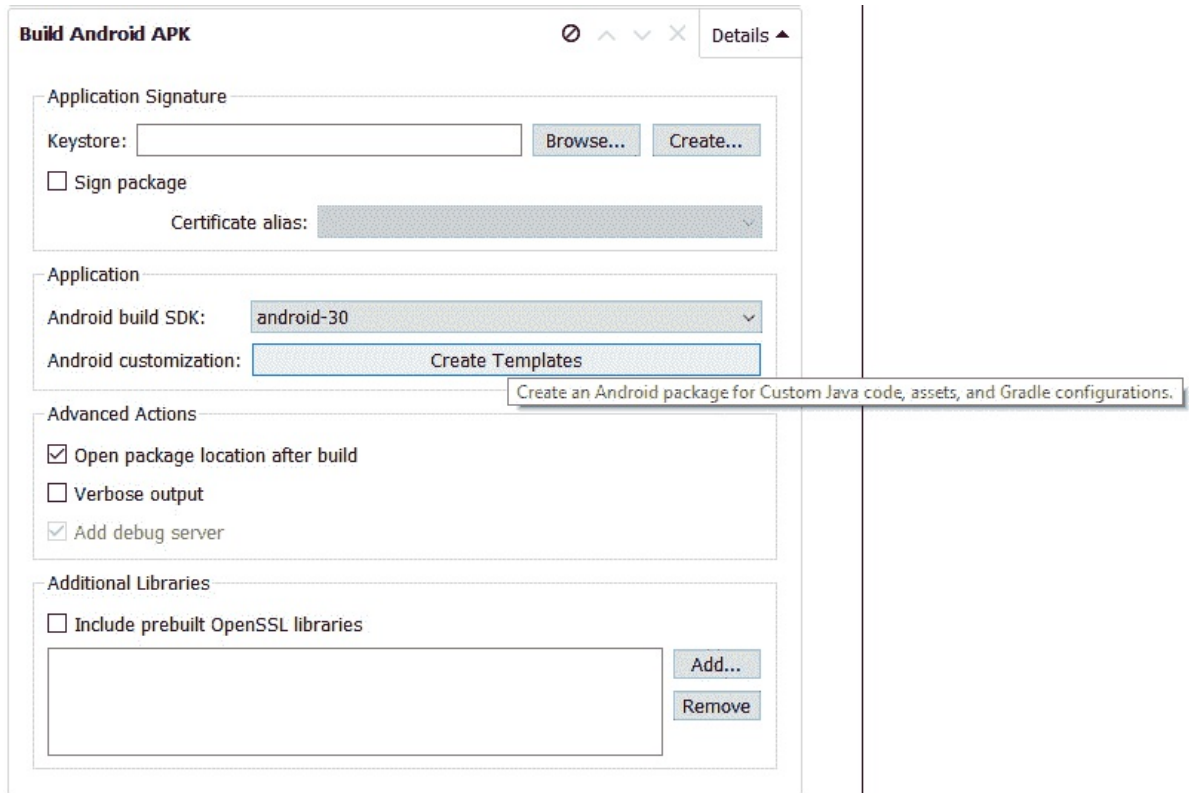


图 10.20-显示构建设置中的 Android 清单选项的屏幕截图

4. 您可以在应用签名部分下看到**Keystore**字段。单击浏览...。按钮选择您的现有密钥库文件或使用**Create...**。按钮创建新的密钥库文件。它可以保护关键的材料不被未经授权地使用。这是一个可选步骤，仅在签署部署二进制文件时需要。
5. When you click the **Create...** button, then you will see a dialog with several fields. Fill in the related fields and click on the **Save** button. *Figure 10.21* shows the keystore creation dialog:

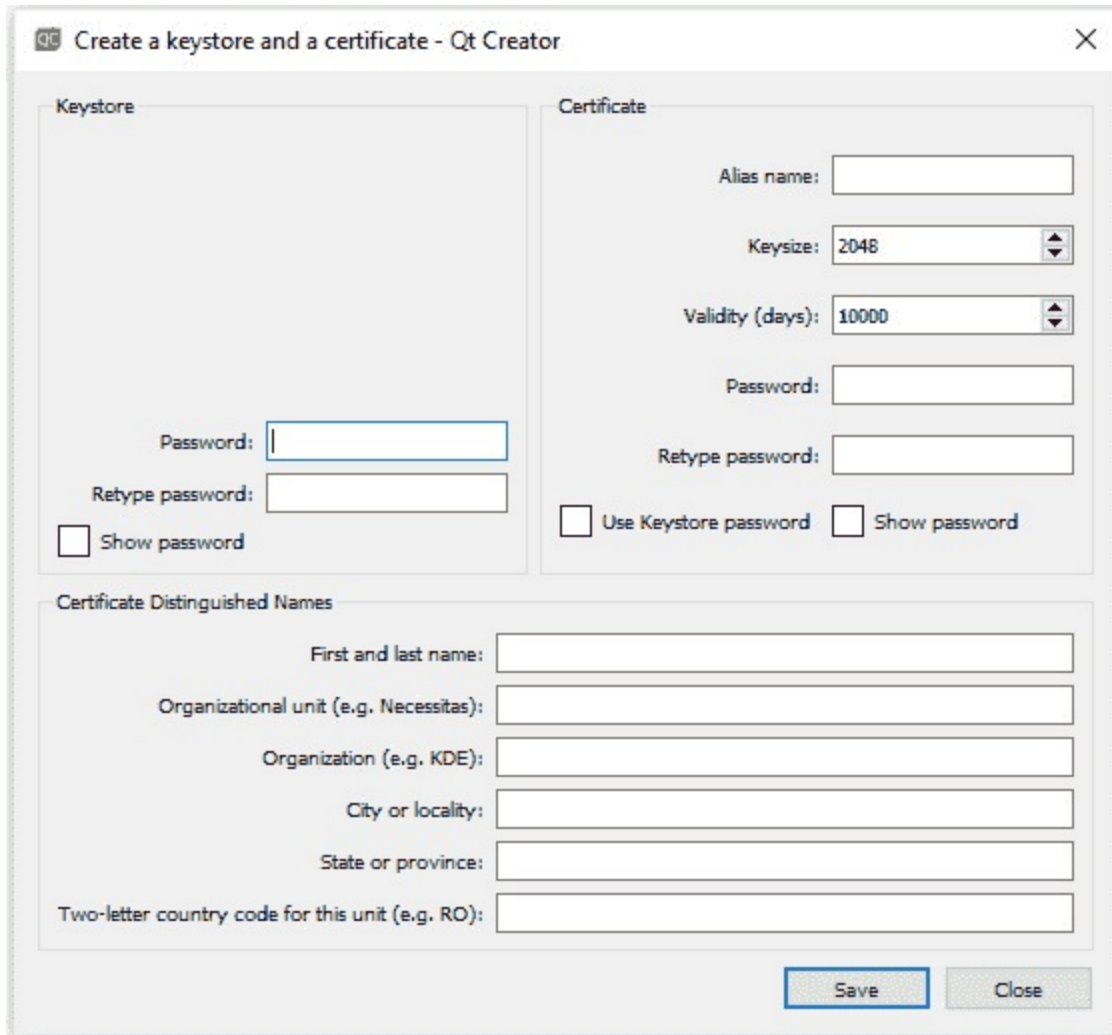


图 10.21-显示密钥库创建屏幕的屏幕截图

6. Save the keystore file anywhere, making sure to end the filename with `.keystore`.

下一步是在申请包上签字。这也是一个可选步骤，仅对 Play Store 发布是必需的。您可以在官方文档<https://developer.android.com/studio/publish/app-signing>中了解有关应用签名的更多信息。

7. You can select the target Android version and configure your



Android app by creating the `AndroidManifest.xml` file with Qt Creator. To do that, click on the **Create Templates** button on the **Build Android APK** screen. You will see a dialog appear, as shown in the following screenshot:

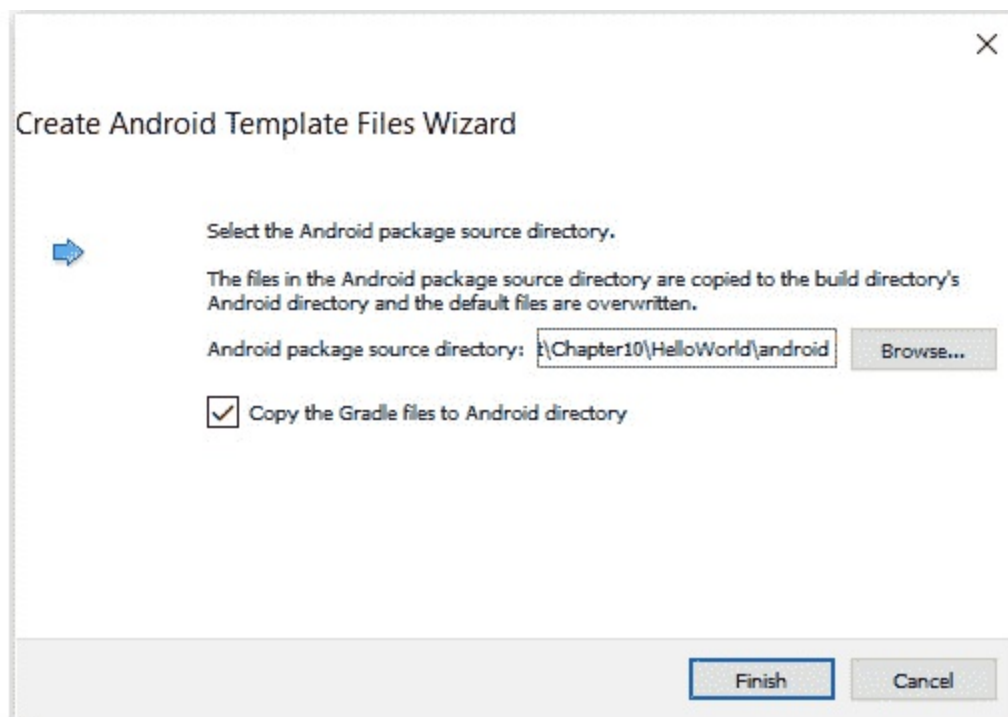


图 10.22-显示清单文件创建向导的屏幕截图

8. 打开清单文件。 您将看到您的 **Android** 应用的几个选项。
9. 您可以设置包名、版本编码、**SDK** 版本、应用图标、权限等。 如果你添加了一个独特的图标，那么你的应用的默认 **Android** 图标将不会出现在设备上。 它将使您的应用独一无二，并且很容易在屏幕上被发现。
10. Let's add *HelloWorld* as the application name and add the Qt icon as our application icon, as shown in the following screenshot:

! [Figure 10.23 - Android manifest file showing different available



图 10.23-显示不同可用选项的 **Android** 清单文件

1. 如果您正在使用任何第三方库，如 **OpenSSL**，请添加其他库。
2. 点击 **Qt Creator** 左下角的**Run**按钮，在 **Android** 设备上构建并运行该应用。您还可以点击**Run**按钮下方的**Deployment**按钮来创建部署二进制文件。
3. 您将看到一个新的对话框出现在屏幕上。此对话框允许您在物理 **Android** 硬件或软件仿真虚拟设备之间进行选择。
4. Connect your Android device and click on the **Refresh Device List** button. Don't forget to enable **Developer options** from your Android device settings. Allow **USB Debugging** when your Android device prompts you:

![Figure 10.24 - Android device selection dialog ](img/Figure\_10.24)

图 10.24-Android 设备选择对话框

5. If you want to use a virtual device, click on the **Create Android Virtual Device** button. You will see the following screen appear:



图 10.25-Android 虚拟设备创建屏幕

6. 如果屏幕警告您无法创建新的 **AVD**，则从 **Android SDK** 管理器更新 **Android** 平台工具和系统镜像。您可以从命令行更新这些内容，如下所示：

```
```\ncpp
> sdkmanager "platform-tools" "platforms;android-30"
> sdkmanager "system-images;android-30;google_apis;x86"
```

```
> sdkmanager --licenses  
```
```

7. 然后，运行以下命令以运行 `avdmanager`:

```
```cpp  
> avdmanager create avd -n Android30 -k "system-images;android-30;e  
```
```

8. The final step is to click on the **Run** button on the Qt Creator. You will see that the Android deployment package was created with the `.apk` extension inside the `build` folder:

![Figure 10.26 - Android installer file generated inside the build

图 10.26-在 `build` 目录内生成的 Android 安装程序文件

9. Internally, Qt runs the `androiddeployqt` utility. Sometimes, the tool may fail to create the package with the following error:

```
```cpp  
error: aidl.exe ...Failed to GetFullPathName  
```
```

在这种情况下，请使用较短的文件路径放置应用，并确保文件路径中没有目录包含

10. 您可以将 `.apk` 文件分发给您的朋友或用户。用户必须在他们的 **Android** 手机或平板电脑上接受一个选项，上面写着安装来历不明。为了避免这种情况，您的应该在 **Play Store** 上发布您的应用。
11. 但是，如果您想在 **Google Play Store** 上分发您的应用，那么您必须注册为 **Google Play** 开发者并在包上签名。谷歌向开发者收取少量一次性费用，以允许他们发布自己的应用。

2. 请注意，Qt 将 Android 应用视为封闭源代码。因此，如果你想让你的 Android 应用代码保密，你需要一个商业 Qt 许可证。

祝贺你!。您已成功生成可部署的 Android 应用。与 iOS 不同，Android 是一个开放系统。您可以将 `.apk` 文件复制或分发到运行相同 Android 版本的其他 Android 设备中并进行安装。

在本节中，我们为 Android 设备创建了一个可安装的软件包。在下一节中，我们将了解更多的安装工具。

## 其他安装工具

---

在本节中，我们将讨论一些可用于创建安装程序的其他工具。请注意，我们不会详细讨论这些工具。我还没有用 Qt6 验证这些安装框架。您可以访问各自的工具网站并了解它们的文档。除了 Qt 提供的安装框架和工具外，您还可以在 Windows 计算机上使用以下工具：

- **CQtDeployer** 是一个应用，用于提取所有相关的可执行程序库，并为您的应用创建启动脚本。该工具声称可以更快地部署应用，并提供灵活的基础设施。它同时支持 Windows 和 Linux 平台。您可以通过以下链接了解有关该工具的更多信息：<https://github.com/QuasarApp/CQtDeployer>。
- **Nullsoft Scriptable Installation System(NSIS)** 是由 Nullsoft 提供的脚本驱动安装创作工具，Winamp 就是由该公司创建的。它已经成为 InstallShield 等专有商业工具的流行替代品。当前版本的 NSIS 具有现代图形用户界面、LZMA 压缩、多语言支持和一个简单的插件系统。您可以在[https://nsis.sourceforge.io/Main\\_Page](https://nsis.sourceforge.io/Main_Page)浏览有关该工具的更多信息。
- **InstallShield** 是专有的软件应用，允许您创建安装程序和软件捆绑

包。InstallShield 通常用于在 Windows 平台台式机和服务系统上安装软件，但也可用于管理各种便携式和移动设备上的软件应用和软件包。看看它的功能并试用试用版吧。您可以在<https://www.revenera.com/install/products/installshield.html>下载试用版并阅读有关它的更多信息。

- **Inno Setup**是用 Delphi 创建的自由软件脚本驱动安装系统。它于 1997 年首次发布，但由于其出色的功能集和稳定性，仍然与许多商业安装程序竞争。通过以下链接了解有关此安装程序的更多信息：<https://jrsoftware.org/isinfo.php>。

您可以选择任何安装框架并部署您的应用。最后，它应该符合您的安装目标。

在本节中，我们讨论了其他几个可能对您的需要有帮助的安装工具。现在让我们总结一下本章的要点。

## 摘要

---

本章首先讨论了应用部署问题，并了解了静态库和动态库之间的区别。然后讨论了 Qt 中不同的部署工具以及 Windows 部署和安装的具体案例。有了这些知识，我们在 Windows 上部署了一个示例应用，并使用 Qt Installer Framework 创建了一个安装程序。此外，我们还发现在 Linux 和 MacOS 上部署应用，并磨练了在各种平台上部署应用的技能。稍后，我们解释了在将基于 Qt 的 Android 应用发布到 Play 商店之前需要考虑的一些重要问题。

最后，我们介绍了一些第三方安装实用程序。总之，您已经学会了在各种平台上开发、测试和部署 Qt 应用。有了这些知识，您应该能够创建您的安装包并与世界共享它们。

在第 11 章，国际化中，我们将了解如何开发支持翻译的 Qt 应用。

# 十一、国际化

---

原文: <https://gitee.com/apacheecn/apacheecn-c-cpp-zh-pt2/blob/master/docs/cross-plat-dev-qt6-mod-cpp/11.md>

在前面的章节中，我们学习了如何使用 **Qt** 控件或 **Qt Quick** 创建 **GUI** 应用。要使我们的应用在世界各地可用，我们需要向应用添加翻译。

使应用支持翻译的过程称为国际化。它使得为来自不同文化、地区或语言的观众本地化内容变得很容易。使用 **Qt** 可以非常轻松地将 **Qt** 小工具和 **Qt Quick** 应用翻译成当地语言。这些根据目标市场的地理和技术标准使应用适应不同语言的过程称为国际化。

您将学习如何制作具有多语言支持的应用。在整个章节中，我们将探索制作翻译感知应用的不同工具和流程。在本章中，我们将讨论以下内容：

- 国际化基础知识
- 编写翻译源代码
- 正在加载翻译文件
- 使用 **Qt** 小部件实现国际化
- 通过 **Qt Quick** 实现国际化
- 部署翻译

在本章结束时，您将能够使用 **Qt** 小部件和 **Qt Quick** 创建翻译感知应用。

## 技术要求

---

本章的技术要求包括在 **Windows 10**、**Ubuntu 20.04** 或 **MacOS 10.14**

等最新桌面平台上安装 Qt 6.0.0 和 Qt Creator 4.14.0 的最低版本。

本章使用的所有代码都可以从以下 giHub 链接下

载：<https://github.com/PacktPublishing/Cross-Platform-Development-with-Qt-6-and-Modern-Cpp/tree/master/Chapter11>。

## 重要音符

本章中使用的截图是在 Windows 平台上拍摄的。根据您的机器上的底层平台，您将看到类似的屏幕。

# 理解国际化和 Qt 语言学家

---

根据目标市场的不同语言、地理变化和技术规范调整应用的过程称为国际化和本地化。国际化是指创建一个软件应用的过程，该软件应用可以翻译成各种语言，并适用于不同的地区，而不需要进行重大的技术更改。国际化通常缩写为 **I18n**，**18** 是英文单词中字母 *i* 和 *n* 之间的字母数。产品本地化的难易程度很大程度上受其国际化程度的影响。为全球市场创建以语言和文化为重点的应用是一个复杂得多、耗时长得多的过程。因此，公司从产品开发开始就专注于为全球市场创建支持 **I18N** 的应用。

对于国际化，您应该以这样的方式设计您的应用，以避免稍后本地化或全球部署的障碍。这涵盖了一些方面，例如允许 **Unicode** 或在适当的情况下谨慎处理遗留字符编码、注意字符串连接、防止代码依赖于用户界面字符串值等。您应该提供对一些功能的支持，比如识别以后国际化可能需要的可翻译字符串和系统语言。

您的应用应该了解当地语言、日期和时间格式、数字系统或文化偏好。修改产品、应用或文档的内容以满足特定目标市场的语言、文化和其他

偏好称为本地化。本地化通常用英语写成*l10n*，其中 10 是介于*l*和*n*之间的字母数。本地化需要结合特定地区的要求并翻译特定地区或语言的应用。可本地化的功能应该与源代码分开，允许根据用户的文化偏好进行调整。

**Qt Language ist**是一个使用户能够创建您的 Qt 应用的翻译的工具。Qt Language ist 可以从安装目录或 IDE 启动。该工具附带两个集成程序，称为*lupdate*和*lrelease*。这些程序可以与 **qmake** 项目一起使用，也可以直接与文件系统一起使用。

*lupdate*工具在项目的源文件、头文件和*.ui*或*.qml*文件中查找可翻译字符串。然后创建或更新翻译文件(*.ts*文件)。您可以在命令行或在*.pro*文件中指定要处理的文件作为参数。*.ts*文件使用文档类型定义(**DTD**)格式，如下链接所述：

<https://doc.qt.io/qt-6/linguist-ts-file-format.html>

Qt 为国际化提供了出色的支持。Qt 在所有用户界面元素中都内置了对多种语言的支持。但是，在为您的应用编写源代码时，您必须遵循某些实践。这包括标记可翻译字符串、避免歧义字符串、使用编号参数(*%n*)作为占位符，以及加载正确的翻译文件。您可以使用 **C++** 和用户界面文件，也可以在这两个源代码中使用可翻译的字符串。该工具定位来自所有来源的字符串，并将其添加到具有相应上下文的单个*.ts*文件中。

应用开发时使用扩展名为*.ts*的翻译文件。这些文件可以编译成紧凑的二进制格式。编译后的翻译文件以*qm*格式编码，并具有*.qm*文件扩展名。在运行应用时，Qt 运行时使用*.qm*文件而不是*.ts*文件。您可以使用*lrelease*工具将*.ts*转换为*.qm*文件。*.qm*文件是轻量级二进制文件。

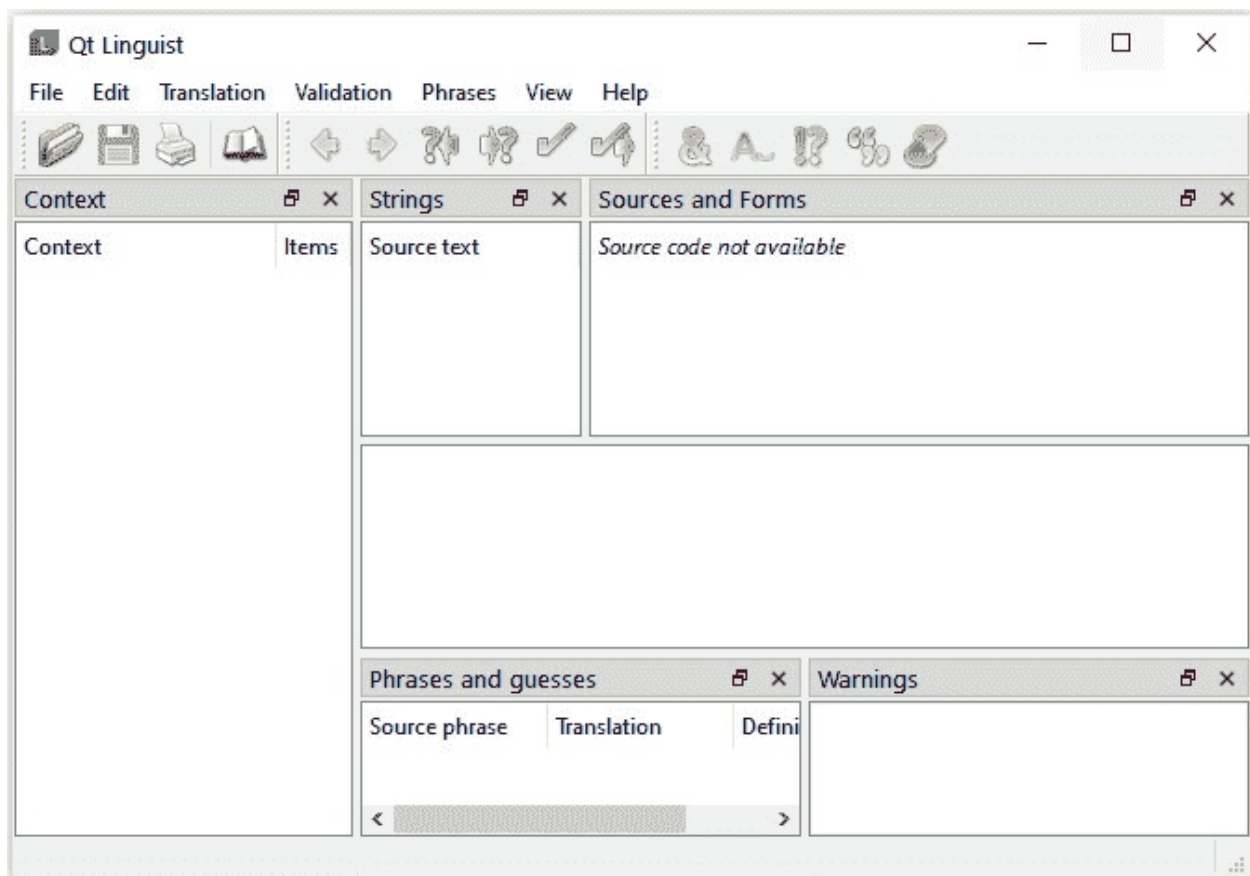


它允许闪电般的翻译查找。您可以在命令行上指定 `.ts` 文件，也可以在 `.pro` 项目文件中指定要由 `lrelease` 处理的文件。从测试版本到最终生产版本，每次发布应用时都会使用此工具。如果 `.qm` 文件不可用，则应用仍将正常运行，并使用源文件中的原始文本。

对于语言的选择，Qt 语言学家和 `lrelease` 使用一定的内部规则。您可以在以下链接中找到有关这些规则的详细信息：

<https://doc.qt.io/qt-6/i18n-plural-rules.html>

让我们来看看 Qt 语言学家的用户界面。您可以通过双击 **linguist** 可执行文件或从命令提示符中选择它，从 Qt 安装目录启动 Qt Language ist。您将看到以下用户界面出现在您的屏幕上：



## 图 11.1-Qt 语言学家用户界面

在上图中，您可以看到多个部分，工具栏中有几个禁用的按钮。您可以从文件菜单打开 `.ts` 文件。在本章后面的小节中，我们将在讨论示例的同时讨论这些部分。

您可以在以下网页上了解有关 Qt 语言学家和 GUI 界面的更多信息：

<https://doc.qt.io/qt-6/linguist-translators.html>

在本节中，您熟悉了与国际化相关的术语和 Qt 框架提供的工具。在很好地理解了基础知识之后，我们准备在下一节编写一个支持翻译的应用。

## 编写翻译源代码

---

在本节中，我们将讨论如何将字符串标记为可翻译字符串，以及如何使用 Qt 提供的工具。只要您的应用使用对用户可见的带引号的字符串，请确保 `QCoreApplication::translate()` 方法处理它。为此，只需使用 `tr()` 方法将用于显示的字符串标记为可翻译。此功能用于显示在 C++ 源文件中哪些文本字符串是可翻译的。

例如，如果要使用 `QLabel` 在用户界面上显示文本，则将文本嵌入到 `tr()` 方法中，如下所示：

```
QLabel *label = new QLabel(tr("Welcome"));
```

类名是 `QObject` 及其派生类的翻译上下文。要覆盖上下文，`QObject` 派生类必须在其类定义中使用 `Q_OBJECT` 宏。此宏设置派生类的上下文。

Qt 为国际化提供了几个方便的宏和方法。用于翻译的几个最常用的宏如下：

- `tr()` 如果翻译在 C++ 源文件中可用，则返回翻译后的字符串。
- `qsTr()` 如果 QML 文件中有翻译，则返回翻译后的字符串。
- `qtTrId()` 在 C++ 文件中查找并返回由 ID 标识的已翻译字符串。
- `qsTrId()` 在 QML 文件中查找并返回由 ID 标识的已翻译字符串。
- `QT_TR_NOOP()` 告诉 `lupdate` 收集当前上下文中的字符串，以便稍后翻译。
- `QT_TRID_NOOP()` 标记动态转换的 ID。
- `QCoreApplication::translate()` 通过查询已安装的翻译文件提供翻译。
- `qsTranslate()` 为 QML 文件中的给定上下文提供翻译版本。
- `QQmlEngine::retranslate()` 使用标记为翻译的字符串更新所有绑定表达式。

C++ 文件中的可翻译字符串使用 `tr()` 标记，而在 QML 文件中则使用 `qsTr()`。我们将在本章中讨论这些宏和方法。

所有可翻译字符串都由 `lupdate` 工具获取，并在翻译源(TS)中更新。TS 文件是 XML 文件。通常，TS 文件遵循以下命名约定：

```
ApplicationName>_<LanguageCode>_<CountryCode>.ts
```

在本约定中，`LanguageCode` 是小写的 ISO 639 语言代码，`CountryCode` 是大写的 ISO 3166 双字母国家代码。您可以使用特定的国家/地区代码通过为相同的语言创建针对不同国家/地区的翻译。在通过 Qt Creator 的新建项目向导创建 Qt 应用时，您可以使用语言代码和国家代码创建默认翻译文件。

一旦创建了 `.ts` 文件，就可以运行 `lupdate` 用所有用户可见的字符串更新 `.ts` 文件。您可以从命令行运行 `lupdate`，也可以从 Qt Creator 和

Visual Studio 外接程序运行 `lupdate`。让我们使用 Qt 的命令提示符为 `HelloWorld` 应用运行以下命令：

```
>lupdate HelloWorld.pro
```

`lupdate` 从不同的源文件(如 `.cpp`、`.h`、`.qml` 和 `.ui`) 获取可翻译字符串。要使 `lupdate` 有效工作，您应该在应用的 `.pro` 文件中的 `TRANSLATIONS` 变量下指定翻译文件。请看下面的 `.pro` 文件部分，我们在其中添加了六个翻译源文件：

```
TRANSLATIONS = \
 HelloWorld_de_DE.ts \
 HelloWorld_fi_FI \
 HelloWorld_es_ES.ts \
 HelloWorld_zh_CN.ts \
 HelloWorld_zh_TW.ts \
 HelloWorld_ru_RU.ts
```

您还可以使用 `*.ts` 添加基于通配符的转换文件选择。

要翻译 Qt Quick 应用，请使用 `qsTr()` 方法标记 `.qml` 文件内的字符串。您可以为单个 QML 文件创建翻译文件，如下所示：

```
>lupdate main.qml -ts HelloWorld_de_DE.ts
```

您可以为不同的语言创建多个翻译文件，并将它们放入一个 `.qrc` 文件中：

```
RESOURCES += translations.qrc
```

您可以使用 `lupdate` 处理 `.qrc` 文件中的所有 QML 文件，如下所示：

```
>lupdate qml.qrc -ts HelloWorld_de_DE.ts
```

要在不使用 `.qrc` 文件的情况下处理所有 QML 文件，请在 Qt 的命令提

示符下键入以下内容：

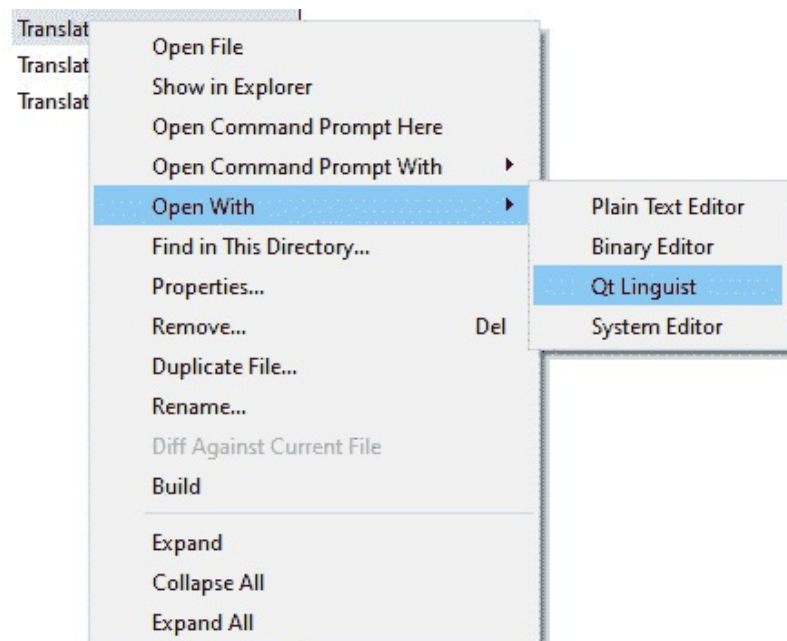
```
>lupdate -extensions qml -ts HelloWorld_de_DE.ts
```

您还可以将 C++ 源文件作为参数与资源文件一起传递。在 .pro 文件中提及翻译文件是可选的。您可以通过在命令行中指定转换文件来执行此操作，如下所示：

```
>lupdate qml.qrc messages.cpp -ts HelloWorld_de_DE.ts HelloWorld_
```

lrelease 集成标记为 finished 的翻译。如果字符串缺少翻译并标记为 unfinished，则使用原始文本。翻译人员或开发人员可以通过以下步骤修改 TS 文件内容并将其标记为 finished：

1. Launch Qt Linguist and open the .ts file from the **File** menu. Alternatively, right-click on the .ts file in the project structure and open with Qt Linguist, as shown here:



## 图 11.2-Qt Creator 中的使用 Qt Languist 选项打开

2. 然后单击**Context**视图中的任何上下文，在**Strings**视图中查看该上下文的可翻译字符串。
3. 在源文本视图中，输入当前字符串的翻译。您可以在短语和猜测视图找到现有的翻译和类似短语。
4. 翻译人员可以在翻译人员备注字段中输入备注。
5. 要完成转换，请按**Ctrl+Enter**并从工具栏中选择勾号图标。您将看到翻译后的字符串有一个绿色的勾号标记。
6. 最后，保存文件并退出 **Qt Languist** 工具。

您可以在不指定`.pro`文件的情况下运行`lrelease`。当您运行`lrelease`读取`.ts`文件时，它会生成应用在运行时使用的`.qm`文件：

```
>lrelease *.ts
```

生成`.qm`文件后，将其添加到`.qrc`文件。您的应用现在可以进行翻译了。

您还可以使用基于文本 ID 的转换机制。在这种方法中，应用中的每个可翻译字符串都被分配了一个唯一的标识符。这些唯一的文本标识符在源代码中直接用作实际字符串的替换。用户界面开发人员需要在这方面投入更多精力，但如果您的应用包含大量翻译后的字符串，则维护起来要容易得多。

在某些应用中，某些类可能不使用`QObject`作为基类或在其类定义中使用`Q_OBJECT`宏。但是这些类可能包含一些可能需要翻译的字符串。为了解决这个问题，Qt 提供了某些宏来添加翻译支持。

您可以使用 `Q_DECLARE_TR_FUNCTIONS(ClassName)` 为您的非 Qt 类启用翻译，如下所示：

```
class CustomClass
{
 Q_DECLARE_TR_FUNCTIONS(CustomClass)
public:
 CustomClass();
 ...
};
```

此宏在 `qcoreapplication.h` 内定义。当您添加此宏时，Qt 会将以下函数添加到您的类中以启用翻译：

```
static inline QString tr(const char *sourceString, const char
*disambiguation = nullptr, int n = -1)
{
 return QApplication::translate(#className, sourceString,
disambiguation, n);
}
```

从前面的代码中，您可以注意到 Qt 使用类名作为上下文调用 `QCoreApplication::translate()`。

您还可以在类或方法之外拥有可翻译的字符串；`QT_TR_NOOP()` 和 `QT_TRANSLATE_NOOP()` 用于将这些字符串标记为可翻译。基于文本 ID 的翻译有不同的宏和函数可用。您可以用 `qsTrId()` 代替 `qsTr()`，用 `QT_TRID_NOOP()` 代替 `QT_TR_NOOP()`。您可以在用户界面中使用与用户界面字符串相同的文本 ID，而不是普通字符串。

在 Qt 语言学家中，可以同时加载和编辑多个翻译文件。您还可以使用短语书重用现有翻译。短语手册是标准的 XML 文件，其中包含个典型短语及其翻译。这些文件是由 Qt 语言学家创建和更新的，可以由任意数量的项目和应用使用。如果您想要翻译短语手册中提供的源字符

串，可以使用 Qt 语言学家的批处理翻译功能。选择批量翻译以指定在批量翻译过程中要使用的短语手册以及使用的顺序。只应考虑没有当前翻译的条目，批量翻译的条目应标记为已接受。您还可以从**New Pase Book**选项创建新的短语书。

## 重要音符

`lupdate`默认情况下要求所有源代码都使用 UTF-8 编码。具有字节顺序标记(**BOM**)的文件也可以以 UTF-16 或 UTF-32 编码。您必须将`CODECFORSRCqmake` 变量设置为`UTF-16`，才能将没有 BOM 的文件解析为 UTF-16。默认情况下，某些编辑器(如 Visual Studio)使用单独的编码。通过将源代码限制为 ASCII 并对可翻译字符串使用转义序列，可以避免编码问题。

在本节中，我们讨论了如何使用`lupdate`和`lrelease`创建和更新转换文件。接下来，我们将学习如何在您的 Qt 应用中安装翻译器并加载翻译文件。

## 在 Qt 应用中加载翻译

在上一节中，我们创建了翻译文件，并了解了工具的用法。要在 TS 文件中查找翻译，可使用`QTranslator`函数。转换器必须在应用的 GUI 对象之前实例化。

让我们看看如何使用以下代码片段中的`QTranslator`加载这些转换文件：

```
QTranslator translator;
if(translator.load(QLocale(),QLatin1String("MyApplication")
 , QLatin1String("_"), QLatin1String(":/i18n")))
{
 application.installTranslator(&translator);
}
```



```
 }
 else
 {
 qDebug() << "Failed to load. "
 << QLocale::system().name();
 }
}
```

在前面的代码中，您可以看到我们已经创建了一个`translator`对象并加载了相应的转换文件。`QLocale`用于获取底层系统语言。您还可以使用`QLocale`本地化数字、日期、时间和货币字符串。

或者，您也可以加载转换文件，如下所示：

```
QString fileName = ":/i18n/MyApplication_"+QLocale::system().name()
+ ".qm";
translator.load(fileName);
```

这里，我们正在研究系统语言并加载对应的翻译文件。当您想要将系统语言用作应用语言时，上述方法效果很好。但是，一些用户可能喜欢使用与系统语言不同的区域语言。在这种情况下，我们可以根据用户选择更改语言。我们将在下一节中学习如何做到这一点。

## 动态切换语言

到目前为止，您已经了解了如何使用 **Qt** 应用的系统语言或默认语言。在大多数应用中，您只需检测`main()`中的语言并加载适当的`.qm`文件。有时，您的应用必须能够支持在仍在运行时更改用户的语言设置。多人轮班使用的应用可能需要在不需要重新启动的情况下切换语言。

要在基于 **Qt** 小部件的应用中实现这一点，您可以覆盖`QWidget::changeEvent()`。然后，您必须检查事件是否属于`QEvent::LanguageChange`类型。您可以相应地重新翻译用户界面。

以下代码片段解释了如何在基于 Qt 小部件的 GUI 中实现动态翻译：

```
void CustomWidget::changeEvent(QEvent *event)
{
 if (QEvent::LanguageChange == event->type())
 {
 ui->retranslateUi(this);
 }
 QWidget::changeEvent(event);
}
```

`QEvent::LocaleChange`可能会导致已安装的转换器列表切换。您可以创建具有用户界面的应用，该用户界面为用户提供更改当前应用语言的选项。当`QEvent::LanguageChange`事件发生时，`QWidget`子类的默认事件处理程序将调用此方法。如果您使用`QCoreApplication::installTranslator()`函数安装新的翻译，您将获得一个`LanguageChange`事件。此外，通过向其他小部件发送`LanguageChange`事件，GUI 将强制它们进行更新。任何其他事件都可以传递给基类进行进一步处理。

要启用动态转换，可以在命令行中或通过 GUI 提供一个选项。默认情况下，Qt 将`.ui`文件中的所有可翻译字符串放在`retranslateUi()`内。无论何时更改语言，都必须调用此函数。您还可以创建并调用自定义方法，以基于`QEvent::LanguageChange`事件重新转换通过 C++ 代码创建的字符串。

在本节中，我们讨论了如何在应用运行时实现动态翻译。在下一节中，我们将使用 Qt 小部件创建一个支持翻译的应用。

## 使用 Qt 小工具实现国际化

在前几节中，我们讨论了如何创建转换文件以及如何使用`QTranslator`

加载转换文件。让我们使用 Qt 小部件创建一个简单的示例，并实现我们的学习。

按照以下步骤创建示例应用：

1. 使用 **Qt Creator** 的新项目创建向导创建基于 **Qt** 小部件的应用，并按照前面章节中讨论的步骤操作屏幕。
2. 在翻译文件屏幕上，选择\*\*德语(德国)\*\*作为语言选项，或选择任何首选语言。
3. 完成项目创建。您将看到Simplei18nDemo\_de\_DE.ts已在您的项目结构中创建。
4. 接下来，将QLabel添加到.ui文件并添加Welcome文本。
5. Next, run `lupdate`. You can run `lupdate` from the command line as well as from the Qt Creator interface, as shown in *Figure 11.3*:

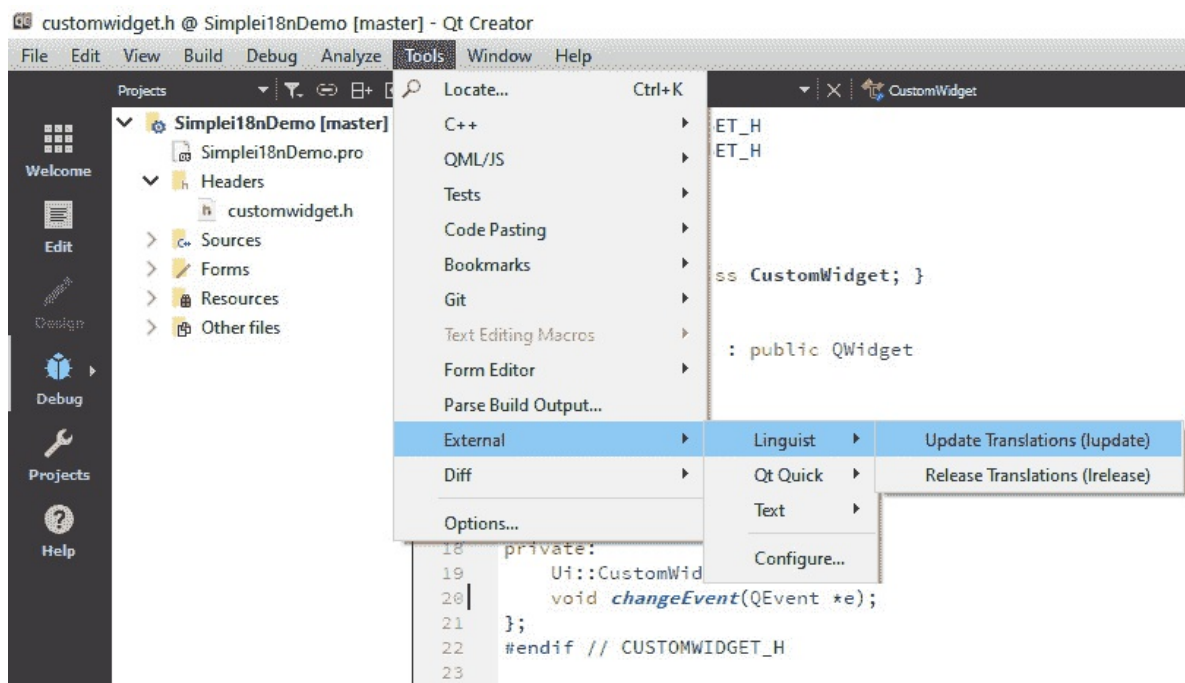


图 11.3-Qt Creator 中的 Qt 语言学家选项

6. 运行 `lupdate` 时，您将在控制台窗口中看到以下输出：

```
C:\Qt6Book\Chapter11\Simplei18nDemo>lupdate Simplei18nDemo.pro
Info: creating stash file C:\Qt6Book\Chapter11\Simplei18nDemo\
Updating 'Simplei18nDemo_de_DE.ts'...
Found 2 source text(s) (2 new and 0 already existing)
```

7. Now, the `.ts` file is updated with strings. Open `Simplei18nDemo_de_DE.ts` with a plain text editor. You should see the following content:

```
<?xml version="1.0" encoding="utf-8"?>
<!DOCTYPE TS>
<TS version="2.1" language="de_DE">
<context>
 <name>CustomWidget</name>
 <message>
 <location filename="customwidget.ui"
 line="14"/>
 <source>Simple i18n Demo</source>
 <translation type="unfinished"></translation>
 </message>
 <message>
 <location filename="customwidget.ui"
 line="25"/>
 <source>Welcome</source>
 <translation type="unfinished"></translation>
 </message>
</context>
</TS>
```

您可以看到，用户界面字符串在 `.ts` 文件中更新了，并且在文件顶部为翻译定义了语言。您可以通过修改代码中的此字段来创建各自的翻译文件：

```
<TS version="2.1" language="de_DE">
```

您还将看到翻译状态为 `unfinished`。

8. So, let's open the file with Qt Linguist and complete the translation:

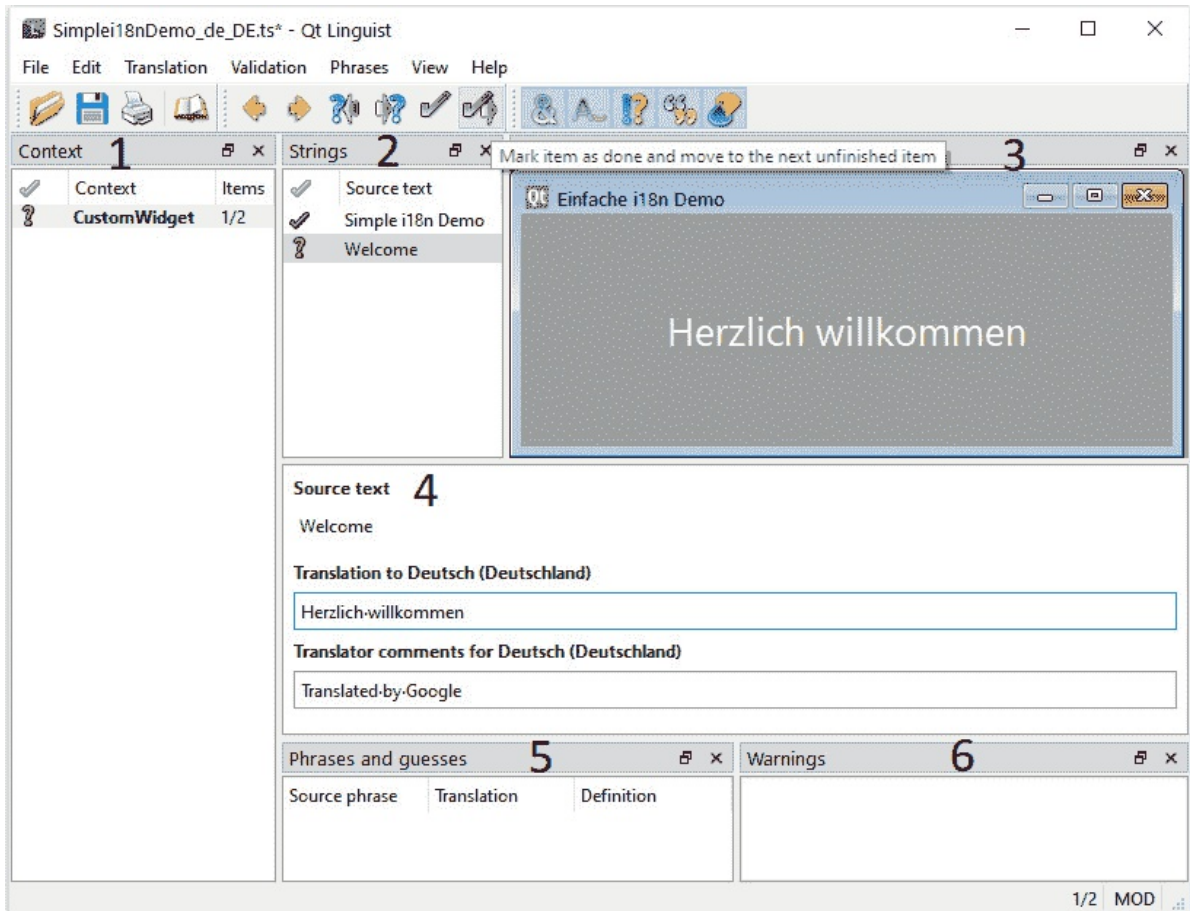


图 11.4-显示 Qt 语言学家界面不同部分的示例

9. 您将在用户界面中看到六个个不同的部分。 在上下文视图中选择上下文以加载相应的字符串。
0. Add a translation in the **Source text** view. You can use Google Translate to translate the string into the desired language. Here we have translated the strings to the German language using Google Translate.

使用了多个翻译。 如果字符串没有确切含义，请忽略。 我不熟悉德语。 我已经

1. 要完成翻译，请按**Ctrl+Enter**或单击工具栏上的绿色勾号图标。
2. 下一步是保存翻译。 对上下文中列出的所有可翻译字符串重复此操作。
3. 从 Qt 的命令提示符或 IDE 的选项运行**lrelease**。 您将看到生成了**.qm**文件：

```
```cpp
C:\Qt6Book\Chapter11\Simplei18nDemo>lrelease *.ts
Updating 'Simplei18nDemo_de_DE.qm'...
Generated 2 translation(s) (2 finished and 0 unfinished)
```
```

4. 让我们将翻译器添加到**main.cpp**并加载翻译文件：

```
```cpp
#include "customwidget.h"
#include <QApplication>
#include <QTranslator>
#include <QDebug>
int main(int argc, char *argv[])
{
    QApplication app(argc, argv);
    QTranslator translator;
    if(translator.load(":/translations
                        /Simplei18nDemo_de_DE.qm"))
    {
        app.installTranslator(&translator);
        qDebug()<<"Loaded successfully!";
    }
    else
    {
        qWarning()<<"Loading failed.";
    }
    CustomWidget customUI;
    customUI.show();
    return app.exec();
}
```

```
}..
```

5. 最后一步是运行 **qmake** 并构建应用。然后，点击左下角的**Run**按钮。
6. 我们已经成功地将我们的图形用户界面翻译成了德语。您将看到以下输出：

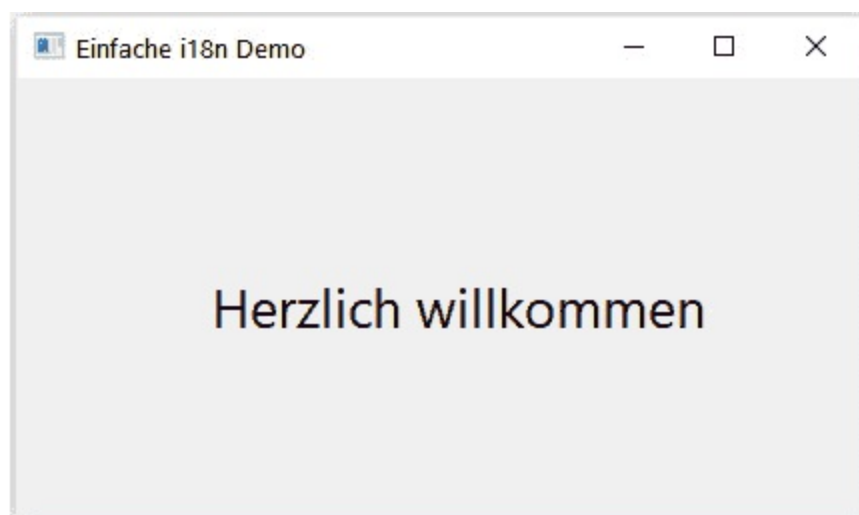


图 11.5-使用 Qt 小部件的翻译示例的输出

祝贺你!。您学习了如何将应用翻译成种不同的语言。现在，您可以将 Qt 应用翻译成当地语言，并与您的朋友和同事分享。

在本节中，我们讨论了如何使用 Qt 小部件创建翻译感知应用。在下一节中，我们将向 Qt 小部件应用添加动态翻译功能。

向 Qt 小部件应用添加动态翻译

在上一节中，您了解了如何创建基于 Qt 小部件的应用并将语言更改为首选语言。但是，与大多数全球应用一样，您可能需要提供更多翻译，并允许用户动态更改语言。

让我们用一些额外的实现来修改上一节中的示例：

1. Add a combo box to the `.ui` file and add three languages to it. For explanation purposes, we have used English, German, and Spanish. We have added a message at the center and added a language-switching option in a dropdown:

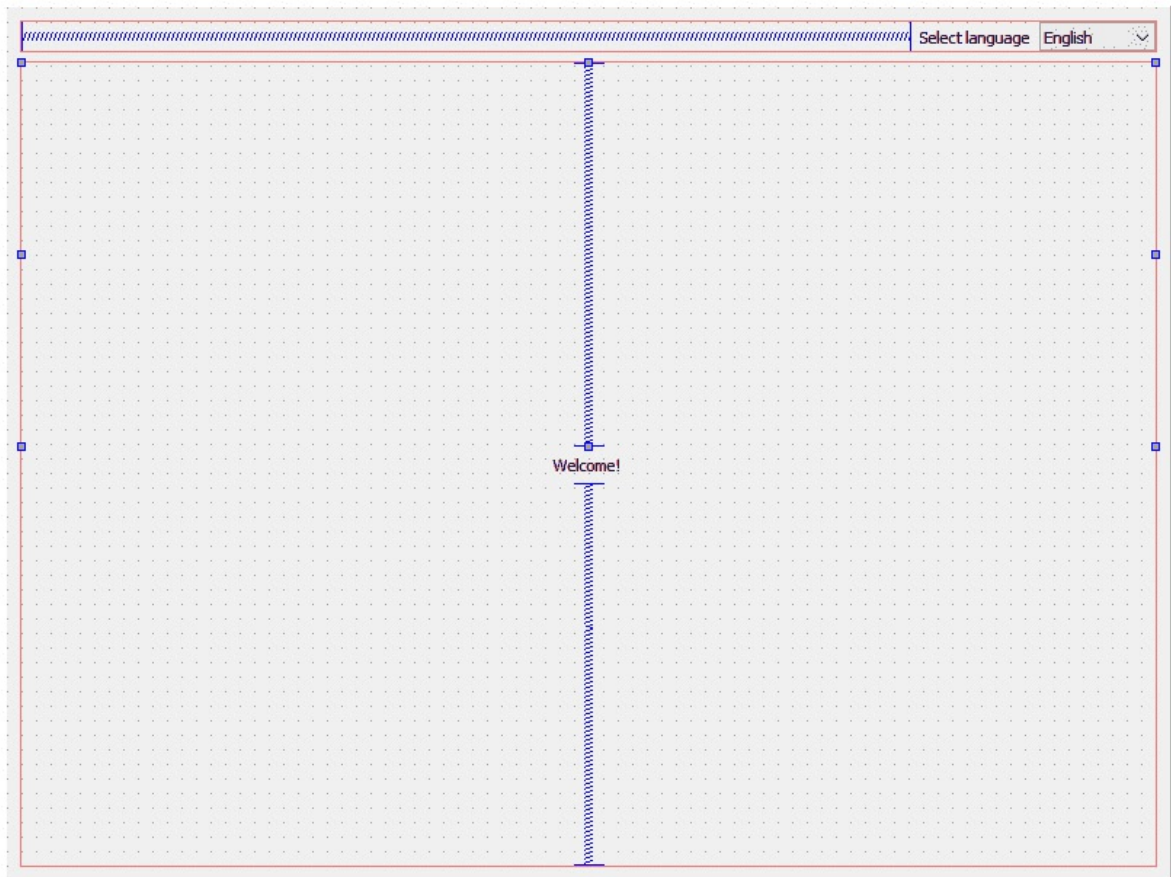


图 11.6-Qt Designer 中显示示例中使用的布局的表单

2. 将新的翻译文件添加到项目文件中，如下所示：

```
TRANSLATIONS += \  
    WidgetTranslationDemo_en_US.ts \  
    WidgetTranslationDemo_de_DE.ts \  
    WidgetTranslationDemo_es_ES.ts
```


3. 让我们修改 `CustomWidget` 类，并为动态转换添加以下方法：

```
#ifndef CUSTOMWIDGET_H
#define CUSTOMWIDGET_H
#include <QWidget>
#include <QTranslator>
QT_BEGIN_NAMESPACE
namespace Ui { class CustomWidget; }
QT_END_NAMESPACE
class CustomWidget : public QWidget
{
    Q_OBJECT
public:
    CustomWidget(QWidget *parent = nullptr);
    ~CustomWidget();
public slots:
    void languageChanged(int index);
    void switchTranslator(const QString& filename);
    void changeEvent(QEvent *event);
private:
    Ui::CustomWidget *ui;
    QTranslator m_translator;
};
#endif // CUSTOMWIDGET_H
```

4. 下一步是连接信号和插槽。我们已经在构造函数中创建了连接：

```
CustomWidget::CustomWidget(QWidget *parent)
    : QWidget(parent), ui(new Ui::CustomWidget)
{
    ui->setupUi(this);
    connect(ui->languageSelectorCmbBox,
            SIGNAL(currentIndexChanged(int)), this,
            SLOT(languageChanged(int)));
    qApp->installTranslator(&m_translator);
}
```

5. Let's add the following code to the slot definition:

```
void CustomWidget::languageChanged(int index)
{
    switch(index)
```

```

{
case 0: //English
    switchTranslator(":/i18n/
        WidgetTranslationDemo_en_US.qm");
    break;
case 1: //German
    switchTranslator(":/i18n/
        WidgetTranslationDemo_de_DE.qm");
    break;
case 2: //Spanish
    switchTranslator(":/i18n/
        WidgetTranslationDemo_es_ES.qm");
    break;
}
}

```

这里，我们通过组合框索引更改信号从用户界面接收语言选择。

6. 下一步是安装新的转换器：

```

void CustomWidget::switchTranslator(const QString& filename)
{
    qApp->removeTranslator(&m_translator);
    if(m_translator.load(filename))
    {
        qApp->installTranslator(&m_translator);
    }
}

```

7. 最后一步是重新实现 `changeEvent()`：

```

void CustomWidget::changeEvent(QEvent *event)
{
    if (event->type() == QEvent::LanguageChange)
    {
        ui->retranslateUi(this);
    }
    QWidget::changeEvent(event);
}

```

8. Run qmake and hit the **Run** button on the IDE.

将出现以下屏幕：

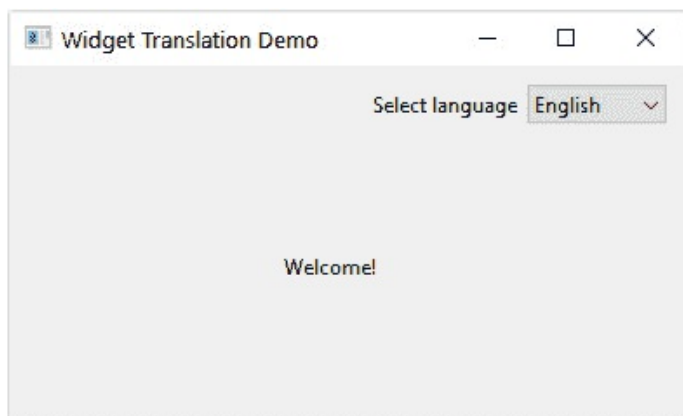


图 11.7-示例显示了选择英语时的输出

9. Change the language from the language selection dropdown. Let's select **German** as the new language. You will see the entire GUI changed with German strings:

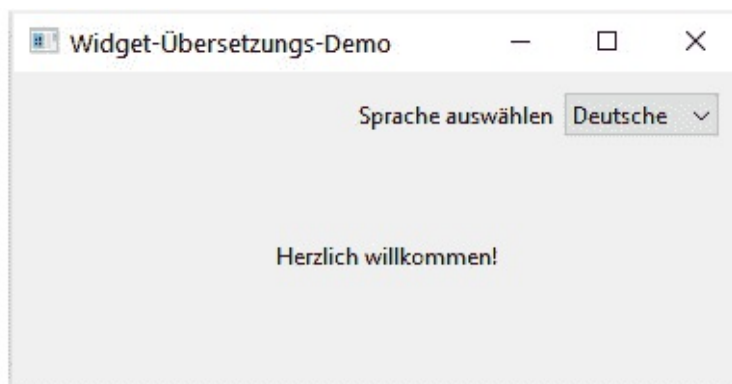


图 11.8-显示选择德语时的输出示例

0. 再次将语言切换为西班牙语。 您将看到 **GUI** 文本更改为西班牙语：

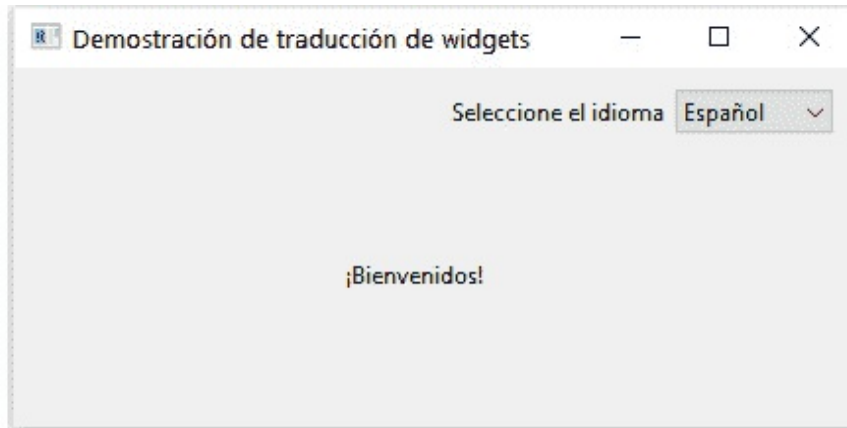


图 11.9-选择西班牙语时显示输出的示例

祝贺你!。您已经成功创建了一个多语言 Qt 小部件应用。

在本节中，您了解了如何在运行时转换基于 Qt 小部件的 GUI。在下一节中，我们将使用 Qt Quick 创建一个支持翻译的应用。

通过 Qt Quick 实现国际化

在上一节中，我们讨论了 Qt 小部件中的国际化。在本节中，我们将讨论国际化 Qt Quick 应用的不同方面。Qt Quick 应用中的底层本地化方案类似于 Qt 窗口小部件应用。Qt Quick 中也使用了 Qt 语言手册中描述的相同工具集。您可以翻译同时使用 C++ 和 QML 的应用。

在 Qt 项目文件中，`SOURCES` 变量用于 C++ 源文件。如果在此变量下列出 QML 或 JavaScript 文件，编译器将尝试使用这些文件，并将其视为 C++ 文件。作为解决办法，您可以使用 `update_only {...}` 条件声明使 QML 文件仅对 `update` 工具可见，但对 C++ 编译器不可见。

请考虑以下示例。应用的 `.pro` 文件片段列出了两个 QML 文件：

```
update_only {
SOURCES = main.qml \
          HomeScreen.qml
```

```
}
```

您还可以使用通配符来指定 **QML** 源文件。由于搜索不是递归的，因此必须列出在源代码中可以找到用户界面字符串的每个目录：

```
lupdate_only{  
  SOURCES = *.qml \  
            *.js  
}
```

让我们用一个简单的翻译来创建一个例子。我们将创建一个与在 **Qt** 小部件应用中创建的屏幕类似的屏幕。遵循以下步骤：

1. 使用 **Qt Creator** 的新项目创建向导创建基于 **Qt Quick** 的应用，并按照前面章节中讨论的步骤操作屏幕。
2. 在翻译文件屏幕上，选择**德语(德国)**作为语言选项或任何首选语言。
3. 完成项目创建。您将看到**QMLTranslationDemo_de_DE.ts**已在您的项目结构中创建。
4. 接下来，将**Text**添加到**.qml**文件并添加**Welcome**文本：

```
import QtQuick  
import QtQuick.Window  
Window {  
    width: 512  
    height: 512  
    visible: true  
    title: qsTr("QML Translation Demo")  
    Text {  
        id: textElement  
        anchors.centerIn: parent  
        text: qsTr("Welcome")  
    }  
}
```

5. 将以下行代码添加到 `main.cpp`:

```
#include <QGuiApplication>
#include <QQmlApplicationEngine>
#include <QTranslator>
#include <QDebug>
int main(int argc, char *argv[])
{
    QGuiApplication app(argc, argv);
    QTranslator translator;
    if(translator.load(":/translations/
        QMLTranslationDemo_de_DE.qm"))
    {
        app.installTranslator(&translator);
        qDebug()<<"Loaded successfully!";
    }
    else
    {
        qWarning()<<"Loading failed.";
    }
    QQmlApplicationEngine engine;
    const QUrl url(QStringLiteral("qrc:/main.qml"));
    QObject::connect(&engine,
        &QQmlApplicationEngine::objectCreated,
        &app, [url](QObject *obj, const QUrl
            &objUrl)
        {
            if (!obj && url == objUrl)
                QCoreApplication::exit(-1);
        }, Qt::QueuedConnection);
    engine.load(url);
    return app.exec();
}
```

6. 这些步骤与 Qt 小部件示例类似。接下来，运行 `lupdate`。
7. 按照相同的步骤使用 Qt Linguist 更新 `.ts` 文件中的翻译。
8. 从 Qt 的命令提示符或 IDE 的选项运行 `lrelease`。您将看到生成了 `.qm` 文件。

9. 将 `.qm` 文件添加到资源(`.qrc`)文件并运行 `qmake`。
10. 最后一步是构建和运行应用。 点击 **Qt Creator** 中的 **Run** 按钮。
11. 您将看到与我们在 **Qt** 小部件示例中看到的相同的输出：

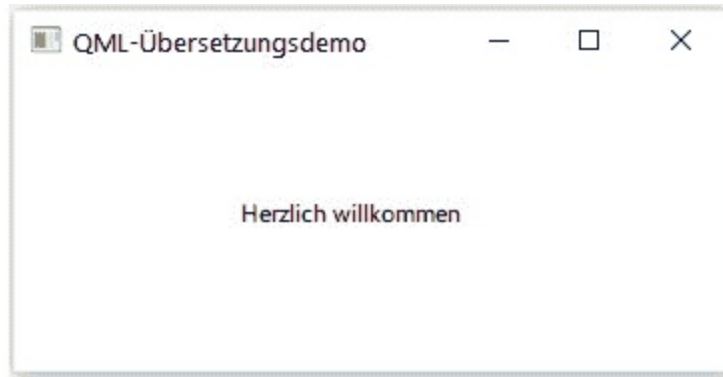


图 11.10-使用 Qt Quick 的翻译示例的输出

在前面的示例中，我们将我们的 Qt Quick 应用翻译成了德语。

在本节中，我们讨论了如何使用 Qt Quick 创建翻译感知应用。 在下一节中，我们将向 Qt Quick 应用添加动态翻译功能。

在 Qt Quick 应用中动态翻译

在上一节中，您了解了如何创建基于 Qt Quick 的应用，以及如何将语言更改为首选语言。 就像 Qt 小部件示例一样，您还可以向 Qt Quick 应用添加动态翻译。

让我们用一些额外的实现来修改前面的示例：

1. Create an i18n support class named `TranslationSupport` and add the following lines:

```
#ifndef TRANSLATIONSUPPORT_H
#define TRANSLATIONSUPPORT_H
#include <QObject>
```

```

#include <QTranslator>
class TranslationSupport : public QObject
{
    Q_OBJECT
public:
    explicit TranslationSupport(QObject *parent =
                                nullptr);
public slots:
    void languageChanged(int index);
    void switchTranslator(const QString& filename);
signals:
    void updateGUI();
private:
    QTranslator m_translator;
};
#endif // TRANSLATIONSUPPORT_H

```

前面的代码是一个助手类，它支持 **QML** 中的翻译特性。用于更新翻译器中的翻译文件。

2. 在下一步中，添加下面的代码来切换转换器：

```

void TranslationSupport::switchTranslator(const QString& filename)
{
    qApp->removeTranslator(&m_translator);
    if(m_translator.load(filename))
    {
        qApp->installTranslator(&m_translator);
        emit updateGUI();
    }
}

```

3. 然后，将以下代码添加到 **QML** `INVOKABLE` 方法定义中：

```

void TranslationSupport::languageChanged(int index)
{
    switch(index)
    {
        case 0: //English
            switchTranslator(":/i18n/
                            QMLDynamicTranslation_en_US.qm");
    }
}

```



```

        break;
    case 1: //German
        switchTranslator(":/i18n/
            QMLDynamicTranslation_de_DE.qm");
        break;
    case 2: //Spanish
        switchTranslator(":/i18n/
            QMLDynamicTranslation_es_ES.qm");
        break;
    }
}

```

4. 在 `main.cpp` 文件中，添加下面的代码。请注意，我们已将 `TranslationSupport` 实例公开给 QML 引擎：

```

#include <QGuiApplication>
#include <QQmlApplicationEngine>
#include <QQmlContext>
#include "translationsupport.h"
int main(int argc, char *argv[])
{
    QGuiApplication app(argc, argv);
    TranslationSupport i18nSupport;
    QQmlApplicationEngine engine;
    engine.rootContext()->setContextProperty(
        "i18nSupport", &i18nSupport);
    const QUrl url(QStringLiteral("qrc:/main.qml"));
    QObject::connect(&i18nSupport,
        &TranslationSupport::updateGUI, &engine,
        &QQmlApplicationEngine::retranslate);
    engine.load(url);
    return app.exec();
}

```

5. 然后用 `QQmlApplicationEngine::retranslate()` 方法将 `updateGUI()` 信号相加。
6. 让我们来看看 `main.qml` 文件。我们已经向 `.qml` 文件添加了一个组合框，并向其中添加了三种语言。出于解释目的，我们使用了英语、德语和西班牙语：

```

Text {
    id: textElement
    anchors.centerIn: parent
    text: qsTr("Welcome!")
}
Row {
    anchors {
        top: parent.top;    topMargin: 10 ;
        right: parent.right; rightMargin: 10;
    }
    spacing: 10
    Text{
        text: qsTr("Select language")
        verticalAlignment: Text.AlignVCenter
        height: 20
    }
    ComboBox {
        height: 20
        model: ListModel {
            id: model
            ListElement { text: qsTr("English")}
            ListElement { text: qsTr("German")}
            ListElement { text: qsTr("Spanish")}
        }
        onCurrentIndexChanged: {
            i18nSupport.languageChanged(currentIndex)
        }
    }
}

```

7. 运行 `lupdate` 并继续执行转换过程。
8. 按照相同的步骤使用 **Qt Linguist** 更新 `.ts` 文件中的翻译。
9. 从 **Qt** 的命令提示符或 **IDE** 的选项运行 `lrelease`。您将看到生成了 `.qm` 文件。
0. 将 `.qm` 文件添加到资源(`.qrc`)文件并运行 `qmake`。
1. The last step is to build and run the application. Hit the **Run** button in Qt Creator.

将出现以下屏幕：

![Figure 11.11 - Qt Quick example showing the output when the Eng]

图 11.11-Qt 快速示例显示了选择英语时的输出

2. Change the language from the language selection dropdown. Let's select **German** as the new language. You will see the entire GUI changed with German strings:

![Figure 11.12 - Qt Quick example showing the output when the Germ]

图 11.12-Qt 快速示例显示了选择德语时的输出

3. 再次将语言切换为西班牙语。 您将看到 **GUI** 文本更改为西班牙语：

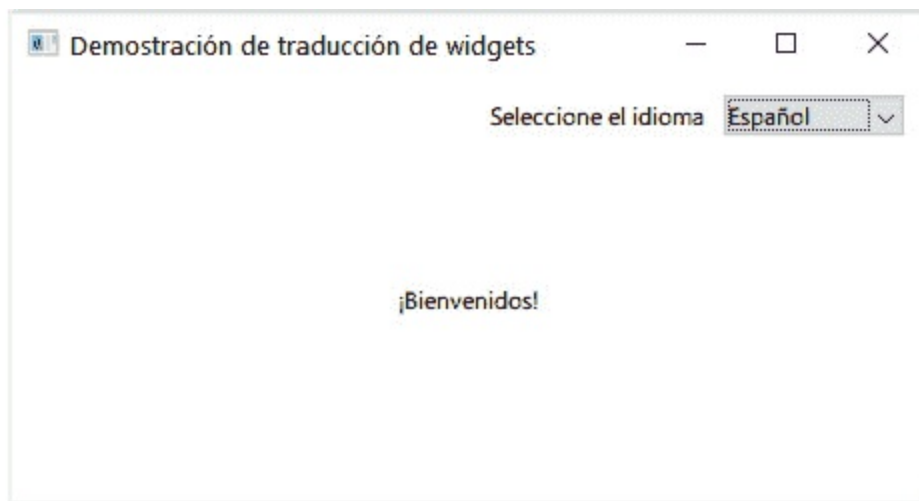


图 11.13-Qt 快速示例显示了选择西班牙语时的输出

祝贺你！。 您已经成功创建了一个多语言 Qt Quick 应用。

在本节中，您了解了如何在运行时转换基于 Qt Quick 的 GUI。在下一节中，我们将讨论如何部署翻译文件。

部署翻译

在前面的小节中，我们了解了如何使用 **Qt** 小部件和 **QML** 创建翻译感知应用。您不必将 `.ts` 文件与您的应用一起发送。要部署翻译，您的发布团队必须使用更新后的 `.qm` 文件，并将其与应用包一起提供。应用所需的 `.qm` 文件应放置在 `QTranslator` 可以找到它们的位置。通常，这是通过在资源(`.qrc`)文件中嵌入 `qm` 文件或指定包含相对于 `QCoreApplication::applicationDirPath()` 的 `.qm` 文件的路径来实现的。`rcc` 工具用于在构建过程中将翻译文件嵌入到 **Qt** 应用中。它的工作方式是生成包含指定数据的相应 **C++** 文件。

通过向 `.pro` 文件添加脚本，可以自动生成 `.qm` 文件。您可以按照以下步骤执行此操作：

1. 首先，使用语言代码在 **Qt** 项目(`.pro`)文件中的 `LANGUAGES` 变量下声明语言。
2. 将 `lrelease` 和 `embed_translations` 添加到 `CONFIG` 变量。
3. 然后添加一个函数来生成目标语言的 `.ts` 文件。
4. 最后，定义 `TRANSLATIONS_FILES` 变量，使用 `lrelease` 创建 `.qm` 文件，并将它们嵌入到应用资源中。

前面的步骤将自动运行 `lrelease` 并生成 `.qm` 文件。`lrelease` 工具处理在 `TRANSLATIONS` 和 `EXTRA_TRANSLATIONS` 下面列出的翻译文件。

与 `TRANSLATIONS` 变量不同，`EXTRA_TRANSLATIONS` 下面列出的文件只由 `lrelease` 工具处理，而不是由 `lupdate` 处理。您需要将 `.qm` 文件嵌入到您的资源中，或者将 `.qm` 文件与部署包一起提供。

您可以在此处了解有关自动生成 **QM** 文件的更多信息：

https://wiki.qt.io/Automating_generation_of_qm_files。

在本节中，您了解了如何部署翻译文件。在下一节中，我们将总结本章的要点。

摘要

在本章中，我们介绍了 **Qt** 中国际化和本地化的核心概念。我们讨论了 **Qt** 为国际化提供的不同工具。我们学习了如何使用 **Qt** 语言学家。我们还了解了如何将 **Qt** 小部件应用翻译成不同的语言。然后，我们学会了如何动态翻译。

在本章的后半部分，我们讨论了如何翻译 **Qt Quick** 应用。之后，我们学习了如何在 **Qt Quick** 应用中动态切换语言。现在，您可以创建具有多种语言的应用，并将其与不同地理区域的客户或朋友共享。

在[第 12 章](#)，性能注意事项中，我们将了解在 **Qt** 应用中优化性能的工具和技巧。

十二、性能注意事项

原文: <https://gitee.com/apacheecn/apacheecn-c-cpp-zh-pt2/blob/master/docs/cross-plat-dev-qt6-mod-cpp/12.md>

在本章中，我们将概述性能优化技术，以及如何在基于 **Qt** 的应用开发环境中应用这些技术。性能是应用成功的一个非常重要的因素。业绩失败可能导致业务失败、客户关系不佳、竞争力下降和收入损失。延迟性能优化可能会给您的声誉和组织形象带来巨大损失。因此，进行性能调优非常重要。

您还将了解性能瓶颈以及如何克服它们。我们将讨论诊断性能问题的不同分析工具，特别是一些流行的工具。然后，您将学习如何分析和评估性能。本章还介绍了**Qt** 建模语言(**QML**)Profiler 和 Flame Graph，以查找 **Qt Quick** 应用中的潜在瓶颈。您还将了解在开发 **Qt** 应用时应该遵循的一些最佳实践。

我们将讨论以下主题：

- 了解性能优化
- 优化 **C++** 代码
- 使用并发性、并行性和多线程
- 使用 **QML Profiler** 和 **Flame Graph** 分析 **Qt Quick** 应用
- 其他 **Qt Creator** 分析工具
- 优化图形性能
- 创建基准
- 不同的分析工具和优化策略
- **Qt** 小部件的性能注意事项
- 学习 **QML** 编码的最佳实践

在本章结束时，您将学会为基于 C++ 和 QML 的应用编写高性能优化代码。

技术要求

本章的技术要求包括在 Windows 10、Ubuntu 20.04 或 MacOS 10.14 等最新桌面平台上安装 Qt 6.0.0 和 Qt Creator 4.14.0 的最低版本。

本章使用的所有代码都可以从以下 giHub 链接下

载：<https://github.com/PacktPublishing/Cross-Platform-Development-with-Qt-6-and-Modern-Cpp/tree/master/Chapter12/QMLPerformanceDemo>。

重要音符

本章中使用的截图是在 Windows 平台上拍摄的。根据计算机中的底层平台，您将看到类似的屏幕。

了解性能优化

进行性能优化是为了提高应用的性能。您可能想知道为什么这是必要的。应用需要性能优化的原因有很多。当您的用户或质量保证(QA)团队报告存在性能问题时，开发人员可能会发现一些影响整体应用性能的问题。这可能是由于底层硬件限制、代码实现不佳或存在可伸缩性问题造成的。

优化是应用开发过程的一部分。这可能涉及优化代码以提高性能或优化内存使用。优化旨在优化应用的行为，使其满足产品在速度、内存占用、功耗等方面的要求。因此，优化几乎与生产阶段的编码功能一样重要。客户可能会报告性能问题，如故障、响应速度慢和功能缺

失。速度较快的应用执行效率更高，同时消耗的资源更少，并且可以在与速度较慢的应用相同的时间内处理更多任务。在当今竞争激烈的世界里，更快的软件意味着相对于竞争对手的竞争优势。在嵌入式和移动平台上，性能非常重要，速度、内存和功耗等因素非常普遍。

在瀑布流程中，性能改进是在应用开发之后、集成和验证阶段进行的。然而，在当今的敏捷世界中，代码性能应该每隔几个冲刺就进行一次评估，以获得整体的应用性能。性能优化是一个持续的过程，而缺陷修复是一次性任务。这是一个迭代的过程，在这个过程中，您总是会找到需要改进的地方，并且在您的应用中总是有改进的余地。根据约束理论(**TOC**)，在复杂应用中通常有一个问题限制应用实现其最佳性能。这种约束称为瓶颈。应用的顶级性能受到瓶颈的限制，因此您应该在应用开发生命周期中考虑性能优化。如果忽视，你的新产品可能会成为一场彻底的灾难，甚至可能毁了你的声誉。

在进入优化阶段之前，您应该定义一个目标。然后，您应该确定瓶颈或约束。在此之后，考虑如何修复约束。您可以改进代码并重新评估性能。如果它没有达到设定的目标，你需要重复这个过程。但是，请记住，过早优化可能是一切邪恶的根源。在验证您的产品和实现早期用户的反馈之前，您应该首先实现主要功能。请记住，首先要使应用运行，然后使其功能正确，然后使其速度更快。

当您设定性能目标时，您需要选择正确的技术。可以有多个目标，例如更快的启动时间、更小的应用二进制文件或更少的随机存取存储器(**RAM**)使用量。一个目标可能会影响另一个目标，因此您必须根据预期标准找到平衡点-例如，优化代码的性能可能会影响内存优化。提高整体性能可能有不同的方法；但是，您还应该遵循组织编码指南和最佳实践。如果您正在为一个开放源码项目做贡献，或者是一名自由职

业应用开发人员，那么您应该遵循标准的编码实践来维护整体代码质量。

以下是我们将遵循的一些提高性能的重要技巧：

- 使用更好的算法和库
- 使用最佳数据结构
- 负责任地分配内存并优化内存
- 避免不必要的复制
- 消除重复计算
- 提高并发性
- 使用编译器二进制优化标志

在接下来的部分中，我们将讨论在我们的 **C++** 代码中提高整体应用性能的机会。

优化 **C++** 代码

在大多数 Qt 应用中，很大一部分代码是用 **C++** 完成的，因此您应该知道 **C++** 优化技巧。本节介绍在编写 **C++** 代码时实现一些最佳实践。如果 **C++** 实现是在未经优化的情况下编写的，则运行速度很慢，并且会消耗大量资源。更好地优化 **C++** 代码还可以更好地控制内存管理和复制。有很多改进算法的机会，从小的逻辑块到使用标准模板库 (**STL**)，再到编写更好的数据结构和库。有几本关于这个主题的优秀书籍和文章。我们将讨论更快运行代码和使用更少资源的几个要点。

下面列出了一些重要的 **C++** 优化技术：

- 专注于算法，而不是微观优化
- 不要构造对象并不必要地复制

- 使用 C++ 11 特性，如移动构造函数、`lambdas` 和 `constexpr` 函数
- 选择静态链接和位置相关代码
- 首选 64 位代码和 32 位数据
- 最大限度地减少数组写入，优先使用数组索引而不是指针
- 首选常规内存访问模式
- 减少控制流量
- 避免数据依赖
- 使用最佳算法和数据结构
- 使用缓存
- 使用预计算表避免重复计算
- 首选缓冲和批处理

由于本书需要以前的 C++ 知识，我们希望您了解这些最佳实践。作为一名 C++ 程序员，请始终保持与最新的 C++ 标准(如 C++ 17 和 C++ 20)保持同步。这些标准将帮助您编写具有强大功能的高效代码。我们不会在这一节详细讨论这些问题，但这将留给您自己探索。

您可以通过以下链接阅读关于 C++ 核心指南的更多内容：<https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines>。

您可以通过以下链接了解有关优化 C++ 代码的更多信息：<https://www.agner.org/optimize/>。

通过列出的方法来改进您的 C++ 代码。接下来，我们将在下一节讨论如何使用并发和多线程提高应用性能。

使用并发性、并行性和多线程

因为您已经是一名 C++ 开发人员，所以您可能知道这些术语，它们可

以互换使用。然而，在这些术语中有不同之处。让我们在这里重温这些术语：

- 并发是同时(并发)执行多个程序的。
- 并行性是利用多核处理器中的多核同时并行运行一部分程序的。
- 多线程是操作系统支持的中央处理器(**CPU**)为同一程序同时运行多个线程的能力。

例如，您可以启动便携文档格式(**PDF**)阅读器和 **Qt Creator** 的多个实例。**Qt Creator** 本身可以运行多个工具。您的系统任务管理器可以向您显示同时运行的所有进程。这称为并发性。它也通常被称为多任务处理。

但是如果你使用并行计算技术来处理你的数据，那么这就叫做并行性。具有巨大数据处理需求的复杂应用使用此技术。请注意，在单核处理器上进行并行计算是一种错觉。

线程是进程的最小可执行单元。一个进程中可以有多个线程，但只有一个主线程。多线程是同一进程中的并发。传统的单线程应用只使用一个内核。具有多个线程的程序可以分发到多个内核，从而实现真正的并发性。因此，多线程应用可以在多核硬件上提供更好的性能。

让我们讨论一下 **Qt** 中提供并发和多线程的几个重要类，如下所示：

- **QThread**用于管理程序中的一个控制线程。
- **QThreadPool**用于管理和回收各个**QThread**对象，以帮助降低多线程应用中的线程创建成本。
- **QRunnable**是一个接口类，用于表示需要执行的任务或代码段。
- **QtConcurrent**提供高级应用编程接口(**API**)，帮助编写多线程程序，

而无需使用低级线程原语。

- `QFuture` 允许根据稍后可用的多个计算结果同步线程。
- `QFutureWatcher` 使用信号和插槽提供有关 `QFuture` 对象的信息和通知。
- `QFutureSynchronizer` 是一个方便的类，它简化了一个或多个 `QFuture` 对象的同步。

线程主要用于两种场景，如下所示：

- 利用多核 **CPU** 加快处理速度
- 卸载长时间运行的处理或阻塞对其他线程的调用，以保持图形用户界面(**GUI**)线程或其他时间关键型线程响应

让我们简要地讨论一下称为线程的最基本的并发概念。 `QThread` 类使用方便的方法在 **Qt** 中提供线程抽象。您可以通过将 `QThread` 类子类化来启动新的自定义线程，如下所示：

```
class CustomThread : public QThread
{
    public:
    void run(){...}
};
```

您可以创建此类的新实例并调用其 `start()` 函数。这将创建一个新线程，然后在这个新线程的上下文中调用 `run()` 函数。另一种方法是直接创建 `QThread` 对象并调用 `start()` 函数，该函数将启动一个事件循环。与传统的 **C++** 线程类相比，`QThread` 支持线程中断，这在 **C++ 11** 和更高版本中不受支持。您可能想知道为什么我们不能只使用 **C++** 标准线程类。这是因为您可以以多线程安全的方式将信号和槽机制与 `QThread` 一起使用。

您还可以使用 `WorkerScript` 在 QML 中使用多线程机制。JavaScript 代码可以使用 `WorkerScript` QML 类型与 GUI 线程并行执行。要在 Qt Quick 应用中启用线程，请按如下方式导入模块：

```
import QtQml.WorkerScript
```

每个 `WorkerScript` 对象可以附加一个 JavaScript。当调用 `WorkerScript.sendMessage()` 时，该脚本将在不同的线程和 QML 上下文中运行。当脚本完成时，它可以向 GUI 线程发送响应，调用 `WorkerScript.onMessage()` 信号处理程序。您可以使用信号和信号处理程序在线程之间交换数据。让我们看一看简单的 `WorkerScript` 用法，如下所示：

```
WorkerScript {  
    id: messagingThread  
    source: "messaging.mjs"  
    onMessage: (messageObject)=> textElement.text =  
        messageObject.reply  
}
```

前面的代码片段使用 JavaScript 文件 `messaging.mjs`，该文件在新线程中执行操作。让我们看一下示例脚本，如下所示：

```
WorkerScript.onMessage = function(message) {  
    //Perform complex operations here  
    WorkerScript.sendMessage({ 'reply': 'Message ' +  
        message})  
}
```

您可以通过单击按钮或基于某些用户操作来发送消息。它将调用 `sendMessage(jsobject message)` 方法，复杂的消息传递操作将在其中进行。您可以通过以下链接阅读有关不同线程机制和用例的更多信息：<https://doc.qt.io/qt-6/threads-technologies.html>。

由于本书是为有经验的 C++ 开发人员编写的，因此您应该熟悉 `mutex`、`semaphore`、`read-write lock` 等术语。Qt 为实现多线程应用时使用这些机制提供了方便的类。我们不会通过示例深入研究这些 Qt 类。您可以通过以下链接了解有关 `QMutex`、`QSemaphore`、`QReadWriteLock` 和 `QWaitCondition` 用法的更多信息：<https://doc.qt.io/qt-6/threads-synchronizing.html>。

在本节中，我们了解了如何使用并发机制来提高应用的整体性能。对于简单的任务，不要不必要地实现它，因为这可能会导致性能下降。在下一节中，我们将讨论如何使用 QML Profiler 工具来分析 Qt Quick 应用。

使用 QML Profiler 和 Flame Graph 评测 Qt Quick 应用

Qt6 中的 QML 利用个图形处理单元(个 **GPU**)，并使用硬件加速进行渲染。这一特性使 QML 在性能上优于 Qt 小部件。但是，QML 代码中可能存在个瓶颈，可能会影响整体应用性能。在本节中，我们将重点介绍如何使用内置工具来查找这些瓶颈。Qt Creator 提供与多个工具的无缝集成。最重要的工具是 **QML Profiler**。它由 Qt 提供，可在所有 Qt 支持的平台上运行。除了 QMLProfiler 之外，Qt Creator 还提供第三方工具，如 **Valgrind**、**Heob** 和 **Performance Analyzer**。您可以启用新插件或从中删除某些插件关于插件...。 ，在帮助菜单下可用。

让我们讨论一下 QML Profiler，您将大部分时间使用它来查找 QML 代码中的瓶颈。QML Profiler 的目标是通过向您提供代码块执行特定操作所需的时间等详细信息来帮助您识别瓶颈，之后您可以决定使用合适的 GUI 元素或更好的数据结构或算法重新实现代码。

按照以下步骤开始评测和优化您的 Qt Quick 应用：

1. 使用 Qt Creator 的新建项目创建向导打开现有的 Qt Quick 项目或创建新的 Qt Quick 应用。
2. Once the project is created, add some code to it. Then, select **QML Profiler** under the **Analyze** menu to run the QML Profiler tool. The **Analyze** context menu can differ from platform to platform based on installed plugins. The following screenshot shows the **QML Profiler** option in the Windows platform. In Linux, you may see a few more options, such as **Valgrind Memory Analyzer**, **Valgrind Memory Analyzer with GDB**, and **Valgrind Function Profiler**:

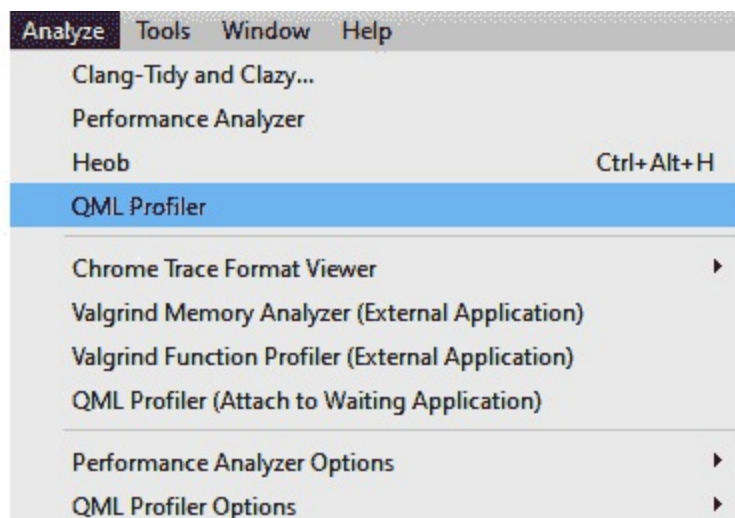


图 12.1-Qt Creator 集成开发环境(IDE)中的 QML Profiler 选项

3. When you hit the **QML Profiler** option, your Qt Quick application will run by QML Profiler. You will see the **QML Profiler** window appear below the code editor. You may also see the following message:

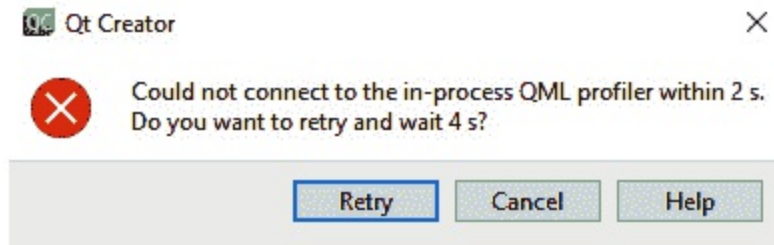


图 12.2-QML Profiler 重试信息

4. If you get this popup, just hit **Retry**. You will notice that the profiling will begin and you will also notice the output screen. In the sample application, we are creating new rectangles on a mouse click, as illustrated in the following screenshot:



图 12.3-示例 Qt Quick 应用的输出

5. On the **user interface (UI)**, perform some user interactions—such as click a button—to do a certain operation. Then, click the **Stop** button located on the title bar of the profiler window. You will also see two more buttons on both sides of the **Stop** button. If you hover your mouse over them, you will see their functionalities, such as **Start QML Profiler analysis** and **Disable Profiling**.

下面的屏幕截图显示了**QML Profiler**窗口的概述：

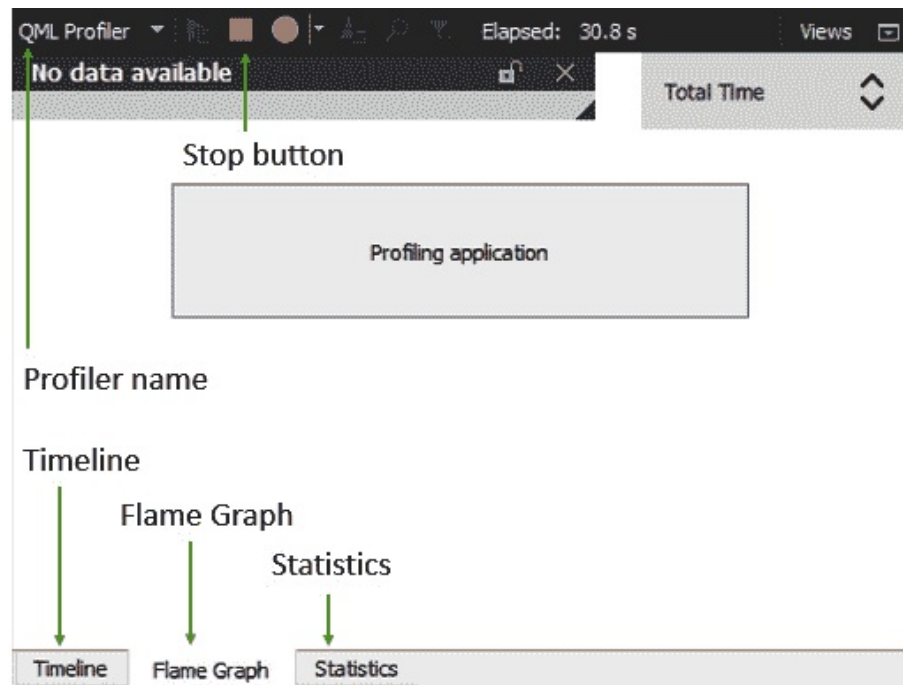


图 12.4-显示停止按钮和选项卡式视图的 QML Profiler 窗口

6. 一旦您停止 Profiler，您将看到**QML Profiler**窗口已用一些视图更新。您会注意到 Profiler 窗口下有三个选项卡-即**Timeline**、**Flame Graph**和**Statistics**。
7. Let's look at the first tab on QML Profiler—click on the **Timeline** tab. The following screenshot shows a sample view of the output:

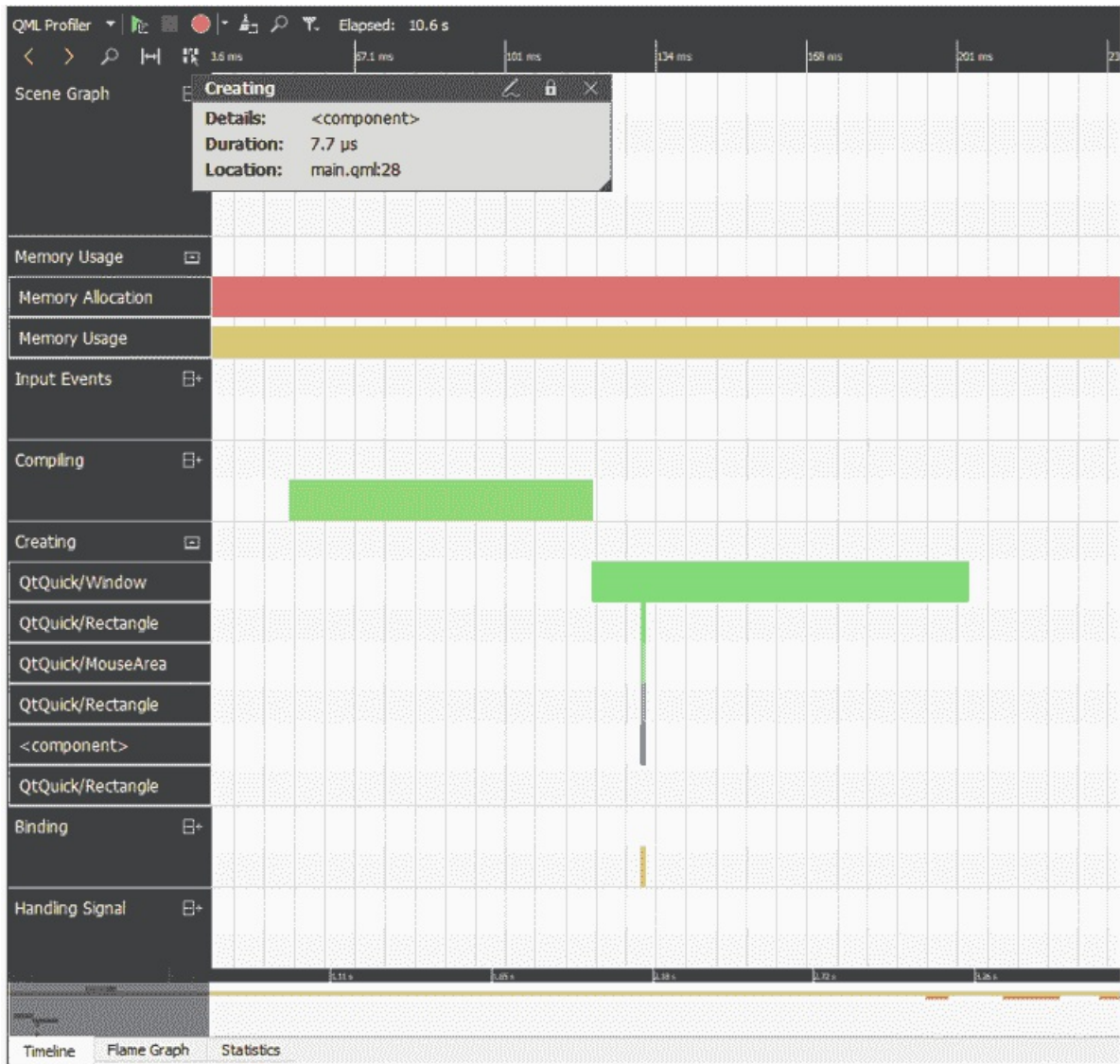


图 12.5-显示时间线详细信息的 QML Profiler

您将注意到，在时间线显示下有六个不同的部分：场景图、内存使用、编译、创建、绑定和**JavaScript**。这些部分概述了应用处理的不同阶段，如编译、组件创建和逻辑执行。

8. You can find colorful bars on the timeline. You can use the mouse wheel to zoom in and zoom out on specific timeline sections. You can also move the timeline by pressing the left mouse button at the

bottom region of the timeline and move in either direction to locate an area of interest.

时间线选项卡的不同部分如以下屏幕截图所示:

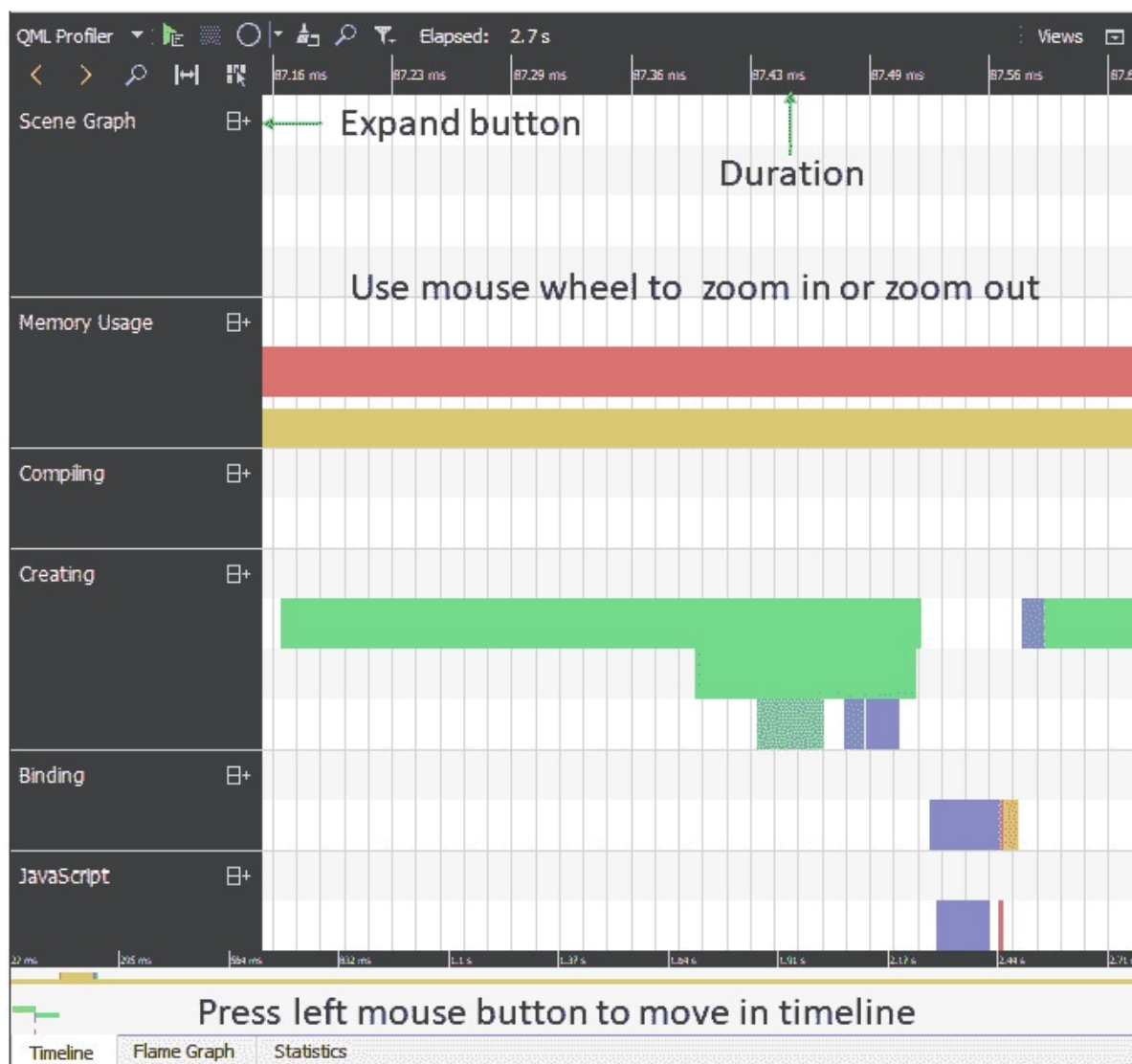


图 12.6-显示不同部分的时间线选项卡

9. You can click on the **Expand** button to see further details under each section, as illustrated in the following screenshot:

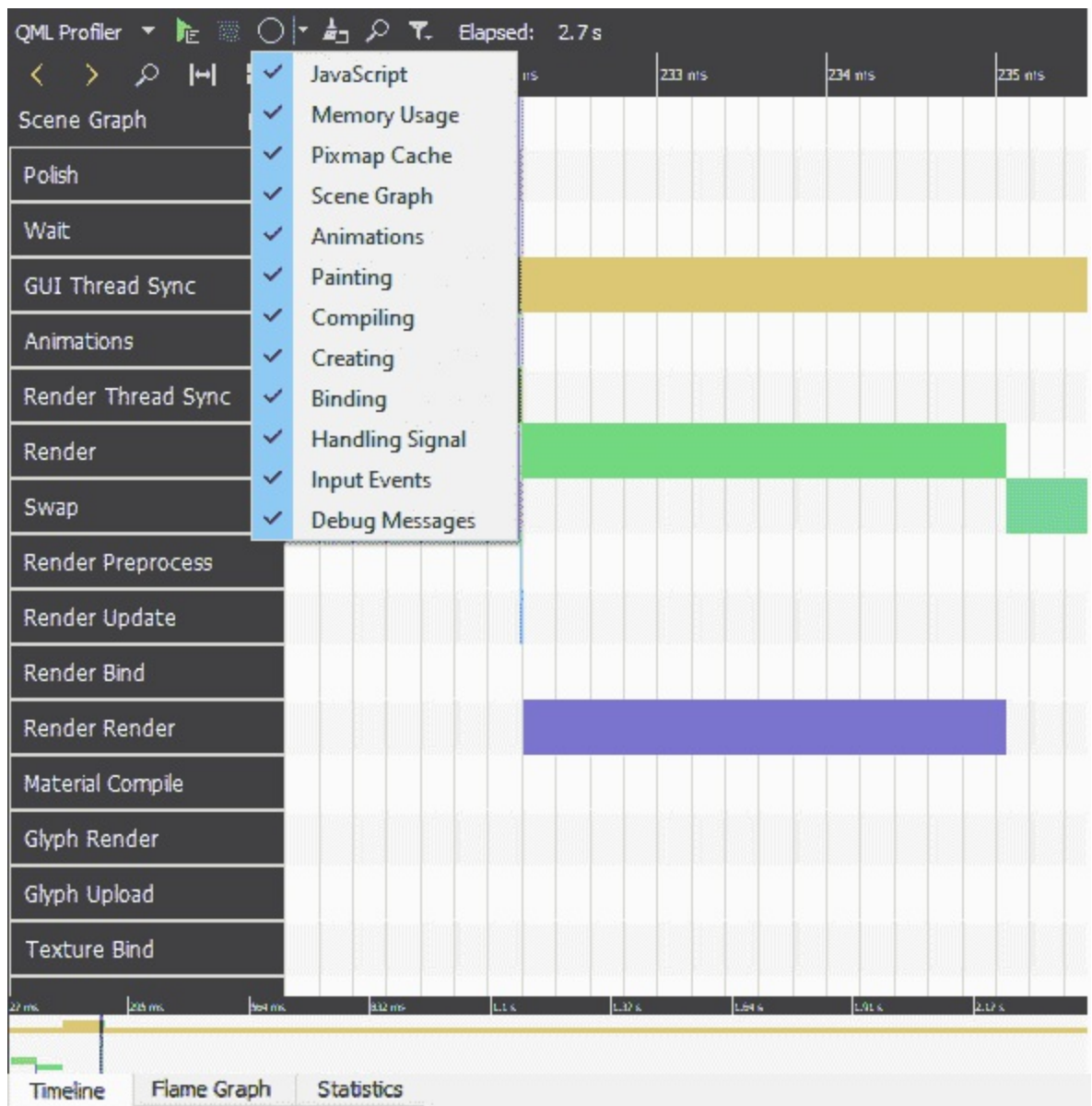


图 12.7-显示场景图和配置文件选项下不同小节的时间线选项卡

0. If you click on one of the bars under the **Creating** section, you can find component details such as the `QtQuick/Rectangle` type, total duration taken for creating an object, and the location of code displayed on a pop-up window, over the **QML Profiler** window. You can use the yellow arrows in the top-left corner to jump to previous or next events. This section is illustrated in the following

screenshot:

![Figure 12.8 - Details of an object under the Creating section](img/figure12.8.png)

图 12.8-创建部分下的对象详细信息

1. You can switch between different tabs at the bottom of the **QML Profiler** window. Once you have explored the **Timeline** tab, let's open up the **Flame Graph** tab. Under this tab, you will find a visualization of the **Total Time**, **Memory**, and **Allocations** of your application as a percentage. You can switch between these views by clicking on the dropdown located in the top-right corner of the **QML Profiler** window, as shown in the following screenshot:

![Figure 12.9 - Flame Graph showing Allocations view](img/figure12.9.png)

图 12.9-显示分配视图的火焰图表

2. **Flame Graph**视图提供更紧凑的统计摘要。水平条描绘了为特定功能收集的样本的一个方面，与组合的所有样本的相同方面相比较。嵌套指示例如显示哪些函数调用另一个函数的调用树。
3. As seen in the following screenshot, you can also see the percentage value displayed on the left side of the code editor. Based on which component is consuming more time, you can tweak your code:

![Figure 12.10 - QML Profiler showing percentage time spent for a

图 12.10-QML Profiler 显示特定代码部分花费的时间百分比

4. 由于数据采集需要时间，您可能会注意到在显示数据之前有个小小的延迟。当您单击启用性能分析按钮时，数据将被传输到 **QML Profiler**，因此不要立即终止应用。
5. 要在应用启动时禁用自动开始数据采集，请选择禁用评测按钮。当您切换该按钮时，数据收集将再次开始。
6. Let's move to the next tab: the **QML Profiler** window. This tab reveals statistical details about the processes in a table structure. The following screenshot illustrates the statistics of the code execution for our sample code:

![Figure 12.11 - QML Profiler showing statistics of code execution]

图 12.11-显示代码执行统计信息的 QML Profiler

7. You can also attach QML Profiler to an externally started application through **QML Profiler (Attach to Waiting Application)** under the **Analyze** menu. Once you select the option, you will see the following dialog:

![Figure 12.12 - QML Profiler showing remote execution option](in

图 12.12-显示远程执行选项的 QML Profiler

8. 要保存收集的所有数据，请右键单击任何 **QML Profiler** 视图，然后在上下文菜单中选择 **Save QML Trace**。您可以选择加载 **QML** 轨迹来查看保存的数据。您还可以将保存的数据发送给其他人以供查看或加载他们保存的数据。

在本部分中，我们讨论了 **QMLProfiler** 中可用的不同选项。通过使用此工具，您可以轻松找到导致性能问题的代码。有关更多详细信息，请

访问此链接：<https://doc.qt.io/qtcreator/creator-qml-performance-monitor.html>。

在的下一节中，我们将进一步讨论如何使用其他分析工具来优化您的 QT 代码。

其他 Qt Creator 分析工具

在前面的小节中，我们讨论了 QML Profiler，但是您可能需要分析您的 C++ 和 Qt 小部件代码。Qt Creator 提供了与一些著名的分析工具的集成，以帮助您分析 Qt 应用。下面列出了 Qt Creator 附带的一些工具：

- **Heob**
- 性能分析器
- ==同步，由 Elderman 更正==@ELDER_MAN
- 嘎吱作响的工具：嘎吱-Tidy 和 Clazy
- **Cppcheck**
- 铬轨迹格式(CTF)可视化工具

在了解它们的文档之前，我们先简要讨论一下这些工具并熟悉它们。

要使用 Heob，您首先需要下载并安装它。使用 Heob 可以轻松检测到缓冲区溢出和内存泄漏。它的工作方式是覆盖调用方进程的堆函数。发生缓冲区溢出时会引发访问冲突，并记录违规代码的堆栈跟踪和缓冲区分配。当应用正常退出时，您会发现堆栈跟踪。它不需要重新编译或重新链接目标应用。

您可以在官方文档链接<https://doc.qt.io/qtcreator/creator-heob.html>上了解其用法。

您可以从sourceforge.net下载二进制文件或从源代码构建它。Heob 的

SO 源码可以在以下链接中找到：<https://github.com/ssbssa/heob>。

Linux Performance Analyzer 工具与 Qt Creator 集成，可用于分析 Linux 桌面或基于 Linux 的嵌入式系统上应用的 CPU 和内存利用率。`perf` 工具定期拍摄应用调用树的快照，并使用 Linux 内核附带的实用程序在时间线视图或火焰图中可视化它们。您可以从分析菜单下的 **Performance Analyzer** 选项在 Linux 计算机上启动它，如以下屏幕截图所示：

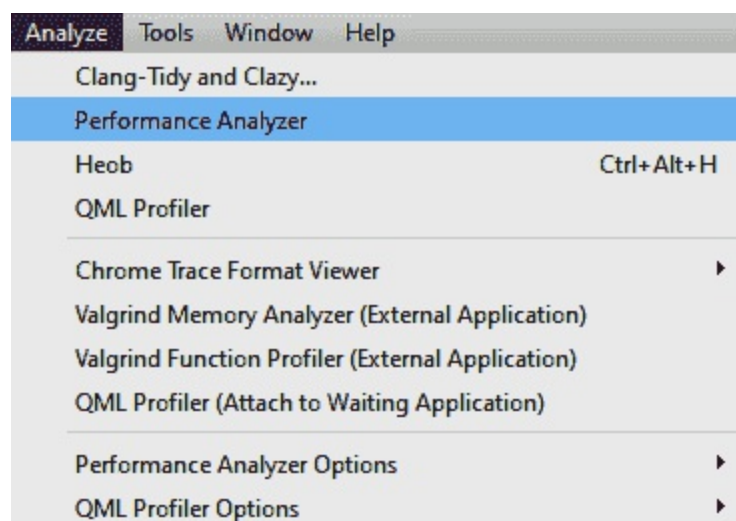


图 12.13-Qt Creator 显示性能分析器选项

请注意，性能分析器不能在 Windows 平台上运行。即使在 Linux 发行版上，如果找不到 `perf` 实用程序，您也会看到一个等效的警告对话框，如下面的屏幕截图所示：

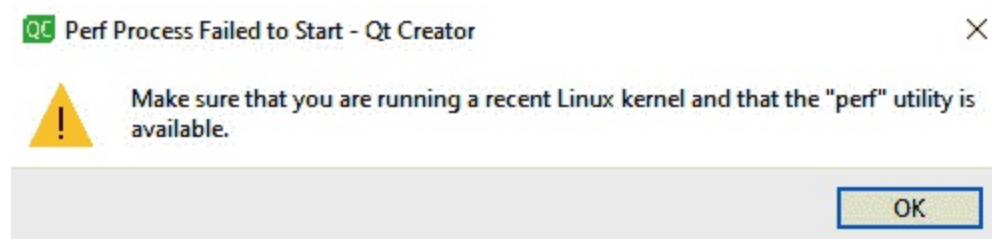


图 12.14-Qt Creator 显示性能分析器警告对话框

使用以下命令在您的 Ubuntu 机器上安装perf工具：

```
$sudo apt install linux-tools-common
```

如果您使用的是不同的 Linux 发行版，则可以使用相应的命令。perf对于特定的 Linux 内核可能会失败，并显示有关内核版本的警告。在这种情况下，请使用适当的内核版本键入以下命令：

```
$sudo apt install linux-tools-5.8.0-53-generic
```

完成perf设置后，您可以使用以下命令在命令提示符中看到预定义的事件：

```
$perf list
```

接下来，启动 Qt Creator 并打开一个 Qt 项目。从分析菜单中选择性能分析器。**Performance Analyzer**将在您开始检查应用后立即开始收集数据，**Recorded**字段将显示持续时间详细信息。由于数据是通过perf工具处理的，并且 Qt Creator 附带了一个附加的辅助程序，因此它可能会在创建几秒钟后出现在 Qt Creator 中。处理延迟字段包含该延迟的估计值。数据收集将继续，直到您单击停止收集配置文件数据按钮或关闭应用。

您还可以从分析菜单下的**Performance Analyzer Options**加载perf.data和分析应用，如下所示：

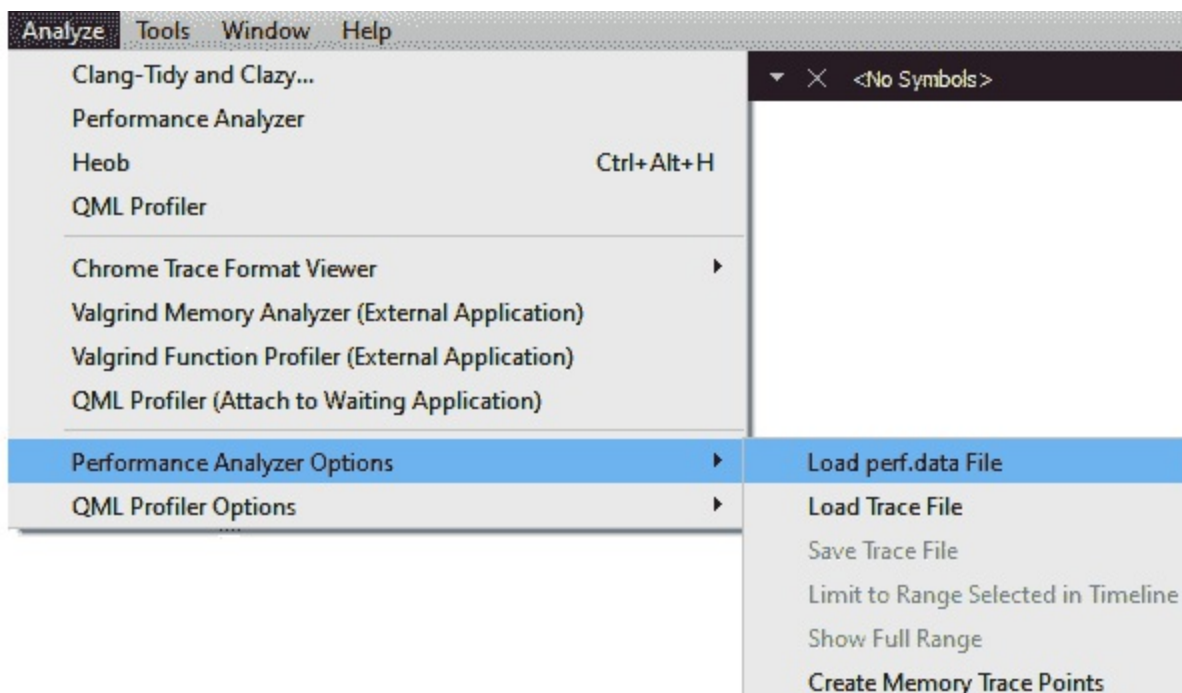


图 12.15-显示 Performance Analyzer 选项的上下文菜单

您可以通过以下链接阅读有关性能分析器用法的详细信息

息：<https://doc.qt.io/qtcreator/creator-cpu-usage-analyzer.html>。

在 MacOS 上，有一个名为**Instructions**的等效工具；但是，它没有与 Qt Creator 集成。您可以单独启动它，然后查看**Time Profiler**部分。

在 Linux 和 MacOS 上，**Valgrind**是调试各种问题的首选工具。个别技术(如分析和内存检查)用于专门的分析。Qt Creator 中的**Analyze**菜单结合了 Valgrind，并允许从 IDE 中进行内存测试和评测。要使用 Valgrind，必须安装它。它在 Windows 上不可用。然而，由于内存问题通常不是特定于平台的，您可以在 Linux 或 MacOS 上进行分析。

KCachegrind是 Valgrind 评测结果的可视化工具。当您运行 Valgrind 时，您会注意到 Profiler 窗口打开，并显示memcheck。您可以从 Profiler 下拉选项将其更改为callgrind。

您可以通过以下链接了解有关 Valgrind 的更多信息：
<https://doc.qt.io/qtcreator/creator-valgrind-overview.html>。

Qt Creator 中下一个可用的工具是**Clang-Tidy**和**Clazy...**。这些工具可用于通过静态分析定位 C++ 代码中的问题。**Clang-Tidy**提供诊断和修复常见编程错误，如样式冲突或接口误用。另一方面，**Clazy**强调了与 Qt 相关的编译器错误，如浪费的内存分配和 API 使用，并建议通过重构活动来解决一些问题。**Clang-Tidy** 包括 Clang 静态分析器功能。您不需要单独设置 Clang 工具，因为它们是分布式的，并与 Qt Creator 集成。当您运行**Clang-Tidy** 和 **Clazy...**时，如以下屏幕截图所示，您将在**Profiler**窗口下看到分析详细信息，并在代码编辑器下方的**Application Output**窗口下看到进度：

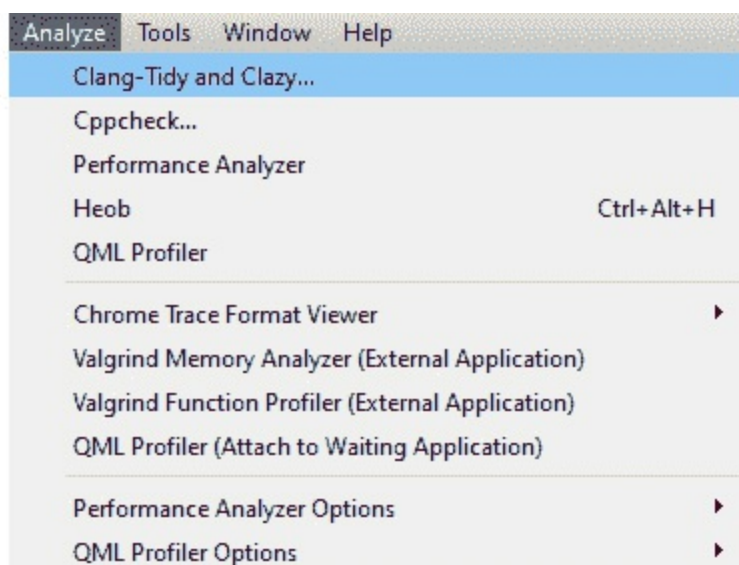


图 12.16-上下文菜单，显示了整齐和整齐的...。 选择权

让我们在现有的 Qt 示例上运行该工具。在应用窗口中，您将看到分析正在运行，在 **Profiler** 窗口中，您将看到结果。

您可以通过以下链接进一步浏览文

档: <https://doc.qt.io/qtcreator/creator-clang-tools.html>。

Qt Creator 还包括另一个名为**cppcheck**的工具。此工具与 Qt Creator 进行了实验性集成。您可以从关于插件...启用它。，位于帮助菜单下。您可以使用它来检测未定义的行为和危险的编码构造。该工具提供了检查警告、样式、性能、可移植性和信息的选项。

与 Qt Creator 集成的最后一个分析工具是**CTF**可视化工具。您可以将其与 QML Profiler 一起使用。跟踪信息可能会让您进一步了解 QML Profiler 收集的数据。您可以找到简单绑定耗时如此之长的原因，例如可能受到 C++ 代码或磁盘操作缓慢的影响。完整堆栈跟踪可用于从顶级 QML 或 JavaScript 向下跟踪到 C++，再向下跟踪到内核区域。这使您可以评估应用的性能，并确定性能不佳是由同一系统上的 CPU 还是其他程序造成的。跟踪可以深入了解系统在做什么，以及应用为什么会以不需要的方式运行。要查看 Chrome 跟踪事件，请使用 CTF 可视化工具。

您可以通过以下链接了解关于 CTF 可视化工具的更多信息: <https://doc.qt.io/qtcreator/creator-ctf-visualizer.html>。

在本节中，我们已经讨论了 Qt Creator 中可用的不同分析工具。在下一节中，我们将进一步讨论如何优化和定位图形性能问题。

优化图形性能

我们在第 8 章，图形和动画中讨论了图形和动画。在本节中，我们将探讨影响图形和动画性能的因素。图形性能在任何应用中都至关重要。如果您的应用实现不佳，则用户可能会看到 UI 闪烁，或者 UI 可能不会按预期进行更新。作为开发人员，您必须尽一切努力确保渲染

引擎保持每秒 60 帧(FPS)刷新率。每帧之间只有 16 毫秒(毫秒)，其中应以 60 FPS 进行处理，这包括将绘制基元上载到图形硬件所需的处理。

为避免图形性能出现任何故障，应尽可能使用异步的事件驱动编程。如果您的应用有大量的数据处理需求和复杂的计算，那么可以使用辅助线程来进行处理。切勿手动旋转事件循环。在阻塞功能中，每帧花费的时间不要超过几毫秒。如果您不遵循这些要点，用户将看到 GUI 闪烁或冻结，导致糟糕的用户体验(UX)。当涉及到在 UI 上生成图形和动画时，QML 引擎非常高效和强大。然而，有几个技巧你可以用来让事情进行得更快。与其自己编写，不如利用 Qt6 的内置功能。

在绘制图形时，如果可能，您应该选择不透明的基元。不透明基本体在渲染器渲染和在 GPU 上绘制时速度更快。因此，在便携网络图形(PNG)和联合图像专家组(JPEG)文件之间，渲染 JPEG 格式更快。将照片传递给 QQuickImageProvider 时，您应该使用 QImage::Format_RGB32。请注意，重叠的复合项目不能批量处理。如果可能的话，避免剪裁，因为这样会破坏批次。使用 QQuickImageProvider 生成裁剪图像，而不是裁剪图像。需要单色背景的应用应该使用 QQuickWindow::setColor() 而不是顶级 Rectangle 元素。QQuickWindow::setColor() 调用更快的 glClear()。

在使用 Image 时，请利用 sourceSize 属性。sourceSize 属性使 Qt 能够在将图像加载到内存之前缩小图像的大小，从而确保巨大的图像占用的内存不会超过所需的内存。当 smooth 属性设置为 true 时，Qt 会过滤图像，使其在缩放或更改原始大小时看起来更平滑。如果图像以其 sourceSize 属性相同的大小呈现，则没有区别。在某些较旧的硬件上，此属性将影响应用的性能。antialiasing 属性指示 Qt 平滑图像边

缘周围的锯齿瑕疵。此属性将影响程序的性能。

通过有效的分批可以获得更好的图形化性能。呈现器可以提供有关批处理运行得有多好、利用了多少批、保留了哪些批、哪些是不透明的以及哪些不是的统计信息。要启用此功能，请添加环境变量

(如`QSG_RENDERER_DEBUG`)并将值设置为`render`。除非图像太大，否则`Image`和`BorderImageQML`类型都会使用纹理贴图集。如果使用C++创建纹理，则调用`QQuickWindow::createTexture()`并传递`QQuickWindow::TextureCanUseAtlas`。您可以使用另一个环境变量`QSG_ATLAS_OVERLAY`为贴图集纹理上色，这有助于轻松识别它们。

要可视化场景图的默认渲染器的各个方面，可以将`QSG_VISUALIZE`环境变量设置为其中一个值。您可以在Qt Creator中执行此操作，方法是转到**Projects**选项卡，展开**Build Environment**部分，单击**Add**，然后输入变量名称`QSG_VISUALIZE`并设置该变量的值，如下所示：

- `QSG_VISUALIZE=overdraw`
- `QSG_VISUALIZE=batches`
- `QSG_VISUALIZE=clip`
- `QSG_VISUALIZE=changes`

当`QSG_VISUALIZE`设置为`overdraw`时，透支在渲染器中可见。为了突出显示透支，所有元素都在三维(3D)中可视化。在某种程度上，此模式也可用于标识视口外部的几何体。半透明项目显示为红色，而不透明项目显示为绿色。视口的边界框显示为蓝色。不要只使用`Rectangle`来绘制白色背景，因为`Window`也有白色背景。在这种情况下，使用`Item`属性而不是`Rectangle`可以提高性能。

将`QSG_VISUALIZE`设置为`batches`会使批在渲染器中可视化。未合并的批

次使用对角线图案绘制，而合并的批次使用纯色绘制。少量不同的颜色表示有效的配料。如果未合并的批次包含大量单个节点，则不需要这些批次。

从 `Item` 派生的所有 `QML` 组件都有一个称为 `clip` 的属性。默认情况下，`clip` 值设置为 `false`。此属性通知场景图不呈现超出其父元素边界的任何子元素。当 `QSG_VISUALIZE` 设置为 `clip` 时，场景顶部会出现红色斑点以指示剪切。因为默认情况下 `Qt Quick Items` 不剪裁，所以剪裁通常不会显示。剪裁阻止将多个组件批处理在一起，这会影响图形性能。

当 `QSG_VISUALIZE` 设置为 `changes` 时，将显示渲染器中的更改。随机颜色的闪烁覆盖用于高亮显示场景图形中的更改。对基本体的修改以纯色显示，但对祖先的更改(如对矩阵或不透明度的更改)由图案显示。

在 `Qt Quick` 应用中试验这些环境变量。您可以通过以下链接了解有关这些呈现标志的更多信息：<https://doc.qt.io/qt-6/qtquick-visualcanvas-scenegraph-renderer.html>。

`Qt Quick` 通过流畅的 `UI` 和动态转换帮助构建出色的应用。但是，您应该考虑一些因素以避免影响性能。将动画添加到属性时，引用该属性的所有绑定都会受到影响并重新求值。若要避免性能问题，可以在运行动画之前移除绑定，然后在动画完成后重新分配绑定。在动画过程中，请避免使用 `JavaScript`。脚本动画应该谨慎使用，因为它们在主线程中运行。

您可以使用 `Qt 快速粒子 ES` 来创建漂亮的粒子效果。但是，其性能取决于底层硬件功能。要渲染更多粒子，需要更快的图形硬件。您的图形硬件应该能够以 `60 FPS` 或更高的速度绘制。您可以通过以下链接了

解有关优化粒子性能的更多信息：<https://doc.qt.io/qt-6/qtquick-particles-performance.html>。

在本节中，我们讨论了优化图形性能的不同注意事项。在下一节中，我们将进一步讨论如何对您的应用进行基准测试。

创建基准

我们已经在第 9 章，测试和调试中了解了基准测试。让我们看看基准测试的一些方面，以评估性能问题。我们已经讨论了 **Qt Test** 对基准测试的支持，基准测试是对特定任务所需平均时间的计算。**QBENCHMARK** 宏用于对函数进行基准测试。

以下代码片段显示了在行编辑上的基准按键单击：

```
void LineEditTest::testClicks()
{
    auto tstLineEdit = ui->lineEdit;
    QBENCHMARK { QTest::keyClicks(tstLineEdit, "Some
                                   Inputs"); }
}
```

您还可以对 **Qt** 提供的便利函数进行基准测试。以下代码对 **QString::localeAwareCompare()** 函数进行基准测试。让我们看一下这里的示例代码：

```
void TestQStringBenchmark::simpleBenchmark()
{
    QString string1 = QLatin1String("Test string");
    QString string2 = QLatin1String("Test string");
    QBENCHMARK { string1.localeAwareCompare(string2); }
}
```

您还可以在 **QML** 中运行基准测试。**Qt** 基准测试框架将多次运行名称以 **benchmark_** 开头的函数，并记录运行的平均计时值。它类似于 **C++**

版本的 `QTestLib` 中的 `QBENCHMARK` 宏。您可以在测试函数名前加上 `benchmark_once_` 前缀，以获得 `QBENCHMARK_ONCE` 宏的效果。

您还可以使用 Qt 实验室提供的 **qmlbench** 工具。这是一个基准测试工具，它将您的 Qt 应用作为单个堆栈(而不是孤立的堆栈)进行评估，这些基准测试使您能够深入了解您的 Qt 应用的整体性能。它有几个现成的外壳，带有内置的基准测试逻辑。您可以使用 **qmlbench** 进行两种不同类型的基准测试，例如普通的 `Benchmark` 或 `CreationBenchmark`。它还允许您执行自动和手动基准测试。自动化测试可用于回归测试，而手动测试可用于了解新硬件的功能。它具有内置功能，如 **FPS** 计数器，这对 **GUI** 应用非常重要。您可以通过运行以下命令来查找帧率：

```
>qmlbench --shell frame-count
```

您还可以使用一个简单的命令运行所有自动化测试，如下所示：

```
>qmlbench benchmarks/auto/
```

要了解有关该工具的更多信息并查看示例，请参阅以下链接：<https://github.com/qt-labs/qmlbench>。

我们已经看到在 Qt 小部件和 QML 中对对象创建进行基准测试，我们还对 Qt 函数进行了基准测试。您还可以在不使用任何宏的情况下进行分析。您只需使用 `QTime` 或 `QElapsedTimer` 来测量代码或函数的一部分所需的时间，如以下代码片段所示：

```
QTime* time = new QTime;
time->start();
int lastElapsedTime = 0;
QDebug()<<"Start:"<<(time->elapsed()-
    lastElapsedTime)<<"msec";
//Do some operation or call a function
QDebug()<<"End:"<<(time->elapsed()-
```

```
lastElapsedTime)<<"msec";
```

在前面的代码片段中，我们使用 `elapsed()` 来测量代码段所花费的时间。不同之处在于，您可以计算一个函数中的几行代码-您不需要来编写单独的测试项目。这是一种无需评估整个项目即可快速发现性能问题的方法。

您还可以对您的 Qt Quick 3D 应用进行基准测试。这里有一篇关于如何做到这一点文章：<https://www.qt.io/blog/introducing-qtquick3d-benchmarking-application>。

在本节中，我们讨论了基准测试技术。在下一节中，我们将讨论更多分析工具。

不同的分析工具和优化策略

您可以在多个级别优化应用，而不仅仅是在代码级别。也可以在内存或二进制上进行优化。您可以通过使用更少的资源来修改您的应用，使其更高效地工作。但是，可能会在内存和性能之间进行权衡。根据您的硬件配置，您可以决定内存使用或处理时间是否重要。在一些具有内存限制的嵌入式平台中，您可以允许处理时间稍长一些，以使用更少的内存并保持应用的响应。您还可以将部分优化任务委托给编译器。

让我们来看看可以用来更快地构建、分析和部署的不同策略。

内存分析和分析工具

在本节中，我们将讨论一些可用于分析应用的其他工具。请注意，我们不会详细讨论这些工具。您可以访问各自的工具网站并了解它们的文档。除了 Qt Creator 中的可用工具外，您还可以在 Windows 计算机

上使用以下工具。

让我们来看一下工具列表，如下所示：

- **AddressSanitizer(Asan)**是 Google 开发的地址监控工具，是 Sanitizer 的一部分。
- **AQTime Pro**查找与应用运行时分析和性能分析相关的问题和内存泄漏。
- **Deleaker**是一个工具，适用于希望发现项目中所有可能的已知漏洞的 C++ 开发人员。它可以检测内存泄漏、图形设备接口(GDI)泄漏和其他泄漏。
- **Intel Inspector XE**是英特尔的内存和线程调试器。
- **PurifyPlus**是一个运行时分析工具套件，它在程序运行时监视您的程序，并报告其行为的关键方面。
- 可视化泄漏检测器是一个免费的、健壮的、开源的 Visual C++ 内存泄漏检测系统。
- **Very Sleepy**是一个基于采样的 CPU 分析器。
- **Visual Studio Profiler(VSTS)**可以用于 CPU 采样、检测和内存分配。
- **MTuner**利用一种新颖的方法进行内存分析和分析，保存整个基于时间的内存操作历史。
- 内存泄漏检测工具是一款高性能内存泄漏检测工具。
- **Heob**检测缓冲区溢出和内存泄漏。集成到 Qt Creator 中。
- **Process Explorer**可以查询并可视化每个进程的多个系统和性能计数器，我经常使用它进行初步调查。
- 系统资源管理器在一个长列表中显示任何正在运行的进程发出的所有系统调用，并支持筛选以选择我们想要观察的进程。
- **RAMMap**检查系统的全局内存使用情况，这需要相当多的 Windows

内部知识。

- **VMMMap**显示有关单个应用内存使用情况的详细信息。
- **Coreinfo**给出了关于处理器的详细信息，这是您在进行低级优化工作时可能需要的信息。
- 膨胀对二进制数据执行深入的分析。它的目标是准确地将二进制的每个字节归于一个符号或编译生成它的单元。

在本节中，我们向您简要介绍了一些第三方评测工具。在下一节中，我们将讨论如何在链接期间优化二进制文件。

链接过程中的优化

在前面的部分中，我们讨论了如何查找瓶颈并优化影响应用性能的代码段。幸运的是，大多数编译器现在都包含一种机制，允许您在保持代码的模块性和整洁性的同时进行此类优化。这称为链接时间代码生成(**LTCG**)或链接时间优化(**LTO**)。LTO 是程序在链接过程中的优化。链接器收集所有目标文件，并将它们集成到单个程序中。因为链接器可以查看整个程序，所以它可以进行整个程序的分析和优化。但是，链接器通常只在程序被翻译成机器码之后才能看到它。我们没有将每个源文件逐个转换为机器码，而是将代码生成过程推迟到最后的链接时间。在链接时生成代码不仅可以实现代码的智能内联，还可以实现诸如取消虚拟化函数和更好地消除冗余代码等优化。此技术可用于缩短应用启动时间。

要在 **Qt** 中启用此机制，您必须从源代码构建。在配置步骤中，将 `-ltcg` 添加到命令行选项中。在编译阶段一次编译所有源代码将为您提供完整 LTO 的所有优化优势。您可以在工具链、平台和应用级别优化应用启动时间。

通过以下链接了解有关这些性能提示的更多信息：https://wiki.qt.io/Performance_Tip_Startup_Time。

有时，您可以将优化任务委托给编译器。当您启用优化标志时，编译器将试图提高性能并优化代码块，代价是编译时间和(可能)调试能力。您可以为您想要的编译器启用编译器级别的优化标志，比如**GNU** 编译器集合(**GCC**)或 Clang。

请通过以下链接查看可用的 C++ 编译器的 GCC 优化选项：<http://gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html>。

您可以通过以下链接了解 CLANG 中的不同标志：<https://clang.llvm.org/docs/CommandGuide/clang.html>。

在本节中，您了解了链接时间优化。在下一节中，我们将讨论如何更快地构建 Qt 应用。

更快地构建 Qt 应用

在一个大型的复杂项目中，花在项目建设上的时间变得越来越宝贵。一般来说，构建时间越长，您每天损失的时间就越多。如果将乘以一个完整团队所需时间，那么仅仅等待构建完成就会浪费大量时间。虽然必须等待数小时才能重新构建每个小更改，可能会让您更仔细地考虑细节，并促使您深入思考每一步，但这也可能会限制更加敏捷的流程或协作。在本节中，我们将提供使用 Qt 在 C++ 中进行优化的简短指南。

请注意以下几点，以加快构建过程：

- 使用平行建筑旗帜
- 利用预编译头(pch)

- 从 **Makefile** 中删除冗余目标
- 在类中使用转发声明

构建大型项目时最有效的方法是使用并行构建方法。可以通过传递一个附加参数来启用并行构建。在 **Qt Creator** 中，您可以在生成设置下启用并行生成。您可以在构建步骤下找到以**Make**和**Details**按钮开始的可编辑字段。单击**Details**按钮，并在**Make Arguments**字段中输入 **-j8**。您可以通过以下命令行语句指示编译器以并行方式生成：

```
>make -j8
```

最后一个数字取决于您的硬件。 **-j8**指示并行运行八个线程。根据您的机器配置，您可以使用 **-j4**。

通过启用 **-MP** 标志，还可以为**Microsoft Visual C++(MSVC)**编译器启用并行构建。您可以通过在 **.pro** 文件中添加以下标志来指示 **cl** 并行运行：

```
*msvc* {  
    QMAKE_CXXFLAGS += -MP  
}
```

预编译头是一种极好的技术，可以极大地减少编译器的负载。当编译器解析文件时，它必须解析整个代码，以及标准标头和其他第三方源代码。 **pch** 允许您定义经常使用的文件，以便编译器可以在开始构建之前预编译这些文件，并在构建每个 **.cpp** 文件时利用结果。

要使用预编译头文件，请将以下代码行添加到 **.pro** 文件：

```
PRECOMPILED_HEADER = ../pch/your_precompiled_header.h  
CONFIG += precompile_header
```

如果使用 `Q_OBJECT` 宏，元对象编译器会生成其他文件。不要不必要地使用 `Q_OBJECT` 宏，除非您需要相关功能，如信号和插槽机制或转换。当您添加 `Q_OBJECT` 宏时，`moc` 将生成一个 `moc_<ClassName>.cpp` 文件，这增加了编译的复杂性。

您可以在 `.cpp` 文件的末尾包含此文件，如下所示：

```
#include "moc_<ClassName>.cpp"
```

您还可以降低每个 `.cpp` 文件的依赖关系，方法是对小项目使用转发声明，在大型项目中使用转发标头。转发类将缩短标准工作期间部分构建的持续时间。大多数类可以在 `forwards.h` 文件中包含转发声明。通过拥有这样的文件，您可以极大地减少头文件中包含的数量，通常是通过包含 `forwards.h`。

因此，`qmake` 会注意到这一点，并将该文件从目标列表中删除。这将减少编译器的负载。

在本节中，您了解了如何缩短应用的构建时间。在下一节中，我们将讨论基于 Qt 小部件的应用中的一些最佳实践。

Qt 小工具的性能注意事项

Qt Widgets 模块利用光栅引擎渲染 Widget，这是一种使用 CPU 而不是 GPU 渲染的软件。在大多数情况下，它可以提供所需的性能。然而，Qt 窗口小部件模块非常陈旧，缺乏最新功能。由于 QML 完全是硬件加速的，您应该考虑将其应用于应用的 UI。

如果您的部件不需要 `mouseTracking`、`tabletTracking` 或类似的事件捕获，请将其关闭。由于此跟踪，您的应用将使用更多的 CPU 时间。

维护较小的样式表，并将其全部保存在一个样式表中，而不是将其应用于单个小部件。较大的样式表将花费较长时间让 **Qt** 将信息处理到呈现系统中，这可能会影响应用的性能。使用自定义样式而不是样式表，因为这样可以提供更好的性能。

不要不必要地创建屏幕，并将其隐藏起来。仅在需要时创建屏幕。在使用 `QStackedWidget` 时，请避免添加过多页面，并在其中填充许多小部件。它要求 **Qt** 在呈现和事件处理阶段递归地发现它们，导致程序运行缓慢。

对于大型操作，只要可行，就使用异步方法，以避免阻塞主进程，并保持软件平稳运行。多线程对于并行化事件循环中的多个进程非常有用。但是，如果操作不当，例如通过重复创建和删除线程，或者通过实现不佳的线程间通信，可能会导致不希望出现的结果。

不同的 **C++** 容器产生不同的速度。**Qt** 的向量容器比 **STL** 中的略慢。总体而言，旧的 **C++** 数组仍然是最快的，但它缺乏排序功能。使用最适合你需要的东西。

在本节中，您了解了使用 **Qt** 小部件模块时的最佳实践。在下一节中，我们将讨论 **QML** 中的最佳实践。

学习 **QML** 编码的最佳实践

在用 **QML** 编码时遵循某些最佳实践是很重要的。您应该将文件保持在一个特定的行限制内，并且应该具有一致的缩进和结构属性，并遵循标准的命名约定。

您可以按以下顺序构建 **QML** 对象属性的结构：

```
Rectangle {
```



```
// id of the object
// property declarations
// signal declarations
// javascript functions
// object properties
// child objects
// states
// transitions
}
```

如果要使用一组属性中的多个属性，请使用组表示法，如下所示：

```
Rectangle {
  anchors {
    left: parent.left; top: parent.top
    right: parent.right; leftMargin: 20
  }
}
```

将属性组视为一个块可以减少混淆，并有助于将属性与其他属性关联起来。

QML 和 JavaScript 不像 C++ 那样强制私有属性。需要隐藏这些私有属性-例如，当属性是实现的一部分时。要有效地获取 QML 项中的私有属性，可以嵌入到 `QtObject{...}` 中以隐藏属性。这样可以防止在 QML 文件和 JavaScript 之外访问属性。要将对性能的影响降至最低，请尝试将所有私有属性分组到相同的 `QtObject` 作用域中。

下面的代码片段说明了 `QObject` 的用法:

[illegible]

解决属性问题需要时间。虽然有时可以缓存和重用查找结果，但如果可行，通常最好避免做额外的工作。您应该尝试在循环中只使用一次公用库。

如果任何属性发生更改，则重新计算属性绑定表达式。如果您有一个执行某些处理但只有结果重要的循环，那么最好创建一个临时累加器，然后将其分配给要更新的属性，而不是递增更新属性本身，以防止触发绑定表达式的重新求值。

为了防止因为是不可见活动元素的子项而留下不可见项的持续开销，应该延迟初始化它们，并在不再使用时销毁它们。使用 `Loader` 元素加载的对象可以通过重置 `Loader` 的 `source` 或 `sourceComponent` 属性来释放，但可以显式销毁其他项。在某些情况下，可能需要保持该项目处于活动状态，在这种情况下，应将其设置为不可见。

通常，不透明内容的绘制速度比半透明内容快得多。这是因为半透明内容需要混合，渲染器可能能够更好地优化不透明内容。即使图像只有一个半透明像素，它也会被视为完全透明。对于具有半透明边缘的 `BorderImage` 元素也可以这样说。

避免在 `QML` 中进行冗长的逻辑计算。使用 `C++` 实现业务逻辑。如果您仍然需要使用基于 `JavaScript` 的实现来执行一些复杂的操作或处理，那么可以使用 `WorkerScript`。

`Qt` 快速编译器允许您将 `QML` 源代码编译成最终的二进制代码。通过启用此功能，可以大大缩短应用的启动时间。您不必将 `.qml` 文件与应用一起部署。您可以通过将以下行添加到 `Qt` 项目(`.pro`)文件来启用 `Qt` 快速编译器：

```
CONFIG += qtquickcompiler
```

要了解有关 **Qt** 快速最佳实践的更多信息，请阅读以下链接上的文档：<https://doc.qt.io/qt-6/qtquick-bestpractices.html>。

您还可以在以下链接的文档中了解有关 **Qt** 快速性能的更多信息：<https://doc.qt.io/qt-6/qtquick-performance.html>。

在本节中，我们学习了使用 **QML** 编码时的一些最佳实践。我们现在将在本章中总结我们所学到的内容。

摘要

在本章中，我们讨论了性能注意事项以及如何提高应用的整体性能。我们从改进 **C++** 代码开始。然后，我们解释了并发技术如何帮助您提高应用的速度。您了解了 **QML Profiler** 和其他性能分析工具。您还了解了在 **Qt** 中编码时使用最佳实践的重要性。现在，您可以在日常编码中使用这些技术。要进行性能优化，您不必是出色的应用开发人员。如果您遵循最佳实践、设计模式并编写更好的算法，那么您的应用将会有更少的缺陷和更少的客户投诉。这是一个持续的过程，你会逐渐变得更好。

祝贺你！您已经学习了性能优化的基础知识。如果您想了解更多信息，可以阅读更多专门为性能调优编写的书籍。快乐的 **Qt** 编码。记住-编写更好和高性能的代码可以减少 **CPU** 周期，进而减少碳足迹，因此，如果您编写更好的代码，就可以有效地拯救地球，抗击气候变化！